

FROM FLOW TO SERVICES

Designing Microservices from Advanced DDD
in the NEXUS-1 Digital Twin



SERVICE BOUNDARIES - DATA OWNERSHIP - CONTRACTS - EVENTUAL CONSISTENCY

GRIGORIOS AGATHANGELIDIS

A NEXUS-1 Companion Volume - Microservice Architecture After Advanced DDD

DEDICATION

*This book is dedicated to my friend
Anna.*

*They stood face to face only three or four times. And yet,
the distance between them was never measured in miles
or hours - it was measured in words. The ones he
entrusted to her, the ones she gave back to him with her
patience and her undivided attention.*

*She, who had read so many books and traveled through
thousands of pages, always had the humility to look at
him as if he were the story worth reading. She
appreciated him - he knows it now - far more than he
ever dared to appreciate himself.*

*He thanks her. For her time, for her devotion,
for her quiet magic.*

FRONT MATTER

Standing Boundary and Copyright Note

From Flow to Services - Designing Microservices from Advanced DDD in the NEXUS-1 Digital Twin. A NEXUS-1 Companion Volume and the natural continuation of From Context to Flow. Copyright (c) 2026 Grigorios Agathangelidis. All rights reserved.

STANDING BOUNDARY

This volume is a software architecture and learning companion. It is not safety-class software, not certified operational guidance, and not connected to any real facility. NEXUS-1 remains a Phase-0 educational demonstrator: useful for learning architecture, language, modelling, boundaries, event flow, service design, reliability patterns, API discipline, messaging, delivery guarantees, distributed workflows, eventual consistency, reporting projections, compliance views, audit trails, traceability, correlation IDs, delayed-truth communication, security boundaries, identity propagation, service-to-service trust, observability, health checks, resilience, deployment units, independent release, containers, orchestration concepts, runtime supervision, operational dashboards, migration strategy, safe service extraction, strangler migration patterns, coexistence planning, contract-test discipline, rollback planning, safe deployment, release gates, recovery evidence, final candidate mapping, service catalog discipline, API and event catalog discipline, data ownership catalogues, failure catalogues, checklist discipline, glossary vocabulary, source/reference accountability, working-index navigation, final consistency audit, and final checkpoint discipline - never for operating a plant.

STANDING RULE

Nothing claims to exist that does not. Every service boundary is presented as an architectural candidate, not as a production command. Every benefit is paired with cost. Every distributed decision must explain what becomes harder.

SCOPE OF THIS DRAFT

This PDF keeps the established cover page, boxed dedication, expanded readable contents, front matter, Parts I-XI, Chapters 1-72, and Back Matter: Glossary of Microservice Terms, Sources and Reference Notes, Working Index, Final Consistency Audit, and Final Checkpoint. This closes the draft body and back matter before any later polishing, pagination refinement, or final publication pass.

Contents

page-aware draft through Final Checkpoint - 1 of 6

SEC	TITLE	PAGE
Front	Cover; Dedication; Boundary; Contents; Preface; How to Read; Series Position	1-14
Part I	Why Microservices Are Not the Starting Point	15
1	The Microservice Temptation	16
2	Why a Bounded Context Is Not Automatically a Service	27
3	The Cost of Distribution	34
4	When the Modular Monolith Is Still the Better Architecture	40
5	NEXUS-1 Before Service Extraction	50
Part II	Finding Service Boundaries	63
6	From Context Map to Service Candidate	64
7	Business Capability Boundaries	75
8	Data Ownership Boundaries	86
9	Workflow Boundaries	98
10	Team and Deployment Boundaries	110
11	Service Boundary Checklist for NEXUS-1	120

Contents

page-aware draft through Final Checkpoint - 2 of 6

SEC	TITLE	PAGE
Part III	Service Ownership and Data Independence	136
12	Database-per-Service Is a Rule About Ownership	137
13	Why Shared Databases Break Microservices	148
14	Cross-Service References and Identity	159
15	Local Truth vs Remote Truth	170
16	Reference Data, Snapshots, and Replication	178
17	NEXUS-1 Data Ownership Map	189
Part IV	APIs and Synchronous Communication	197
18	Designing Service APIs Without Leaking the Domain	198
19	Commands, Queries, and Public Contracts	216
20	REST APIs Between Services	228
21	Versioning Public APIs	239
22	When Synchronous Calls Are Acceptable	251
23	When Synchronous Calls Become a Trap	263
24	API Gateway and Backend-for-Frontend for the Operator Console	275

Contents

page-aware draft through Final Checkpoint - 3 of 6

SEC	TITLE	PAGE
Part V	Messaging and Asynchronous Communication	287
25	Why Services Need Messages	288
26	Integration Events Revisited	300
27	Message Brokers and Delivery Guarantees	312
28	Outbox in a Microservice Architecture	324
29	Inbox and Idempotent Consumers	336
30	Ordering, Duplicates, and Lost Assumptions	348
31	Dead-Letter Queues and Poison Messages	360
32	Event Contracts in NEXUS-1	371
Part VI	Distributed Workflows	388
33	Process Managers Across Services	389
34	Saga Coordination vs Choreography	399
35	The Alarm-to-Compliance Distributed Flow	407
36	Compensation Without Distributed Transactions	417
37	Timeouts, Escalations, and Human Review	425
38	What Must Never Be Hidden in a Workflow Engine	433
39	NEXUS-1 Distributed Process Manager Example	441

Contents

page-aware draft through Final Checkpoint - 4 of 6

SEC	TITLE	PAGE
Part VII	Consistency, Reporting, and Read Models	455
40	The End of Single-Database Thinking	456
41	Eventual Consistency in the Operator Console	464
42	Building Reporting Projections	470
43	Compliance Views Across Services	478
44	Audit Trails Across Service Boundaries	485
45	Traceability and Correlation IDs	492
46	Explaining Delayed Truth to Users	498
Part VIII	Security, Identity, and Access Boundaries	504
47	Authentication vs Authorization Across Services	505
48	Service-to-Service Authentication	511
49	Claims, Roles, and Policy Boundaries	517
50	Secrets, Certificates, and Configuration	523
51	Least Privilege Between Services	530
52	NEXUS-1 Security Boundary Map	538

Contents

page-aware draft through Final Checkpoint - 5 of 6

SEC	TITLE	PAGE
Part IX	Observability and Operations	545
53	Logs, Metrics, and Traces	546
54	Correlation IDs and Causality Chains	554
55	Health Checks and Readiness Checks	562
56	Resilience: Retry, Timeout, Circuit Breaker	570
57	Deployment Units and Independent Release	580
58	Containers and Orchestration Concepts	589
59	Operational Dashboards for NEXUS-1	597
Part X	Migration Strategy	605
60	From Modular Monolith to Microservices	606
61	Strangler Fig Pattern	611
62	Extracting the First Service	617
63	Keeping the Old System Alive During Migration	624
64	Contract Tests During Extraction	630
65	Rollback and Safe Deployment	636
66	NEXUS-1 Migration Roadmap	642

Contents

page-aware draft through Final Checkpoint - 6 of 6

SEC	TITLE	PAGE
Part XI	Final Architecture	648
67	The NEXUS-1 Microservice Candidate Map	649
68	Service Catalog	656
69	API and Event Catalog	663
70	Data Ownership Catalog	670
71	Failure Mode Catalog	676
72	Final Pre-Implementation Checklist	682
Back	Glossary of Microservice Terms	689
Back	Sources and Reference Notes	695
Back	Working Index	701
Back	Final Consistency Audit	704
Back	Final Checkpoint	710

PREFACE

Why Microservices Must Wait

A microservice architecture should not be the first shape of a system. It should be the result of boundaries that have already become true.

The previous NEXUS-1 volume, From Context to Flow, stopped deliberately before deployment. It explained context maps, domain events, integration events, outbox, inbox, process managers, compensation, anti-corruption layers, published language, and eventual consistency. It asked whether the domain was ready to move across boundaries. This book begins at that exact point.

The question is no longer only: what does the RootCause context publish? The question is harder: can RootCause become a separately deployed service without damaging the domain model, the audit trail, the public language, or the reliability rules that make the flow trustworthy?

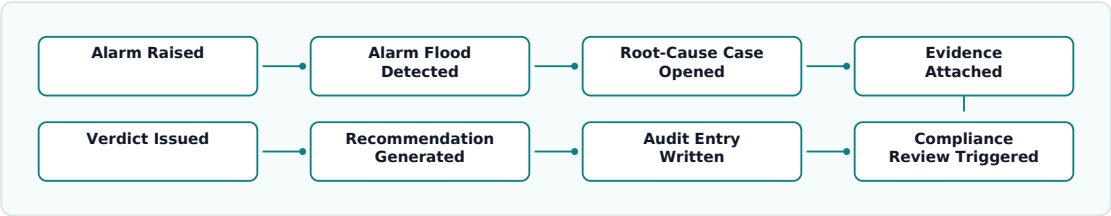


Figure 0.1 - The recurring NEXUS-1 flow. The book follows this path as it moves from modular core to possible service boundaries.

This book is therefore not a celebration of microservices. It is a controlled trial of them. Some NEXUS-1 sectors are strong candidates for extraction. Some are conditional. Some should remain together until the cost of distribution is justified by an equal architectural benefit.

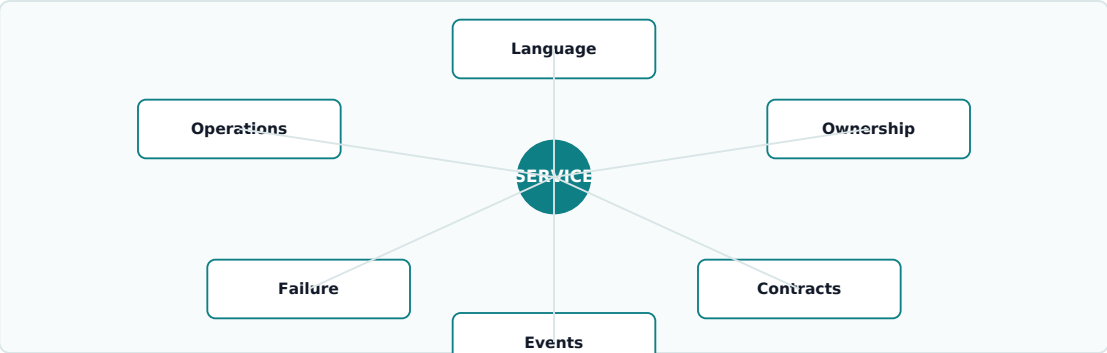
POSITIONING SENTENCE

Microservices begin only after the system already understands its boundaries, contracts, events, ownership, failure modes, and consistency model.

How to Read This Book

Read this book as a bridge from design judgement to service implementation. It does not replace From Context to Flow. It uses that book as the architectural runway.

Every chapter follows a stable rhythm: first the ordinary developer problem, then the naive solution, then the principle, then the NEXUS-1 application, then a small C#/.NET-oriented implementation shape where it helps, and finally a checkpoint.



IF YOU ARE...	READING PATH
Coming from From Context to Flow	Start here. The first chapters turn microservice readiness into service-boundary judgement.
A C# backend developer	Read in order. The implementation shapes are small, but the architectural reasoning matters more than the code.
A software architect	Focus on Parts II, III, V, VI, IX, and X. They carry the boundary, data, messaging, operations, and migration decisions.

WRITING RULE

Never treat a tool as the architecture. Docker, Kubernetes, RabbitMQ, OpenTelemetry, and API gateways appear only after the service responsibility they support is clear.

Where This Book Sits in the NEXUS-1 Series

The NEXUS-1 series has already built several layers of truth: physical simulation, alarm causality, readable policy, schema discipline, EF Core mapping, domain language, flow discipline, and backend core. From Flow to Services asks what happens when those truths are forced to survive distribution.

EARLIER VOLUME	WHAT THIS BOOK INHERITS
From Domain to Twin	The basic DDD vocabulary: domain, model, language, entities, value objects, aggregates, events, bounded contexts, and context maps.
From Context to Flow	The advanced DDD bridge: published language, outbox, inbox, process managers, compensation, and eventual consistency before microservices.
From Schema to System	The seventeen-sector data backbone and the lesson that ownership must be visible in keys, schemas, and audit relationships.
From Entity to Context	The EF Core discipline of one entity, one mapping, one clear context - useful when each service owns its persistence rules.
From Blueprint to Core	The invariant-first backend core: domain and application layers that can be tested before infrastructure arrives.

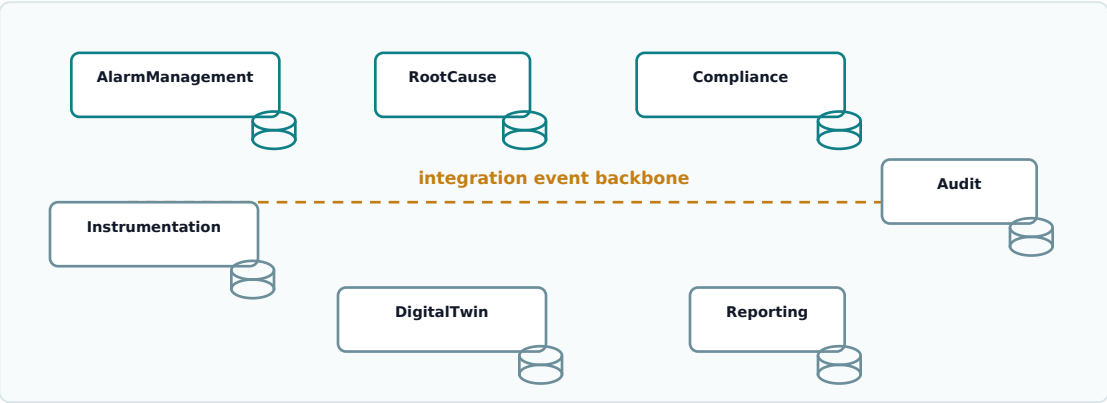


Figure 0.2 - A service candidate map is not a deployment order. It is a responsibility map that asks which sectors can own data, language, contracts, and failure independently.

POSITIONING SENTENCE

This book turns selected context boundaries into service boundaries only where the boundary is already true.

The Service-Readiness Thesis

The central claim is intentionally conservative: microservices are not a maturity badge. They are an architectural cost that becomes worthwhile only when a boundary is already strong enough to pay for it.

QUESTION BEFORE EXTRACTION	WHY IT MATTERS
Who owns the language?	Without language ownership, two services can use the same word and mean different things.
Who owns the data?	Without data ownership, the system becomes a shared database with network calls around it.
Who owns the decision?	Without decision ownership, orchestration logic leaks into the wrong service.
What is the public contract?	Without a stable contract, private aggregate shape becomes accidental API law.
What happens when a dependency fails?	Without failure behaviour, a service boundary is only a latency boundary.
How is the case traced?	Without correlation, the distributed flow cannot be explained after the fact.

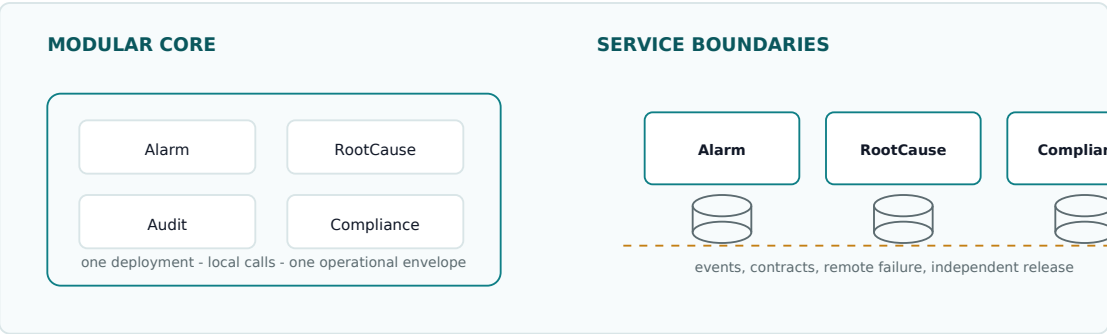


Figure 0.3 - A modular core can be clean without being distributed. A service boundary adds data independence, remote communication, operational responsibility, and failure modes that did not exist before.

RULE OF THE BOOK

Do not split a system because microservices are modern. Split only when the boundary is already true.

PART I

Why Microservices Are Not the Starting Point

Part I protects the reader from the easiest mistake: turning every bounded context, controller, or table group into a service before the system has earned that boundary.

It begins with the temptation itself, then explains why a bounded context is not automatically a microservice, what distribution costs, when the modular monolith is still better, and how NEXUS-1 looks before service extraction begins.

WHAT PART I GIVES YOU

By the end of Part I, a developer should be able to distinguish a clean module, a bounded context, a service candidate, and a deployable microservice - and should know why the four are not the same thing.

CHAPTERS 1-5 - TEMPTATION - CONTEXTS - DISTRIBUTION COST - MODULAR CORE

PART I - WHY MICROSERVICES ARE NOT THE STARTING POINT

1. The Microservice Temptation

The first microservice mistake is not technical. It is psychological. A system becomes difficult, and a team begins to believe that deployment boundaries will fix modelling problems.

Every developer has felt the attraction. The codebase is large. The solution has many projects. The database has too many tables. A feature crosses AlarmManagement, RootCause, Audit, Compliance, and Reporting. Someone says the sentence that sounds practical and modern at the same time: let us split it into microservices.

The sentence is dangerous because it can mean two completely different things. It can mean: we have mature ownership boundaries and independent release pressure. Or it can mean: the application is hard to understand, so we want the network to make the boundaries visible. The second meaning creates pain, not architecture.

PLAIN EXPLANATION

A microservice is not a small Web API. It is a responsibility boundary that owns data, language, contracts, failure behaviour, and operations. If those responsibilities are unclear before extraction, distribution will make them harder to see, not easier.

Chapter 1 has one job: slow the decision down. It does not argue against microservices. It argues against beginning with them.

What you already know

You already know how a clean modular application can be organized. You may have a Domain project, an Application project, an Infrastructure project, and an API project. You may already separate commands from queries. You may already keep EF Core mapping out of your entities. You may already publish domain events inside the application boundary.

That structure is good. It is also not a microservice architecture. It is a modular architecture. The difference matters.

STRUCTURE	WHAT IT PROVES	WHAT IT DOES NOT PROVE
Project boundary	The code can be organized and dependencies can be controlled.	That the project deserves a separate runtime.
Bounded context	A language and model boundary exists.	That the context needs independent deployment.
SQL schema	Data ownership can be made visible in the database.	That the database should be split immediately.
Web API controller	A client can call a capability.	That the controller is a service boundary.
Integration event	A fact can be published outside the model.	That every reaction should be asynchronous.

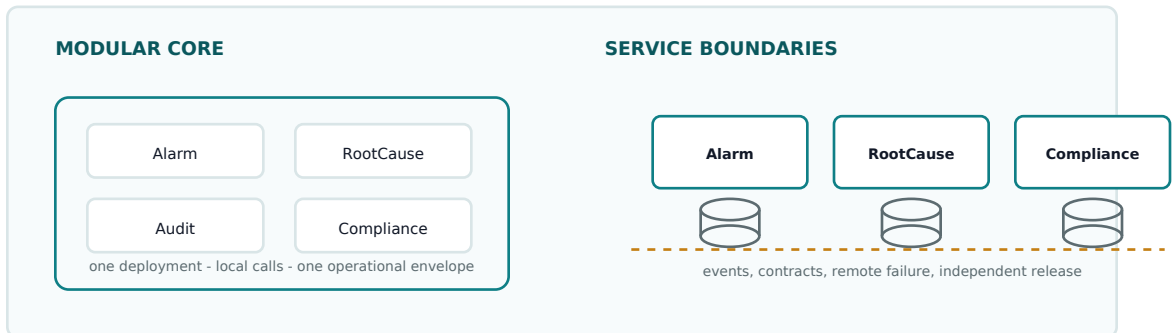
The temptation begins when we confuse these proofs. A bounded context proves a language boundary. A service requires more: independent data, stable contracts, failure isolation, observability, deployment reason, and a team that can own the runtime consequences.

NEXUS-1 REMINDER

AlarmManagement, RootCause, Audit, Compliance, Reporting, DigitalTwin, Instrumentation, and ReactorFleet are meaningful sectors. Meaningful sectors are service candidates, not automatic services.

The naive split

The naive split takes the visible folders and turns them into deployables. AlarmManagement becomes an Alarm service. RootCause becomes a RootCause service. Compliance becomes a Compliance service. Audit becomes an Audit service. Reporting becomes a Reporting service. At first the diagram looks cleaner because each box has a name.



The diagram has become more impressive, but the system has not necessarily become more coherent. A local method call has become a remote call. A database transaction has become a consistency question. A class reference has become a public contract. A log line has become a distributed trace. A unit test has become a contract test. A deployment has become an operational event.

COMMON MISTAKE

Do not mistake the visual neatness of a distributed diagram for architectural maturity. A distributed monolith often looks excellent on a slide and behaves badly in production.

The microservice temptation is powerful because it promises clarity without first doing the slow work that clarity requires: language ownership, data ownership, contract ownership, and failure ownership.

Why the naive solution fails

When a modular flow is pushed across service boundaries too early, normal software events become distributed failure modes. Nothing supernatural happens. The system simply starts charging interest on every unclear boundary.

NAIVE MOVE	IMMEDIATE RESULT	HIDDEN COST
Put RootCause behind an HTTP API	Other services can call it directly.	Every caller now depends on RootCause availability and contract stability.
Let services share the same database	Migration looks easy.	No service truly owns its data; the system remains coupled at the most dangerous layer.
Publish every domain event publicly	Integration looks rich.	Private model language becomes public contract and is difficult to change.
Make Reporting query operational tables	Screens are easy to build.	Reporting becomes a hidden dependency on source-of-truth schemas.
Retry every failed call	Failures appear recoverable.	Retry storms can amplify an outage across the system.

The cost is not only latency

Latency is the easiest cost to notice. It is not the most important. The deeper cost is the loss of local certainty. Inside one transaction, the system either commits the root-cause verdict and its local state or it does not. Across services, the verdict may be committed, the event may be waiting in the outbox, Audit may be temporarily unavailable, Compliance may not yet know, and Reporting may still show the old read model.

ARCHITECTURAL PRINCIPLE

A distributed system must be designed around partial progress. If the design assumes that all services move together, it is not a microservice architecture. It is a distributed transaction wish.

NEXUS-1: a context map is not a deployment map

NEXUS-1 already has strong names: ReactorFleet, Instrumentation, AlarmManagement, RootCause, DigitalTwin, ReinforcementLearning, Audit, Compliance, Reporting, and the operator console. The names are valuable because they express different responsibilities and language areas. But the context map answers a different question from the deployment map.

MAP TYPE	QUESTION IT ANSWERS	NEXUS-1 EXAMPLE
Context map	Where does language change?	AlarmEvent is not Evidence; Recommendation is not Command.
Data map	Who owns the source of truth?	RootCause owns RootCauseCase and its verdict. Reporting owns projections, not truth.
Contract map	What can cross the boundary?	RootCauseVerdictIssued.v1 is public; VerdictIssued remains internal.
Deployment map	What can be released and operated independently?	RootCause may become a service only if the operational cost is justified.

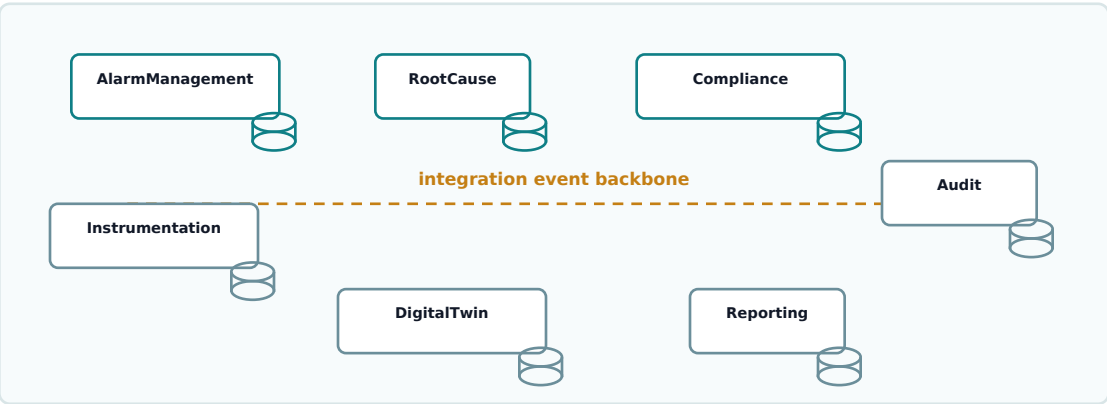


Figure 1.1 - NEXUS-1 sectors as service candidates. A candidate map is an evaluation surface, not a promise that every box will become a separate deployment.

TERMINOLOGY DISCIPLINE

Use RootCauseVerdictIssued.v1 for the public integration event. Keep VerdictIssued as the internal domain event. Do not replace this with RootCauseCaseClosed or RootCauseVerdictPublished, because those names change the language of the decision.

What a service must own

The smallest useful definition of a microservice is not about size. It is about ownership. A service is a deployable unit that owns a business capability, protects its internal model, controls its data, exposes stable contracts, tolerates remote failure, and can be observed when something goes wrong.

OWNERSHIP	MEANING	QUESTION TO ASK
Language	The service controls what its words mean inside its boundary.	Does RootCause define Evidence, Hypothesis, and Verdict without borrowing UI or Reporting terms?
Data	The service owns the source-of-truth tables for its capability.	Can another service change a RootCauseCase directly? The answer should be no.
Decision	The service decides only what it is responsible for deciding.	Does Compliance decide compliance review, while RootCause decides root cause?
Contract	The service publishes APIs and events that do not leak private aggregate shape.	Is RootCauseVerdictIssuedV1 stable even if RootCauseCase internals change?
Failure	The service has explicit behaviour when dependencies are unavailable.	What happens if Audit or Compliance is down when a verdict is issued?
Operations	The service can be monitored, deployed, configured, and diagnosed.	Can we trace a case from alarm flood to compliance review?

Only after these ownerships are credible does the microservice conversation become honest. Before that point, the safer design is usually a modular core with strong internal boundaries and a small number of well-governed integration seams.

SERVICE READINESS TEST

A service is ready when deleting the network boundary would not be the only thing keeping its model understandable. The boundary must already exist in language, data, and rules.

A small C# shape: the tempting synchronous chain

The following example is intentionally simple. It shows the style of code that often appears when a modular flow is split into services too quickly. The handler looks readable, but it hides a distributed transaction expectation inside ordinary sequential code.

```
public sealed class AlarmFloodHandler
{
    private readonly IRootCauseClient _rootCause;
    private readonly IAuditClient _audit;
    private readonly IComplianceClient _compliance;

    public async Task Handle(AlarmFloodDetectedV1 flood)
    {
        var caseId = await _rootCause.OpenCaseAsync(flood);
        var verdict = await _rootCause.AttachEvidenceAndDecideAsync(caseId);

        await _audit.WriteAsync(verdict);
        await _compliance.RequestReviewAsync(verdict);
    }
}
```

Nothing in this code says what happens if Audit is unavailable after the verdict has been committed. Nothing says whether Compliance may receive the review request twice. Nothing says whether the operator console should show a verdict before Reporting catches up. Nothing says which correlation ID ties the calls together. The code is clean; the architecture is not finished.

BETTER READING OF THE SAME FLOW

The verdict should be committed by the service that owns it. A public integration event should be recorded in an outbox in the same local transaction. Other services should react idempotently, with correlation, retries, and observable failure handling.

This does not mean synchronous calls are forbidden. It means synchronous calls must be chosen deliberately. Some queries are acceptable. Some commands are acceptable. Chained decisions that pretend to be one transaction are the trap.

The better first question

The first question is not how many services should we create? The better first question is: which boundaries are already true even if we do not deploy them separately?

BOUNDARY SIGNAL	WHAT IT LOOKS LIKE IN NEXUS-1
Different language	Evidence, Hypothesis, Verdict, Recommendation, AuditEntry, and ComplianceFinding are not synonyms.
Different source of truth	RootCause owns the verdict; Reporting owns a projection of it; Audit owns evidence of change.
Different reason to change	Alarm flood detection evolves differently from compliance review policy.
Different failure tolerance	Reporting may lag; RootCause verdict issuance must remain locally consistent.
Different operational load	Alarm events and signal readings may have different volume and retention needs from compliance records.
Different public contract	RootCauseVerdictIssued.v1 is a stable public fact; RootCauseCase internals are private.

When several signals point to the same boundary, the service candidate becomes credible.
When only the folder structure points to the boundary, extraction is premature.

DESIGN ATTITUDE

A mature architecture is allowed to say no. Keeping a context inside a modular monolith can be the disciplined choice when independent deployment would add more risk than value.

The first NEXUS-1 service-candidate reading

Chapter 1 does not decide the final service architecture. It only establishes the first reading: some sectors are strong candidates, some are conditional, and some should remain inside a protected modular core until later evidence justifies extraction.

CATEGORY	CANDIDATES	EARLY REASONING
Strong candidates	AlarmManagement, RootCause, Compliance, Audit, Reporting	They have clearer language, data, public events or read models, and plausible independent evolution.
Conditional candidates	DigitalTwin, Instrumentation, ReinforcementLearning Advisory	They may benefit from separation, but coupling, volume, timing, and advisory/control boundaries must be evaluated carefully.
Possibly kept together	ReactorFleet, simulation internals, UI composition, shared demonstration infrastructure	These may be too coupled, too foundational, or too demonstration-specific to split early.
Never implied by this book	Safety-like control logic for a real plant	NEXUS-1 is educational and advisory. It does not claim operational plant authority.

The point is not to be timid. The point is to be exact. Strong boundaries should be made stronger. Weak boundaries should be studied before they are distributed. Artificial boundaries should be removed, not wrapped in HTTP.

MICROSERVICE TEMPTATION RESOLVED

The promise of microservices is not smaller code. The promise is independent evolution of responsibly owned capabilities. When that independence is not real, a service is only another place where the same coupling can fail.

Chapter checkpoint

You are ready to continue when you can answer these questions without using tool names as explanations.

- Why is a bounded context not automatically a microservice?
- What ownerships must a service have before extraction is credible?
- Why can a shared database make a microservice architecture false even when the APIs are separate?
- What failure is hidden inside a simple synchronous service chain?
- Why is `RootCauseVerdictIssued.v1` a public integration event while `VerdictIssued` remains internal?
- Which NEXUS-1 sectors look like strong candidates, and which require caution?
- What does the phrase split only when the boundary is already true mean in practical design work?

HONEST BOUNDARY

This chapter has not designed a broker, a database-per-service topology, an API gateway, deployment units, containers, or observability. Those come later. Chapter 1 only establishes the judgement rule: distribution must follow responsibility, not fashion.

Next chapter

Chapter 2 will separate four ideas that are often collapsed into one: module, bounded context, service candidate, and deployable microservice. The distinction is the foundation for every service boundary decision that follows.

Do not split a system because microservices are modern. Split only when the boundary is already true.

PART I - WHY MICROSERVICES ARE NOT THE STARTING POINT

2. Why a Bounded Context Is Not Automatically a Service

A bounded context is a language boundary first. A microservice is a runtime boundary as well. Confusing the two turns a good DDD idea into a distributed-system problem before the system is ready for it.

The previous chapter slowed down the microservice decision. This chapter makes the main distinction precise. In Domain-Driven Design, a bounded context tells us where a model has a controlled meaning. It says that words such as Alarm, Evidence, Verdict, Recommendation, AuditEntry, and ComplianceFinding must not be blended into one universal model. That is already valuable. But it is not yet a deployment decision.

A microservice adds more than a boundary of meaning. It adds a process boundary, a network boundary, a data-ownership boundary, a release boundary, and an operational boundary. Those additions are not free. They only become worthwhile when the context can carry them without leaking its model or depending on another service to remain correct.

PLAIN EXPLANATION

A bounded context answers: what does this word mean here, and who owns the rule? A microservice also answers: who owns the data, what is the public contract, what happens when the network fails, and who operates this thing at runtime?

FOUR THINGS THAT ARE NOT THE SAME



A system may stop at any point. Stopping before runtime separation can be the correct design.

Figure 2.1 - Module, bounded context, service candidate, and microservice are related but not identical. A clean architecture can stop before runtime separation and still be correct.

What you already know

You already use boundaries before you call them microservices. A namespace is a boundary. A project is a boundary. A SQL schema is a boundary. A folder of EF Core configuration classes is a boundary. An aggregate is a boundary. A REST controller is a boundary. None of these, by itself, proves that the bounded thing should run as a separate service.

This is especially important in NEXUS-1 because the system already has many meaningful sectors. ReactorFleet, Instrumentation, AlarmManagement, RootCause, DigitalTwin, ReinforcementLearning, Audit, Compliance, and Reporting all have a reason to exist. If we treat existence as extraction, the service map becomes automatic. Architecture should not be automatic.

BOUNDARY TYPE	WHAT IT PROTECTS	WHAT IT DOES NOT SETTLE
Aggregate boundary	Local invariants and consistency rules.	Whether the aggregate deserves a service.
Bounded context	Language, model, and ownership of meaning.	Whether runtime separation is justified.
SQL schema	Database namespace and referential ownership.	Whether database-per-service is ready.
Project boundary	Compile-time dependencies and code organization.	Whether remote communication is needed.
API boundary	How a caller reaches a capability.	Whether the internal model is protected.

NEXUS-1 REMINDER

A sector is a candidate, not a verdict. Each candidate must be tested for language ownership, data ownership, contract stability, failure behaviour, and operational responsibility.

The four-step confusion

Many microservice failures begin with a quiet substitution. The team says bounded context, then draws a service. It says service, then creates a Web API. It says independent data, then keeps the same shared database. It says event-driven, then publishes whatever the aggregate currently looks like. Each step sounds close enough to the previous one that the mistake is easy to miss.

CONFUSED STATEMENT	WHY IT IS WRONG	BETTER STATEMENT
This is a bounded context, so it is a service.	A bounded context is a model boundary; a service is a runtime and operational boundary.	This context may become a service if the readiness tests pass.
This has its own controller, so it is a service.	A controller is an entry point, not an ownership model.	The controller must protect a capability owned behind it.
This has its own tables, so it owns data.	Tables in one shared database may still be changed by other modules.	Ownership means only the owning service writes source-of-truth data.
This publishes events, so it is loosely coupled.	Bad events leak private model shape and create brittle consumers.	Public integration events must be deliberate contracts.

The difference is not academic. In the alarm-to-compliance flow, a careless split can make every step appear independent while still requiring all services to be online, all databases to be consistent immediately, and all consumers to understand the private structure of RootCauseCase. That is not microservice architecture. It is a distributed monolith with better labels.

COMMON MISTAKE

Do not let the bounded-context diagram become a deployment diagram by habit. A context map is a map of language and relationships. A service topology is a map of runtime responsibility.

The service extraction ladder

A safer way to think is to place every context on a ladder. The bottom steps are DDD steps. The upper steps are distributed-system steps. A context may be valuable even if it never reaches the top.

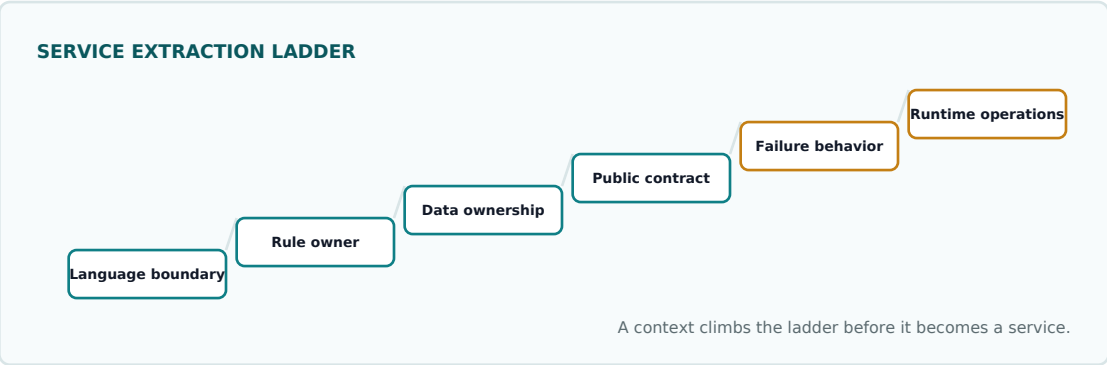


Figure 2.2 - The extraction ladder. Runtime separation comes after language, rule ownership, data ownership, public contract shape, failure behaviour, and operational ownership.

The ladder matters because it prevents a premature leap. A context that has a clear language but no independent data ownership should not become a service yet. A context that owns data but has no stable public contract should not expose its aggregate shape as an API. A context that publishes a stable contract but has no failure policy is still incomplete.

STEP	NEXUS-1 QUESTION
Language boundary	Does RootCause own Evidence, Hypothesis, Verdict, and Case without borrowing Reporting or UI words?
Rule owner	Which aggregate or domain service protects the invariant?
Data ownership	Can any other service write the source-of-truth rows directly?
Public contract	Is the contract stable enough for consumers that do not know the private aggregate?
Failure behaviour	What happens if Compliance, Audit, or Reporting is down?
Runtime operations	Can the service be deployed, monitored, traced, and diagnosed independently?

A NEXUS-1 example: RootCause

RootCause is a strong teaching example because it looks like an obvious service candidate. It has a language of its own. It owns a decision. It protects invariants. It publishes a meaningful public fact. It is also dangerous to extract casually, because its inputs and outputs sit inside a wider flow.

Inside the RootCause context, an internal domain event may be named VerdictIssued. That name is allowed to be local because it belongs to the model. Outside the context, the published integration event should be RootCauseVerdictIssued.v1, represented in C# by a public contract type such as RootCauseVerdictIssuedV1. The version number is not decoration. It is a promise that consumers can depend on.

```
// Internal model event: local language, not a public contract.
public sealed record VerdictIssued(
    RootCauseCaseId CaseId,
    VerdictText Verdict,
    DateTime OccurredAtUtc);

// Public integration event: stable contract for other services.
public sealed record RootCauseVerdictIssuedV1(
    Guid EventId,
    string CorrelationId,
    int RootCauseCaseId,
    int UnitId,
    string VerdictCode,
    string VerdictSummary,
    DateTime IssuedAtUtc);
```

The two events are related, but they are not the same object. The internal event can change when the aggregate changes. The public integration event must change only through a versioned contract. This is the kind of discipline a bounded context must show before it becomes a service.

TERMINOLOGY DISCIPLINE

AuditEntry is evidence, not workflow. A recommendation is not a command. A read model is not the source of truth. These sentences are not style preferences; they are service-boundary protection rules.

When a context should remain inside the modular core

The modular monolith is not a failure state. It is often the healthiest place for a context while its language, data, contracts, and failures are still being learned. A context can be well designed inside one deployable application. It can have its own namespace, project, schema, tests, domain events, and outbox table without being separately deployed yet.

KEEP IT MODULAR WHEN...	PRACTICAL MEANING
The boundary is mostly technical.	A folder exists, but no separate business capability is visible yet.
Data ownership is unclear.	Several modules still write the same source-of-truth rows.
Contracts are unstable.	Consumers need private aggregate details to do their work.
Failure behaviour is unknown.	The team cannot explain what happens when a dependency is unavailable.
Operational ownership is missing.	No one can own monitoring, deployment, incidents, and versioning for the service.
The flow requires immediate consistency.	Splitting would force distributed transactions or fragile synchronous chains.

For NEXUS-1, this means we should not rush every sector out of the core. Some sectors are shared foundations. Some are high-volume fact stores. Some are read-side projections. Some are advisory. Each has a different reason to exist and therefore a different extraction threshold.

HONEST DESIGN CHOICE

Keeping a context modular is not avoiding architecture. It can be the architecture that protects the model until the evidence for distribution is strong enough.

When a context can become a service candidate

A context becomes a credible service candidate when its ownership remains meaningful even after the method call becomes remote. The boundary must survive latency, retries, duplicates, versioning, partial availability, and independent deployment.

SERVICE-CANDIDATE SIGNAL	NEXUS-1 EXAMPLE
It owns a decision.	RootCause issues the verdict; Compliance decides review status; AlarmManagement detects the flood.
It owns source-of-truth data.	RootCause owns root-cause cases and verdicts, not Reporting.
It publishes stable facts.	RootCauseVerdictIssued.v1 and AlarmFloodDetected.v1 are public facts, not private entities serialized by accident.
It can tolerate asynchronous reaction.	Compliance review can be triggered after the verdict event without rolling back the verdict.
It has a different scaling or release pressure.	Reporting projections and alarm ingestion may evolve differently from compliance review.
It can be observed independently.	Correlation IDs connect AlarmManagement, RootCause, Audit, Compliance, and Reporting.

Notice the order. The signal is not that the code is large. The signal is that the responsibility is independent enough to justify the cost of remote communication and independent operation.

SERVICE CANDIDATE RULE

A context can become a service candidate when its independence is real before deployment. Deployment should reveal a boundary, not invent one.

Crossing the boundary without leaking the model

The strongest test of a service candidate is what it exposes. A weak service exposes tables, entity names, or aggregate internals. A stronger service exposes commands, queries, and events that describe public intent or public facts.

PRIVATE MODEL SHAPE	PUBLIC CONTRACT SHAPE
RootCauseCase.FinalVerdict	RootCauseVerdictIssued.v1
Hypothesis.Status changed to Promoted	RootCauseVerdictIssued.v1 after decision
AuditEntry inserted	No workflow event; audit records evidence
Recommendation row created	RecommendationAvailable.v1 or queryable read model, not an actuator command
ComplianceFinding table updated	ComplianceReviewRequested or ComplianceReviewCompleted, depending on the owned decision

This distinction protects future change. RootCause may later change its internal hypothesis scoring, evidence storage, or case lifecycle. Consumers should not break because they were coupled to private shape. They should depend on a public fact: a verdict was issued, by this service, for this case, at this time, under this correlation.

PUBLISHED LANGUAGE

A public contract is not a convenient DTO. It is the published language of the service boundary. It deserves versioning, examples, tests, and review.

A first readiness reading of NEXUS-1 sectors

The table below is not the final architecture. It is the first disciplined reading. It asks whether each sector looks closer to a module, a bounded context, a service candidate, or a deployable microservice. Later chapters will test these readings against data ownership, APIs, messaging, workflows, and operations.

SECTOR	CURRENT READING	REASON
AlarmManagement	Strong service candidate	Owens alarm definitions, alarm events, flood detection language, and the public AlarmFloodDetected.v1 fact.
RootCause	Strong service candidate	Owens cases, evidence, hypotheses, verdicts, and RootCauseVerdictIssued.v1.
Compliance	Strong but workflow-sensitive	Owens review policy and findings, but must not confuse audit evidence with workflow authority.
Audit	Infrastructure-like service candidate	Owens evidence of what happened; it should record, not decide business flow.
Reporting	Read-side service candidate	Owens projections and generated views; it is not source of truth.
Instrumentation	Conditional	High-volume facts and signal ownership matter, but timing and data volume require careful design.
DigitalTwin	Conditional	Model state and divergence are meaningful, but coupling to instrumentation and fleet must be tested.
ReinforcementLearning Advisory	Conditional/advisory	Recommendations are not commands; separation depends on safety of the advisory boundary.

This reading may change as the book develops. That is not weakness. It is the point of the process. Service boundaries should be argued into existence, not assumed.

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- What does a bounded context protect that a microservice also must protect?
- What extra responsibilities does a microservice add beyond a bounded context?
- Why is a Web API controller not proof of a service boundary?
- Why should `VerdictIssued` and `RootCauseVerdictIssued.v1` not be treated as the same event?
- When is the modular monolith still the better home for a context?
- Which NEXUS-1 sectors look like strong service candidates, and which are conditional?
- Why is a public contract more than a DTO?

HONEST BOUNDARY

This chapter has not yet assigned databases per service, API endpoints, broker topics, deployment units, or observability pipelines. It has only separated the language boundary from the runtime boundary. That separation is the necessary starting point for every later design decision.

Next chapter

Chapter 3 will examine the cost of distribution. The purpose is not to frighten the reader away from microservices, but to count the price honestly before deciding which NEXUS-1 boundaries are worth paying for.

A bounded context is the beginning of service reasoning, not the end of it.

PART I - WHY MICROSERVICES ARE NOT THE STARTING POINT

3. The Cost of Distribution

From a clean module to a networked system. Microservices do not only move code into separate projects; they move certainty out of the process and into the network.

A modular monolith lets one process hold a set of strong assumptions. A method call either reaches the method or the process is already broken. A transaction can protect more than one table. A debugger can step from the use case into the domain rule and back out through the repository. None of that is glamorous, but it is simple in the specific way software needs to be simple: the failure surface is small enough that the team can keep it in its head.

Distribution changes that surface. The same business flow now crosses processes, databases, queues, deployments, clocks, retry policies, schemas, and operational dashboards. The code may look cleaner on a diagram, but the runtime has become more complicated. This chapter counts that cost before the book asks which NEXUS-1 boundaries deserve to pay it.

What you already know

You already understand this from ordinary backend work. Calling a C# method is not the same as calling an HTTP endpoint. Reading a navigation property is not the same as asking another service for data. Saving two aggregates in one database transaction is not the same as publishing an event and hoping every consumer catches up.

PLAIN EXPLANATION

A microservice boundary is not just a line in the solution. It is a line where certainty is exchanged for autonomy. The team gains independent deployment and ownership, but it loses local calls, shared transactions, simple joins, and immediate truth.

The local call that became a remote failure

The first cost is psychological. Inside one application, a developer is tempted to treat the boundary as harmless because the code still compiles. After extraction, the same boundary has network behavior. The call can time out. The receiving service can be healthy but slow. The contract can be versioned differently. The database commit in the caller can succeed while the publish step fails. The log line you need can sit in another process.

Inside a modular core	After service extraction	New question
Method call	HTTP, gRPC, or message exchange	What happens when the receiver is unavailable?
One database transaction	Local transaction plus event publication	Which state is committed first and what retries exist?
Direct join	Local projection or remote query	Who owns the truth and who owns the copy?
One deployment	Multiple deployment units	Can one service change without breaking another?
One log stream	Distributed traces and correlation IDs	Can the whole case be reconstructed later?

The table does not mean service extraction is wrong. It means the word service must carry the cost of everything it implies. A service is not a folder with an endpoint. It is a runtime promise that must survive failure, versioning, and independent change.

HONEST BOUNDARY

This chapter is deliberately conservative. It is not trying to prove that microservices are bad. It is trying to prevent the most common mistake: counting only the architectural elegance and ignoring the operational invoice.

Cost 1 - latency and partial failure

In a single process, most failures are total enough to be obvious. If the application is down, the call does not happen. In a distributed system, failures are partial. AlarmManagement may be alive while RootCause is restarting. RootCause may issue a verdict while Compliance is temporarily unavailable. Reporting may show yesterday's projection while Audit already contains today's evidence.

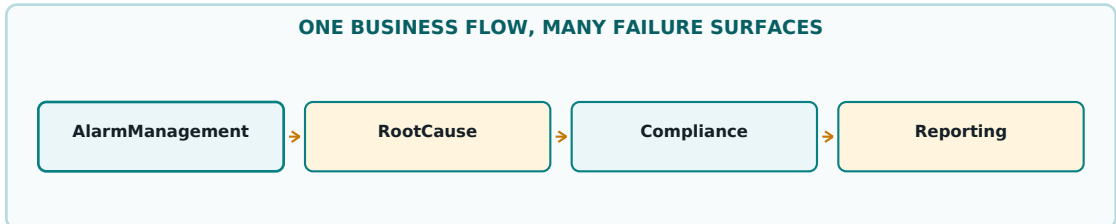


Figure 3.1 - The alarm-to-compliance flow after extraction. Every arrow is no longer a simple function call; it is a place where latency, retry, timeout, idempotency, and versioning must be designed.

The dangerous version of microservices hides this complexity behind generated clients and clean interfaces. A call that looks like this:

```
await rootCauseClient.OpenCaseAsync(command, cancellationToken);
```

may actually mean DNS lookup, connection pooling, TLS, serialization, gateway rules, authentication, timeout policy, retry policy, circuit breaking, deserialization, and error mapping. The method name stayed small; the runtime did not.

NEXUS-1 RULE

Do not extract a context until the book can state its timeout behavior, retry behavior, failure message, and user-visible degraded state. A service that cannot fail clearly is not ready to be independent.

Cost 2 - data ownership replaces easy joins

The second cost is data. In the schema atlas version of NEXUS-1, the database can express cross-sector relationships directly. A unit can be joined to its signals, alarms, causal case, audit entries, and compliance findings because the schema is one controlled backbone. That is a strength of the modular design, not an embarrassment.

```
SELECT u.Code, a.AlarmCode, rc.CaseNumber, cv.ControlCode
FROM ReactorFleet.Unit u
JOIN AlarmManagement.AlarmEvent a ON a.UnitId = u.UnitId
LEFT JOIN RootCause.RootCauseCase rc ON rc.SourceAlarmEventId = a.AlarmEventId
LEFT JOIN Compliance.ComplianceReview cv ON cv.RootCauseCaseId = rc.RootCauseCaseId
WHERE u.Code = 'U1';
```

After extraction, that query becomes illegal as an application habit. It crosses ownership boundaries. RootCause does not get to read AlarmManagement tables directly just because the SQL would be convenient. Compliance does not get to update RootCause rows because a review needs a new status. The join is replaced by contracts, projections, and copies whose staleness must be admitted.

Need	Modular monolith answer	Microservice answer
Show a case dashboard	Query the shared read model or joined schema	Build a Reporting projection from events
Validate an alarm reference	Foreign key or local query	Reference by public identity and optionally cache a snapshot
Attach evidence	Single transaction if same boundary	Command RootCause; publish event; Audit records separately
Run compliance review	Read current case state directly	Consume RootCauseVerdictIssued.v1 and keep a local review record

Cost 3 - consistency stops being automatic

In the modular core, consistency can often be strong inside one transaction. The use case opens a root-cause case, attaches evidence, and saves. If the save fails, nothing is committed. If it succeeds, the next query sees the truth. This simplicity disappears when the same flow spans services.

After extraction, each service owns only its local truth. A service can publish a fact, but it cannot force every other service to observe it instantly. The system becomes eventually consistent by design, and that phrase must not be used as decoration. It must tell the reader exactly what can be temporarily old, what must never be old, and how the user will know the difference.

Truth type	Owner	Allowed to lag?
RootCause verdict	RootCause service	No inside RootCause; yes in downstream projections
Audit evidence	Audit service or audit store	Append must be reliable; display may lag
Compliance review status	Compliance service	May lag behind verdict until review is requested
Reporting dashboard	Reporting service	Yes; it is a projection, not the source of truth

USER EXPERIENCE RULE

If the operator console reads from projections, the UI must say so. It should show pending, last updated, awaiting compliance review, or projection delayed instead of pretending that every screen is immediate truth.

The cost is not only technical. It is explanatory. The user, the tester, and the auditor must understand why the verdict exists in one place before it appears in another.

Cost 4 - contracts become public promises

A class inside one bounded context can change because only that context uses it. A public event contract cannot change that casually. Once RootCause publishes RootCauseVerdictIssued.v1, other services may store it, replay it, validate it, index it, and build operational behavior around it.

```
public sealed record RootCauseVerdictIssuedV1(
    Guid EventId,
    DateTime OccurredAtUtc,
    string CaseNumber,
    string UnitCode,
    string VerdictCode,
    decimal Confidence,
    IReadOnlyList<string> EvidenceIds,
    string CorrelationId);
```

That record is not merely a DTO. It is a promise. It says which facts are public, which names are stable, which values consumers may rely on, and which internal details remain private. The public contract protects the consumer from the publisher's internal model and protects the publisher from consumers depending on its private objects.

Bad extraction habit	Cost introduced
Expose the aggregate as the API payload	Internal model becomes impossible to change safely.
Rename events without versioning	Consumers break or silently interpret the wrong meaning.
Publish every domain event	Private implementation detail becomes public integration noise.
Let AuditEntry drive workflow	Evidence becomes accidental orchestration.

LANGUAGE DISCIPLINE

VerdictIssued may be an internal domain event. RootCauseVerdictIssued.v1 is the public integration event. Confusing the two is how a clean model becomes a brittle distributed model.

Cost 5 - operations become architecture

A microservice is not finished when the controller compiles. It must be deployable, observable, recoverable, and supportable. The operational design is part of the architecture because runtime behavior is now distributed across many processes.

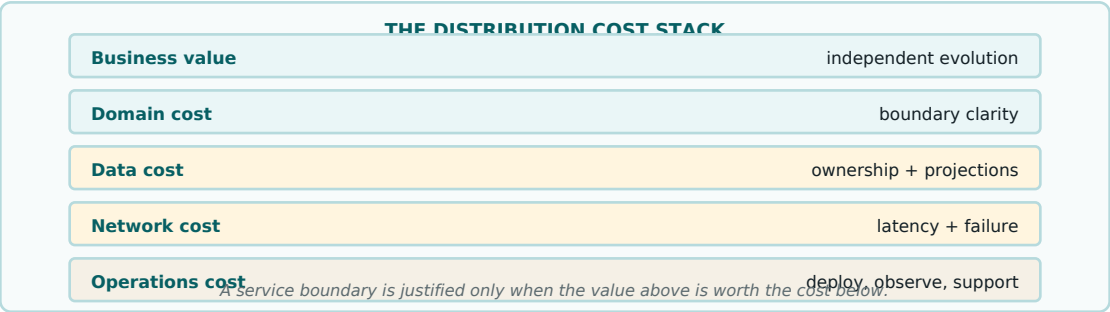


Figure 3.2 - The cost stack. Independent evolution is the visible benefit; latency, data ownership, contracts, and operations are the supporting costs that must be paid for the benefit to be real.

Operational concern	Question the architecture must answer
Health checks	Can the service report dependency health without lying about business readiness?
Correlation IDs	Can one alarm-to-compliance case be followed across all services?
Structured logs	Can an investigation find the command, event, consumer, and result?
Metrics	Can the team see queue depth, retry count, latency, and failure rate?
Deployment	Can one service roll forward or back without breaking contracts?
Data recovery	Can the service rebuild projections from events or snapshots?

The NEXUS-1 distribution cost ledger

The cost is not equal everywhere. Some NEXUS-1 sectors naturally carry independent ownership and runtime pressure. Others are better left inside the modular core until the evidence says otherwise.

Sector	Possible value of extraction	Main cost to pay first
AlarmManagement	Independent alarm ingestion and flood detection cadence.	Backpressure, ordering, duplicate alarms, and reliable handoff.
RootCause	Owns causal reasoning and verdict language clearly.	Contract discipline and traceability from alarm to verdict.
Compliance	Can evolve review workflow separately from diagnosis.	Eventual consistency and human-review state transitions.
Reporting	Naturally projection-based and read-side oriented.	Staleness, rebuilds, and projection versioning.
Audit	Strong cross-cutting evidence role.	Append reliability and avoiding workflow ownership.
ReactorFleet	Stable reference model used by many contexts.	Avoid turning reference ownership into a chatty dependency.

This ledger is not the final service map. It is the price list. A later chapter will decide which boundaries are candidates, which remain modules, and which should be treated as shared reference or platform capabilities.

DESIGN CHECK

A service candidate should have a reason stronger than "it has its own tables". It should have ownership, change pressure, operational need, and a contract that other parts of the system can understand without seeing its internals.

When the cost is worth paying

The cost of distribution is justified when the boundary protects something important enough: independent change, independent scaling, independent failure isolation, or independent team ownership. Without one of those pressures, extraction is often ceremony.

Question	Service extraction becomes more plausible when...
Does the context change at a different pace?	RootCause rules evolve without requiring a full platform release.
Does it have a clear owner?	A team can own the contract, data, uptime, and incidents.
Does failure isolation matter?	Reporting can fail without stopping alarm ingestion or diagnosis.
Does it need a different scaling profile?	Telemetry ingestion or projections need more workers than the domain core.
Can the contract stay stable?	Consumers can rely on versioned events rather than internal classes.

When those answers are weak, the modular monolith is not immature. It is honest. Keeping a boundary inside one deployable unit can be the more disciplined architecture when the domain is still changing, the team is small, the operational environment is not ready, or the only benefit is a prettier diagram.

THE CENTRAL TRADE

A modular monolith optimizes for local coherence. Microservices optimize for independent ownership and evolution. Neither is automatically superior. The correct choice is the one whose costs match the system you actually have.

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- What certainty does a modular monolith keep that microservices give up?
- Why is partial failure more dangerous than total failure?
- Why does database-per-service change the meaning of a simple SQL join?
- Which NEXUS-1 data can be eventually consistent, and which data must remain locally true?
- Why is RootCauseVerdictIssued.v1 a public promise rather than a DTO?
- What operational evidence should exist before a service boundary is trusted?
- When is keeping a context inside the modular core the more honest choice?

HONEST BOUNDARY

This chapter counted the cost of distribution but did not yet choose the service boundaries. The next chapters will use this cost model to decide when the modular monolith is still better and how NEXUS-1 looked before extraction.

Next chapter

Chapter 4 will examine the case for keeping a modular monolith. It will show why service readiness sometimes means refusing extraction until the model, data ownership, contracts, and operations are mature enough to survive it.

Microservices are not the reward for having boundaries. They are the price paid when boundaries need independent runtime life.

PART I - WHY MICROSERVICES ARE NOT THE STARTING POINT

4. When the Modular Monolith Is Still the Better Architecture

Refusing extraction can be an architectural decision, not a lack of ambition. A modular monolith is still a serious architecture when the boundaries are real, the language is disciplined, and the runtime cost of distribution has not yet earned its place.

The previous chapter counted the cost of distribution. This chapter draws the practical consequence: sometimes the best microservice decision is not to create a microservice yet. That sentence is not conservative by habit. It is conservative by evidence. A system that has not stabilized its language, ownership, data rules, and failure behavior is not made clearer by adding a network boundary. It is made harder to understand at exactly the moment understanding matters most.

NEXUS-1 is a useful example because its sectors are already meaningful. ReactorFleet, Instrumentation, AlarmManagement, RootCause, Audit, Compliance, and Reporting are not random packages. They are genuine regions of language and responsibility. But genuine regions do not automatically require separate deployment. In many phases of the system, the more honest architecture is one deployable core with strict internal boundaries.

PLAIN EXPLANATION

A modular monolith is not a big ball of mud. It is one deployable application built from modules that have controlled names, controlled dependencies, and clear ownership. The runtime is one. The model is not one blob.

What a modular monolith is - and is not

The phrase modular monolith is often misunderstood because the word monolith has been treated as an insult. It should not be. A monolith can be careless, tangled, and dangerous. It can also be disciplined, explicit, testable, and easier to operate than a premature service mesh. The difference is not the number of deployables. The difference is whether the internal boundaries are enforced.

Confusion	Careless monolith	Modular monolith
Boundary	Folders are present but dependencies cross freely.	Modules expose explicit contracts and hide internals.
Language	The same word means different things everywhere.	Each bounded context controls its vocabulary.
Data	Tables are used by any code that can reach them.	Schemas and repositories express ownership rules.
Transaction	Everything is updated from anywhere.	Strong consistency is used only inside the correct boundary.
Change	A small rule change risks unrelated screens.	A module can be refactored without leaking private details.

The modular monolith is therefore not the opposite of DDD. It can be the first runtime shape that lets DDD breathe. It gives the team one process while the language is still being discovered, one database while the ownership map is still being tested, and one deployment while the product is still learning what must change independently.

HONEST BOUNDARY

A modular monolith is not a license to avoid boundaries. It is only a better architecture when the modules are real. If every module can reach every table and every service can call every other service directly, the system is not modular; it is merely undelivered distribution.

The NEXUS-1 modular core

Before service extraction, NEXUS-1 can be read as one deployable backend core composed of explicit modules. The modules mirror the language boundaries already established in the earlier books. The runtime is still single, but the internal shape is not accidental.

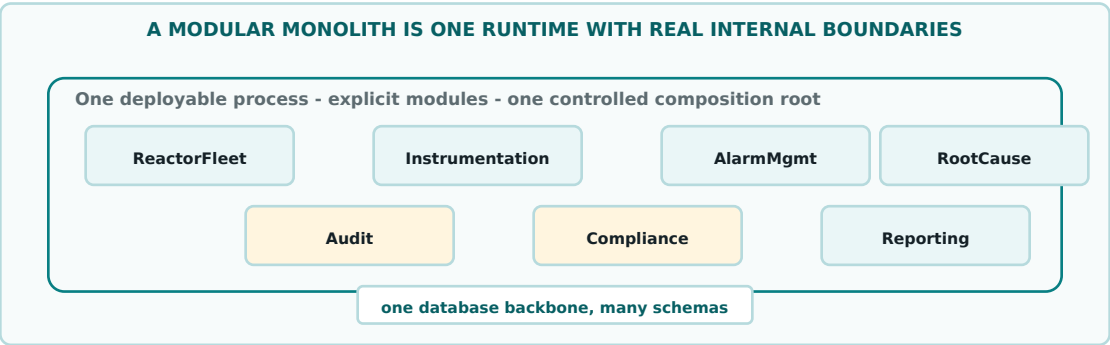


Figure 4.1 - The modular monolith shape for NEXUS-1. One runtime does not mean one model. The sectors remain visible as modules and schemas while the system avoids premature network boundaries.

This shape has a strong advantage during learning and implementation: the team can prove the model before it distributes the model. RootCause can still have its own aggregate, application use cases, repository interface, and events. AlarmManagement can still own the alarm flood language. Compliance can still own review state. Audit can still remain evidence rather than workflow. None of that requires a separate process on day one.

NEXUS-1 RULE

A context may be a module, a schema, a namespace, a project, or later a service. The bounded part is language and ownership first. Runtime separation is a later decision with an operational price.

Benefit 1 - strong consistency where the rule needs it

The first reason to keep the modular monolith is consistency. Some rules should not be spread across asynchronous messages until the boundary is mature. A `RootCauseCase` should not close without evidence. A duplicate alarm should not open two competing cases if the business rule says one case owns the flood. A command should not write a verdict and then discover that the evidence record never existed.

```
public void CloseWithVerdict(string verdict, DateTime closedAtUtc)
{
    if (!_evidence.Any())
        throw new InvalidOperationException(
            "A root-cause case cannot close without evidence.");

    if (FinalVerdict is not null)
        throw new InvalidOperationException(
            "A closed case cannot be closed again.");

    FinalVerdict = verdict;
    ClosedAtUtc = closedAtUtc;
    AddDomainEvent(new VerdictIssued(Id, verdict, closedAtUtc));
}
```

Inside the modular core, this rule can be tested in memory and saved in one local transaction with the aggregate state. The event can be recorded to an outbox in the same database transaction. Nothing here requires `RootCause` to be a separate service. In fact, extracting too early can make the guarantee harder to see.

DESIGN PRINCIPLE

Do not distribute a rule until you know which aggregate owns it, what transaction protects it, and which event expresses the result. Distribution should follow the invariant; it should not discover the invariant by accident.

Benefit 2 - queries are simpler while the model is still being discovered

A second reason to stay modular is discovery. Early in an architecture, the team still needs to ask questions that do not yet deserve a public contract: which alarm produced this case, which signal did the alarm reference, which unit was involved, which audit entries record the decision, and what will the operator console need to show?

In a modular monolith, these questions can be answered with controlled read models, SQL views, or application queries without pretending that every view is already a separate Reporting service. That is not a weakness. It is how the team learns which read models are stable enough to become projections later.

Discovery question	Modular answer	Why it helps before extraction
What does the operator need on one screen?	Build a local read model over owned schemas.	The shape can change without breaking service contracts.
Which fields belong in RootCauseVerdictIssued.v1?	Compare domain event, audit need, and UI need.	The public contract is not guessed from the first screen.
How often does Reporting lag matter?	Measure query cost and user expectation locally.	Projection design is driven by evidence, not fashion.
Which joins are accidental?	Review cross-schema query usage.	The real service boundaries become visible.

READ MODEL RULE

A read model is allowed to be convenient while it is still private. It becomes dangerous only when other contexts start treating that convenience as source-of-truth ownership.

Benefit 3 - refactoring remains cheap while language stabilizes

Microservices make public language expensive. Once a service publishes an API or an integration event, every consumer becomes part of the cost of changing a name. That is exactly why the previous book insisted on published language. But published language should not be rushed. Some words need to be wrong privately before they become right publicly.

Private language still in motion	Why not publish yet
RootCauseCaseClosed	It hides the important business fact. The public event should be RootCauseVerdictIssued.v1 when the verdict is the meaningful outcome.
RecommendationCreated	It may suggest command authority unless the contract clearly says advisory recommendation.
AuditEntryWritten	It records evidence, but it should not drive business workflow as the canonical trigger.
SignalProblemDetected	It may blur signal quality, alarm condition, and causal symptom into one vague phrase.

Inside one modular codebase, a wrong name can be repaired by a controlled refactor. Across services, the same wrong name becomes a versioning burden. The modular monolith gives the team a safer place to finish the language before the language becomes a contract.

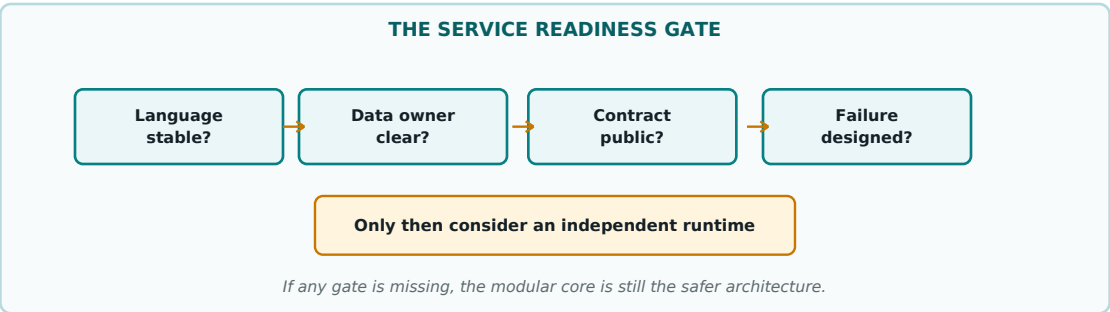


Figure 4.2 - The readiness gate. A service boundary should wait until language, data ownership, contract shape, and failure behavior can all be stated clearly.

When one database is still honest

Microservice discussions often reduce data independence to one slogan: database per service. The slogan is useful later, but it can become misleading too early. The deeper rule is not physical separation first. The deeper rule is ownership. A table should have a clear owner, and other parts of the system should not mutate it as if ownership did not exist.

In the NEXUS-1 modular stage, one SQL Server database can still be honest if it preserves sector schemas, named foreign keys, explicit repositories, append-only event records, and audit history. The database is shared physically, but not semantically. ReactorFleet.Unit is not a casual lookup for every module to rewrite. RootCause.RootCauseCase is not a Compliance table. Audit.AuditEntry is evidence, not a workflow command.

One database can be acceptable when...	It becomes a problem when...
Schemas map to bounded contexts.	Every module reads and writes every schema freely.
Foreign keys act as visible passports.	Foreign keys are used to justify cross-owner mutation.
Repositories enforce owner access.	Controllers bypass owners with direct DbContext access.
Outbox records integration facts.	Events are guessed from after-the-fact database polling.
Reporting is marked as projection.	Reporting tables become hidden source of truth.

OWNERSHIP RULE

Physical sharing is not the same as conceptual sharing. A modular monolith may share a database while still enforcing that each context owns its own truth. The danger begins when convenience erases ownership.

When the modular monolith starts to fail

The modular monolith is not a permanent answer for every system. It remains the better architecture only while its advantages are real. When independent change, scaling, failure isolation, and team ownership become stronger than local coherence, the cost equation changes.

Warning sign	What it means	Possible response
One team waits on another for every release.	Deployment coupling is now harming ownership.	Extract the context with the clearest contract first.
A module needs a different scale profile.	The runtime is paying for one hot path everywhere.	Move ingestion, projection, or background workers out carefully.
A failure in Reporting blocks diagnosis.	Failure isolation is insufficient.	Separate read-side runtime or asynchronous projection.
Schema changes constantly break unrelated modules.	Ownership rules are not strong enough.	Tighten module access before extracting.
Public contracts are already stable and replayable.	The boundary may be ready for service life.	Prepare versioned APIs, events, observability, and migration.

Notice that not every warning sign says extract immediately. Some say enforce the modular boundary better. A tangled monolith does not become a clean distributed system by being split. It becomes distributed tangle. The first repair is often dependency discipline, not Kubernetes.

CALIBRATION POINT

Service extraction should be a response to measured pressure: release pressure, ownership pressure, scale pressure, or failure-isolation pressure. Without pressure, microservices are only architectural ornament.

NEXUS-1 modular-or-service decision map

The following map is deliberately provisional. It does not declare the final service architecture. It states how a careful architect would judge the sectors at this point in the series, before the next parts of the book formalize data ownership, APIs, messages, and distributed workflows.

NEXUS-1 area	Better kept modular for now when...	Extraction becomes plausible when...
ReactorFleet	The fleet model is stable reference data used by many contexts.	Ownership, caching, and reference snapshots are designed.
Instrumentation	Signal definitions and readings are still being aligned with the twin.	Ingestion volume or historian integration creates independent scale pressure.
AlarmManagement	Flood rules are still changing with RootCause collaboration.	Alarm ingestion, ordering, and handoff contracts are stable.
RootCause	Verdict language and evidence rules are still being refined.	RootCauseVerdictIssued.v1 is stable, replayable, and observed.
Compliance	Review workflow depends heavily on evolving verdict language.	Human review state has clear commands and event triggers.
Reporting	Screen needs are still changing rapidly.	Read models are stable projections with rebuild strategy.
Audit	Audit is implemented as evidence infrastructure in the core.	Append reliability and trace correlation require independent hardening.

IMPORTANT DISTINCTION

A sector can be architecturally distinct without being operationally separate. The goal of Part I is not to avoid services. It is to make sure every future service begins as a mature boundary rather than a deployed wish.

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- Why is a modular monolith different from a big ball of mud?
- Which kind of consistency is easier to protect inside one deployable core?
- Why should public event language wait until private language has stabilized?
- How can one database still preserve ownership in a modular architecture?
- Which warning signs suggest that the modular monolith is beginning to fail?
- Why does a tangled monolith usually become a tangled distributed system if it is split too early?
- Which NEXUS-1 sectors look like early extraction candidates, and which should probably remain modular longer?

HONEST BOUNDARY

This chapter defended the modular monolith when it is disciplined. It did not claim that NEXUS-1 should remain modular forever. The next chapter will describe NEXUS-1 before service extraction so the later service map has a concrete starting point.

Next chapter

Chapter 5 will freeze the starting architecture: the NEXUS-1 modular core before extraction. It will show the contexts, schemas, application flows, event seams, and operational gaps that must be understood before any service boundary is chosen.

The right question is not "Can this be a service?" The right question is "What would be damaged if it became a service today?"

PART I - WHY MICROSERVICES ARE NOT THE STARTING POINT

5. NEXUS-1 Before Service Extraction

Before the first service boundary is drawn, the starting system must be made explicit. A service map that does not begin from a truthful modular map is only decoration.

The previous chapters argued against beginning with microservices. This chapter finishes the argument by freezing the starting point. Before NEXUS-1 becomes a distributed system, it is a coherent modular system with known sectors, known flows, known data ownership pressure, and known places where the next design will hurt.

This matters because service extraction is not an act of cutting code into smaller pieces. It is an act of deciding what can survive alone. A context that becomes a service must carry its language, data ownership, contracts, failure behavior, observability, and deployment responsibility with it. If those things are not visible before extraction, the extraction will invent them under pressure.

PLAIN EXPLANATION

Chapter 5 is the architectural freeze-frame. It asks: what does NEXUS-1 look like immediately before we start splitting it into services? Only after that picture is honest can later chapters talk about service boundaries responsibly.

The starting point is a modular core, not a service diagram

A premature microservice diagram often begins with boxes named after nouns: Alarm Service, Root Cause Service, Audit Service, Reporting Service. That looks clean, but it skips the hard question. What exactly exists inside each box before the network arrives? Which rules does it protect? Which database tables does it own? Which words does it publish? Which failures is it prepared to absorb?

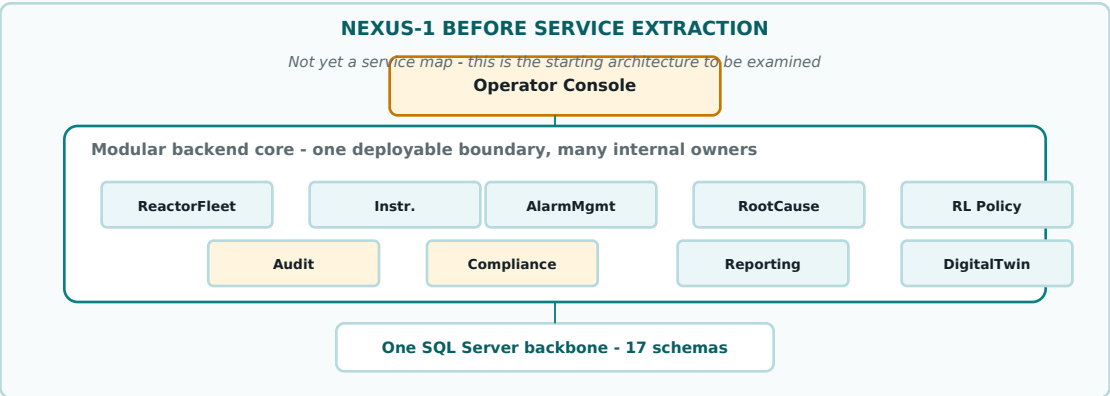


Figure 5.1 - NEXUS-1 before service extraction. The picture is deliberately not a final service topology. It is the modular starting point from which service candidates will be judged.

At this stage, NEXUS-1 should be read as one deployable core with internal architecture. The modules are not random folders. They are the named sectors that earlier books already prepared: physical fleet, instrumentation, digital twin, alarm management, root cause, reinforcement-learning policy, audit, compliance, and reporting. The service question begins only after those sectors are treated as serious owners.

The sectors before extraction

The first discipline is to state each sector without using the word service. A sector can be a bounded context, a schema, a namespace, a module, or eventually a service. But if we call it a service too soon, we may stop seeing what kind of boundary it really is.

Sector	Primary responsibility before extraction	Important boundary warning
ReactorFleet	Owens the physical plant vocabulary: plant, unit, reactor, equipment, rods, steam generators, and structural identity.	It is reference truth for many contexts, so extraction must handle caching and stale references.
Instrumentation	Owens signals, tags, sensors, readings, quality, and measurement identity.	It can become high-volume and may need a different scale profile from the rest of the system.
AlarmManagement	Owens alarm definitions, alarm events, alarm state, and flood detection rules.	It should not decide root cause; it raises and groups alarm facts.
RootCause	Owens cases, evidence, hypotheses, rejected candidates, verdicts, and causal reasoning state.	Its verdict is a decision; its public event must be stable and versioned.
ReinforcementLearning	Owens advisory policy, Q-table style reasoning, training runs, and recommendations.	A recommendation is not a command and must not become one through integration language.
Audit	Owens evidence of change, lineage, correlation, and replayable history.	Audit entries are evidence, not business workflow triggers.
Compliance	Owens review state, findings, controls, and human accountability workflow.	It consumes decisions; it should not rewrite root-cause truth.
Reporting	Owens projections, dashboards, exports, and read models.	A read model is not source of truth.

The recurring flow before distribution

The book will return again and again to one flow because it forces almost every microservice question to appear. An alarm is raised. A flood is detected. A root-cause case opens. Evidence is attached. A verdict is issued. A recommendation becomes available. Audit records what happened. Compliance review is triggered. Reporting catches up later.

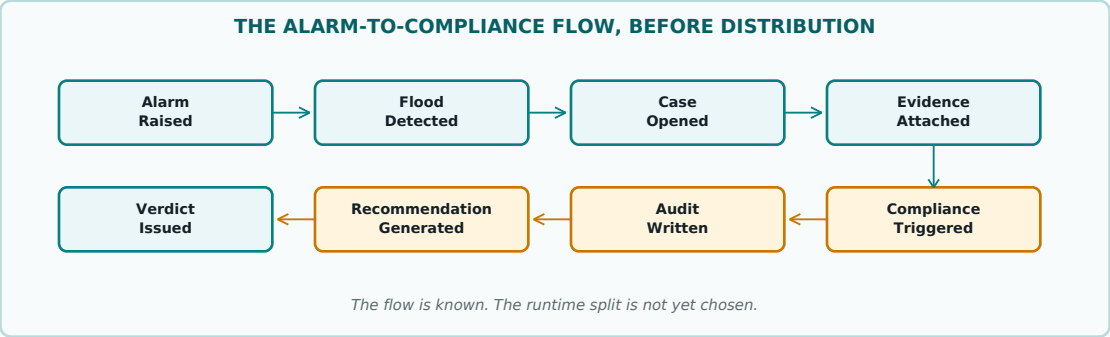


Figure 5.2 - The alarm-to-compliance flow before distribution. The flow is already meaningful as a modular process; later chapters will decide which steps deserve runtime boundaries.

The critical point is that the flow already crosses ownership boundaries before it crosses process boundaries. AlarmManagement owns the alarm. RootCause owns the verdict. Audit owns evidence of the decision. Compliance owns the review. Reporting owns projection. If these ownership changes are unclear inside one process, distributing the flow will not clarify them. It will multiply the places where confusion can hide.

NEXUS-1 SENTENCE TO REMEMBER

An alarm can open a case, but it does not decide the cause. A verdict can trigger review, but it does not become a compliance finding. An audit entry records the trail, but it does not own the workflow.

What must remain strongly consistent at the start

Before service extraction, we need to know which rules are protected by a local transaction or by a single aggregate boundary. These are not implementation details. They are the reasons some parts of the modular core should not be split casually.

Consistency point	Owner	Why it should not be distributed casually
A root-cause case cannot close without evidence.	RootCause	The verdict depends on evidence already accepted by the case. Splitting this prematurely invites partial truth.
An alarm flood should not open duplicate active cases for the same controlled incident rule.	AlarmManagement / RootCause seam	The handoff must be idempotent before it becomes asynchronous.
A recommendation remains advisory.	ReinforcementLearning	The language must prevent any consumer from treating advice as actuator authority.
Audit records the decision with correlation data.	Audit	The audit trail must connect command, event, evidence, and user action without gaps.
Compliance review begins from a issued verdict, not from a draft hypothesis.	Compliance	The review workflow should react to a stable public fact, not a private internal state.

These consistency points do not forbid microservices. They tell us what a future service boundary must preserve. In a modular core, the preservation may be a transaction and an aggregate method. In a distributed architecture, the preservation may require idempotency, an outbox, an inbox, correlation identifiers, retries, and compensation. The rule stays; the mechanism changes.

DESIGN PRINCIPLE

Do not ask the network to protect a rule that the model has not yet expressed locally. First make the invariant visible. Then decide whether a distributed mechanism can preserve it.

What is already asynchronous in design

Even before extraction, NEXUS-1 already contains asynchronous seams in its architecture. From Context to Flow prepared the distinction between internal domain events and public integration events. That distinction now becomes the bridge to services.

Inside `RootCause`, the domain event can be simple and private: `VerdictIssued`. It belongs to the model. Outside `RootCause`, the public contract must be deliberate: `RootCauseVerdictIssued.v1`, represented in C# as `RootCauseVerdictIssuedV1`. The name says what happened and it says that consumers are depending on version one of a contract.

```
// Internal domain fact inside the RootCause model.
public sealed record VerdictIssued(
    RootCauseCaseId CaseId,
    string Verdict,
    DateTime IssuedAtUtc) : IDomainEvent;

// Public integration contract outside the RootCause boundary.
public sealed record RootCauseVerdictIssuedV1(
    Guid EventId,
    string CaseId,
    string UnitCode,
    string Verdict,
    DateTime IssuedAtUtc,
    string CorrelationId);
```

In the modular stage, the public contract may still be written to an outbox table rather than sent through a broker. That is not a weakness. It is preparation. The outbox lets the architecture separate local decision from external publication before the system is physically distributed.

LANGUAGE RULE

`VerdictIssued` is allowed to be private domain language. `RootCauseVerdictIssued.v1` is public integration language. Do not blur them. A private event can evolve with the model; a public event must evolve with consumers.

The database backbone before database-per-service

A future microservice architecture may move toward database-per-service. But the starting point is the NEXUS-1 data backbone: one SQL Server database divided into explicit schemas. The point is not that one database is the final answer. The point is that the current database already teaches ownership if we read it correctly.

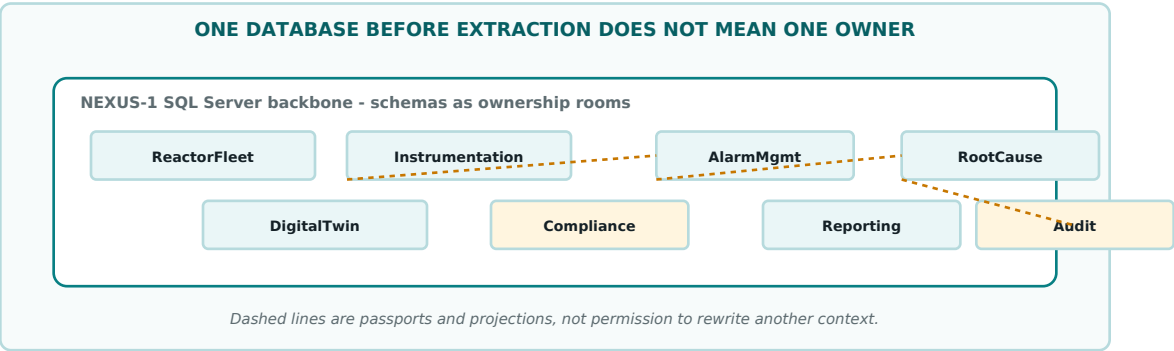


Figure 5.3 - Before extraction, NEXUS-1 can use one database while still keeping ownership visible through schemas, keys, and controlled access paths.

ReactorFleet owns physical structure. Instrumentation owns signals and readings. RootCause owns causal cases and verdict state. Compliance owns findings and review state. Reporting owns projections. Audit owns append-only evidence. A foreign key across schemas is a passport, not a transfer of ownership. It allows one context to refer to another context truthfully, but it does not allow that context to rewrite the truth.

OWNERSHIP RULE

Database-per-service is not a slogan about technology first. It is a rule about ownership. If ownership is unclear in one database, splitting the database will only distribute the confusion.

The operator console before UI composition

The operator console is also part of the starting architecture. Before service extraction, it can still be treated as one browser-based console reading a coherent backend contract. That does not mean the future UI will call every service directly. In fact, it should not.

Console need	Modular-stage answer	Microservice-stage concern
Show unit, alarms, case state, recommendations, audit, and compliance status together.	A local application query or projection can compose the screen.	Direct browser calls to many services can leak topology into the UI.
Explain why a verdict was issued.	RootCause can expose evidence and causal reasoning through an application use case.	Evidence must remain traceable across service boundaries.
Show reporting views that may lag.	Reporting can be marked as a projection even inside one database.	The UI must explain eventual consistency instead of pretending all views are immediate.
Correlate a user action with backend changes.	A single correlation id can pass through the modular core.	Correlation must survive broker hops and retries.

Later, the book will discuss API gateways and Backend-for-Frontend patterns. For now, the rule is simpler: do not make the operator console responsible for knowing the entire service topology. A UI should compose experience; it should not become the distributed transaction coordinator of the system.

UI RULE

The console needs a coherent view, not a direct dependency on every future service. Service extraction should improve backend ownership without turning the UI into a map of backend pain.

The extraction pressures already visible

Not every NEXUS-1 sector feels the same pressure. Some are stable reference owners. Some are high-volume ingestion candidates. Some are decision owners. Some are read-side projection candidates. A serious microservice design notices these differences instead of drawing every box the same size.

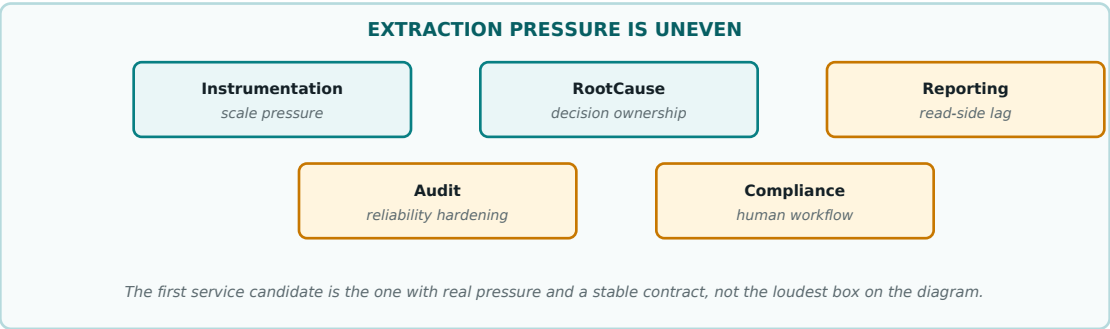


Figure 5.4 - Extraction pressure is uneven. The best first candidate is not necessarily the most interesting domain; it is the boundary with stable language and real operational pressure.

Pressure type	Where it may appear in NEXUS-1	What the architect should check first
Scale pressure	Instrumentation readings, telemetry ingestion, reporting projections.	Is the workload truly different, or can it be handled by a worker or separate read model?
Ownership pressure	RootCause verdicts, Compliance reviews, Audit evidence.	Is the public contract stable enough for other teams or services to depend on?
Release pressure	Reporting changes faster than RootCause rules.	Can the module release independently without breaking shared contracts?
Failure isolation pressure	Reporting or exports should not block diagnosis.	Can the flow tolerate stale projections and delayed delivery?
Compliance pressure	Audit and review records need stronger traceability.	Are correlation, immutability, retention, and replay requirements explicit?

What Chapter 5 freezes for the rest of the book

The rest of the book can now proceed from a fixed starting picture. This does not mean the picture is final. It means we have enough architectural honesty to begin changing it. The later parts will turn this starting model into service-boundary decisions, data-ownership rules, API contracts, integration events, process managers, consistency maps, observability, deployment strategy, and migration steps.

Frozen starting fact	Why it matters later
NEXUS-1 begins as a modular core with explicit sectors.	Service extraction will be judged against existing ownership, not against a blank diagram.
The alarm-to-compliance flow is the recurring test case.	Every communication pattern must prove itself against the same flow.
VerdictIssued and RootCauseVerdictIssued.v1 are not the same event.	Domain truth and public contract evolution will be treated separately.
AuditEntry is evidence, not workflow.	Audit will support traceability without becoming the process manager.
Reporting is projection, not source of truth.	Eventual consistency will be explained rather than hidden.
A recommendation is not a command.	The advisory boundary will survive service design.

IMPORTANT DISTINCTION

This chapter does not choose the final service topology. It chooses the starting truth. That is enough for Part II: finding service boundaries without pretending that every bounded context deserves immediate runtime separation.

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- Why must NEXUS-1 be described as a modular system before it is described as a microservice system?
- Which NEXUS-1 sectors own reference truth, decision truth, evidence truth, workflow truth, and projection truth?
- Why is the alarm-to-compliance flow a better teaching example than a static service diagram?
- Which rules must remain strongly consistent at the start?
- What is the difference between VerdictIssued and RootCauseVerdictIssued.v1?
- Why can one database still express ownership before database-per-service?
- Why should the operator console not become responsible for knowing every backend service?
- Which extraction pressures are already visible in NEXUS-1?

PART I CLOSING

Part I has now made the central warning explicit: microservices are not the starting point. The starting point is mature language, ownership, contracts, consistency, and failure awareness. Part II begins the constructive work: finding service boundaries.

Next chapter

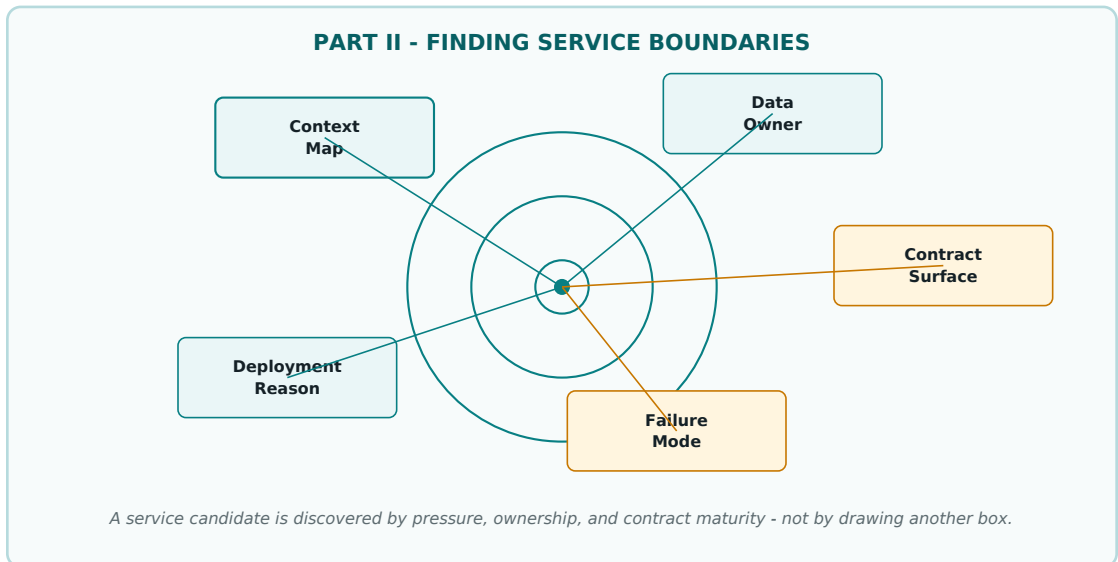
Chapter 6 begins Part II by turning the context map into service candidates. It will ask which contexts are merely modules, which are potential services, and which should remain deliberately inside the modular core for longer.

A service boundary is not drawn where the diagram looks neat. It is drawn where ownership, pressure, and contract maturity meet.

PART II

Finding Service Boundaries

Part II turns the context map into service candidates. It does not ask which boxes can be deployed. It asks which boundaries have enough language, ownership, pressure, and contracts to deserve deployment independence.



CHAPTERS 6-11 - SERVICE CANDIDATES - BUSINESS CAPABILITIES - DATA OWNERSHIP - WORKFLOW - TEAM AND DEPLOYMENT BOUNDARIES

WHAT PART II GIVES YOU

By the end of Part II, the reader should be able to look at ReactorFleet, Instrumentation, AlarmManagement, RootCause, DigitalTwin, ReinforcementLearning, Audit, Compliance, Reporting, and the operator console and classify each one as a strong service candidate, a conditional candidate, or something that should remain deliberately modular for now.

PART II - FINDING SERVICE BOUNDARIES

6. From Context Map to Service Candidate

A context map is not yet a deployment map. It is the evidence from which deployment candidates can be reasoned about without pretending that every boundary deserves a network hop.

Part I closed by making the starting system honest. NEXUS-1 begins as a modular core with explicit sectors and a recurring alarm-to-compliance flow. Part II now asks the next question: which of those sector boundaries could become runtime boundaries, and which ones should remain internal for longer?

The naive answer is simple and wrong: draw one microservice for every bounded context. That answer feels tidy because the names already exist. ReactorFleet, Instrumentation, AlarmManagement, RootCause, Audit, Compliance, and Reporting look like ready-made service names. But a name is not a service. A service must carry responsibility under failure, own write truth, publish stable contracts, and justify its operational cost.

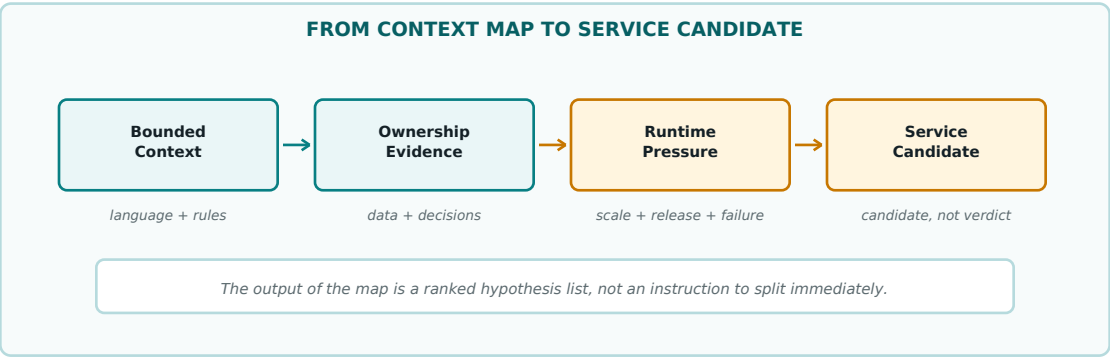


Figure 6.1 - A context becomes a service candidate only after ownership evidence and runtime pressure are examined. The result is a candidate, not a command to extract.

What a context map can and cannot tell you

A context map tells us where language changes and how models depend on one another. It can show upstream and downstream relationships, anti-corruption layers, shared kernels, conformist dependencies, and published language. That is valuable because service extraction without this map is mostly guesswork.

But the map does not answer every runtime question. It does not tell us whether a context has enough traffic to scale separately. It does not tell us whether a team exists to operate it. It does not tell us whether the public contract is stable. It does not prove the database can be separated. It certainly does not guarantee that the operator console will remain usable once the screen needs data from several services.

Context-map signal	What it helps you see	What it does not prove
Separate language	The model has a meaningful semantic boundary.	That the boundary needs independent deployment.
Upstream/downstream dependency	One model depends on another model's published meaning.	That synchronous service calls are safe or acceptable.
Published language	A stable external vocabulary may already exist.	That all consumers are ready for versioned contracts.
Anti-corruption layer	The downstream model needs protection from an external shape.	That a new service should be extracted immediately.
Event flow	A fact crosses a boundary and can be reacted to.	That eventual consistency is acceptable to every user screen.

PLAIN EXPLANATION

The context map is the architectural map of meaning. The service-candidate map is the architectural map of runtime responsibility. The first is necessary for the second, but it is not identical to it.

The dangerous shortcut: service per context

The most common shortcut is to translate every bounded context directly into a service. That shortcut ignores the fact that bounded context is a modelling term while microservice is a runtime term. A model boundary protects language and rules. A runtime boundary adds network failure, independent deployment, security boundaries, monitoring, data replication, versioning, and operational ownership.

The damage appears slowly. A team extracts ReactorFleet because it is a context, then discovers that nearly every other service needs unit identity and reference data. It extracts Reporting because dashboards look separate, then discovers that reporting is a projection and should not own the facts it displays. It extracts Audit because audit feels generic, then accidentally lets audit events drive workflow. The boxes were correct as language boundaries; they were immature as runtime boundaries.

Naive extraction	Why it looks reasonable	What can go wrong
One service per SQL schema.	The schema already marks ownership.	Database ownership is not yet separated; cross-schema joins become remote calls overnight.
One service per aggregate.	Aggregates already protect consistency.	The system becomes too fine-grained and chatty.
One service per screen.	The UI needs visible features.	The UI steals domain ownership and duplicates rules.
One service per team preference.	Teams want independence.	The domain boundary follows organization charts before the model is ready.
One service per fashionable noun.	The diagram looks modern.	The runtime becomes a distributed monolith.

COMMON MISTAKE

Do not convert every context box into a deployable service box. First ask what the context owns, who depends on it, what it publishes, how it fails, and why independent deployment would help.

The five gates of service candidacy

A service candidate is a bounded context that has passed enough readiness gates to be worth deeper design. The gate language is deliberate. A gate can be passed, failed, deferred, or made explicit as a risk. The purpose is not to forbid microservices; it is to stop distribution from becoming a substitute for thinking.

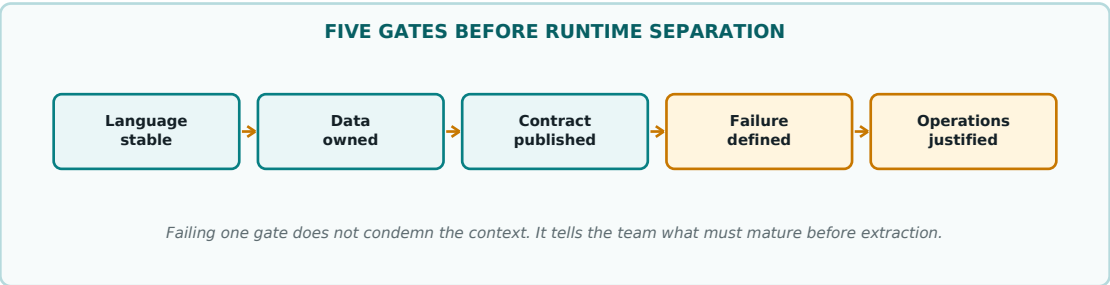


Figure 6.2 - Five gates before runtime separation. The same idea closes From Context to Flow and becomes the starting discipline of this book.

Gate	Question	Ready answer
Language stable	Does this context use words in a controlled way?	Consumers can depend on the published vocabulary without learning private internals.
Data owned	Who owns the write truth?	One context accepts commands and enforces rules for its own facts.
Contract published	What may other contexts call or consume?	APIs and events are smaller than the internal model and versioned deliberately.
Failure defined	What happens when the context is unavailable?	Callers can degrade, retry, queue, or wait without corrupting the domain.
Operations justified	Why deploy independently?	Scale, release cadence, team ownership, resilience, or compliance creates real pressure.

A first NEXUS-1 candidate map

The first candidate map should be conservative. It should not flatter every module by calling it a future service. It should separate strong candidates from conditional ones and from parts of the demonstrator that should remain protected inside a modular core until a much stronger reason appears.

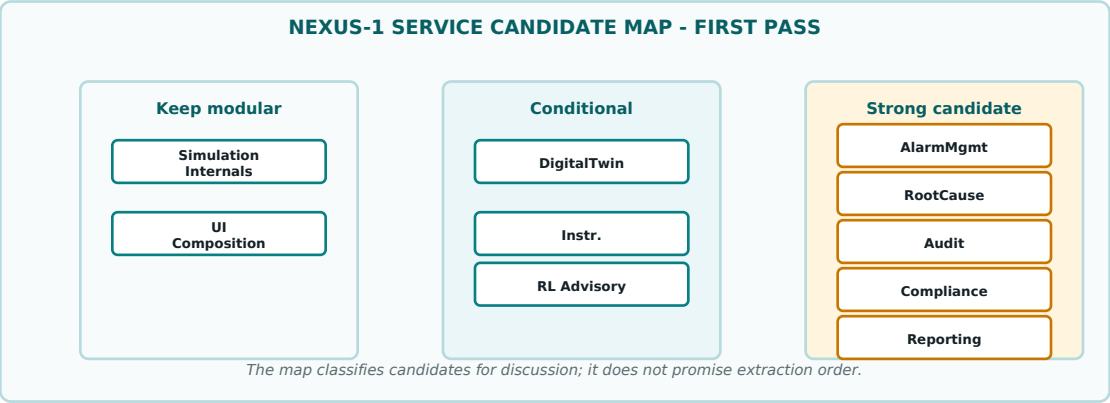


Figure 6.3 - First-pass service-candidate map. The classification is an architectural hypothesis to test in later chapters, not a final deployment promise.

The strong candidates are strong for different reasons. RootCause owns a decision. AlarmManagement owns a stream of alarm facts and flood detection. Compliance owns human review workflow. Audit owns evidence and lineage. Reporting owns projections that can lag without changing source truth. The conditional candidates are not weak; they simply need more careful pressure analysis. Instrumentation may deserve separation for scale. DigitalTwin may deserve separation for model evolution. ReinforcementLearning may become an advisory service if its release cycle and compute profile diverge. ReactorFleet is central reference truth and should be handled with caution.

Candidate evaluation criteria

A service candidate should be evaluated with the same set of questions every time. Without a stable scorecard, the architect will unconsciously reward the most interesting domain or the module with the most dramatic name. The scorecard keeps the decision sober.

Criterion	What to look for	NEXUS-1 example question
Business capability clarity	The context performs a recognizable business or operational capability.	Is RootCause clearly responsible for diagnosis, not alarms or compliance?
Data ownership clarity	One owner changes the write model.	Who may change a RootCause verdict?
Language independence	The context has words that must not leak or be redefined elsewhere.	Does evidence mean the same thing in RootCause and Audit? If not, how is it translated?
Independent deployment value	Separate release would reduce coordination.	Can Reporting evolve dashboards without redeploying diagnosis rules?
Failure isolation value	Unavailable behavior can be isolated.	Can Compliance be delayed without preventing RootCause from issuing a verdict?
Scalability need	The workload profile differs from the rest.	Does Instrumentation ingestion grow faster than case management?
Contract stability	The public API or event can remain stable across releases.	Is RootCauseVerdictIssued.v1 small enough to version safely?
Consistency cost	The split does not require distributed transactions for normal behavior.	Can review be triggered after the verdict event instead of inside the verdict transaction?

Worked candidate: RootCause

RootCause is the clearest early candidate because it owns a real decision. It is not merely a data store and not merely a screen. It protects the RootCauseCase, its evidence, hypotheses, rejections, and verdict. It can publish a public fact when a verdict is issued, and other contexts can react without entering the private model.

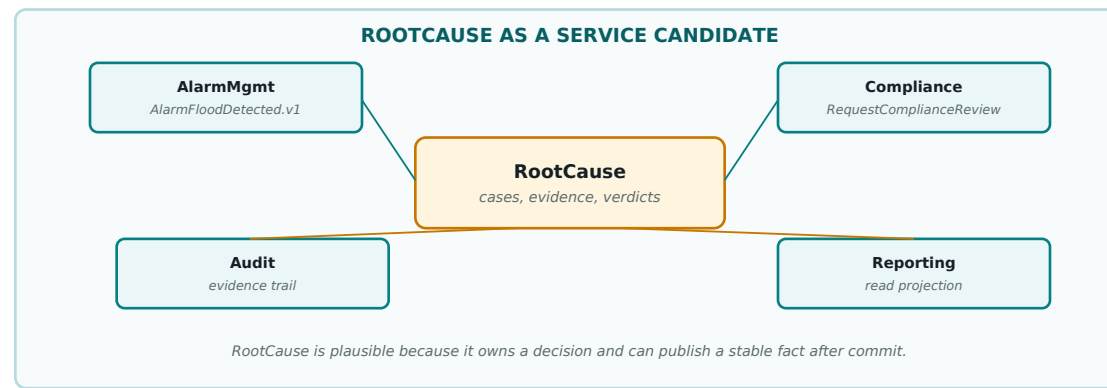


Figure 6.4 - RootCause as a service candidate. The service boundary is plausible because the context owns decisions and can publish facts after committing its own state.

Gate	RootCause assessment
Language stable	Strong. Case, evidence, hypothesis, rejected candidate, verdict, and recommendation are not interchangeable words.
Data owned	Strong. RootCause should own cases, evidence, hypotheses, and verdict state.
Contract published	Strong if the public event remains RootCauseVerdictIssued.v1 and does not expose the aggregate shape.
Failure defined	Moderate. Callers must know what happens if diagnosis is delayed or the service is temporarily unavailable.
Operations justified	Conditional. Technical readiness alone does not prove a separate team, release cadence, or scale requirement.

Worked candidate: AlarmManagement

AlarmManagement is also plausible, but for a different reason. It is closer to event ingestion and operational state than RootCause. It owns alarm definitions, alarm events, acknowledgements, and alarm-flood detection. Its value as a service would come from isolating alarm workload and giving downstream contexts a clean stream of alarm facts.

Aspect	AlarmManagement as a service candidate	Boundary warning
Capability	Raise alarms, track alarm state, detect floods, publish AlarmFloodDetected.v1.	It must not decide root cause.
Data	Owens alarm definitions and alarm events.	Other services may reference alarm ids but should not mutate alarm state.
Contract	Can publish AlarmFloodDetected.v1 to RootCause or a process manager.	The event should not expose every internal alarm-table column.
Failure	Downstream diagnosis may lag if flood publication is delayed.	The system must distinguish alarm truth from diagnosis progress.
Pressure	Possible scale and throughput pressure from alarm streams.	In a demonstration system this pressure may remain simulated, not proven.

TERMINOLOGY DISCIPLINE

AlarmFloodDetected.v1 is a public fact that a downstream context can react to. It is not a command to issue a verdict. It gives RootCause a reason to open or evaluate a case; it does not decide the cause.

Candidates that should remain conditional

A conditional candidate is not a rejected candidate. It is a boundary that may become a service once a specific pressure becomes real. This distinction is important because many architecture mistakes come from extracting a service because it could be a service, not because it needs to be one now.

Conditional candidate	Why it may become a service	Why it may remain modular for now
Instrumentation	High-volume readings, signal quality processing, and ingestion pipelines may need independent scaling.	If the build remains small and demonstrator-grade, a module plus worker may be enough.
DigitalTwin	Model snapshots, divergence detection, and simulation evolution may follow a different release cadence.	If it depends heavily on ReactorFleet and Instrumentation, early separation may create many remote dependencies.
ReinforcementLearning Advisory	Training runs and advisory policy may have separate compute and versioning needs.	An advisory policy can remain a module until independent training/deployment becomes real.
ReactorFleet	Fleet reference data is central and widely used.	Because it is central, extracting it too early can turn every service into a client of reference data. Cache and snapshot strategy must come first.

HONEST BOUNDARY

This draft does not claim that NEXUS-1 currently runs as microservices. It evaluates candidates in an educational architecture. The later implementation chapters will show possible shapes, not certified operational software.

A small .NET shape for candidate analysis

The service-candidate decision can be written down as data before it becomes deployment. That is useful because it keeps the discussion reviewable. A team can score a candidate, attach reasons, and revisit the decision when pressure changes.

```
public sealed record ServiceCandidateAssessment(  
    string ContextName,  
    CandidateStrength Strength,  
    bool StableLanguage,  
    bool OwnsWriteData,  
    bool HasPublishedContracts,  
    bool DefinesFailureBehavior,  
    bool HasOperationalReason,  
    string DecisionNote);  
  
public enum CandidateStrength  
{  
    KeepModular = 0,  
    Conditional = 1,  
    StrongCandidate = 2  
}  
  
var rootCause = new ServiceCandidateAssessment(  
    ContextName: "RootCause",  
    Strength: CandidateStrength.StrongCandidate,  
    StableLanguage: true,  
    OwnsWriteData: true,  
    HasPublishedContracts: true,  
    DefinesFailureBehavior: false,  
    HasOperationalReason: false,  
    DecisionNote: "Decision ownership is strong; runtime extraction waits for failure and operations evidence")
```

This is not intended as production code. It is a modelling tool. The important thing is the shape of the decision: a candidate is not strong because someone likes the name. It is strong because the gates are visible, and the missing gates are not hidden.

SEE IT AS ARCHITECTURE

A service-candidate assessment is like an architecture decision record: it preserves why a context was kept modular, marked conditional, or selected for extraction.

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- Why is a context map not the same thing as a deployment map?
- What goes wrong when every bounded context becomes a service automatically?
- What are the five service-candidate gates?
- Why is RootCause a strong candidate but not automatically the first service to extract?
- Why is AlarmManagement a different kind of candidate from RootCause?
- Why are Instrumentation, DigitalTwin, ReinforcementLearning, and ReactorFleet conditional candidates?
- Why should UI composition logic not become a service merely because the operator console assembles many views?
- How does a service-candidate scorecard protect the architecture discussion from fashion?

CHAPTER 6 CLOSING

The first service-candidate map is now visible. Chapter 7 narrows the lens to business capability boundaries: a service should not only have a name; it should own a meaningful capability that users, operators, or the platform can recognize.

Next chapter

Chapter 7 asks how business capabilities differ from technical folders, database schemas, and UI screens, and why service boundaries become stronger when they follow a capability rather than a noun.

A good service candidate does something the system can name as a responsibility, not just something the codebase can name as a folder.

PART II - FINDING SERVICE BOUNDARIES

7. Business Capability Boundaries

A service boundary becomes stronger when it follows a capability the system must perform, not merely a noun the codebase already contains.

Chapter 6 turned the context map into service candidates. That was a necessary first filter, but it was not enough. A context can be stable and still be the wrong unit of deployment. A service needs a reason to exist at runtime, and one of the strongest reasons is a business capability: a responsibility the platform can name, own, measure, operate, and evolve.

In NEXUS-1, the tempting nouns are everywhere: Unit, Signal, Alarm, Case, AuditEntry, Finding, Report. Those nouns are useful, but they are not automatically service names. A service should not exist because a table exists. It should exist because the system has a capability that needs clear ownership and a public boundary.

PLAIN EXPLANATION

A business capability is something the system must be able to do in domain language: detect an alarm flood, diagnose a cause, preserve evidence, trigger review, project operational status. It is not a table, folder, screen, or controller.

The normal developer trap

A developer opening the solution sees projects. A database engineer sees schemas. A UI developer sees screens. An operator sees work. Microservice design fails when the architecture follows only the first three and forgets the fourth.

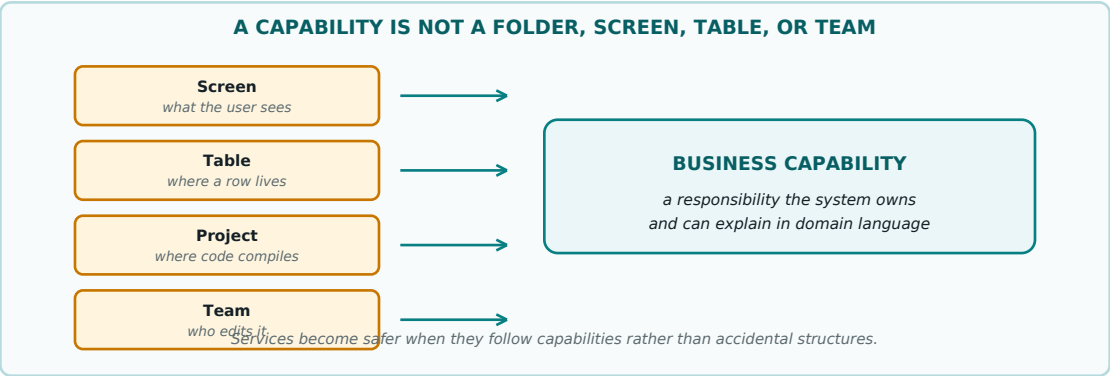


Figure 7.1 - A capability is the responsibility behind the structure. Screens, tables, projects, and teams can reveal a capability, but none of them proves it alone.

This distinction sounds simple, but it changes the whole extraction discussion. The Operator Console may display RootCause, Audit, Compliance, and Reporting in one page, but that does not make the page a service. The RootCause schema may contain the case tables, but that does not make every table an independent service. The Reporting module may have many queries, but a read model is not the source of truth.

COMMON MISTAKE

Do not ask first: which folder should become a service? Ask: which capability needs its own owner, contract, lifecycle, and failure behavior?

What makes a capability service-shaped?

A capability becomes service-shaped when it owns an outcome, not just data. It is responsible for doing something meaningful and for protecting the rules behind that responsibility. That outcome may be immediate, like raising an alarm. It may be analytical, like issuing a root-cause verdict. It may be evidential, like recording an immutable audit trail. It may be read-side, like building a projection for the operator console.

Capability test	Question	NEXUS-1 example
Named responsibility	Can the capability be stated as a verb phrase?	Detect alarm floods; issue root-cause verdicts; trigger compliance review.
Decision ownership	Does it make or protect a decision?	RootCause decides the verdict; Reporting does not.
Write ownership	Does it own the authoritative state it changes?	Compliance owns review state; Audit owns immutable evidence entries.
Public contract	Can other contexts interact through a stable API or event?	RootCauseVerdictIssued.v1, RequestComplianceReview.
Useful failure boundary	Can the rest of the system survive if it is delayed?	Reporting can lag; Compliance can queue review; RootCause delay must be visible.
Independent evolution	Would a separate release cadence reduce coordination?	Dashboards may evolve faster than diagnosis rules.

A capability that fails these questions may still be a good module. It may still deserve a schema, a namespace, and a clean owner. It simply does not yet justify the cost of network separation.

The NEXUS-1 capability map

The NEXUS-1 sectors from the earlier books can be read as capabilities before they are read as services. This reading is more useful because it names what the platform is responsible for, not merely where its code or tables live.

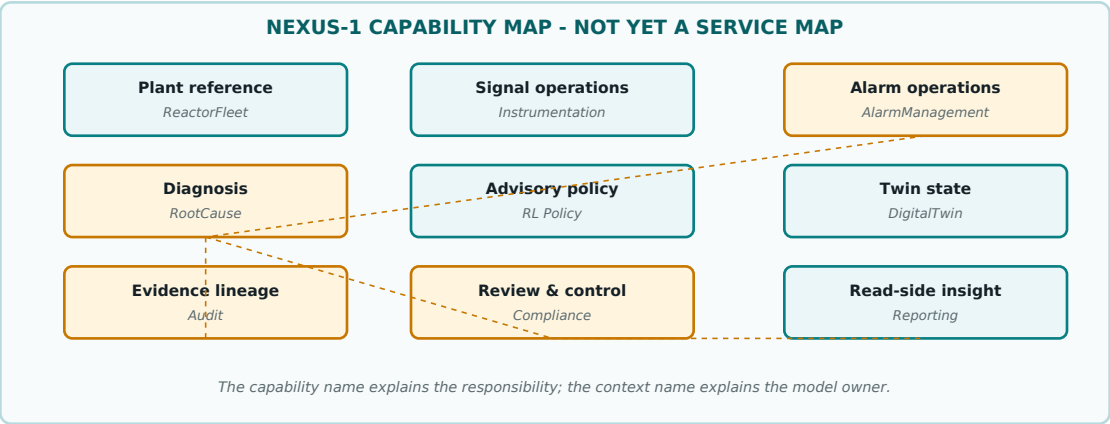


Figure 7.2 - First capability map for NEXUS-1. The map names responsibilities and points at model owners. It remains deliberately separate from the eventual deployment map.

Notice the language. ReactorFleet becomes plant reference, Instrumentation becomes signal operations, AlarmManagement becomes alarm operations, RootCause becomes diagnosis, Audit becomes evidence lineage, Compliance becomes review and control, Reporting becomes read-side insight. The context names are still valid. The capability names say why the context matters.

SERVICE DESIGN RULE

A service name should make sense to a domain conversation before it makes sense to a solution explorer. If a non-technical stakeholder cannot understand what responsibility the service owns, the boundary may still be too technical.

Reading the alarm-to-compliance flow as capabilities

The recurring NEXUS-1 flow can now be read in a different way. Instead of asking which object changes next, we ask which capability is carrying responsibility at each step. This is a better microservice lens because services do not merely hold objects. They accept responsibility for part of a system outcome.

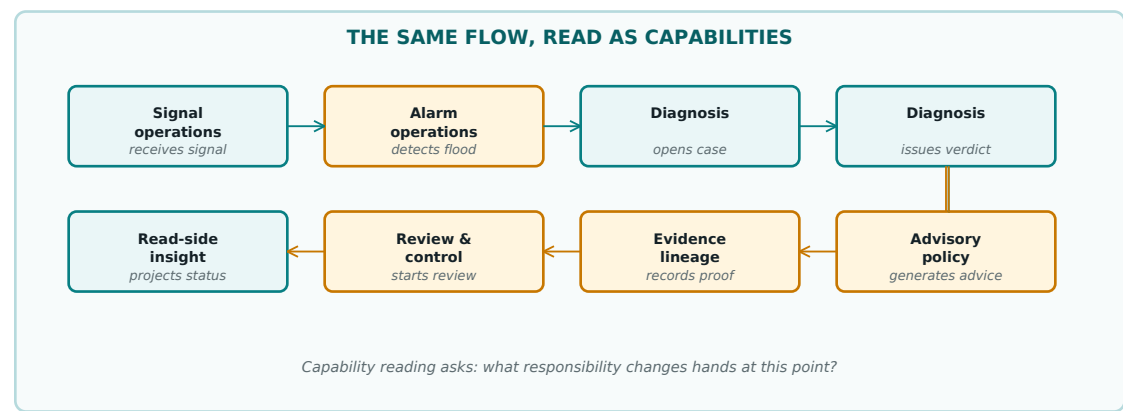


Figure 7.3 - The same alarm-to-compliance flow read as capability transfer. At each handoff, responsibility changes before deployment changes.

Flow step	Capability owner	Boundary meaning
Instrumentation detects or receives signals.	Signal operations	Signal truth is captured before any alarm interpretation.
AlarmManagement raises alarms and detects a flood.	Alarm operations	Alarm truth is created; cause is still unknown.
RootCause opens a case and evaluates evidence.	Diagnosis	A decision process begins, but private hypotheses remain internal.
A verdict is issued.	Diagnosis	A public fact can now be published.
Recommendation is generated.	Advisory policy	Advice is not a command and does not rewrite the verdict.
Audit records evidence.	Evidence lineage	Audit preserves what happened; it does not drive the workflow.
Compliance review is triggered.	Review and control	Human review becomes its own workflow.
Reporting builds a read model.	Read-side insight	Projection follows truth; it does not own truth.

Capability boundaries and data ownership

A capability boundary is weak if it cannot say which data it owns. It is also weak if it owns data but not the decision that changes it. Microservices make this painful because shared writes across services create distributed ambiguity: two services can mutate the same truth, and neither is fully accountable.

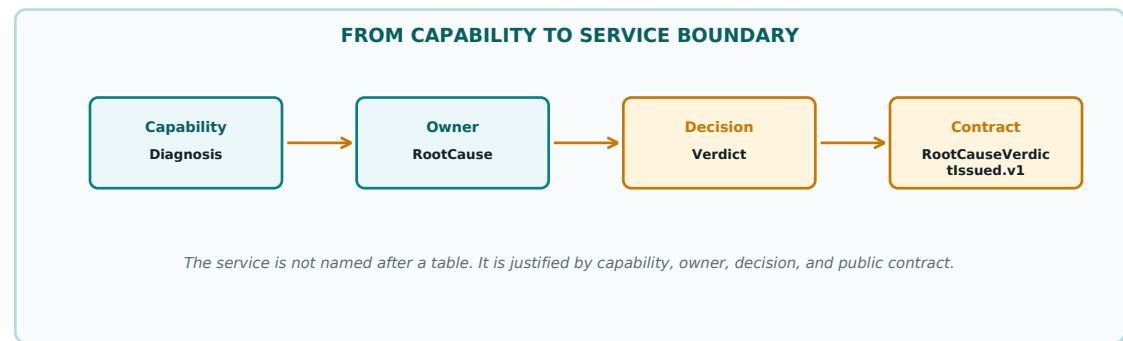


Figure 7.4 - Diagnosis becomes service-shaped only when capability, owner, decision, and public contract line up.

For RootCause, the alignment is strong. The capability is diagnosis. The owner is the RootCause context. The protected decision is the verdict. The public fact is RootCauseVerdictIssued.v1. That chain gives a possible service boundary a reason to exist. It also tells other services what they must not do: they must not write the verdict, reinterpret private hypotheses, or treat an audit entry as a workflow command.

Capability	Owns write truth?	Owns decision?	Public fact or request
Alarm operations	Yes - alarm definitions, events, flood records	Partly - flood detection, not cause	AlarmFloodDetected.v1
Diagnosis	Yes - cases, evidence, hypotheses, verdicts	Yes - verdict	RootCauseVerdictIssued.v1
Evidence lineage	Yes - immutable audit records	No business workflow decision	Audit evidence available to authorized readers
Review and control	Yes - review cases, findings, statuses	Yes - compliance finding and review outcome	RequestComplianceReview
Read-side insight	Yes - projections only	No source-of-truth decision	Queries and dashboard read models

Capability boundaries are vertical slices

A common misunderstanding is to treat a service as one horizontal layer. A team says: the API is a service, the database is another service, the messaging layer is another service. That is not a business capability architecture. It is a distributed layer cake. A capability boundary cuts vertically through API, application use case, domain model, persistence, and contracts.

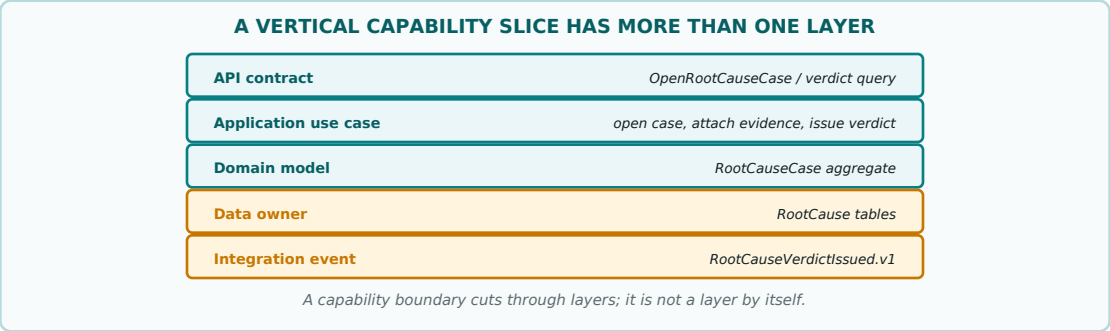


Figure 7.5 - A capability slice crosses layers. The service boundary is not the API alone and not the database alone; it is the responsibility carried through all layers.

This is why the previous backend volumes matter. From Blueprint to Core put the domain and application layers in place. From Context to Flow explained contracts, outbox, inbox, and process managers. This book now asks which vertical slices deserve independent runtime ownership. Without those earlier layers, a microservice would be only a controller pointing at a table.

ARCHITECTURAL MEMORY

A service is not a Web API project with a smaller name. It is a capability boundary with its own model, data ownership, contracts, and operational behavior.

When capability language exposes false services

Capability language is useful because it exposes false services. A false service is a deployable boundary that sounds independent but does not own a meaningful responsibility. It often owns a technical mechanism, a table group, or a UI convenience rather than a domain outcome.

Proposed service	Why it sounds plausible	Capability test result
AuditEntryService	AuditEntry is an important table and appears everywhere.	Too narrow. Evidence lineage is the capability; AuditEntry is one record type.
DashboardService	The operator console needs dashboard data.	UI-shaped. Read-side insight or BFF composition may be valid, but dashboard is not source truth.
UnitService	Many contexts reference Unit.	Dangerous central noun. Plant reference may be a capability, but extraction needs caching and reference-data strategy.
RecommendationService	Recommendations are visible outcomes.	Incomplete. Advisory policy must own recommendation rules and must stay separate from commands.
TableExportService	Exports are used by reporting.	Technical sub-capability. It belongs under read-side insight unless export lifecycle becomes independent.

The cure is to rename proposed services in verbs or responsibilities. If the name becomes awkward, the boundary probably is awkward. If the name becomes clearer - diagnose causes, preserve evidence, review compliance, project status - the boundary is becoming more mature.

NAMING TEST

Try naming the service as a responsibility instead of a noun. If the responsibility is not clear, do not distribute it yet.

A small .NET shape for capability boundaries

A capability boundary can be represented in code before it becomes a service. The code should show which use cases belong together, which contracts leave the boundary, and which internal model remains protected. This is not deployment yet. It is architecture made reviewable.

```
// Capability module before runtime extraction.
namespace Nexus.RootCause;

public static class DiagnosisCapability
{
    public const string Name = "Diagnosis";
    public const string ModelOwner = "RootCause";
    public const string PublicVerdictEvent = "RootCauseVerdictIssued.v1";
}

public interface IRootCauseCommands
{
    Task OpenCaseAsync(OpenRootCauseCase command, CancellationToken ct);
    Task AttachEvidenceAsync(AttachEvidence command, CancellationToken ct);
    Task IssueVerdictAsync(IssueVerdict command, CancellationToken ct);
}

public interface IRootCauseQueries
{
    Task<RootCauseCaseView?> GetCaseAsync(Guid caseId, CancellationToken ct);
}

// The integration contract is smaller than the aggregate.
public sealed record RootCauseVerdictIssuedV1(
    Guid CaseId,
    string UnitCode,
    string VerdictCode,
    DateTime IssuedAtUtc,
    string CorrelationId);
```

The important detail is not the exact interface. The important detail is the boundary discipline. Commands and queries are public surfaces. RootCauseCase remains the protected domain model. RootCauseVerdictIssuedV1 is a stable contract, not a copy of the aggregate.

IMPLEMENTATION SHAPE

Before extraction, a capability can live as a module. Later, the same public surface may become an API and the same event may leave through an outbox. The boundary matures before the deployment changes.

Capability boundaries and the modular monolith

A mature modular monolith can already be capability-oriented. That is one of the central arguments of this book. If the code has capability boundaries, explicit contracts, private models, and clear data ownership inside one deployable unit, then extraction becomes a later operational decision. If those things do not exist, extraction only distributes confusion.

Inside the modular monolith	Later service architecture
Capability module with private domain model.	Service with private domain model.
Internal command handler.	HTTP endpoint or message consumer.
Internal integration event recorded in outbox table.	Message published to broker from outbox.
Local transaction protects aggregate and outbox row.	Local transaction remains; no distributed transaction is introduced.
Other modules consume a stable contract.	Other services consume the same contract, now over the network.
Tests enforce model boundary.	Contract tests and consumer tests enforce runtime boundary.

This is why Chapter 4 defended the modular monolith. It is not a lesser architecture. It is often the place where capability boundaries can be made honest before they become expensive. The microservice version should not invent the boundaries; it should expose boundaries that already work.

HONEST BOUNDARY

This chapter still does not decide final service extraction. It strengthens the candidate map by adding capability ownership. The next chapters add data ownership, workflow boundaries, team and deployment pressure, and finally a NEXUS-1 checklist.

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- Why is a business capability not the same thing as a screen, table, folder, or team?
- Why should a service own an outcome rather than only a noun?
- How does the alarm-to-compliance flow change when read as capability transfer?
- Why does RootCause become stronger as a candidate when described as the diagnosis capability?
- Why is Reporting a read-side capability rather than a source-of-truth owner?
- Why is Audit evidence lineage rather than workflow ownership?
- What is a vertical capability slice?
- How can capability boundaries exist inside a modular monolith before microservices are extracted?

CHAPTER 7 CLOSING

A service candidate now needs more than a context name. It needs a capability, an owner, a decision or responsibility, and a public surface. Chapter 8 continues with the hardest part of all: data ownership boundaries.

Next chapter

Chapter 8 asks who owns the data, who may write it, who may only observe it, and why shared databases become dangerous when service boundaries become real.

A service boundary that cannot say who owns the data is only a network boundary waiting to become a conflict.

PART II - FINDING SERVICE BOUNDARIES

8. Data Ownership Boundaries

A service cannot own a runtime boundary if it does not own a write truth. Data ownership is where a model boundary becomes accountable for facts.

Chapter 7 looked at business capabilities. That gave each candidate service a reason to exist in domain language. This chapter adds the harder question: who owns the data? In a modular monolith the answer can be blurred by one database, one DbContext, one migration set, and one team with permission to fix anything. Across microservices, that blur becomes dangerous.

A service boundary without data ownership is mostly a deployment boundary. It may have its own URL, its own process, and its own dashboard, but if another service can update the same rows, the truth is still shared. The system has bought the cost of distribution without earning the independence that distribution was supposed to provide.

CHAPTER ARGUMENT

Data ownership means one context owns the rules that make a fact valid, the commands that are allowed to change it, the storage boundary that persists it, and the public language by which other contexts learn about it. Reading can be shared. Writing must be owned.

What you already know

In an ordinary database-backed application, it is easy to treat the database as neutral ground. A handler needs a Unit, so it joins ReactorFleet.Unit. A compliance use case needs the root-cause verdict, so it reads the RootCause tables. A reporting query needs everything, so it joins everything. Inside one application this can feel practical, even responsible.

The problem starts when reading and writing are treated as the same kind of access. A screen reading a RootCause verdict is not a problem by itself. A Compliance workflow changing that verdict because the review disagrees is a boundary violation. A Reporting job correcting AlarmEvent rows because the projection looks wrong is a boundary violation. A service that updates another service's tables has not integrated; it has invaded.

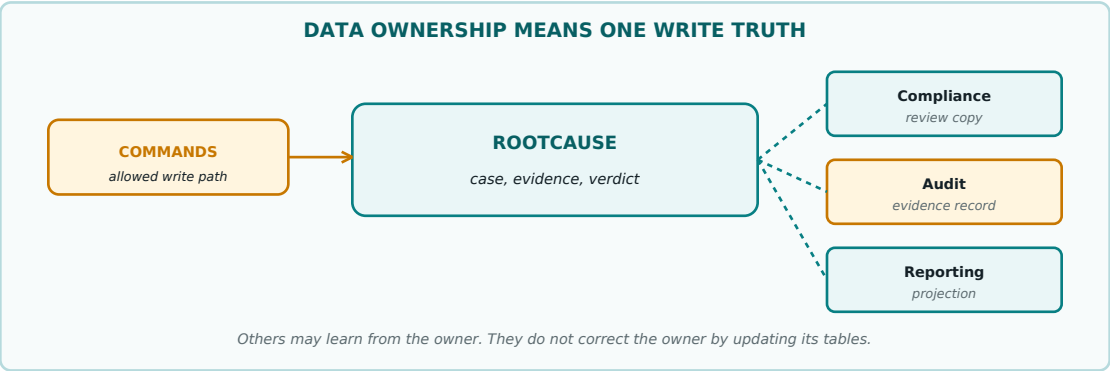


Figure 8.1 - Data ownership in one sentence: RootCause owns the verdict. Compliance, Audit, and Reporting may learn from it, but they do not correct it by writing RootCause tables.

PLAIN EXPLANATION

Data ownership asks: who is allowed to decide what this fact means, who may change it, and which rule must pass before the change is accepted?

Ownership is not the same as possession

A service can possess a copy of data it does not own. A read model may store UnitCode, VerdictCode, or AlarmCount. A Compliance review may store the verdict that caused the review to open. An Audit entry may store a payload containing the same verdict. Those copies are not violations if everyone understands what they are: observations, snapshots, or projections, not the source of truth.

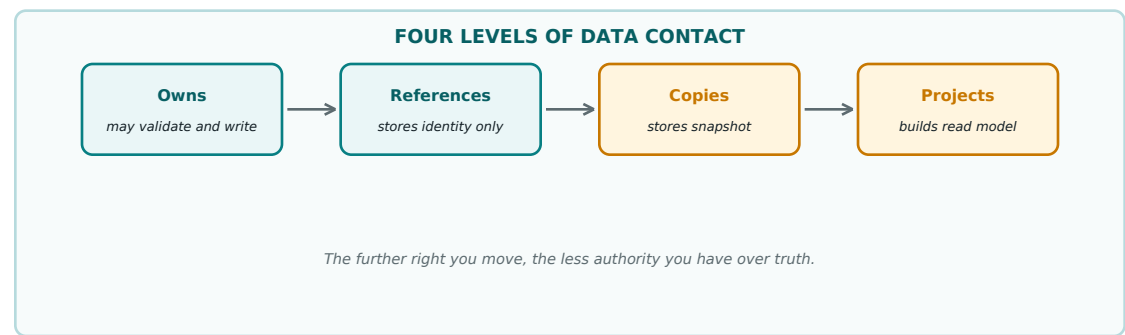


Figure 8.2 - Four levels of contact with data. Only ownership gives write authority over truth. References, copies, and projections need clear labels.

Contact type	What it means	NEXUS-1 example
Owns	This context validates and writes the authoritative state.	RootCause writes RootCauseCase and Verdict.
References	This context stores an identity owned elsewhere.	AlarmEvent stores UnitId or UnitCode to identify the physical unit.
Copies	This context stores a snapshot for its own process.	Compliance stores the verdict code that opened a review.
Projects	This context stores a read-side shape built from events.	Reporting stores an operator dashboard projection.

The first NEXUS-1 ownership map

The NEXUS-1 sectors already suggest a data ownership map, but the map must be stated explicitly. In the earlier schema books, cross-schema foreign keys were passports between sectors. In a microservice discussion, those passports become design pressure: they tell us where a future service must replace direct table access with references, contracts, snapshots, or events.

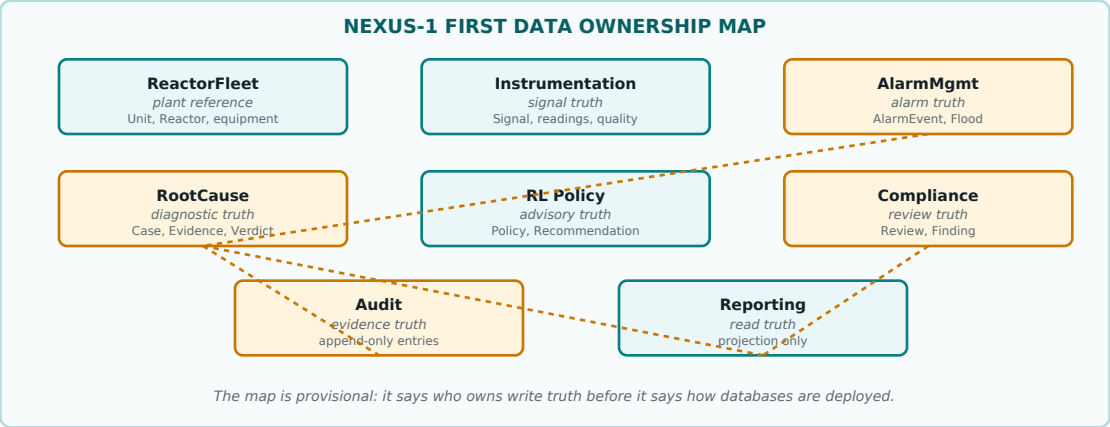


Figure 8.3 - First ownership map for NEXUS-1. Each box names a write truth. This is not yet database-per-service implementation; it is the accountability map that must exist before implementation.

The map also warns us where extraction will be hardest. ReactorFleet is central reference truth. Instrumentation owns high-volume signal truth. AlarmManagement owns alarm lifecycle truth. RootCause owns diagnostic truth. Audit owns evidence truth. Reporting owns no business decision; it owns read models.

SERVICE BOUNDARY RULE

A service candidate becomes weaker if it cannot name the facts it owns and the facts it only observes. A service that owns nothing usually becomes a facade. A service that writes everything becomes a distributed monolith.

Who may write, who may observe

The most practical rule is also the most severe: only the owner writes the owner's truth. Everyone else observes through a contract. This does not mean every read must become slow or remote. It means a non-owner cannot quietly correct another model by reaching into its tables.

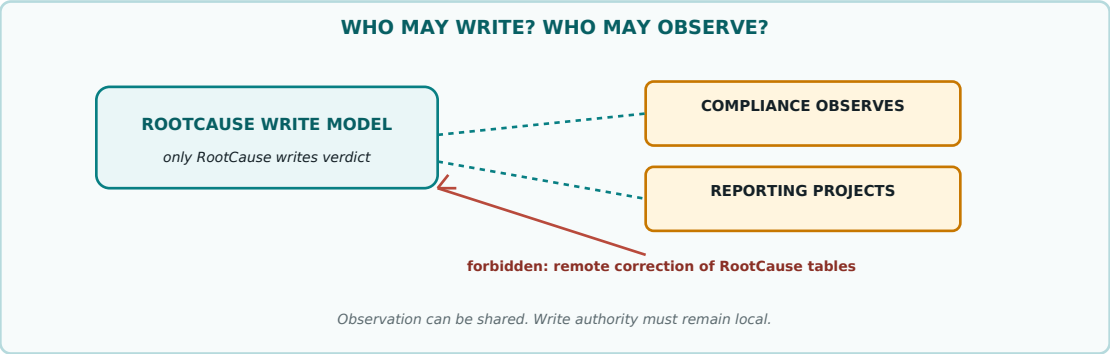


Figure 8.4 - RootCause owns the verdict write model. Other contexts may observe through events, APIs, or projections. They may not update the verdict directly.

Fact	Owner	Allowed non-owner use	Forbidden use
Alarm lifecycle	AlarmManagement	RootCause references alarm ids and consumes flood events.	RootCause changes alarm state to fit a diagnosis.
Root-cause verdict	RootCause	Compliance opens review from RootCauseVerdictIssued.v1.	Compliance edits the RootCause verdict row.
Audit evidence	Audit	All contexts append through an audit contract.	Audit becomes the workflow source of truth.
Compliance finding	Compliance	Reporting projects finding status.	Reporting closes or supersedes a finding.
Report projection	Reporting	Operator Console reads it.	Core domain decisions depend on it as truth.

The shared database temptation

The easiest way to fake microservices is to put APIs in front of modules while leaving the database shared and mutable. The system looks distributed in deployment diagrams, but every service still reaches the same truth store. That is not a transition state to celebrate; it is a risk to manage consciously.

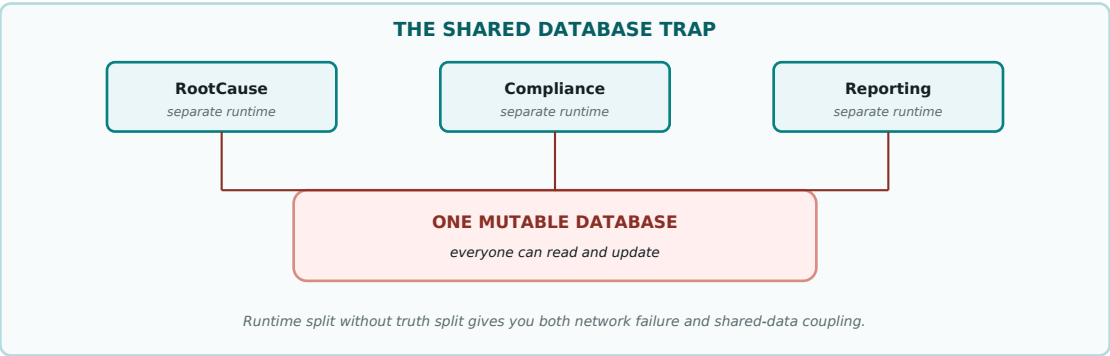


Figure 8.5 - The shared database trap. Services are separate at runtime but coupled by one mutable truth store. Failures now come from the network and the database contract at the same time.

Symptom	What it reveals	Consequence
Two services update the same table	No single owner of write truth.	Invariants can be bypassed.
One service depends on another service's private view	Private model leaked as public contract.	Schema change becomes system-wide breakage.
Reporting table is used by workflow	Projection confused with source truth.	Lag or rebuild changes business behavior.
Manual fixes run across schemas	Operational ownership is unclear.	Incidents become data archaeology.
Every service needs every database permission	Security boundary is fictional.	A compromise in one service can mutate all truth.

Cross-service data without shared writes

A microservice architecture still needs data from outside its boundary. The discipline is not isolation from knowledge. The discipline is isolation of authority. A service may store an external identifier, cache a reference, copy a snapshot, subscribe to an integration event, or build a projection. What it should not do is treat another service's tables as its own persistence layer.

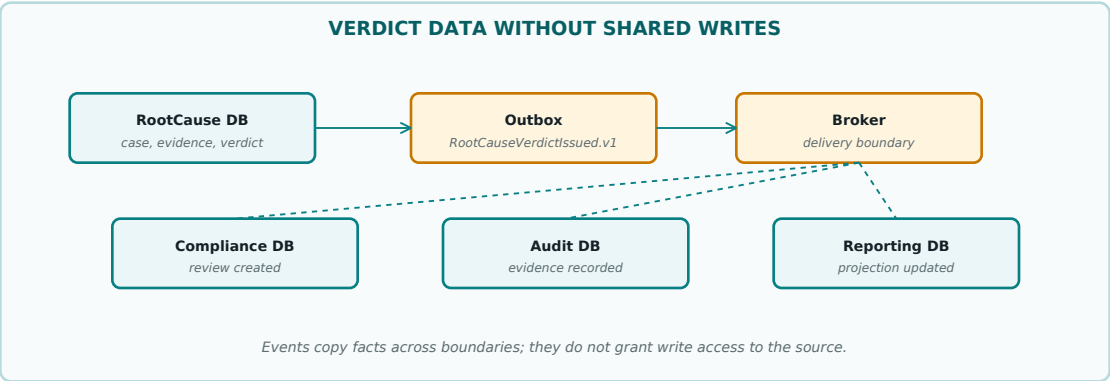


Figure 8.6 - RootCause publishes a verdict event. Compliance, Audit, and Reporting build their own state from that fact without sharing RootCause write tables.

Pattern	Use it when	Cost
Reference by ID	The other context owns a stable identity.	You may need lookup or validation at the boundary.
Snapshot copy	Your process needs the value as it was when the process began.	The copy can become stale and must be labelled as a snapshot.
Event subscription	You need to react when another context changes truth.	Delivery, duplicates, ordering, and retries must be handled.
Read projection	A screen or report needs fast combined data.	Projection lag must be visible and acceptable.
Synchronous query	You need current external state before proceeding.	Availability and latency of the other service enter your use case.

A small .NET shape for ownership

Code can enforce the ownership rule before the system becomes distributed. The first step is not a broker or Kubernetes. The first step is to stop other modules from writing through the wrong repository. A RootCause use case should depend on RootCause persistence. Compliance should react to a public contract and write Compliance state, not RootCause state.

```
// RootCause owns the write model for cases and verdicts.
namespace Nexus.RootCause.Application;

public interface IRootCauseWriteStore
{
    Task<RootCauseCase?> LoadCaseAsync(RootCauseCaseId id, CancellationToken ct);
    Task SaveCaseAsync(RootCauseCase rootCauseCase, CancellationToken ct);
    Task AddOutboxAsync(IntegrationMessage message, CancellationToken ct);
}

// Compliance consumes the published fact and writes its own review state.
namespace Nexus.Compliance.Application;

public sealed class RootCauseVerdictConsumer
{
    private readonly IComplianceWriteStore _store;

    public async Task HandleAsync(RootCauseVerdictIssuedV1 message, CancellationToken ct)
    {
        var review = ComplianceReview.OpenFromVerdict(
            caseId: message.CaseId,
            unitCode: message.UnitCode,
            verdictCode: message.VerdictCode,
            requestedAtUtc: message.IssuedAtUtc);

        await _store.SaveReviewAsync(review, ct);
    }
}

// Notice what is missing: Compliance does not receive IRootCauseWriteStore.
```

The missing dependency is the design. If Compliance cannot see the RootCause write store, it cannot accidentally correct the verdict by bypassing the published language. It can only create its own review state from the public event.

Transactions stop at the ownership boundary

Inside one owner, strong consistency can still be local. RootCause can load a case, attach evidence, issue a verdict, and record an outbox message in one local transaction. That is not the same as making Compliance review, Audit evidence, and Reporting projection update in the same transaction. The first is local consistency. The second is a distributed transaction disguised as convenience.

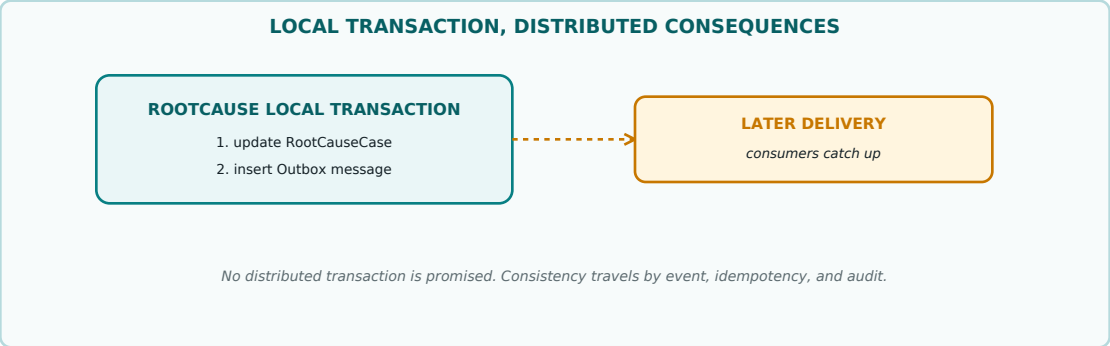


Figure 8.7 - The source service commits its own state and outbox atomically. Consumers catch up later through delivery, idempotency, and audit.

Boundary	Consistency rule	NEXUS-1 consequence
Inside RootCause aggregate	Strong local consistency.	A case cannot issue a verdict without evidence.
RootCause DB + Outbox	Strong local consistency.	The verdict and message record commit together.
RootCause to Compliance	Eventual consistency.	Review may start seconds later and must be traceable.
RootCause to Reporting	Eventual consistency.	Operator projection can lag but must show freshness.
Audit timeline	Append-only evidence discipline.	Late events are recorded, not hidden.

IMPORTANT DISTINCTION

Data ownership does not eliminate transactions. It puts transactions in the right place: inside the owner, not across every consumer that wants to react.

When shared data is still acceptable

This chapter should not be read as a simplistic rule that no table, reference, or database can ever be shared during a transition. Real migrations are staged. A modular monolith may still hold all schemas in one database while ownership rules are being made executable. A service extraction may temporarily read from a shared database while writes are being moved. The honest question is whether the sharing is a controlled transition or the permanent architecture in disguise.

Situation	Acceptable if	Warning sign
One database, schema-per-context	Each schema has one write owner and tests prevent cross writes.	Any module can update any schema because it is convenient.
Temporary direct read during extraction	There is a removal plan and read contract replacement.	The direct read becomes undocumented platform behavior.
Shared reference data	The data is stable, low-risk, and governed centrally.	Every service blocks on central lookup availability.
Shared identity provider	Identity is infrastructure, not domain truth.	External claims are treated as business rules without translation.
Reporting database	It is clearly projection-only and rebuildable.	Workflows depend on reporting rows as source truth.

The NEXUS-1 books have already used a single data backbone as a teaching device. That remains valuable. This book now adds a different lens: when a sector becomes a service, the shared backbone must be replaced by ownership boundaries, published contracts, and read-side copies. The old schema map teaches where the dependencies are; it does not by itself grant service autonomy.

HONEST BOUNDARY

This chapter defines ownership boundaries. It does not yet implement database-per-service. Part III will return to that rule in detail and explain why it is a rule about ownership before it is a rule about servers.

Data ownership checklist for service candidates

A context that passed the capability test in Chapter 7 should now pass the data ownership test. If it cannot, keep it modular. Fix the ownership first. Distribution will not repair an unclear write truth; it will only make the confusion harder to observe and more expensive to recover from.

Question	Good answer	Weak answer
What facts does the candidate own?	A short list of authoritative write models.	A list of screens or tables it happens to use.
Who may change those facts?	Only commands handled inside the owner.	Several services with database access.
How do others learn about changes?	Public API, integration event, projection, or snapshot.	They query private tables or views.
Can consumers work with copies?	Yes, copy semantics and freshness are explicit.	No, everyone needs the live table.
Can the owner enforce invariants locally?	Yes, within one aggregate or local transaction.	No, correctness requires a distributed write.
Can the data be restored or replayed?	Events, audit, and ownership make recovery explainable.	Manual cross-table repair is required.

For NEXUS-1, RootCause looks stronger after this checklist than many other candidates because it owns a meaningful write model: cases, evidence, hypotheses, rejected candidates, and verdicts. Reporting looks different: it may deserve independent scaling and deployment, but it owns projections, not business truth. ReactorFleet is important but dangerous to extract too early because almost every other sector depends on plant identity.

DESIGN MEMORY

The owner of data is the owner of correction. If a verdict is wrong, RootCause corrects it by publishing a correction event. Compliance and Reporting react. They do not overwrite the original truth silently.

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- Why is data ownership stronger than simply possessing a copy of data?
- Why is writing another context's tables a service-boundary violation?
- What is the difference between a reference, a snapshot, and a projection?
- Why is Reporting not the source of truth even if the operator reads Reporting screens?
- Why can RootCause commit a verdict and an outbox message locally without using a distributed transaction?
- Why can one database still exist during a modular-monolith phase if write ownership is enforced?
- Which NEXUS-1 sectors look strong as data owners, and which are more dangerous to extract early?

CHAPTER 8 CLOSING

A service that owns no data is usually a facade. A service that lets everyone write its data is not autonomous. The next chapter adds workflow boundaries: where does a process begin, who carries it forward, and what happens when ownership changes hands over time?

Next chapter

Chapter 9 studies workflow boundaries. It asks which service owns a step, which service owns the overall process, and why a distributed flow cannot be treated like one long application service.

A data boundary says who owns a fact. A workflow boundary says who is responsible for what happens next.

PART II - FINDING SERVICE BOUNDARIES

9. Workflow Boundaries

A data boundary tells you who owns a fact. A workflow boundary tells you who owns the next responsibility when that fact begins to move.

Chapters 6 through 8 looked at service candidates from three directions: context maps, business capabilities, and data ownership. Those are necessary tests, but they are not enough. A system also has movement. A signal becomes an alarm. An alarm flood becomes a root-cause case. A case gathers evidence. A verdict becomes a public fact. That fact may request compliance review, feed reporting, and leave an audit trail.

The mistake is to treat that movement as one long function. In a modular monolith, this may begin as a convenient application service. In a microservice architecture, the same habit becomes a distributed transaction by instinct, even if no transaction coordinator is present. The result is not a workflow. It is a chain of hopes.

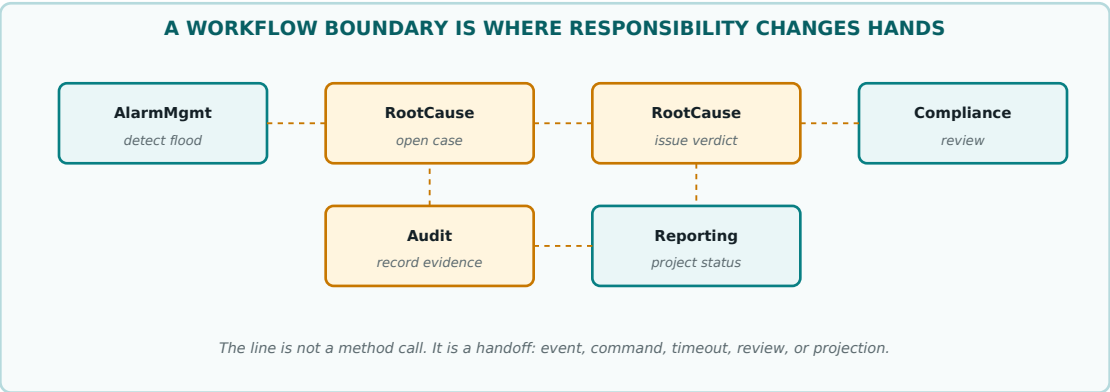


Figure 9.1 - A workflow boundary appears when responsibility moves from one owner to another. The arrow should be a named fact, command, timeout, or projection - not an invisible call stack.

CHAPTER ARGUMENT

A workflow boundary is a responsibility boundary over time. It decides which service owns a step, which component remembers the cross-step process, what public fact or command crosses the boundary, and what happens when the next service is unavailable.

What you already know

You have written use cases that felt innocent at first and then became too long. The method starts by validating input. Then it loads an aggregate. Then it calls another module. Then it writes an audit row. Then it updates a read model. Then it sends an email. Then it retries something because the downstream call failed. Six months later, nobody knows whether the method is a use case, a workflow engine, a transaction script, or a nervous system.

That is the ordinary developer problem behind workflow boundaries. The code does not become dangerous because it is long. It becomes dangerous because it hides responsibility. It no longer says which context owns which decision, which failure is local, which failure is downstream, which part can be retried, and which part must be shown to a human as pending.

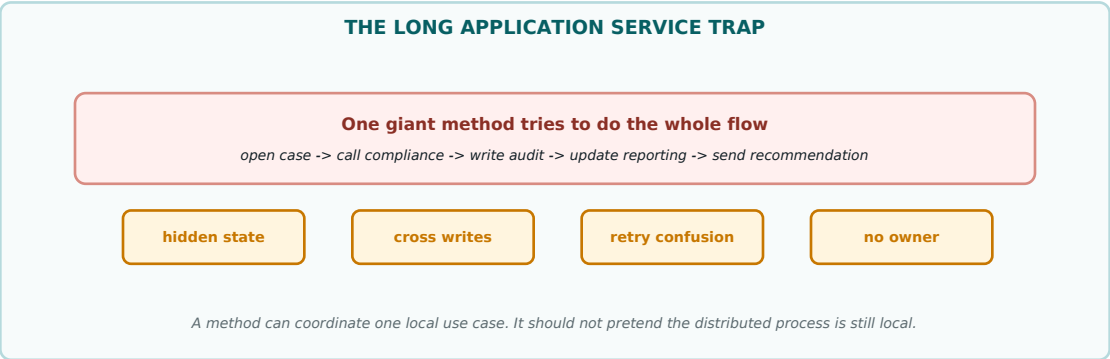


Figure 9.2 - The long application service trap. A single method attempts to perform a cross-context process as if all participating owners were still local.

PLAIN EXPLANATION

A use case completes one local intention. A workflow remembers progress across several intentions. When the process needs memory, waiting, retry, escalation, or cross-context handoff, it has crossed the line from application service to workflow.

Step owner, process owner, observer

The first distinction is simple but powerful. The step owner owns a local business decision. The process owner remembers where the multi-step process is and what should happen next. An observer builds visibility from events but does not drive the process. When these roles are blurred, services start doing each other's work.

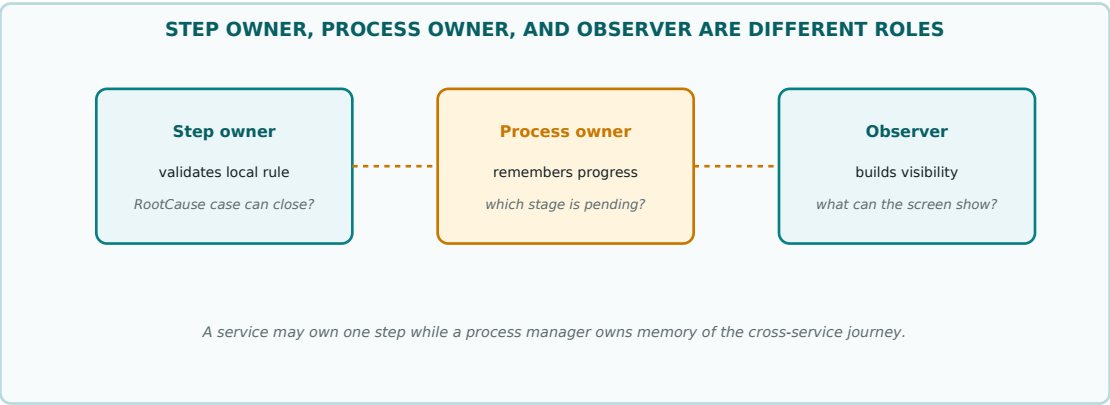


Figure 9.3 - A workflow has roles. A service can own a step without owning the entire process. Reporting can observe the process without controlling it.

Role	Question it answers	NEXUS-1 example
Step owner	Is this local action valid now?	RootCause decides whether a case may issue a verdict.
Process owner	What stage is the cross-context process in?	Alarm-to-compliance process tracks verdict observed, review requested, review opened, or escalated.
Observer	What should a user or report see?	Reporting projects case status and review status for the operator console.
Audit evidence	What happened, when, and because of what?	Audit records events and commands but does not decide the next business step.

Applying the idea to NEXUS-1

The NEXUS-1 alarm-to-compliance flow is useful because no single context can honestly own the whole thing. Instrumentation owns signals. AlarmManagement owns alarm lifecycle and flood detection. RootCause owns cases, evidence, hypotheses, and verdicts. ReinforcementLearning owns advisory policy and recommendation generation. Audit owns append-only evidence. Compliance owns review and findings. Reporting owns read-side visibility.

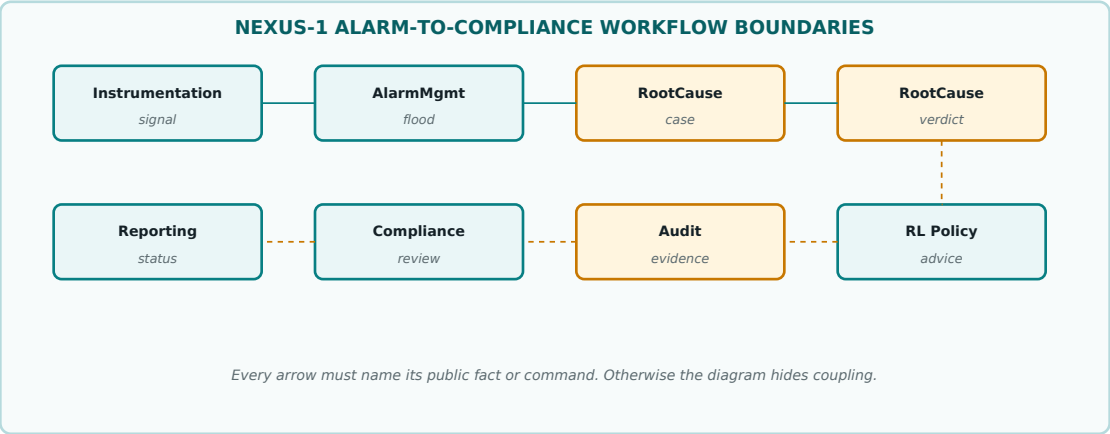


Figure 9.4 - The recurring NEXUS-1 flow read as workflow boundaries. The flow is coherent, but responsibility changes several times.

The diagram does not say that every box must be a separate service today. It says something more important: if a box later becomes a service, the handoff lines already need names. A handoff from RootCause to Compliance cannot be an informal database join. It needs a published fact and, when action is requested, a command with an owner allowed to refuse it.

NEXUS-1 RULE

Do not ask one service to own the whole alarm-to-compliance journey. Ask each service to own its step, and introduce durable process memory only for the parts that cross ownership boundaries over time.

Events and commands at workflow boundaries

A healthy workflow boundary keeps facts and requests separate. An event says something has happened and cannot be refused by the past. A command asks another owner to try something and can be rejected by that owner. Confusing the two makes workflows brittle.

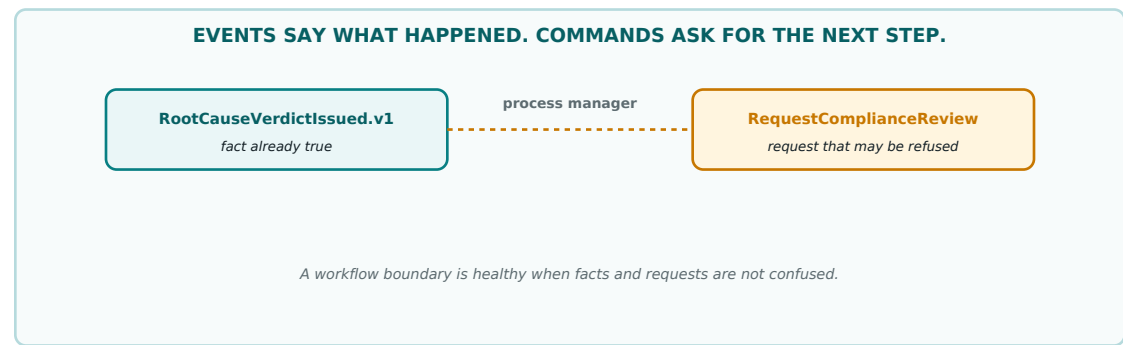


Figure 9.5 - RootCause emits a public fact. A process manager may request a compliance review. Compliance still decides whether the request is valid in its own model.

Boundary message	Kind	Meaning	Owner of validity
VerdictIssued	Internal domain event	RootCause aggregate changed inside its own boundary.	RootCause model
RootCauseVerdictIssued.v1	Integration event	A verdict is now a public fact for consumers.	RootCause published language
RequestComplianceReview	Command/request	Please open a review for this verdict.	Compliance application and domain rules
RootCauseVerdictCorrected.v1	Integration event	The earlier public fact was corrected.	RootCause published language
AuditEntry	Evidence record	Something happened and was recorded.	Audit evidence model, not workflow control

Waiting is part of the model

The moment a workflow crosses service boundaries, waiting becomes real. Compliance may be temporarily unavailable. The compliance review may be created but not acknowledged. Evidence may arrive late. A recommendation may be delayed because the policy service refuses to produce advice under uncertain data. A human reviewer may need time. None of those states should be hidden inside a blocked thread, a retry loop, or an operator's memory.

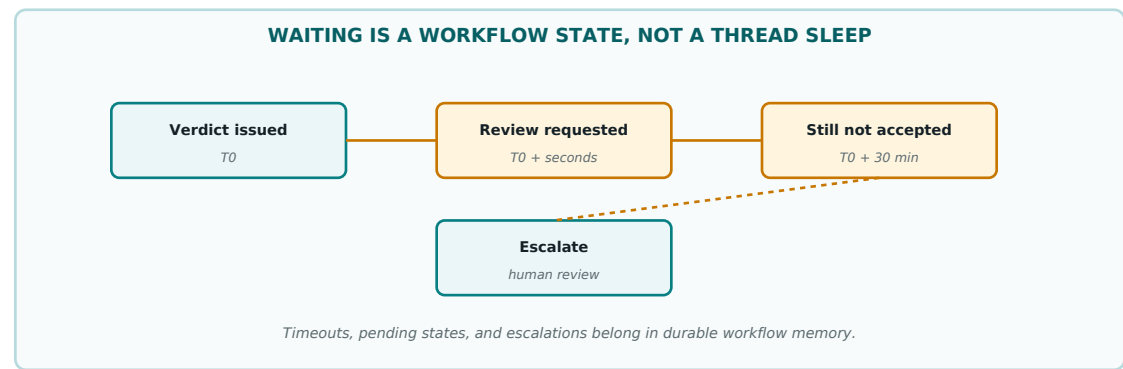


Figure 9.6 - Waiting must be represented as durable process state. A timeout is not impatience; it is a named rule for what should happen if the next responsibility does not arrive in time.

Workflow condition	Bad design	Better design
Compliance service unavailable	Keep retrying inside the request until it times out.	Record pending command, retry with policy, expose pending status.
Review not acknowledged	Assume success because the command was sent.	Track review requested and escalate after configured time.
Evidence arrives after verdict	Rewrite history silently.	Publish correction or add follow-up evidence according to RootCause rules.
Recommendation cannot be generated	Return fake advice for UI completeness.	Record abstention or pending recommendation honestly.

Durable workflow memory

A workflow that remembers only in code cannot survive a restart. A workflow that remembers only in audit cannot easily decide what to do next. The process needs operational memory: correlation id, current stage, pending command, last observed event, retry count, timeout, and failure reason. Audit can prove what happened. Process state tells the system what to do now.

DURABLE PROCESS STATE MAKES THE FLOW AUDITABLE AND RESTARTABLE

CorrelationId	<i>ties events and commands together</i>
Stage	<i>VerdictObserved / ReviewRequested / Escalated</i>
PendingCommand	<i>what should be sent or retried</i>
NextTimeoutUtc	<i>when waiting becomes action</i>
LastEventId	<i>idempotency and replay safety</i>

Process state is operational memory. Audit is evidence. They cooperate, but they are not the same table.

Figure 9.7 - Process state is not audit. It is the durable memory that lets the workflow resume, retry, escalate, or declare itself complete.

```
public sealed class AlarmToComplianceProcess
{
    public Guid CorrelationId { get; private set; }
    public string Stage { get; private set; } = "WaitingForVerdict";
    public string? PendingCommandType { get; private set; }
    public DateTime? NextTimeoutUtc { get; private set; }
    public Guid? LastEventId { get; private set; }

    public void RecordVerdict(RootCauseVerdictIssuedV1 message, DateTime nowUtc)
    {
        if (LastEventId == message.EventId) return; // idempotent replay
        Stage = "VerdictObserved";
        PendingCommandType = "RequestComplianceReview";
        NextTimeoutUtc = nowUtc.AddMinutes(30);
        LastEventId = message.EventId;
    }
}
```

A small .NET process manager shape

The implementation shape remains modest. The process manager reacts to a public event, loads or creates process state by correlation id, records progress, and emits the next command through an outbox. Notice what it does not do: it does not directly mutate Compliance tables, it does not treat Audit as workflow memory, and it does not call every participant in one synchronous chain.

```
public sealed class AlarmToComplianceProcessManager
{
    private readonly IProcessStore _processStore;
    private readonly ICommandOutbox _commandOutbox;
    private readonly IClock _clock;

    public async Task HandleAsync(RootCauseVerdictIssuedV1 message, CancellationToken ct)
    {
        var process = await _processStore.GetOrCreateAsync(message.CorrelationId, ct);
        process.RecordVerdict(message, _clock.UtcNow);

        if (process.ShouldRequestComplianceReview())
        {
            await _commandOutbox.EnqueueAsync(new RequestComplianceReview(
                correlationId: message.CorrelationId,
                rootCauseCaseId: message.RootCauseCaseId,
                verdictCode: message.VerdictCode), ct);
        }

        await _processStore.SaveAsync(process, ct);
    }
}
```

DESIGN BOUNDARY

The process manager may request the next step. The target service still owns whether the request is valid and what state changes follow. Coordination is not authority over every model.

Workflow boundary smells

Workflow boundary problems usually appear before the team names them. You see a use case that needs too many repositories. You see a controller waiting for three services to answer. You see retry logic scattered in handlers. You see audit rows being queried to decide the next business action. You see reporting projections used to control workflow because they are convenient to join.

Smell	What it probably means	NEXUS-1 correction
One handler injects many context stores	Several owners are being coordinated as if local.	Introduce a process manager or split into events and commands.
A service updates another context's table	Step ownership has been bypassed.	Send a command or publish an event; let the owner write.
Audit is queried for next action	Evidence is being confused with process state.	Create durable workflow state.
A screen shows impossible status	The process has no visible stage model.	Expose pending, failed, retrying, and escalated states.
Timeout logic lives in HTTP request	Waiting is not durable.	Store timeout and run scheduled/resilient handling.
Reporting projection drives decisions	Read model became false source of truth.	Move decision to owning service or process state.

The smell is not that several services participate. That is the point of a workflow. The smell is that the participation is hidden, unnamed, or made to look atomic when it is not.

COMMON MISTAKE

Do not solve a workflow boundary by creating one service that owns every step. That only moves the distributed monolith into a new box. The question is not "where can we put all the logic?" The question is "which owner is responsible for each decision, and what durable memory connects the decisions?"

What should remain local

Not every sequence is a workflow boundary. Some sequences belong inside one aggregate or one local application service. The difference is not the number of lines of code. The difference is ownership. If one owner can see the rule, change the state, and commit the result locally, keep it local. Distribution would only add failure modes.

Scenario	Keep local when	Make workflow when
Attach evidence to a case	RootCause owns the case and evidence rule.	Evidence comes from another owner after delays or approvals.
Issue verdict	RootCause can validate evidence and close locally.	Verdict triggers review, reporting, audit, and delayed recommendations.
Acknowledge alarm	AlarmManagement owns state change and actor rule.	Acknowledgement must coordinate external investigation or compliance SLA.
Generate recommendation	RL Policy can create advisory output from local inputs.	Recommendation requires pending external evidence or human confirmation.
Update dashboard	Reporting rebuilds projection from events.	The dashboard update is used to drive business workflow.

This is the discipline that keeps a microservice design from becoming theatrical. A workflow boundary should earn its existence. It should appear because ownership, time, and failure require it - not because the diagram looks more impressive with more arrows.

LOCALITY RULE

Keep local invariants local. Make cross-owner progress explicit. Do not distribute a rule that one aggregate can enforce, and do not hide a cross-context journey inside one aggregate or handler.

Workflow boundary checklist

Before a service candidate is extracted, run the workflow checklist. The goal is not to produce a beautiful choreography diagram. The goal is to know what happens when the next owner is slow, unavailable, duplicate, out of order, or allowed to refuse the request.

Question	Strong answer	Weak answer
Where does the workflow begin?	A named event or command starts it.	A controller method happens to call several services.
Who owns each step?	Each step has a local owner with rules.	One coordinator mutates everyone's state.
What crosses the boundary?	Versioned event, command, or query contract.	Private entity, table row, or informal DTO.
What if the next owner is down?	Pending state, retry, timeout, and visibility exist.	The request fails halfway and nobody knows what completed.
Can the workflow be resumed?	State is durable and idempotent.	Progress lives only in memory or logs.
How is it traced?	Correlation id follows events, commands, audit, and projections.	Each service has unrelated logs.
What must not be hidden?	Business decisions, refusals, timeouts, and escalations.	A workflow engine hides why the process moved.

HONEST BOUNDARY

This chapter identifies workflow boundaries before building a full distributed workflow engine. Later chapters will return to brokers, outbox, inbox, sagas, retries, dead-letter queues, and observability. The judgement comes first; the infrastructure comes later.

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- Why is a workflow boundary different from a data ownership boundary?
- Why is one long application service dangerous when the process crosses context owners?
- What is the difference between a step owner, a process owner, and an observer?
- Why does `RootCauseVerdictIssued.v1` express a public fact while `RequestComplianceReview` expresses a request?
- Why is waiting a durable workflow state rather than a thread sleep or hidden retry loop?
- Why is audit evidence not the same thing as process state?
- Which parts of the NEXUS-1 alarm-to-compliance flow should remain local, and which parts need handoff semantics?

CHAPTER 9 CLOSING

A workflow boundary makes time, ownership, and failure visible. It prevents a service extraction from hiding cross-context coordination inside one method, one aggregate, or one workflow engine that nobody can audit.

Next chapter

Chapter 10 studies team and deployment boundaries. It asks when architectural ownership is mature enough to justify independent release, operations, and responsibility.

A workflow boundary says who owns the next responsibility. A deployment boundary asks who can operate that responsibility independently.

PART II - FINDING SERVICE BOUNDARIES

10. Team and Deployment Boundaries

A service boundary is not complete when the code compiles. It is complete only when a team can change it, release it, observe it, and repair it without asking the whole system to move.

The previous chapters tested service candidates through context maps, capabilities, data ownership, and workflow handoffs. Chapter 10 adds a harder test: can anyone actually own the service at runtime? Microservices are often discussed as a technical decomposition. In practice, they are also an ownership decomposition.

A bounded context can be technically clean and still not ready for independent deployment. It may have a clear language but no team able to maintain its contracts. It may own its data but lack monitoring, runbooks, and release discipline. It may have perfect package boundaries while every production incident still returns to one overloaded central group.

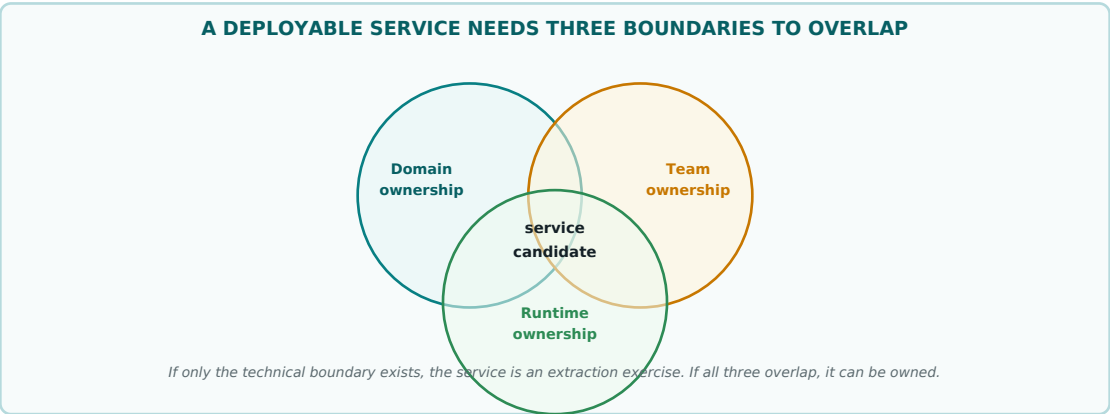


Figure 10.1 - A deployable service boundary is credible only when domain ownership, team ownership, and runtime ownership overlap.

CHAPTER ARGUMENT

Team and deployment boundaries are not decorative. They decide whether a microservice can evolve independently, fail independently, and be supported independently. Without that ownership, distribution increases cost without buying autonomy.

What you already know

You have probably seen teams divided by layers: one group owns the database, another owns APIs, another owns the UI, another owns deployment scripts. This can work for small systems or stable products, but it becomes painful when every feature must cross all teams before it can ship. The delay is not technical only. It is a coordination tax.

Microservices promise to reduce that tax by giving a team a slice of the system it can own end to end. But that promise is not automatic. If service extraction keeps the same layer ownership underneath, the system becomes slower, not faster. The API team still waits for the database team. The UI team still waits for the API team. The operations team still knows every service less deeply than the people who built it.

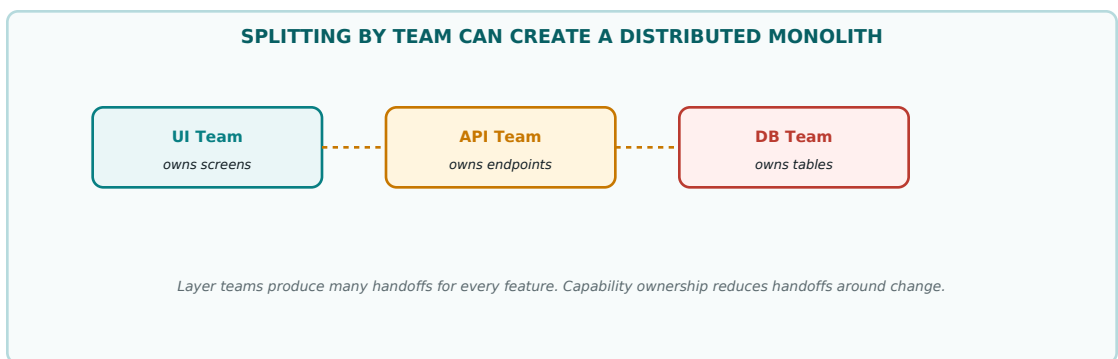


Figure 10.2 - A layer-based team split can survive inside a modular monolith, but it often becomes an expensive distributed monolith after service extraction.

PLAIN EXPLANATION

A team boundary answers: who can change this capability safely? A deployment boundary answers: who can release and operate it independently? A microservice boundary should not be drawn unless those answers are realistic.

Three boundaries, three questions

Chapter 10 uses three questions together. The first is a domain question: who owns the language and rules? The second is a team question: who owns change? The third is a deployment question: who owns release and runtime behavior? A service candidate becomes stronger as the answers converge.

Boundary	Main question	Weak answer	Strong answer
Domain boundary	Who owns the meaning?	Several services use the same word differently without translation.	One context owns the language and publishes a stable contract.
Team boundary	Who owns change?	Every feature needs database, API, UI, and operations handoffs.	One accountable team can change the capability end to end.
Deployment boundary	Who owns release?	A change requires coordinated redeploy of neighbours.	The service can deploy behind compatible contracts.
Runtime boundary	Who owns failure?	Incidents go to a central group that lacks domain context.	The owning team has metrics, alerts, runbooks, and decision authority.

The first boundary can exist without the other two. That is common in a modular monolith. The important mistake is to confuse the existence of the first boundary with readiness for the others. A context is not yet a service if nobody can own its release and production behavior.

NEXUS-1 TRANSLATION

RootCause may be a strong domain boundary. That does not automatically mean it should be independently deployed tomorrow. It becomes a service candidate only when its contracts, data store, operators, tests, and release path can stand on their own.

Team boundary is not an organisation chart

A team boundary should follow change pressure, not manager names. If RootCause, AlarmManagement, and ReinforcementLearning change together every week, splitting them across three isolated teams may increase coordination rather than reduce it. If Compliance changes because of regulatory language and audit expectations while RootCause changes because of causal reasoning, separating ownership may be justified.

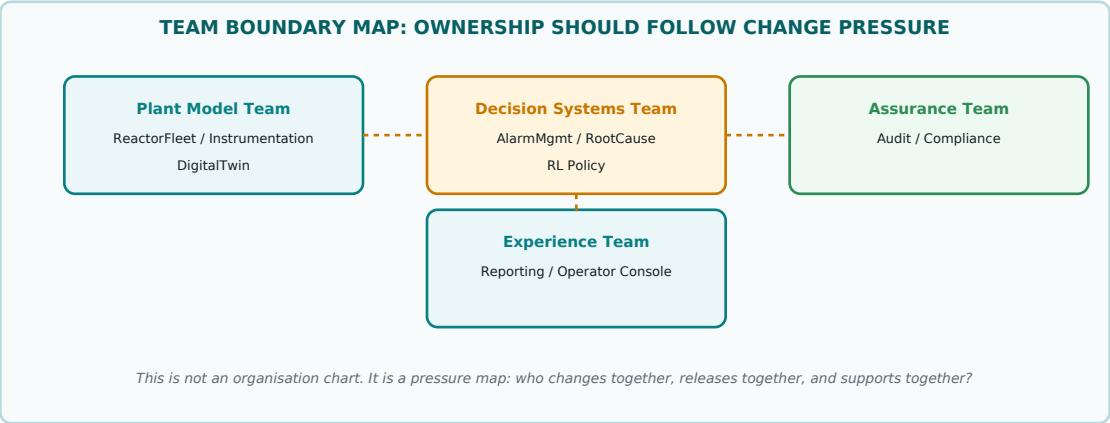


Figure 10.3 - A possible NEXUS-1 ownership map. The purpose is not to freeze an organisation; it is to reveal which capabilities change together and which require different expertise.

Ownership pressure	What to look for	Possible NEXUS-1 signal
Language expertise	Different experts use different words and rules.	Verdict, evidence, finding, recommendation, and audit entry must not collapse into one vocabulary.
Change frequency	One area changes faster or for different reasons.	Reporting screens may change often while RootCause invariants should remain stable.
Compliance pressure	A capability has review, traceability, or policy obligations.	Compliance review and Audit evidence need explicit accountability.
Operational pressure	A capability has distinct failure behavior.	Alarm ingestion failure is not the same incident as report projection lag.

Deployment boundary must be earned

Independent deployment is often named as the main benefit of microservices. But deployment independence is not created by putting code in a new repository. It requires compatible contracts, backward-readable messages, database migration discipline, feature flags, health checks, rollback strategy, and a team that knows which failure modes are acceptable.

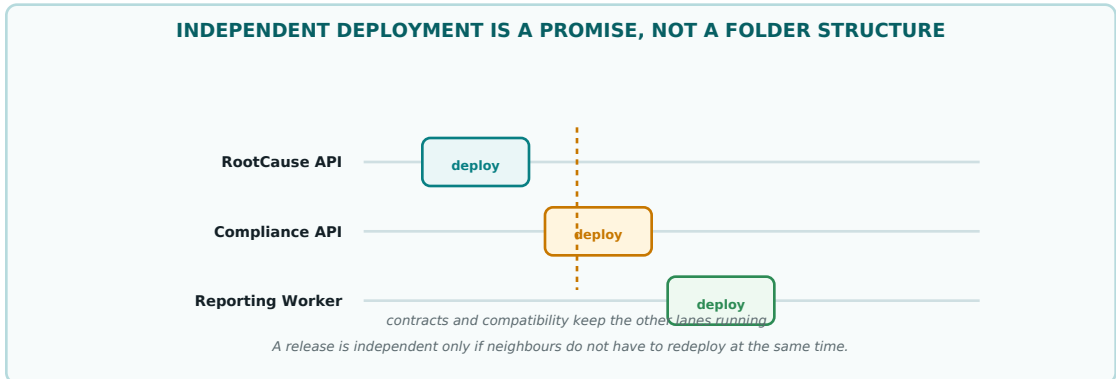


Figure 10.4 - Deployment independence means one lane can release while neighbouring lanes continue to run. If every release requires a synchronized train, the architecture has not bought autonomy.

DEPLOYMENT RULE

A service is independently deployable only if its neighbours do not need to redeploy at the same time for a normal compatible change. If release still requires choreography across the whole platform, the boundary is conceptual but not operational.

```
public sealed record ServiceReleaseReadiness(
    string ServiceName,
    bool OwnsPublicContracts,
    bool SupportsBackwardCompatibility,
    bool HasDatabaseMigrationPlan,
    bool HasHealthChecks,
    bool HasRollbackOrForwardFix,
    bool HasRunbook,
    string OwningTeam);
```

The record is intentionally boring. That is the point. A deployable service boundary depends on dull, explicit responsibilities more than on glamorous infrastructure.

Independent release versus independent chaos

Release independence is valuable only when it is disciplined. A team that can deploy whenever it wants can also break consumers whenever it wants unless compatibility is treated as part of the service contract. Microservices replace compile-time coordination with contract discipline. They do not remove coordination. They move it earlier.

Release habit	Healthy meaning	Dangerous version
Versioned API	Consumers can move at their own pace.	Version numbers exist but payloads change incompatibly.
Versioned events	Old consumers can still read public facts.	The event schema changes because one service refactored internally.
Feature flags	New behavior can be enabled gradually.	Flags hide unfinished design forever.
Database migration	Schema moves forward safely.	Service release assumes manual database changes.
Rollback strategy	Bad releases have a known recovery path.	Rollback breaks because messages or migrations already moved on.
Consumer tests	Contracts are checked before release.	Consumers discover breakage in production.

For NEXUS-1, this means that `RootCauseVerdictIssued.v1` cannot change just because the `RootCause` aggregate was refactored. The public contract belongs to the boundary. Internal code is allowed to evolve behind it. That is the whole value of the boundary.

CONTRACT DISCIPLINE

Independent deployment is not permission to surprise consumers. It is permission to change internal implementation while preserving the public contract consumers rely on.

Applying the test to NEXUS-1 candidates

A NEXUS-1 service map should not begin by turning every sector into a service. It should begin by asking where independent ownership would reduce pressure. Some sectors are obvious future candidates. Others may remain modules, libraries, projections, or shared platform capabilities for a long time.

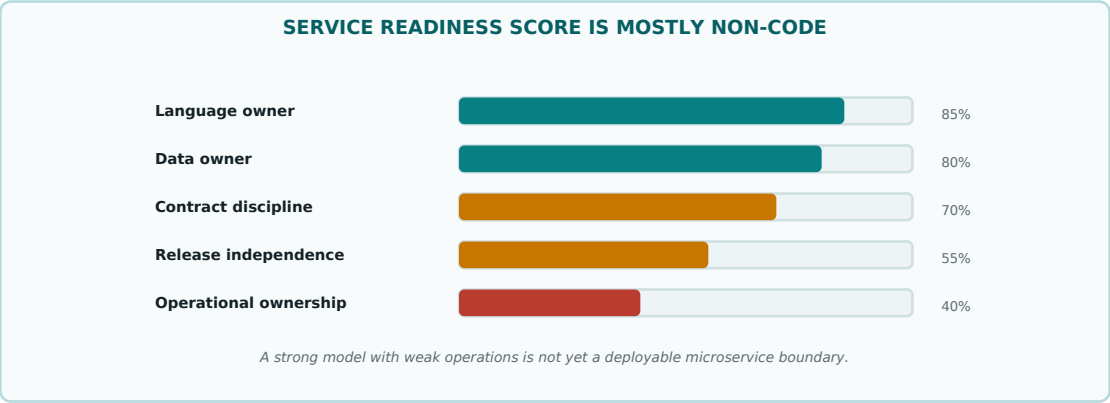


Figure 10.5 - A service-readiness score is mostly about ownership maturity. Code separation is only one piece.

Candidate	Why it may deserve ownership	Why it may wait
RootCause Service	Strong language, strong rules, public verdict contract, clear audit need.	Needs robust event publication, replay, and operator support before extraction.
AlarmManagement Service	Distinct alarm lifecycle and flood detection capability.	High coupling to instrumentation timing may require careful migration.
Compliance Service	Different language, review lifecycle, evidence obligations, and policy pressure.	Must avoid using Audit as workflow state.
Reporting Service	Read-side autonomy and independent scaling.	Must remain projection, not source of truth.
Audit Service	Cross-cutting evidence, append-only integrity, traceability.	Every service depends on it, so failure and backpressure design matter.
DigitalTwin Service	Model snapshots and divergence tracking can evolve independently.	May remain close to instrumentation until signal ownership is stable.

Operational ownership

The strongest reason to delay service extraction is not technical fear. It is operational honesty. A service has a life after deployment. It emits metrics. It fails. It accumulates queues. It fills a dead-letter topic. It runs migrations. It loses contact with dependencies. Someone must know what those situations mean and what to do next.

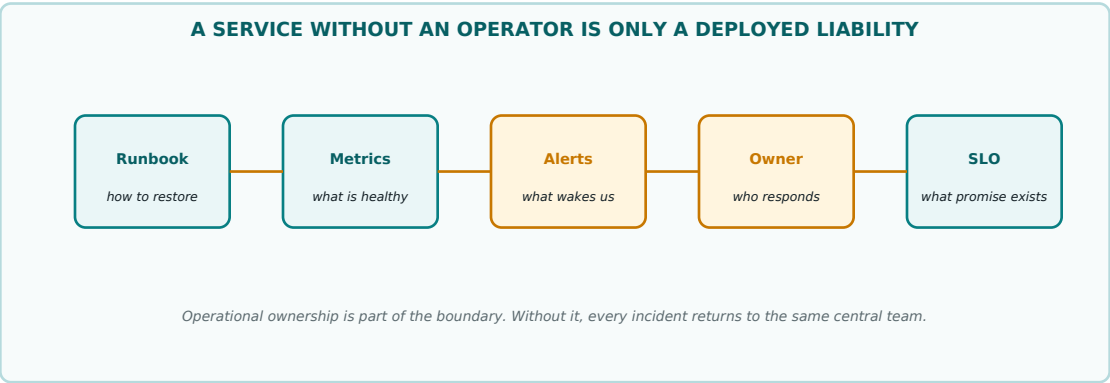


Figure 10.6 - Operational ownership is not a final DevOps detail. It is part of whether the boundary is real.

Operational artifact	Question it answers	NEXUS-1 example
Health check	Can this service accept work?	RootCause can read its store, publish to outbox, and accept case commands.
Metric	Is behavior within expectation?	Verdict publication latency, inbox duplicate count, pending review count.
Alert	When should a human wake up?	Compliance review requests have exceeded timeout threshold.
Runbook	What does the owner do?	Replay outbox, pause consumer, inspect poison message, escalate review.
SLO	What promise is made?	Public verdict event appears within a defined time after local commit.
Incident owner	Who decides during failure?	The owning team, not a generic central operator without domain context.

When team boundaries should not become service boundaries

Sometimes a team owns a capability, but the deployment boundary should still wait. The team may be too small to support another runtime. The contracts may still be changing weekly. The data ownership may still be unclear. The service may need synchronous calls for every useful operation. In those cases, the right answer is not to pretend the architecture is ready. The right answer is to keep the boundary modular and learn.

Warning sign	What it means	Better move now
One or two people own too many future services	Operational load will exceed capacity.	Keep modules inside one deployable and clarify ownership.
Contracts change with every feature	Public language is not stable.	Use internal interfaces and publish only after vocabulary settles.
Every request needs three downstream calls	The boundary will be latency-sensitive and fragile.	Revisit data ownership and local projections.
No runbook or metrics exist	Nobody can operate the split.	Add observability before extraction.
Database migrations require manual coordination	Release independence is false.	Build migration discipline first.
The team split is by technology layer	Feature ownership is still fragmented.	Move toward capability teams before service explosion.

This is not an argument against microservices. It is an argument for sequence. The modular monolith is not a failure state when it is used deliberately. It is a safe training ground where domain, team, and deployment ownership can mature before network boundaries make every weakness more expensive.

HONEST BOUNDARY

A team can own a module before it owns a service. That is often the correct transitional architecture. Ownership should become real before deployment becomes independent.

The team-and-deployment checklist

Run this checklist before treating a NEXUS-1 context as a deployable service. A weak answer does not forbid extraction forever. It tells you which missing ownership discipline must be built first.

Question	Ready answer	Not ready answer
Who owns the language?	One accountable domain owner exists.	The same word changes meaning across teams.
Who owns change?	A team can implement most feature work end to end.	Every change crosses layer teams.
Who owns the contract?	Public API and events are versioned and reviewed.	DTOs mirror internal entities.
Who owns deployment?	The service can release without neighbour redeploy.	Every release is a platform train.
Who owns data migration?	Schema changes are scripted, reviewed, and reversible or forward-fixable.	Manual DB steps are normal.
Who owns production failure?	Runbook, metrics, alerts, and incident owner exist.	Central operations guesses from logs.
Who owns learning?	Post-incident feedback changes the service and its contracts.	Incidents are treated as temporary surprises.

SERVICE EXTRACTION TEST

A service candidate is mature when the team that owns its language can also own its contract, release, data migration, runtime health, and failure response. Without that, the boundary is still mainly architectural, not operational.

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- Why is a team boundary different from a context boundary?
- Why does independent deployment require contract compatibility rather than only separate repositories?
- Why can a context be owned as a module before it is owned as a service?
- Why does operational ownership belong in the service-readiness decision?
- Which NEXUS-1 sectors look like strong future service candidates, and which should probably wait?
- Why can splitting by UI, API, and database teams create a distributed monolith?
- What evidence would prove that RootCause is ready for independent deployment?

CHAPTER 10 CLOSING

Team and deployment boundaries turn architecture into responsibility. A service is not a drawing of a box around code. It is a promise that someone can change, release, observe, and repair that box without moving the whole system.

Next chapter

Chapter 11 closes Part II with the Service Boundary Checklist for NEXUS-1. It gathers the context, capability, data, workflow, team, and deployment tests into one practical extraction decision.

A microservice should not begin when the code can be separated. It should begin when responsibility can be separated.

PART II - FINDING SERVICE BOUNDARIES

11. Service Boundary Checklist for NEXUS-1

A checklist does not make the architectural decision for you. It prevents the decision from being made by fashion, pressure, or the misleading comfort of a clean diagram.

Part II has asked the same question from several angles. Does the context have its own language? Does it describe a business capability? Does it own data? Does it participate in a flow in a way that can fail independently? Can a team own it? Can it be deployed, observed, and repaired without moving the whole platform?

Chapter 11 gathers those tests into one service-boundary checklist for NEXUS-1. The purpose is not to force every strong context into a microservice. The purpose is to make the extraction decision explicit enough that a future team can disagree with it honestly.

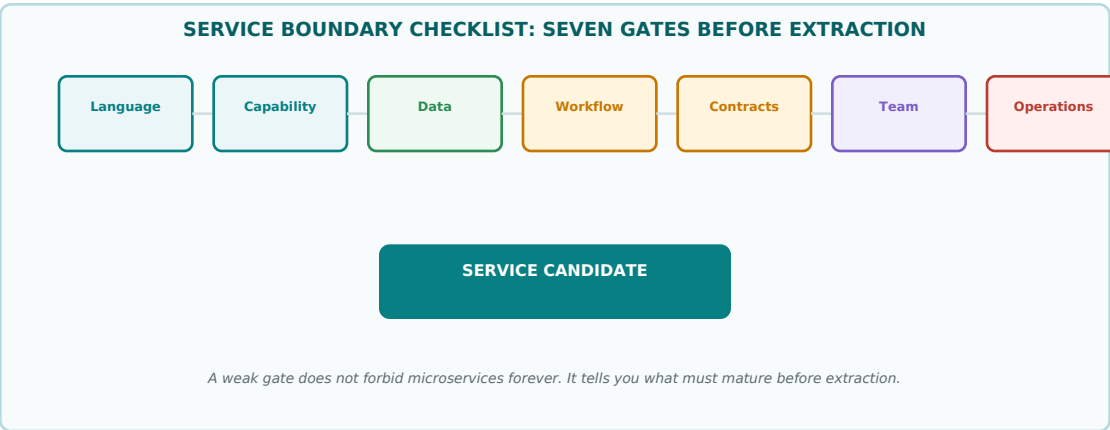


Figure 11.1 - The service boundary checklist turns the Part II discussion into seven readiness gates.

CHAPTER ARGUMENT

A microservice candidate is mature when language, capability, data, workflow, contract, team, and operational ownership point in the same direction. If only one or two of those gates are strong, the right answer may still be a modular boundary, not a runtime boundary.

Why a checklist is necessary

Without a checklist, service extraction tends to follow visible surfaces: tables, controllers, screens, or team preferences. Those surfaces are useful clues, but they are not enough. A table may hold data for several capabilities. A controller may expose a use case without owning the language. A screen may combine projections from many contexts. A team may prefer a service because it gives autonomy, while the system still lacks stable contracts.

The checklist slows the decision down. It asks whether the boundary is already true in the model before it becomes true in deployment. That is the central discipline of this book: do not distribute a lie and call it architecture.

Naive trigger	Why it is not enough	Checklist correction
There is a folder named RootCause	Folders show organisation, not runtime readiness.	Test language, contracts, data ownership, workflow, and operation.
There is a SQL schema named Audit	Schema ownership is a clue, not a deployment promise.	Ask whether the audit service can own append-only evidence independently.
The UI has a Reporting page	Screens often compose data from several owners.	Separate source-of-truth from projection responsibility.
A team wants autonomy	Autonomy without contracts becomes local speed and global pain.	Check whether neighbours can keep running after a release.
The service looks small	Small does not mean stable or valuable.	Ask whether independent deployment buys real change or failure isolation.

PLAIN EXPLANATION

The checklist is a decision record. It tells the team why a boundary was extracted, prepared, or kept inside the modular core. Six months later, it should still be possible to read the decision and understand the trade-off.

The seven readiness gates

The checklist uses seven gates. They are deliberately architectural rather than tool-specific. A team can apply them before choosing RabbitMQ, Kafka, Docker, Kubernetes, SQL Server, or any other infrastructure product. The question is first about responsibility. The tools arrive later.

Gate	Ready answer	Not-ready answer
Language	The context owns words that would be dangerous elsewhere.	The same terms are still negotiated informally across contexts.
Capability	The boundary represents a meaningful business or operational capability.	The boundary is an entity, CRUD table, or technical layer.
Data	The service can own local truth and expose references or events.	Other contexts still need to read or write its private tables.
Workflow	Cross-boundary flow uses named commands/events and visible failure states.	The flow depends on hidden calls, shared transactions, or implicit ordering.
Contract	Public APIs/events are small, versioned, documented, and tested.	Public contracts mirror internal aggregates or EF entities.
Team	A team can change the capability end to end.	Feature work still crosses layer teams for every change.
Operations	Health, metrics, alerts, runbooks, and incident ownership exist.	The service would be deployed before anyone can operate it.

A strong candidate does not need a perfect score. Real architecture rarely gives perfect scores. But a weak score in data ownership, contract stability, or operations should stop immediate extraction. Those weaknesses become more expensive after the network boundary is drawn.

NEXUS-1 candidate bands

The project description already warns that not all NEXUS-1 contexts should become services. Chapter 11 turns that warning into a current candidate map. The map is intentionally provisional. It shows what looks strong now, what needs preparation, and what should remain together or be protected.

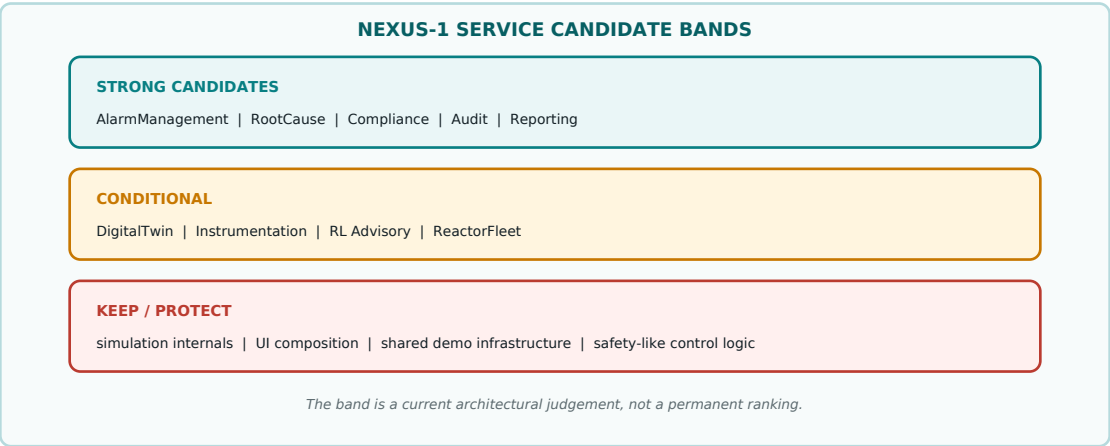
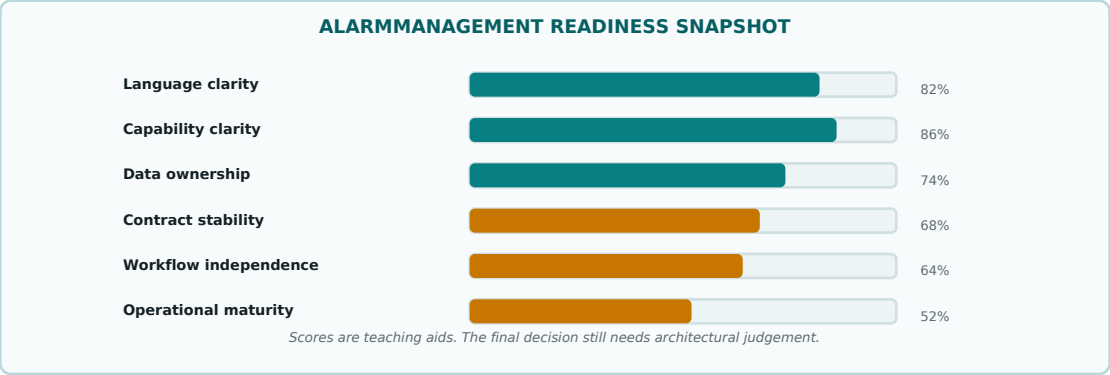


Figure 11.2 - NEXUS-1 service candidates arranged by present readiness, not by importance.

Band	Meaning	Decision posture
Strong candidates	The boundary has clear language, ownership, and a useful reason to evolve independently.	Prepare service design in later parts; still prove data and contracts.
Conditional candidates	The capability is real, but coupling, data ownership, or operational risk remains high.	Keep modular now; build missing discipline before extraction.
Keep/protect	Distribution would weaken integrity or create false authority.	Do not extract unless the purpose and safety boundary change dramatically.

AlarmManagement: a strong but sensitive candidate

AlarmManagement is one of the strongest candidates because alarms have a clear lifecycle, a clear vocabulary, and a clear reason to evolve independently from root-cause reasoning. It can own alarm definitions, alarm events, acknowledgements, flood detection, and the publication of AlarmFloodDetected.v1. It should not own the final causal verdict.



The main risk is not that AlarmManagement lacks identity. It has identity. The risk is that it becomes too chatty with Instrumentation and RootCause if the event boundary is not designed carefully. If every alarm evaluation needs a synchronous call to read signal state, the service will amplify latency. If RootCause needs to query AlarmManagement internals instead of consuming a published fact, the boundary will leak.

Decision point	Recommended posture
Service candidate	Strong, but extraction should wait until event and data contracts are stable.
Public facts	AlarmRaised.v1, AlarmFloodDetected.v1, AlarmAcknowledged.v1 where needed.
Private model	AlarmDefinition, alarm rules, internal flood grouping details.
Main preparation	Local signal snapshot strategy and outbox discipline.

RootCause: the clearest decision boundary

RootCause is the clearest service candidate in Part II because it owns a decision. It owns hypotheses, evidence, verdicts, corrections, and the language that separates symptoms from causes. It has a canonical internal event, `VerdictIssued`, and a public integration event, `RootCauseVerdictIssued.v1`. That distinction is exactly the kind of contract discipline a future service needs.

```
public sealed record ServiceBoundaryAssessment(  
    string ContextName,  
    int Language,  
    int Capability,  
    int DataOwnership,  
    int Workflow,  
    int Contract,  
    int Team,  
    int Operations)  
{  
    public int Total => Language + Capability + DataOwnership +  
        Workflow + Contract + Team + Operations;  
  
    public bool CanBeExtractedNow =>  
        DataOwnership >= 4 && Contract >= 4 && Operations >= 3 && Total >= 28;  
}  
  
var rootCause = new ServiceBoundaryAssessment(  
    ContextName: "RootCause",  
    Language: 5,  
    Capability: 5,  
    DataOwnership: 4,  
    Workflow: 4,  
    Contract: 4,  
    Team: 3,  
    Operations: 3);
```

Listing 11.1 - A deliberately small C# assessment type. The numbers are not magic; the rule is that weak data, contract, or operations should block extraction even when the model looks elegant.

ROOTCAUSE DECISION

RootCause can become a service candidate earlier than many other contexts because it owns a decision and publishes a stable fact. The extraction still depends on outbox, idempotency, traceability, and operator-visible consistency.

Audit, Compliance, and Reporting

Audit, Compliance, and Reporting are all tempting service candidates, but they are strong for different reasons. Audit is strong because it should be append-only evidence and should not drive workflow. Compliance is strong because it owns review obligations and findings. Reporting is strong because it owns projections, but dangerous if anyone mistakes those projections for source-of-truth.

Context	Why it is a candidate	Main danger	Current decision
Audit	Append-only evidence stream; cross-cutting consumers; strong need for independence and integrity.	Audit entries accidentally become workflow commands.	Strong candidate, but must stay evidence-only.
Compliance	Owens review, findings, obligations, closure, and regulatory-facing language.	RootCause or Reporting begins deciding compliance state.	Strong candidate after event contracts and process state mature.
Reporting	Owens read models, dashboards, exports, and operator-facing status composition.	Projection data is treated as domain truth.	Strong read-side candidate with freshness metadata.

The checklist helps avoid a common mistake: grouping these three together because they all feel administrative. They do not own the same truth. Audit proves what happened. Compliance decides whether a review or finding exists. Reporting displays a shaped view of facts owned elsewhere. A single service that owns all three would be convenient but conceptually dishonest.

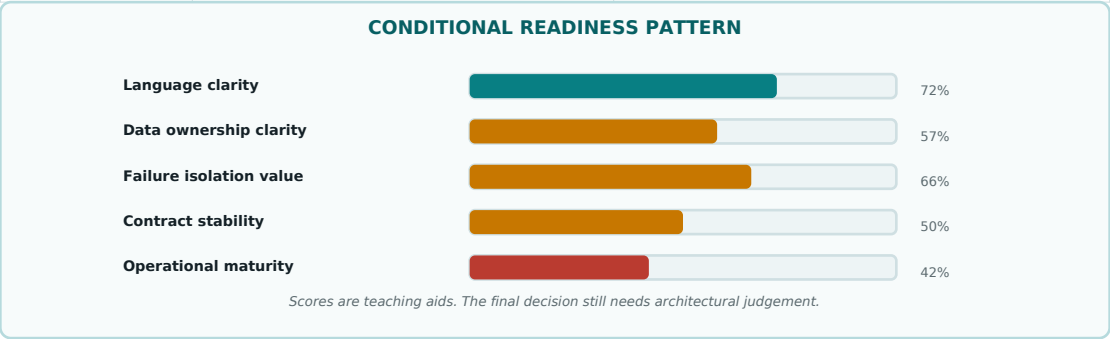
TERMINOLOGY DISCIPLINE

AuditEntry is evidence, not workflow. A ComplianceFinding is a domain state owned by Compliance. A Reporting projection is a view, not the source of truth. These three sentences prevent an enormous amount of distributed confusion.

Conditional candidates

The conditional candidates are not weaker because they are less important. In several cases they are more fundamental. They are conditional because extraction could damage integrity if done too early. DigitalTwin, Instrumentation, ReactorFleet, and ReinforcementLearning all touch foundational truth, high-volume data, or advisory boundaries that need careful protection.

Context	Why it may become a service	Why it should wait
DigitalTwin	Twin models, bindings, snapshots, divergence detection, and model state can evolve independently.	It depends heavily on ReactorFleet and Instrumentation truth; stale data can create false confidence.
Instrumentation	Signal catalogue, readings, quality, and historian integration have clear ownership.	High-volume telemetry, latency, and external historian vocabulary complicate the boundary.
ReactorFleet	Physical unit structure and equipment identity are upstream for many contexts.	It may be stable reference truth better served as a protected core until contracts mature.
RL Advisory	Policy and recommendation logic can evolve independently from plant model and UI.	Recommendations are advisory, not commands; extraction must not imply control authority.



The correct posture is preparation, not rejection. These contexts should continue to harden their contracts, projections, and ownership rules while remaining inside a modular core or carefully protected runtime boundary.

What should remain together

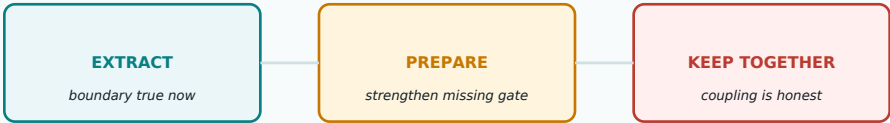
A mature microservice book must name what it will not split. In NEXUS-1, the most important do-not-split area is anything that looks like safety-like control logic. The demonstrator has advisory recommendations and educational simulation behavior. It does not control a plant. Service extraction must never blur that boundary.

Area	Why it should remain protected	Safe treatment
Highly coupled simulation internals	Splitting numerical or tightly coupled model internals can create false runtime independence.	Keep inside a coherent simulation module; expose snapshots or advisory outputs.
UI composition logic	The operator console composes status from many sources and should not become source-of-truth.	Use BFF/read models later, but keep truth in owning services.
Shared demonstration infrastructure	Common demo helpers do not justify independent services.	Keep as shared libraries or platform modules.
Safety-like control logic	The series standing boundary forbids operational plant control claims.	Keep outside service extraction claims; recommendations remain advisory.

HONEST BOUNDARY

NEXUS-1 can demonstrate microservice architecture without pretending to build certified operational control software. The checklist therefore includes a negative decision: some boundaries are not extracted because not extracting them is the more honest architecture.

THREE POSSIBLE DECISIONS AFTER THE CHECKLIST



Do not turn every weak score into a rejection. Some weak scores are preparation work.

The checklist creates a decision record: why we split, why we waited, or why we kept a boundary inside the modular core.

Figure 11.3 - The checklist produces three valid decisions: extract, prepare, or keep together.

From scores to a decision record

The score is not the architecture. It is a conversation starter that forces the team to state evidence. A service with high language and capability scores but low operations should not be extracted immediately. A service with modest capability value but extremely strong operational isolation may still be worth extracting. The final decision must explain the trade-off.

Decision	Use when	Decision record should say
Extract now	Most gates are strong and independent deployment has a real benefit.	What contract will be public, what data is local truth, how failure is handled, and who operates it.
Prepare for extraction	The boundary is promising but one or two gates are weak.	Which gates must improve: contract tests, local store, outbox, projections, runbooks, team ownership.
Keep modular	The cost of distribution is higher than the autonomy gained.	Which coupling is intentional and what evidence would reopen the decision later.

```
public enum BoundaryDecision
{
    ExtractNow,
    PrepareForExtraction,
    KeepModular
}

public sealed record BoundaryDecisionRecord(
    string ContextName,
    BoundaryDecision Decision,
    string PrimaryReason,
    string[] RequiredPreparations,
    DateOnly ReviewAfter);
```

Listing 11.2 - A decision record is more valuable than a score. It explains what the team believed, what it deferred, and when the judgement should be reviewed.

The Part II consistency audit

Part II began with the context map and ends with an extraction checklist. The chapters between them deliberately did not jump straight to APIs, brokers, or containers. They asked whether the boundary itself was honest. That is the sequence this book wants the reader to keep.

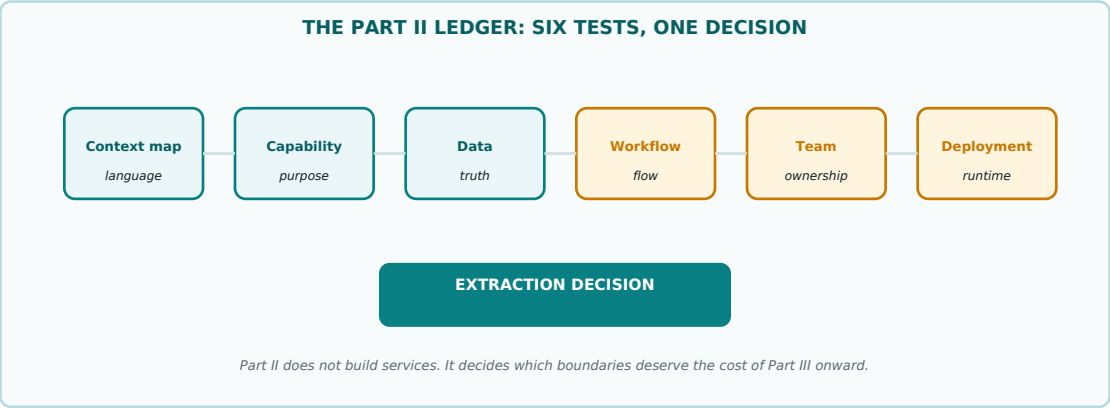


Figure 11.4 - Part II turns six kinds of boundary evidence into one extraction decision.

Part II test	Question it answered	What it prevents
Context map	Where does language and ownership change?	Services built from ambiguous words.
Capability boundary	What useful responsibility is being owned?	Service-per-entity decomposition.
Data boundary	Who owns the source-of-truth?	Shared database microservices.
Workflow boundary	What crosses the boundary, and how can it fail?	Hidden distributed transactions and workflow lies.
Team boundary	Who changes and supports the service?	Layer-team distributed monoliths.
Deployment boundary	Can it release and fail independently?	Separate processes that still move as one system.

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- Why is a service-boundary checklist better than a simple list of contexts?
- Which seven gates should be tested before extracting a NEXUS-1 microservice?
- Why are AlarmManagement, RootCause, Audit, Compliance, and Reporting stronger candidates than some foundational contexts?
- Why are DigitalTwin, Instrumentation, ReactorFleet, and RL Advisory conditional rather than rejected?
- Why must Reporting remain a projection owner rather than a source-of-truth owner?
- Why should safety-like control logic remain outside real operational claims?
- What is the difference between Extract now, Prepare for extraction, and Keep modular?

PART II CLOSING

Part II has not built a single microservice. That is the point. It has built the judgement needed before services are built: which boundaries are real, which are promising, and which should stay inside the modular core until the system is ready.

Next chapter

Chapter 12 opens Part III: Service Ownership and Data Independence. The next question is no longer whether a boundary deserves service treatment. It is what data ownership actually means once the boundary becomes a runtime boundary.

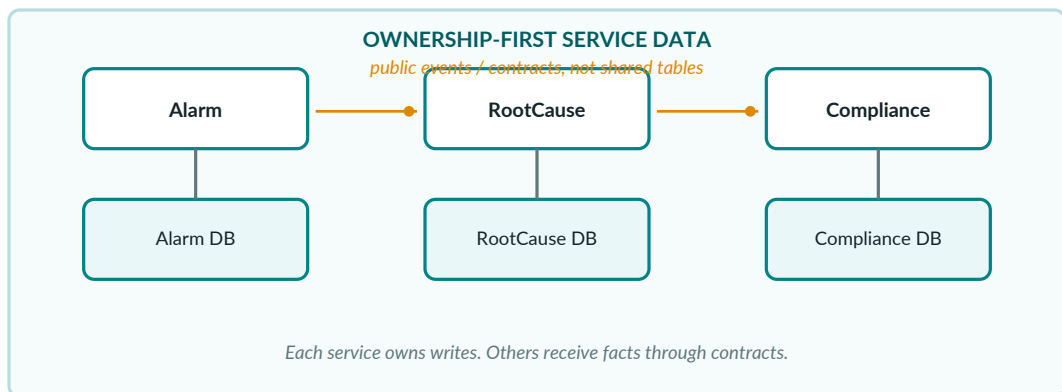
A service boundary begins as a responsibility boundary. Only then does it deserve a database, an API, a broker topic, and a deployment unit.

PART III

Service Ownership and Data Independence

Once a boundary becomes a runtime boundary, ownership stops being a diagram and becomes a rule the system must enforce.

Part II asked whether a context deserves the cost of service extraction. Part III assumes that at least one boundary is serious enough to examine further. The next question is data: who owns truth, who may write it, who may copy it, and who is only allowed to observe it.

**CHAPTERS 12-17 - DATABASE OWNERSHIP - LOCAL TRUTH - REMOTE TRUTH - REFERENCES - SNAPSHOTS****WHAT PART III GIVES YOU**

By the end of Part III, the reader should be able to explain why database-per-service is not a fashion rule, why shared writes destroy autonomy, how cross-service references should be represented, and why a projection or snapshot must never be confused with source-of-truth.

PART III - SERVICE OWNERSHIP AND DATA INDEPENDENCE

12. Database-per-Service Is a Rule About Ownership

The rule is often misunderstood. It does not begin with server topology. It begins with a simpler sentence: no service may secretly write another service's truth.

A developer hears database-per-service and often imagines one SQL Server instance for every box on a diagram. That can happen, but it is not the heart of the pattern. The heart is ownership. A service owns its data model, its invariants, its migrations, its transaction boundary, and the decision about how other services may learn facts from it.

In NEXUS-1 this matters immediately. If RootCause becomes a service, the RootCauseCase cannot be half-owned by AlarmManagement, half-owned by Compliance, and queried directly by Reporting whenever a dashboard is convenient. The moment that happens, the service is a process boundary wrapped around somebody else's database.

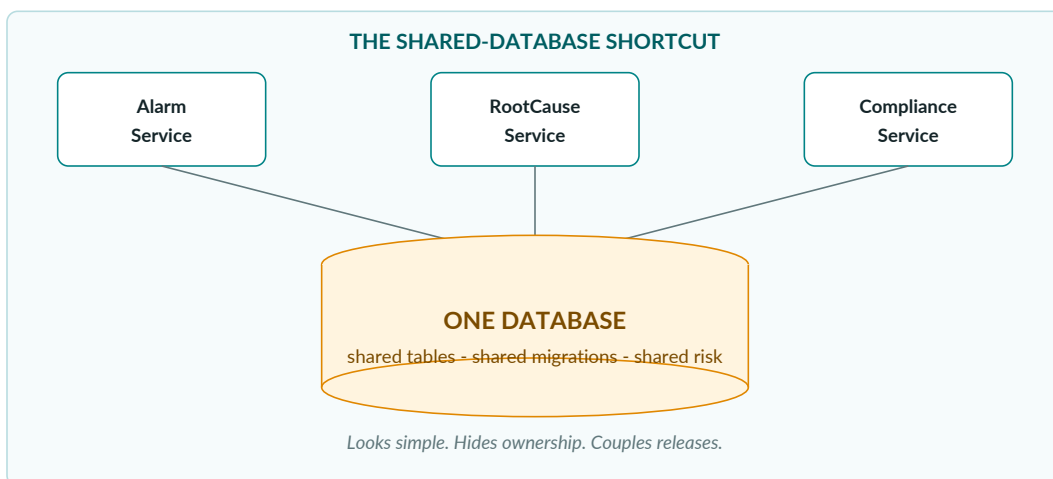
OLD MONOLITH COMFORT	SERVICE ANSWER	NEW COST
Join any table	Ask the owner or consume an event	Queries become designed contracts
One transaction	Local transaction plus message flow	Consistency becomes explicit
Foreign key everywhere	Stable identifier plus validation/projection	Integrity is enforced differently
One migration train	Owner-controlled migrations	Versioning and compatibility matter
One reporting query	Read model built from published facts	Freshness must be visible

CHAPTER ARGUMENT

Database-per-service is not primarily about where bytes live. It is about who is allowed to change truth.

The naive shortcut

The easiest way to pretend a modular monolith has become microservices is to leave the database alone and place several APIs in front of it. The deployment diagram changes. The ownership model does not.



This shortcut feels safe because the team keeps all the conveniences it already had: joins, foreign keys, one reporting surface, and one migration path. But those conveniences are exactly why the boundary is not real. If AlarmManagement and RootCause both write tables in the same database, then neither service can evolve independently. If Reporting queries their tables directly, then Reporting becomes coupled to their private storage decisions.

COMMON MISTAKE

Do not call a group of APIs microservices while they still share one writable database model. That is usually a distributed monolith with extra network calls.

What the rule actually says

The useful version of the rule is more precise than the slogan. A service should own the data it is responsible for protecting. Other services should not reach around its API, message contracts, or projections in order to change that data.

OWNERSHIP RULE	MEANING IN PRACTICE
Owns writes	Only the service changes its source-of-truth tables.
Owns invariants	Rules are enforced inside the owner, not in another service's orchestration code.
Owns migrations	Schema changes are released by the team that owns the service.
Owns transaction boundary	A commit protects only local truth; downstream consequences use messages.
Publishes facts	Other services learn stable facts through events, APIs, or read models.

Physical database vs ownership boundary

FORM	CAN BE VALID EARLY?	RISK
Separate schemas in one database	Yes, as a transition discipline if writes are enforced by code and permissions.	Teams may still rely on cross-schema joins and shared release timing.
Separate databases on one server	Yes, stronger than schemas.	Operational failure may still be shared.
Separate database products or clusters	Yes, when autonomy and operations justify it.	Highest operational cost.
One shared writable database	No, not for real microservices.	Ownership is only decorative.

HONEST BOUNDARY

The series can demonstrate microservice architecture in stages. Early drafts may use schemas or separate databases on one server, but the ownership rule must already be visible: private writes stay private.

The NEXUS-1 ownership map begins here

Part II identified candidates. Part III starts assigning truth. The table below is intentionally conservative: it names local truth, public access, and the thing each service must not allow.

SERVICE	LOCAL SOURCE-OF-TRUTH	PUBLIC ACCESS	NOT ALLOWED
AlarmManagement	AlarmDefinition, AlarmEvent, AlarmFlood, acknowledgement state	AlarmFloodDetected.v1, alarm query API, operator read model	RootCause writes alarm rows directly
RootCause	RootCauseCase, Evidence, Hypothesis, Verdict	RootCauseVerdictIssued.v1, case query API	Compliance changes verdict state
Audit	AuditEntry, AuditBatch, hash-chain or lineage records	append API/event subscriber, audit query API	AuditEntry drives workflow decisions
Compliance	ComplianceReview, Finding, Obligation, closure state	RequestComplianceReview command/API, review events	Reporting decides compliance state
Reporting	Projection rows, dashboard snapshots, exports	read-only UI/data API	Projection becomes source-of-truth

TERMINOLOGY DISCIPLINE

A service owns facts because it owns the language and rule that make those facts meaningful. RootCause owns a verdict. Compliance owns a review. Audit owns evidence. Reporting owns a view.

The moment a join becomes a contract

Inside a modular monolith, joining tables is often the simplest honest expression of the model. Across services, the same join becomes a hidden contract. Reporting may believe it is only reading, but it has become coupled to private table names, column names, indexes, meanings, and migration timing.

```
-- Bad across service boundaries: Reporting knows private storage.
SELECT r.CaseId, r.FinalVerdict, c.ReviewStatus
FROM RootCause.RootCauseCase r
JOIN Compliance.ComplianceReview c ON c.CaseId = r.CaseId;
```

The service version asks a different question: which public fact should Reporting be allowed to know? In NEXUS-1 the canonical fact is not `RootCauseCaseClosed` and not `RootCauseVerdictPublished`. The public integration event remains `RootCauseVerdictIssued.v1`.

```
public sealed record RootCauseVerdictIssuedV1(
    Guid CaseId,
    string UnitCode,
    string VerdictCode,
    decimal Confidence,
    DateTime IssuedAtUtc,
    string EvidenceDigest);
```

DESIGN RULE

A query can be convenient and still be architectural leakage. A public contract is slower to design, but safer to evolve.

A small C# implementation shape

The implementation shape should make the rule hard to violate. The RootCause service may depend on its own persistence boundary. It should not inject AlarmDbContext, ComplianceDbContext, or ReportingDbContext simply because a join would be useful.

```
// Inside the RootCause service boundary.
public sealed class RootCauseDbContext : DbContext
{
    public DbSet<RootCauseCase> Cases => Set<RootCauseCase>();
    public DbSet<Evidence> Evidence => Set<Evidence>();
    public DbSet<OutboxMessage> Outbox => Set<OutboxMessage>();
}

public sealed class IssueVerdictHandler
{
    private readonly IRootCauseCaseRepository _cases;
    private readonly IOutboxWriter _outbox;
    private readonly IUnitOfWork _uow;

    public async Task Handle(IssueVerdict command)
    {
        var rootCase = await _cases.Get(command.CaseId);
        rootCase.IssueVerdict(command.Verdict, command.Confidence);

        _outbox.Add(RootCauseVerdictIssuedV1.From(rootCase));
        await _uow.CommitAsync(); // one local transaction
    }
}
```

The handler records the local decision and the outbox message in one local transaction. It does not open a compliance review directly. It does not write an audit row by reaching into Audit storage. Those consequences are handled by contracts and consumers.

IMPLEMENTATION BOUNDARY

The code does not prove the architecture by itself. It only helps if project references, database permissions, contract tests, and operational ownership enforce the same boundary.

What other services may store

Database ownership does not mean services know nothing about one another. A useful distributed system often stores references, snapshots, and projections. The important discipline is to label those copies honestly.

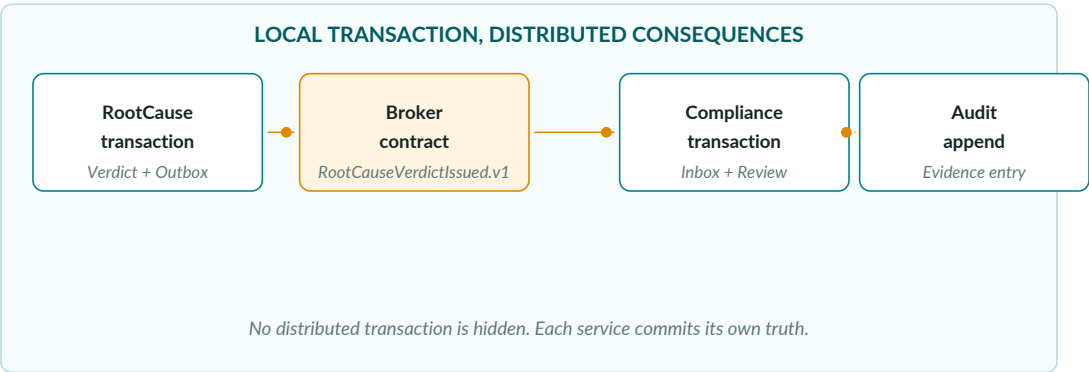
LOCAL COPY TYPE	EXAMPLE	SAFE WHEN	DANGER
Identifier reference	RootCause stores AlarmFloodId from AlarmManagement	The ID is stable and the owner can validate it.	The consumer assumes it can update the alarm.
Snapshot	Compliance stores verdict text and confidence as received	The snapshot has source version and received time.	Users think it always reflects current root-cause state.
Projection	Reporting stores a dashboard row for case status	Freshness and rebuild rules are visible.	Projection is treated as domain truth.
Reference cache	Service caches UnitCode or display name	Cache invalidation is acceptable and bounded.	Local cache becomes a second owner.
Full aggregate copy	Copy of RootCauseCase in Compliance	Almost never needed.	Two services now own one model.

SNAPSHOT RULE

A copied fact should carry its origin: source service, contract name, version, and time received. Without that metadata, a snapshot masquerades as truth.

The transaction boundary changes the flow

The alarm-to-compliance flow was easy to draw as one chain. Once services own their data independently, the chain is no longer one transaction. That is not a defect. It is the reality the architecture must show instead of hiding.



RootCause commits the verdict and the outbox record together. A publisher later emits RootCauseVerdictIssued.v1. Compliance consumes the event idempotently and opens or updates a review. Audit records evidence as evidence, not as the driver of the process. Reporting updates a projection after the facts arrive.

STEP	LOCAL OWNER	CONSISTENCY
Issue verdict	RootCause	Strong inside RootCause database
Publish event	RootCause outbox/publisher	At-least-once delivery must be tolerated
Open review	Compliance	Strong inside Compliance database
Write audit evidence	Audit	Append-only local truth
Update dashboard	Reporting	Eventually consistent projection

What becomes harder

A serious architecture book should say what the rule costs. Database-per-service removes one kind of coupling and introduces several forms of operational work. The point is not that the new work is pleasant. The point is that it is visible and owned.

WHAT GETS HARDER	WHY IT GETS HARDER	DISCIPLINE REQUIRED
Cross-service queries	There is no legal direct join across private tables.	Read models, APIs, and projection freshness.
Transactions	One service cannot roll back another service.	Outbox, inbox, idempotency, compensation.
Debugging	A single user action spans multiple processes.	Correlation IDs and distributed tracing.
Testing	Local tests no longer prove public compatibility.	Contract tests and consumer tests.
Migrations	Private schemas evolve independently.	Versioned contracts and backward compatibility.
Operations	Every store needs backups, health checks, and ownership.	Runbooks, monitoring, and team responsibility.

HONEST BOUNDARY

Database-per-service is not a performance pattern. It is a responsibility pattern. Use it only when responsibility is worth the operational cost.

For NEXUS-1, this means the first extracted services should be those whose ownership and contracts are already mature enough to pay the price: RootCause, Audit, Compliance, and selected read-side Reporting capabilities before the more foundational plant and instrumentation truth.

Applying the data rule to NEXUS-1 candidates

The service-boundary checklist of Chapter 11 now receives a data-specific interpretation. A candidate is not ready because it has a table group. It is ready when its data can be private without making the rest of the system lie.

CANDIDATE	DATA READINESS	DECISION FOR NOW
RootCause	Owens cases, evidence, hypotheses, verdicts. Public event can carry the verdict fact.	Strong candidate for separate persistence.
Audit	Append-only evidence is naturally owned and should not be mutated by others.	Strong candidate, evidence-only.
Compliance	Owens review state and findings; reacts to public facts from RootCause.	Strong after contract and inbox discipline.
Reporting	Owens projections, not domain truth.	Strong read-side candidate with freshness labels.
AlarmManagement	Owens alarm events and floods; upstream for RootCause.	Strong but must publish stable flood contracts.
DigitalTwin	Depends on ReactorFleet and Instrumentation truth.	Prepare carefully; snapshot rules are essential.
Instrumentation	High-volume signal truth and external historian vocabulary.	Conditional; operational complexity is high.

SENTENCE TO REMEMBER

A service may keep a local projection of another service's facts. It may not pretend that projection is the owner. Reporting can show a verdict; RootCause owns the verdict.

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- Why is database-per-service mainly an ownership rule rather than a server-count rule?
- Why does a shared writable database usually produce a distributed monolith?
- What is the difference between local source-of-truth, snapshot, cache, and projection?
- Why should Reporting never become the owner of RootCause or Compliance truth?
- How does RootCause publish a verdict without writing directly into Compliance storage?
- Which NEXUS-1 candidates are strongest from a data-ownership perspective?
- What new operational costs appear when every service owns its own data?

NEXT CHAPTER

Chapter 13 continues Part III by looking at the failure mode this chapter has been avoiding: shared databases that look practical at first and then quietly destroy service autonomy.

A service database is not a storage preference. It is the physical consequence of saying: this service owns this truth.

PART III - SERVICE OWNERSHIP AND DATA INDEPENDENCE

13. Why Shared Databases Break Microservices

A shared database looks like the practical compromise. It keeps the joins, keeps the old reports, and lets services arrive one by one. It also keeps the old coupling alive.

Chapter 12 framed database-per-service as an ownership rule. This chapter explains the opposite failure mode: extracting Web APIs while leaving the tables as a common write surface. The system gains network calls, deployment complexity, and operational cost, but keeps the same hidden dependencies that the service split was supposed to remove.

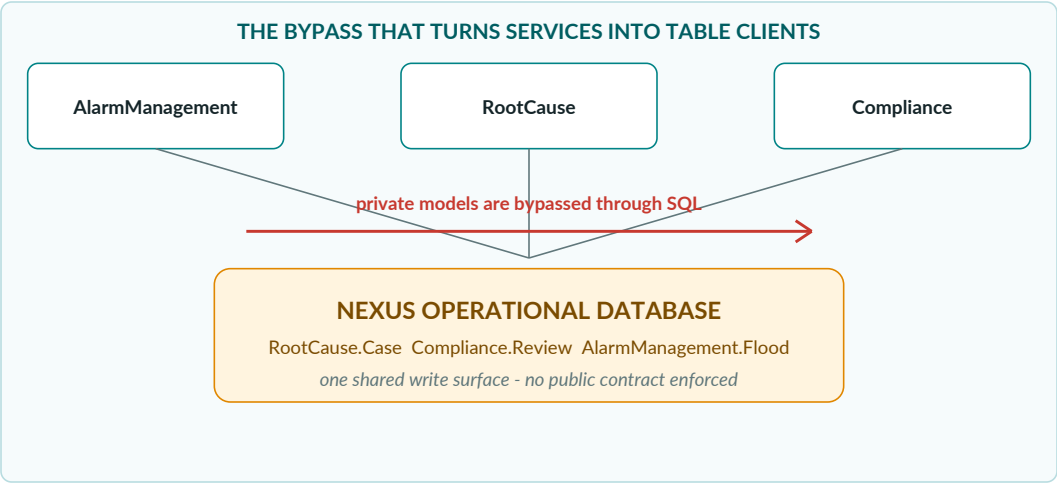
For NEXUS-1, the temptation is obvious. The existing schema atlas already gives us named sectors: AlarmManagement, RootCause, Compliance, Reporting, Audit, DigitalTwin, Instrumentation. Why not deploy services around those sectors and let them all continue to read and write the same SQL Server database?

CENTRAL WARNING

Because a service boundary that can be bypassed with SELECT, INSERT, UPDATE, or DELETE is not yet a service boundary. It is only a network boundary placed in front of a shared model.

The false comfort of one database

A single database feels safe because it preserves familiar habits. Developers can still join across contexts. Reporting still works. Old stored procedures still run. Foreign keys still appear to protect integrity. During the first extraction sprint, the architecture seems to have gained services without losing convenience.



The problem is not that one physical SQL Server instance exists. The problem is that several services still treat the same tables as their private property. Once two services write the same table, no diagram can honestly say which service owns the rule.

WHAT LOOKS CONVENIENT	WHAT IT HIDES
Direct joins	A consumer that depends on private table shape.
Shared foreign keys	A cross-service invariant the owner cannot change alone.
Shared migrations	A release train disguised as autonomy.
Shared reporting views	A read model that may become the unofficial truth.

The private model is bypassed

A service protects its model by forcing callers through its public contracts. A shared database offers a back door around those contracts. The caller no longer asks RootCause to issue or read a verdict; it reads the RootCause tables and assembles its own interpretation.

```
-- Looks harmless inside Reporting.  
-- It is not harmless if RootCause is a service.  
SELECT  c.RootCauseCaseId,  
        c.FinalVerdict,  
        c.Confidence,  
        e.EvidenceCount,  
        r.ReviewStatus  
FROM RootCause.RootCauseCase c  
LEFT JOIN RootCause.CaseEvidenceSummary e  
    ON e.RootCauseCaseId = c.RootCauseCaseId  
LEFT JOIN Compliance.ComplianceReview r  
    ON r.RootCauseCaseId = c.RootCauseCaseId  
WHERE c.UnitId = @unitId  
    AND c.ClosedAtUtc >= @fromUtc;
```

Inside a modular monolith, that query may be acceptable because the schema is part of one deployable model. After extraction, the same query becomes a contract bypass. Reporting now knows table names, join paths, nullability assumptions, and status meanings that RootCause and Compliance might need to change.

BOUNDARY RULE

The danger is not the SELECT statement by itself. The danger is treating a private table shape as a public contract.

Four hidden couplings

Shared databases break microservices because they turn private implementation details into system-wide dependencies. The coupling is often stronger than a synchronous API call because it is less visible.

COUPLING	HOW IT APPEARS	NEXUS-1 EXAMPLE	FAILURE MODE
Schema coupling	Consumers depend on columns and joins.	Reporting reads RootCause.RootCauseCase directly.	RootCause cannot rename or split a table safely.
Transaction coupling	One service expects another table to change in the same unit of work.	RootCause verdict and Compliance review inserted together.	A local failure becomes a cross-service rollback problem.
Release coupling	Migrations must be coordinated across services.	Compliance deploy waits for RootCause column change.	Independent deployment becomes ceremonial.
Language coupling	Services share row meanings without a public language.	RootCauseCaseClosed means different things to Audit and Compliance.	The system drifts into ambiguous events and statuses.

PRACTICAL TEST

The more convenient the shared database feels, the more likely it is carrying architectural coupling that has not been paid for yet.

NEXUS-1 example: verdict, review, and audit

Consider the moment a root-cause verdict is issued. In a shared database design, one handler could update `RootCause.RootCauseCase`, insert `Compliance.ComplianceReview`, write `Audit.AuditEntry`, and refresh `Reporting.RootCauseCaseStatus` in one SQL transaction. It feels reliable because everything commits together.

WRITE	LOCAL OWNER	WHY SHARED WRITE IS A PROBLEM
<code>RootCause.RootCauseCase.FinalVerdict</code>	<code>RootCause</code>	This is the domain decision. Only <code>RootCause</code> should enforce it.
<code>Compliance.ComplianceReview</code>	<code>Compliance</code>	A review is a compliance workflow, not part of the verdict transaction.
<code>Audit.AuditEntry</code>	<code>Audit</code>	Audit records evidence; it should not become process orchestration.
<code>Reporting.RootCauseCaseStatus</code>	<code>Reporting</code>	This is a projection and can lag; it is not source-of-truth.

The distributed design accepts a less comfortable truth: the verdict can be issued before the review appears, the audit entry can be written after the event is consumed, and reporting can be stale for a short time. The system becomes harder, but its ownership becomes honest.

HONEST BOUNDARY

If the business rule requires a compliance review to exist before a verdict is visible, model that as a workflow rule. Do not hide it as a cross-service table transaction.

The schema is not the public contract

From Schema to System treated SQL schemas as serious architectural sectors. That remains valuable. But a SQL schema boundary inside one database is not the same thing as a microservice contract. A table is optimized for local persistence. A contract is optimized for safe communication with outsiders.

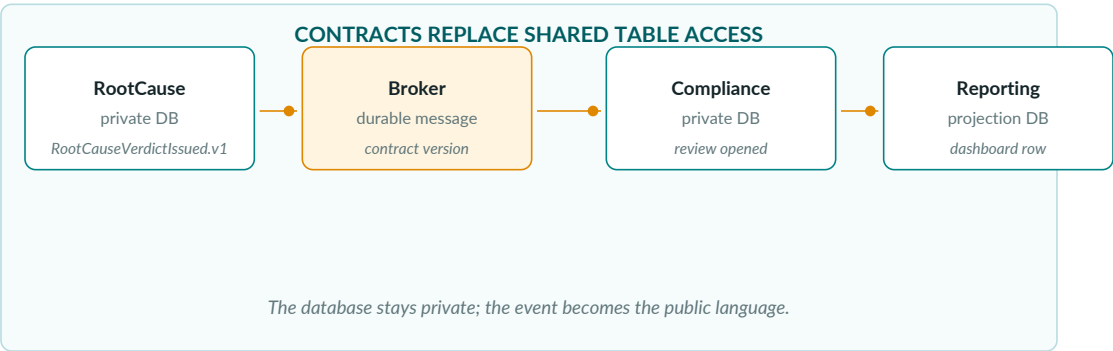
PRIVATE TABLE SHAPE	PUBLIC CONTRACT SHAPE
May contain persistence IDs, nullable columns, lifecycle flags, and rowversion.	Contains only the facts the consumer is allowed to rely on.
Changes through migrations controlled by the owning service.	Changes through versioned API or event contracts.
Can be normalized for local integrity.	Can be denormalized for consumer stability.
May reveal implementation names.	Uses published language: RootCauseVerdictIssued.v1.
Should not be queried by other services.	Is explicitly meant to be consumed by other services.

```
public sealed record RootCauseVerdictIssuedV1(
    Guid EventId,
    int RootCauseCaseId,
    int UnitId,
    string VerdictCode,
    decimal Confidence,
    DateTime IssuedAtUtc,
    string IssuedBy);
```

The event contract says less than the tables know, on purpose. It says the stable facts other services may consume. It does not export every column of RootCauseCase.

What the legal interaction looks like

The alternative to shared tables is not ignorance. Services still collaborate, but they collaborate through contracts that carry facts across boundaries. The receiver stores what it needs locally and labels it as received truth, not owned truth.



Reporting is the exception that proves the rule

Reporting often tempts teams back into shared databases. A dashboard wants one view of everything: alarm flood status, verdict, compliance state, audit evidence, and operator-facing recommendation. Direct SQL makes that view easy.

The correct conclusion is not that reporting owns the truth. Reporting owns a read model. A read model can duplicate data, join across facts, and optimize for the screen - but it must be rebuildable from public facts and must show freshness when users might misread delay as certainty.

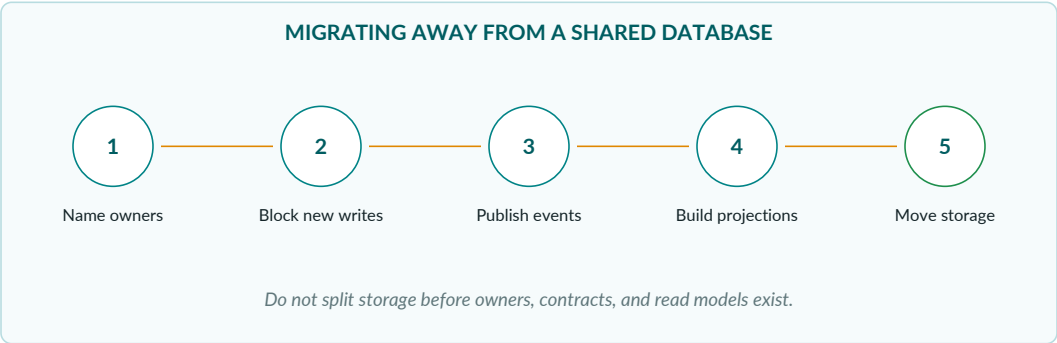
REPORTING NEED	SAFE DESIGN	UNSAFE DESIGN
Operator status page	Projection fed by events and read APIs.	Live join across private service tables.
Compliance dashboard	Compliance-owned view or reporting projection with source markers.	Reporting updates Compliance status directly.
Audit evidence timeline	Audit API/projection ordered by correlation ID.	Dashboard reconstructs audit meaning from raw tables.
Historical analytics	Replicated warehouse or analytical store.	Production service tables queried by everyone.

REPORTING RULE

A projection can be shared. Source-of-truth should not be.

How to migrate away from a shared database

Most real systems do not begin perfectly. A mature modular monolith may already have a shared database, and NEXUS-1 deliberately starts as an integrated demonstrator. The migration path should be gradual, not theatrical.



STEP	ACTION	DONE WHEN
Name owners	Mark every table as owned by exactly one context/service.	No table has two write owners.
Block new cross-writes	Prevent new code from writing another context's tables.	Code review and permissions enforce the rule.
Publish facts	Add outbox events for decisions others need.	Consumers can stop polling private tables.
Build projections	Replace cross-service joins with read models.	Dashboards read projections, not owners.
Move storage	Split schema/database when operationally justified.	The split changes deployment, not the model.

MIGRATION ADVICE

Do not start by moving tables to another server. Start by removing illegal reasons to touch them.

Shared database decision checklist

Before allowing any service to touch a table owned by another service, ask the uncomfortable questions in writing. The answer is usually not never, but it must be exceptional, temporary, and named.

QUESTION	ACCEPTABLE ANSWER
Who owns this table?	One named service or context, with write authority.
Is the access read-only or writable?	Writable access across ownership is a red flag.
Is there a public event or API that should be used instead?	If yes, use the contract; do not bypass it.
Is this for reporting?	Prefer a projection, warehouse, or owner-provided read API.
What happens when the owner changes the schema?	Consumers should not break on private migrations.
Can database permissions enforce the boundary?	The runtime account should not be able to write foreign truth.
Is the exception temporary?	Give it an owner, expiry, and replacement path.

HONEST BOUNDARY

A shared database can be a transitional state. It should not be the architecture you pretend is microservices.

For the NEXUS-1 journey, this means the schema atlas remains invaluable as the map of existing truth, but each future service must convert that map into private ownership, public contracts, and explicit read models.

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- Why does a shared writable database often create a distributed monolith?
- What is the difference between a SQL schema and a service contract?
- Why is direct Reporting access to RootCause tables dangerous after service extraction?
- Which NEXUS-1 writes belong to RootCause, Compliance, Audit, and Reporting respectively?
- What does RootCauseVerdictIssued.v1 protect that a direct SQL join does not protect?
- How can a projection duplicate data without becoming the source of truth?
- What is the safest migration order away from shared database access?

NEXT CHAPTER

Chapter 14 continues Part III by examining cross-service references and identity: how one service can point to another service's object without owning it, joining to it, or pretending a foreign key still crosses the network.

The shared database is not evil. It is simply honest about being shared. The mistake is calling it microservices while every service still reaches into the same private truth.

PART III - SERVICE OWNERSHIP AND DATA INDEPENDENCE

14. Cross-Service References and Identity

When the database was one place, a foreign key could make another context feel near. Once the contexts become services, that comfort disappears. The question becomes simple and dangerous: how do I point to something I do not own?

The previous two chapters separated ownership from storage and warned against shared writable databases. This chapter handles the next practical problem. RootCause still needs to know which Unit a case belongs to. Compliance still needs to know which verdict triggered a review. Audit still needs to say what entity was affected. The difference is that those references can no longer behave like ordinary in-process navigation properties or cross-schema foreign keys.

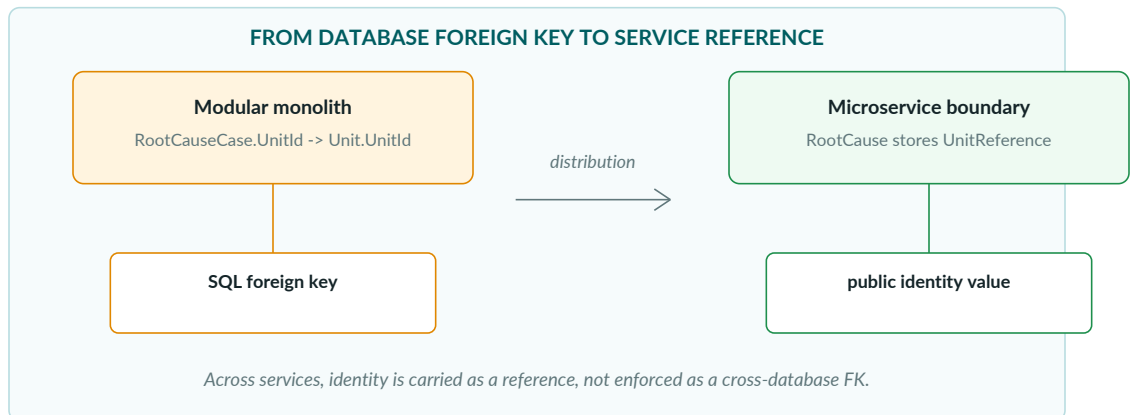
CENTRAL RULE

Across a microservice boundary, an identifier is not a database relationship. It is a public promise made by the owning service and stored by another service as a reference.

The design goal is not to remove references. The design goal is to make every reference honest about what it can and cannot guarantee.

What was easy in the modular core

Inside the modular monolith, the NEXUS-1 schema could use foreign keys as passports between sectors. ReactorFleet.Unit could be referenced by Instrumentation.Signal, AlarmManagement.AlarmEvent, RootCause.RootCauseCase, and Compliance records. The database enforced existence. EF Core could expose navigation properties where appropriate. A join could prove that two rows were speaking about the same physical unit.



That discipline was correct for an integrated data backbone. It made the model explicit and prevented ambiguous strings from spreading through the system. But the same mechanism cannot simply cross a service boundary. A foreign key is a database feature; a microservice boundary is an ownership and runtime boundary.

FROM PASSPORT TO REFERENCE

The old passport idea survives, but it changes form. In a service architecture, the passport is no longer a cross-schema FK. It is a stable public identity value, usually carried inside an API response or integration event.

Every identity needs an owner

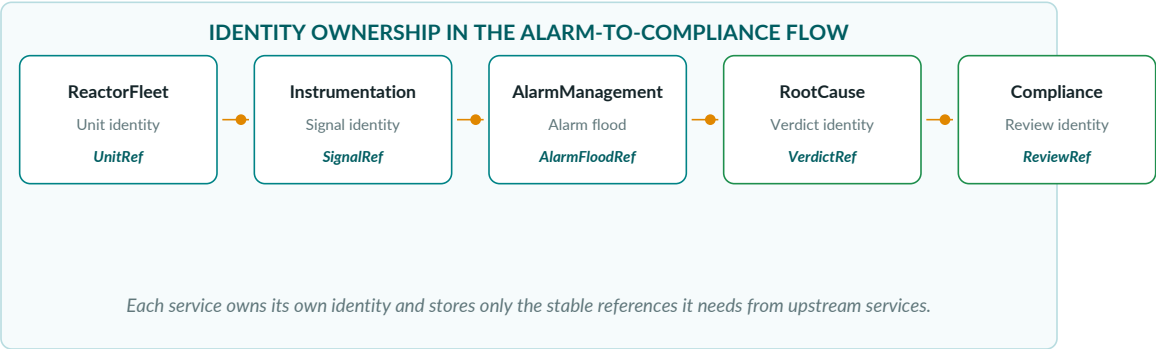
The first rule is ownership. The service that creates and governs an entity owns its identity. Other services may store that identity, but they do not own the thing it names. This sounds obvious until a downstream service starts validating, formatting, reclassifying, or deleting a remote entity as if it were local.

IDENTITY	OWNER	WHO MAY STORE A REFERENCE	WHAT THE REFERENCE MEANS
UnitRef	ReactorFleet	Instrumentation, AlarmManagement, RootCause, Compliance, Reporting	This record concerns that physical unit.
SignalRef	Instrumentation	AlarmManagement, DigitalTwin, RootCause, Reporting	This alarm or model variable concerns that signal.
AlarmFloodRef	AlarmManagement	RootCause, Audit, Reporting	This case was opened because that flood was detected.
RootCauseVerdictRef	RootCause	Compliance, Audit, Reporting	This review or projection was triggered by that verdict.
ComplianceReviewRef	Compliance	Audit, Reporting, Operator Console	This compliance workflow has its own local identity.

The reference travels downstream, but ownership does not. RootCause can refer to a Unit; it cannot redefine what a Unit is. Compliance can refer to a verdict; it cannot change the verdict.

NEXUS-1 identity map

The alarm-to-compliance flow is a chain of local decisions linked by public references. Each step creates or consumes identity. Each step must be clear about whether it is owning a fact or merely pointing to one.



STEP	LOCAL DECISION	UPSTREAM REFERENCE USED
AlarmManagement raises alarm	AlarmEventId / AlarmFloodId	UnitRef, SignalRef
RootCause opens case	RootCauseCaseld	AlarmFloodRef, UnitRef
RootCause issues verdict	RootCauseVerdictRef	RootCauseCaseld, evidence refs
Compliance opens review	ComplianceReviewId	RootCauseVerdictRef, UnitRef
Reporting builds read model	Projection row id	All public references and correlation id

TRACE IS NOT OWNERSHIP

A correlation id traces the flow. It does not replace ownership. It tells us which events belong to the same story, not which service owns the objects inside that story.

Public identity is not always the database key

A common mistake is to expose every database primary key as the public identity of a service. Sometimes that is acceptable, especially in an internal system. But it should be a deliberate decision, not an accident. Database keys are optimized for persistence. Public identities are optimized for contracts, logs, support, and long-term reference.

IDENTIFIER TYPE	GOOD FOR	RISK
INT identity key	Fast local storage, joins, EF Core identity map.	Leaks implementation and can become hard to merge, migrate, or expose safely.
GUID / UUID	Global uniqueness, event IDs, distributed creation.	Harder for humans to read and support.
Business code	Readable references such as UNIT-01 or SIG-FW-104.	Can change if not governed as immutable public identity.
Composite reference	Service + type + public id, useful for generic audit subjects.	Needs strict formatting rules and validation.
Versioned reference	References a specific contract version or snapshot.	More verbose, but safer for historical evidence.

For NEXUS-1, a sensible rule is mixed. Internal tables may keep integer keys. Public contracts should carry stable reference values: EventId, CorrelationId, UnitCode or UnitPublicId, SignalTag, RootCauseCaseNumber, and RootCauseVerdictIssued.v1 payload identifiers.

IDENTITY RULE

Do not expose a primary key merely because it exists. Expose the identity that the receiving service is allowed to remember.

Store a reference, not a remote object

A local service often needs to remember enough about a remote object to remain useful even when the owner is unavailable. That does not mean it owns the remote object. It means it stores a local reference record: the public id, the type of thing referenced, where it came from, when it was observed, and sometimes a small display snapshot.

A LOCAL REFERENCE RECORD

ReferencedService	ReactorFleet
ReferencedType	Unit
PublicId	UNIT-EDESSA-1
DisplaySnapshot	Unit 1 / PWR-900
ObservedAtUtc	2026-07-08T12:00:00Z

The reference is local evidence of what was known, not ownership of the remote object.

```
public sealed record UnitReference(
    string PublicId,
    string DisplayNameSnapshot,
    DateTime ObservedAtUtc);

public sealed class RootCauseCase
{
    public int RootCauseCaseId { get; private set; }
    public UnitReference Unit { get; private set; }
    public AlarmFloodReference TriggeringFlood { get; private set; }
    // No ReactorFleet.Unit navigation property here.
}
```

The object above does not pretend that RootCause owns Unit. It only records which unit the case concerns and what was known about that unit when the case was opened.

No cross-service navigation properties

In the modular core, an EF Core navigation property can be helpful. Across services, it becomes misleading. A property named Unit on RootCauseCase looks like an object the aggregate can navigate to and trust. In a service design, the RootCause service cannot lazy-load ReactorFleet.Unit, cannot rely on a local transaction around it, and cannot enforce its invariants.

IN THE MODULAR MONOLITH	AFTER SERVICE EXTRACTION
RootCauseCase has UnitId and maybe Unit navigation.	RootCauseCase has UnitReference value object.
Database FK proves the unit row exists at commit time.	Reference validity is checked through contract, cache, or event history.
A query can join RootCause to ReactorFleet.	A projection or API composition creates the joined view.
EF Core can track object graphs.	Each service tracks only its local aggregate graph.
Cascade and restrict rules can be database-enforced.	Lifecycle rules become explicit policies and events.

AGGREGATE BOUNDARY

A reference can be inside an aggregate. A remote aggregate should not be inside an aggregate.

This is one of the places where microservices make the code less comfortable but more honest. The model stops pretending that remote state is local state.

How reference validation works

A service still needs protection against invalid references. The mistake is to demand the same kind of guarantee a local foreign key provided. Distributed validation comes in several strengths, and each has a cost.

VALIDATION STYLE	WHEN IT FITS	COST
Synchronous owner check	Creating a case from an operator command where immediate feedback matters.	Couples availability to the owner service.
Cached reference catalogue	Frequently used stable references such as Unit or Signal.	Cache staleness must be visible and acceptable.
Event-built local table	Downstream service listens to UnitRegistered / SignalPublished events.	Requires messaging and replay discipline.
Accept and reconcile later	Low-risk references where human review can correct mistakes.	Needs exception workflow and audit trail.
No validation at write time	Append-only audit or event ingestion where evidence must not be lost.	Readers must handle unknown subjects.

```
public interface IUnitReferenceChecker
{
    Task<UnitReferenceCheckResult> CheckAsync(
        string unitPublicId,
        CancellationToken cancellationToken);
}

// Application layer policy: fail fast for commands that require a known unit;
// accept-and-mark-unknown for evidence ingestion where losing evidence is worse.
```

The important point is that validation style is a policy choice. It should be named in the use case, not hidden behind a database join.

What if the referenced thing changes?

Remote identity creates uncomfortable lifecycle questions. What happens if a signal is renamed? What happens if a unit is decommissioned? What happens if a root-cause verdict is corrected? A local foreign key did not make those questions disappear; it only made them look like database rules. In services, they must become domain policies.

REMOTE CHANGE	LOCAL RESPONSE
Unit display name changes	Keep the old display snapshot for history; optionally refresh current display views.
Unit is retired or decommissioned	Existing cases remain valid; new commands may reject the reference.
Signal tag is replaced	Store both old tag evidence and new public signal identity if a mapping event exists.
RootCause verdict is corrected	Publish RootCauseVerdictCorrected.v1; Compliance reacts according to review state.
Referenced object becomes unknown	Do not delete local evidence; mark reference resolution as unknown and audit the condition.

HISTORICAL TRUTH

Historical evidence should not change merely because the current name of a referenced object changed. A case was opened with the knowledge available at that time.

This is especially important for NEXUS-1 because auditability is part of the architectural discipline. A reference is not only a pointer; it is also evidence of what the system believed when a decision was made.

Audit references are intentionally generic

Audit has a special problem. It must record facts about many kinds of subjects without owning any of them. If Audit tried to hold a foreign key to every possible table, it would become coupled to the whole platform. In a service architecture, Audit should store a generic subject reference and enough context to replay the story.

```
public sealed record AuditSubjectReference(
    string Service,
    string EntityType,
    string PublicId,
    string? DisplaySnapshot,
    DateTime ObservedAtUtc);

public sealed record AuditEntryWrittenV1(
    Guid EventId,
    Guid CorrelationId,
    AuditSubjectReference Subject,
    string Action,
    DateTime WrittenAtUtc);
```

SUBJECT	EXAMPLE REFERENCE
Root-cause verdict	Service=RootCause, EntityType=Verdict, PublicId=RCV-2026-0418-01
Compliance review	Service=Compliance, EntityType=Review, PublicId=CR-2026-00041
Unit	Service=ReactorFleet, EntityType=Unit, PublicId=UNIT-01
Signal	Service=Instrumentation, EntityType=Signal, PublicId=SIG-FW-104

This is not weaker than a foreign key for audit purposes. It is more honest: Audit records the subject as evidence, while the owning service remains the owner of the subject.

Reference rules for NEXUS-1 services

The following rules keep references useful without allowing them to become hidden ownership.

RULE	MEANING
1. Every public reference has an owner.	The owner service creates it, validates it, and defines what it means.
2. Downstream services store references as value objects.	They do not store remote aggregates or EF navigation properties.
3. Do not enforce cross-service identity with database FKs.	Use contracts, events, caches, or projections instead.
4. Store display snapshots for historical readability.	Do not rewrite old evidence when the current name changes.
5. Treat reference validation as a use-case policy.	Some commands fail fast; some evidence is accepted and reconciled.
6. Correlation IDs trace stories, not ownership.	Tracing and identity serve different purposes.
7. Reporting composes references into views.	A composed view is not the source of truth.

SERVICE IDENTITY QUESTION

The old relational model asked: can this row point to that row? The service model asks: who owns the thing being named, and what promise is the reference allowed to carry?

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- Why can a SQL foreign key not simply cross a microservice boundary?
- Which service owns Unit identity, Signal identity, AlarmFlood identity, RootCauseVerdict identity, and ComplianceReview identity?
- What is the difference between storing a UnitReference and owning a Unit?
- Why should RootCauseCase not contain a ReactorFleet.Unit navigation property after extraction?
- When is a synchronous reference check acceptable, and when should a cache or event-built catalogue be preferred?
- Why should historical display snapshots be stored with some references?
- How does Audit record subjects without becoming coupled to every service database?

NEXT CHAPTER

Chapter 15 continues the same theme by separating local truth from remote truth: what a service knows for itself, what it has heard from another service, and what must be treated as a projection or snapshot.

A service may remember another service's identity. It must not confuse memory with ownership.

PART III - SERVICE OWNERSHIP AND DATA INDEPENDENCE

15. Local Truth vs Remote Truth

A service is allowed to be certain about what it owns. About everything else, it must remember how it knows, when it learned it, and who remains the owner.

The previous chapter separated references from ownership. This chapter takes the same discipline deeper. Once NEXUS-1 is distributed, not every fact available to a service has the same authority. RootCause may know that a unit exists because ReactorFleet published it. Compliance may know that a verdict was issued because RootCause published an integration event. Reporting may know that both exist because it built a read model. Those are not the same kind of truth.

The word truth must therefore become more precise. Local truth is the data a service owns and protects. Remote truth is data owned somewhere else but observed here through an API, event, snapshot, cache, or projection. The architecture becomes safer when the code says which kind of knowledge it is handling.

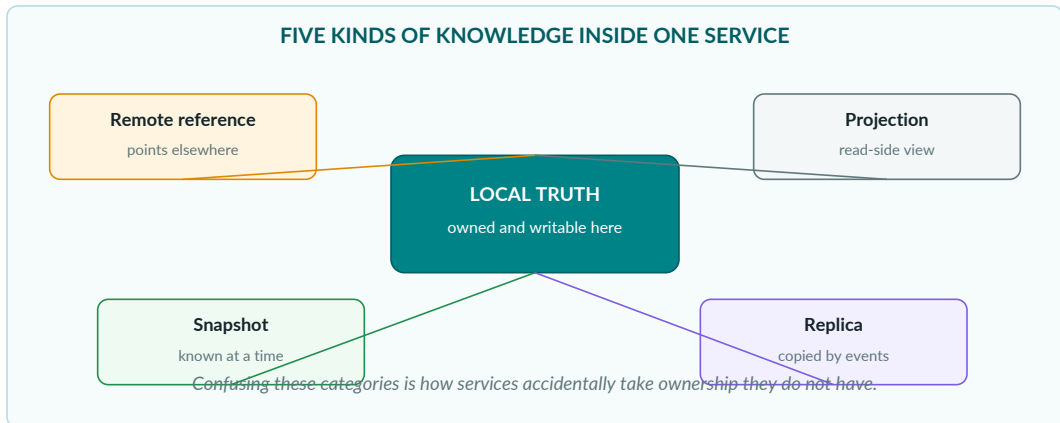
KIND OF KNOWLEDGE	QUESTION IT ANSWERS
Local truth	What is this service allowed to decide and write?
Remote reference	Which outside thing is this local record about?
Snapshot	What did we know about that outside thing at a specific time?
Replica / cache	What copy has this service built for local use?
Projection	What read-side view has been assembled for the user?

CHAPTER ARGUMENT

The danger is not that services remember remote facts. They must. The danger is that they forget those facts are remote.

Why this distinction matters

In a monolith, a developer can ask the database for the current answer. Join the tables, inspect the latest row, and the result looks authoritative. In a distributed architecture, a local service often holds an answer that was true when it was received, but may not be current when it is used.



That does not make the architecture broken. It makes the architecture honest. A root-cause case can be opened using a `UnitReference` that was valid when the alarm flood arrived. A compliance review can be triggered by `RootCauseVerdictIssued.v1` even if the operator console has not yet refreshed. A report can display a status that lags the owner by a few seconds. What must not happen is hiding that lag and calling it immediate truth.

PLAIN SENTENCE

A distributed system does not eliminate truth. It multiplies the distance between the owner of truth and the readers of truth.

Local truth is writable, protected, and accountable

Local truth is not simply data stored in a local database. It is the data the service is responsible for governing. The service owns the rules, the transaction boundary, the lifecycle, and the correction path. It can say yes or no because the decision belongs to it.

SERVICE	LOCAL TRUTH	LOCAL DECISION
ReactorFleet	Units, reactors, equipment identity, lifecycle state	Is this unit active, retired, simulated, or unavailable?
Instrumentation	Signals, tags, quality state, readings, calibration metadata	Is this signal valid enough to be used?
AlarmManagement	Alarm definitions, alarm events, alarm floods, acknowledgements	Has an alarm flood occurred?
RootCause	Cases, evidence, hypotheses, verdicts	Can this case close, and what verdict is issued?
Compliance	Reviews, findings, obligations, closure state	Does this verdict require compliance action?
Reporting	Projection definitions, dashboard snapshots, exports	What read model is available to display?

The local truth rule is the same rule the DDD volumes already taught at the aggregate level, now raised to the service level. The object that owns the rule must be the object that protects the change. The service that owns the language must be the service that publishes the public fact.

OWNERSHIP TEST

When a service changes local truth, it should be able to explain the change from its own model and its own audit trail.

Remote truth is observed, copied, or projected

Remote truth is not forbidden. A useful service architecture would be impossible if every service knew only itself. The rule is that remote truth must be labelled by how it arrived and what authority it carries.

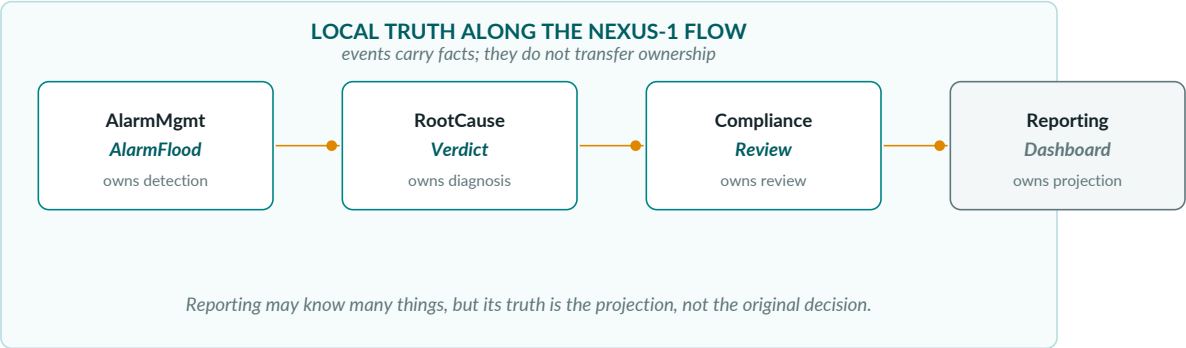
REMOTE KNOWLEDGE	HOW IT ARRIVES	WHAT IT MAY BE USED FOR	WHAT IT MUST NOT DO
Reference	Stored public id from event or command	Link a local record to an outside owner	Pretend to be the outside aggregate
Snapshot	Event payload or API response captured at a time	Historical readability and local decision context	Silently update old evidence
Cache / replica	Subscribed event stream or periodic sync	Fast validation and local read performance	Become an independent source-of-truth
Projection	Read model assembled from many owners	Operator display, report, dashboard	Drive authoritative workflow decisions
Owner query	Synchronous call to owning service	Fresh confirmation when necessary	Replace clear ownership with chatty coupling

PROJECTION WARNING

A projection can be correct for display and still wrong as a place to make decisions. Reporting truth is user-interface truth, not domain authority.

The NEXUS-1 flow as local truths

The recurring alarm-to-compliance flow becomes clearer when each step is read as a local truth produced by one owner and consumed as remote truth by the next owner.

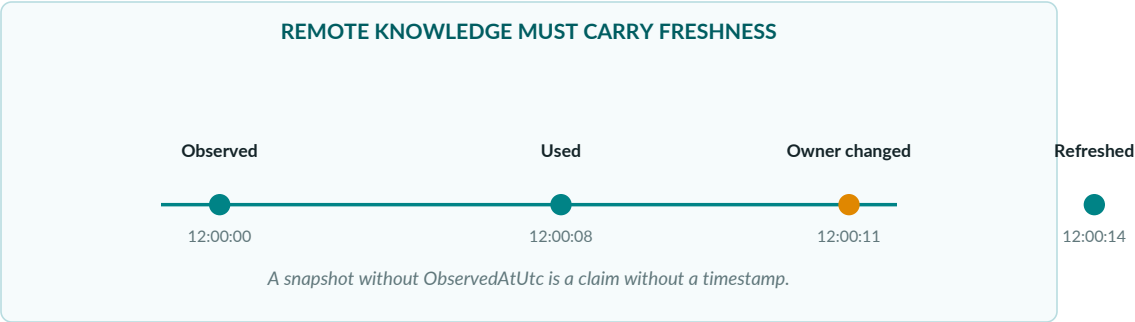


FLOW STEP	OWNER	LOCAL TRUTH PRODUCED	REMOTE TRUTH CONSUMED
Alarm flood detected	AlarmManagement	AlarmFloodDetected.v1	Signal and Unit references from Instrumentation/ReactorFleet
Root-cause case opened	RootCause	RootCauseCase state	AlarmFlood reference and alarm snapshot
Verdict issued	RootCause	RootCauseVerdictIssued.v1	Evidence snapshots, case history, signal references
Compliance review triggered	Compliance	ComplianceReview state	RootCause verdict event and Unit reference
Dashboard updated	Reporting	Read-model projection	Events from AlarmManagement, RootCause, Compliance, Audit

No row in that table transfers ownership. Events transfer facts. APIs answer questions. Projections compose views. The owner remains the owner.

Staleness is a design property, not an embarrassment

A service that uses remote truth must be able to say how fresh it is. Sometimes the answer does not matter: a display name snapshot on an old audit entry can remain old forever. Sometimes it matters greatly: a command that opens a new review may need the latest state from the owner.



CONTEXT	FRESHNESS EXPECTATION	DESIGN RESPONSE
Historical evidence	It should remain as it was observed.	Store ObservedAtUtc and do not rewrite it.
Operator dashboard	A few seconds may be acceptable if visible.	Show last updated time and status.
Command validation	May require current owner state.	Call owner, check cache version, or reject with retry.
Compliance decision	Usually needs stable event evidence, not live UI state.	Base decision on event payload and review policy.
Reporting export	Must be reproducible.	Stamp generation time, source versions, and query window.

FRESHNESS RULE

A stale value is not necessarily a bug. An unlabelled stale value is a bug waiting to become an argument.

Make remoteness visible in code

The code should not hide remote truth behind ordinary properties that look local. A value object can carry the source, the public identity, the observation time, and the version or event that produced it.

```
public sealed record RemoteSnapshot<T>(  
    string SourceService,  
    string PublicId,  
    T Value,  
    DateTime ObservedAtUtc,  
    string? SourceVersion,  
    Guid? SourceEventId);  
  
public sealed record UnitSnapshot(  
    string UnitPublicId,  
    string DisplayName,  
    string LifecycleState);  
  
public sealed class ComplianceReview  
{  
    public RemoteSnapshot<RootCauseVerdictSummary> Verdict { get; private set; }  
    public RemoteSnapshot<UnitSnapshot> Unit { get; private set; }  
  
    // The review owns review state, not the verdict and not the unit.  
}
```

This is more verbose than a navigation property. That is the point. The model tells the reader that the review has copied evidence from outside owners. It does not pretend that Compliance owns the verdict or the unit.

NAMING DISCIPLINE

Type names are architectural documentation. UnitSnapshot, UnitReference, and RemoteSnapshot say something that Unit does not.

When may a service decide from remote truth?

The strict answer is not "never." The useful answer is: only when the decision explicitly accepts the authority and freshness of the remote knowledge it uses.

DECISION	MAY USE REMOTE SNAPSHOT?	WHY
Display case status in operator console	Yes	The screen is a projection and can show freshness.
Append audit evidence	Yes	Audit records what was known when the event arrived.
Open compliance review from verdict event	Yes	The event is the public fact RootCause intended to publish.
Close a root-cause case	No	Only RootCause local evidence and rules can close the case.
Mark a unit retired	No	Only ReactorFleet owns unit lifecycle.
Reject a command because referenced unit is unknown	Yes, with policy	A cache or owner check may be sufficient, depending on risk.

The decision matrix belongs near application services and process managers, not hidden inside generic infrastructure. A handler that accepts a remote snapshot should make that acceptance visible in its policy name, tests, and audit output.

AUTHORITY BEFORE MECHANISM

Never let convenience decide authority. Decide authority first; then choose the technical mechanism.

Reporting is not the source of truth

Reporting is the place where local truths are composed for reading. That makes it powerful and dangerous. A dashboard can display the alarm, the verdict, the compliance review, and the audit trail on one screen. Because it sees everything, it is tempting to let it decide something. It should not.

REPORTING MAY	REPORTING MUST NOT
Join projected facts for operator readability.	Change RootCause verdict state.
Show stale/refreshing/missing indicators.	Pretend projection freshness is owner freshness.
Export a reproducible snapshot.	Write back into owner service tables.
Support search, filtering, and historical views.	Become the hidden integration database.
Reveal conflicts and missing references.	Resolve conflicts by silently overwriting owner truth.

A good reporting service is honest about its own nature. It owns the projection. It owns the export. It owns the query experience. It does not own the domain facts it assembled.

CONSOLE BOUNDARY

The operator console may be the place where truth is seen. It is not automatically the place where truth is made.

Audit records the knowledge path

Auditability in a distributed system is not only the question "what happened?" It is also the question "how did this service know what it used?" That means audit entries should record source service, event id, correlation id, observed time, and sometimes the snapshot that justified the local decision.

```
public sealed record DecisionEvidence(  
    Guid CorrelationId,  
    string SourceService,  
    string ContractName,  
    string ContractVersion,  
    Guid SourceEventId,  
    DateTime ObservedAtUtc,  
    string SnapshotJson);  
  
// The audit trail records the knowledge path, not just the final result.
```

AUDIT FIELD	WHY IT MATTERS
SourceService	Names who owned the fact.
ContractName / version	Identifies the public language used.
SourceEventId	Lets the decision be traced to a published fact.
ObservedAtUtc	Separates historical knowledge from current owner state.
Snapshot payload or hash	Makes the decision replayable without rewriting history.

This is the same NEXUS-1 discipline carried forward: explanation without evidence is not enough. In microservices, evidence includes the path by which a remote fact arrived.

Local/remote truth rules for NEXUS-1

The rules below are deliberately simple. They are the kind of rules a team can put in an architecture decision record and then check in code review.

RULE	MEANING
1. Only the owner writes source-of-truth.	No service writes another service's private tables or private model.
2. Remote facts must carry source and time.	ObservedAtUtc and SourceService are not optional decoration.
3. Snapshots are evidence, not ownership.	A copied unit name does not make Compliance the owner of Unit.
4. Projections are read models.	They may be queried and displayed; they must not become command authority.
5. Freshness is part of the contract.	A reader must know whether the value is current, stale, unknown, or historical.
6. Decisions state their authority.	A handler should reveal whether it used local truth, owner confirmation, or accepted snapshot evidence.
7. Audit records the knowledge path.	A future reader can replay not only the decision but the facts available when it was made.

NEXUS-1 RULE

The sentence to remember: local truth decides; remote truth informs; projections display; audit remembers.

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- What is the difference between local truth and remote truth?
- Why is a UnitSnapshot not the same thing as a Unit aggregate?
- Which service owns the truth of an alarm flood, a root-cause verdict, a compliance review, and a dashboard projection?
- When is stale data acceptable, and what must the system show or record when using it?
- Why must Reporting not become the hidden authority just because it sees many facts?
- What fields make a remote snapshot auditable?
- How does this chapter prepare the ground for reference data, replication, snapshots, and the NEXUS-1 data ownership map?

NEXT CHAPTER

Chapter 16 continues from this point by asking which remote facts are stable enough to become reference data, which should remain snapshots, and when replication is acceptable rather than accidental.

A service does not become safer by knowing everything. It becomes safer by knowing exactly what kind of knowledge it holds.

PART III - SERVICE OWNERSHIP AND DATA INDEPENDENCE

16. Reference Data, Snapshots, and Replication

Once local truth and remote truth have been separated, the next question is practical: which remote facts may be copied, which must be frozen as evidence, and which should be kept in sync as a replica?

Chapter 15 gave the rule: local truth decides, remote truth informs, projections display, audit remembers. This chapter turns that sentence into a data design discipline. It is the bridge between service ownership and real database independence, because no microservice can remain useful if it refuses to know anything outside itself - and no microservice can remain honest if it copies everything and calls the copy truth.

THREE WAYS REMOTE KNOWLEDGE ENTERS A SERVICE

REFERENCE DATA

slow, governed vocabulary
versioned and seeded

SNAPSHOT

fact captured at a moment
immutable evidence

REPLICA

local copy kept in sync
eventually consistent

The mistake is not copying data. The mistake is copying it without naming the kind of copy.

CENTRAL DISTINCTION

The question is not "may services duplicate data?" They often must. The real question is: what kind of duplication is this, who owns the original, how fresh must the copy be, and what is the copy allowed to decide?

Not all copied data has the same authority

A service can hold data for several reasons. Some values are stable vocabulary. Some are historical evidence. Some are read-side acceleration. Some are accidental copies made because an API call was inconvenient. Only the first three deserve architectural names.

COPY TYPE	QUESTION IT ANSWERS	ALLOWED AUTHORITY	NEXUS-1 EXAMPLE
Reference data	What controlled vocabulary do we share?	Validation and display within a known version	AlarmSeverity, EngineeringUnit, FindingSeverity
Snapshot	What did we know when this decision happened?	Evidence for a past decision	Unit name and verdict summary captured by Compliance
Replica	What current read-side view do we need locally?	Fast queries, local validation, dashboard composition	Reporting copy of RootCause verdict status
Cache	What did the owner answer recently?	Temporary performance help only	Short-lived Unit lookup in an API gateway
Accidental copy	What was easiest to store?	None; should be redesigned	Copying RootCauseCase rows into Compliance tables for convenience

The table is intentionally strict. A copy without a type becomes a design trap. It will first be used for display, then for validation, then for a decision, and finally someone will ask why two services disagree.

NAMING RULE

A copied value must carry its role. Reference data is governed. A snapshot is historical. A replica is operational. A cache is temporary.

Reference data is shared vocabulary, not shared ownership

Reference data is the controlled vocabulary of the system: severities, statuses, engineering units, finding categories, signal qualities, contract versions, and other small lists whose meaning must remain stable enough for many services to use them.

REFERENCE DATA	OWNER	WHY IT MAY BE COPIED
EngineeringUnit	CorePlatform or a dedicated reference service	Instrumentation, alarms, twin variables, and reports must agree on symbols and precision.
AlarmSeverity	AlarmManagement public vocabulary	RootCause and Reporting may display severity without owning alarm rules.
HypothesisStatus	RootCause internal or published vocabulary	Only public statuses should cross the boundary; internal workflow states should not.
FindingSeverity	Compliance public vocabulary	Reporting and dashboards can filter findings consistently.
Contract version	Platform governance	Consumers need to know which public language version they are reading.

A common mistake is to put all reference data into a shared database because it is small. Size is not the criterion. Ownership is. If a list is global platform vocabulary, it may belong to a platform/reference service. If it is domain vocabulary, the domain that owns the word owns the list.

REFERENCE-DATA DISCIPLINE

Reference data should be versioned, seeded, named, and governed. It should not be a folder of enums that every service copies differently.

Reference data needs versions and public codes

A service should not depend on another service's private integer primary keys. Public reference data needs stable codes and, when meaning changes, visible versions. The code is the public language. The surrogate key is local storage.

```
public sealed record ReferenceCode(
    string Code,
    string Name,
    string Version,
    bool IsActive,
    DateTime EffectiveFromUtc,
    DateTime? EffectiveToUtc);

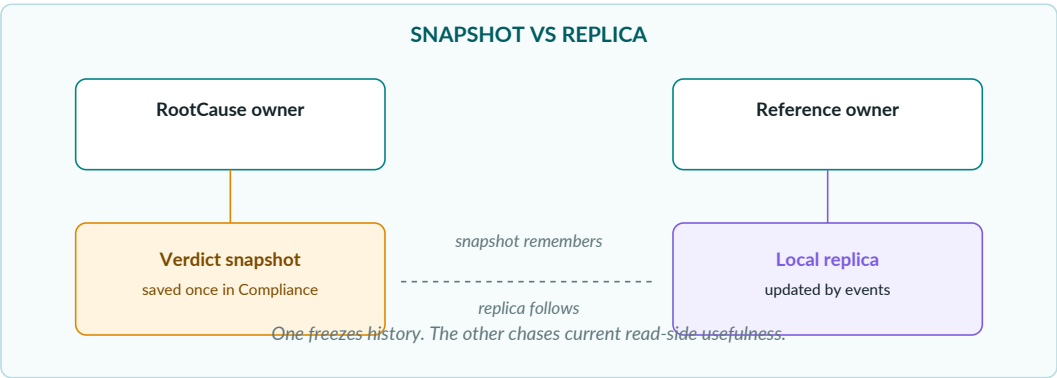
// Local database key is private.
// Public Code + Version is the cross-service contract.
public sealed record AlarmSeverityRef(
    string Code,           // INFO, WARN, CRITICAL
    string Version,       // v1
    string DisplayName,
    int SortOrder);
```

FIELD	WHY IT EXISTS
Code	Stable public identifier; survives local database changes.
Version	Lets old messages remain interpretable when meaning changes.
IsActive	Prevents new use without breaking old history.
Effective dates	Make time-based interpretation possible.
DisplayName	Supports UI without turning UI labels into rules.

In NEXUS-1 this matters because old alarm floods, old verdicts, old compliance reviews, and old reports must remain readable. A severity code that meant one thing in January should not silently mean another thing in June.

Snapshots freeze evidence

A snapshot is not a poor replica. It has a different job. A snapshot captures what a service knew at the time a decision was made, so that the decision remains explainable later even if the owning service changes.



SNAPSHOT SHOULD CONTAIN	REASON
Public identity of the original	Lets the owner be queried later if needed.
Human-readable summary	Keeps old audit and review pages understandable.
ObservedAtUtc	Separates past knowledge from current state.
Source service and contract version	Names the public language used.
Source event id or correlation id	Makes the knowledge path replayable.
Optional hash of payload	Allows tamper-evident comparison without storing everything twice.

HISTORICAL RULE

A snapshot is allowed to become old. That is its purpose. Rewriting snapshots to make them current destroys evidence.

A snapshot shape for NEXUS-1

The shape below is deliberately plain. It can be represented as owned type, JSON column, or separate snapshot table depending on the service. The architectural point is that the copy announces itself as a snapshot.

```
public sealed record RootCauseVerdictSnapshot(  
    string VerdictPublicId,  
    string CasePublicId,  
    string VerdictText,  
    decimal Confidence,  
    string IssuedByService,  
    string ContractName,      // RootCauseVerdictIssued  
    string ContractVersion,   // v1  
    Guid SourceEventId,  
    DateTime IssuedAtUtc,  
    DateTime ObservedAtUtc);  
  
public sealed class ComplianceReview  
{  
    public RootCauseVerdictSnapshot Verdict { get; private set; }  
    public ReviewState State { get; private set; }  
  
    // Compliance owns State. RootCause owns the verdict.  
}
```

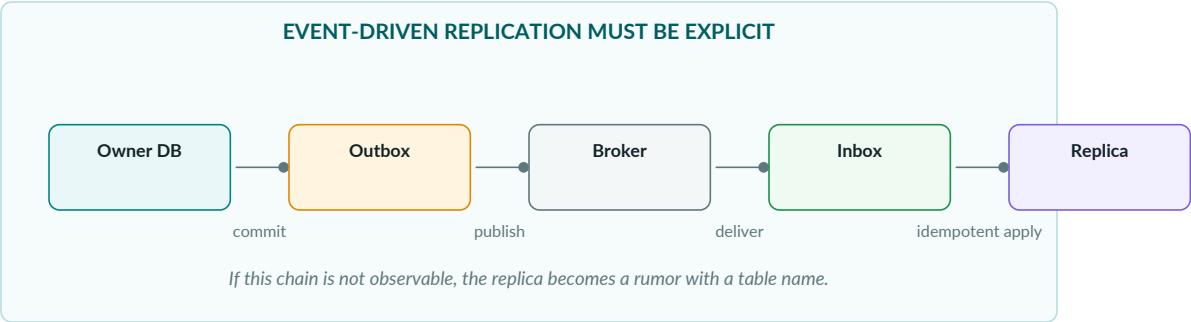
A reader can see immediately what is local and what is remote. Compliance may decide whether to open, assign, escalate, or close a review. It may not revise the verdict. The snapshot gives it enough evidence to explain why the review exists.

WHY THIS MATTERS

Snapshot objects are one of the cleanest ways to preserve auditability after service extraction.

Replication is a promise to keep a copy useful

A replica is different from a snapshot because it is expected to change. It is a local copy maintained from the owner's published facts so that the consuming service can query quickly, validate locally, or build a user experience without calling several owners for every screen.



REPLICA USE	ACCEPTABLE WHEN	DANGER
Reporting dashboard	The UI can tolerate eventual consistency and show freshness.	Users may think the projection is source-of-truth.
Local validation cache	Rejecting unknown public ids is useful and low risk.	False rejection if cache is stale.
Search index	The owner model is not optimized for global search.	Index lag and missing updates.
Read-side aggregate	The query needs facts from many owners.	Replica becomes a hidden integration database.

OPERATIONAL PRICE

Replication is acceptable only when lag, duplicates, reordering, replay, and rebuild are designed - not discovered in production.

A replica must be rebuildable

The most important property of a replica is not speed. It is rebuildability. If the local copy becomes corrupt, stale, or incomplete, the service must know how to discard and rebuild it from authoritative events or owner APIs.

REPLICA RULE	MEANING
Consume public events, not private tables.	A replica follows the owner's published language.
Store source event id.	Idempotency and replay require a stable event identity.
Record last applied position.	The service must know how far it has caught up.
Handle duplicates.	At-least-once delivery is normal, not exceptional.
Handle reordering or reject it explicitly.	Some flows require ordering; others can converge without it.
Expose freshness.	Dashboards and diagnostics need last updated time and lag.
Support rebuild.	A broken replica should be a repair task, not a data mystery.

```
public sealed class ProjectionCheckpoint
{
    public string ProjectionName { get; init; } = null!;
    public string SourceStream { get; init; } = null!;
    public long LastAppliedPosition { get; set; }
    public DateTime LastAppliedAtUtc { get; set; }
}
```

A checkpoint table is not glamorous. It is the line between a read model and a pile of copied rows.

NEXUS-1 classification examples

The same NEXUS-1 data can fall into different categories depending on where it is held. That is why the classification is always from the point of view of a specific service.

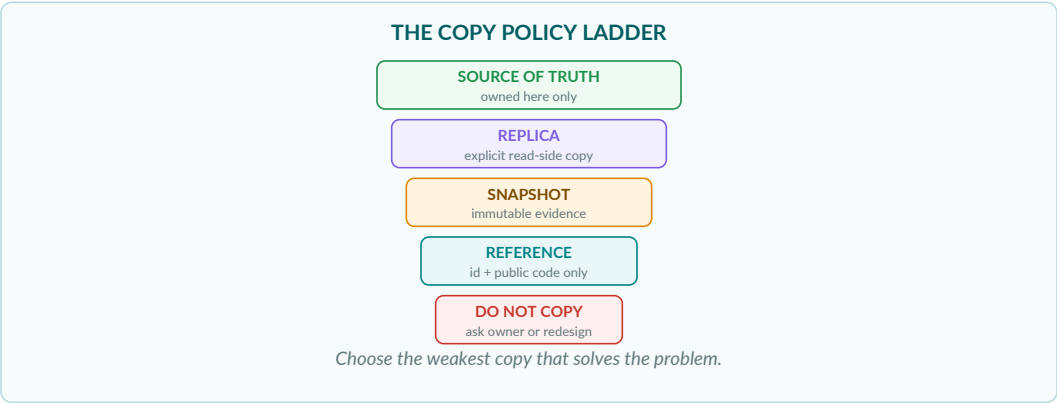
DATA HELD BY SERVICE	CATEGORY	OWNER	COMMENT
Compliance stores Verdict summary	Snapshot	RootCause	Evidence for why a review was opened.
Reporting stores Verdict status	Replica / projection	RootCause	For dashboards and exports; not authority.
RootCause stores AlarmFlood id	Reference	AlarmManagement	It points to the owner; may include a flood snapshot.
AlarmManagement uses SignalQuality codes	Reference data	Instrumentation or CorePlatform	Published vocabulary, not private signal state.
Operator Console shows complete flow	Projection	Multiple owners	UI composition, not domain truth.
Audit stores decision evidence	Snapshot / audit evidence	Owner of each source event	Immutable path of knowledge.

PERSPECTIVE RULE

The same value can be source-of-truth in one service, snapshot in another, replica in a third, and projection in the UI. The label depends on ownership.

Choose the weakest copy that works

Microservice data design becomes safer when the default question is not "can we copy it?" but "what is the weakest form of knowledge that solves the problem?" Weak means less authority, less synchronization pressure, and fewer false claims.



NEED	WEAKEST SUITABLE FORM
Display a name next to an old review	Snapshot.
Validate that a public id is syntactically known	Reference or short-lived cache.
Build a dashboard across many owners	Projection or replica with freshness.
Make a domain decision owned by another context	Do not copy; ask the owner or move the decision.
Keep audit evidence forever	Immutable snapshot and source event link.
Search across verdicts and findings	Replica / index, rebuilt from public events.

DESIGN HEURISTIC

The weakest-copy rule prevents a read convenience from becoming an accidental business authority.

What should not be replicated

The presence of a message broker does not mean every fact deserves an event, and the presence of an event does not mean every consumer should store it forever. Some data should remain behind the owner boundary.

DO NOT REPLICATE	WHY
Private aggregate internals	They change with the owner model and are not public language.
Temporary workflow steps that have no outside meaning	They leak implementation instead of publishing facts.
Sensitive operational details not needed by the consumer	Minimum necessary knowledge protects the model and the people using it.
Values that require immediate correctness for a command	Use owner confirmation or redesign the boundary.
Large payloads copied only because joins are inconvenient	This creates a distributed shared database in disguise.
Anything whose owner cannot be named	If ownership is unknown, copying spreads the ambiguity.

In NEXUS-1 terms, RootCause may publish a verdict. It does not need to publish every internal hypothesis score. AlarmManagement may publish that a flood was detected. It does not need to expose every private suppression rule. Compliance may publish review status. It does not need to expose every draft reviewer note.

EVENT BOUNDARY

A public event is not a database dump. It is a sentence in the published language.

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- What is the difference between reference data, a snapshot, a replica, a projection, and a cache?
- Why should public codes and versions cross service boundaries instead of private database keys?
- Why is a snapshot allowed to become old, while a replica is expected to be refreshed?
- What makes a replica rebuildable and auditable?
- Why should Reporting not become the hidden integration database?
- What is the weakest-copy rule, and how does it protect service ownership?
- Which NEXUS-1 facts should be snapshots, which should be replicas, and which should remain owned by their source service?

NEXT CHAPTER

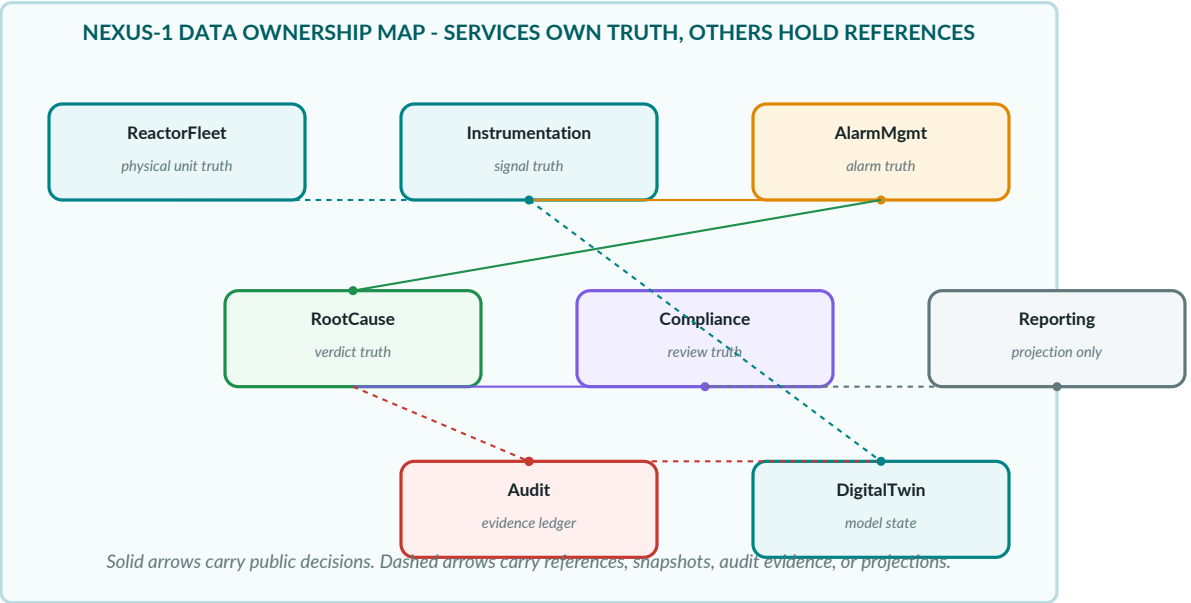
Chapter 17 will use these rules to draw the NEXUS-1 data ownership map: which service owns which truth, which public facts cross boundaries, and which read models are deliberately only projections.

Service independence does not mean no copying. It means every copy has a name, an owner, a timestamp, and a limit.

PART III - SERVICE OWNERSHIP AND DATA INDEPENDENCE

17. NEXUS-1 Data Ownership Map

Part III has moved from rule to practice: database-per-service is about ownership, shared databases break that ownership, identities must cross boundaries carefully, local truth and remote truth must be separated, and copies must be labelled as reference data, snapshots, replicas, projections, or caches. This chapter draws the map those rules imply for NEXUS-1.



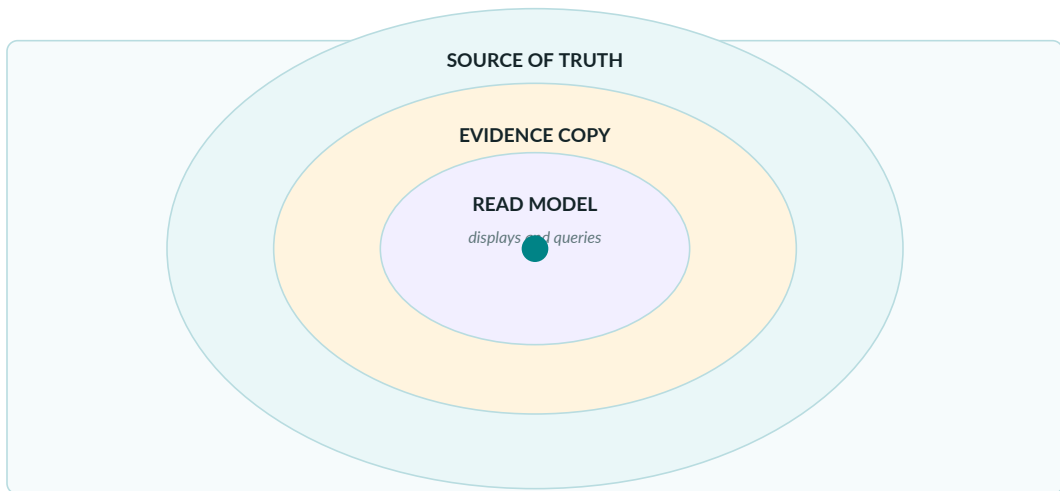
MAP RULE

A data ownership map is not an ER diagram. It does not show every table. It shows which service is allowed to change which truth, which facts it publishes, and which other services are allowed to remember or project those facts.

Why the map matters before code

A service boundary without a data ownership map is only a deployment wish. The team may create separate APIs, separate repositories, separate containers, and separate pipelines, but the system will still behave like one database if nobody can answer who owns each fact.

In NEXUS-1 the risk is high because the domain is naturally connected. A signal belongs to a unit. An alarm references a signal. A root-cause case uses alarms as evidence. A compliance review references the verdict. Reporting wants to join everything. Audit wants to remember everything. Without a map, every service has a reason to reach into every other service.



Only the inner owner changes truth. Outer layers preserve or present it.

BEFORE EXTRACTION

The ownership map forces a design conversation before a migration creates a hidden contract. It asks: who writes, who publishes, who consumes, who snapshots, who projects, and who must never decide?

The five ownership roles

For this book, each data relationship is classified using five roles. The same service can play different roles for different facts, but each individual fact should have one primary source-of-truth owner.

ROLE	MEANING	EXAMPLE
Owner	May create, change, validate, and retire the fact.	AlarmManagement owns AlarmEvent and AlarmFlood.
Publisher	Emits a public fact in a stable contract.	RootCause publishes RootCauseVerdictIssued.v1.
Consumer	Reacts to the public fact but does not rewrite it.	Compliance consumes the verdict event.
Evidence holder	Stores a snapshot because a later decision must be explainable.	Audit stores payload, source id, and hash.
Projector	Builds a read model for queries or UI composition.	Reporting stores denormalized flow status.

The words matter because they stop a common mistake. A consumer is not a co-owner. A projector is not a decision maker. An evidence holder does not drive workflow. A publisher does not leak its private aggregate state.

CONFLICT TEST

If two services both believe they own the same mutable fact, the boundary is not ready for service extraction.

Primary ownership map

The first table names the main source-of-truth service for each NEXUS-1 sector. It is deliberately conservative: foundational truth stays stable; service extraction begins where ownership and public contracts are already mature.

SECTOR / SERVICE	OWNS AS SOURCE OF TRUTH	PUBLIC FACTS IT MAY PUBLISH
ReactorFleet	Units, physical equipment, plant hierarchy, stable equipment identity.	UnitRegistered, UnitLifecycleChanged, EquipmentIdentityPublished.
Instrumentation	Signals, tags, sensor metadata, measurement quality, signal readings.	SignalRegistered, SignalQualityChanged, MeasurementAccepted.
DigitalTwin	Twin models, model variables, bindings, snapshots, divergence records.	TwinSnapshotRecorded, TwinDivergenceDetected.
AlarmManagement	Alarm definitions, alarm events, alarm floods, acknowledgements.	AlarmRaised.v1, AlarmFloodDetected.v1, AlarmAcknowledged.v1.
RootCause	Cases, hypotheses, evidence classification, verdicts, corrections.	RootCauseVerdictIssued.v1, RootCauseVerdictCorrected.v1.

READING THE MAP

This map does not say all five should be extracted first. It says that if they are extracted, these are the truths they must own.

Review, evidence, and read-side ownership

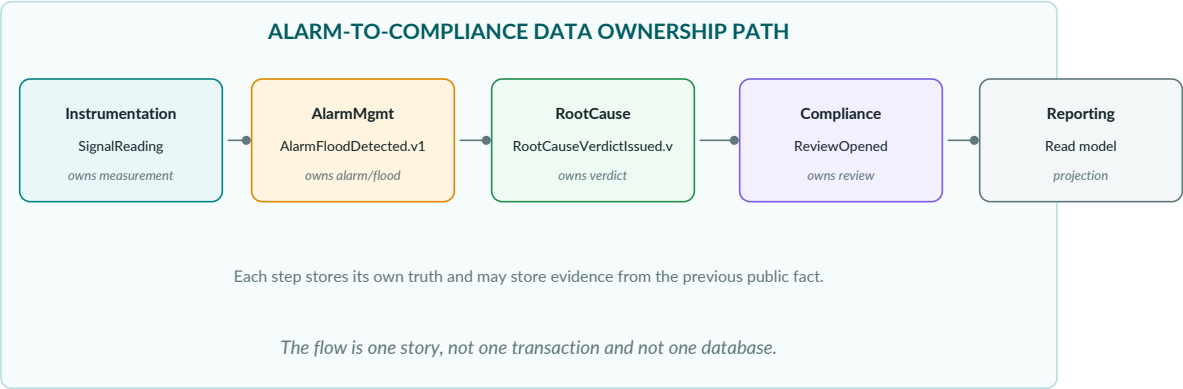
The second table covers the services most likely to appear early in a microservice transition because their ownership is clearer and their public contracts can be carefully limited.

SECTOR / SERVICE	OWNS AS SOURCE OF TRUTH	WHAT IT MUST NOT OWN
Compliance	Reviews, controls, findings, review decisions, regulatory workflow state.	It must not revise root-cause verdicts or alarm facts.
Audit	Append-only evidence ledger, event hashes, actor/timestamp evidence, correlation trail.	It must not drive the business workflow simply because it records it.
Reporting	Report definitions, generated report snapshots, read models, exports.	It must not become the source of truth for operational facts.
ReinforcementLearning	Training runs, policies, Q-table entries, advisory recommendations.	It must not issue actuator commands.
Operator Console / BFF	UI composition, session-specific view models, screen aggregation.	It must not invent domain truth when a service is unavailable.

This is where the earlier NEXUS-1 vocabulary protects the architecture. A recommendation is not a command. An audit entry is evidence, not workflow. A read model is not the source of truth. Those sentences are not style preferences; they are service-boundary safety rails.

The alarm-to-compliance ownership path

The central NEXUS-1 example can now be drawn as a data ownership path. The flow is one narrative, but the facts do not belong to one table, one transaction, or one service.



FLOW STEP	OWNER	DATA IT WRITES LOCALLY	WHAT CROSSES THE BOUNDARY
Signal received	Instrumentation	Measurement, SignalQuality	Public signal identity, quality fact, optional measurement summary.
Alarm raised	AlarmManagement	AlarmEvent, AlarmFlood	AlarmRaised.v1 or AlarmFloodDetected.v1.
Case opened	RootCause	RootCauseCase, Evidence, Hypothesis	CaseOpened internal or public only if needed.
Verdict issued	RootCause	Verdict, case state, outbox message	RootCauseVerdictIssued.v1.
Review triggered	Compliance	ComplianceReview, ReviewSnapshot	Verdict snapshot and source event id.
Status displayed	Reporting / BFF	Projection rows or composed view	Read model with freshness indicator.

Event ownership and public language

A public event belongs to the service that owns the fact being announced. Consumers may depend on the contract, but they do not own the event name, meaning, version, or correction rules.

PUBLIC CONTRACT	PUBLISHER / OWNER	CONSUMERS	DISCIPLINE
AlarmFloodDetected.v1	AlarmManagement	RootCause, Reporting, Audit	Publishes a flood fact, not root-cause judgment.
RootCauseVerdictIssued.v1	RootCause	Compliance, Audit, Reporting, Operator Console	Canonical event for verdict publication.
RootCauseVerdictCorrected.v1	RootCause	Compliance, Audit, Reporting	Correction is explicit; old facts are not silently overwritten.
RequestComplianceReview	RootCause or process manager	Compliance	Command/request contract, not an audit entry.
ComplianceReviewCompleted.v1	Compliance	Reporting, Audit	Compliance owns review outcome.
AuditEntryRecorded.v1	Audit	Diagnostics and evidence tooling	Evidence notification, not workflow trigger.

TERMINOLOGY DISCIPLINE

The canonical verdict event remains `RootCauseVerdictIssued.v1`. The book avoids `RootCauseCaseClosed` as the main public event because closing a case is workflow state; issuing a verdict is the domain fact other services need.

One writer per mutable truth

A practical data ownership map should be testable. The simplest test is a write matrix: for each mutable fact, exactly one service is allowed to write it. Other services may hold references, snapshots, replicas, or projections, but not the write authority.

WRITE AUTHORITY MATRIX					
	Alarm	RootCause	Compliance	Audit	Reporting
AlarmEvent	W	-	-	-	-
Verdict	-	W	-	-	-
Review	-	-	W	-	-
AuditEntry	-	-	-	W	-
ReportView	-	-	-	-	W

Every row has one writer. Readers use contracts, snapshots, or projections.

MUTABLE FACT	ONLY WRITER	OTHERS MAY HOLD
AlarmEvent state	AlarmManagement	Alarm id, severity snapshot, flood reference.
RootCause verdict	RootCause	Verdict snapshot, public verdict id, published event id.
Compliance review state	Compliance	Review status projection, audit evidence.
Audit entry	Audit	Audit entry id and correlation reference.
Report projection row	Reporting	No authority; rebuildable read-side copy.

WRITE RULE

One writer does not mean one reader. It means one service protects the invariant and accepts the cost of changing the fact.

Foreign keys become passports, not chains

Inside the modular core, cross-schema foreign keys acted as passports between sectors. In a service architecture, the passport idea survives, but it changes form. A foreign key between databases becomes a public identity carried by a contract, validated by published language, and sometimes cached or replicated locally.

MONOLITHIC / MODULAR CORE	SERVICE ARCHITECTURE VERSION
RootCause.Evidence -> AlarmManagement.AlarmEvent FK	Evidence stores AlarmEventPublicId plus snapshot of alarm evidence.
ComplianceFinding -> RootCauseCase FK	Compliance stores VerdictPublicId / CasePublicId and the triggering event id.
Reporting joins RootCause and Compliance tables	Reporting consumes public events and builds a read model.
Audit FK to changed entity table	Audit stores entity type, public id, event id, hash, actor and timestamp.
DigitalTwin binding FK to Signal	Twin service uses SignalPublicId and synchronized signal catalogue.

This is not weaker discipline. It is different discipline. The database can no longer enforce all relationships with a local foreign key, so the architecture must enforce them through contracts, idempotency, validation, reconciliation, and audit.

PASSPORT RULE AFTER EXTRACTION

A service boundary does not remove referential integrity concerns. It moves them from the database engine into contracts, validation, and operational checks.

The ownership table behind the contracts

Every public contract should be traceable back to the service that owns the truth. The table below is the minimum governance ledger a team should keep beside its schema, event catalog, or API documentation.

OWNER	TRUTH	PUBLIC ID	PUBLIC CONTRACT	COPY ALLOWED IN
AlarmManagement	Alarm flood detected	AlarmFloodId	AlarmFloodDetected.v1	RootCause evidence, Reporting projection, Audit evidence.
RootCause	Verdict issued	VerdictId	RootCauseVerdictIssued.v1	Compliance snapshot, Reporting projection, Audit evidence.
RootCause	Verdict corrected	CorrectionId	RootCauseVerdictCorrected.v1	Compliance correction workflow, Reporting projection, Audit evidence.
Compliance	Review state	ReviewId	ComplianceReviewStatusChange.v1	Reporting projection, Audit evidence.
Audit	Evidence recorded	AuditEntryId	AuditEntryRecorded.v1	Diagnostics projection only.
Reporting	Report snapshot generated	ReportSnapshotId	ReportGenerated.v1	Download/export workflow, not domain decisions.

CONTRACT READINESS

If a contract cannot name its owner, public id, and allowed consumers, it is not ready to become a stable microservice contract.

A conservative extraction sequence

The ownership map also suggests an extraction order. NEXUS-1 should not begin by tearing out the deepest foundational data. It should begin where ownership is clear, public facts are meaningful, and the operational price is justified.

STAGE	CANDIDATE	WHY THIS ORDER
0	Keep modular monolith with clear modules	The system still benefits from one transaction while boundaries are being proven.
1	Reporting projections / BFF read side	Read-only extraction teaches composition and freshness without moving decisions first.
2	Audit service	Append-only evidence is a strong service candidate if correlation and payload rules are stable.
3	Compliance workflow	Review state has clear ownership and consumes RootCause verdicts.
4	RootCause service	High-value boundary; requires mature AlarmFloodDetected and Verdict contracts.
5	AlarmManagement and Instrumentation	Foundational and high-volume; extract only after event and operational discipline are proven.
6	DigitalTwin and RL policy services	Useful later, but they depend on stable references, signals, and advisory/control separation.

HONEST BOUNDARY

This is an architectural recommendation for the demonstrator, not a universal rule. The real sequence depends on team ownership, operational maturity, and which deployment pressures are real.

Ownership-map anti-patterns

The map is useful only if it prevents mistakes. These are the warning signs that the system is drifting back toward a distributed monolith.

ANTI-PATTERN	WHY IT BREAKS THE MAP
Reporting becomes the place where truth is reconciled.	Reporting is a projection owner, not the judge of disagreement.
Compliance updates RootCause verdict fields after review.	A review outcome is not a verdict correction.
Audit events drive workflow because they are convenient.	Audit is evidence; process managers and domain events drive workflow.
Services share lookup ids with private integer keys.	Public codes and versions should cross the boundary, not local surrogate keys.
Consumers depend on private event payload fields.	The public contract is the contract; private state must not leak.
A service uses another service database for one quick query.	One quick query is how the shared database returns through the side door.

DISTRIBUTED-MONOLITH SMELL

The most dangerous microservice mistake is not network failure. It is pretending ownership exists while every service still knows every other service's private tables.

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- Which NEXUS-1 service owns AlarmEvent, RootCauseVerdict, ComplianceReview, AuditEntry, and ReportingProjection?
- Why is the data ownership map not the same thing as an ER diagram?
- What is the difference between owner, publisher, consumer, evidence holder, and projector?
- Why does RootCauseVerdictIssued.v1 remain the canonical public verdict event?
- How do cross-schema foreign keys become public identities and snapshots after service extraction?
- Which services are safer early extraction candidates, and which should wait?
- What anti-patterns would turn NEXUS-1 into a distributed monolith?

PART III CLOSE

Part III is now complete. The book has moved from the temptation of services to a defensible service-data map. Part IV will change the question: once ownership is clear, what should synchronous APIs expose, and where do they become a trap?

Service ownership becomes real when the map says who may change truth - and everyone else is forced to ask, remember, or project it honestly.

PART IV

APIs and Synchronous Communication

Part IV starts after ownership is clear. Once a service owns its data and language, the next question is how other services and the operator console are allowed to ask for something without entering the private model.

Synchronous APIs are useful, familiar, and dangerous. They make a distributed system feel like a local method call, which is exactly why they can hide coupling. This part keeps the NEXUS-1 rule visible: expose contracts, not aggregates; expose intent, not tables; expose stable public language, not internal convenience.

CHAPTERS 18-24

- 18 Designing Service APIs Without Leaking the Domain
- 19 Commands, Queries, and Public Contracts
- 20 REST APIs Between Services
- 21 Versioning Public APIs
- 22 When Synchronous Calls Are Acceptable
- 23 When Synchronous Calls Become a Trap
- 24 API Gateway and Backend-for-Frontend for the Operator Console

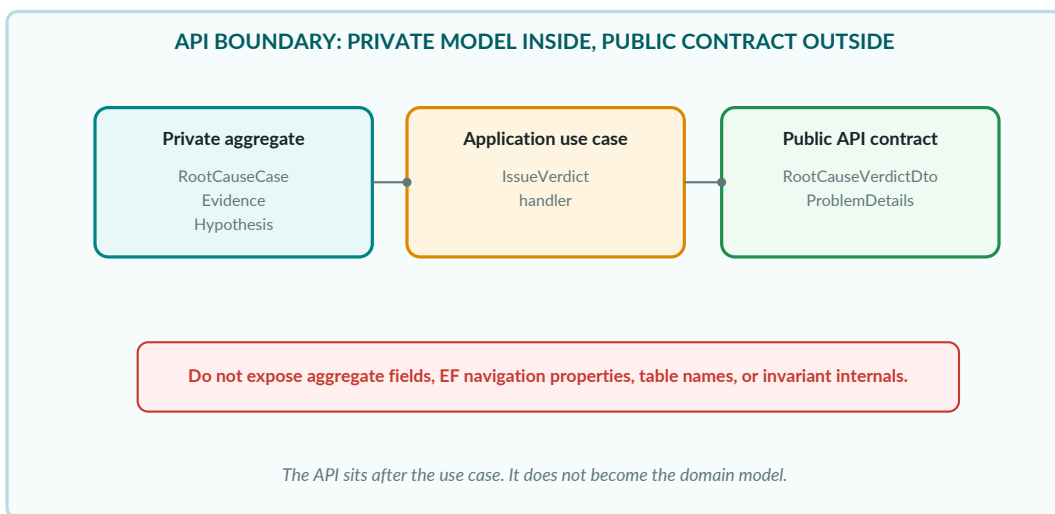
PART IV BOUNDARY

Part IV does not claim that every service must call every other service synchronously. It does the opposite: it teaches how to recognize the few synchronous calls that are justified, and how to avoid turning APIs into a distributed version of private table access.

PART IV - APIS AND SYNCHRONOUS COMMUNICATION

18. Designing Service APIs Without Leaking the Domain

A service API is a promise made to the outside world. It should expose the language other services are allowed to depend on - not the aggregate, table, EF entity, private status enum, or shortcut the team happens to use today.



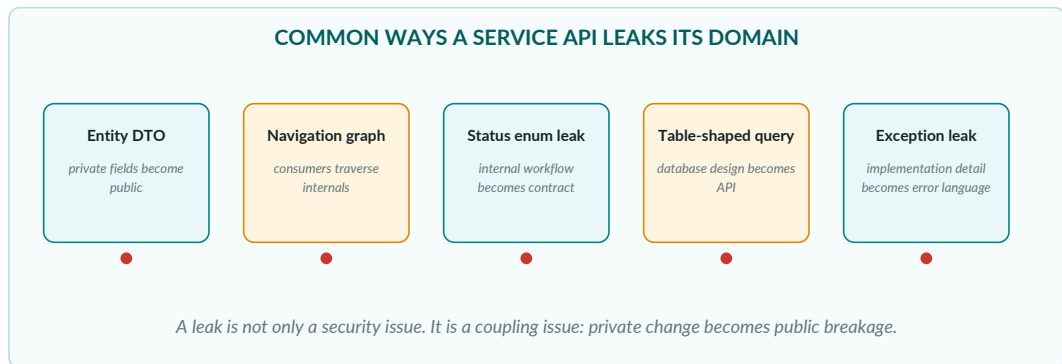
API RULE

The first rule of a microservice API is not HTTP, REST, OpenAPI, or JSON. The first rule is boundary discipline: the public contract must be smaller, more stable, and more intentional than the domain model behind it.

What it means to leak the domain

A domain leak happens when a consumer can see, rely on, or be broken by a private modelling decision. The leak may look harmless: the API returns an entity, serializes a navigation property, exposes an internal enum, or uses the database column name because it was convenient.

The damage appears later. A RootCauseCase changes how it stores hypotheses, and a reporting consumer breaks. An internal case state is renamed, and a compliance client treats the new value as unknown. A private collection becomes part of the response, and another team builds a workflow around it. The service boundary existed on a deployment diagram, but not in the contract.



LEAKY BOUNDARY

A leaky API turns private refactoring into public migration. That is the opposite of independent service evolution.

Internal model vs public contract

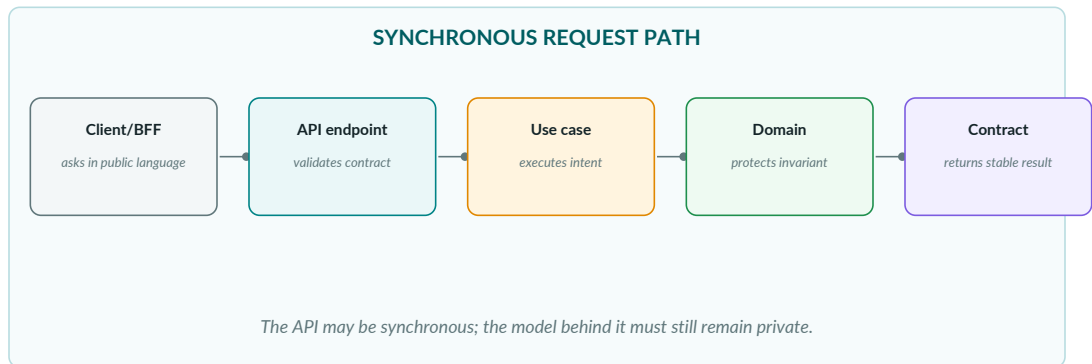
The internal model is allowed to be rich. It may contain invariants, domain events, private constructors, value objects, collections, lifecycle state, and methods that protect rules. The public contract should be much narrower. It should say only what a consumer is allowed to know and rely on.

INTERNAL MODEL DETAIL	PUBLIC CONTRACT VERSION	REASON
RootCauseCase aggregate	RootCauseVerdictIssuedV1 response or event payload	Consumers need the verdict fact, not the aggregate.
Evidence collection and hypothesis objects	EvidenceSummary items with public ids and reasons	The internal reasoning structure can evolve.
EF Core RowVersion	ETag or version token when concurrency is public	Do not expose persistence mechanics directly.
Internal CaseStatus enum	Stable public status vocabulary	Workflow internals should not become accidental contracts.
Navigation properties	Links or public ids	Consumers should not traverse private object graphs.

In NEXUS-1 this distinction is especially important because the domain language is precise. A verdict is not a case closure. A recommendation is not a command. An audit entry is evidence, not workflow. The API must preserve those distinctions instead of flattening them into convenient JSON.

The API belongs after the use case

A common mistake is to put the API controller too close to the entity. The controller reads an entity from the database, serializes it, and calls the result a service boundary. That is not a boundary; it is remote table access with HTTP syntax.



The safer flow is: endpoint receives a public request, application use case performs the action or query, the domain model protects the rule, and a deliberate contract is returned. The controller does not expose entities and does not become a place where domain decisions are made.

CONTROLLER DISCIPLINE

Controllers should translate at the boundary. They should not own policy, rebuild aggregates, or leak private state.

NEXUS-1 example: RootCause verdict API

The RootCause service owns the verdict. Other services may ask for a verdict, subscribe to a verdict event, or hold a snapshot of a verdict. They should not read the RootCause database, and they should not receive the private RootCauseCase aggregate as JSON.

CONSUMER NEED	BAD API	BETTER API
Compliance needs to open a review.	GET /rootcause/cases/{id} returns full case graph.	GET /verdicts/{publicid} returns verdict summary and evidence references.
Reporting needs status.	GET /rootcause/tables/case rows.	GET /rootcause/cases/{id}/public-status.
Operator console needs explanation.	GET /rootcause/entities/{id}?include=evidence,hypotheses.	GET /rootcause/verdicts/{id}/explanation.
Audit needs trace.	RootCause writes directly to Audit tables.	RootCause emits event id, payload hash, actor, timestamp.

NEXUS-1 API SHAPE

The contract is shaped by consumer need, not by the private shape of the aggregate. Consumers get the fact they need and the trace they can audit.

A public contract is a deliberately boring type

Public API types should be boring on purpose. They should be simple, versioned, serializable, and independent of the domain entity. Their job is not to express every internal rule. Their job is to preserve a stable agreement between services.

```
public sealed record RootCauseVerdictResponseV1(
    Guid VerdictId,
    Guid CaseId,
    string UnitCode,
    string VerdictCode,
    string VerdictText,
    decimal Confidence,
    IReadOnlyList<EvidenceSummaryV1> Evidence,
    DateTime IssuedAtUtc,
    string IssuedBy,
    string CorrelationId);

public sealed record EvidenceSummaryV1(
    Guid EvidenceId,
    string EvidenceType,
    string SourceReference,
    string Summary);
```

This type deliberately does not expose `RootCauseCase`, `Hypothesis`, EF navigation properties, row versions, internal collections, or database keys. It exposes public identity, public language, the result, the evidence summary, and the trace.

CONTRACT OBJECT

A DTO is not a weak domain model. It is a boundary object. Its weakness is its strength: it gives consumers the agreed shape and nothing more.

Endpoint names should expose intent

In service design, endpoint names are language. They teach consumers what the service believes exists. A table-shaped endpoint says, "here are my rows". An intent-shaped endpoint says, "here is the fact or action this boundary supports".

INTENT	PREFERRED SHAPE	AVOID
Issue a verdict	POST /root-cause/cases/{caseId}/verdicts	PUT /rootcausecase/{id}/status
Read a verdict	GET /root-cause/verdicts/{verdictId}	GET /rootcausecases/{id}?include=all
Correct a verdict	POST /root-cause/verdicts/{verdictId}/corrections	PATCH /rootcausecase/{id} with arbitrary fields
List case summaries	GET /root-cause/cases?unit={unitCode}&state=open	GET /RootCauseCase table rows
Explain a verdict	GET /root-cause/verdicts/{id}/explanation	GET private evidence/hypothesis graph by default

The goal is not to worship REST naming rules. The goal is to make the public surface reflect the ubiquitous language: case, verdict, correction, evidence summary, explanation, and correlation.

Version the contract, not the class name accident

From Context to Flow already established the canonical public event name `RootCauseVerdictIssued.v1` and the public C# contract `RootCauseVerdictIssuedV1`. The same discipline applies to synchronous API contracts. Versioning is not decorative; it is the promise that consumers will not break because the provider refactored its internals.

CHANGE	NEW VERSION NEEDED?	WHY
Add optional display field	Usually no	Old consumers can ignore the field.
Rename <code>VerdictCode</code> to <code>CauseCode</code>	Yes	Existing consumers depend on the name and meaning.
Change confidence scale from 0-1 to 0-100	Yes	Same field would mean a different value.
Add internal hypothesis ranking	No, unless public	Private reasoning can evolve behind the contract.
Remove evidence summary	Yes	Consumer capability is removed.

VERSIONING RULE

Version public meaning. Do not version every private refactoring. The boundary exists so internal change can happen without external breakage.

Errors are part of the public contract

A service API does not only return success. It also returns absence, conflict, invalid intent, unavailable dependency, stale projection, and forbidden access. If those error shapes leak exceptions, SQL errors, stack traces, or internal enum values, the boundary has failed.

SITUATION	PUBLIC RESPONSE IDEA	DO NOT EXPOSE
Verdict not found	404 with public problem code <code>verdict_not_found</code>	Table name or repository exception.
Case not ready for verdict	409 conflict with rule code <code>evidence_required</code>	Private invariant method or stack trace.
Consumer asks for stale read model	200 with freshness metadata or 202 pending	Pretend the projection is immediately true.
Downstream unavailable	503 with retry guidance and correlation id	Broker/client library exception.
Unauthorized decision attempt	403 with policy code	Internal role table structure.

The error contract matters for operators too. A screen that shows "verdict not yet projected" is safer than a screen that silently displays an old value as if it were current.

HONEST ABSENCE

A well-designed API tells the truth about absence and delay. It does not force the client to guess whether nothing exists, something failed, or the read model has not caught up.

The public surface of a service

A service API should have a small public surface. Each endpoint should be traceable to a use case, a public contract, an owner, and a known reason for being synchronous.

API SURFACE ITEM	QUESTION IT MUST ANSWER
Public request type	What intent or query is the consumer allowed to express?
Public response type	What fact, status, or view is the service willing to promise?
Problem response	How does the service express absence, conflict, and delay?
Correlation id	How will the call be followed through logs, audit, outbox, and UI?
Freshness metadata	Is the response source-of-truth, projection, cache, or snapshot?
Authorization policy	Who is allowed to ask, decide, or observe?
Version	What public meaning is being preserved?

ENDPOINT READINESS TEST

If an endpoint cannot name its use case, owner, contract, freshness, error shape, and version, it is not ready to be a microservice boundary.

API anti-patterns in NEXUS-1

The following mistakes are especially tempting in a system like NEXUS-1 because the underlying model is rich and the dashboard wants a complete picture.

ANTI-PATTERN	WHY IT IS DANGEROUS
Expose everything because the operator console needs it.	The console needs a composed view, not every private domain object.
Return EF entities from controllers.	Persistence shape becomes public API shape.
Let Reporting call private RootCause endpoints.	Reporting becomes dependent on internal reasoning structure.
Use synchronous calls for every cross-service step.	The flow becomes slow, fragile, and harder to recover.
Hide eventual consistency from the UI.	Operators cannot distinguish fresh truth from delayed projection.
Expose audit as a workflow API.	Evidence capture becomes confused with business process.

BOUNDARY CURE

The cure is not fewer APIs. The cure is more disciplined APIs: smaller contracts, clearer ownership, explicit freshness, and no private model leakage.

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- What is the difference between an internal domain model and a public API contract?
- Why is returning an EF entity from a service endpoint a boundary leak?
- Why should RootCause expose a verdict contract rather than a full RootCauseCase graph?
- What does it mean to version public meaning instead of private refactoring?
- How should an API express not-found, conflict, stale projection, and downstream unavailability?
- Why must the operator console receive composed views without gaining access to every private model?
- Which API details help preserve traceability across services?

NEXT STEP

Chapter 18 establishes the API boundary. Chapter 19 will sharpen the public language further by separating commands, queries, and public contracts.

A service API should be small enough to protect the model and honest enough to let other services depend on it.

PART IV - APIS AND SYNCHRONOUS COMMUNICATION

19. Commands, Queries, and Public Contracts

A service boundary is easier to trust when every public shape says what it is: a request to change truth, a request to read truth, or a contract other services may depend on.

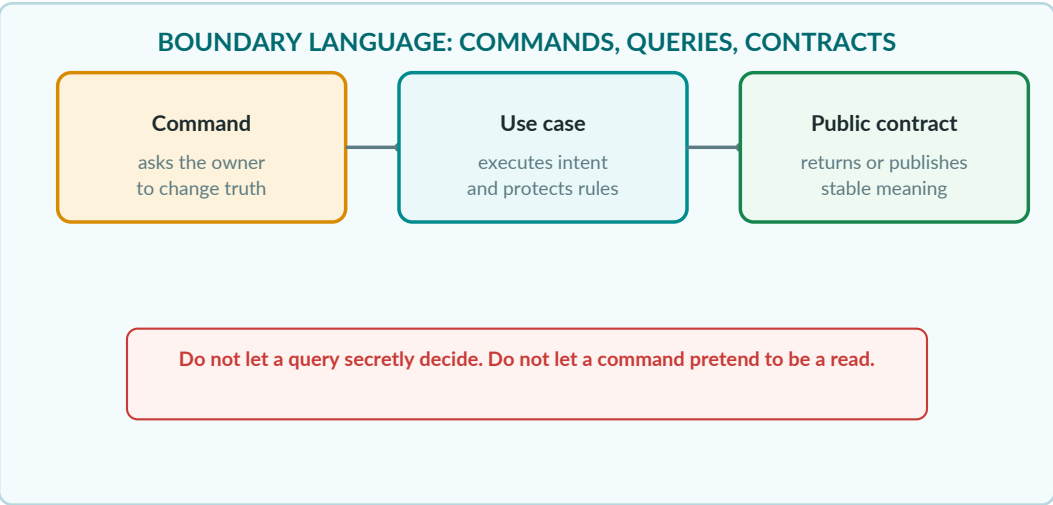


Figure 19.1 - Commands, queries, and contracts are separate boundary languages. A service may use HTTP, but the real design is the intention and ownership behind the call.

BOUNDARY RULE

A command expresses intent to change owned state. A query asks for a view of state. A public contract is the stable language shared beyond the service boundary. Mixing them makes a distributed system harder to reason about before a single message is sent.

Why the distinction matters

In a modular monolith the damage caused by confused naming is often local. A method called `GetAndUpdateStatus` is ugly, but the same process, database transaction, and code review can still reveal what it really does. In a microservice architecture the same confusion becomes a public lie. Other services must decide whether a call can be retried, cached, audited, authorized, or replayed. They can do that only when the boundary language is honest.

Commands, queries, and contracts therefore are not only programming categories. They are reliability categories. They tell consumers what may change, what may be stale, what can be cached, what must be idempotent, and what deserves an audit trail.

SHAPE	PLAIN MEANING	RUNTIME CONSEQUENCE
Command	A request for the owner to perform an intention.	May change state; needs authorization, validation, idempotency, and audit.
Query	A request for information.	Should not change source-of-truth state; may use projections, cache, freshness metadata.
Public contract	A stable request, response, or event shape.	Can be depended on by other services; must be versioned and protected.
Internal model	The private domain shape behind the boundary.	Can evolve without forcing consumers to migrate.

LANGUAGE BEFORE PROTOCOL

HTTP verbs, routing conventions, and OpenAPI descriptions help only after the service has named its intentions correctly. A bad contract documented beautifully is still a bad contract.

Commands: requests to change owned truth

A command does not describe a database update. It describes an intention in the public language of the service owner. The RootCause service may accept IssueVerdict. It should not accept SetCaseStatus or UpdateRootCauseCaseRow, because those names expose implementation mechanics rather than domain intention.

A good command is narrow. It contains the data needed to evaluate the intention, not a complete copy of the aggregate. The service that receives the command loads its private model, checks its invariants, and decides whether the requested change is allowed.

```
public sealed record IssueRootCauseVerdictCommandV1(
    Guid CaseId,
    string VerdictCode,
    string VerdictText,
    decimal Confidence,
    IReadOnlyList<Guid> EvidenceIds,
    string IssuedBy,
    string CorrelationId,
    string IdempotencyKey);
```

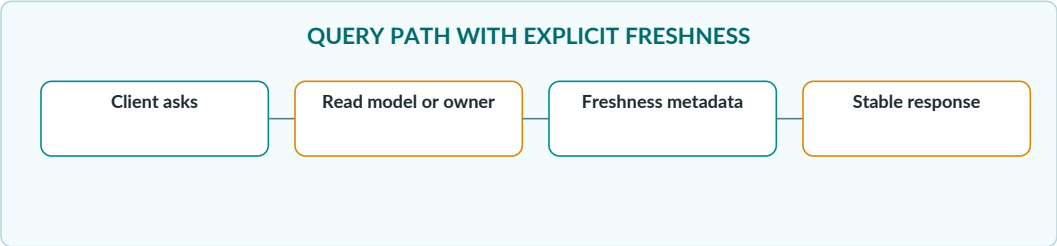
FIELD	WHY IT BELONGS
Caseld	Identifies the public case whose owner must decide.
VerdictCode / VerdictText	States the intended decision in public language.
Evidencelds	References evidence; does not embed private Evidence objects.
CorrelationId	Connects API, logs, audit, outbox, and operator view.
IdempotencyKey	Allows safe retry when the caller does not know whether the command was processed.

COMMAND DISCIPLINE

A command may be rejected even when its JSON is valid. Valid syntax is not valid intention. The domain still decides.

Queries: reading without pretending to own

A query asks for a view. It should not change source-of-truth state and should not smuggle decisions into a read path. This sounds obvious until the operator console wants one convenient endpoint that returns status, updates acknowledgement, marks a notification as seen, and recalculates a recommendation. That endpoint is not a query; it is a hidden workflow.



QUERY NEED	ACCEPTABLE SHAPE	BOUNDARY WARNING
Operator console needs case status.	GET /root-cause/cases/{id}/summary	Do not return the full aggregate graph.
Reporting needs a list of verdicts.	GET /root-cause/verdicts?from=&to=	Do not read RootCause tables from Reporting.
Compliance needs the verdict fact.	GET /root-cause/verdicts/{id}	Do not infer verdict from a status enum.
Audit needs trace.	GET /traces/{correlationId}	Do not turn audit evidence into workflow state.

QUERY HONESTY

A query response should say whether it is source-of-truth, projection, snapshot, or cached view. A stale but labelled answer is safer than a stale answer pretending to be immediate truth.

Public contracts are not internal DTOs

A common mistake is to take a DTO originally created for an MVC screen, publish it in OpenAPI, and call it a microservice contract. That DTO may contain exactly what one screen needed on one day, but it has not necessarily been designed as a durable promise to other services.

A public contract should be deliberately boring, but not accidental. It should have stable names, stable meaning, explicit versioning, trace fields where needed, and no dependency on EF Core entities, domain navigation properties, private enum names, or table-shaped collections.

TYPE	PURPOSE	SERVICE-BOUNDARY RULE
Screen DTO	Optimized for one UI view.	May change when UI layout changes; not automatically reusable.
Domain entity	Protects business rules internally.	Should not be serialized as a public API shape.
Public request contract	Represents allowed consumer intent.	Must be validated, versioned, and authorized.
Public response contract	Represents a promised fact or view.	Must declare identity, meaning, and freshness where relevant.
Integration event contract	Represents something that already happened.	Must be append-friendly and safe for asynchronous consumers.

CONTRACT TEST

Ask this before publishing a type: would I be comfortable supporting this shape after the private model changes? If the answer is no, the type is not a public contract yet.

NEXUS-1 contract set for the alarm-to-compliance flow

The recurring NEXUS-1 flow gives a practical contract set. Some shapes are commands, some are queries, and some are events. Treating all of them as generic messages would erase the difference between intention, observation, and historical fact.

NAME	KIND	OWNER AND MEANING
AlarmFloodDetected.v1	Integration event	AlarmManagement publishes that a flood was detected.
OpenRootCauseCase	Command	RootCause is asked to open a case for investigation.
RootCauseCaseSummaryV1	Query response	UI or Reporting reads public case progress.
IssueRootCauseVerdictCommandV1	Command	RootCause is asked to issue a verdict.
RootCauseVerdictIssued.v1	Integration event	RootCause publishes that the verdict is now a fact.
RootCauseVerdictIssuedV1	C# public contract	The versioned shape used by consumers.
RequestComplianceReview	Command	Compliance is asked to review a verdict.
RootCauseVerdictCorrected.v1	Integration event	A correction occurred; consumers must update projections.

TERMINOLOGY DISCIPLINE

RootCauseCaseClosed is not the main public fact. The public fact is the verdict. AuditEntry is evidence, not workflow. A recommendation is advice, not a command.

A versioned contract shape in C#

The public C# contract should make the public meaning obvious from the type name. Versioning belongs in the contract because consumers depend on the shape, not on the private class that produced it.

```
public sealed record RootCauseVerdictIssuedV1(  
    Guid EventId,  
    Guid VerdictId,  
    Guid CaseId,  
    string UnitCode,  
    string VerdictCode,  
    string VerdictText,  
    decimal Confidence,  
    IReadOnlyList<EvidenceSummaryV1> Evidence,  
    DateTime IssuedAtUtc,  
    string IssuedBy,  
    string CorrelationId,  
    string CausationId,  
    string SchemaVersion = "1.0");  
  
public sealed record EvidenceSummaryV1(  
    Guid EvidenceId,  
    string EvidenceType,  
    string SourceReference,  
    string Summary);
```

This contract is not the RootCauseCase aggregate. It is the public fact that a verdict has been issued, together with enough evidence summary and trace information for downstream consumers to behave responsibly.

VERSIONING RULE

Version the public meaning that consumers depend on. Do not create a new version merely because an internal field, entity, or table was renamed.

Validation is not the same as invariants

Public contracts need validation, but validation at the boundary is not the same as domain correctness. The API can check whether fields exist, values have acceptable formats, and the caller is allowed to ask. The aggregate still decides whether the command is legal in the current state.

LAYER	WHAT IT CHECKS	WHAT IT MUST NOT PRETEND
API contract validation	Required fields, formats, ranges, version, authorization.	It does not prove the business decision is allowed.
Application use case	Idempotency, loading, transaction boundary, orchestration.	It should not duplicate aggregate invariants.
Aggregate invariant	Current-state rule such as evidence required before verdict.	It should not know HTTP, JSON, broker, or endpoint names.
Persistence constraint	Unique keys, FK integrity, concurrency tokens.	It is not the whole domain model.

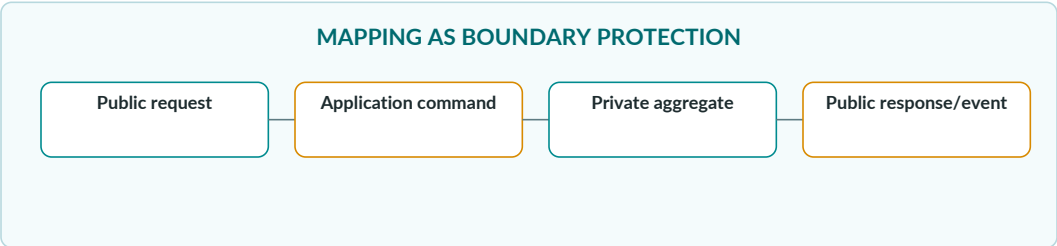
```
public Result Handle(IssueRootCauseVerdictCommandV1 command)
{
    ValidatePublicContract(command);
    var rootCauseCase = repository.Load(command.CaseId);
    rootCauseCase.IssueVerdict(
        command.VerdictCode,
        command.VerdictText,
        command.EvidenceIds,
        command.IssuedBy);
    return Result.Accepted(rootCauseCase.PublicVerdictId);
}
```

INVARIANT BOUNDARY

The contract carries intention. The use case coordinates. The aggregate decides. Keeping those roles separate is what lets the public API stay stable while the model remains rich.

Mapping protects both sides of the boundary

Mapping is sometimes treated as boring plumbing. At a service boundary it is architectural protection. The mapper is where private terms are translated into public terms and where the service refuses to export details that consumers should not depend on.



MAPPING	PURPOSE	AVOID
Request to command	Translate public intent into use-case input.	Do not pass API objects deep into the domain.
Aggregate to response	Expose stable facts and ids.	Do not serialize private collections and navigation graphs.
Aggregate/domain event to integration event	Publish only public meaning.	Do not publish internal domain event objects by accident.
Exception/result to problem response	Return useful public failure language.	Do not leak stack traces or SQL/provider errors.

MAPPING RULE

A mapper is not a place to invent business decisions. It translates between languages. Decisions still belong in the owner: use case, aggregate, policy, or process manager.

Synchronous commands and queries need different promises

The next chapters will examine REST and synchronous calls directly. This chapter prepares that discussion by separating the promises. A synchronous command does not promise that every downstream projection is already updated. A synchronous query does not promise that it owns the fact it displays unless it says so.

PUBLIC PROMISE	MEANING	WARNING
POST command accepted	The owner accepted the intention and will process it inside its consistency boundary.	Downstream projections may still lag.
POST command completed	The owner completed the decision and returns the public result.	Only use when completion belongs in the same boundary.
GET source-of-truth query	The owner returns its own current truth.	May be slower; should be protected by authorization and versioning.
GET projection query	A read model returns a view composed from events or replication.	Must include freshness or lag information when it matters.
GET cache/snapshot query	A cached answer is returned deliberately.	Must not be mistaken for live decision truth.

DO NOT OVER-PROMISE

The safest distributed API is often one that promises less but tells the truth. Accepted, pending, projected, stale, and unavailable are legitimate states, not embarrassments to hide.

NEXUS-1 example: verdict issuance as public language

When RootCause issues a verdict, three different public surfaces may appear around the same business moment. They should not be collapsed into one generic object.

SURFACE	CANONICAL SHAPE	MEANING
Command	IssueRootCauseVerdictCommandV1	A caller asks RootCause to issue a verdict.
Response	RootCauseVerdictResponseV1	RootCause answers with the public result or accepted status.
Integration event	RootCauseVerdictIssued.v1 / RootCauseVerdictIssuedV1	RootCause announces that the verdict is now a fact.
Query response	RootCauseVerdictSummaryV1	Consumers read the public verdict later.
Correction event	RootCauseVerdictCorrected.v1	The public history records that a prior verdict was corrected.

The names are repetitive on purpose. Repetition is cheaper than ambiguity. The reader of a log, an OpenAPI page, an event stream, or an audit trace should know whether they are seeing a request, an answer, or a fact that already happened.

NEXUS-1 NAMING DISCIPLINE

Use VerdictIssued internally when it is a domain event. Use RootCauseVerdictIssued.v1 when it leaves the boundary. Use RootCauseVerdictIssuedV1 when naming the public C# contract. Do not replace this with a vague RootCauseCaseClosed event.

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- What is the difference between a command, a query, and a public contract?
- Why should IssueRootCauseVerdict be a command rather than a table-shaped update?
- Why should a query never hide a workflow side effect?
- What makes a public contract different from a screen DTO or EF entity?
- Why is validation at the API boundary not the same as enforcing a domain invariant?
- What fields help a command survive retries and support traceability?
- Why does RootCauseVerdictIssued.v1 need a stable public shape?
- How can a response tell the truth about source-of-truth, projection, cache, or snapshot freshness?

NEXT STEP

Chapter 20 will move from language to protocol. It will ask how REST APIs between services can support this discipline without becoming remote CRUD over private models.

A service boundary becomes readable when a request says what it wants, a response says what it promises, and an event says what already happened.

PART IV - APIS AND SYNCHRONOUS COMMUNICATION

20. REST APIs Between Services

REST is useful when one service needs a clear, immediate answer from another service. It becomes dangerous when it hides ownership, spreads private models, or turns a workflow into a chain of waiting machines.

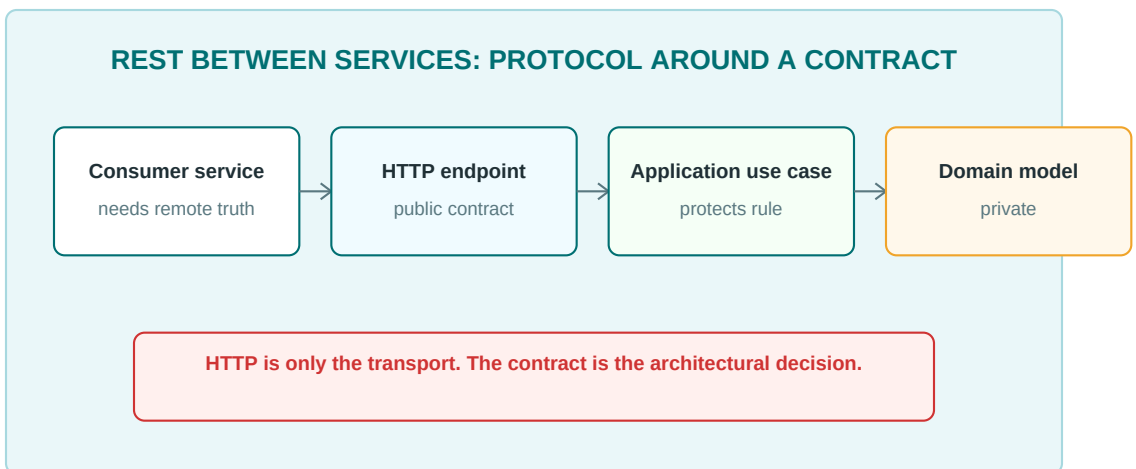


Figure 20.1 - A REST call is not just a route and a verb. It is a public contract around a use case, with the domain model protected behind the boundary.

BOUNDARY RULE

A service API should expose what another service is allowed to ask for, not how the owning service stores, models, or reasons about its private domain. If the HTTP contract mirrors the aggregate, the aggregate has leaked.

The comfortable lie of HTTP

HTTP makes distribution feel simple. A controller has a route. A client sends JSON. A response returns JSON. The first demo works, and the system appears to have become a set of services. But the hard part was never the wire format. The hard part is deciding what the caller is allowed to know, what it is allowed to request, and what happens when the owner cannot answer.

Inside one deployable application, a bad method call can often be found and repaired by one team. Across services, the same bad call becomes a dependency. The caller compiles against it, caches it, retries it, documents it, and builds its own assumptions around it. A careless endpoint therefore becomes an architectural fact.

HTTP surface	What it can express	What it cannot prove by itself
GET /cases/42	A request for a representation.	That the response is source-of-truth, fresh, or safe to cache.
POST /cases	A request to create or start something.	That the caller owns the decision or that retrying is harmless.
PUT /cases/42	A replacement or update convention.	That the whole resource should be writable by the caller.
Status 200	The server produced a response.	That the business operation was accepted, durable, or audited.

PROTOCOL IS NOT OWNERSHIP

REST gives a service a language for request and response. It does not decide who owns truth, who owns the rule, or which data is allowed to cross the boundary. Those decisions must already exist before the route is designed.

A service API is not a controller mirror

A common first mistake is to take the internal application controller and publish it as a service boundary. Every table receives CRUD endpoints. Every entity receives a DTO with the same properties. Every navigation property becomes a nested JSON object. The result looks convenient because it resembles the database and the domain model the team already has. It is also the fastest way to make the private model public.

The owning service must be free to change its aggregate structure without forcing every consumer to migrate. That freedom disappears when outside services depend on internal entity shapes such as `RootCauseCase`, `Evidence`, `Hypothesis`, `AuditEntry`, or `ComplianceFinding` as if those were public contracts.

Boundary shape

```
// Bad boundary: private structure exposed as remote CRUD.  
GET /api/rootcause/rootcausecases/42  
PUT /api/rootcause/rootcausecases/42/evidence/17  
PUT /api/rootcause/rootcausecases/42/hypotheses/9/status  
  
// Better boundary: ask for a named use case or a stable view.  
POST /api/root-cause/cases/open  
POST /api/root-cause/cases/42/evidence  
GET /api/root-cause/cases/42/summary
```

PUBLIC DOES NOT MEAN EVERYTHING

A service may publish a command endpoint, a query endpoint, or a stable read representation. It should not publish its aggregate as a remote object graph that other services learn to manipulate field by field.

Resource language and use-case language

REST literature often begins with resources. That is useful, but service boundaries in a DDD system also need use-case language. A remote caller is not inside the aggregate. It should not be invited to perform small private mutations as if it were. It should ask the owner to perform a meaningful action or return a meaningful representation.

In NEXUS-1, this distinction protects the alarm-to-compliance flow. AlarmManagement may announce that an alarm flood exists. RootCause may open a case and attach evidence. Compliance may request a review. Reporting may ask for a projection. None of those services should reach through HTTP and rearrange another service's private state.

Naive route	Stronger route	Reason
PUT /cases/{id}/verdict	POST /root-cause/cases/{id}/verdicts	A verdict is a decision the owner issues, not a field assignment.
GET /evidence?caseId=42	GET /root-cause/cases/{id}/evidence-summary	The consumer receives a representation, not a private collection.
POST /auditentries	POST /audit/evidence-records	The endpoint names the public purpose, not the table.
GET /compliancefindings	GET /compliance/reviews/{id}/status	The caller asks for review state, not database rows.

ROUTE NAMES ARE MODEL NAMES

A route teaches consumers how to think about the service. If the route language is table-shaped, consumers will treat the service as a remote database. If the route language is use-case-shaped, consumers are reminded that an owner is protecting a rule.

The synchronous call chain in NEXUS-1

The recurring NEXUS-1 flow tempts a designer to make every step call the next one synchronously: AlarmManagement detects a flood and calls RootCause; RootCause issues a verdict and calls Recommendation; Recommendation calls Audit; Audit calls Compliance; Compliance calls Reporting. This is simple to draw and painful to operate.

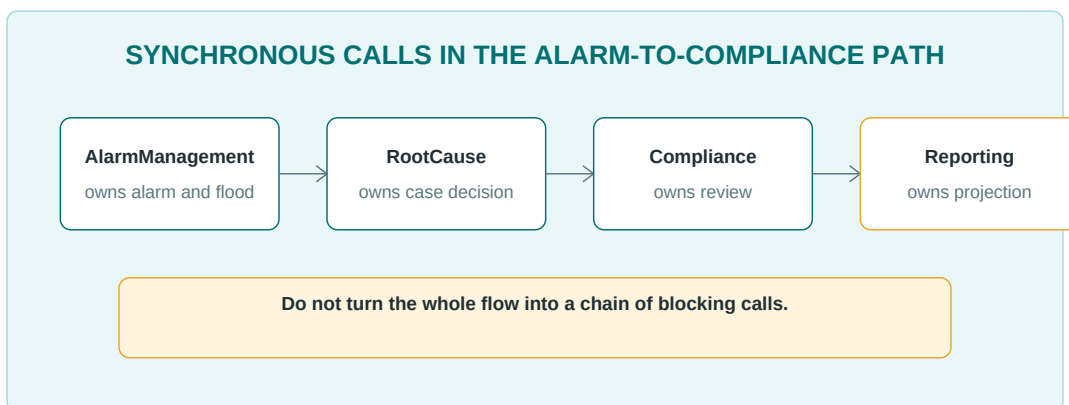


Figure 20.2 - Synchronous calls can be useful at a single boundary. A whole workflow made of blocking service-to-service calls becomes fragile quickly.

A synchronous call is most defensible when the caller cannot proceed without an immediate answer and the called service owns that answer. It is less defensible when the caller only wants to notify another service that something happened. For notification, an integration event is usually a better fit. For immediate validation, lookup, or short query, REST may be acceptable.

ONE CALL CAN BE CLEAN; A CHAIN IS A SMELL

The problem is not that services use HTTP. The problem is when an entire business flow depends on every downstream service being available at the same moment.

What a service may ask and what it may not steal

A REST boundary is a permission boundary. The caller is allowed to ask certain questions and request certain actions. It is not allowed to steal another service's decision, bypass its invariants, or treat its database as remote storage.

Caller wants	Acceptable REST shape	Boundary protection
Open a root-cause case	POST /root-cause/cases/open	RootCause validates the request and creates its own case.
Read case status	GET /root-cause/cases/{id}/status	The caller receives status, not the aggregate internals.
Attach evidence	POST /root-cause/cases/{id}/evidence	RootCause checks whether the case is still open.
Trigger compliance review	POST /compliance/reviews/request	Compliance owns review creation and priority.
Display operator view	GET /operator/cases/{id}/timeline	A BFF or read model composes the view without exposing source models.

THE OWNER STILL DECIDES

Even when a remote service sends a command-like request, the receiving service owns the decision. A request can be rejected, delayed, validated, logged, or translated before it touches the domain model.

A small C# implementation shape

The caller should depend on a typed client or application port, not on scattered `HttpClient` calls. This keeps protocol details out of the use case and makes the boundary visible in tests. The client speaks in public contracts, not in EF entities or domain aggregates.

Typed client boundary

```
public interface IRootCauseServiceClient
{
    Task<RootCauseCaseSummaryResponse> GetCaseSummaryAsync(
        RootCauseCaseId caseId, CancellationToken ct);

    Task<AttachEvidenceResponse> AttachEvidenceAsync(
        AttachEvidenceRequest request, CancellationToken ct);
}

public sealed record AttachEvidenceRequest(
    string CaseId,
    string EvidenceType,
    string SourceService,
    string CorrelationId,
    string Summary);

public sealed record AttachEvidenceResponse(
    string CaseId, string EvidenceId, string Status);
```

Notice the missing types. There is no `RootCauseCase` entity in the request. There is no `DbContext`. There is no navigation graph. The contract says what the caller may request and what the service promises to return.

IMPLEMENTATION DETAIL

In a real .NET backend this client might be implemented with `HttpClientFactory`, Polly resilience policies, `OpenTelemetry` tracing, and generated clients from `OpenAPI`. Those tools matter, but they do not replace the boundary design.

NEXUS-1 REST API sketch

The sketch below is not a complete API catalogue. It is a boundary-language sample. The point is to show which service owns the endpoint and what kind of public shape it exposes. Each endpoint should be judged by whether it protects the service owner's model.

Service	Endpoint	Public meaning
AlarmManagement	POST /alarm-floods/detect	Ask the alarm owner to evaluate a flood condition.
RootCause	POST /root-cause/cases/open	Ask RootCause to open a case from a public incident reference.
RootCause	GET /root-cause/cases/{id}/summary	Read a stable public summary of the case.
Compliance	POST /compliance/reviews/request	Ask Compliance to create or schedule a review.
Reporting	GET /reports/operator-flow/{caseId}	Read a projection composed for display.
Audit	GET /audit/traces/{correlationId}	Read evidence of what happened across the flow.

DO NOT CONFUSE API MAP WITH CALL GRAPH

This table names possible REST surfaces. It does not say every service must call every other service synchronously. Some relationships should be HTTP queries, some should be messages, and some should not exist.

Time budgets, errors, and retry boundaries

Once REST crosses a service boundary, failure becomes ordinary behavior. The network may be slow. The caller may time out. The called service may be healthy but overloaded. A load balancer may return an error before the application even sees the request. The contract therefore needs runtime rules, not only JSON fields.

Concern	Question to answer	NEXUS-1 example
Timeout	How long may the caller wait?	Operator screen status query must return quickly or show stale/unknown.
Retry	Can the request be safely repeated?	AttachEvidence needs idempotency key or duplicate detection.
Failure response	What can the caller do next?	RootCause unavailable should not erase the original alarm flood.
Correlation	How is the call traced?	CorrelationId follows alarm, case, verdict, audit, compliance.
Authorization	Who may ask?	Reporting may read summaries; it may not issue verdicts.

RETRY ONLY WHAT YOU UNDERSTAND

A GET may be retryable. A POST may also be retryable if it carries an idempotency key and the service contract says how duplicates are handled. Without that rule, retries can create the very corruption they were meant to survive.

REST and the operator console

The operator console should not become a direct client of every internal service. If the UI must call `ReactorFleet`, `Instrumentation`, `AlarmManagement`, `RootCause`, `DigitalTwin`, `Compliance`, `Reporting`, and `Audit` to render one screen, the browser has become the integration layer. That is not composition; it is distributed coupling moved to the front end.

For NEXUS-1, the safer shape is a Backend-for-Frontend or operator-facing API that returns screen-specific read models. Internally, that API may query `Reporting` projections, `RootCause` summaries, and `Audit` traces. Externally, the console receives a stable view designed for human use.

BFF/read-model shape

```
// Operator-facing API: shaped for the screen, not for every source service.
GET /operator/cases/{caseId}/overview

public sealed record OperatorCaseOverview(
    string CaseId,
    string CurrentStatus,
    string VerdictSummary,
    string RecommendationState,
    string ComplianceReviewState,
    string AuditTraceId);
```

UI COMPOSITION IS STILL ARCHITECTURE

A screen that needs data from many services should usually read a projection or a BFF response. Letting the browser orchestrate service calls makes latency, authorization, and partial failure visible in the worst possible place.

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- Why is a REST endpoint not automatically a good service boundary?
- What is the danger of exposing entity-shaped CRUD across services?
- When is a synchronous service call acceptable?
- Why is a chain of blocking calls dangerous in the alarm-to-compliance flow?
- What is the difference between a route map and a call graph?
- Why should a typed client depend on public contracts rather than domain entities?
- How do timeouts, retries, idempotency, authorization, and correlation become part of the API design?
- Why should the operator console usually read a projection or BFF response?

NEXT STEP

Chapter 21 will move from endpoint shape to contract survival. It will ask how public APIs evolve when other services already depend on them, and why versioning is not a cosmetic suffix but a promise to consumers.

A REST API between services is healthy only when the contract is smaller than the domain model and clearer than the transport.

PART IV - APIS AND SYNCHRONOUS COMMUNICATION

21. Versioning Public APIs

A public API becomes part of another service's architecture the moment that service depends on it. Versioning is the discipline that lets a contract evolve without pretending every consumer can change at the same instant.

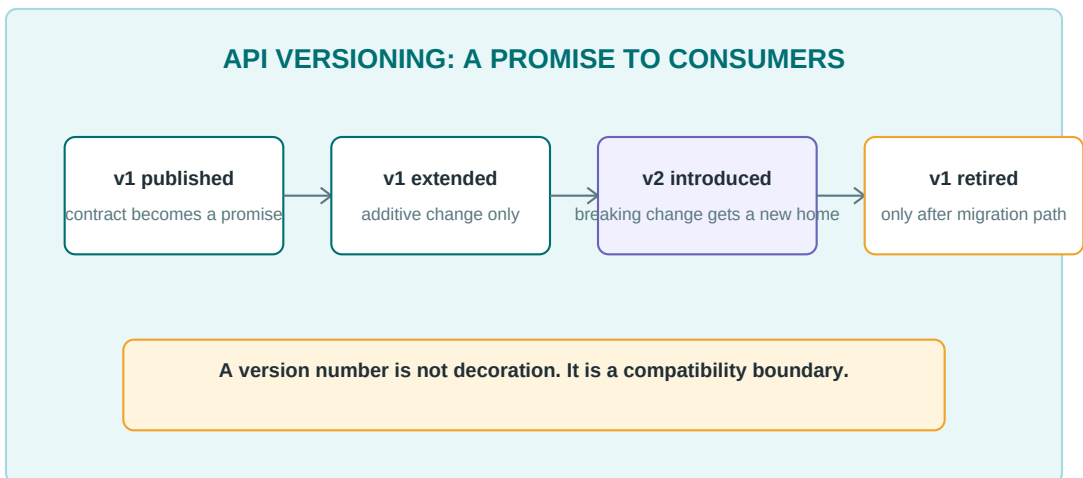


Figure 21.1 - Versioning is a consumer-protection mechanism. It lets a contract change without forcing every caller, listener, and client package to move in lockstep.

VERSIONING RULE

Do not version an API because the route looks tidy. Version it because consumers need a stable promise. Once a public contract is consumed, changing its meaning is no longer a private refactor.

Why versioning is not cosmetic

Inside one codebase, a method signature can be changed and every compilation error points to the caller. Across services, there is no single compiler protecting the whole system. One service may already be deployed. Another may be pinned to an older client. A third may store your response shape in a projection. A route change that feels small to the provider can be a production break for the consumer.

This is why versioning belongs to architecture, not decoration. A version names the compatibility promise a provider makes to its consumers. The promise may be explicit in a URL, a header, a media type, an event name, or a client package. The location matters less than the discipline behind it.

Provider thinks	Consumer experiences	Architectural lesson
We renamed a field.	Deserializer fails or maps null.	A public name is part of the contract.
We changed an enum meaning.	Old business logic makes a wrong decision.	Meaning is versioned, not only shape.
We removed a route.	Deployed consumer cannot complete its workflow.	Retirement needs a migration window.
We added a required property.	Old client cannot send a valid request.	Required input is a breaking change.

PUBLIC MEANS BORING ON PURPOSE

A private model may improve quickly. A public contract must evolve slowly, visibly, and with migration paths. That boring discipline is what lets independent deployment remain honest.

The NEXUS-1 contract names must stay disciplined

From Context to Flow already established a naming discipline that this book should preserve. The internal domain event inside the RootCause context is VerdictIssued. The public integration event is RootCauseVerdictIssued.v1. The C# public contract type is RootCauseVerdictIssuedV1. Those names are not interchangeable.

Name	Kind	Versioning meaning
VerdictIssued	Internal domain event	Private to the RootCause model; not a public contract.
RootCauseVerdictIssued.v1	Public integration event	Versioned promise consumed outside the owning context.
RootCauseVerdictIssuedV1	C# contract type	Serializable public shape; stable for consumers.
RootCauseVerdictCorrected.v1	Related public event	Separate business meaning; not a silent mutation of the original event.
RootCauseCaseClosed	Rejected as canonical	Closing is not the important public decision; verdict issuance is.

LANGUAGE IS PART OF THE API

If the event name changes from issued to published to closed without discipline, consumers cannot know what happened. Versioning protects payload shape; language discipline protects meaning.

What counts as a breaking change?

A change is breaking when an existing correct consumer can no longer keep working without modification, or worse, keeps working while misunderstanding the meaning. Some changes break compilation, some break serialization, and some break trust silently.

Change	Usually safe?	Why
Add optional response field	Yes	Old consumers ignore unknown data if serialization is tolerant.
Add required request field	No	Old consumers cannot create a valid request.
Rename field	No	The old public name disappears.
Change field meaning	No	The shape survives while the interpretation breaks.
Add enum value	Risky	Old consumers may treat unknown value as failure or default.
Remove status code	No	Consumer error handling may depend on it.
Add new endpoint	Yes	Existing consumers are not forced to use it.

THE SILENT BREAK IS THE MOST DANGEROUS

A deserialization failure is visible. A field with the same name and a different meaning can pass every technical check while corrupting a business decision.

HTTP API versioning choices

There is no single perfect place to put the version. A URL version is obvious and easy to route. A header version keeps resource names clean but hides compatibility in metadata. A media-type version is precise but heavier. The important thing is not fashion; it is consistency, documentation, and testability.

Strategy	Example	Trade-off
URL segment	/api/v1/root-cause/cases/{id}	Obvious and easy to operate; version becomes part of route structure.
Query parameter	/cases/{id}?api-version=1	Easy to add; often feels less canonical for public APIs.
Header	Api-Version: 1	Keeps URL clean; easier to miss in logs and manual testing.
Media type	application/vnd.nexus.rootcause.v1+json	Precise contract negotiation; more complex for simple teams.
Separate endpoint	/root-cause-verdicts/v1	Clear for contract-shaped resources; can fragment the API surface.

PICK ONE PRIMARY RULE

A platform can support more than one mechanism internally, but a book, team, and API catalogue should teach one primary convention. Mixed conventions create accidental complexity before the first business feature ships.

Event versions are not the same as API versions

REST APIs and integration events evolve differently. A REST caller is present at the moment of the request. An event consumer may process the message later, replay it from a log, or store it forever in a projection. That makes event versioning especially strict. The event name, payload, schema, topic, and serializer settings all become part of the historical record.



Figure 21.2 - A public contract may appear as an HTTP API, an integration event, or a client package. Versioning must keep these surfaces aligned.

EVENTS HAVE MEMORY

An API response disappears after the call. An event may be replayed months later. Do not mutate the meaning of `RootCauseVerdictIssued.v1` after it has entered audit, reporting, or compliance projections.

C# contract shape: v1 remains v1

A public contract type should be small, explicit, and boring. It should not inherit from the domain entity and should not carry navigation properties. If v2 is needed, create a v2 type. Do not silently modify v1 and hope consumers notice.

Versioned public contract types

```
public sealed record RootCauseVerdictIssuedV1(
    string EventId,
    string CaseId,
    string UnitId,
    string VerdictCode,
    string VerdictSummary,
    decimal Confidence,
    DateTimeOffset IssuedAtUtc,
    string CorrelationId);

public sealed record RootCauseVerdictIssuedV2(
    string EventId,
    string CaseId,
    string UnitId,
    string VerdictCode,
    string VerdictSummary,
    decimal Confidence,
    string EvidenceDigest,
    DateTimeOffset IssuedAtUtc,
    string CorrelationId);
```

The example adds EvidenceDigest in v2. That may be useful for compliance or reporting, but it does not justify changing the v1 meaning. Existing consumers that understand RootCauseVerdictIssuedV1 should continue to receive the shape they were promised.

Compatibility tests are part of the contract

A public API version should be tested as a contract, not merely as controller code. The provider should prove that v1 still serializes and deserializes in the shape consumers expect. Consumers should prove that they do not accidentally depend on private fields. For important flows, a consumer-driven contract test is often more valuable than another isolated unit test.

Test type	Protects	NEXUS-1 use
Serialization snapshot	Field names and JSON shape	RootCauseVerdictIssuedV1 still emits the same public fields.
Consumer contract test	Real consumer expectations	Compliance can still request review from the published verdict.
Backward-read test	Old messages remain readable	Reporting can replay RootCauseVerdictIssued.v1 from history.
Negative compatibility test	Breaking changes are visible	Removing Caseld fails the contract suite.
Semantic test	Meaning remains stable	Confidence still means calibrated verdict confidence, not UI severity.

TEST MEANING, NOT ONLY JSON

Schema compatibility is necessary but not sufficient. The same field names can carry a different meaning. For NEXUS-1, meaning errors matter because audit, compliance, and reporting depend on language discipline.

Deprecation without surprise

A version is not forever, but retirement must be deliberate. Consumers need notice, documentation, telemetry, and a migration path. The provider should know which consumers still call v1 before turning it off. Guessing is not operational discipline.

Step	Provider action	Consumer protection
Announce	Publish v2 and migration notes.	Consumers know what changed and why.
Run side by side	Keep v1 and v2 alive together.	Teams migrate without a forced big bang.
Observe	Log version usage by consumer and route.	Provider knows who still depends on v1.
Warn	Return deprecation headers or dashboard alerts.	Consumers receive visible pressure, not surprise.
Retire	Remove v1 only after agreed window.	Retirement becomes planned work, not outage.

DEPRECATION IS A FLOW

API retirement is itself a workflow. It needs owners, dates, evidence, communication, and rollback thinking. Treating deprecation as a code deletion is how independent deployment becomes accidental breaking change.

NEXUS-1 versioning map

The map below shows a disciplined first versioning convention for the NEXUS-1 microservice discussion. The point is not to finalize every URL. The point is to keep public names, event names, and contract types aligned so that services can evolve without language drift.

Surface	Versioned example	Owner
RootCause REST API	/api/v1/root-cause/cases/{id}/summary	RootCause service
RootCause event	RootCauseVerdictIssued.v1	RootCause service
RootCause C# contract	RootCauseVerdictIssuedV1	Contracts package
Compliance request API	/api/v1/compliance/reviews/request	Compliance service
Operator BFF view	/api/v1/operator/cases/{id}/overview	Operator API / BFF
Audit trace query	/api/v1/audit/traces/{correlationId}	Audit service

AUDIT IS EVIDENCE, NOT WORKFLOW

The Audit service may expose versioned trace queries and may record versioned events. It should not become the service that drives the business process. AuditEntry remains evidence, not orchestration.

What not to version too early

Versioning is a discipline, but over-versioning is also a smell. Do not create v2 because a private handler changed. Do not version an internal domain event that never leaves its context. Do not version a DTO that is only used behind one application boundary. Public compatibility deserves ceremony; private refactoring should remain cheap.

Do not version	Reason	Better action
Internal domain event	It is not consumed outside the context.	Refactor with local tests.
Private application command	It belongs to one service boundary.	Change handler and tests together.
Database table shape	Storage is not the public contract.	Use migrations and repository mapping.
Temporary UI model	If only the BFF consumes it, keep it internal.	Stabilize before making public.
Experimental endpoint	No consumer promise exists yet.	Mark as internal or preview.

PUBLIC BEFORE VERSIONED

Do not turn every class into a versioned artefact. First decide whether the thing is public. Then decide how its compatibility will be protected.

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- Why is versioning a consumer-protection mechanism?
- What makes a change breaking even when the JSON still parses?
- Why must RootCauseVerdictIssued.v1 remain distinct from the internal VerdictIssued domain event?
- Which changes are usually additive, and which require a new version?
- Why do integration events need stricter historical discipline than ordinary REST responses?
- How do contract tests protect public API versions?
- What should happen before v1 is retired?
- Why should internal domain events and private handlers not be over-versioned?

NEXT STEP

Chapter 22 will ask when synchronous calls are actually acceptable. Versioning protects the contract; the next question is whether the call should happen synchronously at all.

A versioned contract is not a suffix. It is a promise that another service may safely build upon.

PART IV - APIS AND SYNCHRONOUS COMMUNICATION

22. When Synchronous Calls Are Acceptable

A synchronous call is not a failure of microservice design. It is acceptable when the caller is supposed to wait, the provider owns the answer, the failure can be shown honestly, and no distributed transaction is being smuggled into the request.

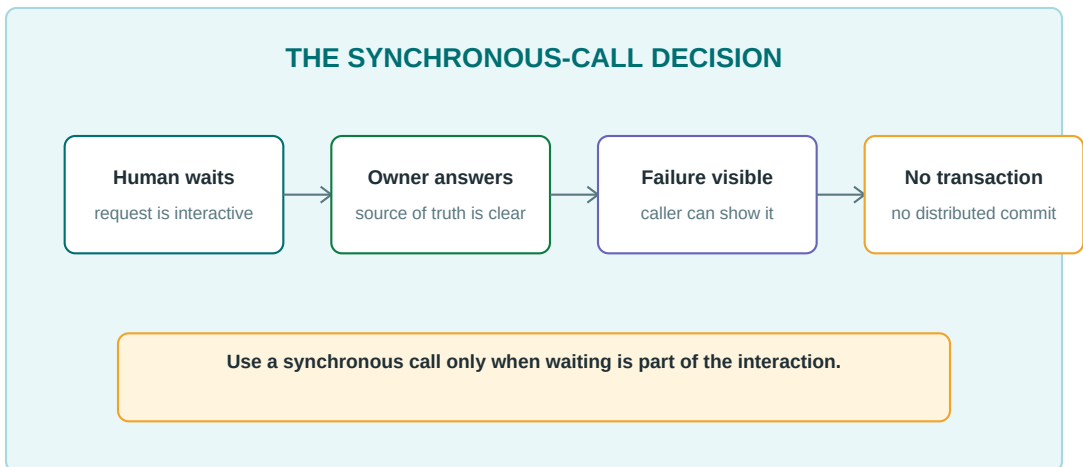


Figure 22.1 - A synchronous call is acceptable when waiting is part of the user or service interaction, not when it is hiding a cross-service transaction.

CHAPTER RULE

Synchronous communication is allowed when it makes the interaction clearer and the failure mode honest. It becomes dangerous when it tries to make a distributed system feel local.

Synchronous does not mean wrong

Microservice discussions often overcorrect. After a team learns that distributed calls create coupling, the answer sometimes becomes: never call another service synchronously. That rule is too simple. A user opening a case screen expects the screen to wait for current information. A backend-for-frontend composing an operator view may need to ask several owning services for their public read APIs. A validator may need a quick answer from an identity or reference service before accepting a command.

The problem is not the HTTP call. The problem is using the call to pretend that another service is local, always available, and part of the same transaction. A synchronous call is a request across a boundary. It must be designed with timeout, failure, ownership, and user experience in mind.

Naive rule	Better rule	Reason
Never use sync calls.	Use them only when waiting is honest.	Some interactions are naturally request-response.
HTTP is simple.	HTTP is simple until failure matters.	Latency and availability become part of design.
Just call the other service.	Call only a public contract.	Internal models must remain protected.
Retry until it works.	Timeout, fail clearly, and avoid duplicate side effects.	Infinite waiting is not reliability.

HONEST WAITING

The question is not whether HTTP is allowed. The question is whether the caller is allowed to wait for that answer and what the system will do if the answer does not arrive.

The four conditions for an acceptable call

A synchronous call is acceptable only when four conditions are visible. First, the caller has a legitimate reason to wait. Second, the provider clearly owns the data or decision being requested. Third, failure can be handled locally and shown honestly. Fourth, the call is not required to complete the same business transaction as the caller.

Condition	Question	NEXUS-1 example
The caller may wait.	Is this an interactive or immediate request?	Operator opens a root-cause case summary.
The provider owns the truth.	Is the called service the source of truth?	RootCause owns verdict state, not Reporting.
Failure is local and visible.	Can the caller show unavailable/degraded?	Operator BFF shows RootCause unavailable.
No distributed transaction.	Can the caller commit without remote commit?	AlarmManagement should not require Compliance commit before recording alarm.

A SIMPLE TEST

If the caller cannot continue, cannot explain failure, and cannot time out safely, the call is probably not an acceptable synchronous dependency. It is a workflow dependency looking for an event.

Interactive reads are the clearest case

The strongest case for synchronous calls is an interactive read. A human asks for a screen. The system composes the answer. If one public read API is unavailable, the UI can show a partial or degraded state. No domain decision is hidden inside the composition. No service is pretending to own another service's data.

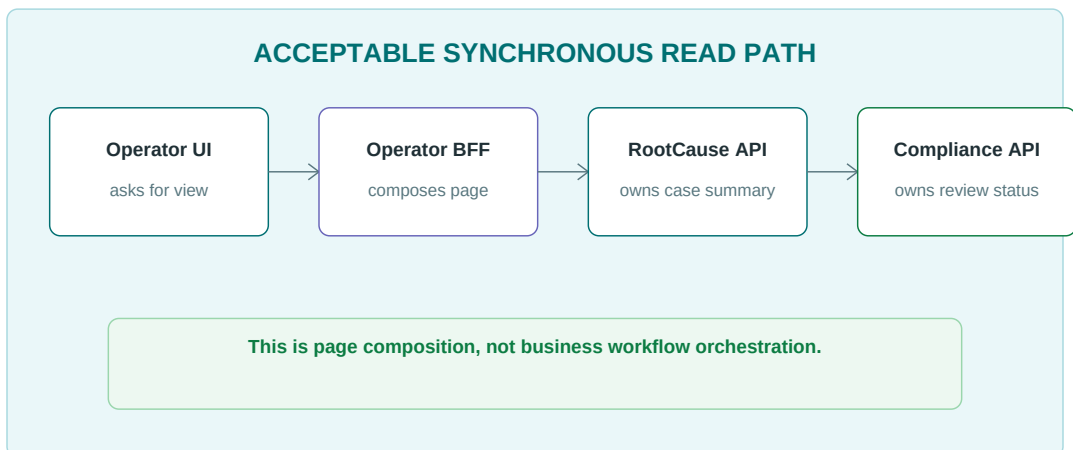


Figure 22.2 - A BFF may synchronously compose a view from public read APIs. That does not make the BFF the owner of RootCause or Compliance truth.

READ COMPOSITION IS NOT A SAGA

An operator overview may call RootCause and Compliance APIs to display a case. That is different from driving the alarm-to-compliance workflow. The former is view composition; the latter belongs to events and process management.

When a command may call another service

Commands are more dangerous than reads because they change state. A command handler may sometimes call another service synchronously, but only for a bounded, non-transactional decision that belongs outside the caller and can fail before the caller commits. The call must not create a two-service commit disguised as a method call.

Command situation	Synchronous call acceptable?	Reason
Check identity or permission before command	Usually yes	The command has not committed yet; failure rejects the request.
Validate a public reference value	Sometimes	Acceptable if cached fallback or clear timeout exists.
Reserve remote resource and commit local state	Risky	This begins distributed transaction territory.
Open local case only if remote workflow also starts	No	Use event/process manager; do not require dual commit.
Ask remote service for advisory read-only data	Sometimes	Acceptable if advisory and not source of final decision.

BEFORE THE COMMIT IS DIFFERENT

A synchronous validation before local state changes is often acceptable. A synchronous requirement that two services commit together is not the same thing and should not be treated as a normal API call.

NEXUS-1 acceptable synchronous calls

The table below gives a first NEXUS-1 judgement map. It does not say that every implementation must look exactly like this. It teaches the reasoning: who is waiting, who owns the answer, what fails, and whether the call is part of a business transaction.

Call	Judgement	Why
Operator BFF -> RootCause case summary	Acceptable	Human waits for a view; RootCause owns case truth; failure can be shown.
Operator BFF -> Compliance review status	Acceptable	View composition from public read API; no workflow decision.
RootCause -> Audit trace query	Acceptable for display	Read-only trace lookup; Audit remains evidence.
AlarmManagement -> RootCause OpenRootCauseCase	Usually avoid	Alarm flood should publish event; RootCause reacts without dual commit.
RootCause -> Compliance RequestComplianceReview	Avoid as sync workflow	Verdict issuance should publish event/process manager should coordinate.
Reporting -> every service during report generation	Avoid at scale	Reporting should build projections, not depend on live availability of all services.

THE WORD USUALLY MATTERS

Some calls are acceptable in a Phase-0 demonstrator but poor in a mature distributed design. The book keeps the judgement explicit rather than pretending every answer is universal.

Timeouts are part of the contract

A synchronous call without a timeout is not a designed dependency. It is a hope. The caller must decide how long it is willing to wait, what message it will show, whether a retry is safe, and whether partial data is acceptable. Those choices belong beside the API contract because they shape the user experience and the failure mode.

A bounded synchronous read client

```
public sealed class RootCauseClient
{
    private readonly HttpClient _http;

    public RootCauseClient(HttpClient http)
    {
        _http = http;
        _http.Timeout = TimeSpan.FromSeconds(2);
    }

    public async Task<CaseSummary?> GetSummaryAsync(string caseId, CancellationToken ct)
    {
        using var response = await _http.GetAsync($"api/v1/root-cause/cases/{caseId}/summary",
            ct);
        if (response.StatusCode == HttpStatusCode.NotFound) return null;
        response.EnsureSuccessStatusCode();
        return await response.Content.ReadFromJsonAsync<CaseSummary>(cancellationToken: ct);
    }
}
```

In a real implementation the timeout and policy would normally be configured outside the client. The important point is architectural: the call has a bounded wait and a clear contract. It does not block indefinitely.

Partial failure and degraded answers

The caller must be able to say what happens when only part of a composed view is available. This is especially important for the operator console. A page that combines RootCause, Compliance, Audit, and Reporting must not pretend all pieces share the same availability. One panel may be current, another stale, another unavailable.

Panel	Owning service	Degraded behaviour
Root-cause summary	RootCause	Show unavailable if case summary cannot be read.
Compliance status	Compliance	Show pending/unknown rather than inventing a review state.
Audit trace	Audit	Show trace unavailable; do not block case display.
Reporting projection	Reporting	Show last refreshed timestamp and stale marker.
Recommendation	ReinforcementLearning / advisory service	Hide or mark advisory unavailable; never replace with command.

DEGRADED IS BETTER THAN FALSE

A partial page with visible unavailable panels is more honest than a page that silently substitutes stale or invented truth. In NEXUS-1, visible uncertainty is part of the safety-adjacent teaching discipline.

Synchronous calls and ownership

The safest synchronous calls respect ownership. The caller asks the owner for something the owner is allowed to answer. The caller does not tunnel into the owner's database, copy private fields, or depend on internal lifecycle states. This keeps the call a boundary interaction rather than a leak.

Bad sign	Why it is a problem	Repair
Caller needs private status enum.	Private model leaked into public contract.	Publish stable public state.
Caller joins remote database table.	Data ownership is bypassed.	Use public read API or projection.
Caller needs provider transaction to commit.	Distributed transaction is hidden.	Use event/process manager.
Caller retries command blindly.	Duplicate side effects possible.	Use idempotency key or asynchronous workflow.
Caller treats remote timeout as business fact.	Technical failure becomes domain truth.	Represent unavailable separately.

BOUNDARY LANGUAGE AGAIN

A synchronous API is still a published language. Its job is to expose the owner's public meaning, not the owner's internal model.

Observability makes waiting accountable

A synchronous call creates a visible wait. That wait should be measurable. The caller should record duration, timeout, status code, correlation id, and the dependency name. Without those facts, a slow screen becomes an argument instead of an investigation.

Telemetry fact	Question it answers	NEXUS-1 value
Dependency duration	Which service made the request slow?	Operator BFF can identify RootCause latency.
Timeout count	Is the provider unavailable or merely slow?	Separates network failure from domain state.
Correlation id	Which calls belong to the same case view?	Connects RootCause, Compliance, Audit, Reporting.
Status code distribution	Is the contract being used correctly?	Finds NotFound, Unauthorized, or version errors.
Fallback/degraded flag	Did the user see a partial answer?	Makes UI truth auditable.

NO INVISIBLE DEPENDENCIES

If a service depends on another service synchronously, that dependency must be visible in logs, traces, dashboards, and architecture diagrams. Hidden runtime coupling is the beginning of operational surprise.

The decision checklist

Before accepting a synchronous call, use the checklist below. A yes to every line does not make the call free. It only means the call is architecturally honest enough to consider.

Checklist question	Required answer
Is the caller allowed to wait?	Yes; waiting is part of the interaction.
Does the provider own the requested truth?	Yes; the provider is the source of truth or public read owner.
Is the contract public and versioned where needed?	Yes; no internal model leak.
Can the caller time out safely?	Yes; failure path is designed.
Can the user or upstream caller understand degraded state?	Yes; unavailable is not confused with false.
Is the call outside a shared transaction?	Yes; no dual commit is required.
Is the dependency observable?	Yes; traces and metrics identify it.
Is asynchronous messaging worse for this interaction?	Yes; request-response is the clearer shape.

IF ONE ANSWER IS NO

A single no does not always forbid the call, but it demands design work. The more no answers you collect, the more likely the interaction belongs to messaging, projection, or process management instead.

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- Why is a synchronous call not automatically wrong?
- What are the four conditions that make such a call acceptable?
- Why are interactive reads the safest case?
- Why is a command-to-command synchronous dependency more dangerous?
- Which NEXUS-1 calls are acceptable as view composition?
- Why should AlarmManagement not usually call RootCause synchronously to open a case?
- What is the difference between unavailable and false?
- Why must timeout, degradation, and observability be treated as part of the design?

NEXT STEP

Chapter 23 will show the other side of the same decision: when synchronous calls become a trap. The point is not to ban request-response. The point is to notice when it starts carrying workflow, ownership, and transaction weight it cannot safely hold.

A good synchronous call tells the truth immediately. A bad one hides a distributed workflow behind a local-looking method.

PART IV - APIS AND SYNCHRONOUS COMMUNICATION

23. When Synchronous Calls Become a Trap

A synchronous call becomes dangerous when it stops being a simple request-response interaction and starts carrying workflow, transactions, ownership, retries, and availability assumptions that belong to a distributed design.

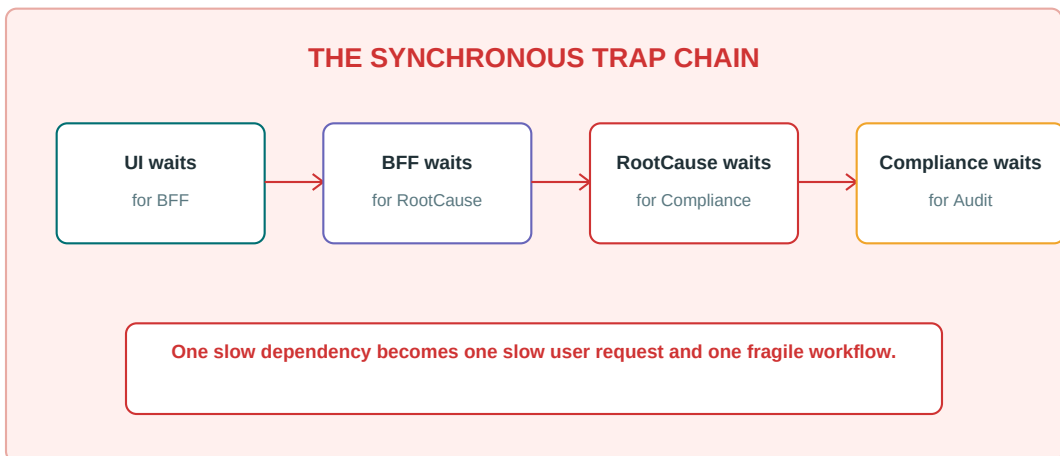


Figure 23.1 - The trap is not one HTTP call. The trap is a chain of waits where every service inherits the availability and latency of the next.

CHAPTER RULE

A synchronous call becomes a trap when the caller cannot complete, fail, explain, or recover without another service behaving like a local object.

The trap begins when the network disappears from the design

Chapter 22 allowed synchronous calls under strict conditions. This chapter draws the opposite boundary. The problem is not that service A calls service B. The problem is that service A begins to depend on service B as if B were a method in the same process, a table in the same database, and a participant in the same transaction.

That is how a distributed system becomes more fragile than the modular monolith it replaced. The monolith at least failed inside one runtime and one database boundary. A careless microservice design spreads the same workflow across a network and then hopes the network behaves like memory.

Local assumption	Distributed reality	Result
The call is quick.	Latency varies and queues form.	User requests slow down unpredictably.
The call succeeds.	The provider may be down or partitioned.	The caller cannot finish its work.
The transaction is one thing.	Each service owns its own transaction.	Partial completion becomes normal.
A retry is harmless.	The first attempt may have committed.	Duplicates and side effects appear.
The response is truth.	The response may be stale or unavailable.	Technical failure is mistaken for domain fact.

DISTRIBUTION IS NOT A DETAIL

If the design requires a remote service to behave like a local object, the service boundary has not been accepted. It has only been hidden behind HTTP.

Trap one: command chains

The most dangerous synchronous dependency is a command chain. A command enters one service, which must command another service, which must command a third, all before the original request can return. This looks tidy in a sequence diagram, but it makes every service responsible for every downstream service being available at the same moment.

A synchronous workflow disguised as an application service

```
// Smells simple, but it is a distributed workflow hidden inside a request.
public async Task IssueVerdictAsync(IssueVerdict command)
{
    var verdict = _case.IssueVerdict(command.CaseId, command.Text);
    await _rootCauseRepository.SaveAsync(verdict);

    await _complianceClient.RequestReviewAsync(command.CaseId);
    await _auditClient.WriteEvidenceAsync(command.CaseId, verdict);
    await _reportingClient.RefreshCaseProjectionAsync(command.CaseId);
}
```

The code reads like normal orchestration, but the transaction boundary is false. If the verdict is saved and the compliance call times out, what is the truth? Was the verdict issued? Should the compliance review exist? Should the request be retried? Every answer is harder than the code suggests.

NEXUS-1 REPAIR

The RootCause service should commit the verdict and publish RootCauseVerdictIssued.v1 through an outbox. Compliance, Audit, and Reporting react from their own boundaries. The workflow becomes explicit instead of hidden inside one HTTP request.

Trap two: distributed transactions by politeness

A team may say it is not using distributed transactions because there is no two-phase commit. But it can still create a distributed transaction by expectation. If the business operation is considered successful only when three services have all changed state, then the design is still asking for one transaction across three owners.

Operation	Hidden expectation	Why it traps the design
Issue root-cause verdict	Compliance review must be created before response returns.	RootCause cannot succeed without Compliance availability.
Raise alarm flood	RootCause case must be opened in same request.	AlarmManagement becomes blocked by diagnostic workflow.
Generate recommendation	Audit and Reporting must update instantly.	Advisory result depends on unrelated runtime paths.
Close incident	All projections must refresh before command returns.	Read models become part of command transaction.

A BETTER QUESTION

Ask what must be strongly consistent inside one owner and what may become consistent after an event. If the answer crosses service ownership, a synchronous call is usually the wrong tool.

Trap three: availability multiplication

A single service may be reliable enough. A chain of services is less reliable than every link in the chain. If a request requires four services to be available at the same time, the request availability is roughly the product of those dependencies. The exact math is not the point. The intuition is enough: every required synchronous dependency makes the user-facing operation easier to break.

Required dependency shape	User-facing effect	Architectural meaning
One service answers.	One dependency can fail.	Simple request-response.
BFF calls three read APIs.	Partial page may degrade.	Acceptable if panels fail independently.
Command calls three command APIs.	Whole command fails if any link fails.	Trap: workflow hidden in request.
Service calls service which calls service.	Latency and failure propagate.	Trap: cascading wait chain.
Report calls all live services.	Report unavailable when any owner is slow.	Trap: projection should exist locally.

READ COMPOSITION VS WORKFLOW DEPENDENCY

Calling several read APIs to compose a page can be acceptable when each panel can degrade. Calling several services to complete one command usually means the workflow needs events or a process manager.

Trap four: retry without idempotency

Retries are often added to make synchronous calls feel safer. They can do the opposite. If the first request reached the provider and committed but the response was lost, a retry may repeat the side effect. This is not a rare distributed edge case; it is normal uncertainty.

Retry is not reliability unless duplicates are designed

```
// Dangerous when the provider command is not idempotent.
await retryPolicy.ExecuteAsync(async () =>
{
    await _complianceClient.RequestReviewAsync(caseId);
});

// Safer contract shape: the operation carries a stable idempotency key.
await _complianceClient.RequestReviewAsync(new RequestComplianceReviewV1
{
    CaseId = caseId,
    VerdictId = verdictId,
    IdempotencyKey = $"rc-verdict:{verdictId}:review"
});
```

Even with idempotency, the deeper question remains: should this be a synchronous command at all? Idempotency makes retries safer. It does not automatically make the boundary correct.

RULE OF THUMB

If the caller needs retries because the operation is important, the operation probably deserves durability. Durable usually means outbox, inbox, idempotent consumer, and observable delivery - not a longer HTTP wait.

Trap five: leaking private states

A synchronous call can also become a trap when the caller needs too much detail from the provider to decide what to do. Instead of asking for a public answer, the caller asks for private lifecycle states, internal flags, or domain terms that are not part of the published language. The call then becomes a semantic leak.

Caller asks for	Smell	Better contract
RootCauseCase.InternalLifecycle State	Caller knows private aggregate lifecycle.	CaseSummaryStatus in public API.
ComplianceReview.DbStatusCode	Database detail becomes public language.	ReviewStatus.v1 with stable values.
AuditEntry table row shape	Evidence store leaks persistence model.	AuditTraceItem response contract.
Reporting projection rebuild flag	Caller depends on reporting internals.	ProjectionFreshness response.

BOUNDARY PROTECTION

When a caller needs private states from another service, do not solve it with one more endpoint. Revisit the public contract, the ownership decision, or the event that should have crossed the boundary earlier.

NEXUS-1 trap cases

The recurring alarm-to-compliance flow gives concrete examples. Some interactions may look like ordinary service calls, but they carry the weight of workflow ownership. Those are the cases to move away from synchronous request chains.

Naive synchronous design	Trap	Preferred shape
AlarmManagement calls RootCause.OpenCase during flood detection.	Alarm handling depends on diagnostic service availability.	Publish AlarmFloodDetected.v1; RootCause consumes.
RootCause calls Compliance.RequestReview before returning verdict.	Verdict success depends on review creation.	Publish RootCauseVerdictIssued.v1; process manager or Compliance consumes.
Compliance calls Reporting.Refresh after review decision.	Workflow waits for projection update.	Reporting consumes events and rebuilds read model.
Operator BFF calls Audit to display trace.	Acceptable if read-only and degradable.	Keep synchronous but mark panel unavailable on failure.
RootCause calls Audit as mandatory command after every evidence change.	Evidence write waits on audit runtime.	Local outbox/audit event with durable delivery.

THE SAME CALL CAN CHANGE CHARACTER

A read-only, degradable call to Audit is acceptable. A mandatory command call to Audit inside the same business operation is a trap. The boundary is not the service name; it is the responsibility carried by the call.

Replacing the trap with flow

The repair is not always messaging, but messaging is the usual repair when the problem is workflow. The service that owns the decision commits its own transaction. It records the public event in the outbox. Another process publishes the event. Downstream services consume it idempotently and update their own truth.

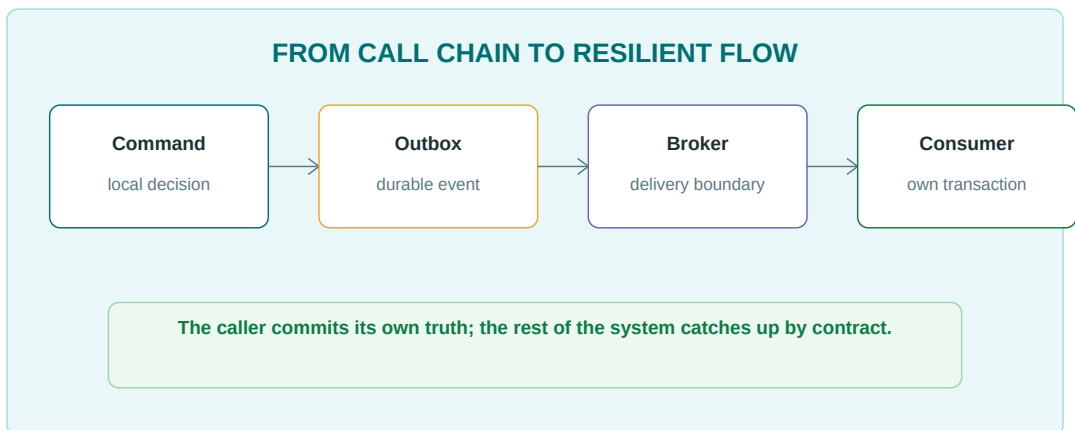


Figure 23.2 - The repair separates local decision, durable publication, delivery, and consumer-owned transactions.

This is not free. It introduces eventual consistency, duplicate handling, ordering questions, dead-letter queues, monitoring, and operational complexity. But that complexity is honest. The synchronous trap keeps the same complexity and hides it inside a request timeout.

The synchronous-trap checklist

Use this checklist when a proposed API call feels natural. Natural is not enough. A call can be easy to write and still be architecturally wrong.

Question	Trap answer
Does the caller need the remote command to commit before it can commit?	Yes - hidden distributed transaction.
Does a retry risk repeating a side effect?	Yes - missing idempotency or wrong interaction shape.
Does the caller know provider internals?	Yes - private model leaked.
Does one user request wait through a long service chain?	Yes - availability and latency multiply.
Does failure leave the workflow half-done?	Yes - process manager or event flow needed.
Does the call exist only to refresh a read model?	Yes - projection should be asynchronous.
Would the modular monolith handle this better today?	Possibly - extraction may be premature.

WHEN IN DOUBT

Draw the failure path. If the failure path is longer than the happy path, the synchronous call is probably hiding the real design problem.

Honest boundary

This chapter is deliberately strict because microservices are often damaged by calls that were individually reasonable and collectively disastrous. The warning does not mean every synchronous call must disappear. It means every synchronous call must be small enough to fail honestly.

Keep synchronous when	Move away from synchronous when
The caller is composing an interactive read view.	The caller is trying to complete a cross-service workflow.
Failure can be shown as unavailable or partial.	Failure leaves business state ambiguous.
The provider owns a public answer.	The caller needs provider internals.
The call has a short timeout and clear fallback.	The call sits inside a long dependency chain.
No shared transaction is implied.	The operation succeeds only if several services commit.

NEXUS-1 POSITION

For NEXUS-1, root-cause verdict issuance is a domain decision owned by RootCause. Compliance review, audit evidence, and reporting projection should follow from public contracts and durable flow, not from a command chain that makes the verdict wait for every downstream sector.

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- Why can one acceptable synchronous call become dangerous when placed in a chain?
- What is a command chain, and why is it worse than read composition?
- How can a team create a distributed transaction without using two-phase commit?
- Why do retries require idempotency?
- Why does a lost response create uncertainty about whether the provider committed?
- What private-state leaks should be avoided in NEXUS-1 public APIs?
- Which parts of the alarm-to-compliance flow should move through events rather than command calls?
- Why is visible eventual consistency often more honest than hidden synchronous waiting?

NEXT STEP

Chapter 24 completes Part IV by looking at the API Gateway and Backend-for-Frontend for the Operator Console. After that, the book can move from synchronous communication into messaging and asynchronous service integration.

A bad synchronous call does not merely wait for another service. It borrows another service's failure and calls it its own.

PART IV - APIS AND SYNCHRONOUS COMMUNICATION

24. API Gateway and Backend-for-Frontend for the Operator Console

The operator console should not know every service topology, every retry rule, every internal contract, and every failure mode. The edge of the system needs protection, and the UI needs a composition layer shaped for humans, not for service internals.

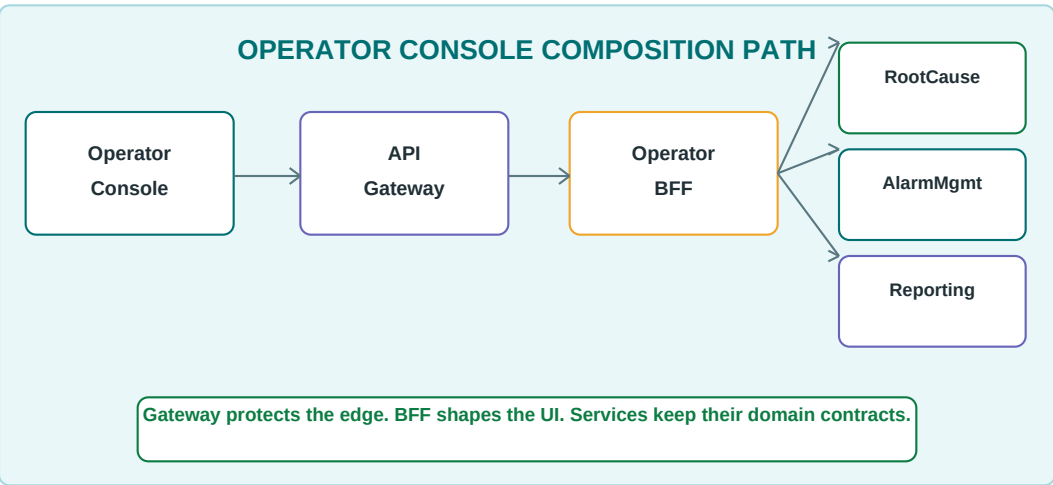


Figure 24.1 - The gateway is the controlled edge; the BFF is the console-specific composition layer; the services remain owners of their own contracts.

CHAPTER RULE

An API Gateway is not a place to hide domain coupling. A Backend-for-Frontend is not a place to make business decisions. The gateway protects the edge; the BFF composes a user experience.

The operator console is not one service

The NEXUS-1 console looks like one application to the operator. Architecturally, it is many questions on one screen. What is the unit state? Which alarms are active? Is there an alarm flood? Is a root-cause case open? Has a verdict been issued? Is compliance review pending? Is the reporting view fresh? No single service should own all of those answers.

Without a composition layer, the browser becomes the integration layer. That is tempting because modern front-end code can call many APIs. But it gives the UI knowledge it should not have: service locations, token rules, version choices, retry policy, timeout policy, correlation propagation, fallback behaviour, and the shape of every internal contract.

Screen need	Likely owner	UI should know
Active alarms	AlarmManagement	Public alarm summary, not alarm aggregate internals.
Root-cause status	RootCause	Case status and verdict summary, not hypothesis lifecycle tables.
Compliance status	Compliance	Review state, not control catalogue internals.
Traceability	Audit	Trace items and correlation ids, not audit table shape.
Dashboard counts	Reporting	Read-model freshness and counts, not source table joins.

NEXUS-1 POSITION

The operator console may display a single coherent picture, but it must not collapse service ownership into one front-end dependency map.

API Gateway and BFF are not the same thing

The two patterns are often spoken about together because both live near the edge. They solve different problems. The gateway is the controlled entrance into the service estate. The BFF is a client-specific adapter that speaks in the shape of one user experience.

Concern	API Gateway	Backend-for-Frontend
Primary question	May this request enter?	What does this client need to see?
Scope	All or many clients.	One client or client family.
Typical duties	Routing, TLS, authentication, rate limits, coarse policy, request logging.	Screen-shaped queries, response composition, fallbacks, UI DTOs.
Should it know domain rules?	No.	No. It may orchestrate reads, not decide domain truth.
NEXUS-1 example	Route /api/rootcause/* to RootCause service.	Build OperatorOverviewDto from alarms, root-cause status, compliance and reporting.

BOUNDARY LANGUAGE

Gateway is an edge-control pattern. BFF is a client-composition pattern. Confusing them leads either to a gateway full of business logic or to a BFF that behaves like a hidden monolith.

Why the UI should not call every service directly

A browser-based operator console can call RootCause, AlarmManagement, Compliance, Reporting, Audit, and DigitalTwin directly. The fact that it can does not mean it should. Direct calls make the browser aware of the runtime map. Every service split, version change, authentication difference, and temporary outage becomes a front-end problem.

Direct-call problem	Architectural consequence
The UI knows service addresses.	Deployment topology leaks into client code.
The UI chooses API versions.	Version migration becomes a browser compatibility problem.
The UI retries each panel differently.	Reliability policy is duplicated and inconsistent.
The UI joins cross-service data.	A screen becomes an accidental integration service.
The UI handles every partial failure alone.	Operator experience becomes inconsistent across screens.
The UI receives too much private data.	Front-end DTOs drift toward domain entities.

NOT A FRONT-END CRITIQUE

This is not a criticism of Angular, React, or any client framework. It is a boundary rule: user-interface code should express user interaction, not service topology.

What belongs in the API Gateway

The gateway is a controlled edge. It should be boring in the best sense: narrow responsibilities, predictable behaviour, and no temptation to become the new application layer. Its value is that every inbound request crosses one accountable entrance before it reaches a service.

Gateway responsibility	Reason
TLS termination and edge routing	The outside world should not know every internal service endpoint.
Authentication token validation	Services receive a trusted identity context, not anonymous edge traffic.
Coarse authorization policy	Obviously invalid traffic can be rejected before service work begins.
Rate limits and request size limits	The edge absorbs abusive or accidental overload.
Correlation id creation and propagation	A request can be traced across services and logs.
Protocol translation where justified	External simplicity can be separated from internal transport details.

GATEWAY ANTI-RULE

Do not put root-cause verdict rules, compliance review decisions, recommendation logic, audit interpretation, or reporting joins inside the gateway. If the gateway starts deciding domain truth, the architecture has created a new central brain by accident.

What belongs in the Operator BFF

The BFF exists because the operator console is not shaped like a service boundary. A screen is shaped around human attention. Services are shaped around ownership. The BFF translates between those two shapes without stealing the service decisions.

BFF responsibility	NEXUS-1 example
Screen-shaped read DTOs	OperatorOverviewDto, AlarmFloodPanelDto, CaseTimelineDto.
Read composition	Combine alarm status, root-cause summary, compliance review state, and reporting freshness.
Degradation policy	Return partial view with panel status instead of failing the entire dashboard.
Client-specific naming	Use words the console displays, not table or aggregate internals.
Caching short-lived read data	Cache dashboard summaries where staleness is visible and acceptable.
Correlation propagation	Forward TraceId/CorrelationId to every service call used in a composite view.

BFF DISCIPLINE

The BFF may compose reads. It should not issue domain verdicts, close cases, create compliance findings, or compensate workflows. Those decisions remain in the services that own the language.

A screen-shaped contract

The BFF contract should be stable for the console, not for the database. It should answer the question the screen asks. For the operator overview, the question is not "give me rows from five services". The question is "what should the operator see right now, and which parts are fresh?"

A console-oriented BFF DTO

```
public sealed class OperatorOverviewDto
{
    public string UnitCode { get; init; } = null!;
    public PanelStatus Fleet { get; init; } = null!;
    public PanelStatus Alarms { get; init; } = null!;
    public PanelStatus RootCause { get; init; } = null!;
    public PanelStatus Compliance { get; init; } = null!;
    public PanelStatus Reporting { get; init; } = null!;
    public DateTime AsOfUtc { get; init; }
    public string CorrelationId { get; init; } = null!;
}

public sealed class PanelStatus
{
    public string State { get; init; } = "available"; // available, stale, unavailable
    public string? Message { get; init; }
    public object? Data { get; init; }
}
```

The important detail is not this exact class. The important detail is that availability and freshness are part of the contract. The UI is not forced to guess whether missing data means no problem, stale projection, or service failure.

Degradation is a user-experience contract

In Chapter 23 we saw that synchronous calls become a trap when one user request waits on a long chain. The BFF repairs part of that problem by making degradation explicit. A dashboard can remain useful when one panel is stale, as long as the operator is told plainly what is fresh, what is old, and what is unavailable.

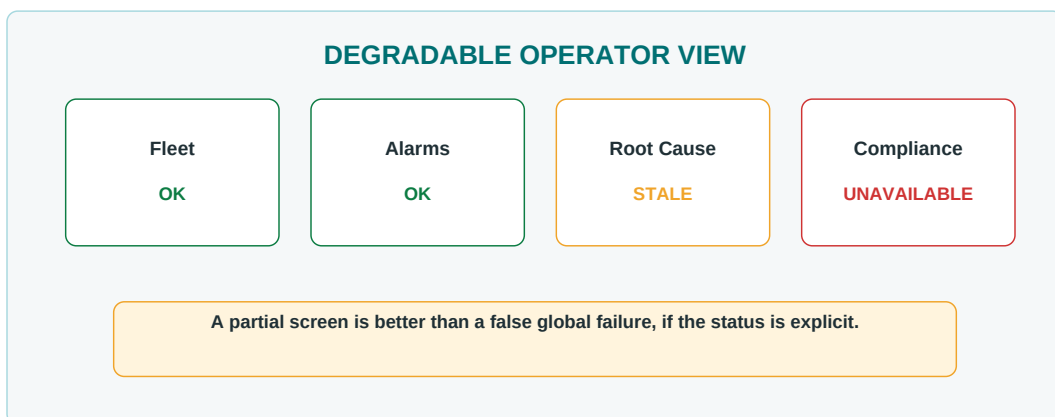


Figure 24.2 - A BFF can return a partial screen contract instead of turning one unavailable service into a total console failure.

This is especially important for NEXUS-1 because the console is an explanatory and advisory surface. A missing compliance panel should not make the active alarm list disappear. A stale reporting projection should not be confused with the absence of an alarm flood.

HONEST UI RULE

A degraded screen is acceptable only when degradation is visible. Silent fallback is worse than failure because it makes stale or incomplete information look authoritative.

Read composition is not write orchestration

The BFF is allowed to compose reads because a screen frequently needs data from several owners. That permission does not extend to writes. A command should enter the service that owns the decision, not the service that happens to draw the screen.

Operation from console	Where it should go	BFF role
Acknowledge alarm	AlarmManagement	Forward command or link to service contract; do not decide alarm state.
Open root-cause case	RootCause	Forward command; do not create related records elsewhere.
Issue verdict	RootCause	Send command to owner; downstream effects follow events.
Request compliance review	Compliance or process flow	Do not manually chain after RootCause verdict.
View operator overview	Operator BFF	Compose read DTO and degradation status.
Export report snapshot	Reporting	Call reporting contract or return export status.

WRITE RULE

The BFF may make the command convenient for the client, but it must not become the owner of the business decision. Convenience is not ownership.

Identity, correlation, and audit through the edge

Gateway and BFF design is also an audit problem. A service must be able to answer who asked, which screen or client initiated the request, which correlation id ties the calls together, and which service actually made the domain decision. Losing that information at the edge damages traceability.

Traceability context, not domain authority

```
// Edge context propagated by gateway and BFF.
public sealed record RequestContext(
    string UserId,
    string UserDisplayName,
    string ClientApplication, // OperatorConsole
    string CorrelationId,
    string CausationId,
    DateTime ReceivedAtUtc);

// Services may record this as metadata,
// but the domain decision still belongs to the receiving service.
```

The BFF can propagate context. It cannot turn its own user-session knowledge into permission to change another service's rules. Authorization and domain invariants still belong at the service boundary and inside the owning model.

AUDIT DISTINCTION

Audit records who requested and what was decided. It should not make the BFF the author of every downstream decision. The deciding service must remain visible in the trace.

What must not move into the edge

Because the gateway and BFF sit in a convenient position, they attract responsibilities. That is why they must be policed. The edge should protect access and compose views. It should not become the place where domain boundaries are bypassed because calling one more service from the middle feels easy.

Do not put this in the gateway or BFF	Why
Root-cause scoring rules	RootCause owns causal reasoning and verdict discipline.
Compliance control interpretation	Compliance owns regulatory language and review rules.
Audit evidence meaning	Audit records evidence; it does not decide workflow.
Cross-service compensations	A process manager or saga owns long workflow state.
Database joins across service stores	This breaks service ownership and re-creates the shared database problem.
Private DTO translation for every service	The edge becomes tightly coupled to internal models.

EDGE ANTI-PATTERN

The worst BFF is a distributed monolith in disguise: all services become data providers, and the BFF becomes the real application. That is not a microservice architecture; it is centralization with network latency.

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- What problem does an API Gateway solve, and what problem does it not solve?
- Why is a BFF useful for the NEXUS-1 operator console?
- Why should the browser not call every service directly?
- What is the difference between read composition and write orchestration?
- Why must degradation be visible in the UI contract?
- Which metadata must travel through the edge for traceability?
- Why should root-cause verdict rules never live in the BFF?
- How does this chapter complete the synchronous-communication arc of Part IV?

NEXT STEP

Part V moves from APIs to messages. Once the service edge is controlled, the next question is how services publish facts, consume events, retry safely, reject poison messages, and keep public contracts durable over time.

A good edge protects the system without becoming the system. A good BFF serves the screen without owning the domain.

PART V

Messaging and Asynchronous Communication

Part V leaves the comfort of direct calls. A service no longer waits for every downstream consequence to finish before it returns. Instead, it publishes facts, consumes public events, records outbox rows, processes inbox messages idempotently, and accepts that delivery, ordering, duplication, and failure are now part of the design.

CHAPTERS 25-32

MESSAGES - INTEGRATION EVENTS - BROKERS - OUTBOX - INBOX -
IDEMPOTENCY - ORDERING - DEAD-LETTER QUEUES - EVENT CONTRACTS

A service boundary becomes real when another service can react without being called directly.

PART V - MESSAGING AND ASYNCHRONOUS COMMUNICATION

25. Why Services Need Messages

A service should not have to call every downstream service just to finish its own work. Messages let a service publish a durable fact, return control to its owner, and allow other services to react without turning the whole system into one long synchronous chain.

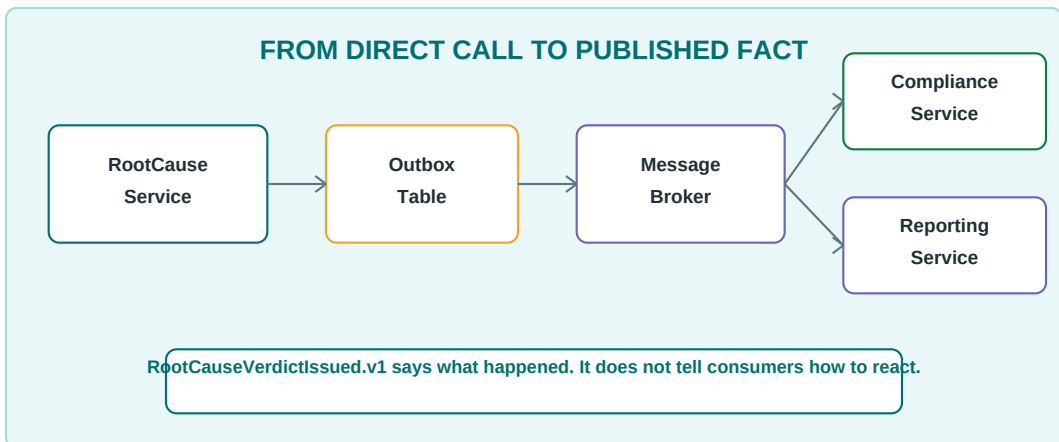


Figure 25.1 - Messaging begins when the owning service publishes a public fact instead of directly commanding every downstream consequence.

CHAPTER RULE

A message is not a smaller HTTP request. A good integration event is a durable public fact: something happened inside one owner, and other services may react according to their own responsibilities.

The problem direct calls cannot solve

At the end of Part IV, the synchronous boundary was clear. Direct calls are acceptable when the caller needs an immediate answer from the owner of a decision. They become dangerous when one action turns into a chain of calls across many services. The alarm-to-compliance flow is exactly that kind of chain.

When RootCause issues a verdict, the root-cause service owns the decision. But the decision does not end the system story. Compliance may need review. Reporting may need a projection. Audit needs evidence. The operator console may need a status change. None of those consequences should force RootCause to wait for every downstream service to be healthy before it can finish issuing the verdict.

Naive synchronous chain	What becomes fragile
RootCause calls Compliance directly.	Compliance outage can make a valid verdict look like a failed verdict.
RootCause calls Reporting directly.	A read-model refresh can delay a domain decision.
RootCause calls Audit directly as workflow.	Evidence recording becomes confused with orchestration.
RootCause waits for all responses.	One slow dependency stretches the critical path.
RootCause retries all calls itself.	The owner of the verdict becomes the owner of every downstream failure.

NEXUS-1 POINT

RootCause must be able to issue a verdict because its own rules are satisfied. It should not need Compliance, Reporting, or the operator console to be available in the same instant.

A message is a fact, not a request for permission

The first mental shift is simple: a message should not mean "please let me finish my work". It should mean "my work reached a fact that others are allowed to know". That difference protects ownership. The publisher does not ask consumers how to interpret its domain. It publishes a stable fact in a stable public language.

Question	Synchronous call	Message
What is the sender asking?	Do this now and give me the result.	This happened; react if it matters to you.
Coupling shape	Sender knows receiver exists.	Producer knows public event contract, not each consumer.
Failure effect	Receiver outage can block sender.	Consumer outage can be retried later.
Consistency model	Often immediate for the request.	Usually eventual across services.
Best use	Queries, validation, small commands to owner.	Facts, notifications, read-model updates, downstream workflows.

In NEXUS-1, the canonical public fact is `RootCauseVerdictIssued.v1`. It says that the `RootCause` context has issued a verdict for a case. It does not say that `Compliance` must open a review, `Reporting` must rebuild a table, or the console must repaint immediately. Those are consumer decisions.

LANGUAGE DISCIPLINE

VerdictIssued is an internal domain event. `RootCauseVerdictIssued.v1` is a public integration event. Keeping those names separate prevents the private model from becoming the public contract.

The canonical NEXUS-1 event

A good event name is boring and precise. It should name the fact that became true, not the reaction someone might perform. That is why the public event is `RootCauseVerdictIssued.v1` rather than `RootCauseCaseClosed`, `RootCauseVerdictPublished`, or `RequestComplianceReview`.

Name	Why it is or is not suitable
VerdictIssued	Good internal domain event name; too private to expose as the public contract.
RootCauseVerdictIssued.v1	Good public integration event: owner, fact, and version are visible.
RootCauseCaseClosed	Too vague; a case could close for reasons other than a final verdict.
RootCauseVerdictPublished	Implies publication mechanism, not the domain fact.
RequestComplianceReview	A command or workflow request, not the canonical event from RootCause.

CANONICAL EVENT RULE

Name the fact the producer owns. Do not name the consumer reaction. A producer publishes what happened; consumers decide what that means for them.

The public contract shape

The public contract is not the aggregate. It is not the database row. It is the minimum durable fact other services need in order to react safely. It should include identity, timing, version, correlation, and enough summary data to avoid immediate callback for basic decisions.

Public integration event contract

```
public sealed class RootCauseVerdictIssuedV1
{
    public string EventId { get; init; } = null!;
    public string EventType { get; init; } = "RootCauseVerdictIssued.v1";
    public DateTime OccurredAtUtc { get; init; }
    public string CorrelationId { get; init; } = null!;
    public string CaseId { get; init; } = null!;
    public string UnitId { get; init; } = null!;
    public string VerdictCode { get; init; } = null!;
    public decimal Confidence { get; init; }
    public IReadOnlyList<string> EvidenceIds { get; init; } = Array.Empty<string>();
}
```

The contract deliberately avoids private details: no aggregate object graph, no EF Core navigation properties, no internal hypothesis lifecycle, no service-specific table names. It is public because it is stable enough for consumers to depend on.

CONTRACT BOUNDARY

The event should contain enough information for a consumer to decide whether it cares. It should not contain the producer's private model just because serialization makes that easy.

Messages reduce temporal coupling

The deepest benefit of messaging is not speed. It is temporal independence. With a synchronous call, the caller and receiver must both be alive at the same time. With a durable message, the producer can finish now and the consumer can catch up later.

Temporal situation	With direct call	With message
Compliance is temporarily down	RootCause call fails or waits.	Event waits; Compliance consumes later.
Reporting projection is slow	User action may become slow.	Projection catches up asynchronously.
New consumer is added	Producer code may change to call it.	Consumer subscribes to existing public event.
Consumer receives duplicate	May repeat side effects if not guarded.	Inbox/idempotency handles duplicate.
Consumer is removed	Producer call path must change.	Subscription is removed; producer still publishes fact.

BENEFIT AND COST

Messaging reduces temporal coupling, but it does not remove design work. It introduces delivery semantics, duplicate handling, ordering questions, dead-letter queues, and event-version governance.

Messages introduce eventual consistency

A message means another service may not have reacted yet. That is not a bug; it is the consistency model. Once RootCause publishes RootCauseVerdictIssued.v1, Compliance may still show "review pending creation" for a short time, Reporting may still show an older read model, and the operator console must display that lag honestly.

View	Source of truth	What may lag
RootCause case page	RootCause service	Nothing about the verdict itself; RootCause owns it.
Compliance queue	Compliance service	The review task may not exist yet.
Reporting dashboard	Reporting projection	Counts may be stale until the event is processed.
Operator overview	BFF/composed view	Panels may have different freshness.
Audit trace	Audit/event log	Projection may be delayed, but event metadata must be traceable.

HONEST UI RULE CARRIED FORWARD

The console must distinguish no verdict, verdict issued but not yet propagated, and propagation failed. Eventual consistency is safe only when the user experience does not pretend everything is immediate.

The outbox is where messaging becomes reliable

Publishing a message after a database transaction sounds easy until the process crashes between the commit and the publish. If the verdict is saved but the event is lost, consumers never know. If the event is published but the verdict transaction rolls back, consumers react to a fact that is not true. The outbox pattern exists to close that gap.

Verdict and event recorded atomically

```
// Inside the same transaction:
case.IssueVerdict(verdict);
db.RootCauseCases.Update(case);

db.OutboxMessages.Add(new OutboxMessage(
    type: "RootCauseVerdictIssued.v1",
    payload: Serialize(contract),
    correlationId: request.CorrelationId));

await db.SaveChangesAsync();

// A background publisher sends outbox rows later.
```

The outbox does not make delivery magical. It makes the first promise reliable: if the domain fact was committed, the intention to publish it was committed with it. Later chapters will handle publishing, retry, poison messages, and cleanup.

OUTBOX PROMISE

The database transaction commits the state change and the message record together. The broker publish can fail and be retried without pretending the state change did not happen.

The inbox protects consumers from normal delivery behaviour

Messaging systems normally prefer at-least-once delivery because losing a message is worse than delivering it twice. That means duplicates are normal. A consumer that cannot tolerate duplicates is not ready to consume messages.

Consumer risk	Inbox/idempotency response
Same event delivered twice	Store EventId and ignore already-processed messages.
Consumer crashes after side effect	Process within a local transaction where possible.
Different events arrive close together	Use local ordering rules or tolerate independent processing.
Message payload cannot be parsed	Reject to dead-letter path with reason.
Consumer version is behind	Accept known versions or route unsupported versions safely.

IDEMPOTENCY RULE

A consumer must be able to say: I have already processed this public fact, so processing it again would be wrong. That memory is part of the consumer's responsibility.

What messages are not for

Messages are powerful enough to be misused. They should not be used to avoid making ownership decisions. They should not turn every method call into a broker hop. They should not hide the fact that a user needs an immediate answer. They should not be used as remote procedure calls with worse debugging.

Misuse	Why it is wrong
Ask another service to validate a command that must finish now	The user needs an immediate owner decision; use a direct call or rethink ownership.
Publish every internal domain event publicly	Private model churn becomes public contract churn.
Use messages for simple local method calls	Adds delivery uncertainty without crossing a meaningful boundary.
Send commands disguised as events	Consumers become hidden subordinates of the producer.
Ignore ordering and duplicates	Normal broker behaviour becomes data corruption.
Use messages to avoid API design	The system still needs explicit contracts, versioning, and meaning.

ANTI-HYPE BOUNDARY

Messaging does not make a system simpler. It makes some forms of coupling weaker and some forms of failure more explicit. That trade is valuable only when the boundary deserves it.

Applying the idea to the alarm-to-compliance flow

The recurring NEXUS-1 flow now has a clean division. Commands go to the service that owns the decision. Events announce facts after the owner has decided. Consumers react without forcing the producer to wait for them.

Step	Communication style	Owner
Raise alarm	Command or internal ingestion to AlarmManagement	AlarmManagement
Detect alarm flood	Internal decision, then integration event AlarmFloodDetected.v1	AlarmManagement
Open root-cause case	Command OpenRootCauseCase	RootCause
Attach evidence	Command to RootCause	RootCause
Issue verdict	Command to RootCause; publish RootCauseVerdictIssued.v1	RootCause
Trigger compliance review	Consumer reaction or process manager command	Compliance / workflow owner
Build reporting projection	Consume events asynchronously	Reporting
Show operator status	BFF reads available views and freshness	Operator console / BFF

CONNECTIVITY CHECK

Messages do not replace the flow. They make the flow honest about ownership: who decides, who publishes, who reacts, and who merely observes.

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- Why should RootCause not synchronously call every downstream service after issuing a verdict?
- What is the difference between an internal domain event and a public integration event?
- Why is RootCauseVerdictIssued.v1 better than RootCauseCaseClosed as the canonical public event?
- What does the outbox protect against?
- Why must consumers be idempotent?
- How does messaging introduce eventual consistency?
- When should you not use a message?
- How does this chapter connect Part IV synchronous APIs to Part V asynchronous communication?

NEXT STEP

Chapter 26 revisits integration events in detail. Now that the need for messages is clear, the next question is how to design an event that is stable, versioned, useful, and still small enough to protect the producer's private model.

A service publishes facts because other services should be able to react without owning the decision that created them.

PART V - MESSAGING AND ASYNCHRONOUS COMMUNICATION

26. Integration Events Revisited

An integration event is not just a domain event sent over a broker. It is a public contract, written in the language of the producing context, versioned for consumers, and small enough to protect the private model that created it.

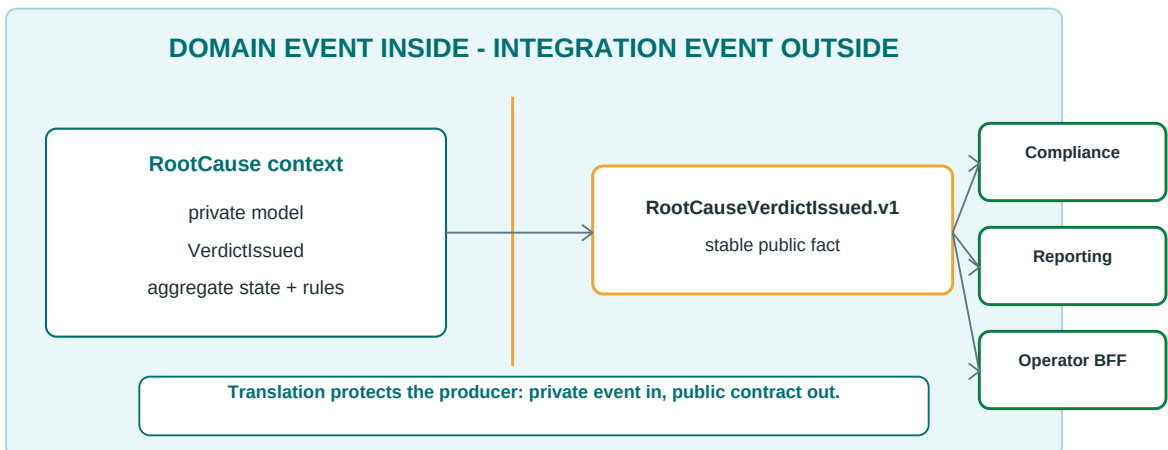


Figure 26.1 - A private domain event is translated at the service boundary into a public integration event.

CHAPTER RULE

Do not publish your aggregate. Do not publish your private domain event just because it already exists. Publish a deliberate integration event whose meaning, owner, version, and consumer obligations are clear.

Why revisit integration events?

From Context to Flow already introduced the distinction between domain events and integration events. Chapter 25 showed why services need messages at all. This chapter now slows down at the boundary and asks the practical design question: what exactly should be published?

The easy answer is dangerous: take the domain event, serialize it, and put it on the broker. That feels efficient because `VerdictIssued` already exists inside the `RootCause` model. But the internal event was created for one purpose: to help the local model record and react to a change inside its own boundary. A public integration event has a different purpose: to let other services react without learning the producer's private shape.

Internal event	Public event
Lives inside one bounded context.	Crosses a service boundary.
May change when the internal model changes.	Must change slowly and deliberately.
Can mention private concepts.	Uses published language only.
Useful for local handlers and audit construction.	Useful for downstream services and projections.
Example: <code>VerdictIssued</code>	Example: <code>RootCauseVerdictIssued.v1</code>

BOUNDARY MEMORY

A domain event says what happened inside the model. An integration event says what the model is willing to make public.

The translation step

The most important line in an event-driven microservice is often not the publish call. It is the mapping line before the publish call. That line decides what private information leaves the boundary, what is renamed into public language, what is omitted, and what version the outside world will see.

Translation from private event to public contract

```
var domainEvent = new VerdictIssued(  
    caseId: rootCauseCase.Id,  
    verdict: rootCauseCase.FinalVerdict,  
    evidence: rootCauseCase.Evidence);  
  
var integrationEvent = RootCauseVerdictIssuedV1.From(  
    domainEvent,  
    correlationId: command.CorrelationId,  
    occurredAtUtc: clock.UtcNow);  
  
outbox.Add(integrationEvent);
```

The integration event is not a mirror of the internal object. It is a curated statement. That curation is not hiding information; it is protecting the producer from accidental contract leakage and protecting consumers from accidental dependency on unstable details.

TRANSLATION RULE

Every public event should pass through an explicit translation step. If there is no translation, the private model is probably being exposed by accident.

The event envelope

A useful integration event has two layers. The payload says what happened. The envelope says how the message participates in a distributed system: identity, time, trace, producer, and version. Without the envelope, consumers may understand the payload but still fail to process it safely.

Field group	Purpose	NEXUS-1 example
Identity	Lets every service identify the message itself.	EventId, EventType, Version
Time	Separates when the fact happened from when it was delivered.	OccurredAtUtc, PublishedAtUtc
Trace	Connects the event to the larger flow.	CorrelationId, CausationId
Owner	Names the service that owns the fact.	Producer = RootCause
Payload	Carries the minimum public fact.	Caseld, UnitId, VerdictCode, Confidence
Contract version	Lets the contract evolve without surprise.	RootCauseVerdictIssued.v1

Envelope shape

```
public sealed class IntegrationEventEnvelope<TPayload>
{
    public string EventId { get; init; } = null!;
    public string EventType { get; init; } = null!;
    public int Version { get; init; }
    public DateTime OccurredAtUtc { get; init; }
    public string Producer { get; init; } = null!;
    public string CorrelationId { get; init; } = null!;
    public string? CausationId { get; init; }
    public TPayload Payload { get; init; } = default!;
}
```

Payload minimalism

Payload design is a discipline of restraint. The event must contain enough information for consumers to decide whether they care. It should not contain every field the producer happens to know. A large payload feels helpful on day one and becomes a coupling mechanism on day one hundred.

Payload choice	Good or risky?	Reason
Caseld, UnitId, VerdictCode	Good	Stable public identifiers and the fact itself.
Confidence and evidence identifiers	Good if public	Useful for triage without exposing full evidence internals.
Full Evidence objects	Risky	Leaks private model and may expose data consumers should not own.
Hypothesis rejection history	Usually risky	Often internal reasoning, not a public contract.
EF Core navigation graph	Wrong	Persistence shape is not a message contract.
Human-readable summary only	Incomplete	Useful for display, but too weak as machine contract.

MINIMUM USEFUL FACT

The event should be small, but not cryptic. A consumer should not need to call the producer immediately just to know whether the event matters.

Published language at the boundary

The event name and payload fields belong to the published language of the producer. That language is not an implementation detail. It is the vocabulary the producer promises to use when speaking to the outside world. In NEXUS-1, that is why the public event says `RootCauseVerdictIssued`, not `RootCauseCaseClosed`.

Private word or shape	Public alternative	Why
FinalVerdict property	VerdictCode	Contract field should be stable and coded.
RootCauseCaseClosed	RootCauseVerdictIssued.v1	Names the fact, not a lifecycle side effect.
Evidence collection objects	EvidenceIds	References evidence without exporting internal structure.
Recommendation command	RecommendationGenerated.v1	A recommendation is a published advisory outcome, not a command.
AuditEntry	Audit evidence reference	Audit is evidence, not workflow ownership.

LANGUAGE DISCIPLINE

Once an event crosses the boundary, its words belong to consumers too. That is why public words must be chosen more slowly than private class names.

Versioning an event is different from versioning code

A code change can be deployed when the team is ready. A public event change must respect every consumer that already reads it. Versioning is therefore part of the event name, schema, documentation, tests, and operational monitoring.

Change	Compatible?	Guidance
Add optional field	Usually yes	Keep old consumers working; document default meaning.
Rename field	No	Create v2 or keep both fields temporarily.
Change meaning of field	No	This is worse than a rename; create a new version.
Remove field	No for old consumers	Deprecate first; remove only after consumers migrate.
Add new event type	Yes	New fact, new subscription; old contracts remain.
Split one event into two	Usually v2 territory	Make consumer migration explicit.

VERSION RULE

A version number is not decoration. It is a promise that consumers can reason about change instead of discovering it through broken deserialization.

Consumer obligations

A public event contract is not only a producer promise. It also imposes obligations on consumers. A consumer must process messages idempotently, tolerate delay, handle unsupported versions safely, and avoid turning the event back into a synchronous dependency on the producer.

Consumer obligation	What it prevents
Store processed EventId	Duplicate delivery causing duplicate side effects.
Validate EventType and version	Accidental processing of unsupported contracts.
Use CorrelationId	Lost trace across alarm, root-cause, compliance, and reporting.
Treat producer data as source-of-truth only for that fact	Read models becoming fake owners.
Avoid immediate callback for every field	Asynchronous flow collapsing back into synchronous coupling.
Dead-letter invalid messages with reason	Silent data loss and impossible support cases.

CONSUMER RULE

Subscribing to an event is accepting responsibility. The producer should not become responsible for every consumer's failure mode.

NEXUS-1 contract example

The RootCauseVerdictIssued.v1 contract is deliberately plain. It gives downstream services enough information to react without exposing how RootCause reached the verdict. Compliance can open a review. Reporting can update a projection. The operator console can show a pending propagation state.

NEXUS-1 public payload

```
public sealed class RootCauseVerdictIssuedV1Payload
{
    public string CaseId { get; init; } = null!;
    public string UnitId { get; init; } = null!;
    public string VerdictCode { get; init; } = null!;
    public decimal Confidence { get; init; }
    public IReadOnlyList<string> EvidenceIds { get; init; } = Array.Empty<string>();
    public string IssuedBy { get; init; } = null!;
}

// Envelope.EventType = "RootCauseVerdictIssued.v1"
// Envelope.Producer = "RootCause"
```

WHAT THIS CONTRACT DOES NOT SAY

It does not command Compliance. It does not contain the RootCauseCase aggregate. It does not expose the full hypothesis history. It does not pretend that Reporting is immediately up to date.

Common mistakes

Most integration-event mistakes are not syntax mistakes. They are boundary mistakes. They either leak too much of the producer or assume too much about the consumer.

Mistake	Why it damages the architecture
Publishing internal domain events directly	Private model changes become public breaking changes.
Naming events after consumer actions	Producer becomes the hidden owner of downstream workflow.
Putting commands on the event bus and calling them events	Consumers are no longer autonomous.
Publishing enormous object graphs	Persistence and domain internals leak across services.
Ignoring version numbers	Every deployment becomes a compatibility gamble.
Treating audit entries as integration events	Evidence recording is confused with business notification.
Assuming events arrive once and in order	Normal broker behaviour becomes a production incident.

ANTI-PATTERN MEMORY

An event bus can distribute confusion faster than HTTP. The cure is not fewer messages only; it is better public language and stricter contracts.

Where this chapter connects

The book now has a clean chain. Part II selected boundaries. Part III separated data ownership. Part IV defined public APIs for direct interaction. Chapter 25 explained why messages are needed. This chapter defines what kind of message is safe to publish.

Earlier idea	Connection to integration events
Bounded context	The producer owns the language of the event.
Data ownership	The event carries facts from the source-of-truth owner.
Public API contract	An event is also a public contract, but asynchronous.
Outbox	The event must be recorded reliably with the state change.
Inbox	Consumers must process the event idempotently.
Operator console/BFF	The UI must expose freshness and propagation, not fake immediacy.

CONNECTIVITY CHECK

Integration events are the bridge between domain truth and distributed reaction. They preserve ownership only when the event remains a public fact, not a remote command.

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- Why is a domain event not automatically an integration event?
- What does the translation step protect?
- What belongs in an event envelope?
- Why should RootCauseVerdictIssued.v1 stay small?
- What does published language mean at a service boundary?
- Which event changes require a new version?
- What obligations does a consumer accept when it subscribes?
- Why is AuditEntry evidence rather than the main public workflow event?

NEXT STEP

Chapter 27 moves from event design to infrastructure. Once a service has a stable integration event, the next question is how a broker carries that event, what delivery guarantees are realistic, and which guarantees the application must provide itself.

An integration event is a public fact with a contract. It is not a leaked object, not a command in disguise, and not a shortcut around ownership.

PART V - MESSAGING AND ASYNCHRONOUS COMMUNICATION

27. Message Brokers and Delivery Guarantees

A broker gives services a place to exchange messages without calling one another directly. It does not remove failure. It only changes where failure appears: routing, acknowledgement, retries, ordering, duplication, and dead-letter handling become part of the design.

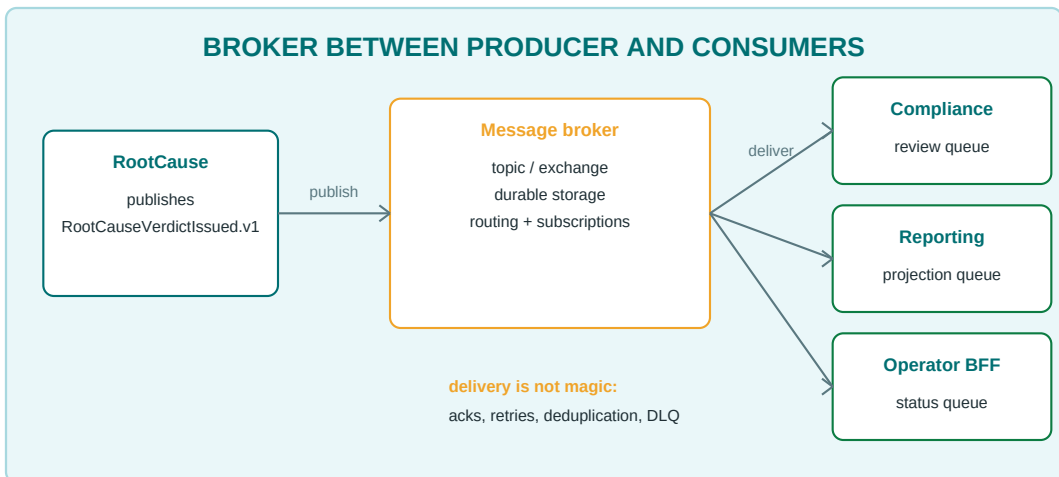


Figure 27.1 - A broker decouples producers and consumers, but delivery behaviour still has to be designed.

CHAPTER RULE

Do not ask the broker to be the architecture. The broker carries messages; the application still owns meaning, idempotency, consistency, ordering assumptions, and recovery behaviour.

Why a broker enters the architecture

Chapter 25 explained why services need messages. Chapter 26 defined the public integration event that is safe to publish. Now the system needs a runtime place where that event can be stored, routed, delivered, retried, and observed. That place is the message broker.

A broker is not used because HTTP is bad. Synchronous APIs are still useful when the caller needs an immediate answer and the callee is part of the immediate decision. A broker is used when the producer should not wait for every downstream reaction and when several consumers may need the same public fact at different speeds.

Without a broker	With a broker
RootCause calls Compliance directly.	RootCause publishes RootCauseVerdictIssued.v1.
RootCause must know every downstream consumer.	Consumers subscribe without changing the producer.
A slow reporting update can delay the producer.	Reporting catches up asynchronously.
A consumer outage becomes producer failure.	The broker can hold messages for later delivery.
Retries are hidden inside call chains.	Retries are visible as message processing behaviour.

BROKER MEMORY

A broker buys temporal decoupling. It lets the fact exist before every interested service has finished reacting to it.

Broker, queue, topic, subscription

The words are often used loosely, so the book needs a small vocabulary. A broker is the infrastructure component. A queue is usually a competing-consumer lane: one message is processed by one consumer instance. A topic or exchange is a routing surface: one published message can be copied into several subscriptions. A subscription is a consumer view of the stream.

Term	Plain meaning	NEXUS-1 use
Broker	The messaging infrastructure process or service.	The runtime that carries integration events.
Queue	A work lane consumed by one logical consumer group.	Compliance review work queue.
Topic / exchange	A publishing surface that fans messages out.	RootCause events routed to many consumers.
Subscription	A consumer-owned feed of matching messages.	Reporting subscribes to verdict events.
Routing key	A value used to decide where a message goes.	rootcause.verdict.issued.v1
Consumer group	Several instances sharing one logical workload.	Multiple Reporting workers building one projection.

NAMING DISCIPLINE

Do not hide architectural meaning behind the phrase event bus. Name the broker concept you are using: queue, topic, subscription, routing key, consumer group.

Delivery guarantees: what they promise and what they do not

Delivery guarantees are often stated as if they were simple. They are not. A guarantee is always scoped: to a queue, a partition, a broker transaction, a storage engine, or a consumer acknowledgement. A guarantee that is true inside the broker may not be true across the application, database, network, and human workflow.

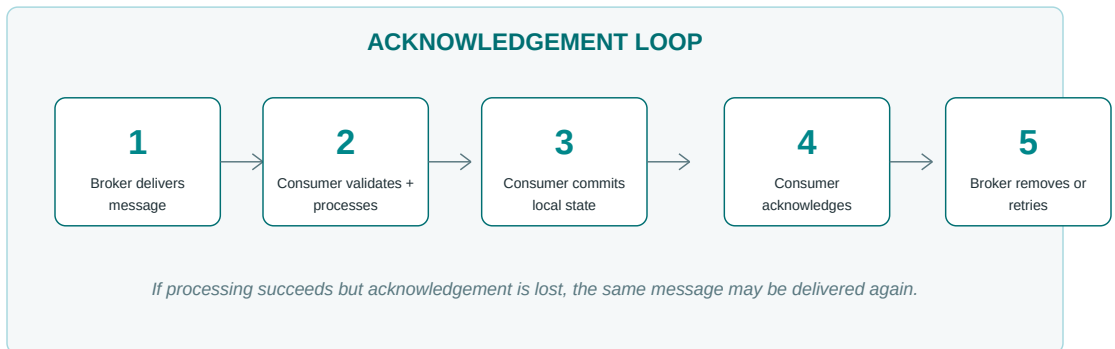
Claim	What it really means	Architectural consequence
At-most-once	The message may be lost, but will not be retried after uncertainty.	Rarely acceptable for important NEXUS-1 facts.
At-least-once	The message will be retried until acknowledged, so duplicates can happen.	Default assumption: consumers must be idempotent.
Exactly-once	Often means exactly-once within one broker/storage boundary, not across the whole workflow.	Do not design as if duplicate processing is impossible.
Ordered delivery	Usually scoped to a queue, partition, key, or single consumer lane.	Define the ordering boundary explicitly.
Durable delivery	The broker persists the message until it can be delivered or expires.	Still needs outbox, monitoring, and dead-letter handling.

DEFAULT ASSUMPTION

For serious business events, design as if delivery is at-least-once. That means the same message can arrive more than once, and processing must remain safe.

Acknowledgement is a contract

A broker usually considers a message handled only when the consumer acknowledges it. That acknowledgement is not a polite thank-you. It is a contract: the consumer is saying that it has safely done whatever local work the message required, or that it intentionally rejects the message.



This loop explains duplicates. The consumer may process the message and commit its local update, then crash before the acknowledgement reaches the broker. From the broker's point of view, the message is still unhandled. It will deliver the same message again. That behaviour is not a broker bug; it is the normal price of reliable retry.

ACK RULE

Acknowledge only after the local side effect is safely committed. Then make the side effect idempotent, because the acknowledgement itself can fail.

Retries do not fix bad contracts

Retries are useful when the failure is temporary: a database deadlock, a short network interruption, a service restart. Retries are harmful when the message is permanently invalid or when repeated processing creates repeated side effects. The broker can try again; it cannot know whether trying again is meaningful unless the application gives it rules.

Failure type	Retry?	Reason
Temporary database timeout	Yes, with backoff	The same message may succeed later.
Consumer service restarting	Yes	The consumer may be healthy in seconds.
Unknown event version	No direct retry	A code deployment or contract decision is needed.
Malformed payload	No direct retry	The message will not become valid by waiting.
Missing reference data	Maybe	Retry if replication lag is plausible; otherwise dead-letter with reason.
Duplicate EventId	No side effect	Treat as already processed and acknowledge.

RETRY MEMORY

A retry policy is not a correctness policy. It must be paired with validation, idempotency, backoff, and a dead-letter path.

Dead-letter queues are not trash cans

A dead-letter queue receives messages that could not be processed after the configured attempts or that failed validation immediately. It is tempting to treat it as a technical dump. In a serious system, the dead-letter queue is an operational evidence stream: it says where the architecture could not complete its promise.

Dead-letter record should include	Why
Original event envelope	Preserves EventId, version, producer, and trace.
Failure reason	Separates invalid payload, missing data, timeout, and consumer bug.
Consumer name	Shows which service failed to process.
Attempt count and timestamps	Supports operational diagnosis and SLA review.
Last exception category	Helps distinguish transient from permanent failures.
Manual action state	Shows whether a human has reviewed or replayed it.

OPERATIONAL RULE

If nobody watches the dead-letter queue, the system has not handled failure. It has merely moved failure to a darker room.

Ordering is always smaller than you want

Developers often ask whether the broker preserves order. The useful answer is: order of what, for whom, and under which key? Global order across a distributed system is expensive and usually unnecessary. What NEXUS-1 needs is local order where meaning depends on sequence: events for the same case, the same unit, or the same projection lane.

Ordering scope	Useful for	Cost or limitation
No ordering claim	Independent facts and best-effort projections.	Consumers must tolerate any sequence.
Per queue	Single logical consumer lane.	Throughput may be limited by one lane.
Per partition key	Events for same CaseId or UnitId.	Wrong key breaks the assumption.
Per aggregate stream	Events from one aggregate.	Requires producer discipline.
Global order	Rare audit or ledger use cases.	High cost; usually over-design for NEXUS-1.

ORDERING RULE

Name the ordering boundary explicitly. In NEXUS-1, CaseId is often a better ordering key than the whole platform.

Broker choice should follow the flow

This book does not require a particular product. RabbitMQ, Azure Service Bus, Kafka, and other systems can all carry useful architectures when used honestly. The important decision is not brand first; it is flow first. What does the NEXUS-1 event need: command-like work distribution, fan-out, replay, retention, ordered streams, or delayed retries?

Messaging need	Architectural shape	Example in NEXUS-1
Work queue	One consumer group processes each item.	Compliance review request.
Fan-out event	Several consumers receive the same fact.	RootCauseVerdictIssued.v1 to Reporting and Compliance.
Stream replay	New consumer can rebuild from retained history.	Reporting projection rebuild.
Delayed retry	Message waits before another attempt.	Temporary missing reference snapshot.
Partitioned ordering	Events ordered per key.	RootCause case lifecycle by Caseld.
Operational DLQ	Failed messages preserved for support.	Invalid contract version or poison message.

PRODUCT BOUNDARY

Tool comparison belongs later. At the design level, choose the messaging semantics first; then select infrastructure that can honestly support them.

NEXUS-1 broker topology

The first NEXUS-1 service split does not need a broker for every table or every change. It needs a small set of public facts whose consumers are genuinely independent. The RootCause verdict event is the teaching case because it has multiple consumers and no single downstream owner.

Published event	Producer	Consumers	Delivery assumption
AlarmFloodDetected.v1	AlarmManagement	RootCause, Operator BFF	At-least-once; keyed by FloodId.
RootCauseVerdictIssued.v1	RootCause	Compliance, Reporting, Operator BFF	At-least-once; keyed by Caseld.
RootCauseVerdictCorrected.v1	RootCause	Compliance, Reporting	At-least-once; ordering by Caseld matters.
RequestComplianceReview	RootCause or process manager	Compliance	Work message; one logical consumer group.
RecommendationGenerated.v1	RL Policy / advisory service	Operator BFF, Reporting	Advisory fact, not a command.

NEXUS-1 RULE

Start with a few high-value public facts. Do not turn every internal domain event into broker traffic.

Common broker mistakes

Message brokers are powerful enough to hide architectural mistakes for a while. Then the mistakes return as duplicate side effects, unbounded queues, poison messages, silent lag, and consumers nobody can safely change.

Mistake	Why it hurts
Treating the broker as a distributed transaction coordinator	The system avoids ownership decisions instead of making them.
Assuming publish means processed	The event may be durable while every consumer is still behind.
Ignoring consumer lag	The UI shows stale projections as if they were immediate truth.
Using one global queue for unrelated facts	Ordering and throughput assumptions become impossible to reason about.
No idempotency table	Duplicate delivery becomes duplicate business action.
No DLQ ownership	Poison messages accumulate without accountability.
No contract registry or tests	Consumer compatibility fails at runtime.

ANTI-HYPE MEMORY

A broker does not make a system event-driven. It only makes it capable of moving events. The architecture is in the contracts, handlers, storage, and recovery rules.

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- What problem does a broker solve in NEXUS-1?
- What is the difference between a queue and a topic or exchange?
- Why is at-least-once delivery the safest default assumption?
- Why can acknowledgement failure produce duplicate delivery?
- When should a message be retried, and when should it be dead-lettered?
- Why is ordering always scoped?
- Which NEXUS-1 events are good candidates for broker publication?
- Why does the broker not replace outbox and inbox discipline?

NEXT STEP

Chapter 28 moves from broker behaviour to producer reliability. Once the service has a broker and a public event, the next question is how to avoid the classic failure where the database commits but the message is never published. That is the job of the outbox pattern.

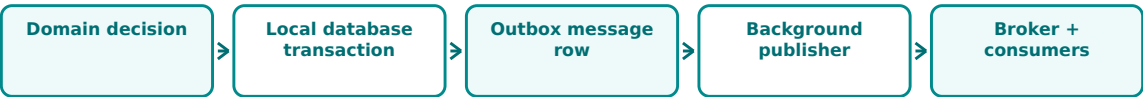
The broker is the road, not the contract. The event is the public fact, the consumer owns its reaction, and the application must still design for duplicates, delay, failure, and recovery.

PART V - MESSAGING AND ASYNCHRONOUS COMMUNICATION

28. Outbox in a Microservice Architecture

The outbox pattern is the small reliability discipline that prevents a service from saving a business decision while losing the public event that should have announced it.

VERDICT SAVED - EVENT DURABLY RECORDED - EVENT PUBLISHED LATER



The previous chapter gave the system a broker. That is not enough. The broker can carry a message only after the service has produced one. The dangerous moment sits before the broker: the moment between committing local state and publishing the event that tells the rest of the system what happened.

In NEXUS-1 the example is the RootCause verdict. If the verdict row is committed but RootCauseVerdictIssued.v1 is lost, Compliance does not open review, Reporting does not update, and the operator console cannot explain the delay. The database is true, but the system is silent. The outbox makes that silence much harder to create by storing the outgoing event in the same transaction as the decision that produced it.

CHAPTER RULE

Do not publish a critical integration event as a hopeful side effect after SaveChanges. Record the event envelope inside the same local transaction as the state change, then publish from the outbox with retries, tracing, and operational visibility.

The outbox does not remove failure. It moves failure into a place the system can see, retry, audit, and explain.

The crash window the outbox closes

The naive implementation has two separate promises: save the verdict, then publish the message. Those promises do not share one atomic boundary. A process can crash between them. A network call can fail after the database commit. A retry can publish a duplicate. A timeout can leave the application unsure whether the broker accepted the message.

Naive step	Possible failure	What the system sees
Save RootCause verdict	Database commit succeeds; process crashes before publish	RootCause is correct locally; consumers never learn the verdict.
Publish before commit	Broker accepts event; database transaction rolls back	Consumers react to a fact that is not actually true.
Publish after commit	Broker timeout leaves outcome unknown	Retry may duplicate, or silence may persist.
Retry blindly	Same event is generated with a new id	Consumers cannot know it is the same business fact.
Ignore publish failure	No event reaches Compliance or Reporting	The workflow becomes inconsistent without evidence.

PLAIN EXPLANATION

The outbox does not make the database and broker one transaction. It changes the problem: instead of trying to save state and publish to the broker at the same instant, the service saves state and the intended publication together in one local database transaction.

The outbox row is an envelope

An outbox table should not be a random JSON dump with a timestamp. It is an operational contract. A publisher must be able to find pending messages, publish them safely, retry failures, preserve ordering where the design requires it, and explain what happened later.

Column / field	Meaning	NEXUS-1 example
OutboxMessageId	Local storage identity for the row.	BIGINT identity or GUID.
EventId	Stable public event identity used for idempotency.	rcv-2026-0418-v1.
EventType	Public contract name and version.	RootCauseVerdictIssued.v1.
OccurredAtUtc	When the domain fact became true.	Verdict issued time.
CorrelationId	Trace across command, case, event, consumers.	Alarm-to-compliance trace id.
AggregateId / CaseId	Ordering and ownership key.	RootCause case id.
PayloadJson	Serialized public contract, not aggregate state.	RootCauseVerdictIssuedV1.
Status / Attempts	Publisher lifecycle and retry evidence.	Pending, Publishing, Published, Failed.

ENVELOPE RULE

Store enough metadata to operate the message without deserializing the whole payload. The payload is for consumers; the envelope is for routing, tracing, retries, diagnostics, and audit.

Saving the decision and the event together

The application handler should treat the domain decision and the outbox record as one unit of local work. The aggregate issues the verdict. The application layer translates the internal fact into the public contract. The same transaction persists both the aggregate state and the outbox envelope.

```
public async Task Handle(IssueVerdict command, CancellationToken ct)
{
    var rootCauseCase = await cases.Load(command.CaseId, ct);

    // Domain decision: private model, private rules.
    var verdictIssued = rootCauseCase.IssueVerdict(
        command.VerdictCode,
        command.Confidence,
        command.EvidenceIds);

    // Public contract: stable language outside RootCause.
    var contract = RootCauseVerdictIssuedV1.From(
        verdictIssued,
        correlationId: command.CorrelationId);

    await db.OutboxMessages.AddAsync(
        OutboxMessage.FromIntegrationEvent(contract), ct);

    await db.SaveChangesAsync(ct); // case state + outbox row together
}
```

TRANSLATION BOUNDARY

The outbox row should contain the public integration event, not the private domain event object. `VerdictIssued` may be a private in-memory event; `RootCauseVerdictIssued.v1` is the contract consumers are allowed to know.

The publisher is a reliability component

The background publisher is not a casual hosted service that loops over rows. It is the component that turns durable local intent into external publication. It must lock or claim rows safely, publish with the correct envelope, mark success, classify failure, and leave enough evidence for operators to understand lag.

```
while (!stoppingToken.IsCancellationRequested)
{
    var batch = await outbox.ClaimPendingBatchAsync(limit: 100, ct);

    foreach (var message in batch)
    {
        try
        {
            await broker.PublishAsync(
                message.EventType,
                message.PayloadJson,
                message.Headers,
                ct);

            message.MarkPublished(clock.UtcNow);
        }
        catch (Exception ex) when (IsTransient(ex))
        {
            message.MarkRetry(ex, clock.UtcNow);
        }
        catch (Exception ex)
        {
            message.MarkFailed(ex, clock.UtcNow);
        }
    }

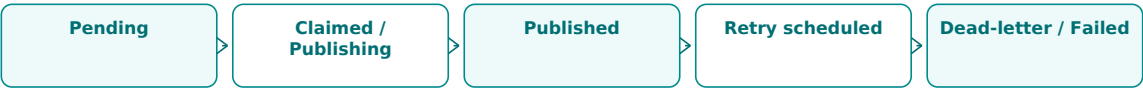
    await db.SaveChangesAsync(ct);
}
```

PUBLISHER RULE

The publisher must be restart-safe. If it dies halfway through a batch, pending or claimed messages must be discoverable again, and already-published messages must not be treated as new business facts.

Publishing is a state machine

OUTBOX MESSAGE LIFECYCLE



A state machine avoids vague operational language. A message is not merely handled or unhandled. It has a lifecycle. Each state answers a support question: is it waiting, being worked, already sent, delayed for retry, or stopped because the system needs human attention?

State	Meaning	Allowed next step
Pending	Recorded with the domain transaction; not yet claimed.	Claim for publishing.
Publishing	A publisher owns the current attempt.	Published, Retry, or Failed.
RetryScheduled	A transient failure occurred; do not hammer the broker.	Pending after backoff.
Published	Broker accepted the message.	Retention or archive later.
Failed / DeadLetter	Permanent failure or retry limit exceeded.	Human review, fix, replay, or abandon with evidence.

STATE RULE

Do not hide the outbox behind a boolean Published flag if the system needs serious operation. Status, attempt count, last error category, next retry time, and timestamps are part of the reliability design.

Producer idempotency matters too

Most discussions of idempotency focus on consumers. The producer also has responsibility. If the same command is retried, the service must not create two different public events for one business fact. The EventId should identify the fact, not the attempt.

Producer risk	Bad result	Outbox discipline
Same command retried after timeout	Two verdict events with different EventIds.	Use command idempotency and stable event creation.
Publisher retries after unknown broker response	Same payload may be delivered twice.	Consumers use EventId; producer does not mutate payload.
Operator corrects a verdict later	Correction confused with duplicate original event.	Publish RootCauseVerdictCorrected.v1 as a new fact.
Event regenerated from changed aggregate	Payload drift between attempts.	Persist payload at outbox creation time.
Manual replay uses new correlation	Trace breaks across services.	Preserve original EventId and CorrelationId unless publishing a new fact.

FACT IDENTITY RULE

A retry is not a new business fact. A correction is. RootCauseVerdictIssued.v1 and RootCauseVerdictCorrected.v1 must be different facts, not two accidental shapes of the same failure recovery.

Ordering begins at the outbox

The broker can only preserve an order the producer makes visible. If events for the same RootCauseCase matter in sequence, the outbox needs an ordering key and, where appropriate, an aggregate sequence. Without those values, the broker cannot know which messages belong together.

Ordering field	What it protects	Example
AggregateType	Separates case events from other facts.	RootCauseCase.
AggregateId	Groups events that must be reasoned about together.	CasId RCA-2026-0418.
AggregateSequence	Defines producer-side order for one aggregate.	Verdict issued before verdict corrected.
OccurredAtUtc	Explains when the fact became true.	Human-readable audit timeline.
OutboxMessageId	Storage order, useful but not domain order.	Do not confuse identity with meaning.
PartitionKey	Broker routing key for scoped ordering.	CasId or UnitId, not entire platform.

ORDERING RULE

If NEXUS-1 cares about order for a root-cause case, use CasId as the ordering boundary. Do not demand global order when local order is the real requirement.

Retention, cleanup, and audit

An outbox table will grow forever unless the architecture gives it a retention policy. But deleting too aggressively removes evidence needed for diagnosis, replay, and audit. The policy should distinguish operational retry data from long-term event evidence.

Data	Operational need	Retention choice
Pending messages	Must remain until published or failed.	Never delete while actionable.
Published envelopes	Useful for replay window and support.	Keep for configured period, then archive or compact.
Payload JSON	May contain business data.	Protect with data classification and retention rules.
Failure details	Needed to diagnose poison messages.	Keep at least through incident review.
Trace metadata	Needed for audit chain and correlation.	Preserve longer than transient logs.
High-volume event streams	May become expensive.	Archive to cold storage or event log if needed.

AUDIT BOUNDARY

The outbox is not a substitute for the Audit context. It is publication evidence. Audit may record the decision and trace; the outbox records the intent and result of publication.

NEXUS-1 RootCause outbox design

For the first service extraction, the RootCause service is the clean teaching case. It owns RootCauseCase, Evidence, Hypothesis, and Verdict. When the verdict becomes final, the service publishes RootCauseVerdictIssued.v1. Compliance, Reporting, and the Operator BFF react later.

Design decision	NEXUS-1 choice	Reason
Producing service	RootCause service	RootCause owns the verdict language and rule.
Internal domain event	VerdictIssued	Private event inside the model.
Public integration event	RootCauseVerdictIssued.v1	Stable consumer contract.
Ordering key	Caseld	Case lifecycle order matters locally.
Correlation id	Alarm-to-compliance trace id	Connects alarm, case, verdict, review, report.
Consumers	Compliance, Reporting, Operator BFF	They observe and react; they do not own the verdict.
Outbox owner	RootCause database	The producer owns its publication evidence.

NEXUS-1 RULE

RootCause does not wait for Compliance to be available before saving a verdict. It saves the verdict and the outbox event together; then the publisher and consumers complete the cross-service consequences.

Operational checks and common mistakes

The outbox is only useful if operations can see it. A growing pending count, repeated failures, old unpublished messages, or replayed poison events are not implementation details. They are signals that the distributed system is no longer keeping its promises.

Check or mistake	What to watch	Why it matters
Oldest pending message age	Seconds or minutes since creation.	Shows publication lag.
Retry count by event type	Repeated transient or permanent failure.	Separates broker issue from bad contract.
No unique EventId	Duplicate business facts.	Breaks consumer idempotency.
No claim timeout	Stuck Publishing rows after crash.	Publisher restart cannot recover cleanly.
Payload built at publish time	Event changes after the decision.	Consumers receive a different fact.
Outbox table unbounded	Storage and query slowdown.	Retention was not designed.
No correlation id	Trace ends at service boundary.	Operator explanation becomes fragmented.

OPERATIONAL RULE

A hidden outbox is a half-built outbox. The table needs metrics, alerts, replay tooling, retention rules, and ownership just like any other reliability component.

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- Why is publishing after SaveChanges a dangerous two-step promise?
- What failure does the outbox pattern close?
- Why should the outbox store the public integration event instead of the private domain event?
- Which fields belong in an outbox envelope besides the payload?
- Why must EventId be stable across retries?
- How does the outbox support ordering for one RootCauseCase?
- Why is the outbox not the same thing as the Audit context?
- Which operational metrics prove that the outbox is healthy?

NEXT STEP

Chapter 29 turns to the consumer side. Once a producer can publish reliably, consumers must prove they can receive the same event twice without doing the business work twice. That is the job of the inbox pattern and idempotent consumers.

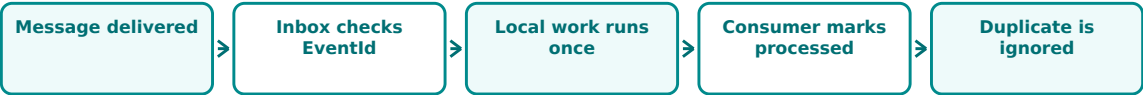
Outbox discipline is the producer half of reliable messaging: save the fact, save the event, publish later, and leave evidence for every step.

PART V - MESSAGING AND ASYNCHRONOUS COMMUNICATION

29. Inbox and Idempotent Consumers

A broker may deliver the same event twice. A serious consumer must treat duplicate delivery as normal distributed behaviour, not as an excuse for duplicate business work.

RECEIVE - RECORD - DECIDE - PROCESS - REMEMBER



Chapter 28 made the producer reliable enough to save its decision and its outgoing event together. That solves only half of the problem. The event now reaches another service, and the receiving service must prove that it can handle the same fact more than once without creating the consequence more than once.

In NEXUS-1, the example is Compliance receiving `RootCauseVerdictIssued.v1`. If that event is delivered twice, Compliance must not open two compliance reviews for the same verdict. Reporting must not count the same verdict twice. The operator console must not show two different stories for one fact. This chapter is about the consumer-side discipline that makes those promises believable.

CHAPTER RULE

A consumer is not safe because the broker promises delivery. A consumer is safe when it can receive the same integration event again and either do no work or repeat the work without changing the final business meaning.

The outbox protects the producer from silent publication loss. The inbox protects the consumer from accidental repetition.

Duplicate delivery is not a rare edge case

A message broker that delivers at least once is making a practical promise: it will prefer repetition over loss. That is usually the correct trade-off. Losing `RootCauseVerdictIssued.v1` would hide a verdict from Compliance. Delivering it twice is annoying, but survivable if the consumer is designed correctly.

Cause of duplicate	What happened	Consumer requirement
Publisher retry after timeout	The broker accepted the first send, but the producer did not receive the answer.	Treat the same EventId as the same fact.
Consumer crash after local work	Business rows were saved, but the broker acknowledgement was not sent.	Detect that the event was already processed.
Broker redelivery	The acknowledgement was late, lost, or the consumer group rebalanced.	Do not assume first delivery is the only delivery.
Manual replay	An operator replays a historical event to rebuild a projection.	Make replay deliberate and traceable.
Poison-message recovery	A fixed consumer reprocesses earlier failed messages.	Separate retry from new business intent.

NORMAL BEHAVIOUR

Duplicate delivery is not proof that the messaging system is broken. It is a normal consequence of choosing at-least-once delivery. The broken part would be a consumer that cannot tolerate it.

What idempotent means in business terms

Idempotent is sometimes described as doing the same operation many times and getting the same result. That definition is useful, but it can sound too mathematical. In application architecture the simpler question is this: if this consumer sees the same event again, will the business meaning change?

Operation	Not idempotent	Idempotent version
Open compliance review	INSERT a new review row every time the event arrives.	Create review using VerdictEventId as a unique business key.
Update reporting counter	Increment TotalVerdicts by 1 for every delivery.	Upsert a verdict projection keyed by EventId or VerdictId.
Append audit note	Write an indistinguishable duplicate note.	Write one audit fact per event identity, or mark replay explicitly.
Notify operator	Send a fresh alert on every redelivery.	Notify once, or display current status from the read model.
Cache recommendation status	Overwrite with no identity check.	Update only the row for the same source event and version.

PLAIN RULE

Idempotency is not a messaging feature alone. It is a business guarantee: the same fact is not allowed to create a second business consequence unless the contract explicitly says a second fact has occurred.

The inbox table

An inbox is the consumer-side memory of integration events. It records which public event identities the service has received and processed. The table does not own the domain fact. It owns the evidence that this consumer has already reacted to that fact.

Column / field	Meaning	NEXUS-1 example
InboxMessageId	Local storage identity for the consumer row.	BIGINT identity or GUID.
EventId	Stable public event identity from the producer.	rcv-2026-0418-v1.
ConsumerName	Which consumer owns this processing record.	Compliance.RootCauseVerdictConsumer.
EventType	Contract name and version.	RootCauseVerdictIssued.v1.
CorrelationId	Trace across services.	Alarm-to-compliance trace id.
ReceivedAtUtc	When this service first saw the event.	Consumer receipt timestamp.
ProcessedAtUtc	When local work completed.	Review opened timestamp.
Status / Attempts	Processing lifecycle.	Received, Processing, Processed, Failed.
PayloadHash	Optional tamper / replay comparison.	Hash of payload JSON.

UNIQUE CONSTRAINT

At minimum, the inbox should prevent the same EventId from being processed twice by the same logical consumer. In SQL terms, that usually means a unique constraint on ConsumerName + EventId.

Processing inside one local transaction

The inbox row and the local business consequence should be committed together inside the consumer service database. This is not a distributed transaction with the producer. It is a local promise: either this service remembers the event and saves its reaction, or it saves neither.

```
public async Task Consume(RootCauseVerdictIssuedV1 message, CancellationToken ct)
{
    if (await inbox.AlreadyProcessed(
        consumer: ConsumerNames.ComplianceVerdict,
        eventId: message.EventId,
        ct))
        return;

    await using var tx = await db.BeginTransactionAsync(ct);

    await inbox.RecordReceived(message, ConsumerNames.ComplianceVerdict, ct);

    var review = ComplianceReview.OpenForRootCauseVerdict(
        verdictEventId: message.EventId,
        rootCauseCaseId: message.RootCauseCaseId,
        unitId: message.UnitId,
        correlationId: message.CorrelationId);

    db.ComplianceReviews.Add(review);

    await inbox.MarkProcessed(message.EventId, ConsumerNames.ComplianceVerdict, ct);
    await db.SaveChangesAsync(ct);
    await tx.CommitAsync(ct);
}
```

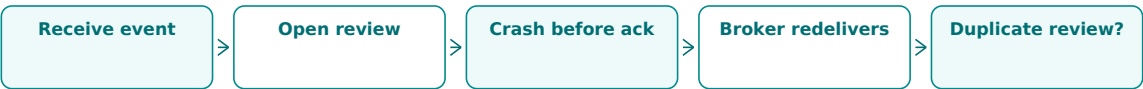
LOCAL TRANSACTION RULE

The consumer cannot make the producer transaction part of its own transaction. It can only make its own reaction and its own inbox evidence atomic inside its own database.

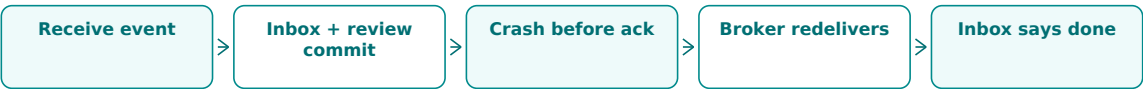
The crash window moves to the consumer

Without an inbox, the consumer has its own two-step danger: do the business work, then acknowledge the broker. If the service crashes after the work but before the acknowledgement, the broker redelivers the message. A naive consumer repeats the work. An idempotent consumer recognizes the event and returns safely.

WITHOUT INBOX: THE CONSUMER CRASH WINDOW



WITH INBOX: DUPLICATE DELIVERY BECOMES SAFE



The inbox does not stop the broker from redelivering. It changes what redelivery means. The second delivery becomes a harmless observation: this event has already produced the local consequence for this consumer.

ACK BOUNDARY

Broker acknowledgement and database commit are not the same thing. The inbox is the consumer memory that survives the gap between them.

NEXUS-1: Compliance opens one review

The cleanest example is Compliance reacting to a root-cause verdict. The event is public: `RootCauseVerdictIssued.v1`. Compliance does not load the `RootCause` aggregate and does not ask `RootCause` whether it may create a review. The published contract is enough. But Compliance must treat the event identity as part of its own rule.

Element	Consumer-side choice	Reason
Source event	RootCauseVerdictIssued.v1	Public fact from RootCause.
Consumer	Compliance service	Owns review workflow and controls.
Business key	VerdictEventId or RootCauseCaseId + VerdictVersion	Prevents duplicate review rows.
Inbox uniqueness	ConsumerName + EventId	Prevents duplicate event processing.
Local result	ComplianceReview	New local truth owned by Compliance.
Trace	CorrelationId carried forward	Connects alarm, case, review, reporting.
Duplicate delivery	No new review; maybe update last-seen metadata	Second delivery is acknowledged safely.

OWNERSHIP RULE

The event tells Compliance that a verdict exists. It does not tell Compliance how to conduct the review. Compliance owns that local workflow and its own idempotency rule.

Reporting consumers are different

Reporting usually builds read models rather than opening business workflows. That changes the idempotency strategy, but not the requirement. A read model consumer may use upsert, replacement, version comparison, or projection rebuild. It still must avoid counting the same public fact twice.

Consumer type	Typical local work	Idempotency strategy
Compliance workflow	Open a review or attach a case to an existing review.	Unique business key + inbox. Detect duplicate before creating work.
Reporting projection	Update a dashboard row or materialized view.	Upsert by event identity, case id, or version. Avoid blind increments.
Operator BFF cache	Refresh cross-service status shown to the UI.	Replace current view by source version and correlation id.
Audit observer	Record evidence that an event was observed.	One audit fact per EventId, or explicit replay flag.
Notification worker	Send a human-facing alert.	Dedupe by EventId and notification channel.

PROJECTION RULE

Do not implement reporting projections as blind counters unless the event stream is already deduplicated for that consumer. Upsert is usually safer than increment.

The exactly-once illusion

Teams often ask whether a broker can give exactly-once delivery. The more useful question is whether the whole business outcome is exactly once. Across a broker, a consumer process, a consumer database, a network, and an acknowledgement, the safe design does not depend on that fantasy. It assumes redelivery and makes redelivery safe.

Claim	Why it is dangerous	Safer wording
The broker guarantees exactly once.	The consumer database and business operation still have crash windows.	The broker may reduce duplicates; the consumer must remain idempotent.
The message was acknowledged, so work happened.	Ack is transport evidence, not domain evidence.	Use local business rows and inbox records as evidence.
A unique constraint is enough.	It may prevent duplicates but still hide errors if not handled deliberately.	Use unique constraints plus explicit duplicate handling.
Retry means do it again.	Retrying a duplicate fact may create duplicate consequences.	Retry means attempt safe processing again.
Idempotency is only technical.	The business key decides what counts as the same fact.	Design idempotency with domain language.

HONEST BOUNDARY

This book does not claim exactly-once distributed business processing. It designs for at-least-once delivery plus idempotent consumers, which is the discipline that survives ordinary failure.

Testing idempotent consumers

An idempotent consumer is not proven by reading its code. It is proven by the duplicate-delivery test. Feed the same message twice and assert that the local business consequence exists once. Then crash or simulate failure between important steps and prove that recovery does not corrupt the local model.

```
[Fact]
public async Task RootCauseVerdictIssued_Opens_One_ComplianceReview()
{
    var message = RootCauseVerdictIssuedV1Builder.Valid()
        .WithEventId("evt-rootcause-0418-verdict-1")
        .Build();

    await consumer.Consume(message, CancellationToken.None);
    await consumer.Consume(message, CancellationToken.None);

    var reviews = await db.ComplianceReviews
        .Where(r => r.VerdictEventId == message.EventId)
        .ToListAsync();

    reviews.Should().ContainSingle();

    var inboxRows = await db.InboxMessages
        .Where(i => i.EventId == message.EventId &&
            i.ConsumerName == ConsumerNames.ComplianceVerdict)
        .ToListAsync();

    inboxRows.Should().ContainSingle();
}
```

TEST RULE

Every consumer that performs non-trivial business work should have a duplicate-delivery test. If the test creates two business consequences, the consumer is not ready for production messaging.

Checklist for NEXUS-1 consumers

Before a NEXUS-1 service consumes an integration event, the team should be able to answer the following questions. They are deliberately practical because idempotency failures are usually practical: one duplicate row, one duplicate email, one duplicate review, one misleading dashboard count.

Question	Good answer for NEXUS-1
What is the event identity?	EventId from the public contract, not a newly generated consumer id.
Who is the logical consumer?	A stable ConsumerName such as Compliance.RootCauseVerdictConsumer.
What is the local consequence?	Review, projection row, cache state, notification, or audit observation.
What key prevents duplicate consequence?	A unique local business key tied to the event or source fact.
Can the local work and inbox commit together?	Yes, inside the consumer service database transaction.
What happens on duplicate delivery?	Return safely, update last-seen metadata, or replay explicitly; no new business fact.
How is failure visible?	Inbox status, attempts, last error, age, and correlation id.
How is replay controlled?	Manual replay preserves EventId and is logged as operational action.

READINESS RULE

A consumer that cannot explain its idempotency key is not ready to consume public integration events in a microservice architecture.

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- Why does at-least-once delivery require consumer idempotency?
- What is the difference between an outbox and an inbox?
- Why is ConsumerName + EventId a useful uniqueness boundary?
- Why must the inbox record and local business consequence commit together?
- How does Compliance avoid opening two reviews for one RootCause verdict?
- Why are reporting projections often better implemented with upsert than with blind increments?
- Why is exactly-once delivery not a sufficient architecture promise?
- Which duplicate-delivery test would you write for a new NEXUS-1 consumer?

NEXT STEP

Chapter 30 moves from duplicate delivery to ordering. Once a consumer can survive repeated messages, it must learn which ordering assumptions are safe, which are local, and which should never be made globally.

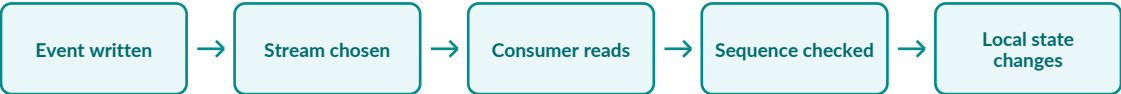
Idempotency is the consumer half of messaging honesty: the same fact may arrive again, but it must not become a second truth.

PART V - MESSAGING AND ASYNCHRONOUS COMMUNICATION

30. Ordering, Duplicates, and Lost Assumptions

A broker can preserve order only inside a boundary. A serious service does not assume that all messages in the platform arrive in the same sequence, or that a duplicate cannot arrive after the newer fact.

LOCAL ORDER - DUPLICATE TOLERANCE - SEQUENCE GUARD - CLEAR ASSUMPTIONS



One message path, not one universal order

Chapter 29 made the consumer safe against repeated delivery. This chapter adds the next problem: the repeated message may not be the oldest message. A consumer may receive version 4, then receive version 3 again. It may receive an event for one aggregate before another event that a human thinks should have happened first. It may process a correction after the original verdict has already reached reporting.

The naive promise is tempting: put everything on one broker, publish everything in timestamp order, and the system will behave like one large transaction log. That promise breaks as soon as services partition work, retry failed deliveries, scale consumers, or publish from separate outboxes. Distributed systems do not give one global story for free. They give many local stories, and each service must state which one it depends on.

CHAPTER RULE

Do not ask the whole platform to preserve one universal order. Name the stream that must be ordered, preserve order only there, and make every consumer safe against duplicates and older messages arriving late.

Ordering is useful only when the boundary that protects it is named.

The ordinary developer trap

Most developers first meet ordering through a queue that behaves politely in a demo. Message A is sent, then Message B is sent, and the consumer prints A before B. The sample works, so the team quietly converts that observation into an architecture rule: the system is ordered.

The rule was never true at that scale. The demo proved only one narrow fact: one producer, one queue, one partition, one consumer, no retry delay, no parallelism, and no competing outbox publisher happened to produce the expected sequence. Change any one of those conditions and the visible sequence can change.

Assumption	Why it feels safe	What distribution changes
Send order equals process order	It works in a single-threaded demo	Parallel consumers and retries move work around
Timestamp order is business order	Clocks look precise	Clocks drift and business causality is not a clock
One queue means one sequence	The queue has a front and back	Partitions, topics, and subscriptions create many sequences
Duplicate means harmless repeat	The event looks the same	The duplicate may arrive after newer local state
Global order solves everything	It sounds clean	It slows the system and still may not match domain causality

The lesson is not that ordering is impossible. The lesson is that ordering is always scoped. The first architectural task is to identify where order is required and where the system only needs convergence.

PLAIN EXPLANATION

Ordering is not a property of the whole platform. It is a property of a stream, a partition, an aggregate, a consumer, or a projection. If you cannot name the boundary, you do not own the ordering assumption.

What actually needs to be ordered?

NEXUS-1 does not need every event in the plant to stand in one line. An alarm acknowledgement on Unit 2 does not need to wait behind a reporting projection for Unit 1. A training-run episode in ReinforcementLearning does not need to be ordered with a compliance review assignment. Forcing those unrelated facts through one global order would make the architecture slower without making it more correct.

The useful question is smaller: which facts belong to the same decision history? A RootCauseCase has a history. Evidence is attached, hypotheses are rejected or promoted, a verdict is issued, and sometimes a correction follows. Inside that case history, order matters. Outside it, order is mostly an illusion of convenience.



Figure 30.1 - Each stream owns its own ordering assumptions; the whole platform does not share one sequence.

The diagram does not describe one total sequence. It describes several streams that cross a business flow. AlarmManagement has its own event history. RootCause has a case history. Compliance has a review history. Reporting has projection updates. Audit has append-only evidence. Each stream can be ordered locally without pretending that the entire platform shares a single clock.

NEXUS-1 RULE

Preserve order per Unit, per AlarmFlood, per RootCauseCase, or per ComplianceReview when the rule depends on that sequence. Do not preserve order merely because the events appear near each other in a diagram.

Ordering is useful only when the boundary that protects it is named.

Ordering by aggregate, not by platform

A practical microservice rule follows naturally from DDD: the aggregate is often the smallest meaningful ordering boundary. If two events change the same RootCauseCase, their order matters because the aggregate protects a rule. If two events belong to different cases, a global order between them is rarely part of the domain.

This is why many messaging systems ask for a partition key or routing key. The key is not just a performance detail. It is a statement about what must be processed sequentially. Use RootCauseCaseId as a key, and all events for one case can be kept together. Use UnitId, and all unit-level events can be kept together. Use no meaningful key, and the system has avoided making the assumption visible.

Stream key	Useful when	Risk if misused
RootCauseCaseId	Evidence, hypothesis, verdict, and correction events must follow in that order	Events that should be grouped by flood or unit may scatter
AlarmFloodId	A flood is opened, enriched, classified, and closed	Separate root-cause cases may be forced into one order
UnitId	Operator console wants a unit-level operational timeline	High-volume units can become hot partitions
ComplianceReviewId	Review state changes must be applied in sequence	It does not order the upstream root-cause decision
EventId only	Good for deduplication	Bad for business ordering because every event becomes its own

The key must be chosen by the model, not by infrastructure habit. A partition key that is convenient for throughput but wrong for the domain creates a fast stream of subtle bugs.

Duplicates and old messages are different problems

Chapter 29 treated duplicate delivery as a repeat of the same event. That is only the first half of the problem. A consumer also has to notice when the event is not new enough to change the local state. A duplicate says: I have seen this exact fact before. An old message says: I may or may not have seen this exact event, but the fact it carries is older than what I already know.

For example, Compliance receives `RootCauseVerdictIssued.v1` and opens a review. Later it receives `RootCauseVerdictCorrected.v1`. The correction becomes the newer truth for that review. If the original verdict is delivered again after the correction, the inbox can reject the duplicate if the `EventId` is the same. But if the original verdict is republished with a new `EventId`, idempotency alone is not enough. The consumer needs a version, sequence number, effective timestamp, or domain-specific rule that says whether this older fact may still change anything.

Consumer check	Question answered	Example
EventId uniqueness	Have I processed this exact message?	Ignore the same <code>RootCauseVerdictIssued.v1</code> delivery
Aggregate version	Is this fact newer than my local version?	Apply case version 7 after version 6, reject version 5
Effective timestamp	Which fact is valid for this business interval?	A corrected verdict supersedes earlier explanation from a stat
Business state rule	Is this transition still allowed?	A closed compliance review may not reopen from an old even
Projection upsert	Can I set the final state rather than increment blindly?	Reporting stores current verdict status, not count +1

IMPORTANT DISTINCTION

Idempotency protects against repeated messages. Sequence and version checks protect against stale meaning. A serious consumer usually needs both.

A small C# shape

The following shape is deliberately small. It is not a broker framework. It is the kind of consumer-side guard a NEXUS-1 service would place before changing local state. The handler first checks whether the event was processed. Then it checks whether the event version is older than the local projection. Only then does it apply the change.

```
public sealed class VerdictProjectionConsumer
{
    private readonly IInboxStore _inbox;
    private readonly IVerdictProjectionStore _projections;

    public async Task Handle(RootCauseVerdictIssuedV1 message)
    {
        if (await _inbox.WasProcessed(message.EventId))
            return; // duplicate delivery, no business work

        var current = await _projections.Find(message.RootCauseCaseId);

        if (current is not null && message.CaseVersion <= current.CaseVersion)
        {
            await _inbox.MarkProcessed(message.EventId, "stale-version");
            return; // not duplicate, but older than local truth
        }

        await _projections.Upsert(new VerdictProjection(
            message.RootCauseCaseId,
            message.CaseVersion,
            message.VerdictCode,
            message.IssuedAtUtc));

        await _inbox.MarkProcessed(message.EventId, "applied");
    }
}
```

The interesting line is not the upsert. The interesting line is the comparison between `message.CaseVersion` and `current.CaseVersion`. That line admits a fact most messaging diagrams hide: the consumer may see a message that is valid, well-formed, signed, and still too old to apply.

The database also needs to know

A consumer cannot rely only on in-memory checks. If the process restarts after applying the business change but before recording the inbox status, the service may repeat work. If two consumer instances race each other, both may believe they are first. The database must carry the uniqueness and version rules that the code claims to enforce.

```
CREATE TABLE Messaging.InboxMessage (  
  ConsumerName      nvarchar(160) NOT NULL,  
  EventId           uniqueidentifier NOT NULL,  
  StreamKey         nvarchar(120) NOT NULL,  
  StreamVersion     bigint NULL,  
  ReceivedAtUtc     datetime2(7) NOT NULL,  
  ProcessedAtUtc    datetime2(7) NULL,  
  Outcome           nvarchar(40) NULL,  
  CONSTRAINT PK_Messaging_InboxMessage  
    PRIMARY KEY (ConsumerName, EventId)  
);  
  
CREATE TABLE Reporting.RootCauseVerdictProjection (  
  RootCauseCaseId   uniqueidentifier NOT NULL,  
  CaseVersion       bigint NOT NULL,  
  VerdictCode       nvarchar(80) NOT NULL,  
  IssuedAtUtc       datetime2(7) NOT NULL,  
  CONSTRAINT PK_Reporting_RootCauseVerdictProjection  
    PRIMARY KEY (RootCauseCaseId)  
);
```

The primary key on `ConsumerName` and `EventId` is the duplicate guard. The `CaseVersion` stored in the projection is the stale-message guard. Together they let the service survive a broker that redelivers, retries, and occasionally exposes an older message after a newer projection has already been written.

IMPLEMENTATION WARNING

Do not place the sequence rule only in application memory. If the ordering assumption protects data, the database must be able to enforce or verify it after a restart.

Ordering is useful only when the boundary that protects it is named.

Lost assumptions in NEXUS-1

A lost assumption is a rule the team believed but never wrote down. Ordering bugs are often lost-assumption bugs. The team assumes the compliance service will always learn about a verdict after the root-cause case is complete. It assumes reporting will never see a correction before the original. It assumes audit can be read as workflow state. It assumes time order is the same as causal order. None of those assumptions is safe until it is named and handled.

Lost assumption	Failure shape	Safer design
A verdict always arrives before its correction	Correction is processed first and the original overrules it	Unit test for correction and correction semantics
Audit order is workflow order	Audit entries are used to infer current business state	Step audit as evidence; use service-owned state for decisions
Timestamps create causal order	Clock or delay makes a later fact look earlier	Use version or domain transition rules
One event means one reaction	A duplicate creates a second compliance task	Inbox plus idempotent command handling
Reporting can increment counters blindly	Duplicate or older event inflates numbers	Upsert state or recompute projection from owned facts

The value of this chapter is not a new tool. It is the discipline of turning invisible assumptions into explicit contracts: stream key, event version, consumer outcome, local transition rule, and projection shape.

When ordering should not be preserved

A common overcorrection is to make everything sequential. The result is a system that is slower, less available, and still not necessarily more correct. Ordering is expensive because it often reduces parallelism. It can create hot partitions. It can block unrelated work behind a poisoned message. It can make one slow consumer look like a whole-platform problem.

For NEXUS-1, Reporting does not need every alarm event to be processed in exact global order before it can update a dashboard tile. The operator console can show a status as of a timestamp. It can label a projection as catching up. It can replace state with newer snapshots. It does not need to serialize the entire plant merely to avoid explaining eventual consistency to the user.

Use ordering when	Prefer convergence when
An aggregate transition would be invalid if applied out of sequence	The consumer builds a read model that can be recomputed or upserted
A correction supersedes an earlier fact for the same case	Events belong to different units, cases, or reviews
A workflow step cannot legally happen before another	The UI can show as-of time and refresh when the projection catches up
The domain has a versioned history	The domain only needs final current state

COST RULE

Ordering is a design cost. Pay it where the domain requires it; do not pay it everywhere merely to make the distributed system feel like a monolith.

See it in the recurring flow

Return to the book's recurring flow. AlarmManagement detects an alarm flood and publishes AlarmFloodDetected.v1. RootCause opens a case, attaches evidence, and later publishes RootCauseVerdictIssued.v1. Compliance reacts to the verdict. Reporting builds a read model. Audit records evidence along the way.

A diagram can make that look like one neat line. The runtime is not one neat line. RootCause may publish a verdict while Reporting is still catching up on an earlier alarm projection. Compliance may fail and retry. Audit may record evidence before the read model has refreshed. A correction may arrive after the original verdict has already reached several consumers. None of this is scandalous. It is the normal price of distributing a flow that used to live inside one process.



Business order must be protected where decisions depend on it.

The service design becomes honest when each edge states its promise. AlarmFloodDetected.v1 does not promise that all alarm projections are current. RootCauseVerdictIssued.v1 does not promise that every downstream service has reacted. RequestComplianceReview does not become a command to the operator console. Each contract says what happened, who owns it, and what a consumer may safely infer.

Checklist for Chapter 30

Before accepting a NEXUS-1 service design, the architect should be able to answer the following ordering questions without hand-waving.

- What is the stream key: UnitId, AlarmFloodId, RootCauseCaseId, ComplianceReviewId, or something else?
- Does the consumer require strict sequence, or only eventual convergence?
- Which field tells the consumer whether a message is stale?
- Is the duplicate guard separate from the stale-message guard?
- Can the projection be updated by upsert instead of increment?
- What happens if a correction arrives before the original event is retried?
- Which assumptions belong in the public contract and which remain private to the consumer?
- Can a poisoned message block unrelated work, or is the partition too wide?

HONEST BOUNDARY

This chapter does not claim that ordering is solved by a single pattern. Brokers, partitions, versions, inbox records, database constraints, and domain rules all participate. The architectural achievement is not perfect order; it is knowing exactly which order matters and which order does not.

Ordering is useful only when the boundary that protects it is named.

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- Why is global ordering usually the wrong promise for a microservice architecture?
- What is the difference between duplicate delivery and stale delivery?
- Why is an aggregate or stream key often a better ordering boundary than a whole platform?
- When would you choose `RootCauseCaseId` instead of `UnitId` as the ordering key?
- Why can timestamp order be different from business causality?
- Why should the database store both inbox status and local projection version?
- When is it better to let a read model converge than to force strict order?
- Which ordering assumption would you write explicitly for `RootCauseVerdictCorrected.v1`?

NEXT STEP

Chapter 31 moves from ordering failures to failed messages themselves: dead-letter queues, poison messages, retry limits, and the discipline of treating failure records as operational evidence rather than hidden garbage.

PART V - MESSAGING AND ASYNCHRONOUS COMMUNICATION

31. Dead-Letter Queues and Poison Messages

A message that cannot be processed should not disappear, loop forever, or quietly block a stream. It should become visible evidence: why it failed, which contract it used, which consumer rejected it, and what a human or operator must decide next.

FAILED DELIVERY - RETRY LIMIT - DEAD LETTER - TRIAGE - REDRIVE WITH CARE

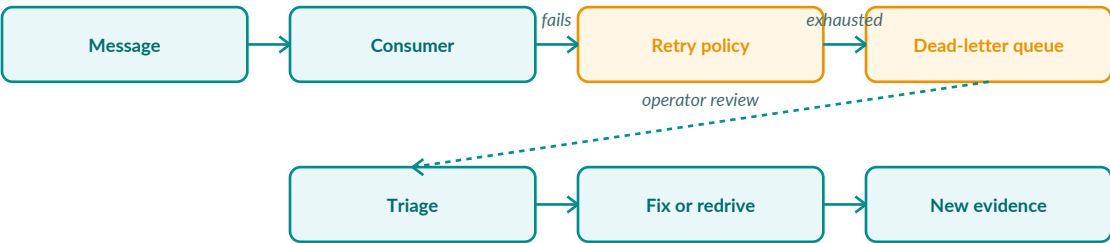


Figure 31.1 - A dead-letter queue is not a trash can. It is a visible operational evidence trail.

Chapter 30 dealt with messages that arrive more than once or arrive in an order the consumer did not expect. This chapter deals with messages that never become acceptable to the consumer at all. They may be malformed. They may refer to an entity the consumer cannot find. They may use a contract version the consumer no longer understands. They may expose a domain conflict that should not be solved by retrying harder.

The naive instinct is to keep retrying. In a distributed system that instinct can become dangerous. A message that always fails is not a temporary failure. It is an operational signal. If the platform hides it behind infinite retries, the failure becomes noise, delay, duplicate work, and sometimes data corruption.

CHAPTER RULE

A dead-letter queue is not a trash can. It is the place where the system admits that automatic processing has stopped and that the failure now needs evidence, diagnosis, and an explicit decision.

A failed message is not garbage; it is evidence that the contract, consumer, or data needs attention.

What a poison message is

A poison message is a message that repeatedly fails for a reason retrying will not fix. A temporary database outage is not a poison message. A network timeout is not a poison message. A malformed `RootCauseVerdictIssued.v1` payload with a missing `RootCauseCaseld` is poison. A `RequestComplianceReview` command for a verdict state the compliance service has explicitly rejected may be poison. The difference is whether time alone can plausibly make the message processable.

Failure kind	Retry may help?	Example in NEXUS-1
Transient infrastructure failure	Yes	Compliance database unavailable for thirty seconds
Rate or capacity pressure	Yes, with backoff	Reporting consumer is behind during an alarm-flood drill
Consumer bug	No, not until code changes	Handler cannot deserialize a valid <code>RootCauseVerdictIssuedV1</code> record
Bad contract payload	No	Required public field is missing or semantically impossible
Missing referenced state	Maybe	<code>RootCauseCaseld</code> not yet replicated into a local reference table
Domain rejection	No, unless a human decides otherwise	Compliance refuses a review request for an invalid verdict state

The table matters because each row deserves a different response. A transient failure should be retried. A capacity problem should be backed off. A consumer bug should be fixed. A bad public contract should be investigated at the producing boundary. A domain rejection should be visible to the business flow, not hidden as a technical retry.

DEFINITION

A poison message is not merely a message that failed. It is a message whose repeated failure is itself evidence that automatic retry is the wrong next action.

A failed message is not garbage; it is evidence that the contract, consumer, or data needs attention.

Why infinite retries are architecture debt

Retry is one of the most useful patterns in distributed systems, but it is also one of the easiest to abuse. A retry policy should express the assumption that the failure may be temporary. When that assumption is false, retry becomes a machine for creating repeated damage.

An infinite retry loop can keep a consumer busy with one impossible message while valid messages wait behind it. It can write the same error thousands of times. It can hide the original failure behind a wall of repeated stack traces. It can trigger unnecessary alerts. In the worst case, it can repeatedly call a partially idempotent external endpoint and create the very inconsistency the retry was meant to avoid.

Retry mistake	What happens	Better rule
No retry limit	Poison message loops forever	Set max attempts and move to dead-letter after exhaustion
Retry every failure equally	Malformed contracts are treated like network timeouts	Classify transient, permanent, and unknown failures
No jitter or backoff	Many consumers retry at the same instant	Use exponential backoff and jitter
No failure record	The cause is lost after the process restarts	Persist attempt count, exception summary, and payload reference
Automatic redrive without fix	The same poison message returns to the same failed consumer	Redrive only after diagnosis or handler change

RETRY RULE

Retry only while the failure model still says time may help. Once the failure has crossed that boundary, the message should move from automation to evidence.

A failed message is not garbage; it is evidence that the contract, consumer, or data needs attention.

What belongs in a dead-letter record

A useful dead-letter record does not merely store a blob. It stores enough context to reconstruct the failure without depending on volatile logs. The goal is not to shame the service that failed. The goal is to make the failure diagnosable, auditable, and safe to handle later.

Field	Why it matters
MessageId	Identifies the failed delivery and prevents confused redrive
ContractName and ContractVersion	Shows which public contract failed
ProducerService and ConsumerService	Names both sides of the boundary
CorrelationId and CausationId	Connects the failure to the larger alarm-to-compliance flow
AggregateId or StreamKey	Shows which local ordering boundary was affected
AttemptCount and FirstFailedAtUtc	Distinguishes a new failure from a repeated one
LastErrorCode and LastErrorSummary	Keeps the triage reason visible without needing a log search
PayloadHash and PayloadLocation	Allows safe comparison without printing sensitive payloads everywhere
Disposition	Records whether the message was redriven, ignored, corrected, or escalated

In NEXUS-1, this kind of record is especially important because the series has treated auditability as a first-class property from the beginning. A failed RootCauseVerdictIssued.v1 message is part of the evidence trail. It says that the root-cause service published something, that another service could not process it, and that the architecture had to make a decision.

A failed message is not garbage; it is evidence that the contract, consumer, or data needs attention.

SQL-friendly shape for NEXUS-1

The exact physical table can vary by broker and infrastructure stack. The important point is that the dead-letter fact must be durable, queryable, and connected to the same correlation discipline used by the outbox and inbox. The shape below is deliberately SQL-friendly because NEXUS-1 treats the database as a source of audit evidence, not merely a cache of runtime state.

```
CREATE TABLE Messaging.DeadLetterMessage (
  DeadLetterMessageId  bigint IDENTITY(1,1) NOT NULL,
  MessageId           uniqueidentifier NOT NULL,
  ContractName        nvarchar(160) NOT NULL,
  ContractVersion      nvarchar(40) NOT NULL,
  ProducerService      nvarchar(120) NOT NULL,
  ConsumerService      nvarchar(120) NOT NULL,
  CorrelationId        uniqueidentifier NULL,
  CausationId          uniqueidentifier NULL,
  StreamKey           nvarchar(160) NULL,
  AttemptCount         int NOT NULL,
  FirstFailedAtUtc     datetime2(7) NOT NULL,
  LastFailedAtUtc      datetime2(7) NOT NULL,
  LastErrorCode        nvarchar(80) NULL,
  LastErrorSummary     nvarchar(1000) NOT NULL,
  PayloadHash          char(64) NOT NULL,
  PayloadLocation      nvarchar(500) NULL,
  Disposition          nvarchar(40) NOT NULL,
  DispositionReason    nvarchar(1000) NULL,
  DisposedAtUtc        datetime2(7) NULL,
  CONSTRAINT PK_Messaging_DeadLetterMessage
    PRIMARY KEY (DeadLetterMessageId),
  CONSTRAINT UQ_Messaging_DeadLetter_Message_Consumer
    UNIQUE (MessageId, ConsumerService)
);

CREATE INDEX IX_Messaging_DeadLetter_Correlation
ON Messaging.DeadLetterMessage(CorrelationId, LastFailedAtUtc DESC);
```

SCHEMA DISCIPLINE

The dead-letter table should not become a dumping ground for every exception. It should record failed message processing after the retry policy has made a boundary decision.

A failed message is not garbage; it is evidence that the contract, consumer, or data needs attention.

The NEXUS-1 poison-message examples

The alarm-to-compliance flow gives several realistic examples. None requires a live plant claim. They are software architecture failures inside the demonstrator boundary.

Message	Failure	Likely disposition
AlarmFloodDetected.v1	Consumer cannot deserialize because the published contract is broken	Pack and fix consumer or maintain compatibility
RootCauseVerdictIssued.v1	Payload omits the public Caseld required by the contract	Dead-letter and investigate producer boundary
RequestComplianceReview	Compliance service rejects because the verdict state is not eligible for review	Make for review rejected and notify process manager
RootCauseVerdictCorrected.v1	Correction references a verdict version older than the projected state applies	Ignore stale update, permits, otherwise dead-letter
Reporting projection message	Read-model update fails because the target row violates a unique projection id	Fix projection id idempotency, then redrive

The important distinction is that not every failed message is a technical incident. Some failures are contract incidents. Some are domain incidents. Some are projection bugs. A serious microservice architecture should not flatten all of them into one red error label.

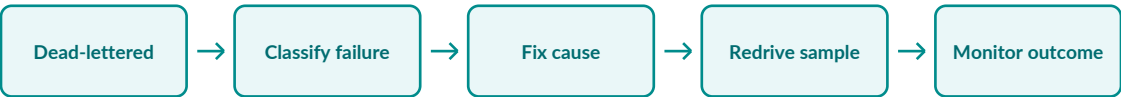
LANGUAGE DISCIPLINE

A dead-lettered message does not automatically mean the business operation failed. It means one consumer could not complete its processing contract after the allowed attempts.

A failed message is not garbage; it is evidence that the contract, consumer, or data needs attention.

Triage before redrive

Redrive means taking a message from the dead-letter queue and sending it back for processing. It is useful, but it is dangerous when treated as a magic reset button. Redrive without diagnosis is just retry with a different name.



Redrive is a controlled operation, not a panic button.

A NEXUS-1 operator or support engineer should classify the failure first. Was the consumer temporarily down? Was the handler fixed? Was the public contract corrected? Was the reference data replicated? Was the domain state corrected by a human decision? Only then does redrive become meaningful.

Disposition	Meaning	When appropriate
Redrive	Try the same message again after a fix	Consumer bug fixed or downstream service recovered
Park	Keep the message for later investigation	Unknown failure, unclear business impact
Ignore	Mark as intentionally not processed	Older stale event superseded by newer state
Correct and replay	Publish a corrected event or command	Original payload violates a contract but business fact remains valid
Escalate	Human review required	Domain conflict cannot be solved by software alone

OPERATIONAL RULE

The redrive action should be audited with who did it, when, why, which fix was believed to make it safe, and what happened after redrive.

A failed message is not garbage; it is evidence that the contract, consumer, or data needs attention.

Dead letters and the process manager

A process manager should not ignore dead-letter outcomes. If Compliance cannot process RequestComplianceReview, the alarm-to-compliance flow has not completed. That does not mean RootCause was wrong, and it does not mean the operator console should invent a clean state. It means the flow has an operational exception that must be visible.

The process manager can treat a dead-letter record as a state transition in the workflow: review requested, delivery failed, retry exhausted, human review required. This is not the same as letting the dead-letter queue drive the domain. The domain fact remains in the owning service. The dead-letter record is evidence about an integration failure across a boundary.

Do this	Do not do this
Expose failed integration state to the workflow owner	Pretend the message disappeared
Connect dead letters to correlation and causation identifiers	Search raw logs manually as the primary recovery method
Let a process manager pause or escalate a distributed flow	Let a broker error become an invisible business delay
Audit redrive and disposition decisions	Redrive messages repeatedly without recording why

BOUNDARY RULE

A dead-letter record is operational evidence. It can influence workflow supervision, escalation, and recovery, but it should not become the source of domain truth for the business fact itself.

A failed message is not garbage; it is evidence that the contract, consumer, or data needs attention.

What the operator console should show

The operator console should not show a raw broker queue as if it were a domain screen. It should show the effect of integration health on the flow the user cares about. For example: root-cause verdict issued, compliance review pending, delivery to Compliance failed after five attempts, triage required. That is a human-readable state, not a dump of middleware internals.

Console label	What it should mean
Compliance review pending	The review request has been published or scheduled, but the review is not complete
Delivery failed	The consumer exhausted its retry policy and a dead-letter record exists
Triage required	A human must classify the failure before redrive, correction, or ignore
Projection delayed	Reporting is behind, but source-of-truth services remain valid
Recovered after redrive	The failed message was processed after an explicit recovery action

This preserves the recurring distinction in the series: audit evidence is not workflow, recommendation is not command, and projection is not source of truth. A dead-letter state belongs to operational flow visibility. It should inform the operator or maintainer without pretending to be a plant fact.

A failed message is not garbage; it is evidence that the contract, consumer, or data needs attention.

Checklist for Chapter 31

Before accepting a NEXUS-1 messaging design, the architect should be able to answer the following questions about failed messages.

- Which failures are transient, permanent, or unknown?
- What is the maximum retry count for each consumer?
- Does the retry policy use backoff and jitter?
- Where is the failure stored after retry exhaustion?
- Can a support engineer see the contract name, version, correlation id, and consumer?
- Can a dead-lettered message be redriven only after diagnosis?
- How is redrive audited?
- Which process manager or support workflow is notified when a critical message is dead-lettered?
- How does the operator console explain integration failure without exposing broker internals?

HONEST BOUNDARY

This chapter does not choose RabbitMQ, Azure Service Bus, Kafka, or any specific broker product. The architecture point is broker-independent: a failed message needs retry limits, durable evidence, triage, disposition, and safe recovery.

A failed message is not garbage; it is evidence that the contract, consumer, or data needs attention.

Chapter checkpoint

You are ready to continue when you can answer these questions clearly.

- What makes a message poison rather than merely failed?
- Why can infinite retry become an architectural bug?
- What fields should a dead-letter record preserve for diagnosis?
- Why is redrive dangerous without triage?
- How can a dead-letter record affect a process manager without becoming domain truth?
- Why should the operator console show workflow impact instead of broker internals?
- What is the difference between correcting and replaying a business event and simply redriving the same payload?
- How would you handle a dead-lettered `RootCauseVerdictIssued.v1` message in NEXUS-1?

NEXT STEP

Chapter 32 closes the messaging part by making event contracts explicit in NEXUS-1: canonical names, payload shapes, version rules, correction events, and the published language that prevents services from leaking private models.

A failed message is not garbage; it is evidence that the contract, consumer, or data needs attention.

PART V - MESSAGING AND ASYNCHRONOUS COMMUNICATION

32. Event Contracts in NEXUS-1

A public event is not the domain model escaping through the broker. It is a deliberately shaped contract: named, versioned, minimal, durable, and understandable by services that do not own the private model that produced it.

PUBLISHED LANGUAGE - EVENT NAME - CONTRACT VERSION - PAYLOAD SHAPE - CORRECTION EVENT

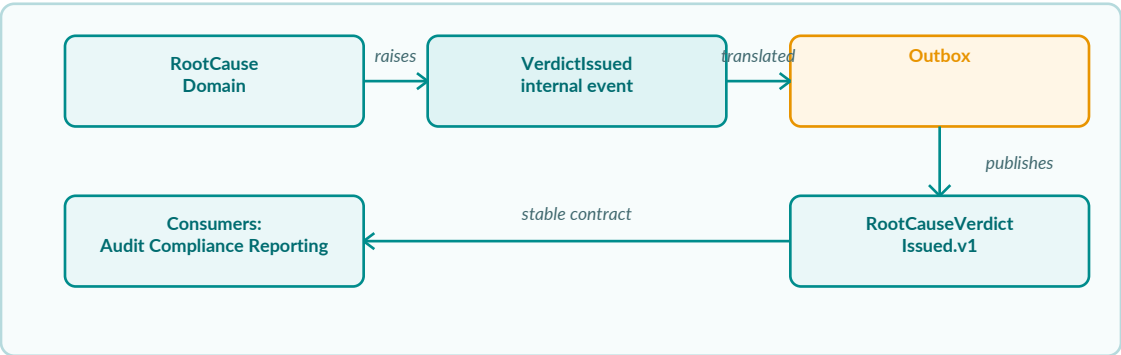


Figure 32.1 - Private domain event translated to a public integration contract.

Part V has walked from the reason services need messages to delivery guarantees, outbox, inbox, ordering, and dead-letter handling. This final messaging chapter asks what exactly travels through the broker. The answer must not be vague. The thing that crosses a service boundary is not just a JSON object. It is a published language artifact.

Inside the RootCause context, the domain may raise an internal event called VerdictIssued. That internal event can be rich, private, and shaped by the aggregate. Outside the RootCause boundary, consumers should not receive the private object. They should receive a stable public integration event: RootCauseVerdictIssued.v1, represented in C# as RootCauseVerdictIssuedV1.

CHAPTER RULE

Never publish the private domain event as the public integration event. Translate it into a deliberately shaped contract at the boundary, then version that contract as part of the published language.

A public event is not a snapshot of the private model; it is a promise made to other services

Internal event versus integration contract

The difference is not cosmetic. The internal event exists to express a fact inside the owning model. The integration event exists to let other services react without depending on that owning model. The first belongs to the aggregate language. The second belongs to the system language.

Concern	Internal domain event	Public integration event
Primary audience	The same bounded context	Other services and external consumers
Example name	VerdictIssued	RootCauseVerdictIssued.v1
C# shape	May reference domain objects or value objects	DTO-style contract with primitives and stable names
Versioning	Usually evolves with the internal model	Must preserve compatibility or publish a new version
Storage	May be transient before outbox translation	Stored in outbox and observable through audit/operations
Rule	Can be refactored with the model	Must not break consumers silently

This distinction protects both sides. RootCause remains free to improve its aggregate, evidence model, and hypothesis language. Compliance and Reporting remain free to consume a stable verdict contract without importing RootCause internals.

TERMINOLOGY DISCIPLINE

RootCauseCaseClosed should not be the canonical public event if the business fact consumers need is the verdict. The canonical public event is RootCauseVerdictIssued.v1.

The canonical event names

The NEXUS-1 project already has a language discipline. This book carries it forward instead of inventing new names. The naming rule is simple: the public event name should say the business fact that became true, not the technical step that happened to an implementation object.

Artifact	Canonical name	Reason
Internal domain event	VerdictIssued	The RootCause aggregate has issued a verdict inside its own boundary.
Public integration event	RootCauseVerdictIssued.v1	Other services can react to the issued verdict without seeing the private model.
C# public contract type	RootCauseVerdictIssuedV1	The type name follows the event contract and the version suffix.
Related event	AlarmFloodDetected.v1	AlarmManagement publishes the fact that a flood exists.
Related command	OpenRootCauseCase	A request to start work; not a fact that already happened.
Related command	RequestComplianceReview	A request to Compliance; not evidence and not a verdict.
Correction event	RootCauseVerdictCorrected.v1	A later correction should not rewrite the historical issued event.

The prohibited names are as important as the accepted names. RootCauseVerdictPublished sounds like a messaging implementation detail. RootCauseCaseClosed sounds like a lifecycle state. Neither is as precise as RootCauseVerdictIssued.v1 when the business fact is that a verdict was issued.

What belongs in the payload

A public payload should be small enough to remain stable and rich enough to let consumers do their work without calling back immediately for the most basic facts. The event should not contain the entire RootCauseCase aggregate. It should carry the identity, correlation, verdict summary, relevant confidence or severity fields, timing, and causation links needed by downstream services.

Field group	Example fields	Why it exists
Identity	eventId, caseId, unitId	Lets consumers store and deduplicate the event.
Traceability	correlationId, causationId, occurredAtUtc	Lets a case be followed across services.
Contract	eventName, eventVersion, schemaVersion	Makes the payload self-describing.
Verdict fact	verdictCode, verdictText, issuedBy, issuedAtUtc	States the business fact that changed.
Evidence summary	evidenceCount, primaryEvidenceIds	Points to evidence without embedding the private evidence model.
Operational tags	sourceService, tenant or environment marker where applicable	Helps support and audit without leaking internals.

PAYLOAD BOUNDARY

A public event should carry stable identifiers and facts, not navigation properties, EF Core entities, aggregate internals, or UI-specific labels.

```
{
  "eventName": "RootCauseVerdictIssued.v1",
  "eventId": "01JX-RCA-VERDICT-00042",
  "eventVersion": 1,
  "occurredAtUtc": "2026-07-09T09:30:00Z",
  "correlationId": "EVT-2026-0418",
  "causationId": "CMD-CLOSE-RCA-0418",
  "sourceService": "RootCause",
  "caseId": "RCA-2026-0418",
  "unitId": "UNIT-1",
  "verdictCode": "FEEDWATER_VALVE_LATENCY",
  "verdictText": "FV-104 actuator latency identified as primary origin.",
  "evidenceCount": 7,
  "primaryEvidenceIds": ["EVD-104", "EVD-219"]
}
```

The C# contract type

The public C# type should read like a contract, not like a domain aggregate. It should be immutable, serializable, and deliberately boring. Boring is a feature here: consumers must understand it without needing the RootCause project referenced in their solution.

```
public sealed record RootCauseVerdictIssuedV1(
    Guid EventId,
    string EventName,
    int EventVersion,
    DateTime OccurredAtUtc,
    string CorrelationId,
    string CausationId,
    string SourceService,
    string CaseId,
    string UnitId,
    string VerdictCode,
    string VerdictText,
    string IssuedBy,
    DateTime IssuedAtUtc,
    int EvidenceCount,
    IReadOnlyList<string> PrimaryEvidenceIds);
```

This type is intentionally not RootCauseCase. It has no behavior, no private state, no EF mapping, and no navigation to hypotheses. It is not the model. It is the published message about one fact produced by the model.

Do	Avoid
Use stable primitive fields and explicit identifiers	Expose aggregate entities or private value objects
Name the event after the business fact	Name the event after the transport action
Include correlation and causation identifiers	Force operators to reconstruct flow from logs only
Keep UI formatting out of the contract	Bake console labels into event payloads
Version intentionally	Let contract changes hide inside a deployment

Versioning rules for event contracts

An event contract is harder to change than an internal class. A private class changes with one service. A public contract changes with every consumer, every stored message, every dead-letter record, every replay, and every projection that knows how to read it.

Change	Compatible?	Preferred action
Add optional field with safe default	Usually yes	Keep v1 and document the default.
Rename a field	No	Publish v2 or support both names during transition.
Change field meaning	No	Publish a new version; do not lie with old names.
Remove required field	No	Publish v2 and migrate consumers.
Add enum value	Conditionally	Consumers must handle unknown values safely.
Change timestamp semantics	No	Publish new field or new version.
Add internal diagnostic metadata	Usually yes	Keep it optional and out of business decisions.

VERSIONING RULE
If a consumer that correctly understands v1 would make the wrong decision after your change, the change is not compatible. Create a new contract version.

For NEXUS-1, RootCauseVerdictIssued.v1 remains the event consumed by Audit, Compliance, Reporting, and the operator-console composition layer. A later RootCauseVerdictIssued.v2 could exist, but v1 messages in old outbox rows and replay archives must remain readable.

Correction is not mutation

A serious audit trail does not pretend that an earlier public event never happened. If a verdict is corrected, the system should not rewrite `RootCauseVerdictIssued.v1` in history. It should publish a correction event that points to the earlier verdict and explains the replacement fact.



Corrections preserve history instead of rewriting the event stream.

The correction event is not a failure of the model. It is part of accountable modelling. In a system that treats audit seriously, the old fact remains visible as the fact that was issued at that time. The correction becomes a new fact with its own timestamp, author, reason, and causation.

```
public sealed record RootCauseVerdictCorrectedV1(  
    Guid EventId,  
    DateTime OccurredAtUtc,  
    string CorrelationId,  
    string OriginalVerdictEventId,  
    string CaseId,  
    string PreviousVerdictCode,  
    string CorrectedVerdictCode,  
    string CorrectionReason,  
    string CorrectedBy);
```

AUDIT DISCIPLINE

A correction event does not erase the original event. It creates a new auditable fact that tells consumers how to adjust their projections and reviews.

Consumer contracts in the alarm-to-compliance flow

Every consumer should be able to say why it subscribes to an event and what it is allowed to do with it. This prevents accidental workflow coupling. Audit records evidence. Compliance starts or updates a review. Reporting builds a read model. The operator console composes status. None of these consumers owns the RootCause verdict.

Consumer	Subscribes to	Allowed reaction
Audit	RootCauseVerdictIssued.v1, RootCauseVerdictCorrected.v1	Append evidence of what was issued and later corrected.
Compliance	RootCauseVerdictIssued.v1	Evaluate whether a compliance review should be opened or updated.
Reporting	RootCauseVerdictIssued.v1 and correction events	Update read-side projections; never become source of truth.
Operator Console BFF	Read models and status endpoints	Compose display state; not decide root cause.
ReinforcementLearning	Usually not direct consumer of verdict events	Recommendations are advisory and should not become commands.

RECURRING NEXUS-1 SENTENCE

AuditEntry is evidence, not workflow. A recommendation is not a command. A read model is not the source of truth. An integration event is not the aggregate.

A small contract registry

A microservice architecture needs a place where public contracts are named and reviewed. It does not have to be heavy at first. It can begin as a small registry in the repository: contract name, version, owning service, schema file, sample payload, compatibility notes, consumers, and deprecation status.

Registry item	Example
Contract name	RootCauseVerdictIssued.v1
Owner	RootCause service
C# type	RootCauseVerdictIssuedV1
Schema artifact	contracts/rootcause/RootCauseVerdictIssued.v1.schema.json
Sample payload	contracts/rootcause/samples/root-cause-verdict-issued-v1.json
Consumers	Audit, Compliance, Reporting, Operator Console BFF
Compatibility policy	Additive optional changes allowed; breaking changes require v2.
Deprecation state	Active

The registry is not bureaucracy. It is the same design habit used throughout the series: one meaning per name, one owner per truth, and an audit trail for decisions that cross boundaries.

```
/contracts
  /rootcause
    RootCauseVerdictIssued.v1.schema.json
    RootCauseVerdictIssued.v1.sample.json
    RootCauseVerdictCorrected.v1.schema.json
  /alarmmanagement
    AlarmFloodDetected.v1.schema.json
  /compliance
    RequestComplianceReview.v1.schema.json
```


Anti-patterns that damage contracts

Most contract damage begins with convenience. A developer publishes the domain object because it is already there. A field is renamed because the UI changed its label. A consumer starts depending on a field that was meant to be diagnostic. A projection treats itself as source of truth. None of these decisions looks catastrophic in isolation. Together, they make services brittle.

Anti-pattern	Damage
Publishing EF entities	Consumers become coupled to persistence and navigation shape.
Using UI labels as event fields	Display changes become contract breaks.
Making every field required	Future compatible evolution becomes difficult.
No event version in the name or metadata	Consumers cannot reason about compatibility.
Renaming events after implementation steps	The language stops describing business facts.
Treating audit events as workflow commands	Evidence begins to drive behavior it should only record.

DESIGN WARNING

Convenience at the boundary becomes long-term coupling. The public contract should be shaped with more discipline than the private class because more systems will depend on it.

Checklist for Chapter 32

Before accepting an event contract in NEXUS-1, the architect should be able to answer the following questions.

- What service owns the event contract?
- Is the name a business fact rather than a transport action?
- Is the internal domain event separated from the public integration event?
- Does the payload avoid aggregate internals, EF entities, UI labels, and navigation properties?
- Does the event include event id, version, occurred time, correlation id, and causation id?
- Which consumers are known, and what are they allowed to do?
- What changes are compatible with the current version?
- What change would require v2?
- How are correction events represented without rewriting history?
- Where is the schema, sample payload, and owner recorded?

HONEST BOUNDARY

This chapter defines contract discipline, naming, payload shape, and version rules. It does not require a specific schema registry product, broker, serialization library, or deployment platform. Those choices belong to implementation and operations.

Chapter checkpoint

You are ready to leave Part V when you can answer these questions clearly.

- Why should VerdictIssued stay internal while RootCauseVerdictIssued.v1 becomes public?
- Why is RootCauseCaseClosed weaker than RootCauseVerdictIssued.v1 as the canonical event?
- What fields make an integration event traceable across services?
- Why should a public event not expose EF Core entities or aggregate internals?
- What kind of change requires a new event version?
- Why is a correction event better than mutating the original issued event?
- What is each consumer allowed to do with RootCauseVerdictIssued.v1?
- How does a contract registry protect the published language of the system?

PART V CLOSING

Messages do not make services reliable by themselves. Reliability comes from contracts, delivery rules, durable publication, idempotent consumption, ordering humility, dead-letter visibility, and a published language that consumers can trust.

NEXT STEP

Part VI moves from messages to distributed workflows: process managers, sagas, choreography, compensation, timeouts, escalations, and the alarm-to-compliance flow as a runtime process.

PART VI

Distributed Workflows

Part VI moves beyond individual messages. A service can publish facts, and another service can consume them, but a real business flow often needs memory: what has already happened, what still has to happen, what failed, what timed out, and what must be brought back to a human. Distributed workflows are the discipline of making that memory explicit without turning the whole system back into one hidden transaction.

CHAPTERS 33-39

PROCESS MANAGERS - SAGAS - CHOREOGRAPHY - ALARM-TO-COMPLIANCE FLOW
- COMPENSATION - TIMEOUTS - HUMAN REVIEW - HIDDEN WORKFLOW RISKS

A distributed workflow is not a transaction stretched across the network. It is a remembered conversation between owners.

PART VI - DISTRIBUTED WORKFLOWS

33. Process Managers Across Services

A process manager is not a smarter service. It is the place where a long-running flow remembers what has already happened, decides what message should be sent next, and refuses to pretend that a distributed workflow is one database transaction.

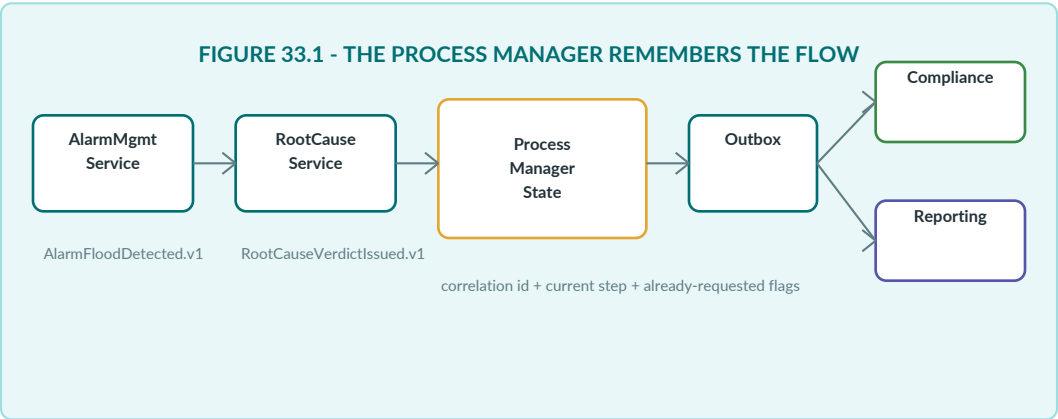


Figure 33.1 - A process manager records workflow progress and sends durable messages through an outbox. It does not own the domain facts produced by the services.

CHAPTER RULE

A process manager owns the memory of a flow, not the truth of the domain. RootCause owns the verdict. Compliance owns the review. Reporting owns its projection. The process manager owns the fact that the flow has reached a step, that a request has been sent, and that another step is now expected.

What you already know

Inside a modular monolith, a long workflow often hides inside one application service. The service loads an aggregate, calls a domain method, writes a row, maybe writes an audit entry, and returns. If the database transaction commits, the developer feels the workflow is finished.

That confidence disappears once the same workflow crosses service boundaries. A root-cause verdict may be issued in one service. A compliance review must start in another. A reporting projection must update somewhere else. Audit must remain replayable. The operator console must show progress even if one downstream service is temporarily unavailable. There is no single call stack that can honestly contain all of that.

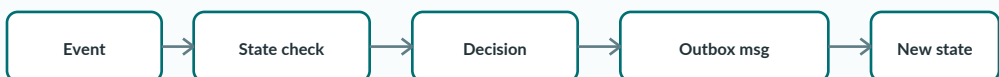
THE NAIVE ANSWER

The naive answer is to make RootCause call Compliance, then Reporting, then Audit, then wait until everyone agrees. That looks simple in a sequence diagram and fails in production because the slowest or least available downstream service becomes part of the root-cause command path.

The better question is not "how do we make one call finish everything?" The better question is "who remembers that this multi-service flow is in progress?" That is the question a process manager answers.

The problem a process manager solves

A process manager is a small state machine that reacts to events, records progress, and sends commands or integration requests. It does not calculate the root cause. It does not decide the compliance result. It does not rewrite the reporting model. It coordinates the flow that connects those owners.



Every step must be replay-safe: the same event twice must not produce two different effects.

NEXUS-1 SENTENCE TO REMEMBER

The process manager does not say what the verdict means. It remembers that RootCauseVerdictIssued.v1 has happened, that compliance must be requested, that reporting must be notified, and that the workflow is not complete until those responsibilities have been handed to their owners.

Why this is different from an application service

Concern	Application service inside one context	Process manager across services
Time span	Usually one request or one local transaction.	May last seconds, minutes, hours, or days.
Memory	Often relies on the current call stack and database transaction.	Persists explicit workflow state and correlation.
Inputs	Commands from a user or API.	Integration events and timeout signals.
Outputs	Local domain changes and local events.	Durable commands or requests to other services, usually through an outbox.
Failure model	Rollback or local retry may be enough.	Retry, idempotency, compensation, timeout, and human review must be designed.

The distinction matters because many failed microservice designs accidentally rebuild the monolith as a chain of blocking calls. The application service becomes a distributed transaction coordinator without admitting it. A process manager makes that responsibility explicit, durable, inspectable, and testable.

LANGUAGE DISCIPLINE

A process manager may send a request such as RequestComplianceReview. It must not rename a root-cause verdict, reinterpret evidence, or pretend that a recommendation is a command. It coordinates ownership; it does not steal it.

The NEXUS-1 flow as a process

The recurring NEXUS-1 alarm-to-compliance flow is an ideal example because it is not one service doing one thing. It is a sequence of ownership transfers. AlarmManagement owns alarms and floods. RootCause owns cases, evidence, hypotheses, and verdicts. Compliance owns review. Reporting owns read models. Audit records evidence of changes and decisions.

A useful process manager does not collapse these responsibilities into one mega-service. It tracks the handoff points:

- An alarm flood has been detected and a root-cause case should be opened or associated.
- A root-cause verdict has been issued and the public event has been observed.
- A compliance review has been requested once, and only once.
- Reporting has been notified to rebuild or update its projection.
- Timeouts and missing downstream acknowledgements are visible to operators or support staff.
- The workflow has reached a completed, waiting, failed, or needs-human-review state.

TABLE SHAPE: RootCauseWorkflowState

Correlation	Caseld	Verdict	Compliance	Reporting	Status
WF-0418	RCA-0418	Issued	Requested	Queued	Waiting

The table is not the source of truth for verdicts. It is the memory of orchestration progress.

Figure 33.2 - One possible workflow-state row. It records orchestration progress, not the root-cause verdict itself.

This row is not a second source of truth for RootCause. The verdict remains in the RootCause service. The process state only says that a verdict event has been seen and that the corresponding downstream work has or has not been requested.

SOURCE-OF-TRUTH RULE

A process manager may store identifiers, step flags, timestamps, last processed event IDs, retry counts, and status. It should not store a private copy of the entire domain model. When it stores a snapshot, the snapshot must be labelled as workflow memory, not domain truth.

A small implementation shape

In C# terms, the process manager usually appears as an event consumer plus a persistence model. The consumer receives a public event, loads the workflow state by correlation ID, checks what has already happened, appends messages to the outbox, and saves the updated state in one local transaction.


```
public sealed class RootCauseWorkflowState
{
    public Guid CorrelationId { get; private set; }
    public Guid RootCauseCaseId { get; private set; }
    public bool VerdictObserved { get; private set; }
    public bool ComplianceReviewRequested { get; private set; }
    public bool ReportingProjectionRequested { get; private set; }
    public string Status { get; private set; } = "Waiting";

    public void MarkVerdictObserved(Guid caseId)
    {
        RootCauseCaseId = caseId;
        VerdictObserved = true;
    }

    public bool NeedsComplianceRequest()
        => VerdictObserved && !ComplianceReviewRequested;

    public void MarkComplianceRequested()
        => ComplianceReviewRequested = true;
}
```

The important point is not this exact class. The important point is the shape: explicit state, explicit step flags, and methods that make repeated event delivery safe. If the same event is delivered twice, the state should prevent the same downstream request from being produced twice.

IDEMPOTENCY CONNECTION

This chapter depends on Chapter 29. A process manager that is not idempotent becomes a duplication engine. It will create two compliance reviews, two reporting rebuilds, or two escalation tasks because the broker did exactly what brokers are allowed to do: deliver more than once.

Handling RootCauseVerdictIssued.v1

The public event `RootCauseVerdictIssued.v1` is the canonical notification that `RootCause` has issued a verdict. The process manager consumes that event because the wider platform has work to coordinate after the verdict. It does not consume a private domain event such as `VerdictIssued`, because that event belongs inside `RootCause`.

```
public async Task Handle(RootCauseVerdictIssuedV1 message, CancellationToken ct)
{
    var state = await workflows.GetOrCreateAsync(
        message.CorrelationId,
        message.RootCauseCaseId,
        ct);

    if (state.HasProcessed(message.EventId))
        return;

    state.MarkVerdictObserved(message.RootCauseCaseId);

    if (state.NeedsComplianceRequest())
    {
        await outbox.AddAsync(new RequestComplianceReview(
            message.CorrelationId,
            message.RootCauseCaseId,
            message.VerdictCode,
            message.IssuedAtUtc), ct);

        state.MarkComplianceRequested();
    }

    state.MarkProcessed(message.EventId);
    await unitOfWork.SaveChangesAsync(ct);
}
```

The handler performs two writes that must survive together: it updates workflow state and inserts an outbox message. That local transaction is the reliability point. It does not call Compliance directly inside the handler and hope the network behaves.

BOUNDARY CONNECTION

This is where Chapter 28 matters. The process manager should write its outgoing request to the outbox. The publisher then delivers it. The state change and the decision to send a message are committed together.

What the process manager may own

May own	Why it belongs here
Correlation ID	The flow needs one traceable thread across services.
Current workflow status	Operators and support staff need to know whether the flow is waiting, completed, failed, or escalated.
Step completion flags	The manager must avoid sending the same request twice.
Last processed event IDs	Idempotency requires durable memory of what has already been handled.
Retry and timeout metadata	Delayed downstream work must become visible and recoverable.

What it must not own is just as important. It must not own the root-cause evidence, the compliance finding, the reporting data model, or the audit truth. Those belong to their services. The process manager coordinates the conversation between them.

COMMON MISTAKE

Do not let the process manager become the place where all business rules go to avoid service ownership. If every service asks the process manager what to do, the system has not become distributed; it has moved the old central brain into a new table.

When a process manager is justified

Not every event-driven interaction needs a process manager. If Reporting simply consumes RootCauseVerdictIssued.v1 and updates a projection, choreography may be enough. A process manager becomes useful when the flow has memory, branching, timeouts, retries, escalation, compensation, or a user-visible status that spans multiple services.

Readiness checklist

Question	Reason
There is a long-running flow	The work cannot honestly complete inside one local transaction.
More than one service owner participates	The flow crosses true ownership boundaries, not just project folders.
The flow needs memory	Someone must remember which steps have already happened.
Duplicate events are possible	Repeated delivery must not create repeated business effects.
Timeouts matter	A missing downstream result must become visible and actionable.
Human review may be required	Some failures should stop and ask for attention rather than retry forever.

If none of these are true, avoid the process manager. Let services publish and consume events directly. Coordination machinery has a cost. The NEXUS-1 principle remains the same as earlier chapters: distribute only when the boundary has earned the burden.

ARCHITECTURAL DECISION

For NEXUS-1, the alarm-to-compliance path earns a process manager because it spans RootCause, Compliance, Reporting, and Audit-facing evidence, while also requiring correlation, retry, timeout, and user-visible status. A simple reporting projection does not earn one by itself.

Process manager vs central command service

A process manager can be abused. The easiest abuse is to turn it into a central command service that tells every other service what to do at every moment. That design looks tidy because all logic appears in one place. It is dangerous because it makes every service a passive worker and makes the process manager the new monolith.

A healthy process manager is thinner. It reacts to facts, records progress, and sends requests. The receiving services still decide according to their own rules. Compliance may reject or defer a review. Reporting may rebuild asynchronously. Audit may record the evidence but should not drive the business workflow. The process manager coordinates; it does not control the domain.

NEXUS-1 EXAMPLE

RequestComplianceReview is a request to the Compliance service, not a forced finding. Compliance still owns regulations, review status, reviewer assignment, and finding severity. The process manager only knows that a review was requested because a root-cause verdict was issued.

Observability is part of the pattern

A process manager that cannot be inspected is worse than no process manager at all. The whole reason to store workflow state is to make the flow understandable: which event started it, which messages were produced, which step is waiting, how many retries have occurred, and what correlation ID connects the records.

In NEXUS-1, this matters because the operator console should not merely show "something is delayed." It should show that the root-cause verdict exists, the compliance review request was sent, reporting is waiting for its projection update, and a timeout will escalate the flow if the next step does not arrive.

Testing the process manager

A process manager is easier to test than many developers expect because it is mostly state transition logic. Given a current state and an incoming event, it should produce a predictable new state and a predictable set of outgoing messages. The test does not need a real broker. It needs a fake outbox and a durable state repository abstraction.

```
[Fact]
public async Task VerdictIssued_requests_compliance_only_once()
{
    var message = RootCauseVerdictIssuedV1Factory.Valid();

    await handler.Handle(message, CancellationToken.None);
    await handler.Handle(message, CancellationToken.None);

    outbox.Messages.OfType<RequestComplianceReview>()
        .Should().ContainSingle();

    var state = await workflows.GetAsync(message.CorrelationId);
    state!.ComplianceReviewRequested.Should().BeTrue();
}
```

The test proves the most important property: repeated delivery does not produce repeated downstream work. More tests should cover timeout escalation, failure transitions, late acknowledgements, and manual recovery.

TESTING RULE

Do not only test the happy path. The value of a process manager appears when events are late, duplicated, missing, corrected, or delivered after the workflow has already moved on.

Connection to the next chapter

This chapter introduced the process manager as explicit workflow memory. The next chapter compares two ways of distributing a workflow: saga coordination, where a coordinating component directs the steps, and choreography, where services react to events without a central coordinator. Neither is automatically superior. Each is a trade-off in visibility, coupling, ownership, and failure handling.

CHECKPOINT

1. What does a process manager own, and what must it not own?
2. Why is a process manager different from an application service?
3. Why must the process manager update its state and write outbox messages in one local transaction?
4. Why is idempotency mandatory for a process manager?
5. In the NEXUS-1 flow, which service owns the verdict and which component owns the memory that compliance was requested?

HONEST BOUNDARY

This chapter does not claim that every NEXUS-1 flow needs a process manager. Many event consumers can remain simple. Reporting projections may consume events directly. Audit records may be written by local append-only mechanisms. The process-manager pattern is justified only where the workflow itself needs durable memory, correlation, retry, timeout, or escalation. Used elsewhere, it becomes ceremony.

Chapter 33 closes the first step of distributed workflow design: make the memory of the flow explicit. Chapter 34 will decide whether that memory should sit in a coordinator, emerge through choreography, or remain inside a simpler local service.

PART VI - DISTRIBUTED WORKFLOWS

34. Saga Coordination vs Choreography

A saga is not a magic transaction replacement. It is a way of admitting that several services must cooperate without pretending they can commit, roll back, and agree at the same instant.

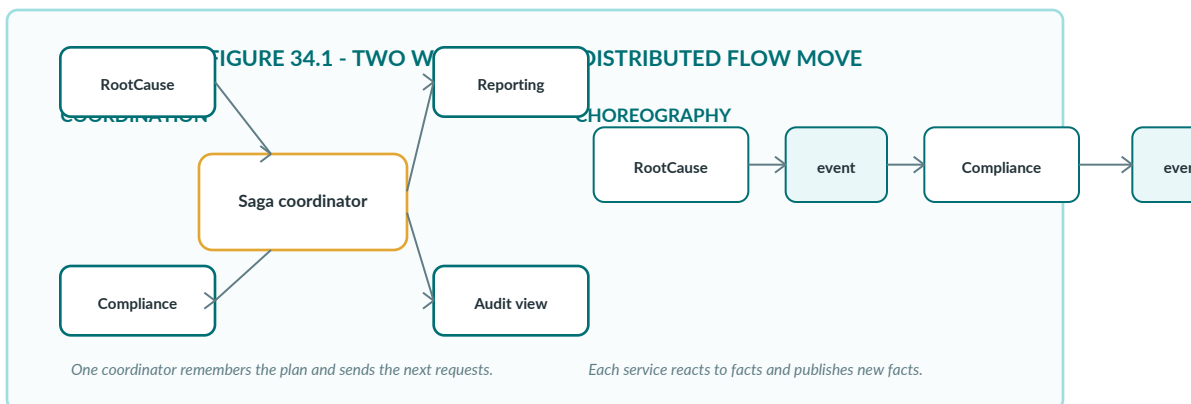


Figure 34.1 - Coordination and choreography are two ways to move a distributed workflow. The first makes the plan visible in one coordinator; the second lets services react to events and publish the next facts.

CHAPTER RULE

Use coordination when the workflow needs visible memory, branching, timers, compensation, or human escalation. Use choreography when the reaction is local, obvious, and stable. Do not use either word to hide ownership.

What you already know

You have already seen the core problem. A root-cause verdict is issued in one service, but the system is not finished. Compliance may need a review. Reporting may need a new read model. The operator console may need a status update. Audit must be able to replay what happened. Some steps may succeed, some may fail, and some may arrive late.

Chapter 33 gave that problem a first answer: a process manager can remember the flow. This chapter asks the next question. Should one component coordinate the steps, or should every service simply react to the events it sees and publish its own follow-up events?

THE FALSE CHOICE

The wrong debate is "saga or no saga" as if the word itself creates reliability. The real debate is where the flow rules live, who can see the state of the flow, and how failure becomes recoverable rather than invisible.

What a saga means in this book

In this book, a saga is a long-running business transaction split across local service transactions. Each service commits its own work. The wider flow reaches completion only after several local facts have been produced and observed. If something cannot complete, the system does not roll back a distributed database transaction; it compensates, retries, escalates, or asks a human to review the situation.

That definition is deliberately modest. A saga is not a guarantee that the whole system behaves like one ACID transaction. It is a design for surviving the fact that it cannot.

Coordination

In a coordinated saga, one component knows the plan. It receives events, stores saga state, decides the next step, sends commands or requests, handles timeouts, and records whether compensation or escalation is needed. This component may be called a saga coordinator, workflow manager, or process manager. The naming is less important than the responsibility.

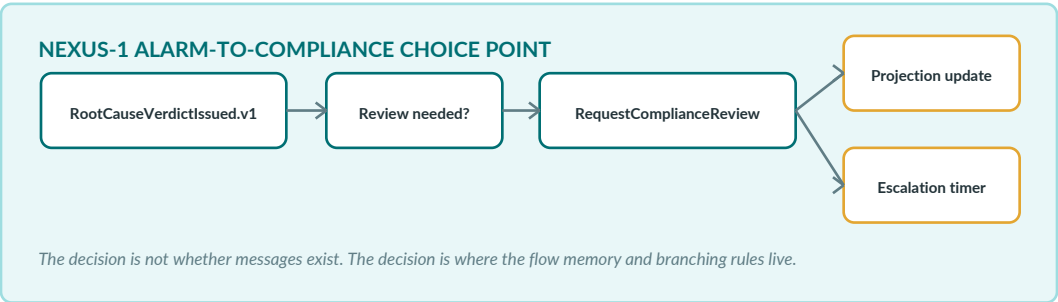


Figure 34.2 - The NEXUS-1 flow contains a branching point: after the root-cause verdict, someone must decide which downstream work is required and whether missing responses need escalation.

COORDINATION FIT

Coordination fits the alarm-to-compliance path when the platform must show one user-visible workflow status: verdict observed, compliance requested, reporting notified, timeout pending, or human review required.

The coordinator is useful because it makes the plan inspectable. A support engineer can query one workflow row and see what the system believes is happening. The cost is coupling: the coordinator knows the order of steps and the names of downstream requests. If every business rule moves there, it becomes a central brain.

Strength	Why coordination helps
Visible workflow state	Operators and support staff can see the current step and the last known result.
Branching and timers	The coordinator can decide when to request compliance, wait, retry, or escalate.
Compensation path	A failed step can trigger a deliberate corrective action rather than silence.
Testable flow logic	Given state plus event, the next message and new state are predictable.

The coordinator should still speak in public contracts. It should not reach into RootCause tables, Compliance tables, or Reporting tables. Its input is a public event such as RootCauseVerdictIssued.v1. Its output is a request such as RequestComplianceReview. It coordinates across boundaries; it does not bypass them.

DANGER

A coordinated saga becomes harmful when every service is reduced to a thin CRUD worker and all domain decision-making moves into the coordinator. That is not microservice architecture. It is a procedural monolith with network hops.

Choreography

In choreography, there is no single coordinator that knows the whole plan. A service publishes a fact. Other services that care about that fact react locally. Their reaction may publish more facts, and other services may react to those. The flow emerges from the subscription graph.

For simple reactions, choreography is elegant. RootCause publishes RootCauseVerdictIssued.v1. Reporting consumes it and updates a projection. Audit consumes it and records evidence. No central component needs to tell Reporting or Audit

what to do, because each reaction belongs clearly to the receiving service.

CHOREOGRAPHY FIT

Choreography fits when the downstream reaction is local, idempotent, stable, and does not need a user-visible shared workflow status. Reporting projections and audit evidence are often good candidates.

The cost is visibility. If the flow fails halfway, there may be no single place to ask what happened. You must reconstruct the story from event logs, outbox rows, inbox rows, projection state, and traces. That is acceptable for some flows and dangerous for others.

Concern	Coordination	Choreography
Flow visibility	High: one coordinator stores progress.	Distributed: visibility comes from logs, traces, and local states.
Coupling	Coordinator knows the sequence and request names.	Services know events they subscribe to, not the whole plan.
Branching	Easy to centralize and inspect.	Possible but can become hard to reason about.
Failure recovery	Coordinator can own retry, timeout, and escalation state.	Each service must own its own recovery; global status is harder.
Best fit	Long-running flows with timers and human review.	Simple local reactions to stable public events.

The table should not be read as a scorecard where one column wins. It is a decision aid. A system can use both. NEXUS-1 should use both.

COMBINED DESIGN

A coordinated saga can manage the critical alarm-to-compliance path while simple consumers remain choreographed. The coordinator does not need to tell Reporting how to build its read model; Reporting can consume the same event and rebuild itself.

The NEXUS-1 split

The cleanest design is not to force the whole platform into one style. NEXUS-1 has at least three kinds of reaction after a root-cause verdict:

- Critical workflow reaction: compliance review should be requested, tracked, retried, timed out, and escalated. This fits coordination.
- Read-side reaction: reporting projections should rebuild from public facts. This often fits choreography.

- Evidence reaction: audit should record what happened, but `AuditEntry` remains evidence, not workflow. This fits append-only recording rather than orchestration.

This split protects ownership. Compliance still owns the review. Reporting still owns the projection. Audit still owns evidence. The coordinator owns only the progress of the critical workflow that connects them.

CANONICAL EVENT DISCIPLINE

The trigger remains `RootCauseVerdictIssued.v1`. Do not invent `RootCauseCaseClosed` as the main public event, and do not rename the contract to `RootCauseVerdictPublished` if the book has already established `RootCauseVerdictIssued.v1` as the public language.

A coordinated shape in C#

A coordinator handler can be small if it respects boundaries. It reads workflow state, checks the next required step, writes outgoing messages to the outbox, and stores its new state. The downstream services still decide locally.

```
public async Task Handle(RootCauseVerdictIssuedV1 message, CancellationToken ct)
{
    var saga = await sagas.LoadOrStartAsync(message.CorrelationId, ct);

    if (saga.AlreadyHandled(message.EventId))
        return;

    saga.ObserveVerdict(message.RootCauseCaseId, message.IssuedAtUtc);

    if (saga.ShouldRequestComplianceReview())
    {
        await outbox.AddAsync(new RequestComplianceReview(
            message.CorrelationId,
            message.RootCauseCaseId,
            message.VerdictCode), ct);

        saga.MarkComplianceReviewRequested();
    }

    saga.MarkHandled(message.EventId);
    await unitOfWork.SaveChangesAsync(ct);
}
```

The handler does not call `Compliance` synchronously. It does not update a reporting database. It does not write audit evidence directly unless that evidence belongs to the local orchestration store. It produces durable intent and leaves each owner to do its own work.

A choreographed shape in C#

A choreographed consumer is even smaller. Reporting consumes the public event because it owns a read model that should reflect root-cause verdicts. It does not need the coordinator to tell it to rebuild a projection if the rule is simple and stable.

```
public async Task Handle(RootCauseVerdictIssuedV1 message, CancellationToken ct)
{
    if (await inbox.HasProcessedAsync(message.EventId, ct))
        return;

    await projectionStore.UpsertRootCauseVerdictAsync(new VerdictProjectionRow(
        message.RootCauseCaseId,
        message.UnitId,
        message.VerdictCode,
        message.IssuedAtUtc), ct);

    await inbox.MarkProcessedAsync(message.EventId, ct);
}
```

This consumer has no global workflow memory. That is acceptable because its responsibility is local: keep a projection current. If it fails, its inbox and projection can be replayed. It does not decide whether compliance review is required.

SEPARATION RULE

Use coordination for the flow that must be tracked as a whole. Use choreography for local reactions that can be replayed independently. Never use choreography as an excuse to make the overall business process impossible to understand.

Compensation changes the decision

The moment a flow needs compensation, coordination becomes more attractive.

Compensation means doing another explicit action because an earlier action cannot simply be rolled back. If a compliance review was opened for a verdict that is later corrected, the system may need a `RootCauseVerdictCorrected.v1` event and a follow-up action that updates or closes the review.

Choreography can still handle this, but the compensation logic becomes scattered.

Compliance must react to correction events. Reporting must rebuild. The operator console must explain why the status changed. Without careful observability, the reader of the system has to infer the process from fragments.

NEXUS-1 CORRECTION CASE

`RootCauseVerdictCorrected.v1` should not pretend the original event disappeared. It is a new fact. The saga or consumers must interpret it as correction, not erasure. Auditability depends on preserving both the original verdict and the correction path.

Timeouts change the decision even more

Timeouts are where choreography often becomes difficult. If Compliance does not acknowledge a review request within a defined window, who notices? If Reporting has not

caught up, who shows the operator that the projection is stale? If a message is stuck in a dead-letter queue, who turns that into a visible workflow status?

A coordinator gives those questions a home. It can store a deadline, schedule a timeout check, and move the workflow to Waiting, Escalated, or NeedsHumanReview. Choreography can achieve similar results, but only if each service publishes enough status events and the observability layer can reconstruct the process reliably.

Flow feature	Prefer coordination when...	Choreography remains enough when...
Timeout	A missed response must change global workflow status.	The local consumer can retry or replay without operator involvement.
Compensation	Several services need a visible corrective sequence.	Each service can independently react to the correction event.
Human review	A person must see one coherent case state.	Humans only inspect local service screens.
Audit explanation	The reason for each step must be reconstructed as one workflow story.	Independent append-only evidence is sufficient.

This is why the architecture decision must be made per flow, not per fashion. A team that says "we use choreography" for everything will eventually hide important workflow state. A team that says "we coordinate everything" will eventually rebuild central control.

Operational visibility

Both approaches need observability. Coordination needs it because the coordinator can become a bottleneck or a single point of misunderstanding. Choreography needs it because the flow is distributed by design. In both cases, correlation IDs are not optional. They are the thread by which the story is followed across services.

For the NEXUS-1 operator console, this has a practical consequence: a user should not need to know which service failed in order to understand the flow state. The console should be able to say: verdict issued; compliance request queued; compliance acknowledgement missing; reporting projection last updated at this time; audit evidence recorded. That view can be built from a coordinator state, a workflow read model, or traces, but it must exist.

CONNECTIVITY RULE

Every event, request, inbox row, outbox row, projection update, and workflow-state change must carry the same correlation ID. Without it, coordination becomes blind and choreography becomes archaeology.

Choosing for NEXUS-1

The recommended NEXUS-1 design is mixed and explicit:

- Coordinate the alarm-to-compliance workflow because it has branching, user-visible status, timeout risk, and human review.
- Choreograph reporting projection updates because Reporting owns its read model and can replay from events.
- Record audit evidence without letting AuditEntry become workflow control.
- Publish correction events as new facts rather than mutating the past.
- Correlate every message and state row so the entire flow can be explained.

ARCHITECTURAL DECISION

NEXUS-1 should not choose coordination or choreography as a platform religion. It should choose coordination for critical, stateful, human-visible workflows and choreography for simple, local, replayable reactions. This preserves both clarity and autonomy.

Connection to the next chapter

The next chapter applies this choice to the full alarm-to-compliance flow. It will walk the path from AlarmFloodDetected.v1 through root-cause case handling, verdict issuance, compliance review, audit evidence, reporting projection, timeout, and operator visibility. The question will no longer be abstract: it will be which step owns which responsibility.

CHECKPOINT

1. What is a saga in this book, and what does it not guarantee?
2. When does coordination help more than choreography?
3. When is choreography the cleaner choice?
4. Why should NEXUS-1 use a mixed design rather than one style everywhere?
5. Why must RootCauseVerdictCorrected.v1 be treated as a new fact rather than an erasure?

HONEST BOUNDARY

This chapter does not implement a production workflow engine, broker scheduler, timeout worker, or distributed tracing stack. It defines the architectural choice. A real implementation would still need persistence, lease-safe workers, retry policies, poison-message handling, correlation propagation, dashboards, alerts, and operational playbooks. Those details are not optional in production; they are simply beyond this chapter.

Chapter 34 closes the choice between coordination and choreography. The next chapter turns that choice into the concrete NEXUS-1 alarm-to-compliance distributed flow.

PART VI - DISTRIBUTED WORKFLOWS

35. The Alarm-to-Compliance Distributed Flow

This is the flow the previous chapters prepared for: a fact born in alarms, interpreted by root-cause reasoning, carried by events, remembered by workflow state, reviewed by compliance, recorded by audit, projected by reporting, and explained to the operator without pretending that one transaction held it all together.

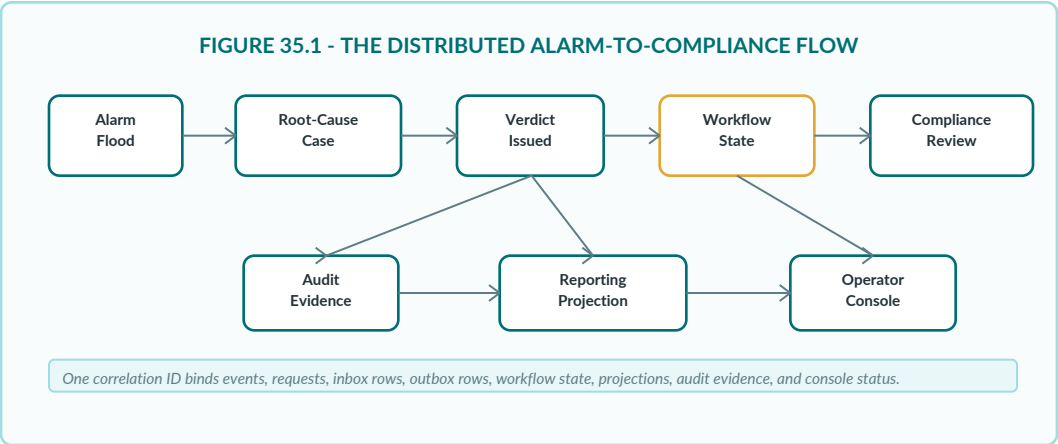


Figure 35.1 - The full distributed path. No service owns the whole domain, but the flow must still be traceable as one story through correlation, workflow state, audit evidence, and projections.

CHAPTER RULE

A distributed workflow must name three things separately: who owns the fact, who remembers the progress, and who explains the status. When those three responsibilities are confused, the system becomes either a hidden monolith or an untraceable event cloud.

What you already know

The book has already built the ingredients. Chapter 18 protected service APIs from leaking domain internals. Chapter 19 separated commands, queries, and public contracts. Chapters 25 to 32 established the messaging discipline: integration events, delivery guarantees, outbox, inbox, idempotency, ordering, dead-letter handling, and contract names. Chapters 33 and 34 then asked where a long-running workflow should live and when coordination is preferable to choreography.

This chapter connects those decisions into the recurring NEXUS-1 path. The path begins when AlarmManagement detects an alarm flood and ends when the operator console can show a coherent status: what happened, what root-cause verdict was issued, whether compliance review was requested, whether audit evidence exists, and whether reporting has caught up.

THE CENTRAL QUESTION

How can the system let several services own their own data and still let a human follow one case from alarm flood to compliance review without guessing? The answer is not a shared database. The answer is explicit contracts, local transactions, workflow state, correlation, and replayable evidence.

The flow as a story

A distributed flow should first be readable in ordinary language. NEXUS-1 should be able to tell the story without mentioning RabbitMQ, SQL Server, HTTP, or any framework:

- AlarmManagement detects a pattern that qualifies as an alarm flood.
- RootCause opens or updates a case and attaches evidence.
- RootCause issues a verdict and publishes RootCauseVerdictIssued.v1.
- The flow manager observes the verdict and decides whether compliance review is required.
- Compliance receives a request and opens a review if the rule is satisfied.
- Audit records evidence but does not drive the workflow.
- Reporting updates read models for dashboards and exports.
- The operator console composes status from workflow state, read models, and service APIs.

That story is simple. Keeping it true after distribution is the hard part.

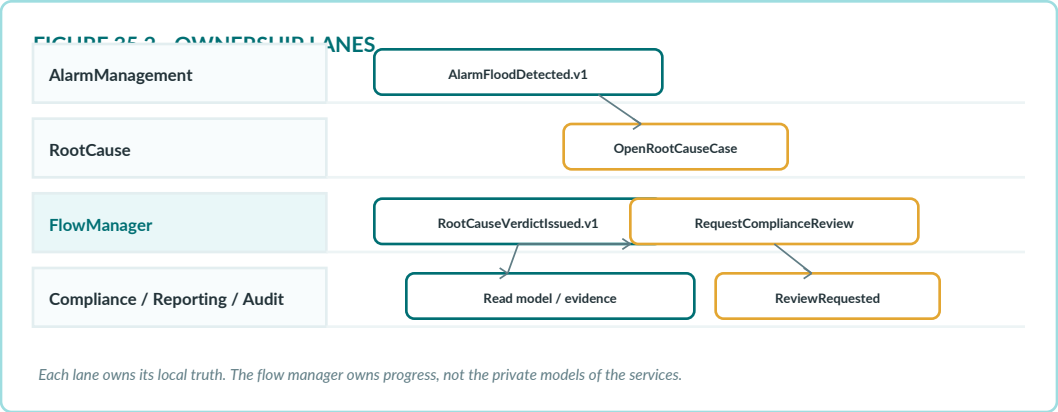


Figure 35.2 - The flow is not one service stretched across several databases. It is a set of ownership lanes connected by public facts and requests.

Step 1 - AlarmManagement owns the flood

The first owner is AlarmManagement. It owns alarm definitions, alarm events, acknowledgements, flood detection rules, and flood state. It may know that fourteen alarms arrived inside a window and that the pattern should be treated as a flood. It does not own a root-cause verdict and it does not own compliance review.

The public fact from this service is not an internal alarm-domain object. It is a contract such as `AlarmFloodDetected.v1`. That contract should contain enough information for downstream services to correlate the event and decide whether to react: flood id, unit id, time window, severity, summary, affected signal references, and correlation id. It should not expose AlarmManagement table names or private aggregate fields.

OWNERSHIP RULE

AlarmManagement can say: a flood was detected. AlarmManagement should not say: this is the root cause, this requires regulatory review, or this dashboard is now complete. Those are different ownership decisions.

Step 2 - RootCause owns the verdict

RootCause consumes the public alarm-flood fact or receives an application request to open a case. It creates a `RootCauseCase`, attaches evidence, promotes or rejects hypotheses, and eventually issues a verdict. The verdict is a domain decision inside RootCause before it becomes a public integration event.

The internal domain event may still be named `VerdictIssued`. The public integration event remains `RootCauseVerdictIssued.v1`. The distinction matters because the internal event can

change with the model, while the public event is a contract other services depend on.

CANONICAL CONTRACT

RootCauseVerdictIssued.v1 is the public fact. Avoid replacing it with vague names such as RootCauseCaseClosed or RootCauseVerdictPublished if the earlier language has already established the issued-verdict contract. The system should have one canonical public name for the same fact.

Fact or request	Owner	Meaning in the flow
AlarmFloodDetected.v1	AlarmManagement	A flood pattern has been detected and can be used as input by other contexts.
OpenRootCauseCase	RootCause API / application layer	A request to create or attach to a root-cause case; not a public fact by itself.
VerdictIssued	RootCause domain	Internal domain event raised by the aggregate after a valid verdict decision.
RootCauseVerdictIssued.v1	RootCause public contract	Stable integration event observed by workflow, compliance, reporting, and audit.
RequestComplianceReview	Flow manager / Compliance boundary	A request that compliance review should be considered or opened.

Step 3 - The flow manager owns progress

The flow manager is not the owner of alarms, root-cause reasoning, compliance rules, reporting projections, or audit evidence. Its job is narrower and more important: it remembers where the distributed flow is, which public fact was observed, which request was sent, what deadline exists, and what status should be visible to the operator.

FIGURE 35.3 - WORKFLOW STATUS IS A FIRST-CLASS MODEL



Figure 35.3 - A human-visible distributed process needs a status model. Without it, the operator only sees isolated service states.

This is the main difference between workflow memory and business ownership. The flow manager may store a row that says ComplianceReviewRequested. That does not mean it owns the compliance review. It means it owns the progress record that says the request was sent and must be acknowledged or escalated.

BOUNDARY

The flow manager stores progress, not domain truth. If it stores the actual root-cause verdict as its own editable truth, it has started duplicating RootCause. If it stores compliance findings as its own truth, it has started duplicating Compliance.

Step 4 - Compliance owns the review

Compliance should not be a passive database table updated by another service. It owns its own rules: which verdicts require review, which control or regulation applies, who can approve, what evidence is required, and what final finding is recorded. The flow manager can request a review, but Compliance decides how that request becomes a local review record.

This protects the Compliance model from becoming a remote extension of RootCause. It also allows Compliance rules to evolve independently. If tomorrow a different verdict severity requires review, the Compliance service changes its own decision rule without forcing RootCause to know regulatory workflow details.

Step 5 - Audit records evidence, not workflow control

Audit is part of the flow but not the conductor of the flow. It records evidence that something happened: event observed, request sent, review opened, verdict corrected, timeout escalated. It should not be used as the mechanism that drives the next business step.

This distinction repeats one of the series rules: an AuditEntry is evidence, not workflow. A service can read audit evidence for inspection or reconstruction, but a normal business process should not depend on scanning audit rows to decide what to do next.

AUDIT RULE

If the only way to know the next step is to query audit entries, the workflow model is missing. Audit must make the story replayable; it must not become the hidden process manager.

Step 6 - Reporting builds read models

Reporting should not become the source of truth. It is the read side. It consumes public events and service-owned data through allowed contracts and builds projections optimized for screens, exports, and summaries. If its projection is stale, that is a read-model condition, not a domain fact.

For the operator console this is crucial. A dashboard can show a combined view of the case, compliance review, and audit evidence, but the dashboard does not own those facts. It is a composed view over multiple service-owned truths and workflow status.

Console field	Source of truth	What the console may display
Alarm flood status	AlarmManagement	Detected, active, acknowledged, closed, or suppressed according to alarm rules.
Root-cause verdict	RootCause	Issued verdict, evidence summary, confidence or abstention state if provided.
Workflow progress	Flow manager	Review requested, waiting, completed, timed out, escalated, or corrected.
Compliance status	Compliance	Review opened, assigned, pending evidence, finding recorded, or closed.
Audit evidence exists	Audit	Evidence recorded with correlation id and immutable timestamps.
Dashboard summary	Reporting projection	A convenient read model, with freshness shown when necessary.

The happy path sequence

A possible happy path is intentionally boring. Each service commits locally. Each public fact is published through the outbox. Each consumer uses an inbox. The correlation id is carried end to end.

```
1. AlarmManagement commits AlarmFlood and outbox event AlarmFloodDetected.v1.
2. Broker delivers the event to RootCause or an application orchestrator.
3. RootCause opens the case, attaches evidence, and later issues VerdictIssued.
4. RootCause outbox publishes RootCauseVerdictIssued.v1.
5. FlowManager inbox records the event and creates workflow state.
6. FlowManager sends RequestComplianceReview and records a deadline.
7. Compliance opens a local review and publishes ComplianceReviewRequested.v1.
8. Reporting updates the read model from public events.
9. Audit records append-only evidence for each important fact.
10. Operator Console displays composed status with correlation and freshness.
```

Nothing in this sequence requires a distributed transaction. Nothing in it requires services to share tables. The price is that the flow is only reliable if every step is durable, idempotent, observable, and correlated.

The failure paths that make the design real

The happy path is not the test of the architecture. Failure paths are the test. The NEXUS-1 flow must be able to explain at least four uncomfortable cases:

Failure	Required behaviour
Duplicate verdict event	FlowManager and downstream consumers ignore it through inbox idempotency, but preserve traceability.
Compliance unavailable	Request remains durable; retry policy and timeout deadline become visible workflow state.
Reporting projection delayed	Console shows stale or pending projection instead of pretending the read model is current.
Verdict corrected later	RootCauseVerdictCorrected.v1 is published as a new fact; workflow and compliance respond without erasing history.
Poison message	Dead-letter handling creates operational visibility and avoids endless invisible retry loops.

FAILURE RULE

If a failure cannot be shown to a support engineer as a named state, it will eventually become a mystery. Distributed workflows need business states and operational states, not just exceptions in logs.

A small implementation shape

The flow manager can be implemented as a service that consumes RootCauseVerdictIssuedV1. The handler is not allowed to call private RootCause tables or mutate Compliance tables directly. It records workflow progress and sends a request through a boundary.

```
public sealed class RootCauseVerdictIssuedConsumer
{
    public async Task Handle(RootCauseVerdictIssuedV1 message, CancellationToken ct)
    {
        if (await inbox.HasProcessedAsync(message.EventId, ct))
            return;

        var flow = await workflowStore.GetOrCreateAsync(
            message.CorrelationId,
            message.RootCauseCaseId,
            ct);

        flow.MarkVerdictObserved(message.EventId, message.IssuedAtUtc);

        if (flow.RequiresComplianceReview(message.VerdictCode))
        {
            flow.MarkComplianceRequested(clock.UtcNow);
            await requestBus.SendAsync(new RequestComplianceReview(
                message.CorrelationId,
                message.RootCauseCaseId,
                message.UnitId,
                message.VerdictCode), ct);
        }

        await workflowStore.SaveAsync(flow, ct);
        await inbox.MarkProcessedAsync(message.EventId, ct);
    }
}
```

The example is deliberately small. It shows the architectural shape, not a production worker. A real worker would need transactional inbox handling, outbox-backed request sending, leases or concurrency protection, retries, metrics, structured logging, and alerting.

CODE DISCIPLINE

The handler works with public contracts and workflow state. It does not receive a `RootCauseCase` entity. It does not expose a `ComplianceReview` entity. It does not update Reporting tables. That is the point of the boundary.

The minimum workflow table

The workflow state can be stored in a small orchestration table or a set of tables. The key design point is not the exact schema. The key point is that status, deadlines, correlation, and last observed event are durable.

```
CREATE TABLE Workflow.AlarmComplianceFlow (  
    AlarmComplianceFlowId BIGINT IDENTITY(1,1) NOT NULL,  
    CorrelationId UNIQUEIDENTIFIER NOT NULL,  
    RootCauseCaseId BIGINT NOT NULL,  
    UnitId INT NOT NULL,  
    CurrentStatus NVARCHAR(40) NOT NULL,  
    LastObservedEventId UNIQUEIDENTIFIER NULL,  
    ComplianceRequestedAtUtc DATETIME2(7) NULL,  
    ComplianceDeadlineUtc DATETIME2(7) NULL,  
    CompletedAtUtc DATETIME2(7) NULL,  
    EscalatedAtUtc DATETIME2(7) NULL,  
    RowVersion ROWVERSION NOT NULL,  
    CONSTRAINT PK_Workflow_AlarmComplianceFlow PRIMARY KEY (AlarmComplianceFlowId),  
    CONSTRAINT UQ_Workflow_AlarmComplianceFlow_Correlation UNIQUE (CorrelationId)  
);
```

This table is not the source of truth for the verdict or the review. It is the source of truth for the progress of this cross-service flow. That is a legitimate service responsibility.

What the operator console should say

The operator console should never need to expose the internal architecture to be honest. It does not have to say: RabbitMQ delivered a message to consumer group X and the outbox publisher is on retry attempt three. It should say the operationally meaningful thing: compliance review request is pending; last attempt failed; retry scheduled; human escalation deadline is approaching.

That is why the book keeps returning to the distinction between technical events and user-facing status. A distributed system has many technical facts, but the user needs a coherent status model.

CONSOLE RULE

The console should compose service-owned truth without becoming the owner of that truth. It may display a workflow state, but it must not invent one from incomplete projections when the workflow service has not confirmed it.

Connection to the next chapter

This chapter established the distributed path. The next chapter goes deeper into the most uncomfortable part of that path: compensation. Once a verdict is corrected, a review request has already been sent, or a projection has already been built, the system cannot pretend time ran backward. It must publish new facts and perform deliberate corrective actions.

CHECKPOINT

1. Which service owns alarm-flood truth?
2. Which service owns the root-cause verdict?
3. What does the flow manager own, and what must it not own?
4. Why is AuditEntry evidence rather than workflow control?
5. What should the operator console show when Reporting is delayed?
6. Why does RootCauseVerdictCorrected.v1 represent a new fact rather than an erasure?

HONEST BOUNDARY

This chapter defines the architecture of the NEXUS-1 alarm-to-compliance flow. It does not certify any operational workflow, prescribe nuclear compliance procedure, or implement a production workflow engine. The flow is a software-architecture demonstration for an educational digital-twin system. Production use would require domain authority, regulatory validation, safety analysis, security review, operational runbooks, and far more testing than a book chapter can provide.

Chapter 35 turns the abstract saga discussion into one concrete NEXUS-1 flow. The next chapter explains how that flow survives when later facts require correction instead of rollback.

PART VI - DISTRIBUTED WORKFLOWS

36. Compensation Without Distributed Transactions

A distributed system cannot make time run backward. Once a service has committed, published, requested, projected, or audited something, the honest repair is not erasure. The honest repair is a new fact and a deliberate compensating action.

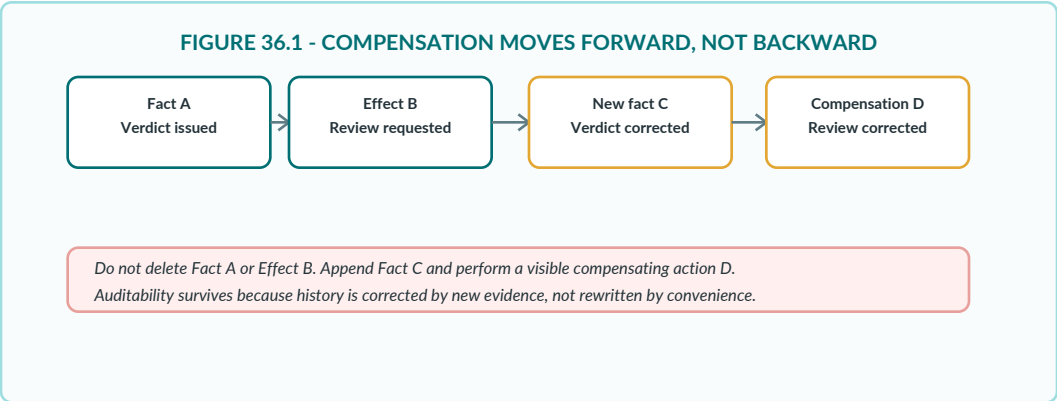


Figure 36.1 - Compensation does not delete the past. It appends a correcting fact and then performs a visible forward action that repairs the business effect.

CHAPTER RULE

In a microservice architecture, compensation is not a technical trick that imitates rollback. It is a domain-visible corrective action. If the earlier action mattered enough to cross a service boundary, the correction must matter enough to be named, stored, published, and audited.

What you already know

In a monolith, a developer can often rely on one database transaction. Insert the case row, update the review row, write the audit row, commit once, and sleep well. The transaction boundary and the model boundary are the same shape, so rollback feels natural.

The previous chapters deliberately broke that comfort. AlarmManagement owns alarm floods. RootCause owns verdicts. Compliance owns reviews. Reporting owns projections. Audit owns evidence. The flow manager owns progress. Chapter 35 showed the whole alarm-to-compliance path as a set of local truths connected by events and requests. This chapter asks what happens when the path must be corrected after it has already moved.

THE DISTRIBUTED FACT

Once RootCause publishes RootCauseVerdictIssued.v1, downstream services may react. Compliance may open a review. Reporting may build a read model. Audit may record evidence. The system cannot safely pretend that none of this happened just because RootCause later learns more.

Why distributed rollback is the wrong mental model

The naive answer is to wish for one transaction that spans services. The wish is understandable and usually dangerous. Distributed transactions couple independent services at runtime, make availability depend on a shared commit protocol, and turn partial failure into a coordination problem that is harder than the business problem the services were meant to solve.

More importantly, distributed rollback often lies about meaning. If a compliance review was opened, an operator saw it, an audit entry recorded it, and a report included it, then the business world has changed. The correction is not to pretend the world never changed. The correction is to say: this earlier action was based on an earlier fact; a newer fact supersedes it; here is the compensating action and here is the trail.

Rollback thinking	Compensation thinking
Try to erase the effect of an earlier step.	Append a new fact that explains why the earlier step must be corrected.
Treat a distributed workflow as if it were one large transaction.	Treat each service as locally consistent and repair cross-service effects explicitly.

Rollback thinking	Compensation thinking
Hide partial progress until everything succeeds.	Expose progress, pending corrections, deadlines, and human review states.
Delete audit evidence when a path is reversed.	Preserve audit evidence and add correction evidence.

The NEXUS-1 compensation cases

The alarm-to-compliance flow gives useful examples because every downstream reaction has a different owner. A correction in RootCause does not give RootCause permission to mutate Compliance, Reporting, or Audit. It gives RootCause the responsibility to publish a new public fact that other services can interpret.

What changed?	Earlier effect	Compensation
A verdict is corrected.	Compliance review was already requested.	Publish RootCauseVerdictCorrected.v1; request review correction or review cancellation depending on Compliance rules.
Evidence is reclassified.	Report projection included old evidence category.	Mark projection stale and rebuild read model from the corrected fact stream.
Recommendation is withdrawn.	Operator console showed an advisory recommendation.	Publish RecommendationWithdrawn.v1 or revised recommendation; show superseded status, not silent removal.
Compliance finding changes.	Audit evidence exists for the earlier finding.	Append correction evidence; never delete the original audit entry.

NEXUS-1 LANGUAGE DISCIPLINE

RootCauseVerdictCorrected.v1 is a new public fact. It is not a patch to an old event and not a hidden database update. The old verdict remains part of history; the corrected verdict becomes the current truth from the time it is issued.

Compensation starts with the service that owns the new fact

The correcting fact begins where the new knowledge belongs. If the verdict changed, RootCause owns the correction. If a review decision changed, Compliance owns the correction. If a projection was stale, Reporting owns the rebuild status. This rule prevents the flow manager from becoming a privileged god-service that edits everyone else's truth.

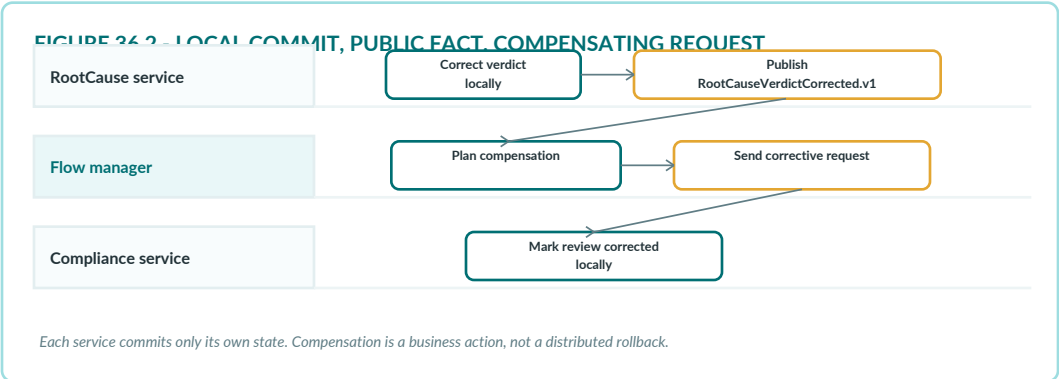


Figure 36.2 - The service that owns the corrected fact commits locally and publishes publicly. Other services compensate their own effects through their own rules.

The flow manager may observe the corrected fact and plan a compensation sequence. It may record that a corrective request was sent. It may track acknowledgement, timeout, escalation, and completion. But it should not reach into Compliance tables to reverse a review, and it should not reach into Reporting tables to overwrite a projection.

The compensation ledger

A serious compensation mechanism needs memory. Otherwise the architecture becomes a collection of clever handlers with no durable explanation of what they have tried to repair. That memory may be a table inside the workflow service: compensation id, correlation id, observed fact, affected effect, compensating action, attempts, deadline, status, and final outcome.

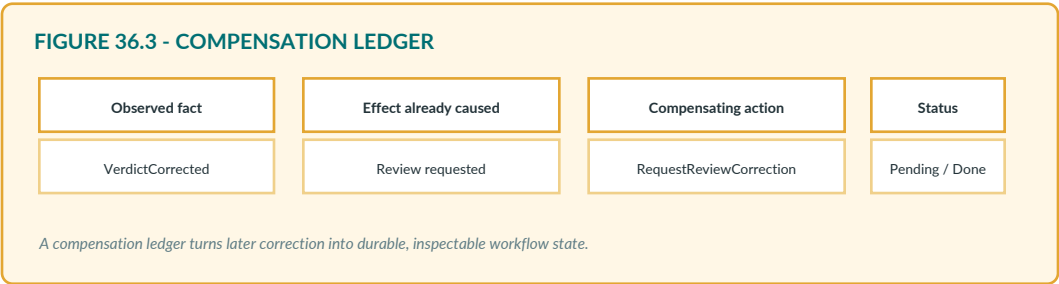


Figure 36.3 - A compensation ledger makes correction inspectable. It gives support engineers and the operator console a place to read what is being repaired.

```
CREATE TABLE Workflow.CompensationAction (
    CompensationActionId BIGINT IDENTITY(1,1) NOT NULL,
    CorrelationId UNIQUEIDENTIFIER NOT NULL,
    TriggerEventId UNIQUEIDENTIFIER NOT NULL,
    TriggerEventType NVARCHAR(120) NOT NULL,
    AffectedService NVARCHAR(80) NOT NULL,
    ActionType NVARCHAR(80) NOT NULL,
    Status NVARCHAR(40) NOT NULL,
    AttemptCount INT NOT NULL CONSTRAINT DF_Compensation_AttemptCount DEFAULT 0,
    NextAttemptAtUtc DATETIME2(7) NULL,
    CompletedAtUtc DATETIME2(7) NULL,
    LastError NVARCHAR(1000) NULL,
    RowVersion ROWVERSION NOT NULL,
    CONSTRAINT PK_Workflow_CompensationAction PRIMARY KEY (CompensationActionId)
);
```

This table does not become the source of truth for the corrected verdict or the compliance review. It becomes the source of truth for the compensating work that must be performed because a new fact made an earlier effect incomplete, outdated, or wrong.

A small implementation shape

A compensation planner consumes a corrected event and creates compensating actions. The important detail is that planning and execution are not the same thing. First the system records what must be done. Then workers execute those actions idempotently.

```
public sealed class VerdictCorrectedConsumer
{
    public async Task Handle(RootCauseVerdictCorrectedV1 message, CancellationToken ct)
    {
        if (await inbox.HasProcessedAsync(message.EventId, ct))
            return;

        var flow = await workflowStore.GetByCorrelationAsync(message.CorrelationId, ct);
        flow.MarkCorrectedVerdictObserved(message.EventId, message.CorrectedAtUtc);

        if (flow.ComplianceReviewWasRequested)
        {
            flow.PlanCompensation(
                affectedService: "Compliance",
                actionType: "RequestReviewCorrection",
                triggerEventId: message.EventId);
        }

        if (flow.ReportingProjectionWasBuilt)
        {
            flow.PlanCompensation(
                affectedService: "Reporting",
                actionType: "MarkProjectionStale",
                triggerEventId: message.EventId);
        }

        await workflowStore.SaveAsync(flow, ct);
        await inbox.MarkProcessedAsync(message.EventId, ct);
    }
}
```

The consumer does not synchronously call every downstream service in the same method and hope the network behaves. It records the need for compensation and lets dedicated, retryable, observable workers carry out each step. That keeps correction durable even when one downstream service is unavailable.

IMPLEMENTATION RULE

Compensation should be planned durably before it is attempted. If the process crashes after observing the correction but before sending the compensating request, the ledger should still tell the system what remains to do.

Commands used for compensation

Compensating requests should be named as clearly as normal business commands. A vague command such as UpdateReview is not enough. The name should reveal why the request exists and what business correction it asks for.

Weak name	Better name	Reason
UpdateCompliance	RequestComplianceReviewCorrection	States that the request corrects a prior review path because a newer fact exists.
DeleteReportRow	MarkRootCauseProjectionStale	Avoids silent erasure and tells Reporting to rebuild its own read model.
UndoRecommendation	WithdrawRecommendation	Creates a visible advisory change rather than pretending the recommendation never existed.
FixAudit	AppendCorrectionEvidence	Audit is append-only evidence, not editable workflow state.

Idempotency still matters

Compensation messages are messages. They can be delivered twice. They can arrive late. A worker can crash after sending a request and before marking the action complete. Therefore compensating requests need the same inbox and idempotency discipline as every other integration message in Part V.

A Compliance service receiving RequestComplianceReviewCorrection should be able to answer: have I already processed this compensation action id? Does this correction apply to a review I own? Is the review already closed? If it is closed, do I reopen, append a correction note, or escalate to human review? Those are domain rules, not messaging details.

IDEMPOTENCY RULE

A compensating action must be safe to retry. If retrying the same correction creates a second review, a duplicate finding, or two contradictory audit notes, the compensation mechanism has created a new failure instead of repairing the old one.

Compensation is not always automatic

Some compensations are safe to automate. Marking a projection stale, rebuilding a read model, or appending correction evidence can usually be automatic. Other compensations should stop at a human review boundary. Withdrawing a compliance review, changing a formal finding, or superseding a recommendation that a person may already have acted on may require an explicit human decision.

This is why Chapter 37 will discuss timeouts, escalations, and human review. Compensation is not a license for software to clean up every distributed consequence by itself. Sometimes the correct compensation is to make the unresolved state impossible to miss.

HUMAN-REVIEW BOUNDARY

A compensating action may be automatic, semi-automatic, or manual. The architecture should store which kind it is. In safety-adjacent or compliance-heavy systems, the most honest action is often to request human review rather than pretend the system knows how to undo a business consequence.

The operator-facing status

A correction should be visible to the operator in human language. The console should not merely show that an event handler failed or that a retry is pending. It should say what the system knows: verdict corrected; compliance review correction requested; reporting projection stale; audit correction recorded; awaiting compliance acknowledgement.

That status does not need to reveal every queue, consumer, and table. But it must not hide the fact that the system is in a correction path. Hidden compensation is worse than visible delay, because hidden compensation makes the screen look certain when the architecture is still repairing itself.

Connection to the next chapter

This chapter focused on correction. The next chapter focuses on time. A compensating action may be planned, but the target service may be unavailable. A review may be requested, but no one may acknowledge it. A projection may be marked stale, but the rebuild may not finish. Distributed workflows therefore need deadlines, timeouts, escalations, and human-visible states.

CHECKPOINT

1. Why is compensation different from rollback?
2. Why should RootCause publish RootCauseVerdictCorrected.v1 instead of editing the old event?
3. What does the compensation ledger remember?
4. Why must compensating actions be idempotent?
5. Which compensations in NEXUS-1 can be automatic, and which should require human review?
6. Why should the operator console expose correction state?

HONEST BOUNDARY

This chapter describes compensation as a software architecture pattern for the educational NEXUS-1 demonstrator. It is not regulatory guidance, not a safety procedure, and not a claim about how a certified operational plant should correct compliance or operational records. The architectural lesson is general: do not erase distributed history; publish corrective facts and perform visible compensating actions under the ownership of the service that owns the affected truth.

Chapter 36 replaces the fantasy of distributed rollback with a discipline of forward correction. The next chapter adds the time dimension: deadlines, escalations, and human review when compensation cannot finish quietly.

PART VI - DISTRIBUTED WORKFLOWS

37. Timeouts, Escalations, and Human Review

A distributed workflow that waits forever has not become reliable. It has only hidden failure. Timeouts make silence visible, escalation gives it an owner, and human review protects decisions that software should not close alone.

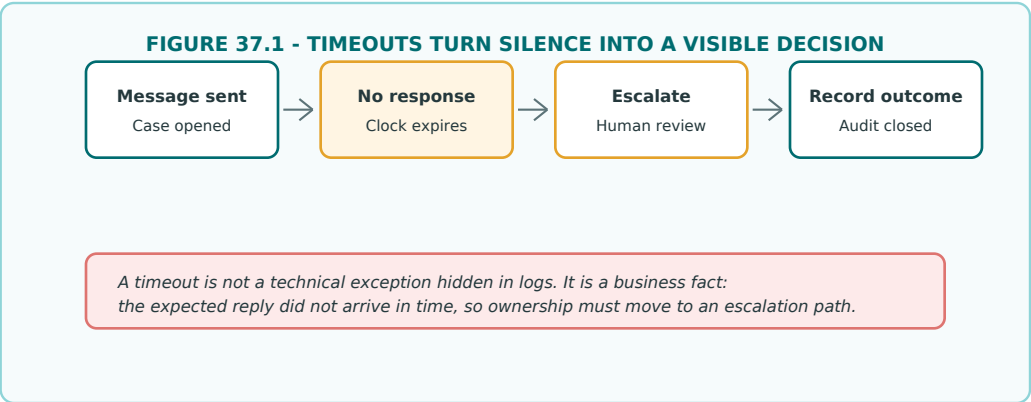


Figure 37.1 - A timeout converts absence into a recorded decision point. The system does not pretend that silence is success; it creates an explicit escalation path.

What you already know

You already know this from ordinary software. A request leaves your service, and nothing comes back. Perhaps the downstream service is busy. Perhaps the message was delivered but not processed. Perhaps a human approval is waiting in a queue nobody opened. The dangerous part is not only the delay. The dangerous part is pretending that the delay has no business meaning.

A timeout is the point at which a workflow stops treating silence as patience and starts treating it as information. In a monolith, this often looks like a synchronous call timing out. In a distributed workflow, the problem is wider: a service may be waiting for an event, a compensating action, a compliance review, or a human decision. If the wait has no deadline, the case can become immortal.

PLAIN EXPLANATION

A timeout is not just a technical setting such as 30 seconds. In a workflow, a timeout means: this expected thing did not happen in the allowed time, therefore the system must record that fact and move responsibility somewhere visible.

This chapter adds the time dimension to the distributed flow built in the previous chapters. Chapter 33 gave the process manager memory. Chapter 34 separated choreography from coordination. Chapter 35 walked the alarm-to-compliance flow. Chapter 36 showed how correction moves forward through compensation. Now we ask what happens when a step neither succeeds nor fails, but simply does not finish.

The hidden danger of waiting

Waiting feels harmless because it does not corrupt data immediately. That is the trap. A stuck workflow can be more dangerous than a failed workflow because failure is visible. A stuck workflow can leave the operator console in a half-truth: the root-cause verdict exists, the recommendation is pending, compliance has not reviewed it, reporting has not caught up, and nobody can say whether the system is late, broken, or merely slow.

For NEXUS-1, that matters because the alarm-to-compliance flow is accountability-heavy. It is not enough to know that a service was called. The book has insisted from the beginning that AuditEntry is evidence, not workflow, that a recommendation is not a command, and that a read model is not the source of truth. The same discipline applies to waiting: a delayed step must be represented as a workflow state, not buried in a retry loop.

A timeout belongs to the owner of the expectation

The first design question is not how many seconds to wait. The first question is who expected something to happen. A timeout belongs to the service or process manager that created the expectation. If RootCause publishes RootCauseVerdictIssued.v1 and Compliance is expected to open a review, the timeout does not belong to RootCause. RootCause issued the verdict. The distributed workflow owner tracks whether the compliance review was requested, accepted, or escalated.

This distinction prevents ownership drift. Without it, every service starts monitoring every other service and the workflow becomes a web of suspicion. With it, the process manager owns the lifecycle of the flow while each service owns its local truth.

Expectation	Wrong owner	Better owner	Reason
Compliance review should start after verdict	RootCause service	Alarm-to-compliance process manager	RootCause owns the verdict, not the downstream review lifecycle.
Recommendation should become visible	Operator console	Recommendation or workflow owner	The UI displays state; it should not invent workflow meaning.
Audit evidence should be written	Reporting service	Audit service plus workflow owner	Audit owns evidence; the workflow owner tracks whether evidence arrived.
Manual reviewer should decide	Any service with a timer	Human-review queue owner	A human decision must have accountable assignment and due time.

Technical timeout vs workflow timeout

A technical timeout protects a thread, connection, or request. A workflow timeout protects a business expectation. They are related, but they are not the same thing.

Kind	Example	What it protects	What it should record
HTTP timeout	Stop waiting after 5 seconds	Connection pool, caller latency, resource pressure	Usually a technical failure metric or retry decision
Message retry delay	Retry publish after 30 seconds	Delivery progress	Attempt count, last failure, next attempt time
Workflow timeout	Compliance review not accepted within 15 minutes	Business accountability	TimedOut state, escalation owner, reason, evidence link

The timeout state is part of the model

A mature distributed workflow should not have only happy-path states. It needs waiting states, retry states, timed-out states, escalated states, and reviewed states. Those states are not accidental implementation details. They are what let an operator, developer, auditor, or tester ask where the flow is and why it has not moved.

A useful NEXUS-1 process manager might track states such as the following:

State	Meaning	Allowed next states
VerdictIssued	RootCause has issued the public event RootCauseVerdictIssued.v1.	ComplianceReviewRequested, RecommendationRequested
ComplianceReviewRequested	A request has been sent and the process manager expects acknowledgement.	ComplianceReviewAccepted, ComplianceReviewTimedOut
ComplianceReviewTimedOut	The expected acknowledgement did not arrive within the configured window.	ComplianceReviewEscalated, RequestRetried
ComplianceReviewEscalated	Ownership has moved to an escalation queue or responsible role.	HumanReviewAssigned, FlowClosedWithException
HumanReviewAssigned	A person or role is responsible for resolving the blocked step.	ReviewCompleted, ReviewTimedOut, CompensationRequested

NEXUS-1 SENTENCE TO REMEMBER

A timeout is a state transition, not a sleep duration. Once the allowed waiting time expires, the workflow should move to a named state with an owner, a timestamp, and an audit trail.

This is the difference between an operationally useful system and a distributed application that only developers can interpret. A log line saying timeout while calling Compliance does not tell the operator whether compliance review is still expected, has been retried, has been escalated, or has been abandoned. A workflow state does.

Escalation is a change of ownership

Escalation is often described emotionally, as if it means something has gone badly wrong. In architecture it should be more precise. Escalation means that the current owner is no longer the right owner for the next decision. The workflow must transfer responsibility to another role, queue, service, or human review path.

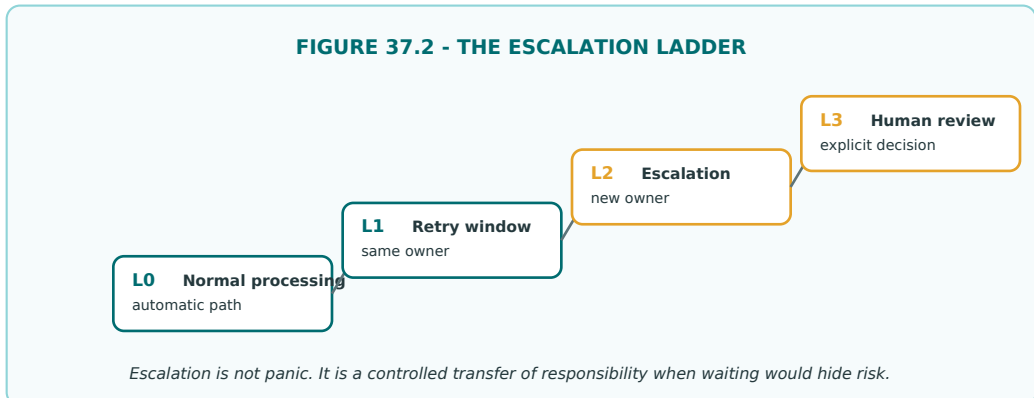


Figure 37.2 - Escalation climbs from ordinary processing to explicit human responsibility. Each step changes ownership, not merely severity.

In NEXUS-1, not every delay should wake a human. A transient message broker failure can be retried. A duplicate event can be ignored by an inbox. A reporting projection can lag and catch up. But a verdict that cannot reach compliance, a correction that changes a public conclusion, or a recommendation that affects operator attention should not remain invisible indefinitely.

Escalation should answer four questions:

1. What expected thing did not happen?
2. Who owns the blocked step now?
3. What can the new owner decide?
4. How will the decision be recorded?

If the escalation path cannot answer those questions, it is not a workflow design. It is only a notification mechanism.

Human review is not a failure of automation

A common microservice mistake is to treat human review as an embarrassment, something to be hidden after automation has failed. In serious systems, human review is part of the model. Some decisions should be delayed until a person has inspected the evidence, accepted the trade-off, or declared that the software should not proceed automatically.

The earlier NEXUS-1 books used the phrase advisory carefully. A recommendation is not a command. A root-cause verdict is evidence-backed but not plant authority. AuditEntry is evidence, not workflow. Those distinctions naturally lead to human review points. The system can calculate, detect, recommend, and notify. But it must also know when to stop and ask for a responsible decision.

Situation	Automatic action?	Human review?	Reason
Reporting projection is late	Yes - retry projection	Usually no	Read model lag is expected under eventual consistency.
Compliance review not acknowledged	Limited retry only	Yes after timeout	Compliance is a responsibility boundary, not just a technical consumer.
Root-cause verdict corrected	Publish correction event	Often yes	A public conclusion changed; downstream interpretation may need review.
Recommendation generated with low confidence	No command	Yes or abstain	Advice that is uncertain should be surfaced as uncertainty, not hidden as action.

HONEST DESIGN POINT

Human review is not a place to dump unclear automation. It needs its own inputs, screen state, decision vocabulary, permissions, deadlines, and audit evidence. Otherwise it becomes a manual black box attached to an automated black box.

A C# shape for workflow deadlines

The code shape does not need to be complicated. A process manager can hold expected replies as deadline records. Each deadline names the workflow, the expected event, the due time, the current owner, and the action to take when time expires.

```
public sealed class WorkflowDeadline { public Guid DeadlineId { get;
init; } public Guid WorkflowId { get; init; } public string
ExpectedEventName { get; init; } = null!; public DateTime DueAtUtc { get;
init; } public string OwnerRole { get; init; } = null!; public string
EscalationPolicy { get; init; } = null!; public bool IsSatisfied { get;
private set; } public void MarkSatisfied(DateTime observedAtUtc) { if
(observedAtUtc > DueAtUtc) throw new InvalidOperationException( "A late
reply must be recorded as late, not silently accepted."); IsSatisfied =
true; } }
```

The important part is not the exact class name. The important part is that the deadline becomes a first-class record. It can be queried, tested, retried, escalated, and audited. A timer hidden in memory cannot provide that discipline.

For a database-backed process manager, this usually means a table such as WorkflowDeadline or ProcessDeadline. It should be owned by the workflow service or process-manager service, not by every participating service. Services publish facts; the workflow owner tracks expected future facts.

A minimal table shape

Column	Purpose
DeadlineId	Stable identity of the deadline record.
WorkflowId	The process instance waiting for a reply or decision.
ExpectedEventName	For example ComplianceReviewAccepted.v1 or RecommendationAvailable.v1.
DueAtUtc	The time after which silence becomes a timeout.
OwnerRole	The role responsible when the deadline expires.
Status	Pending, Satisfied, TimedOut, Escalated, Cancelled.

Designing the NEXUS-1 escalation policy

A policy for the alarm-to-compliance flow should not use one timeout for everything. Different steps have different meanings. A reporting projection can wait longer than a compliance acknowledgement. A message publish retry can happen quickly. A human review may have a longer deadline but must be visible immediately after assignment.

Workflow step	Timeout meaning	First action	Escalation path
Request compliance review	Compliance did not acknowledge the request.	Retry with idempotency key.	Assign compliance coordinator.
Generate recommendation	Recommendation service did not produce advice or abstention.	Retry or mark recommendation unavailable.	Operator sees pending or unavailable state.
Write audit evidence	Audit evidence was not persisted or confirmed.	Stop closure of workflow.	Escalate to audit owner.
Human review assigned	No decision before due time.	Notify assigned role.	Escalate to responsible supervisor.

The operator console must show waiting honestly

Timeouts and escalations are not useful if the UI still shows a single vague status such as processing. A serious console needs a small status vocabulary that separates waiting, retrying, timed out, escalated, under review, completed, and completed with exception. That vocabulary should come from the workflow model, not from frontend guesswork.

OPERATOR CONSOLE RULE

Do not hide distributed uncertainty behind a spinner. A spinner says the user should wait. A workflow state says the system knows what it is waiting for, who owns the wait, and when it will escalate.

This rule connects directly to Part IV. The API gateway or BFF should compose state from service read models, but it should not invent the workflow state. It should display the state owned by the process manager.

What should never be escalated automatically

Not every failure should climb to a person. If every late message becomes an alert, the escalation system becomes an alarm flood. The same discipline that From Flood to Cause applied to alarms applies here: more notifications do not mean more understanding.

Automatic escalation is appropriate when waiting hides a responsibility gap. It is not appropriate when the system is handling an expected transient condition inside its designed retry envelope.

Do not escalate automatically when...	Better behaviour
A duplicate message is detected by the inbox.	Acknowledge and ignore it idempotently. Duplicate delivery is normal distributed behaviour.
A projection is a few seconds behind.	Show eventual consistency honestly and allow catch-up.
A retryable broker outage is still inside the retry policy.	Keep retrying and expose degraded status to operations, not to every business reviewer.
A low-risk noncritical export failed.	Record failure and allow manual re-run; do not create urgent workflow escalation.

What must always be visible

The opposite rule is just as important. Some timeout conditions must be visible even if no human decision is required yet. A service can be allowed to retry quietly only while the workflow remains inside an acceptable uncertainty window. After that window, the workflow must show the state.

VISIBLE STATES

Pending is acceptable only when it has a deadline. Retrying is acceptable only when the next retry is known. Escalated is acceptable only when the new owner is known. Human review is acceptable only when the decision options are explicit.

Testing timeout behaviour

Timeouts are easy to describe and easy to forget to test. A process manager should be tested with a controllable clock, not with real waiting. The test should prove that the deadline is created, that a timely event satisfies it, that a late event does not silently repair it, and that the escalation path is invoked when time expires.

```
[Fact] public void Compliance_review_timeout_escalates_the_flow() { var
clock = new FakeClock("2026-07-01T10:00:00Z"); var flow =
AlarmToComplianceFlow.Start(caseId, clock.UtcNow);
flow.RequestComplianceReview(timeoutAfter: TimeSpan.FromMinutes(15));
clock.Advance(TimeSpan.FromMinutes(16));
flow.CheckDeadlines(clock.UtcNow);
Assert.Equal(FlowState.ComplianceReviewEscalated, flow.State);
Assert.Equal("ComplianceCoordinator", flow.CurrentOwnerRole);
Assert.Contains(flow.Events, e => e.Name ==
"ComplianceReviewTimedOut.v1"); }
```

Notice the event name in the test. `ComplianceReviewTimedOut.v1` is not the same as `ComplianceReviewFailed.v1`. Failure means something happened and failed. Timeout means something expected did not happen in time. That distinction is not pedantry; it is how the workflow remains explainable.

The same test style should exist for human review. The process should not close automatically when a reviewer is late. It should move to `ReviewTimedOut` or `ReviewEscalated` and leave a record of who was assigned, what deadline was missed, and what escalation policy applied.

Chapter checkpoint

CHECKPOINT

1. Why is a workflow timeout different from an HTTP timeout?
2. Who should own the timeout for a compliance review request?
3. Why should a timeout be represented as a workflow state?
4. What changes during escalation: severity, ownership, or both?
5. Why is human review a first-class model concern rather than a failure of automation?
6. What should the operator console show instead of an indefinite spinner?

HONEST BOUNDARY

This chapter describes timeouts, escalation, and human review as software architecture patterns for the educational NEXUS-1 demonstrator. It does not define operational response times, emergency procedures, regulatory review requirements, or certified human-factors design for any real facility. The architectural lesson is general: distributed workflows must make waiting, lateness, ownership transfer, and human decision points explicit.

Chapter 37 turns silence into a visible part of the workflow. The next chapter continues the same discipline by asking where distributed workflows hide risk even when every individual service appears to be working correctly.

PART VI - DISTRIBUTED WORKFLOWS

38. What Must Never Be Hidden in a Workflow Engine

A workflow engine can be useful infrastructure. It can persist waiting, schedule timers, retry activities, and resume after failure. But it must never become the secret place where the domain decision, the public contract, the compensation rule, or the human-review authority disappears.

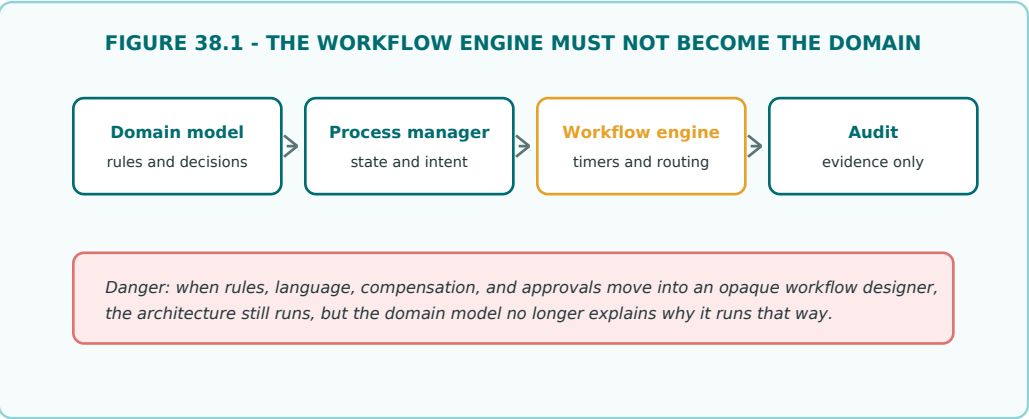


Figure 38.1 - The workflow engine may execute the flow, but the domain model and process manager must still explain the meaning of the flow.

What you already know

You have probably seen a tool that promises to make workflows visible. A diagram is drawn, boxes are connected, and every activity appears on one screen. The first impression is reassuring: finally, the distributed flow can be seen. The danger is that visibility of boxes is not the same as visibility of meaning.

If the diagram says Review Required, Approve, Retry, Compensate, and Close, it may still hide the real questions. Who decided that review is required? Which service owns the rule? What contract caused the transition? What data was used? What happens if the approval is late? Which action is a business decision and which is only routing? A workflow engine can draw arrows while concealing the answers.

PLAIN EXPLANATION

A workflow engine is allowed to help the system remember where a process is and what technical step should run next. It is not allowed to replace the domain model as the place where business meaning, rules, ownership, and decisions live.

This chapter is deliberately placed after timeouts and escalation. Chapter 37 made waiting visible. This chapter prevents the next mistake: making the workflow engine responsible for everything simply because it can store state and move tokens between steps.

The engine is not the architect

A workflow engine is often sold as an architectural center. That is the wrong mental model. The engine is infrastructure. It can run activities, store bookmarks, schedule timers, and resume after a crash. It can coordinate execution. It should not decide what the bounded contexts mean.

In NEXUS-1, the root-cause verdict belongs to RootCause. Compliance review belongs to Compliance. Audit evidence belongs to Audit. Reporting projections belong to Reporting. The process manager may coordinate the story that crosses them, but even the process manager should express its state and rules in code and language that the team owns. The engine should execute that model, not replace it.

The five things that must not be hidden

A workflow engine becomes dangerous when it hides one of five responsibilities: decision, language, contract, compensation, or authority. Each one must remain explicit enough to review, test, version, and explain.

Do not hide...	Why it is dangerous	Keep it explicit as...
Domain decisions	The real rule disappears into a diagram or script nobody tests like domain code.	Aggregate method, domain service, or application policy.
Language	Generic step names replace the ubiquitous language of the bounded context.	Named states and events: VerdictIssued, ReviewRequested, ReviewEscalated.
Public contracts	A diagram transition becomes a private contract between services.	Versioned integration events and commands such as RootCauseVerdictIssued.v1.
Compensation policy	Rollback-like behaviour can undo the wrong thing or hide correction history.	Forward correction events and explicit compensation commands.
Human authority	A person appears as an activity box but no ownership, deadline, or decision options are modeled.	Review assignment, due time, reviewer role, allowed outcomes, audit evidence.

The recurring NEXUS-1 trap

The tempting design is to put the whole alarm-to-compliance flow into a workflow designer: when RootCauseVerdictIssued.v1 arrives, call Compliance, call Recommendation, call Audit, call Reporting, wait for all of them, and close the process. The diagram looks clean. The architecture is not.

The problem is not that a workflow engine runs activities. The problem is that the diagram starts to look like the source of truth. A future developer may change the approval branch in the engine without touching the Compliance model. Another may rename a step from VerdictIssued to CaseClosed and accidentally break the language discipline established in From Context to Flow. The engine becomes a second domain model, but without the protections of the domain model.

Engine responsibility vs domain responsibility

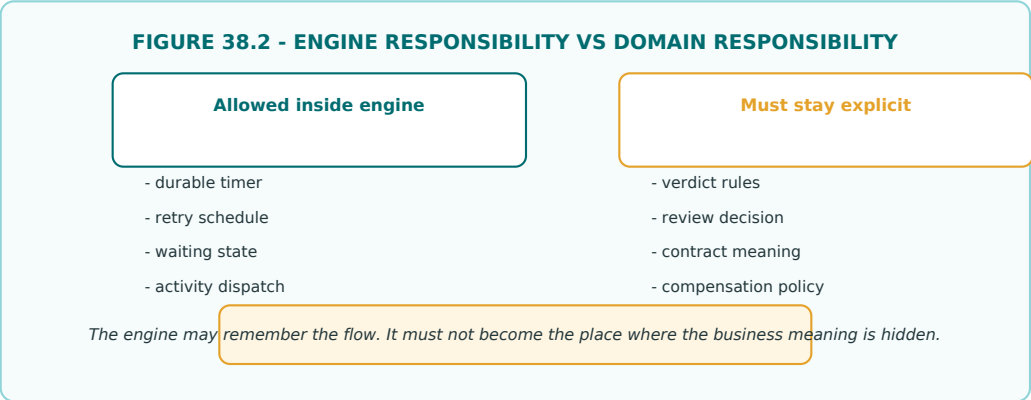


Figure 38.2 - A useful workflow engine holds execution mechanics. The domain and process model hold meaning.

The safest rule is simple: the engine may know that a named activity should run; it should not contain the reason that activity is valid. That reason belongs in code the domain team can test and review.

WORKFLOW ENGINE RULE

The workflow engine may persist a process instance, schedule a timer, dispatch an activity, and resume after failure. It must not be the only place where the business rule, public event meaning, compensation rule, or human decision authority is defined.

This rule protects the same distinction the book has protected everywhere else. A recommendation is not a command. AuditEntry is evidence, not workflow. A read model is not source of truth. A workflow engine is execution support, not domain truth.

What the engine may own

The engine can still own useful technical responsibilities. It can be the durable place where the process is waiting. It can run scheduled checks. It can record technical activity status. It can dispatch work to services. It can expose operational visibility. These are valuable responsibilities because distributed workflows need memory.

Engine responsibility	Allowed meaning
Durable timer	Wake the process when a deadline arrives.
Activity dispatch	Call or publish the next technical step after the process model requests it.
Retry bookkeeping	Track attempts, next attempt time, and last technical error.
Process persistence	Remember waiting state and correlation identifiers after a restart.
Operational visibility	Show which activities are waiting, failed, timed out, or completed.

What the engine must call into

A mature design uses the engine as an execution host around explicit application code. The process manager decides what command should be issued next. The engine stores that the command is scheduled or waiting. The service that receives the command makes its own local decision.

For example, the alarm-to-compliance flow may have a method such as `OnRootCauseVerdictIssued`. That method records the process state and requests compliance review. A workflow engine can host the call and persist the wait. But the method name, input contract, resulting state, and emitted command remain part of the application model.

```
public sealed class AlarmToComplianceProcess
{
    public Result Handle(RootCauseVerdictIssuedV1 e, IWorkflowCommands commands)
    {
        if (State != FlowState.WaitingForVerdict)
            return Result.Success(); // idempotent duplicate

        State = FlowState.ComplianceReviewRequested;
        CorrelationId = e.CorrelationId;

        commands.Send(new RequestComplianceReview(
            e.RootCauseCaseId,
            e.VerdictId,
            e.IssuedAtUtc,
            e.CorrelationId));

        ScheduleDeadline("ComplianceReviewAcknowledged", e.IssuedAtUtc.AddMinutes(15));
        return Result.Success();
    }
}
```

The workflow engine may store and resume this process. It may not replace this process with an unreviewed diagram where the same meaning is hidden in boxes and arrows.

Why diagrams are not enough

A diagram can be excellent documentation. It can also become dangerous camouflage. A box named `ComplianceReview` can mean request sent, request accepted, review opened, review completed, or finding issued. Without explicit states and contracts, the diagram is readable only to the person who drew it.

DIAGRAM RULE

A workflow diagram is acceptable only when every meaningful transition has a named event, command, state, owner, and failure path outside the diagram. Otherwise the diagram is not architecture; it is hidden code with nicer shapes.

NEXUS-1 workflow-engine boundary map

The following map gives a practical boundary for the NEXUS-1 demonstrator. It does not forbid workflow infrastructure. It says which parts of the alarm-to-compliance process the infrastructure may help execute, and which parts must stay in explicit service or domain code.

Flow concern	May live in workflow engine	Must remain outside engine
Verdict event received	Correlation, activity start, durable bookmark.	Meaning of RootCauseVerdictIssued.v1 and RootCause verdict rules.
Compliance review requested	Dispatch request and wait for acknowledgement.	Compliance policy deciding whether and how review opens.
Recommendation unavailable	Timeout tracking and retry status.	Rule that recommendation is advisory and not a command.
Audit evidence written	Activity completion tracking.	Audit meaning, immutability, correction policy, evidence identity.
Human review late	Timer and escalation activity.	Review authority, due rules, allowed outcomes, accountability.

The table shows the mature compromise. The workflow engine can hold process mechanics. The services and process model hold meaning. This is the difference between using infrastructure and surrendering architecture to infrastructure.

Testing against hidden workflow logic

A team can detect hidden workflow logic by asking a blunt question: can the important rule be tested without starting the workflow engine? If the answer is no, the rule probably lives in the wrong place.

Test question	Healthy answer	Smell
Can the process transition be unit-tested?	Yes, with a fake clock and fake command sink.	Only by running the engine designer runtime.
Can the public contract be versioned independently?	Yes, as RootCauseVerdictIssued.v1 and typed DTOs.	Contract fields are hidden in activity input mappings.
Can compensation be reviewed in code?	Yes, as explicit forward commands/events.	Rollback branch lives only in the diagram.
Can human review authority be audited?	Yes, reviewer role, due time, options, and outcome are modeled.	Approval is just a user task with no domain meaning.

NEXUS-1 SENTENCE TO REMEMBER

Use a workflow engine to remember and resume the process. Do not use it to hide why the process is allowed to move.

This sentence also protects future migration. If NEXUS-1 later changes workflow infrastructure, the domain process should survive. The engine can be replaced only if the meaning is not trapped inside it.

Chapter checkpoint

CHECKPOINT

1. Why is a workflow engine infrastructure rather than the domain model?
2. Name five things that must never be hidden inside an engine diagram.
3. What may a workflow engine legitimately own?
4. Why is a diagram not enough to prove architecture?
5. How can tests reveal that business logic has been hidden in the engine?
6. In NEXUS-1, why must `RootCauseVerdictIssued.v1` remain a public contract rather than a private transition label?

HONEST BOUNDARY

This chapter does not argue against workflow engines, orchestration tools, durable task frameworks, or BPMN-style diagrams. They can be valuable when used as execution infrastructure. The boundary is architectural: in the NEXUS-1 demonstrator, domain rules, public contracts, compensation meaning, audit evidence, and human authority must remain visible outside the engine. The chapter is about software architecture discipline, not about certifying or rejecting any specific product.

Chapter 38 keeps distributed workflow infrastructure honest. The next chapter will turn the Part VI lessons into a concrete NEXUS-1 distributed process-manager example.

PART VI - DISTRIBUTED WORKFLOWS

39. NEXUS-1 Distributed Process Manager Example

The previous chapters described the rules. This chapter puts them together as a concrete implementation shape: a small, explicit process manager that listens to public events, records durable state, sends commands through an outbox, handles duplicate messages, waits with deadlines, and escalates without pretending there is a distributed transaction.

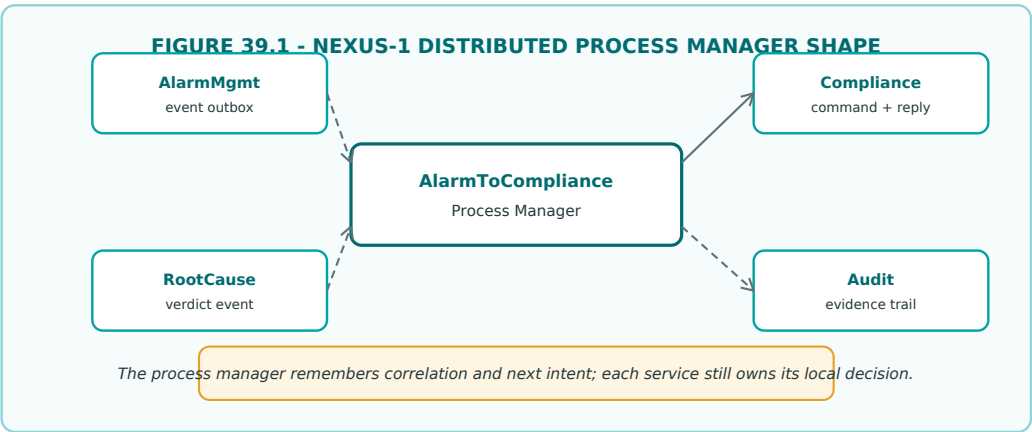


Figure 39.1 - The process manager coordinates across services while local ownership remains inside each service.

What this example is and is not

This chapter is an implementation sketch, not a full production framework. Its purpose is to show the shape a C#/.NET developer should be able to recognize: messages in, process state loaded, decision made, new state saved, outgoing command written to an outbox, and no direct cross-service transaction attempted.

The example follows the recurring NEXUS-1 alarm-to-compliance flow. AlarmManagement detects an alarm flood. RootCause opens and later resolves a root-cause case. The public contract RootCauseVerdictIssued.v1 crosses the boundary. A process manager requests compliance review. Compliance may acknowledge, delay, reject, or time out. Audit records evidence, but audit does not drive the workflow.

DESIGN BOUNDARY

The process manager coordinates the flow; it does not become RootCause, Compliance, Audit, or Reporting. It remembers correlation, stage, deadlines, retries, and outgoing intent. The owning services still make the local decisions.

The message set

A process manager should speak through published language. In this example, it consumes events that are already stable at service boundaries and produces commands directed to owning services. It does not consume private domain events from inside aggregates, and it does not issue commands that pretend to be decisions.

Message	Kind	Owner	Process-manager use
AlarmFloodDetected.v1	Integration event	AlarmManagement	Start or correlate the distributed process.
OpenRootCauseCase	Command	RootCause	Ask RootCause to open a case; RootCause validates.
RootCauseVerdictIssued.v1	Integration event	RootCause	Record verdict seen and request compliance review.
RequestComplianceReview	Command	Compliance	Ask Compliance to open a review.
ComplianceReviewOpened.v1	Integration event	Compliance	Complete or advance the process stage.
ComplianceReviewDeadlineReached	Timer message	Process manager	Turn absence into timeout state and escalation intent.

Process state

Process state is operational memory. It is not an audit trail and not a read model. It exists so the system can answer: which cross-service process is this, which stage is it in, which event IDs has it already handled, what command is pending, what deadline exists, and what failure or escalation is visible?

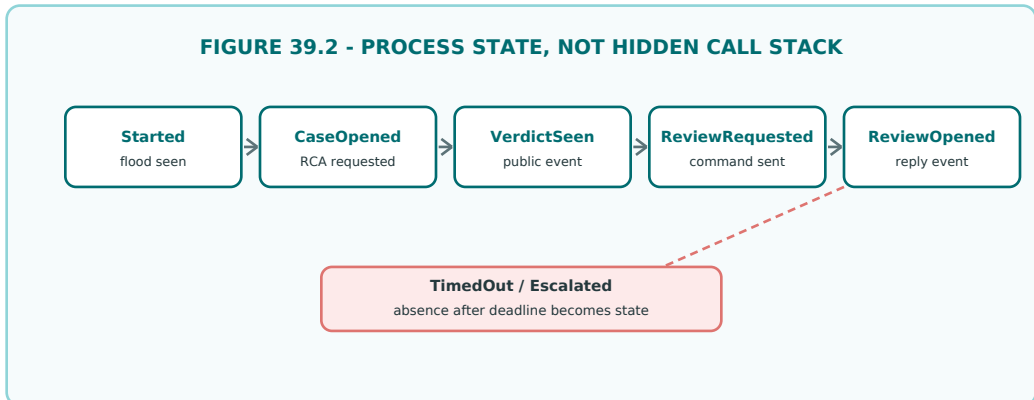


Figure 39.2 - The process manager records named states rather than hiding waiting inside a call stack.

```

public enum AlarmToComplianceStage
{
    Started = 1,
    RootCauseCaseRequested = 2,
    RootCauseCaseOpened = 3,
    VerdictSeen = 4,
    ComplianceReviewRequested = 5,
    ComplianceReviewOpened = 6,
    TimedOut = 7,
    Escalated = 8,
    Completed = 9
}

public sealed class AlarmToComplianceProcess
{
    public Guid ProcessId { get; private set; }
    public string CorrelationId { get; private set; } = null!;
    public AlarmToComplianceStage Stage { get; private set; }
    public string? RootCauseCaseId { get; private set; }
    public string? VerdictId { get; private set; }
    public DateTime? ComplianceReviewDeadlineUtc { get; private set; }
    public string? LastHandledEventId { get; private set; }
    public int RetryCount { get; private set; }
}
  
```

This object is intentionally boring. That is a virtue. It is durable enough to survive restarts, explicit enough to be queried by support, and small enough not to become the domain model of every participant.

The store and the command outbox

A process manager needs a state store and an outgoing-message store. The store may be a table owned by the coordination service. The outgoing command should be persisted durably before any broker publication is attempted. This repeats the outbox discipline from Part V at the process-manager boundary.

Table or component	Purpose	Important columns or methods
ProcessState	Durable state of the long-running flow.	ProcessId, CorrelationId, Stage, Deadlines, RetryCount, RowVersion.
ProcessedMessage	Idempotency memory for consumed events.	MessageId, HandlerName, ProcessedAtUtc, Result.
CommandOutbox	Durable outgoing command intent.	MessageId, CommandType, Payload, Destination, OccurredAtUtc, Attempts.
TimerRequest	Durable wake-up request for deadlines.	DueAtUtc, ProcessId, TimerName, CorrelationId.

```
public interface IProcessStore
{
    Task<AlarmToComplianceProcess?> FindByCorrelationId(
        string correlationId, CancellationToken ct);

    Task Save(AlarmToComplianceProcess process, CancellationToken ct);
}

public interface ICommandOutbox
{
    Task Enqueue<TCommand>(
        TCommand command,
        string destination,
        string correlationId,
        CancellationToken ct);
}

public interface ITimerScheduler
{
    Task Schedule(
        string timerName,
        string correlationId,
        DateTime dueAtUtc,
        CancellationToken ct);
}
```

Handler 1: alarm flood detected

The first handler starts the process. It does not open a root-cause case directly in RootCause storage. It records process memory and enqueues a command. RootCause remains free to accept, reject, validate, or dead-letter the command according to its own model.

```
public async Task Handle(AlarmFloodDetectedV1 e, CancellationToken ct)
{
    if (await inbox.AlreadyProcessed(e.EventId, ct))
        return;

    var process = await store.FindByCorrelationId(e.CorrelationId, ct)
        ?? AlarmToComplianceProcess.Start(e.CorrelationId, e.UnitId, e.EventId);

    if (process.ShouldRequestRootCauseCase())
    {
        process.MarkRootCauseCaseRequested(e.EventId, clock.UtcNow);

        await commandOutbox.Enqueue(
            new OpenRootCauseCase(
                e.CorrelationId,
                e.UnitId,
                e.FloodId,
                e.DetectedAtUtc,
                destination: "RootCause",
                correlationId: e.CorrelationId,
                ct: ct);
        );

        await store.Save(process, ct);
        await inbox.MarkProcessed(e.EventId, ct);
    }
}
```

Two boundaries are visible. First, duplicate events are absorbed before a second root-cause case is requested. Second, the command is a request, not a forced decision. RootCause owns whether a case can be opened.

OWNERSHIP RULE

The process manager may ask. The target service decides. If the target service rejects the command, the process manager records that fact and follows a recovery path. It must not bypass the owner.

Handler 2: verdict issued

The verdict event is the central hand-off. Inside `RootCause`, the domain event may be `VerdictIssued`. Outside `RootCause`, the process manager sees the public integration event `RootCauseVerdictIssued.v1`. That distinction keeps `RootCause` free to evolve internally while preserving the stable external contract.

```
public async Task Handle(RootCauseVerdictIssuedV1 e, CancellationToken ct)
{
    if (await inbox.AlreadyProcessed(e.EventId, ct))
        return;

    var process = await store.FindByCorrelationId(e.CorrelationId, ct)
        ?? AlarmToComplianceProcess.RehydrateFromVerdict(e.CorrelationId);

    process.RecordVerdictSeen(
        eventId: e.EventId,
        rootCauseCaseId: e.RootCauseCaseId,
        verdictId: e.VerdictId,
        occurredAtUtc: e.IssuedAtUtc);

    if (process.ShouldRequestComplianceReview())
    {
        await commandOutbox.Enqueue(
            new RequestComplianceReview(
                e.CorrelationId,
                e.RootCauseCaseId,
                e.VerdictId,
                e.IssuedAtUtc),
            destination: "Compliance",
            correlationId: e.CorrelationId,
            ct: ct);

        process.MarkComplianceReviewRequested(
            deadlineUtc: e.IssuedAtUtc.AddMinutes(15));

        await timers.Schedule(
            "ComplianceReviewAcknowledgementDeadline",
            e.CorrelationId,
            e.IssuedAtUtc.AddMinutes(15),
            ct);
    }

    await store.Save(process, ct);
    await inbox.MarkProcessed(e.EventId, ct);
}
```

The deadline is part of the state

The handler does not sleep. It records a deadline and schedules a timer message. That makes absence visible. If Compliance does not answer before the deadline, a later handler will receive a timer message and decide whether to retry, escalate, or mark the process as timed out.

TIME RULE

Waiting is never an invisible thread. Waiting is a named state with a deadline, a correlation ID, and an expected fact. When the expected fact does not arrive, the absence becomes a process event.

Handler 3: compliance review opened

The happy path remains simple. Compliance opens the review and publishes a public event. The process manager consumes the event, marks the stage, and may emit follow-up intents for audit or reporting. It does not treat the reporting projection as source of truth.

```
public async Task Handle(ComplianceReviewOpenedV1 e, CancellationToken ct)
{
    if (await inbox.AlreadyProcessed(e.EventId, ct))
        return;

    var process = await store.FindByCorrelationId(e.CorrelationId, ct);
    if (process is null)
        return; // or dead-letter if your policy treats this as impossible

    process.MarkComplianceReviewOpened(
        eventId: e.EventId,
        complianceReviewId: e.ComplianceReviewId,
        openedAtUtc: e.OpenedAtUtc);

    await commandOutbox.Enqueue(
        new AppendAuditEvidence(
            e.CorrelationId,
            subject: "ComplianceReview:" + e.ComplianceReviewId,
            action: "ComplianceReviewOpened"),
        destination: "Audit",
        correlationId: e.CorrelationId,
        ct: ct);

    await store.Save(process, ct);
    await inbox.MarkProcessed(e.EventId, ct);
}
```

Handler 4: deadline reached

A deadline handler should not guess that Compliance failed. It should compare state against time. If the expected reply has already arrived, the timer is stale and can be ignored. If the process is still waiting, the handler records timeout and chooses a visible recovery path.

```
public async Task Handle(ComplianceReviewDeadlineReached e, CancellationToken ct)
{
    var process = await store.FindByCorrelationId(e.CorrelationId, ct);
    if (process is null)
        return;

    if (process.Stage == AlarmToComplianceStage.ComplianceReviewOpened)
        return; // stale timer; the reply already arrived

    if (!process.IsWaitingForComplianceReview(e.DueAtUtc))
        return;

    process.MarkComplianceReviewTimedOut(
        dueAtUtc: e.DueAtUtc,
        observedAtUtc: clock.UtcNow);

    if (process.RetryCount < 3)
    {
        process.ScheduleRetry(clock.UtcNow.AddMinutes(5));
        await commandOutbox.Enqueue(
            RequestComplianceReview.FromProcess(process),
            destination: "Compliance",
            correlationId: process.CorrelationId,
            ct: ct);
    }
    else
    {
        process.MarkEscalated("Compliance review not opened after retries");
        await commandOutbox.Enqueue(
            new RequestHumanReview(process.CorrelationId, process.RootCauseCaseId!),
            destination: "OperatorWorkflow",
            correlationId: process.CorrelationId,
            ct: ct);
    }

    await store.Save(process, ct);
}
```

The important design detail is not the number three. The important detail is that retry and escalation are states, not hidden loops. A user can query the process and see whether the flow is waiting, retrying, timed out, or escalated.

Process idempotency rules

Inbox idempotency prevents a message from being processed twice. Process idempotency prevents a process from doing the wrong thing even when messages arrive in surprising order. A mature process manager uses both.

Condition	Process response	Reason
Duplicate AlarmFloodDetected.v1	Ignore if EventId already processed or process already started.	Do not open a second root-cause case.
Verdict arrives before local case-open reply	Rehydrate or advance by CorrelationId if contract is valid.	Distributed order is not guaranteed.
Compliance reply arrives after timeout	Accept late success if escalation is not final, or record semantic correction.	Late truth is still truth; do not erase history.
Deadline fires after success	Ignore stale timer.	Timers are messages too; they can arrive late.
Unknown verdict code	Dead-letter or escalate with visible reason.	Never map unknown domain language silently.

NEXUS-1 SENTENCE TO REMEMBER

The process manager is not reliable because messages arrive perfectly. It is reliable because imperfect arrival is expected, named, stored, and handled.

Where this would live in a .NET solution

The implementation can live in a dedicated coordination service or in a module inside the modular core while the system is still being extracted. The important point is not the deployment unit at first. The important point is that the code already obeys service-boundary rules: public contracts in, commands out, local state, outbox, inbox, correlation, and no shared database shortcut.

```
Nexus.Services.ProcessManager/  
  Contracts/  
    Consumed/  
      AlarmFloodDetectedV1.cs  
      RootCauseVerdictIssuedV1.cs  
      ComplianceReviewOpenedV1.cs  
    Produced/  
      OpenRootCauseCase.cs  
      RequestComplianceReview.cs  
      RequestHumanReview.cs  
  Application/  
    AlarmToComplianceProcessManager.cs  
  Handlers/  
    AlarmFloodDetectedHandler.cs  
    RootCauseVerdictIssuedHandler.cs  
    ComplianceReviewOpenedHandler.cs  
    DeadlineReachedHandler.cs  
  Domain/  
    AlarmToComplianceProcess.cs  
    AlarmToComplianceStage.cs  
  Infrastructure/  
    ProcessStore.cs  
    InboxStore.cs  
    CommandOutboxStore.cs  
    TimerScheduler.cs
```

This structure deliberately mirrors the book thesis. The process manager has its own domain-like process state, but it does not import `RootCauseCase` or `ComplianceReview` as entities. It speaks through contracts. That keeps the coordination model from becoming a distributed monolith with friendlier folder names.

The Part VI closure

Part VI began by asking who remembers a workflow after a service boundary is crossed. The answer is now concrete. A process manager remembers the cross-boundary process, a saga defines the long-running consistency story, compensation replaces impossible rollback, timeouts make absence observable, and workflow engines remain infrastructure. This chapter combines those ideas into one NEXUS-1 shape.

Chapter checkpoint

CHECKPOINT

1. Which public events does the NEXUS-1 process manager consume?
2. Which commands does it produce, and who owns their validation?
3. Why does the process manager need its own state table instead of relying on Audit?
4. Why must a deadline be stored as process state rather than implemented with Task.Delay?
5. What are the two layers of idempotency used in the example?
6. Why does this process manager avoid becoming a distributed monolith?

HONEST BOUNDARY

This chapter provides a deliberately compact implementation shape. It does not choose RabbitMQ, Azure Service Bus, Kafka, MassTransit, Dapr, Temporal, Durable Functions, or any other runtime. Those choices belong to later operational chapters. The architectural point is independent of the tool: process state, public contracts, outbox, inbox, deadlines, and escalation must be explicit before the transport or workflow product is allowed to matter.

Part VI is now complete. The next part turns from workflow coordination to consistency, reporting, read models, and the user experience of delayed truth.

PART VII

Consistency, Reporting, and Read Models

Part VII begins after the workflow is already distributed. The next difficulty is not how to send a message or how to coordinate a saga. The difficulty is truth: which service owns it, which projection displays it, why the operator console may lag, and how the system explains that lag without lying.

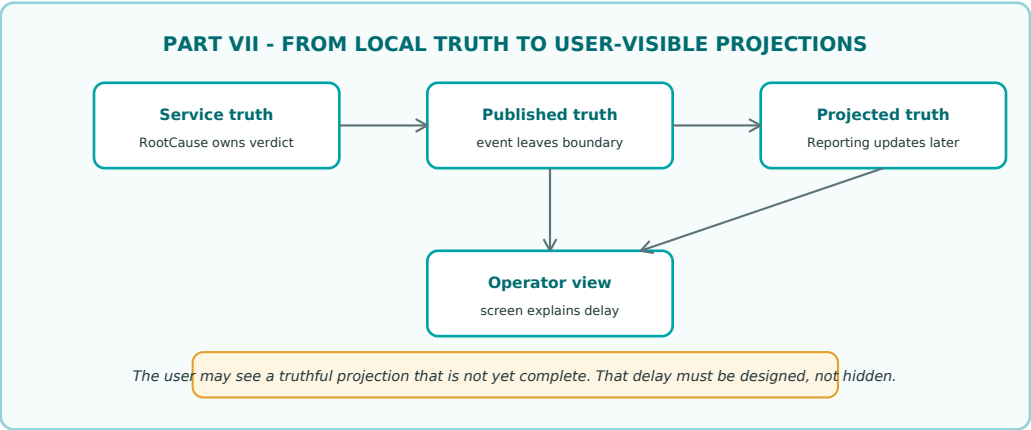


Figure VII.1 - Part VII studies the movement from local service truth to projected user-visible truth.

WHAT PART VII GIVES YOU

By the end of this part, the reader should be able to separate source-of-truth data from reporting projections, design read models that admit delay, build compliance views without taking ownership away from services, preserve audit across boundaries, and explain delayed truth to users.

PART VII - CONSISTENCY, REPORTING, AND READ MODELS

40. The End of Single-Database Thinking

The modular monolith gave the system a comforting illusion: if the rows exist in one database, a join can reveal the truth now. A microservice architecture removes that illusion. The truth still exists, but it no longer sits in one place, arrives at one time, or appears through one query.

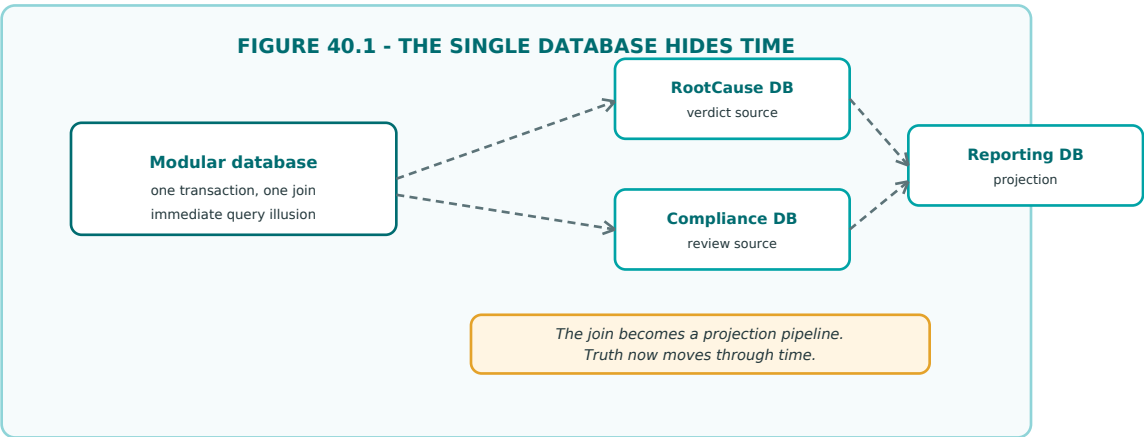


Figure 40.1 - A single database hides time; a service architecture makes time visible.

What single-database thinking gave us

A single database is not automatically bad. For the earlier NEXUS-1 books, it was exactly the right teaching tool. It made the data backbone visible. ReactorFleet, Instrumentation, AlarmManagement, RootCause, Compliance, Reporting, and Audit could be joined, traced, and checked in one place. That was useful because the point was to show the model, the keys, and the ownership map before distribution made everything harder.

But a single database also trains the mind to expect immediate answers. If a RootCauseCase has a verdict and a ComplianceReview row has been created, a query can show both at once. If the Reporting schema has a view, it can read across boundaries and return a complete incident summary. The developer begins to confuse query convenience with architectural truth.

PLAIN EXPLANATION
Single-database thinking says: if the data is related, I should be able to join it now.
Service thinking says: if the data belongs to different owners, I may need to receive, store, project, and explain it over time.

The naive move would be to keep the join by letting Reporting, Compliance, or the operator console read everyone else's tables. That preserves the feeling of immediate truth, but it destroys service ownership. The system becomes distributed in deployment while remaining centralized in data dependency. That is the distributed monolith in its most polite form.

Why the old mental model fails

When services own their data, no service has the right to treat another service's database as a shared library. RootCause owns the verdict. Compliance owns the review. Audit owns evidence records. Reporting owns projections that are useful for reading, not facts that override their sources. The operator console composes views from contracts and read models, not by joining private tables.

Single-database assumption	Distributed correction	NEXUS-1 consequence
A join can answer everything now.	A projection answers what has arrived so far.	IncidentSummary may briefly lack Compliance status.
Foreign keys enforce cross-context truth.	Public identifiers and events connect services.	RootCauseCaseId travels in RootCauseVerdictIssued.v1.
Reporting can read all source tables.	Reporting consumes events and builds read models.	Reporting is query-optimized but not authoritative.
A transaction makes all changes visible together.	Messages make changes visible over time.	Compliance review may appear after the verdict event.
The UI can assume completeness.	The UI must show freshness and pending status.	Operator console shows "verdict issued; review pending".

NEXUS-1 SENTENCE TO REMEMBER

A read model is allowed to be useful before it is complete. It is not allowed to pretend that incomplete means final.

Three kinds of truth

The first repair is vocabulary. A distributed system must distinguish local truth, published truth, and projected truth. They are not competitors. They are different stages of the same fact as it moves through service boundaries.

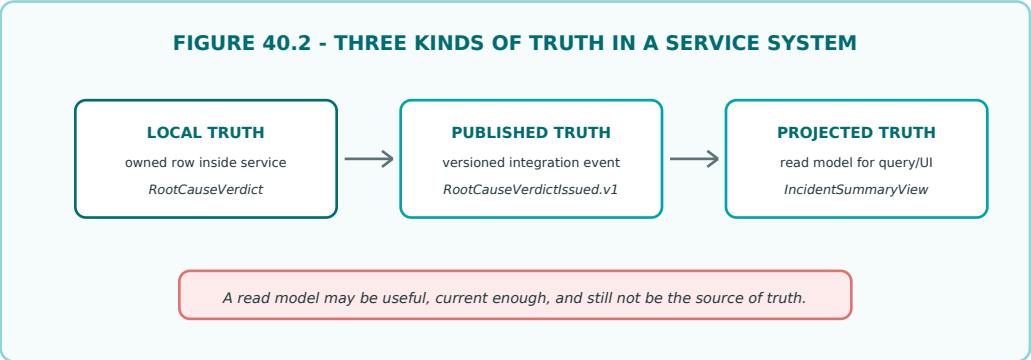


Figure 40.2 - Local truth, published truth, and projected truth serve different purposes.

Local truth is the state owned by a service. RootCause owns the fact that a verdict was issued. Published truth is the versioned event that RootCause is willing to share outside its boundary: RootCauseVerdictIssued.v1. Projected truth is the read-side shape built by Reporting or by the operator-console BFF so a user can query a useful view without calling every service for every screen.

TERMINOLOGY DISCIPLINE

Do not call a projection "the truth" without qualification. Say source of truth when you mean the owning service, public fact when you mean the integration event, and read model when you mean a query-optimized projection.

Applying this to the alarm-to-compliance flow

In the recurring NEXUS-1 flow, the end of single-database thinking appears the moment a verdict is issued. In the modular database, the reporting view could join RootCause, Compliance, Audit, and Reporting tables. In a service architecture, the Reporting service must learn about those facts through events or through carefully designed read APIs. That means it can be behind.

Step	Owning truth	Public fact or projection	Possible delay
Verdict issued	RootCause service	RootCauseVerdictIssued.v1	Event publication may be retried through outbox.

Compliance review requested	Process manager / Compliance command boundary	RequestComplianceReview	Command may wait in outbox or broker.
Review opened	Compliance service	ComplianceReviewOpened.v1	Projection may not yet include review ID.
Evidence recorded	Audit service	Audit evidence query or event	Audit trail may be complete before reporting summary.
Dashboard updated	Reporting/BFF projection	IncidentSummaryView	View may show pending or stale marker.

This table is the end of the old mental model. There is no one query that is allowed to reach into every owner and pretend to be the system. There are owners, public facts, projections, and delays. Good architecture does not eliminate all delays. It makes the important delays visible and bounded.

The reporting projection as a contract with time

A reporting projection is not only a data shape. It is a time contract. It should tell the reader what it contains, what source facts contributed to it, when it was last updated, and whether it is known to be waiting for more facts.

```
public sealed record IncidentSummaryProjection(  
    string CorrelationId,  
    string UnitCode,  
    string AlarmFloodId,  
    string? RootCauseCaseId,  
    string? VerdictCode,  
    string ComplianceStatus,  
    DateTime LastUpdatedAtUtc,  
    bool IsComplete,  
    string[] WaitingFor);  
  
public sealed record ProjectionFreshness(  
    string ProjectionName,  
    string CorrelationId,  
    DateTime LastEventAtUtc,  
    DateTime LastProjectedAtUtc,  
    string[] MissingFacts);
```

The fields `IsComplete` and `WaitingFor` are not UI decoration. They are architecture made visible. They prevent the operator console from presenting a half-built read model as if it were a final incident record.

DESIGN RULE

Every read model that joins facts from multiple services should have a freshness story. At minimum, it should carry last update time, source event position or timestamp, and a way to show pending facts.

A small projection handler

The code below is deliberately small. Its purpose is not to implement a complete reporting service. Its purpose is to show that a projection is built from public facts, not from private table joins. The handler consumes `RootCauseVerdictIssuedV1` and updates a local read model owned by `Reporting`.

```
public async Task Handle(RootCauseVerdictIssuedV1 e, CancellationToken ct)
{
    if (await inbox.AlreadyProcessed(e.EventId, ct))
        return;

    var summary = await projections.FindByCorrelationId(
        e.CorrelationId, ct)
        ?? IncidentSummary.CreatePending(e.CorrelationId, e.UnitId);

    summary.ApplyRootCauseVerdict(
        rootCauseCaseId: e.RootCauseCaseId,
        verdictId: e.VerdictId,
        verdictCode: e.VerdictCode,
        issuedAtUtc: e.IssuedAtUtc,
        sourceEventId: e.EventId);

    summary.MarkWaitingFor("ComplianceReviewOpened.v1");
    summary.SetLastProjectedAt(clock.UtcNow);

    await projections.Save(summary, ct);
    await inbox.MarkProcessed(e.EventId, ct);
}
```

The handler does not ask RootCause for its database. It does not join Compliance. It does not call Audit to fill every gap. It records the fact it has received and marks the missing fact by name. That is the discipline that replaces the old cross-schema join.

Freshness beats false completeness

In a control-room-like console, a stale or partial display is dangerous when it looks complete. The safest user experience is not always the one that updates every panel instantly. The safest one is the one that says what it knows, when it learned it, and what it is still waiting for.

Screen state	Meaning	Acceptable display language
Fresh and complete	All expected facts have arrived for the current workflow stage.	Verdict issued; compliance review opened; audit evidence recorded.
Fresh but incomplete	Latest event was applied, but expected downstream fact has not arrived.	Verdict issued; compliance review pending.
Stale projection	No update has been received beyond the freshness window.	Projection may be delayed; open service status for details.
Contradictory facts	Projection received facts that do not agree by correlation or version.	Incident summary requires review; do not infer final status.

OPERATOR-CONSOLE RULE

A delayed read model is not a lie if it is labelled as delayed. It becomes a lie when the UI removes the delay marker and presents the projection as final truth.

What this changes in architecture reviews

The old review question was often: can this screen query the data it needs? The new question is more precise: which service owns each fact, which public contract carries it, which projection stores it for the screen, how fresh must that projection be, and what does the user see while it is incomplete?

Review question	Why it matters
What is the source of truth?	Prevents Reporting or UI composition from becoming hidden owners.
Which event or API carries the fact?	Prevents private database access from returning through the back door.
What is the acceptable delay?	Turns eventual consistency from a vague excuse into a design constraint.
How is freshness shown?	Protects users from treating partial projections as final records.
What happens when facts contradict?	Defines escalation instead of silent overwrite.

This is not less rigorous than a single database. It is more explicit. The old rigor lived in constraints and joins. The new rigor lives in ownership, contracts, projection rules, freshness markers, and traceability.

Chapter checkpoint

CHECKPOINT

1. Why does a single database encourage immediate-truth thinking?
2. What is the difference between local truth, published truth, and projected truth?
3. Why must Reporting not read private RootCause or Compliance tables?
4. Why should a projection carry freshness or completeness metadata?
5. How should the operator console display a verdict when compliance review has not yet arrived?
6. What question replaces "can I join these tables" in a microservice architecture review?

HONEST BOUNDARY

This chapter does not yet design the full operator-console consistency experience. It establishes the mental shift required before that design is safe: cross-service read models are projections with time, not joins with authority. The next chapter applies that shift directly to the NEXUS-1 operator console.

The next chapter continues from this point: if the read model is allowed to lag, the UI must teach the user how to read lag without panic, confusion, or false certainty.

PART VII - CONSISTENCY, REPORTING, AND READ MODELS

41. Eventual Consistency in the Operator Console

The operator console is where distributed truth becomes human experience. Services may be correct, messages may be delivered, projections may be rebuilding, and yet the screen can still mislead if it presents delayed truth as final truth.

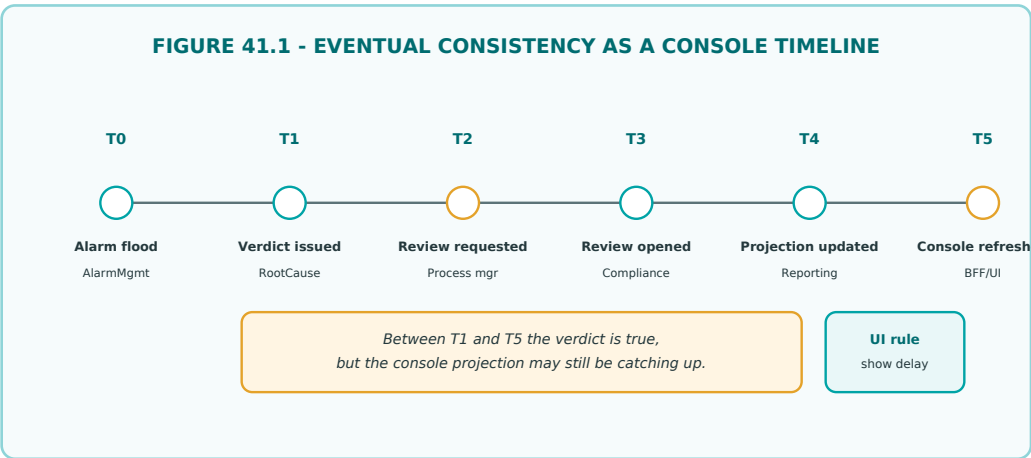


Figure 41.1 - Eventual consistency is not a vague backend excuse; it is a visible timeline the operator console must make readable.

The problem is not delay; the problem is invisible delay

Eventual consistency is often explained from the backend: a service commits its local transaction, publishes an event, another service consumes it, and a read model catches up. That explanation is technically correct and operationally incomplete. The reader of the screen does not experience an outbox, broker, inbox, or projection handler. The reader experiences a panel that either looks current or does not.

In NEXUS-1, the most important example remains the alarm-to-compliance flow. RootCause may issue a verdict before Compliance has opened its review. Audit may have stored the evidence before Reporting has rebuilt the summary. The operator console may therefore be showing a true verdict, a pending review, and a summary timestamp that is a few seconds behind. That is acceptable only if the screen says so.

PLAIN EXPLANATION

Eventual consistency means the system can be correct even when every screen has not caught up yet. The console must show that the view is catching up, instead of pretending everything changed at the same instant.

The naive user-interface answer is a spinner. A spinner is not enough. It says that something is happening, but it does not say what is already true, what is still pending, and whether the view is safe to interpret. A serious console needs more than loading; it needs consistency language.

What the console must distinguish

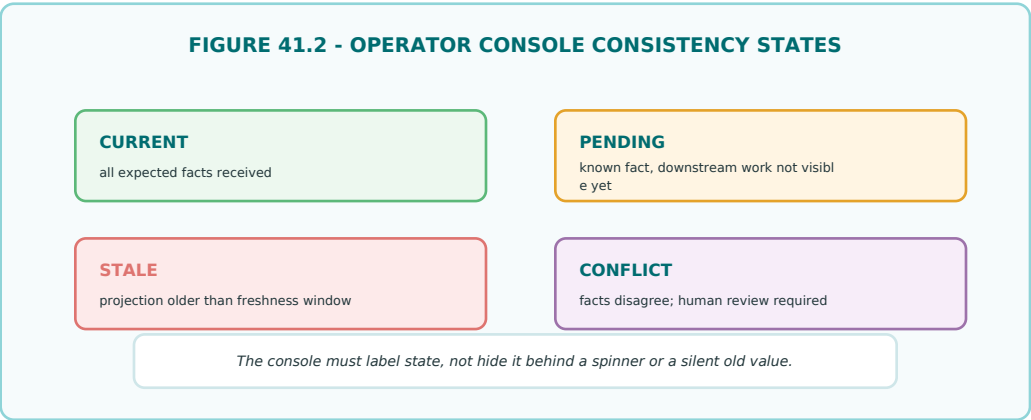


Figure 41.2 - The operator console should make consistency states explicit instead of hiding them behind old values or generic loading indicators.

Console state	Meaning	NEXUS-1 example	User action
Current	The projection has received all expected facts for this workflow stage.	Verdict, review, audit marker, and summary all agree.	Read normally.
Pending	A known upstream fact exists, but a downstream fact has not arrived yet.	RootCause verdict issued; Compliance review pending.	Do not infer compliance completion.
Stale	The projection has not advanced within its freshness window.	Incident summary older than the latest event timestamp.	Open freshness details or service health.
Conflict	Facts disagree or cannot be correlated safely.	Verdict correction event conflicts with displayed summary version.	Escalate for review; do not auto-resolve silently.

The operator should see fact, freshness, and gap

A console card that only displays the latest value is too thin for a distributed architecture. The card should show the fact itself, when that fact was last projected, and which related facts are still missing. That does not make the UI more complicated for its own sake. It makes the hidden distributed state readable.

```
public sealed record OperatorIncidentCard(
    string CorrelationId,
    string UnitCode,
    string AlarmFloodId,
    string? RootCauseVerdict,
    string ComplianceState,
    string AuditEvidenceState,
    ConsoleConsistencyState ConsistencyState,
    DateTime LastProjectedAtUtc,
    DateTime? LastSourceEventAtUtc,
    string[] WaitingFor,
    string[] Warnings);

public enum ConsoleConsistencyState
{
    Current,
    Pending,
    Stale,
    Conflict
}
```

This record is not a domain object. It is a user-facing composition. Its job is not to decide root cause, open compliance review, or write audit evidence. Its job is to show the current projection honestly.

NEXUS-1 SENTENCE TO REMEMBER
The console may compose a view. It may not compose authority. Authority remains with the service that owns the decision.

The BFF as consistency translator

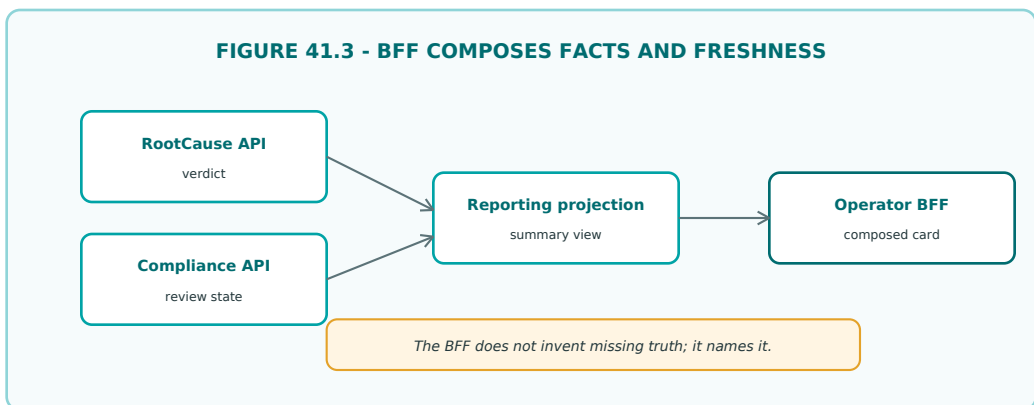


Figure 41.3 - The operator BFF composes facts and freshness markers; it does not invent missing facts.

The backend-for-frontend is a natural place to translate service and projection state into a screen shape. But this does not give the BFF business authority. It does not decide that compliance is complete because the verdict exists. It does not decide that audit evidence is valid because Reporting has a row. It asks each source or projection for what it owns, combines the answers, and labels the gaps.

BFF RULE

A BFF may reduce network chatter and shape a view for the operator console. It must not hide missing downstream facts, suppress freshness metadata, or convert pending work into final status.

A small composition policy

The following policy is intentionally simple. It shows how a composition layer can make consistency visible without owning the business decision. The exact thresholds would be configuration in a real system; the principle is that the view is labelled from evidence, not from optimism.

```

public sealed class IncidentCardConsistencyPolicy
{
    public ConsoleConsistencyState Classify(
        IncidentSummaryProjection projection,
        TimeSpan freshnessWindow,
        DateTime nowUtc)
    {
        if (projection.HasCorrelationConflict)
            return ConsoleConsistencyState.Conflict;

        if (nowUtc - projection.LastProjectedAtUtc > freshnessWindow)
            return ConsoleConsistencyState.Stale;

        if (projection.WaitingFor.Length > 0)
            return ConsoleConsistencyState.Pending;

        return ConsoleConsistencyState.Current;
    }
}
  
```

The policy is deliberately boring. That is a virtue. It does not infer plant truth. It does not score root cause. It does not override the process manager. It gives the UI a safe consistency label from projection metadata.

Input evidence	Console label	Why this is safe
Missing downstream facts listed in WaitingFor.	Pending	The UI admits incomplete projection instead of filling the gap.
Projection older than configured freshness window.	Stale	The user sees time risk instead of trusting an old value.
Correlation or version disagreement.	Conflict	The system stops short of false reconciliation.
No missing facts, no staleness, no conflict.	Current	The view is current enough for the intended display purpose.

How the screen should speak

The wording matters. A technical system can be correct and still train users badly. If the console says nothing while a projection is incomplete, users learn to treat absence as meaning. If the console says "error" every time a projection waits for a downstream event, users learn to distrust normal distributed behaviour. The message must be precise.

Bad wording	Better wording	Reason
No compliance review exists.	Compliance review pending.	Absence in the projection is not proof of absence in the system.
Incident complete.	Incident summary current as of 14:03:22 UTC.	Completion and freshness are different claims.
Loading...	Verdict received; waiting for ComplianceReviewOpened.v1.	Names the missing fact rather than hiding it.
System inconsistent.	Projection conflict: verdict correction requires review.	Separates data conflict from total system failure.

USER-EXPERIENCE RULE

A distributed console should teach the user which truth is already known, which truth is still pending, and which truth requires review. It should not force the user to infer distributed state from silence.

Failure is not the same as delay

One of the most common mistakes in distributed UI design is treating every delayed projection as a failure. Normal delay, service degradation, and contradiction are different states. They deserve different handling.

Condition	What it means	Console response
Normal delay	The upstream event arrived; the downstream projection has not yet caught up.	Show pending state with missing event or expected service.
Service unavailable	The console cannot query a service or projection health endpoint.	Show degraded source and last known projection time.
Poisoned event	A message could not be applied and reached the dead-letter path.	Show delayed or requires-review only if the user should act.
Conflicting correction	A correction event changes an earlier displayed fact.	Show correction marker and audit link.

This is where the work of previous chapters returns. Outbox, inbox, idempotency, dead-letter queues, process managers, and compensation are not hidden backend machinery anymore. They shape what the console can honestly say.

HONEST BOUNDARY

This chapter designs the consistency language of the operator console, not a production human-factors certified interface. NEXUS-1 remains an educational demonstrator. The architectural lesson is transferable: do not let the screen erase time.

Chapter checkpoint

CHECKPOINT

1. Why is invisible delay more dangerous than delay itself?
2. What is the difference between Current, Pending, Stale, and Conflict in the console?
3. Why should the BFF not convert pending downstream work into final status?
4. Which fields help a read model explain its freshness?
5. Why is "Compliance review pending" safer than "No compliance review exists"?
6. How do outbox, inbox, and process-manager design become visible in the UI?

The next chapter moves from the console back into the data layer: how to build reporting projections that are fast to query, honest about their sources, and clearly separate from the services that own the original truth.

PART VII - CONSISTENCY, REPORTING, AND READ MODELS

42. Building Reporting Projections

A reporting projection is not a shortcut around service ownership. It is a deliberately built read-side model that accepts delay, records lineage, and answers questions without pretending to own the truth it displays.

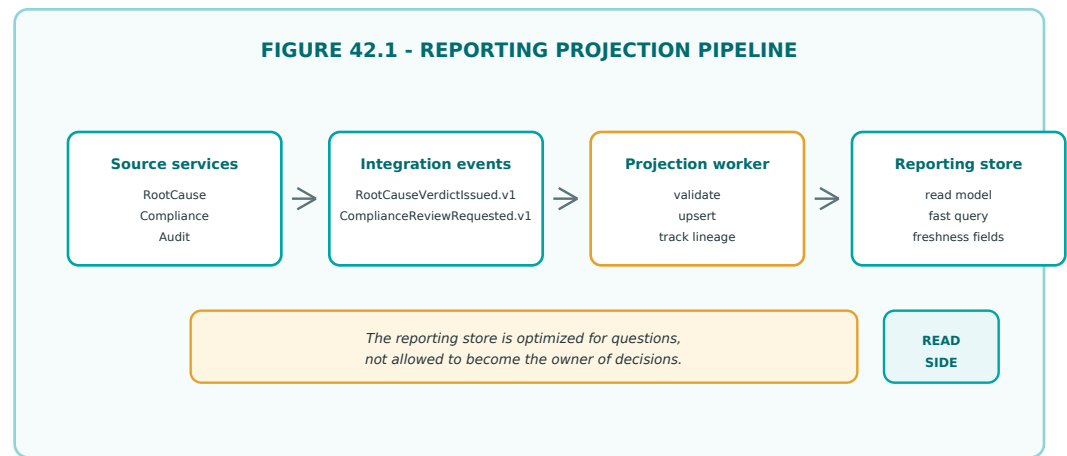


Figure 42.1 - Reporting projections sit on the read side: source services own decisions; the projection owns query shape and freshness metadata.

Why reporting projections exist

In a modular monolith, reporting often feels simple because one database can answer almost any question. A dashboard can join RootCause, Compliance, Audit, AlarmManagement, and Reporting tables in one query. That convenience disappears when those areas become independently owned services. The question does not disappear; the old query does.

A reporting projection is the disciplined replacement for the cross-service join. Instead of asking five services synchronously every time an operator opens a screen, the system listens to public events, updates a read model, and serves a query-optimized view from the reporting side. The projection is allowed to be fast. It is allowed to duplicate selected facts. It is not allowed to become the authority for those facts.

PLAIN EXPLANATION
A projection is a copy shaped for reading. It is not the original decision. It is useful because screens and reports ask different questions than command handlers and aggregates.

The first rule is therefore negative: do not build a projection because you are tired of respecting service boundaries. Build it because a user-facing question needs a stable, fast, explicit read model.

Projection ownership is not domain ownership

Question	Owned by source service	Owned by reporting projection
What was the root-cause verdict?	RootCause owns the verdict and correction rules.	Stores the displayed verdict text and version for reporting.
Is compliance review required?	Compliance or process manager owns the workflow rule.	Shows current known review state and last event applied.
What evidence exists?	RootCause and Audit own evidence and audit records.	Stores evidence counts or references, not editable evidence.
How fast can the screen load?	No source service owns UI query speed alone.	Reporting owns the query shape, indexes, and projection latency.

This table is the heart of the chapter. A reporting service may own a projection table, a report endpoint, a dashboard query, and an export job. It does not own the business truth produced elsewhere. That separation keeps the read side useful without letting it corrupt the write side.

NEXUS-1 SENTENCE TO REMEMBER
Reporting may say what it has seen. It may not decide what happened.

What a useful projection row must carry

FIGURE 42.2 - A PROJECTION ROW CARRIES DATA AND LINEAGE

A read model that cannot explain its sources is only a cache wearing a business name.

Figure 42.2 - A serious read model carries display facts and the metadata needed to judge them.

The weakest reporting models store only the convenient display columns. They look clean at first and become dangerous later, because no one can answer where a value came from, when it was last updated, which event version produced it, or whether it is still waiting for another service.

A NEXUS-1 incident summary projection should therefore carry four categories of data: display facts, status facts, freshness fields, and lineage fields. The first two serve the screen. The last two protect the reader from trusting the screen too much.

```
public sealed record IncidentSummaryProjection(
    string CorrelationId,
    string UnitCode,
    string AlarmFloodId,
    string? RootCauseVerdict,
    string ComplianceReviewState,
    int EvidenceCount,
    int AuditEntryCount,
    DateTime LastProjectedAtUtc,
    DateTime LastSourceEventAtUtc,
    string LastAppliedEventName,
    string LastAppliedEventVersion,
    string[] SourceServices,
    string[] WaitingFor);
```

This record is not the schema of a certified plant system. It is the shape of the architectural lesson: every displayed fact should be accompanied by enough context to explain its freshness and origin.

Building from events, not from private tables

A projection should not read the private database tables of other services. If it does, it recreates the shared-database problem under a more polite name. The correct input is a public contract: an integration event or a deliberately exposed read API. For the recurring flow, the most important event is still `RootCauseVerdictIssued.v1`.

PROJECTION INPUT RULE

The projection consumes published language, not private model structure. It listens to `RootCauseVerdictIssued.v1`, `ComplianceReviewRequested.v1`, `ComplianceReviewOpened.v1`, `AuditEntryRecorded.v1`, and related public events. It should not query `RootCauseCase` tables directly.

The following handler shape is deliberately small. It shows the projection worker as an idempotent updater. The event is not re-decided. It is applied.

```
public sealed class RootCauseVerdictProjectionHandler
{
    private readonly IIncidentSummaryProjectionStore _store;
    private readonly IClock _clock;

    public async Task Handle(
        RootCauseVerdictIssuedV1 message,
        CancellationToken cancellationToken)
    {
        await _store.UpsertAsync(
            message.CorrelationId,
            row => row with
            {
                RootCauseVerdict = message.Verdict,
                LastSourceEventAtUtc = message.IssuedAtUtc,
                LastProjectedAtUtc = _clock.UtcNow,
                LastAppliedEventName = "RootCauseVerdictIssued",
                LastAppliedEventVersion = "v1",
                SourceServices = row.SourceServices.AddDistinct("RootCause"),
                WaitingFor = row.WaitingFor.Remove("RootCauseVerdictIssued.v1")
            },
            cancellationToken);
    }
}
```

The handler is boring for the same reason the inbox handler was boring: boring is reviewable. A projection worker should be easy to test with the same message delivered twice, delivered late, or delivered after a correction event.

Upsert, do not append blindly

The projection is usually not an event log. The event log remains elsewhere. The projection is a query shape. That means the worker often updates one row per incident, one row per case, one row per dashboard card, or one row per reportable aggregate. Appending every event into the reporting table may be useful for trace tables, but it is not the dashboard projection itself.

Pattern	Use when	Risk if misused
Upsert summary row	The screen asks for current state by incident or case.	Must keep lineage fields or the row becomes unexplained.

Append reporting fact	The report needs trends, counts, or time-series facts.	Can become an unreadable duplicate event store.
Materialized join view	The UI needs a fast composition of several known facts.	Can hide which source is stale or missing.
Export snapshot	A report must preserve what the user saw at generation time.	Can be mistaken for live truth if not timestamped.

ANTI-PATTERN

Do not make Reporting the place where business rules are secretly re-implemented because source services are inconvenient to call. Reporting is a read side, not a second domain model with weaker discipline.

Rebuildability is the test

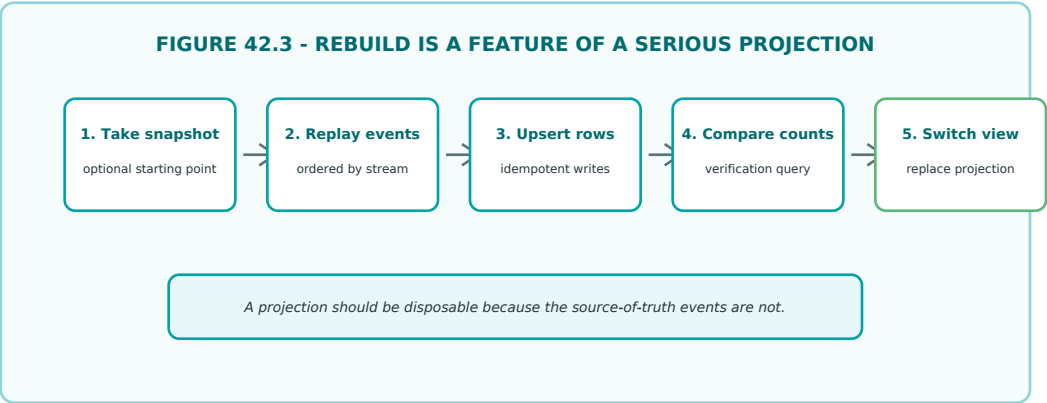


Figure 42.3 - A projection is healthy when it can be rebuilt from source events or approved snapshots.

A projection will eventually need to be rebuilt. Maybe a display column is added. Maybe a bug in the projection handler is fixed. Maybe a new report groups the same facts differently. If rebuilding is impossible, the projection has become an accidental source of truth.

For NEXUS-1, rebuildability means at least three things. First, the source events remain available or are summarized in approved snapshots. Second, the projection worker is deterministic enough that replaying the same inputs produces the same read model. Third, verification queries can compare event counts, case counts, and last-applied positions before the rebuilt projection is trusted.

REBUILD RULE

A reporting projection may be disposable. The events, source records, and audit evidence it derives from are not disposable.

Freshness, lag, and last-applied position

Freshness is not only a timestamp. A useful projection should tell readers when it was last updated, which source event it last applied, and whether it is caught up enough for the screen or report being shown.

Field	Meaning	Why it matters
LastProjectedAtUtc	When the projection row was written.	Shows the age of the read model itself.
LastSourceEventAtUtc	When the latest applied source fact occurred.	Separates event age from projection write time.
LastAppliedEventName	The public event most recently applied.	Explains what changed the row last.
LastAppliedSequence	Stream position or monotonically increasing marker.	Allows rebuild and catch-up verification.
WaitingFor	Expected facts not yet seen.	Lets the UI say pending instead of silent.

These fields look operational, but they are part of the user experience too. Chapter 41 argued that the console must not hide delayed truth. Chapter 42 gives the reporting projection the fields needed to keep that promise.

A NEXUS-1 reporting projection example

The recurring alarm-to-compliance flow needs a read model that can answer a very practical question: what is the current status of the incident, from the operator's point of view? That answer crosses RootCause, Compliance, Audit, and Reporting. In a service architecture it should not be a four-database join. It should be a reporting projection built from public facts.

```
CREATE TABLE Reporting.IncidentSummaryProjection (
  CorrelationId NVARCHAR(80) NOT NULL,
  UnitCode NVARCHAR(40) NOT NULL,
  AlarmFloodId NVARCHAR(80) NOT NULL,
  RootCauseVerdict NVARCHAR(200) NULL,
  ComplianceReviewState NVARCHAR(60) NOT NULL,
  EvidenceCount INT NOT NULL,
  AuditEntryCount INT NOT NULL,
  LastProjectedAtUtc DATETIME2(7) NOT NULL,
  LastSourceEventAtUtc DATETIME2(7) NOT NULL,
  LastAppliedEventName NVARCHAR(120) NOT NULL,
  LastAppliedEventVersion NVARCHAR(20) NOT NULL,
  WaitingForJson NVARCHAR(MAX) NOT NULL,
  RowVersion ROWVERSION NOT NULL,
  CONSTRAINT PK_Reporting_IncidentSummaryProjection
    PRIMARY KEY (CorrelationId)
);
```

This table is intentionally a projection table. It is not normalized like a write-side model. It repeats values that make the query fast and understandable. That repetition is acceptable because the row advertises its source, version, and freshness.

HONEST BOUNDARY

This projection is a software-architecture example for the NEXUS-1 demonstrator. It is not a certified reporting schema, not a plant record system, and not a replacement for regulated audit evidence. It teaches how read models can remain useful without pretending to be source-of-truth tables.

Query design belongs to the projection

Once a projection exists, its schema should be shaped by the questions it must answer. This is not a violation of domain modelling; it is the point of a read model. The write side protects invariants. The read side answers questions.

Report question	Projection design response
Show all open incidents for Unit 1.	Index UnitCode plus ComplianceReviewState and LastProjectedAtUtc.
Show incidents waiting for compliance.	Store ComplianceReviewState as a queryable field, not only inside JSON.
Show stale incident summaries.	Index LastProjectedAtUtc or expose lag through a calculated view.
Export monthly verdict summaries.	Maintain append-only reporting facts or snapshot exports separately from the current summary row.

Notice the discipline. Query design belongs to Reporting, but business decision design does not. Reporting can choose indexes. Reporting cannot decide that a

verdict is valid.

Chapter checkpoint

CHECKPOINT

1. Why is a projection the replacement for a cross-service join?
2. What is the difference between owning a projection and owning domain truth?
3. Why should Reporting consume public events instead of private service tables?
4. Which fields make a projection honest about freshness and lineage?
5. Why is rebuildability an architectural test?
6. What can Reporting optimize without becoming the owner of the business decision?

The next chapter applies this discipline to compliance views: how a compliance screen can gather facts from several services without turning regulatory visibility into a hidden workflow engine.

PART VII - CONSISTENCY, REPORTING, AND READ MODELS

43. Compliance Views Across Services

A compliance view is not a new authority. It is a disciplined window over facts owned by several services: the verdict that RootCause issued, the evidence Audit recorded, the review state Compliance owns, and the freshness limits Reporting must make visible.

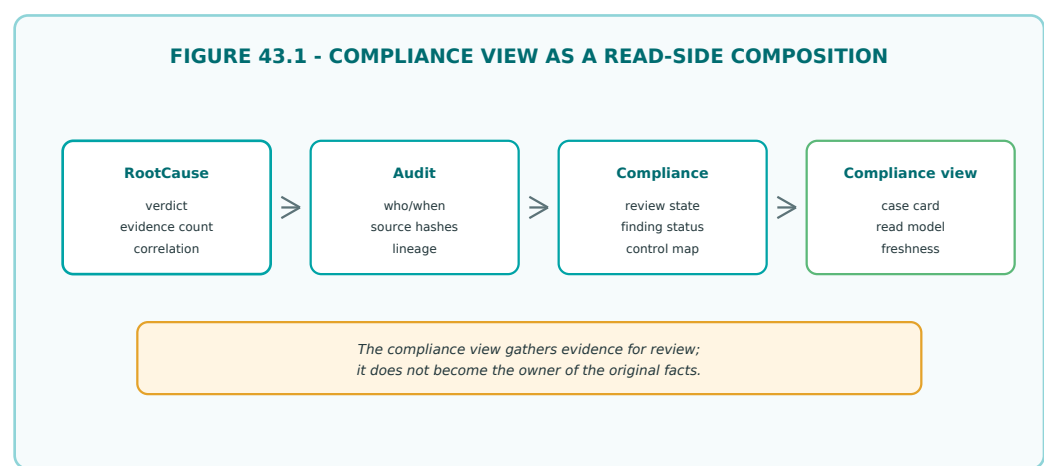


Figure 43.1 - A compliance view composes public facts into a reviewable screen without becoming the owner of those facts.

Why compliance becomes difficult after service extraction

In a single database, a compliance screen can appear deceptively simple. It can join the root-cause case, alarm flood, audit entry, finding, control, and report tables in one query. The screen feels complete because the database boundary hides the ownership boundary.

After service extraction, that convenience is gone. RootCause owns verdicts and evidence rules. Audit owns evidence records and immutable history. Compliance owns review workflow, controls, findings, and regulatory mapping. Reporting owns read models and exports. A compliance view must respect all four truths at once.

PLAIN EXPLANATION
A compliance view is a screen or read model that helps a reviewer see what is known. It is not the place where the original verdict, the audit evidence, or the compliance workflow is secretly recreated.

The architectural problem is therefore not how to display more columns. The problem is how to display cross-service evidence without smuggling cross-service ownership back into one service.

The three-layer responsibility split

Layer	Owns	Must not do
Source services	RootCause verdicts, Audit evidence, Compliance reviews, AlarmManagement floods.	Shape their private models around one UI screen.
Projection / read model	The composed compliance view, freshness fields, lineage, and query shape.	Decide whether a verdict is correct or a review is required.
Operator / reviewer UI	Presentation, pending states, warnings, and links back to source records.	Pretend delayed or missing facts are already complete.

This split is the same discipline used throughout the book. The service that owns a fact owns the rule that creates or changes it. The read model owns the shape of a question. The UI owns how delay is explained to the human being looking at the screen.

NEXUS-1 SENTENCE TO REMEMBER
Compliance may review a root-cause verdict. It does not become the service that issued the verdict.

What a compliance view needs to show

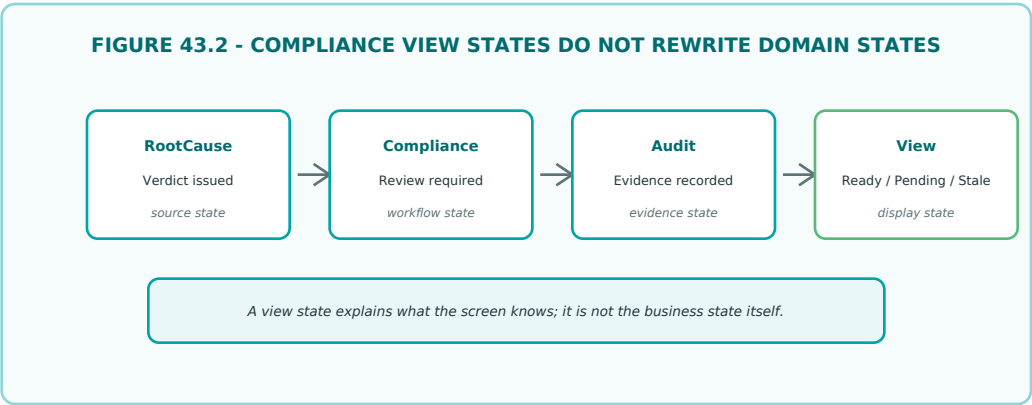


Figure 43.2 - The view has its own display state, but that state must not overwrite the source states it summarizes.

A useful compliance view should not try to show the entire private model of every service. It should show the minimum set of facts needed for review, with enough lineage to let the reviewer inspect the source when necessary.

View element	Source of truth	Displayed as
Root-cause verdict	RootCause service via RootCauseVerdictIssued.v1.	Verdict text, issued time, version, correction marker.
Evidence summary	RootCause evidence and Audit records.	Counts, references, hash markers, source links.
Review state	Compliance service.	Not required, required, open, escalated, closed.
Regulatory mapping	Compliance control catalogue.	Mapped controls and reviewer notes.
Freshness / waiting state	Reporting projection metadata.	Fresh, pending, stale, waiting for source.

The view is intentionally narrow. It is not a data warehouse, not an audit ledger, and not a second root-cause engine. It is a reviewer-facing composition of selected public facts.

Consuming public facts only

The most important implementation rule is simple: the compliance view should consume public contracts, not private tables. If it reads `RootCause.RootCauseCase` directly, it breaks the database-per-service rule. If it reads `Audit.AuditEntry` directly without an approved contract or export, it turns `Audit` into a shared persistence layer. If it calls several services live on every screen load, it risks building a slow synchronous trap.

VIEW INPUT RULE
A compliance view should be built from integration events, approved reporting feeds, or explicit read APIs. It should not join private service tables across runtime boundaries.

A small event-applied handler makes the rule visible. The handler does not decide compliance. It updates what the compliance view can currently display.

```
public sealed class ComplianceViewProjectionHandler
{
    private readonly IComplianceViewStore _store;
    private readonly IClock _clock;

    public async Task Handle(
        RootCauseVerdictIssuedV1 message,
        CancellationToken cancellationToken)
    {
        await _store.UpsertAsync(
            message.CorrelationId,
            row => row with
            {
                RootCauseVerdict = message.Verdict,
                RootCauseVerdictVersion = "v1",
                RootCauseIssuedAtUtc = message.IssuedAtUtc,
                LastSourceEventName = "RootCauseVerdictIssued.v1",
                LastSourceEventAtUtc = message.IssuedAtUtc,
                LastProjectedAtUtc = _clock.UtcNow,
                WaitingFor = row.WaitingFor.Remove("RootCauseVerdictIssued.v1")
            },
            cancellationToken);
    }
}
```

The shape should look familiar by now. The projection is an idempotent reaction to a public fact. It records what it saw, when it saw it, and which part of the view is now satisfied.

Compliance review is workflow, not just display

A compliance view is read-side composition. A compliance review is a workflow. Mixing the two creates a quiet architectural error: the screen starts to decide what the service should decide.

Concern	Belongs in compliance workflow	Belongs in compliance view
Whether review is required	Yes. This is a compliance rule or process-manager decision.	No. The view displays the result.
Which control applies	Yes. Compliance owns the mapping rule.	May display mapped controls.

Whether evidence is present	May participate in workflow rules.	Shows counts and links with source references.
Whether the view is stale	Usually no.	Yes. Reporting exposes freshness and lag.

ANTI-PATTERN

Do not let a compliance screen contain hidden if-statements that decide review obligations because the proper Compliance service has not yet caught up. The screen may say pending. It may not impersonate the workflow.

Freshness is part of compliance meaning

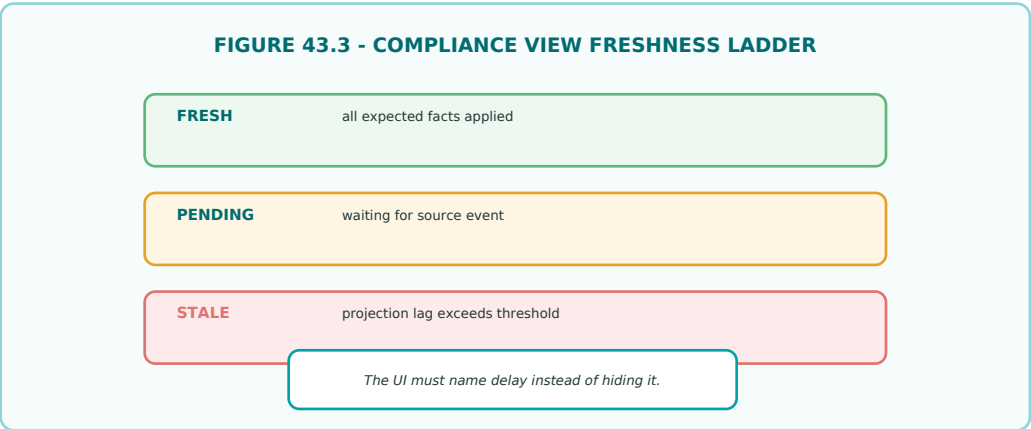


Figure 43.3 - Compliance views should make freshness visible because review decisions depend on knowing whether the view is complete.

For ordinary dashboards, stale data is often only an inconvenience. For a compliance view, stale data changes the meaning of what the reviewer sees. A case that appears to have no audit entries may simply be waiting for AuditEntryRecorded.v1. A review that appears closed may be waiting for a correction event. A verdict that appears final may have been corrected by RootCauseVerdictCorrected.v1 but not yet projected.

The view must therefore carry explicit freshness fields and display them. Not every user needs to see every internal detail, but the system must be able to explain delay when asked.

FRESHNESS RULE

A compliance view should never silently flatten fresh, pending, and stale facts into the same visual state.

A NEXUS-1 compliance view record

The following record shows an implementation shape. It is deliberately designed as a view model, not as a domain aggregate. It contains display facts, source references, freshness metadata, and waiting markers.

```
public sealed record ComplianceCaseView(
    string CorrelationId,
    string UnitCode,
    string RootCauseCaseId,
    string? RootCauseVerdict,
    string? RootCauseVerdictVersion,
    DateTime? RootCauseIssuedAtUtc,
    string ComplianceReviewState,
    string[] MappedControls,
    int EvidenceCount,
    int AuditEntryCount,
    string[] SourceServices,
    string[] SourceEventVersions,
    string[] WaitingFor,
    DateTime LastProjectedAtUtc,
    DateTime LastSourceEventAtUtc,
    bool IsStale);
```

This object is useful precisely because it is not pure. It repeats display facts. It carries arrays of source services and event versions. It contains freshness and waiting fields. A write-side aggregate should not look like this. A serious read-side compliance view often should.

MODEL BOUNDARY

The ComplianceCaseView is not the ComplianceReview aggregate. One is a display composition; the other protects workflow rules. The names should stay different because their responsibilities are different.

Open links back to source records

A compliance view becomes trustworthy when it can point back to its sources. The view itself may show a verdict, a count, and a state. But the reviewer should be able to open the root-cause case, audit evidence, and compliance review record from the same screen.

Displayed item	Source link target	Reason
Verdict	RootCause case or verdict history.	Reviewer can inspect the reasoning chain.
Evidence count	Evidence list or audit evidence index.	Reviewer can see what the count represents.
Audit marker	Immutable audit entry or audit batch.	Reviewer can replay who wrote what, and when.
Control mapping	Compliance control catalogue.	Reviewer can inspect the rule basis.

The practical principle is simple: the view may summarize, but it should not trap the user inside the summary. A serious compliance screen always leaves a path back to evidence.

Chapter checkpoint

CHECKPOINT

1. Why is a compliance view not the owner of the facts it displays?
2. Which services contribute facts to a NEXUS-1 compliance view?
3. Why should the view consume public contracts rather than private tables?
4. What is the difference between a compliance review workflow and a compliance view?
5. Why are freshness and waiting markers especially important for compliance screens?
6. Why should a view link back to source records instead of becoming a self-contained substitute for them?

The next chapter turns from compliance views to audit trails across service boundaries. If Chapter 43 asks how reviewers see distributed facts, Chapter 44 asks how those facts remain provable after they cross service lines.

PART VII - CONSISTENCY, REPORTING, AND READ MODELS

44. Audit Trails Across Service Boundaries

An audit trail is easy to imagine while everything writes into one database. It becomes harder - and more important - when the truth moves through services, messages, projections, retries, and human review.

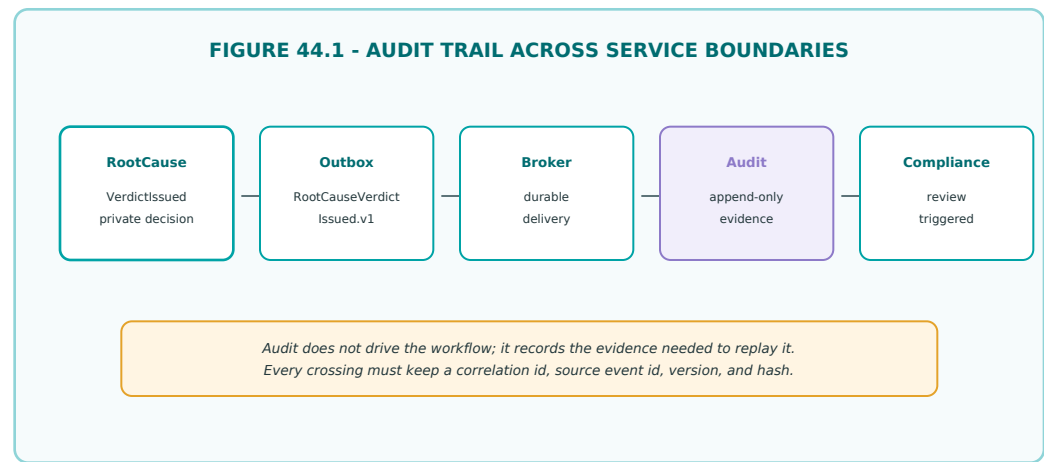


Figure 44.1 - A cross-service audit trail records the movement of evidence without becoming the workflow engine that caused the movement.

Why audit becomes harder after service extraction

Before service extraction, audit often feels like a table at the edge of the application. A controller changes a row, an interceptor records who changed it, and the database transaction keeps the business change and the audit entry together. The design may be simple, but its simplicity depends on one important assumption: the write and the audit record share the same transactional boundary.

Microservices remove that assumption. RootCause can issue a verdict in its own database. Audit can record immutable evidence in another database. Compliance can begin a review later. Reporting can project a composed view even later. The audit trail is no longer one row written beside another row. It is a chain of evidence across service boundaries.

PLAIN EXPLANATION

In a microservice architecture, audit is not only about who changed a table. It is about proving that a fact was created by the service that owns it, published through the contract it claims, consumed by the service that reacted to it, and preserved with enough identity to replay the path later.

That is why audit trails belong in the same architectural conversation as messaging, outbox, inbox, projections, and process managers. They do not replace those patterns. They make those patterns accountable.

Audit is evidence, not workflow

The most important sentence in this chapter repeats a rule from the earlier volumes: AuditEntry is evidence, not workflow. An audit entry may prove that RootCause issued a verdict. It may prove that Compliance opened a review. It may prove that a projection applied an event. But the audit entry itself should not cause the business step.

Thing recorded	Owner of the decision	Role of audit
Verdict issued	RootCause service.	Record that the verdict was issued, by whom/what, from which case, under which event version.
Review requested	Compliance service or process manager.	Record that the review request was produced and correlated to the verdict.
Projection updated	Reporting or projection worker.	Record that a read model applied a source event at a known time.
Human reviewed	Compliance workflow.	Record the reviewer identity, time, decision, and evidence links.

NEXUS-1 SENTENCE TO REMEMBER

Audit records what the system did; it should not secretly decide what the system must do next.

The audit envelope

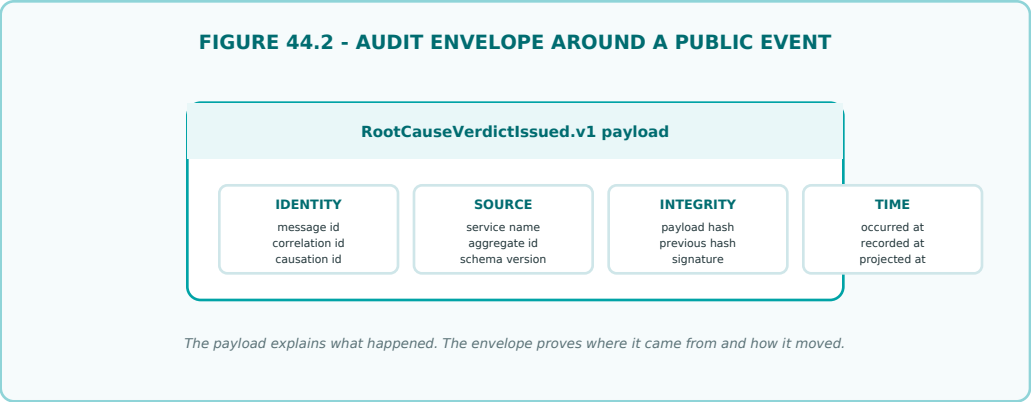


Figure 44.2 - The payload tells the business story; the envelope carries the proof needed to trust and replay the story.

A public event needs more than domain payload. `RootCauseVerdictIssued.v1` should carry the verdict facts that consumers need. Around that payload, the system also needs an audit envelope: message identity, correlation, causation, source service, schema version, timestamps, and integrity markers.

Envelope field	Question answered	Why it matters
MessageId	Which exact message is this?	Supports idempotency, replay, and duplicate detection.
CorrelationId	Which case or flow does this belong to?	Connects alarm, verdict, audit, compliance, and reporting records.
CausationId	What caused this message?	Reconstructs the chain of reactions across services.
SourceService	Who published it?	Prevents anonymous facts from entering the audit trail.
ContractVersion	Which public contract shape was used?	Allows old evidence to remain readable after later contract versions exist.
PayloadHash	Was the payload altered?	Supports tamper evidence and replay verification.

A C# shape for auditable messages

A small envelope type makes the rule executable. The domain event remains private inside the service. The integration event becomes public. The audit envelope travels with the public event so that consumers and auditors can reason about the same identity.

```
public sealed record AuditableMessage<TPayload>(  
    Guid MessageId,  
    string CorrelationId,  
    string? CausationId,  
    string SourceService,  
    string ContractName,  
    string ContractVersion,  
    DateTime OccurredAtUtc,  
    DateTime RecordedAtUtc,  
    string PayloadHash,  
    TPayload Payload);
```

The wrapper is intentionally boring. It does not contain business rules. It names the message, the source, the contract, the time, and the integrity marker. That boredom is a strength because it lets many services use the same evidence grammar without sharing their domain models.

DESIGN RULE
Standardize the audit envelope. Do not standardize the private domain model behind it.

Where the audit record is written

A distributed system can write audit evidence in more than one place. The important point is to know which evidence each place owns and what guarantee it can honestly make.

Audit location	Guarantee it can make	Boundary
Local service audit table	The service recorded its own write in the same local transaction.	Does not prove another service consumed the event.
Outbox record	The service committed a public message to be published.	Does not prove successful business processing downstream.
Central audit service	A durable, queryable evidence ledger collected records from many services.	Must not become a shared write model or workflow brain.
Projection audit metadata	The read model can say which source events it applied.	Does not become the source event itself.

In NEXUS-1, a balanced design uses all four. RootCause keeps local evidence of its decision. The outbox keeps the public event until it is published. Audit receives and stores an immutable record. Reporting stores projection metadata so the console can explain freshness and lineage.

Tamper evidence and hash chaining

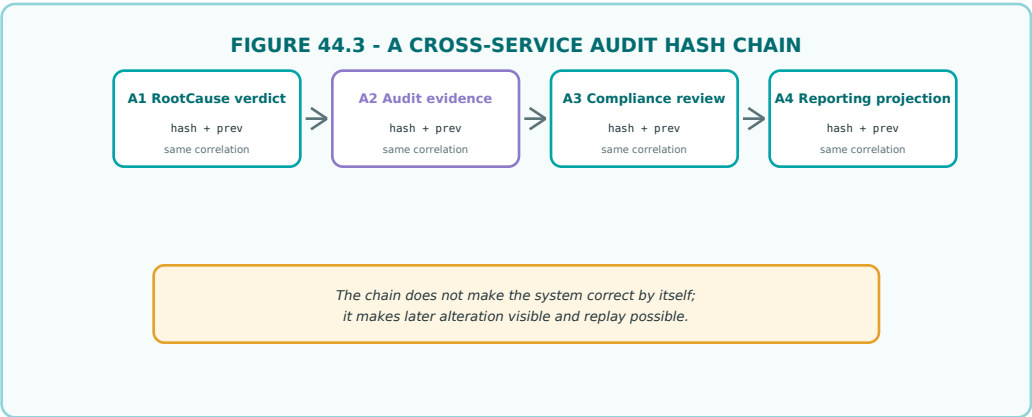


Figure 44.3 - A hash chain lets separate records remain independently stored while still forming a replayable evidence path.

Audit cannot prevent every mistake. It can make hidden alteration difficult. One simple way is to hash the canonical event payload and link audit records through previous hashes within the same correlation chain.

The goal is not theatrical cryptography. The goal is engineering accountability. If a RootCause verdict, Audit evidence record, Compliance review, and Reporting projection all claim to belong to the same flow, a later reviewer should be able to replay their order, compare hashes, and detect that something has changed.

HONEST BOUNDARY

A hash chain does not prove that the original decision was correct. It proves that the recorded evidence path has not silently changed according to the verification rules the system uses.

The NEXUS-1 audit trail for a verdict

The recurring alarm-to-compliance flow gives a concrete audit chain. The chain begins inside RootCause, crosses the outbox boundary, enters Audit, triggers Compliance, and appears later in Reporting. Each step has a different owner, but the correlation identity ties them together.

Step	Service	Audit evidence	Must not claim
1	RootCause	VerdictIssued domain event, local case version, evidence references.	That compliance has already reviewed it.
2	RootCause outbox	RootCauseVerdictIssued.v1, envelope, payload hash.	That consumers have processed it.
3	Audit	Immutable audit entry, source event id, hash link.	That the workflow is complete.
4	Compliance	Review requested/opened, reviewer actions, control mapping.	That RootCause changed its verdict.
5	Reporting	Projection applied event ids, freshness metadata.	That the projection is the source of truth.

Notice the repeated phrase: must not claim. Audit trails are useful only when each record states what it can prove and stops before pretending to prove more.

What the operator console should show

The console should not show the entire audit ledger to every user. It should show enough evidence to make the distributed state understandable: source service, last event applied, correlation id, freshness state, and links to source records. A deeper audit screen can expose the full chain when needed.

UI label	Meaning	Example
Source	The service that owns the fact being displayed.	RootCause
Last event	The most recent public fact applied to the view.	RootCauseVerdictIssued.v1
Correlation	The flow identity connecting alarm, verdict, audit, and review.	EVT-2026-0418 / case correlation id
Freshness	Whether the read model is fresh, pending, or stale.	Pending audit evidence
Evidence link	A path back to the source record or immutable audit entry.	Open audit chain

USER-INTERFACE RULE

A distributed console should not hide audit delay. It should display enough lineage for the user to understand whether the view is complete, pending, or stale.

Chapter checkpoint

CHECKPOINT

1. Why does audit become harder when a system moves from one database to services?
2. Why is AuditEntry evidence rather than workflow?
3. Which fields belong in an auditable message envelope?
4. What is the difference between a local audit record, an outbox record, and a central audit ledger?
5. What can a hash chain prove, and what can it not prove?
6. What should the operator console show so distributed facts remain explainable?

The next chapter continues the same accountability theme from a different angle: traceability and correlation IDs. Chapter 44 makes the evidence durable. Chapter 45 will make the path through the system searchable.

PART VII - CONSISTENCY, REPORTING, AND READ MODELS

45. Traceability and Correlation IDs

A distributed system does not only need to record what happened. It needs to connect what happened. Correlation IDs give one flow a searchable spine, so an alarm, a verdict, an audit entry, a compliance review, and a reporting projection can still be read as one story.

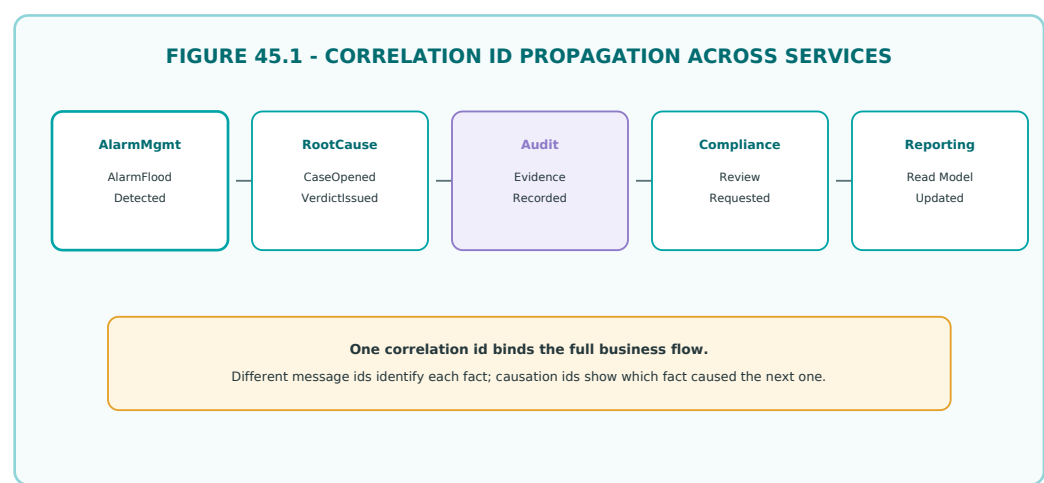


Figure 45.1 - Correlation ID propagation keeps a distributed alarm-to-compliance flow readable even when each service owns its own data and messages.

Why traceability becomes a design concern

Inside a modular monolith, the call stack often gives a temporary sense of traceability. A request enters one process, a service calls another service, a transaction commits, and the debugger can show the sequence while it is still happening. That convenience disappears when the same business flow crosses services, databases, queues, background workers, retries, and projections.

After service extraction, the most important question is often not simply what failed. It is which business flow this failure belongs to. Did the delayed compliance view belong to the same alarm flood as the root-cause verdict? Did the duplicate message belong to the same case or to a later correction? Did the reporting projection apply the same event that the audit ledger recorded?

PLAIN EXPLANATION

Traceability is the ability to follow one business story across technical boundaries. A correlation ID is the identifier that lets the story keep one name while the technical work is split across services.

Without traceability, a system can be technically distributed and operationally blind. Each service may know what it did locally, but no one can reconstruct the path from alarm to compliance review without guessing.

Correlation is not the same as identity

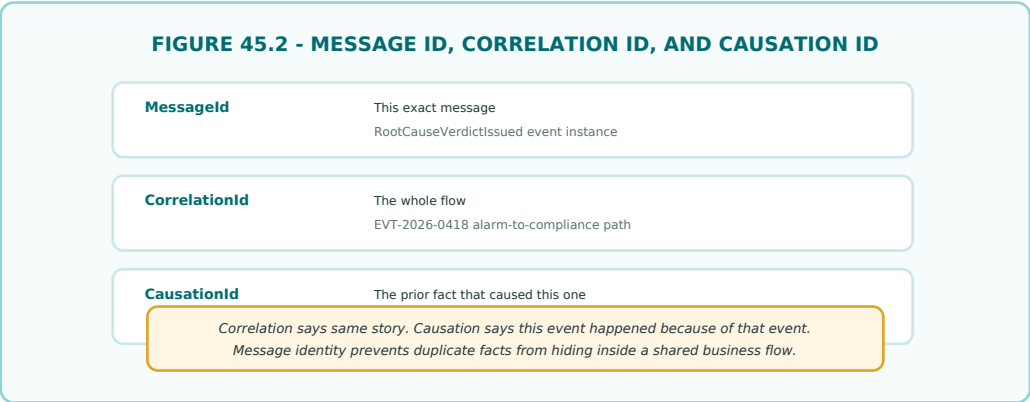


Figure 45.2 - Message identity, correlation identity, and causation identity answer different questions and should not be collapsed into one field.

A common mistake is to put one ID on a message and expect it to answer every question. That creates confusion. The message id identifies one message. The correlation id identifies the whole business flow. The causation id identifies the previous fact that caused this new fact to be produced.

Identifier	Question answered	NEXUS-1 example
MessageId	Which exact message instance is this?	The unique RootCauseVerdictIssued.v1 message delivered to a consumer.

CorrelationId	Which larger flow does this belong to?	The alarm-to-compliance flow created from EVT-2026-0418.
CausationId	Which prior message or command caused this?	RootCauseCaseOpened was caused by AlarmFloodDetected.v1.
AggregateId	Which domain object owns the fact?	RootCauseCaseId or ComplianceReviewId.

DESIGN RULE

Do not reuse MessageId as CorrelationId. A duplicate message and a different message in the same flow must be distinguishable.

The traceability contract

Traceability should not be left to logging convention alone. It belongs in the contract envelope that surrounds public commands, public events, and projection metadata. The payload explains the business fact; the trace envelope explains where the fact came from and how it relates to the surrounding flow.

```
public sealed record TraceEnvelope(
    string CorrelationId,
    string? CausationId,
    Guid MessageId,
    string SourceService,
    string ContractName,
    string ContractVersion,
    DateTime OccurredAtUtc,
    DateTime RecordedAtUtc);
```

This shape is deliberately close to the audit envelope from Chapter 44. The difference is emphasis. Audit asks whether evidence can be replayed and trusted. Traceability asks whether related facts can be found and connected. In practice, the same message envelope often supports both.

Envelope field	Traceability role	When missing
CorrelationId	Groups all records in one business flow.	The operator sees isolated facts, not a coherent incident.
CausationId	Links the current fact to the prior fact that caused it.	The flow becomes a cloud of events with no readable chain.
MessageId	Identifies one exact message instance.	Duplicate detection and replay become unreliable.
SourceService	Names the service that published the fact.	Consumers cannot distinguish owner truth from copied projection.
ContractVersion	Names the public shape used at the boundary.	Old traces become ambiguous after contract evolution.

Where a correlation ID begins

A correlation ID should start where the business story starts, not where a technical request happens to enter. In NEXUS-1, the recurring story begins with an alarm flood or an operator action that opens a case. The ID should then be carried into

RootCause, Audit, Compliance, and Reporting without being replaced at every hop.

Starting point	Correlation strategy	Reason
Alarm flood detected	Create a new flow correlation id if none exists.	The flood becomes the business story that later services follow.
Root-cause case opened	Reuse the alarm correlation id, attach case id separately.	Case identity is not the same as flow identity.
Verdict corrected	Reuse the original correlation id, publish a new message id.	The correction belongs to the same story but is a new fact.
Manual compliance escalation	Reuse the existing correlation id if it reacts to a known event.	Human action remains connected to the system path.

NEXUS-1 SENTENCE TO REMEMBER
RootCauseCaseId identifies a case. CorrelationId identifies the distributed story the case belongs to.

Propagating correlation through commands, events, and reads

Traceability has to survive three kinds of movement: synchronous requests, asynchronous messages, and read-model updates. If any one of these drops the correlation ID, the chain becomes partial. Partial traceability is still useful, but it forces investigation to jump across gaps.

Movement	How correlation travels	NEXUS-1 example
Command	Command metadata carries CorrelationId and optional CausationId.	OpenRootCauseCase is caused by AlarmFloodDetected.v1.
Integration event	Public event envelope carries correlation, causation, and message identity.	RootCauseVerdictIssued.v1 keeps the same correlation id.
HTTP request	Header carries x-correlation-id; server validates or creates one.	Operator console request for case status uses the same flow id.
Projection update	Projection row stores SourceMessageId and CorrelationId.	Compliance dashboard can explain which event it last applied.
Audit record	Audit entry stores correlation plus hash/evidence metadata.	Audit can replay the path for the case.

This is why correlation ID propagation appears in the implementation list for this book. It is not a logging decoration. It is an architectural requirement for distributed accountability.

A small middleware shape

At the edge of a service, middleware can normalize the incoming correlation ID. If the caller provides one, the service validates and stores it in a request context. If the caller does not provide one, the service creates one. Downstream commands and events then receive it from the same context.

```
public sealed class CorrelationContext
{
    public string CorrelationId { get; init; } = default!;
    public string? CausationId { get; init; }
}

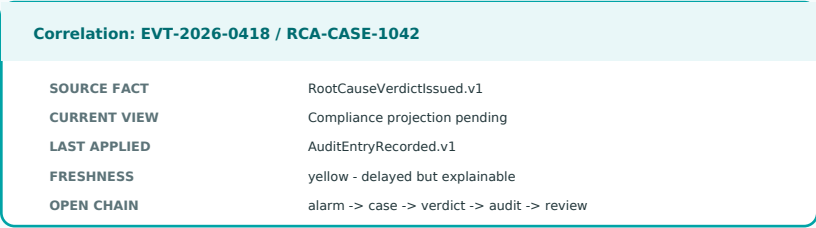
// HTTP edge rule;
// 1. read x-correlation-id if present
// 2. create one if absent
// 3. store it in request context
// 4. copy it into commands and integration events
```

HONEST BOUNDARY

Middleware can carry correlation; it cannot invent business meaning. The domain still decides what happened. Correlation only helps the system trace that decision after it crosses boundaries.

Traceability in the operator console

FIGURE 45.3 - TRACEABILITY PANEL FOR THE OPERATOR CONSOLE



The user should see why a screen is delayed, not just that it is delayed.

Figure 45.3 - A traceability panel turns delayed distributed state into a readable status rather than a mysterious gap.

The operator console should not require a developer to open logs in order to understand a delayed view. If a compliance view is waiting for an audit projection, the UI can show the correlation ID, the last event applied, the source service, and the freshness state. The goal is not to expose internal plumbing to every user. The goal is to prevent delayed truth from looking like missing truth.

Console question	Trace data needed	User-facing answer
Why is compliance pending?	Last applied event and expected next event.	Verdict received; compliance projection not yet applied.
Which incident is this?	CorrelationId and source event id.	This view belongs to EVT-2026-0418.
Is this duplicate?	MessageId and idempotency result.	Duplicate event ignored; view unchanged.
Can I audit it?	Audit entry id and hash link.	Open evidence chain for the same correlation.

What traceability must not become

Traceability is powerful enough to become dangerous if it is misunderstood. A correlation ID does not authorize access. It does not prove correctness. It does not turn a projection into source truth. It does not give a downstream service permission to mutate another service database. It is a line through the evidence, not a master key through the system.

Mistake	Why it is wrong	Correct boundary
Using correlation as authorization	Knowing a flow id does not mean the caller may see every record in that flow.	Security policy still decides access.
Using correlation as truth	Related records may still disagree, lag, or be corrected.	Source service remains the owner of truth.
Changing another service by correlation	A flow id is not permission to write into a remote model.	Use public commands or events only.
Hiding causation under correlation	Same flow does not explain which event caused which reaction.	Carry causation id separately.

SECURITY BOUNDARY

A correlation ID may appear in logs, messages, and read-model metadata. It should never be treated as a secret or as proof that the caller is allowed to read the whole flow.

Chapter checkpoint

CHECKPOINT

1. What problem does a correlation ID solve in a distributed system?
2. Why are MessageId, CorrelationId, and CausationId separate identifiers?
3. Where should a correlation ID begin in the NEXUS-1 alarm-to-compliance flow?
4. How should commands, events, HTTP requests, projections, and audit records carry trace metadata?
5. What should the operator console show when a view is delayed?
6. Why must correlation never be used as authorization or as source-of-truth proof?

Chapter 44 made the evidence durable. This chapter made the path searchable. The next chapter closes Part VII by explaining how the system should speak honestly to users when a distributed view is delayed but not wrong.

PART VII - CONSISTENCY, REPORTING, AND READ MODELS

46. Explaining Delayed Truth to Users

Eventual consistency is not only a backend property. It becomes a user experience. If the screen hides delay, users lose trust. If the screen explains delay with evidence, freshness, and traceability, users can keep working without confusing a delayed view with a wrong system.

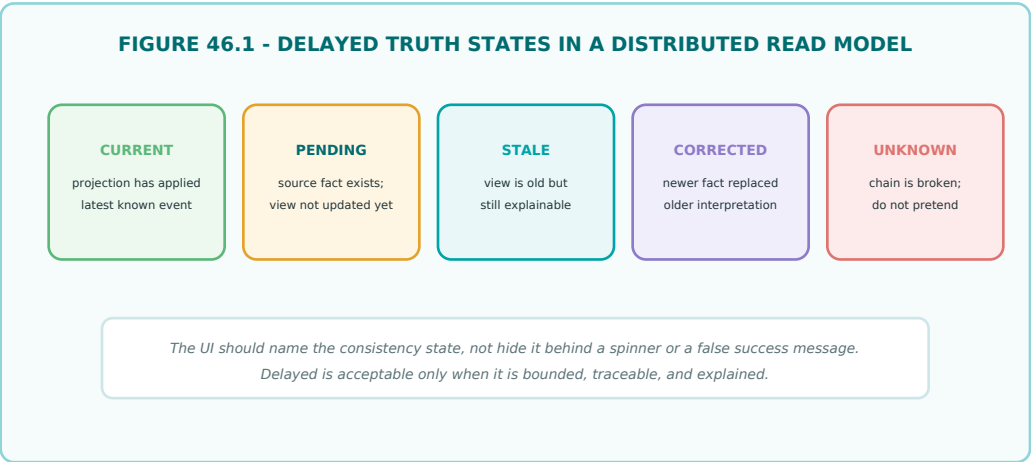


Figure 46.1 - Delayed truth states give the operator console a controlled vocabulary for views that are current, pending, stale, corrected, or unknown.

The user does not care which queue is behind

Developers often explain eventual consistency in technical language: the event is in the broker, the projection worker is behind, the consumer retried, the read model has not caught up. That may be true, but it is not the language a user needs at the console. The user wants to know whether the system is wrong, whether the view is old, whether the next step has been requested, and whether it is safe to continue reading the page.

A distributed architecture therefore creates a communication responsibility. Once a screen is built from multiple services and projections, the UI must explain which truth it is showing. A projection is not allowed to behave like source truth. A stale view is not allowed to look current. A missing event is not allowed to look like a completed workflow.

PLAIN EXPLANATION
Delayed truth means the source service has already accepted or published a fact, but the read model or screen has not yet reflected it. The fact may be correct; the view may simply be behind.

This chapter closes Part VII by turning the technical material of consistency, projections, compliance views, audit trails, and correlation IDs into a user-facing discipline.

Why spinners are not enough

The easiest user-interface response to uncertainty is a spinner. A spinner says wait. It does not say what is being waited for, which source has already moved, which projection is behind, or whether the delay is expected. A spinner can be acceptable for a short technical operation. It is not enough for a distributed operational workflow.

Naive message	Why it fails	Better message
Loading...	Does not distinguish slow network, missing data, delayed projection, or failure.	Compliance view is updating from latest verdict.
No compliance review found	May imply the review does not exist when it is only not projected yet.	Review request pending; last applied event is AuditEntryRecorded.v1.
Something went wrong	Hides whether the system has evidence, a retry, or a traceable cause.	Projection worker is retrying; source verdict remains recorded.
Completed	May hide that downstream projections are still catching up.	Verdict completed; compliance projection pending.

USER-EXPERIENCE RULE
A delayed distributed view must explain four things: what source fact is known, what view is behind, what the last applied event was, and what the user may reasonably do next.

Source truth and projection truth

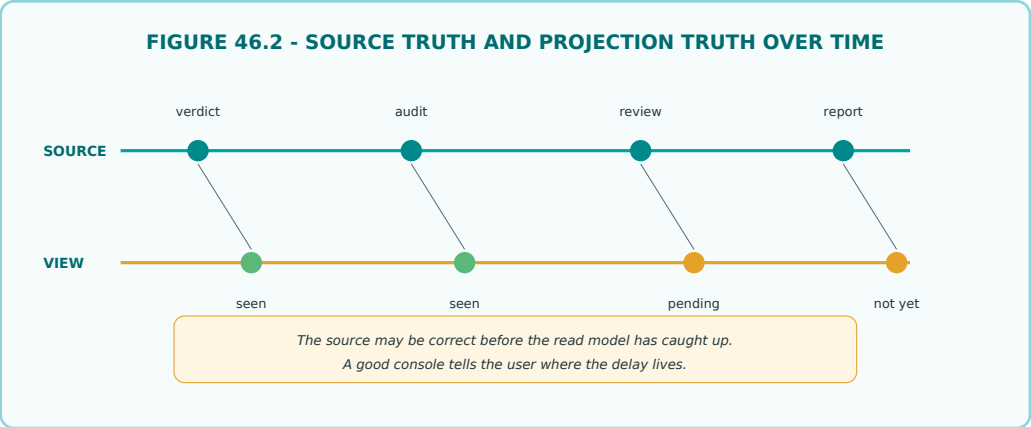


Figure 46.2 - The source service can be ahead of the read model. The user experience must show that difference without turning it into panic.

In NEXUS-1, RootCause may issue a verdict before Compliance has projected the review request. Audit may have recorded the evidence while Reporting still shows the previous summary. This is not automatically a defect. It becomes a defect only when the system presents the older view as if it were the whole truth.

Truth type	Owned by	User-facing meaning
Source truth	The service that owns the decision or fact.	The verdict has been issued by RootCause.
Audit truth	The append-only evidence record.	The verdict was recorded and can be replayed.
Projection truth	The read model built for a screen or report.	The dashboard has or has not caught up.
UI freshness	The console presenting status to the user.	The screen is current, delayed, stale, corrected, or unknown.

Once those truth types are separated, the console can be honest without becoming noisy. It can say: the verdict exists, the audit record exists, the compliance view is pending, and the system is retrying.

A consistency status contract

The backend should not force the UI to infer consistency state from missing fields. A read API can return data and consistency metadata together. The data says what the screen can display. The metadata says how fresh, traceable, and complete that data is.

```
public sealed record ProjectionStatusDto(
    string SourceService,
    string ProjectionName,
    string ConsistencyState,    // Current, Pending, Stale, Corrected, Unknown
    string? CorrelationId,
    Guid? LastAppliedMessageId,
    string? LastAppliedContract,
    DateTime? LastAppliedAtUtc,
    string? ExpectedNextContract,
    string UserMessage);
```

This is not a domain object. It is a boundary contract for a read model. It exists because the console is part of the distributed system. It should not need to guess whether a missing compliance row means no review, delayed review, or broken projection.

DESIGN RULE

A read model should expose freshness explicitly. Missing data plus no freshness metadata is how eventual consistency turns into user confusion.

Writing messages that preserve trust

FIGURE 46.3 - A DELAYED-TRUTH MESSAGE FOR THE OPERATOR CONSOLE

Compliance view is updating

Root-cause verdict RCA-1042 has been issued and audited.
Compliance review request has not yet appeared in this view.
Last applied event: AuditEntryRecorded.v1 at 14:22:08 UTC.
Correlation: EVT-2026-0418 / expected next: RequestComplianceReview.
Status: delayed, traceable, retrying automatically.

The message names the source fact, the missing projection, the last known event, and the next expected step.

Figure 46.3 - A useful delayed-truth message names the source fact, the delayed view, the last applied event, and the next expected step.

Good delayed-truth messages avoid two extremes. They do not bury the user in broker terminology, and they do not pretend everything is finished. They translate technical consistency into operationally useful language.

Message element	Purpose	Example
Known fact	Tells the user what is already true at the source.	Root-cause verdict RCA-1042 has been issued.
Delayed view	Names what has not caught up yet.	Compliance review is not visible in this view yet.
Last applied event	Anchors the view in a traceable event.	Last applied: AuditEntryRecorded.v1.

Expected next step	Shows that the system knows what it is waiting for.	Expected: RequestComplianceReview.
User action	Prevents unnecessary manual work.	No action required; retrying automatically.

What delayed truth must not hide

Explaining delay is not permission to excuse every failure. Some delays are normal. Some delays are warnings. Some delays indicate that a projection has stopped, a message is poisoned, or a contract changed incorrectly. The user-facing status must therefore be tied to operational thresholds.

State	Meaning	Required behavior
Current	The view has applied the latest relevant event.	Show normal status; keep trace metadata available.
Pending	A source fact exists and the view is expected to catch up.	Show calm explanation and expected next step.
Stale	The view has not updated within the expected freshness window.	Show warning and link to trace or operations status.
Corrected	A newer event changed an earlier interpretation.	Show correction clearly; do not silently overwrite history.
Unknown	The system cannot prove where the view stands.	Do not pretend. Show honest uncertainty and escalation path.

HONEST BOUNDARY

A polite message is not a substitute for reliability. If a projection is stale beyond its threshold, the system needs observability, alerting, and operator escalation - not better wording alone.

Applying the rule to the NEXUS-1 flow

Consider the recurring alarm-to-compliance flow. AlarmManagement detects a flood. RootCause opens a case and later issues a verdict. Audit records evidence. Compliance is triggered. Reporting builds a read model. At several points, a user may open the console between source truth and projection truth.

Moment	Bad screen behavior	Better delayed-truth behavior
Verdict issued, audit pending	Show verdict as complete with no evidence status.	Verdict issued; audit evidence is being recorded.
Audit written, compliance pending	Show no compliance action and leave user guessing.	Compliance review request pending from audited verdict.
Compliance projected, reporting behind	Show old report as if current.	Report generated from previous projection; refresh pending.
Correction event published	Replace old verdict silently.	Show corrected verdict and link to prior version.

This is the final lesson of Part VII. Consistency is not only how services store and project data. It is how the system tells the truth about what the user is seeing.

Part VII checkpoint

CHECKPOINT

1. Why is a spinner not enough for a delayed distributed view?
2. What is the difference between source truth, audit truth, projection truth, and UI freshness?
3. What fields should a projection freshness contract expose?
4. What should a delayed-truth message say to the user?
5. When does normal pending state become stale state?
6. Why must a corrected fact be shown as corrected rather than silently overwritten?

Part VII began by ending single-database thinking. It then followed eventual consistency through the operator console, reporting projections, compliance views, audit trails, and traceability. This chapter completes the arc by making delayed truth visible to users. The next part moves to security: once services have independent boundaries, the system must decide who may cross them and how those crossings are proven.

PART VIII

Security, Identity, and Access Boundaries

Once a modular core becomes a distributed system, security stops being a single middleware decision. Every service boundary becomes a security boundary. Every call must answer two separate questions: who is calling, and what is that caller allowed to do?

Part VIII treats identity and access as architecture, not decoration. It does not turn NEXUS-1 into operational plant software and it does not claim a certified security model. It asks the software question that distributed design forces: when AlarmManagement, RootCause, Audit, Compliance, Reporting, and the Operator Console no longer live inside one process, how is trust carried without becoming blind trust?

WHAT PART VIII ADDS

Part VII explained delayed truth to users. Part VIII explains controlled crossing of boundaries: authentication, authorization, service-to-service identity, claims, roles, policies, secrets, certificates, configuration, and least privilege.

Chapter	Security question
47	How is authentication different from authorization once services call other services?
48	How does one service prove its identity to another service?
49	Where do claims, roles, and policies belong, and where do they become dangerous?
50	How are secrets, certificates, and configuration handled without leaking trust?
51	What does least privilege mean between independently deployed services?
52	How does NEXUS-1 draw a security boundary map that is honest about what it protects?

PART VIII - SECURITY, IDENTITY, AND ACCESS BOUNDARIES

47. Authentication vs Authorization Across Services

Knowing who is calling is not the same as deciding what that caller may do. In a distributed system, confusing those two questions is one of the fastest ways to turn service boundaries into security theatre.

In a modular monolith, authentication and authorization are often experienced as one thing. A request enters the web application. Middleware validates the user. Policies run in the same process. Controllers call application handlers. The database receives the change. The same runtime sees the user, the action, and the data.

Microservices break that comfort. The Operator Console may call a Backend-for-Frontend. The BFF may call RootCause. RootCause may publish an integration event. Compliance may react later. Audit may record the chain. Reporting may show a projection. By the time a decision is made, the original user is no longer the only caller in the room. Services are callers too.

PLAIN EXPLANATION

Authentication asks: who is this caller? Authorization asks: is this caller allowed to perform this action on this resource now? A valid identity is not permission. A trusted service is not permission. A network route is not permission.

What you already know

You already know the difference in ordinary life. A badge can prove that a person works in a building. It does not prove that the person may enter every room, approve every document, or override every procedure. Authentication is the badge. Authorization is the door rule, the approval rule, and the context around the action.

Software is the same. A JSON Web Token, cookie, certificate, API key, or managed identity can prove a caller. None of them automatically answers whether that caller may open a root-cause case, issue a verdict, correct a published event, view a compliance finding, or export an audit trail.

Question	Authentication answer	Authorization answer
Who is calling?	An operator, the BFF, RootCauseService, ComplianceService, or a background worker.	Not enough information. Identity is only the starting point.
What is being requested?	Usually not answered by authentication.	OpenRootCauseCase, IssueVerdict, RequestComplianceReview, ExportAuditEvidence.

Question	Authentication answer	Authorization answer
Which resource is affected?	Usually not answered by authentication.	A specific unit, case, verdict, compliance review, report, or audit batch.
Is it allowed now?	No. The caller may be known but not allowed.	Yes or no, based on policy, role, claim, ownership, state, and boundary rules.

The monolith assumption that does not survive

A monolith can accidentally rely on a simple assumption: if the request reached the handler, the security question must already have been answered. That assumption is dangerous even in one process, but distribution makes it worse. One service may receive a message hours after the user clicked a button. Another service may receive a call from a BFF rather than from the browser. A projection worker may run without a human user at all.

The system therefore has to separate four identities that are often blurred in smaller applications:

Identity	Meaning in NEXUS-1	Risk if confused
Human actor	The operator, reviewer, administrator, or analyst who initiated or approved an action.	The system loses accountability for the person behind a change.
Client application	The Operator Console, BFF, job runner, or integration client that submitted the request.	The UI becomes trusted too much, or background jobs appear as humans.
Service identity	RootCauseService, AuditService, ComplianceService, ReportingService, or another backend component.	A service-to-service call is accepted just because it came from inside the network.
Domain authority	The context that is allowed to decide a fact: RootCause issues verdicts; Audit records evidence.	A downstream service starts making decisions it only has permission to observe.

DESIGN RULE

A downstream service should not accept a request merely because the upstream service is authenticated. It must still ask whether the caller has the right authority, for this action, on this resource, at this boundary.

A NEXUS-1 example: opening a root-cause case

Consider the moment after AlarmManagement detects a flood. The Operator Console shows the event. A human operator requests that a root-cause case be opened. In a single application, one controller might check the user role and call the RootCause application service directly. In a service architecture, the flow crosses more than one boundary.

FIGURE 47.1 - AUTHENTICATION AND AUTHORIZATION IN ONE FLOW

Operator -> Operator Console -> BFF -> RootCause API -> RootCause Application Service

|||

user token service identity domain policy

Authentication proves the operator and the service. Authorization decides whether this user, through this client, may request this RootCause action for this unit and case context.

Figure 47.1 - Authentication is carried through the path, but authorization must be evaluated at the service boundary that owns the action.

The BFF may authenticate the user for the console session. It may check that the user has access to the NEXUS-1 operator experience. That is useful, but it is not the RootCause decision. RootCause must still protect its own command boundary. It must ask whether the call is allowed to open a case for this unit, whether the command came through an accepted client, whether the service caller is trusted for this operation, and whether the domain state permits the operation.

Layer	What it authenticates	What it authorizes
Operator Console	The human user session, usually through a token or cookie.	Only UI affordances: which buttons to show, which routes to expose.
BFF	The browser client and user token.	Client-level access to console use cases; it should not become the domain authority.
RootCause API	The calling service or BFF, plus the propagated user context.	Whether OpenRootCauseCase is allowed for this unit, this user, this caller, and this state.
RootCause domain/application layer	Nothing by itself; it receives a trusted actor context from the boundary.	Whether the command violates invariants or domain rules.
Audit	The publisher and trace envelope.	Whether the evidence record can be appended; it does not authorize the original domain decision.

Authorization belongs at more than one level

One common mistake is to put all authorization in the gateway. Another is to put all authorization in each service and ignore the gateway entirely. A serious service architecture needs both, but the responsibilities are different.

The edge can reject obvious wrongness: expired tokens, unknown clients, missing scopes, and users who have no access to the console at all. The service must protect its own authority: commands, resources, aggregate state, tenant or plant scope, and domain-specific permission. The application layer must still protect invariants even if authorization says yes.

Place	Good responsibility	Bad responsibility
Gateway or BFF	Authenticate the user, reject invalid tokens, apply broad route policy, reduce accidental exposure.	Pretend it knows every domain rule inside RootCause, Compliance, Audit, or Reporting.
Service boundary	Check caller trust, user claims, service scopes, resource access, command permission, and contract policy.	Blindly trust every internal call because it came from the cluster or VNet.
Application handler	Apply use-case policy and load resource state needed for authorization decisions.	Depend on HTTP objects or UI roles directly.
Domain model	Protect invariants that must always hold.	Know about JWT claims, OAuth scopes, or gateway details.

NEXUS-1 SENTENCE TO REMEMBER

The gateway may decide whether the caller may reach RootCause. RootCause decides whether the caller may perform a RootCause action. The aggregate still decides whether the action is valid for its current state.

Actor context without leaking infrastructure

A service should not pass raw HTTP tokens deep into the domain model. It should translate external identity into a small actor context that the application layer can reason about. The context should name the human actor when there is one, the service caller, the correlation ID, and the permission evidence used at the boundary.

```
public sealed record ActorContext(
    string? UserId,
    string? UserDisplayName,
    string ClientId,
    string CallingService,
    IReadOnlySet<string> Roles,
    IReadOnlySet<string> Permissions,
    string CorrelationId);

public sealed record OpenRootCauseCaseCommand(
    int UnitId,
    string AlarmFloodId,
    ActorContext Actor);

public interface IRootCauseAuthorizationPolicy
{
    Task EnsureCanOpenCaseAsync(
        OpenRootCauseCaseCommand command,
        CancellationToken cancellationToken);
}
```

This is not a complete security implementation. It is an architectural shape. The HTTP layer validates tokens and certificates. The service boundary translates them into an ActorContext. The application handler asks a policy whether the action is allowed. The domain aggregate receives only the decision and the actor information it needs for accountability.

Service authentication is not user authorization

A service-to-service credential proves that a backend component is allowed to speak on the network. It does not prove that every user action carried by that component is allowed. If the BFF calls RootCause, RootCause should be able to know both the service caller and the represented user. If a background process calls Compliance, Compliance should know that the action is system-initiated and should record that honestly.

Call type	Authenticated caller	Authorization question
User-driven command	BFF service plus propagated operator identity.	May this operator request this command for this unit and context?
System reaction to event	Process manager or event consumer service.	Is this service allowed to react to this event contract in this way?
Projection update	Projection worker identity.	May this worker update this read model from this event version?
Audit append	Publishing service plus trace envelope.	May this evidence be appended, and does it carry enough provenance?

HONEST BOUNDARY

This chapter does not prescribe a particular identity provider, OAuth flow, certificate authority, cloud platform, or secret store. It defines the architectural separation that those tools must support. Tooling choices belong later; confusing authentication with authorization would remain wrong with any tool.

The minimum rule set for Chapter 47

A service boundary becomes credible only when it can answer the following questions without reaching for vague words like internal or trusted:

Question	Required answer
Who is the human actor, if there is one?	A stable user identifier and displayable accountability data.
Which client or service submitted the request?	A client ID or service identity separate from the user.
Which action is requested?	A named command, query, or event reaction.
Which resource is affected?	Unit, RootCauseCase, ComplianceReview, AuditBatch, report, or projection key.
Which policy allowed it?	A named policy, permission, scope, or rule that can be audited.
What trace connects it to the distributed flow?	Correlation ID, causation ID, and message or request ID.

In the next chapter, the same distinction moves one level deeper. Once authentication and authorization are separated, services still need a way to prove their own identities to one another. That is the subject of service-to-service authentication.

Chapter 47 checkpoint

CHECKPOINT

1. Why is authentication not enough to authorize a command?
2. Why should the gateway not become the only authorization authority?
3. What is the difference between a human actor, a client application, and a service identity?
4. Why should RootCause re-check authorization even when the BFF is trusted?
5. What should an ActorContext contain, and what should it avoid leaking into the domain model?

PART VIII - SECURITY, IDENTITY, AND ACCESS BOUNDARIES

48. Service-to-Service Authentication

A user token says something about a human or client session. It does not, by itself, prove that RootCauseService, ComplianceService, AuditService, or a projection worker is really the service it claims to be.

The previous chapter separated authentication from authorization. This chapter narrows the first word. In a distributed NEXUS-1 architecture, authentication no longer happens only at the edge. Services call services. Workers consume messages. Process managers invoke APIs. Projections update read models. Each of those actors must prove its identity before another service should trust the call.

The danger is subtle. A system can authenticate users carefully and still allow backend services to trust each other too loosely. If every request that comes from the internal network is accepted, the network has become the identity provider. If every token forwarded by a BFF is accepted as if it were the whole truth, the BFF has become too powerful. If every consumer of an event is allowed to react as if it were the owner of the decision, boundaries have collapsed.

PLAIN EXPLANATION

Service-to-service authentication asks: which software component is calling? It proves the caller is RootCauseService, ComplianceService, AuditService, ReportingProjectionWorker, or the OperatorConsole BFF. It does not automatically prove what that service may do, and it does not replace user authorization.

What you already know

In ordinary development, you already use different keys for different systems. Your database has a connection string. Your CI pipeline has a deployment credential. Your cloud function has a managed identity. Your API may accept a client credential. You do not give every tool the same password and then call the environment secure.

Microservices formalize that instinct. Each service must have a stable identity, and other services must be able to verify it. A service identity is not a human identity. It is the identity of the software component that is making the call or processing the message.

Caller	Human identity?	Service identity?	Why both may matter
Operator Console through BFF	Yes - the logged-in operator.	Yes - the BFF client or service.	RootCause needs to know the user request and the trusted client path.
RootCause process manager	Usually no direct user.	Yes - RootCauseProcessManager.	Compliance must know a system workflow is requesting review.

Caller	Human identity?	Service identity?	Why both may matter
Reporting projection worker	No.	Yes - ReportingProjectionWorker.	The worker may update projections but should not issue verdicts.
Audit append endpoint caller	Sometimes, if a user action caused the evidence.	Yes - the publishing service.	Audit must record both provenance and the service that appended the record.

Why internal is not an identity

A common early microservice shortcut is to say: this endpoint is internal, therefore it is trusted. That sentence is attractive because it is easy. It is also incomplete. Network position can reduce exposure, but it is not identity. A misconfigured service, compromised worker, overly broad firewall rule, leaked credential, or mistaken deployment can all turn internal trust into accidental permission.

For NEXUS-1, this matters because the most sensitive flows are not always started by a browser. RootCause may request compliance review. Audit may receive append-only evidence. Reporting may consume verdict events. These actions should not be allowed merely because the caller lives in the same cluster, same VNet, or same machine.

DESIGN RULE

A service boundary should authenticate the calling service explicitly. Network location may be one control, but it should not be the proof of identity.

The NEXUS-1 trust path

The alarm-to-compliance flow gives a useful trust path. At the beginning, a human operator may be present. In the middle, a domain service makes decisions. At the end, compliance and audit may react asynchronously. Identity therefore changes shape as the flow moves.

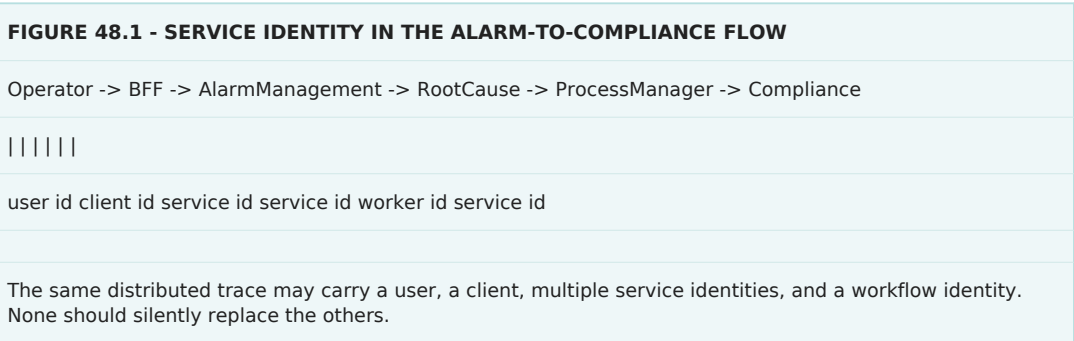


Figure 48.1 - The system does not have one identity. It has a chain of identities that must remain visible and auditable.

Boundary crossing	Caller proves	Receiver checks
BFF -> AlarmManagement	BFF client identity plus operator context.	Route policy, user access, command permission, unit scope.
AlarmManagement -> RootCause	AlarmManagement service identity or signed event publisher.	Allowed publisher, event contract, replay and idempotency policy.

Boundary crossing	Caller proves	Receiver checks
RootCause -> Compliance	RootCause service or process-manager identity.	Whether RootCause may request review for this verdict event.
Any service -> Audit	Publishing service identity plus trace metadata.	Append permission, provenance completeness, correlation and causation IDs.
Projection worker -> Reporting store	Projection worker identity.	Projection write permission for this read model only.

Common mechanisms, architectural meaning

This book is not tied to a single cloud platform or identity provider. The important architectural question is not whether the implementation uses mutual TLS, OAuth client credentials, managed identity, signed messages, SPIFFE/SPIRE, or another mechanism. The important question is what each mechanism proves and what it does not prove.

Mechanism	What it can prove	What it should not be used to pretend
Mutual TLS	The caller presents a trusted certificate and proves possession of its private key.	That the caller is authorized for every domain action.
OAuth client credentials	A registered client or service obtained a token for a service-to-service call.	That the represented human user approved the action.
Managed identity	The cloud/runtime identity of the deployed workload.	That the workload may use every API or every data store.
Signed message envelope	A publisher signed a message, and the payload has not been altered.	That every consumer should trust the message as a command.
Network policy	Traffic came from an allowed network path.	That the caller is who it claims to be.

NEXUS-1 SENTENCE TO REMEMBER

Service-to-service authentication proves the software caller. Domain authorization still decides the allowed action. Audit records both.

Token forwarding vs token exchange

When a user-driven request crosses services, teams often forward the original user token. That can work in simple cases, but it becomes risky when every downstream service receives a token minted for the edge or for a different audience. The receiver may accept a token that was never intended for it. Worse, the token may carry too much information, making every service dependent on user-identity details it should not know.

A cleaner approach is often token exchange or a delegated service token: the BFF authenticates the user, then obtains or sends a token intended for the downstream service, carrying only the necessary claims and an explicit delegation context. The downstream service sees the calling service and the represented user separately.

Approach	Benefit	Cost or risk
Forward original user token	Simple and familiar; useful for early systems or same-audience APIs.	Token audience may be wrong; downstream services may learn too much; hard to separate user from service.
Token exchange / delegated token	Receiver gets a token intended for it, with explicit delegation and reduced claims.	Requires identity-provider support and careful design.
Pure service token with ActorContext payload	Useful for workflows where the service acts on a recorded event or command.	Must protect the actor context from tampering and preserve audit provenance.
No user context, service only	Appropriate for scheduled projection jobs and system maintenance tasks.	Must be recorded honestly as system action, not disguised as a user.

A service identity model in C#

A service boundary should translate infrastructure details into a small model the application layer can use. The domain model should not parse certificates or JWTs. But the application layer may need to know which service called, whether the call was user-delegated, and what trace connects it to the original action.

```
public sealed record ServiceIdentity(
    string ServiceName,
    string ClientId,
    IReadOnlySet<string> ServiceScopes,
    string AuthenticationMethod);

public sealed record DelegatedActor(
    string? UserId,
    string? DisplayName,
    IReadOnlySet<string> Permissions);

public sealed record ServiceCallContext(
    ServiceIdentity Caller,
    DelegatedActor? Actor,
    string CorrelationId,
    string? CausationId);

public interface IServiceTrustPolicy
{
    Task EnsureCallerIsTrustedAsync(
        ServiceCallContext context,
        string operation,
        CancellationToken cancellationToken);
}
```

The important part is the separation. `ServiceIdentity` answers which software caller crossed the boundary. `DelegatedActor` records the represented human when one exists. `ServiceCallContext` connects the action to the distributed trace. `IServiceTrustPolicy` decides whether this caller is trusted for this operation before the use case proceeds.

Service authentication in message-based flows

Synchronous API calls are not the only place identity matters. A message broker can hide the caller if the system treats every message in a queue as equally trustworthy. For NEXUS-1, event consumers should know which service published the event and whether the

event contract version is accepted.

Message concern	What to record or verify
Publisher identity	Which service published the event: AlarmManagement, RootCause, Compliance, or another publisher.
Contract version	The exact public event contract: RootCauseVerdictIssued.v1, AlarmFloodDetected.v1, and so on.
Signature or broker identity	Whether the envelope or broker metadata proves publisher identity sufficiently for the trust model.
Correlation and causation	Which request, command, or previous event caused this message.
Consumer authority	Whether this consumer is allowed to react to this event in this way.

HONEST BOUNDARY

This chapter does not choose a certificate authority, OAuth server, cloud identity provider, service mesh, or broker authentication mode. Those are implementation decisions. The architectural requirement is stable: every service boundary must know which service is calling and must not confuse that fact with domain permission.

The minimum rule set for Chapter 48

A NEXUS-1 microservice design should not move beyond this point until it can answer these questions for every service-to-service path:

Question	Required answer
Which service is calling?	A stable service identity, not only an IP address or internal network assumption.
How was the service authenticated?	mTLS, client credential, managed identity, signed envelope, or another explicit mechanism.
Is a human actor represented?	If yes, the actor is recorded separately from the service identity. If no, the action is marked system-initiated.
What is the intended receiver or audience?	The credential or token is scoped to the service that receives it.
What operation is allowed?	A service scope or trust policy names the allowed operation, not a blanket internal permission.
How is the decision audited?	Correlation ID, causation ID, service identity, actor identity, and policy decision are recorded.

The next chapter moves from the identity of services to the authorization language attached to people and actions: claims, roles, and policy boundaries. That is where the system decides how much permission data should cross a service boundary and how much should remain local to the owning context.

Chapter 48 checkpoint

CHECKPOINT

1. Why is internal network location not enough to prove service identity?
2. What is the difference between a user identity and a service identity?
3. Why can a service-to-service token prove the caller but not the domain permission?
4. Why should delegated user context be separated from service identity?
5. What must Audit record when a system workflow acts without a human user?

PART VIII - SECURITY, IDENTITY, AND ACCESS BOUNDARIES

49. Claims, Roles, and Policy Boundaries

A claim is not a permission everywhere. A role is not a universal truth. A policy belongs to a boundary, and a service that forgets that becomes either too permissive or impossible to change.

The previous chapter proved that services need identities of their own. RootCauseService, ComplianceService, AuditService, and the Operator Console BFF must be recognizable as software callers, not merely as traffic from an internal network. But authentication still leaves a second question open: once we know who the user is, and which service is calling, who is allowed to do what?

This chapter is about that second question. It separates claims, roles, and policies in the same spirit that the book separated domain events from integration events and recommendations from commands. The words are familiar, but in microservices their boundaries matter much more than they do in a single application.

PLAIN EXPLANATION

A claim is a statement about an actor. A role is a coarse membership label. A policy is a rule that decides whether a specific action is allowed inside a specific boundary. In a microservice architecture, policies must not be replaced by raw roles flowing everywhere.

Why the naive solution fails

The naive solution is simple: put every role and permission in the token, forward that token to every service, and let each service check whatever it needs. It works for a small system because there is only one meaning of Operator, Reviewer, Administrator, and Auditor. Once the system is split, that shared meaning starts to crack.

In NEXUS-1, the word operator may mean a user allowed to view plant status, acknowledge an alarm, start a drill, or inspect a root-cause case. It does not automatically mean the same person may issue a root-cause verdict, request a compliance review, append audit evidence, or change service configuration. A role that is too broad becomes a master key. A role that is too detailed becomes an unmaintainable permission catalogue.

Naive shortcut	Why it feels convenient	What breaks in services
Forward every role everywhere	Every service sees the same user labels.	Roles become global vocabulary; local service meaning is lost.
Make one central permission list	One place appears to control everything.	Domain-specific decisions move away from the service that owns the rule.
Let the gateway authorize everything	Backend services look simpler.	The gateway must understand private domain rules it should not own.

Naive shortcut	Why it feels convenient	What breaks in services
Trust any authenticated service	Internal calls become easy.	A trusted service can become an accidental superuser.

Claims, roles, and policies, separated

A claim should be treated as evidence for an authorization decision, not as the decision itself. It may say the user id, organization, site, shift, assurance level, service client id, or scope. A role says the user belongs to a coarse group. A policy uses those facts, plus local service state, to decide whether a command or query may proceed.

Concept	Example in NEXUS-1	Boundary rule
Claim	UserId=U-1042, Site=EDESSA-DEMO, UnitScope=U1, Assurance=interactive-login.	Claims describe the actor or call; they do not by themselves execute a domain action.
Role	Operator, Investigator, ComplianceReviewer, Auditor, ServiceWorker.	Roles are useful shortcuts, but their meaning must be interpreted inside a boundary.
Policy	CanOpenRootCauseCase, CanIssueVerdict, CanRequestComplianceReview, CanAppendAuditEvidence.	Policies name allowed actions in the language of the receiving service.
Domain rule	A RootCauseCase cannot close without evidence.	Domain invariants still run after authorization succeeds. Permission is not correctness.

NEXUS-1 SENTENCE TO REMEMBER

Authentication asks who is calling. Authorization asks what this caller may do here. The domain still asks whether the action is valid.

Policy belongs close to the action

A policy boundary is the place where a service translates general identity facts into a local permission decision. The RootCause service should own the meaning of CanIssueVerdict. The Compliance service should own the meaning of CanOpenComplianceReview. Audit should own the meaning of CanAppendAuditEvidence. Those policies may all use shared claims, but they should not be replaced by a global role string.

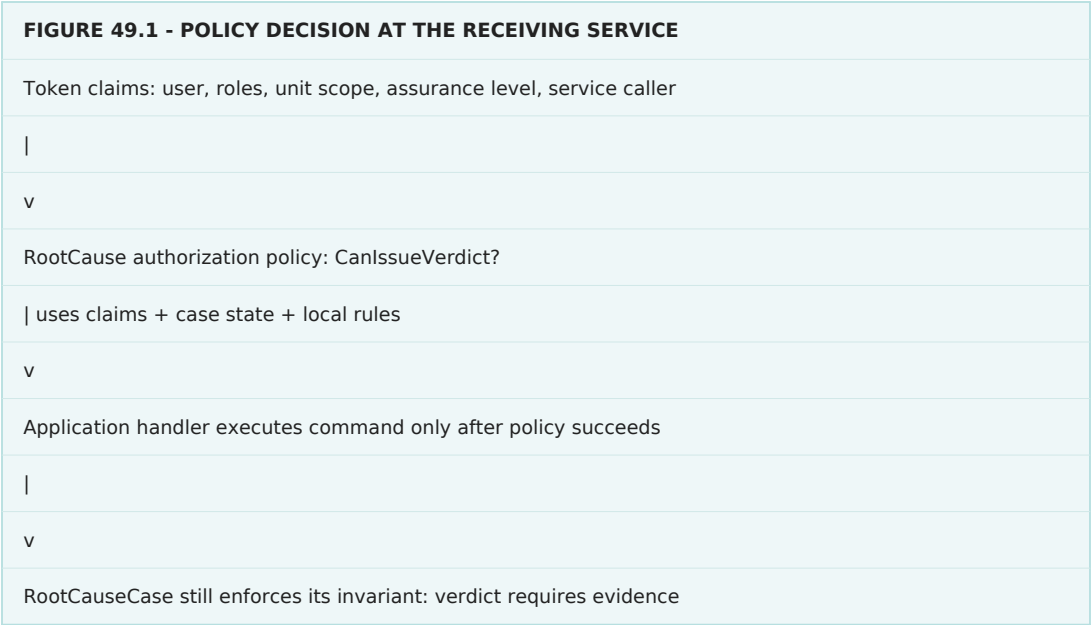


Figure 49.1 - A token supplies facts. The receiving service owns the policy decision. The domain model still protects its invariant.

The difference between permission and validity

This distinction is important enough to repeat. Authorization can say a reviewer is allowed to attempt a verdict. It cannot make an empty case valid. Authorization can say RootCauseService may request compliance review. It cannot make Compliance accept a malformed request or skip its own review-opening policy. Permission opens the door; domain rules still decide whether the action can stand.

Attempted action	Authorization question	Domain or application validity question
OpenRootCauseCase	May this actor open a case for this unit or alarm flood?	Does the alarm flood exist, and is there already an open case?
AttachEvidence	May this actor add evidence to this case?	Is the case still open, and is the evidence grounded?
IssueVerdict	May this actor issue a verdict?	Does the case contain enough evidence and a promoted hypothesis?
RequestComplianceReview	May this service or actor request review?	Is the verdict public, versioned, and review-worthy?
AppendAuditEvidence	May this publisher append to Audit?	Is the evidence envelope complete, immutable, and correlated?

A small .NET implementation shape

The application layer should not spread raw ClaimsPrincipal parsing through every handler. A boundary should translate the incoming principal and service call context into a small authorization input. Then policies can be named in the language of the service.

```

public sealed record ActorContext(
    string ActorId,
    IReadOnlySet<string> Roles,
    IReadOnlySet<string> Permissions,
    IReadOnlySet<string> UnitScopes,
    string AuthenticationMethod,
    string? DelegatedByService);

public sealed record AuthorizationRequest(
    ActorContext Actor,
    string PolicyName,
    string ResourceId,
    string CorrelationId);

public interface IBoundaryAuthorization
{
    Task<AuthorizationResult> AuthorizeAsync(
        AuthorizationRequest request,
        CancellationToken cancellationToken);
}

public static class RootCausePolicies
{
    public const string OpenCase = "RootCause.OpenCase";
    public const string AttachEvidence = "RootCause.AttachEvidence";
    public const string IssueVerdict = "RootCause.IssueVerdict";
}

```

This shape does not dictate an identity provider. It simply prevents the handler from confusing a role with a policy, a service identity with a human actor, or a token claim with a domain invariant.

Policy evaluation in a command handler

The handler below demonstrates the intended sequence. First, authorize the attempt. Second, load the aggregate. Third, let the aggregate enforce its own rule. If authorization fails, the command does not reach the model. If the domain rule fails, the action is refused even for an authorized actor.

```

public sealed class IssueVerdictHandler
{
    private readonly IBoundaryAuthorization _authorization;
    private readonly IRootCauseCaseRepository _cases;

    public async Task<Result> Handle(
        IssueVerdictCommand command,
        ActorContext actor,
        CancellationToken ct)
    {
        var allowed = await _authorization.AuthorizeAsync(
            new AuthorizationRequest(
                actor,
                RootCausePolicies.IssueVerdict,
                command.CaseId,
                command.CorrelationId), ct);

        if (!allowed.Succeeded)
            return Result.Failure(SecurityErrors.Forbidden);

        var rootCauseCase = await _cases.GetAsync(command.CaseId, ct);
        return rootCauseCase.IssueVerdict(command.Verdict, command.IssuedAtUtc);
    }
}

```


DESIGN RULE

Authorization is a gate before the use case. Domain invariants are still the law inside the model. Do not weaken one because the other exists.

Local roles, public claims, and service contracts

The hardest part is deciding what may cross a service boundary. A role name often belongs to an organization or UI experience. A receiving service may not understand it, or worse, may understand it differently. Public contracts should therefore prefer stable, minimal facts: actor id, service caller, unit scope, delegation reason, assurance level, and explicit operation scope when needed.

Should this cross a boundary?	Usually safe?	Reason
User id or subject id	Yes	Needed for audit and actor attribution.
Service caller id	Yes	Needed to distinguish BFF, workflow worker, projection worker, and publisher.
Unit/site scope	Yes, if minimized	Useful for local authorization without exposing the full organization model.
Raw UI role names	Careful	The meaning of Operator or Reviewer may differ by service and over time.
Every permission the user has	No	Over-shares security structure and creates coupling.
Local domain state	No	The receiving service should load its own state or consume a stable event contract.

NEXUS-1 policy boundary map

The following map is a first security-language sketch. It is intentionally conservative. It avoids claiming that one global role controls all service behavior.

Service boundary	Local policies	Must not outsource
AlarmManagement	AcknowledgeAlarm, ShelveAlarm, DetectFlood, PublishAlarmFloodDetected.	Whether an alarm state transition is valid.
RootCause	OpenCase, AttachEvidence, PromoteHypothesis, IssueVerdict.	Whether evidence is sufficient or a verdict can stand.
Compliance	OpenReview, AssignReviewer, RecordFinding, CloseReview.	Whether a finding is valid under compliance workflow.
Audit	AppendEvidence, SealAuditBatch, ReadAuditTrail.	Append-only integrity and provenance checks.
Reporting	ReadProjection, ExportReport, RefreshProjection.	Source-of-truth decisions from write-side services.

Avoiding the distributed admin trap

One of the most dangerous shortcuts is to create a global Administrator role that every service accepts as permission to do everything. It is useful during early development and

harmful as soon as the system becomes serious. In a distributed architecture, even administrators should cross boundaries through named policies. Administrative power must be explainable, scoped, and auditable.

HONEST BOUNDARY

This chapter does not implement a full identity system, role-management UI, policy engine, or OpenID Connect provider. It defines the architectural separation that those tools must respect: claims are facts, roles are coarse labels, policies are boundary-local decisions, and domain invariants remain inside the model.

The minimum rule set for Chapter 49

Rule	Meaning
Do not treat roles as universal language.	A role must be interpreted in the service that owns the action.
Do not put all permissions in every token.	Tokens should carry only what the receiver needs.
Name policies in service language.	RootCause.IssueVerdict is clearer than generic CanWrite.
Separate service caller from human actor.	A process manager may act without a current human session.
Audit authorization decisions.	Record actor, service caller, policy, resource, outcome, correlation, and reason.
Let the domain still refuse invalid actions.	Authorized does not mean valid.

The next chapter moves to the material that makes those identities and policies operational: secrets, certificates, and configuration. If those are treated casually, the cleanest authorization model becomes only text on a page.

Chapter 49 checkpoint

CHECKPOINT

- 1. Explain the difference between a claim, a role, and a policy.
- 2. Why should a service not blindly accept a global Operator role as permission for every action?
- 3. Where should the policy CanIssueVerdict be interpreted in NEXUS-1?
- 4. Why does authorization not replace the RootCauseCase invariant about evidence?
- 5. What security facts should Audit record when a policy decision allows or refuses an action?

PART VIII - SECURITY, IDENTITY, AND ACCESS BOUNDARIES

50. Secrets, Certificates, and Configuration

A secret is not a setting. A certificate is not a decoration. Configuration is not harmless text. In a microservice architecture, these small operational facts become part of the security boundary itself.

The previous chapter separated claims, roles, and policies so that NEXUS-1 services do not confuse identity facts with permission decisions. That separation is necessary, but it is not enough. A service also has to protect the material that lets it prove itself, call other services, decrypt protected data, sign tokens, connect to databases, and trust messages.

That material is often hidden in the most ordinary-looking places: appsettings files, environment variables, local developer secrets, certificate stores, deployment manifests, and cloud key vaults. Because it looks like configuration, teams are tempted to treat it casually. In a monolith, that may stay survivable for a while. In microservices, it becomes a system-wide failure path.

PLAIN EXPLANATION

Configuration tells a service how it should behave. A secret lets a service prove or access something. A certificate binds identity to cryptographic trust. They may be loaded through the same startup pipeline, but they must not be governed as if they were the same kind of thing.

The naive solution

The naive solution is to put everything into one configuration system and call it done. Connection strings, broker passwords, JWT signing keys, certificate thumbprints, token audiences, feature flags, retry counts, endpoint URLs, and service names all become key-value pairs. The application starts, the tests pass, and the system appears disciplined because all values have names.

The problem is that names alone do not create ownership or protection. A retry count can be visible to many people. A signing key cannot. A feature flag can be changed by a release engineer. A private certificate key cannot be copied into a build log. A service endpoint may be documented. A database password must be rotated and audited. Treating all of them as equivalent configuration destroys the difference between behavior and trust.

Item	Looks like configuration?	Actually governed as	Why it matters
API endpoint URL	Yes	Operational setting	Wrong value breaks routing but does not usually expose trust by itself.
Retry count	Yes	Runtime behavior setting	Can be tuned and reviewed like normal service configuration.

Item	Looks like configuration?	Actually governed as	Why it matters
Database password	Yes	Secret	Exposure grants access to source-of-truth data.
JWT signing key	Yes	Cryptographic secret	Exposure allows forged identity or service claims.
TLS private key	Yes	Certificate private material	Exposure lets another process impersonate the service.
Feature flag	Yes	Controlled product setting	Changing it may alter behavior but should not reveal credentials.

Three different responsibilities

A serious service separates three responsibilities before code is written. First, configuration values that describe behavior. Second, secrets that must be hidden, rotated, and accessed only by authorized workloads. Third, certificates or keys that establish cryptographic trust between services and infrastructure.

Responsibility	Examples	Who should own it	Normal change pattern
Configuration	Timeouts, base URLs, feature switches, queue names, projection lag thresholds.	Service team plus operations.	Change through deployment or controlled runtime configuration.
Secrets	Passwords, API keys, broker credentials, encryption keys.	Security/operations with service-specific access.	Rotate, version, audit access, never print.
Certificates	mTLS client certificate, signing certificate, issuer certificate chain.	Platform/security with service owners.	Renew before expiry, validate chain, support overlap.
Derived policy inputs	Allowed audiences, trusted issuers, required scopes.	Security architecture plus receiving service.	Change deliberately because it changes trust.

NEXUS-1 SENTENCE TO REMEMBER

A service may read configuration to decide how to behave, but it uses secrets and certificates to prove what it is allowed to access or whom it can trust.

NEXUS-1 examples

In the NEXUS-1 service architecture, the difference becomes concrete. RootCauseService needs configuration for its event contract version, queue name, timeouts, and outbox publisher cadence. It needs secrets to connect to its own database and publish to the broker. It may need a certificate or client assertion to prove itself when it calls ComplianceService or writes to Audit. These are not one concern.

Service	Configuration examples	Secret examples	Certificate or trust examples
RootCause	Outbox batch size, broker topic, verdict event version.	RootCause DB credential, broker publisher credential.	Client identity for Audit and Compliance calls.
Compliance	Review SLA, allowed event versions, projection refresh interval.	Compliance DB credential, workflow worker secret.	Trusted issuer for RootCauseVerdictIssued.v1 messages.
Audit	Batch sealing cadence, retention class, read API limits.	Audit DB credential, hash-chain signing key.	Sealing/signing certificate and trusted service callers.
Reporting	Projection lag warning threshold, export size limits.	Read-store credential, export storage key.	Certificate for secure export endpoint.
Operator Console BFF	Backend service URLs, cache duration, UX lag thresholds.	BFF client secret if confidential client is used.	Token validation issuer and service-call credential.

Configuration that belongs to the service

Configuration should be local to the service that uses it. RootCauseService may decide how many outbox messages it publishes per batch. Reporting may decide how stale a projection may be before the UI shows a delayed-truth banner. Compliance may decide which review escalation thresholds apply. These are service-owned behavior settings. They should not be hidden as global platform constants.

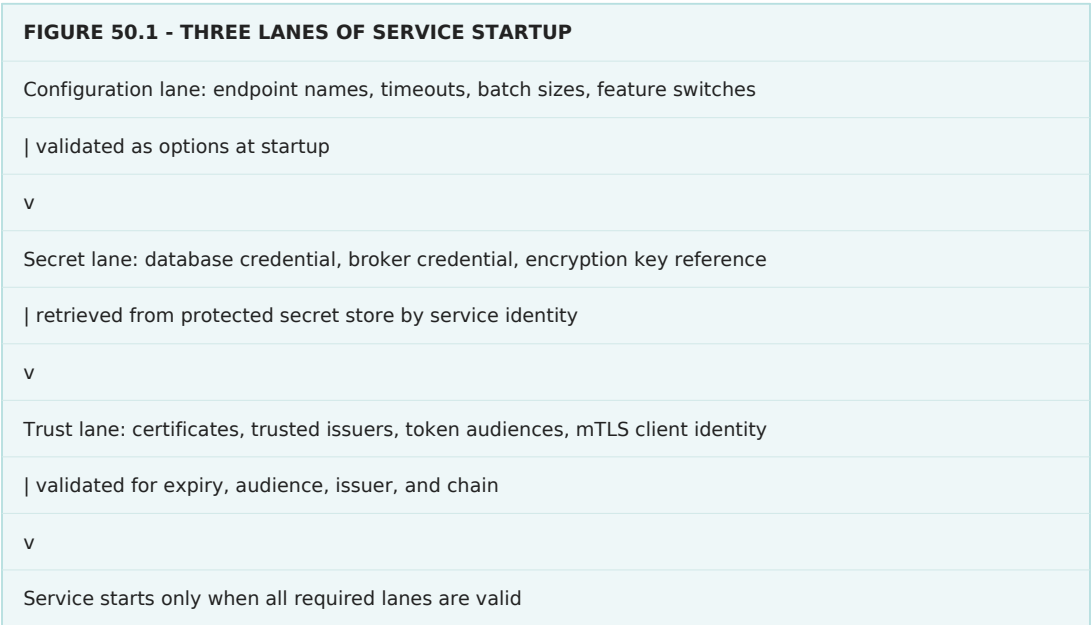


Figure 50.1 - Configuration, secrets, and trust material may be loaded during startup, but they are validated by different rules.

A small .NET implementation shape

The following shape keeps behavior options separate from secrets. Options are validated at startup. Secrets are represented by references, not by raw secret values in the strongly typed configuration object. That distinction prevents accidental logging and makes rotation a first-class concern.

```
public sealed class RootCauseServiceOptions
{
    public string ServiceName { get; init; } = "RootCauseService";
    public string VerdictEventVersion { get; init; } = "v1";
    public int OutboxBatchSize { get; init; } = 100;
    public TimeSpan PublisherInterval { get; init; } = TimeSpan.FromSeconds(5);
    public Uri ComplianceServiceBaseUrl { get; init; } = null!;
}

public sealed class RootCauseSecretReferences
{
    public string DatabaseCredentialRef { get; init; } = null!;
    public string BrokerPublisherCredentialRef { get; init; } = null!;
    public string AuditSigningKeyRef { get; init; } = null!;
}

public interface ISecretProvider
{
    Task<SecretValue> GetAsync(string reference, CancellationToken ct);
}
```

The important point is not the exact class names. The point is that configuration objects do not become convenient buckets for secrets. A reference may appear in configuration; the secret value itself should be retrieved by a provider that can enforce access, auditing, and rotation rules.

Certificates and trust are not optional decoration

In a service-to-service architecture, a certificate or signing key is part of the service identity. If RootCauseService presents a client certificate to ComplianceService, the receiving service does not merely see traffic; it sees a cryptographic claim that this caller is the RootCause service. If that key is copied, expired, logged, or accepted without validation, the trust boundary becomes fictional.

Certificate concern	Question to ask	Failure if ignored
Expiry	Will the service refuse to start or warn before the certificate expires?	A valid service suddenly cannot call dependencies.
Private key protection	Who can read the private key material?	Another process can impersonate the service.
Chain validation	Which issuer is trusted and for what purpose?	A service accepts an identity from the wrong trust root.
Audience/purpose	Is this certificate allowed for this call or token use?	A certificate for one purpose becomes a universal key.
Rotation overlap	Can old and new certificates coexist during rollout?	Zero-downtime rotation becomes impossible.

DESIGN RULE

A certificate should be treated as identity material with lifecycle, ownership, expiry, and audit requirements. Do not reduce it to a thumbprint string pasted into a settings file.

Rotation is part of the design, not a later task

A system that can use a secret but cannot rotate it safely is not finished. Rotation affects more than storage. It affects how services reload values, how producers and consumers overlap credentials, how message signatures are verified during transition, how failed calls are diagnosed, and how long old keys remain valid.

Material	Rotation strategy	NEXUS-1 example
Database credential	Create new credential, grant least required access, deploy service using new reference, revoke old credential after verification.	RootCause DB writer credential changes without giving Reporting write access.
Broker credential	Allow publisher to use new credential while consumers continue reading existing messages.	RootCause publishes RootCauseVerdictIssued.v1 without interrupting Compliance.
Signing key	Publish key id, accept old and new keys during overlap, retire old key after message lifetime expires.	Audit can verify both old and new event signatures during transition.
mTLS certificate	Install new certificate, update trust bundle if needed, deploy, monitor, then remove old certificate.	Compliance accepts RootCauseService old and new cert during rollout.

Configuration drift and the distributed system

Configuration drift is the state where services that should agree no longer agree. In NEXUS-1, this can happen if RootCause publishes v1 events while Compliance expects v2 only, if Reporting reads from a stale topic name, or if Audit trusts an issuer that another service has already retired. Drift is often less dramatic than an outage, but more dangerous because it may look like the system is still working.

Drift type	Symptom	Architectural response
Contract version drift	Consumer rejects messages that producer believes are valid.	Publish compatibility window; document accepted event versions.
Endpoint drift	Service calls old URL or wrong environment.	Central discovery or environment-scoped settings with validation.
Trust drift	One service trusts an issuer another has retired.	Managed trust bundles, rotation windows, and startup checks.
Policy drift	Gateway allows a call the service refuses, or the reverse.	Receiving service remains final policy authority.
Projection configuration drift	UI displays different lag expectations per screen.	Explicit projection metadata and shared UX language.

HONEST BOUNDARY

This chapter does not prescribe a specific vault, certificate authority, service mesh, or cloud platform. The architecture is tool-agnostic: protect secret values, validate configuration, treat certificates as identity material, support rotation, and make trust decisions explicit.

Startup validation checklist

A service should fail early when its required configuration, secret references, or trust settings are invalid. Starting half-configured is worse than not starting at all because the failure may surface later as lost messages, rejected calls, or inconsistent projections.

Startup check	Question
Configuration schema	Are all required behavior settings present and within valid ranges?
Secret references	Do secret references exist, and can the service identity read only the ones it needs?
Certificate expiry	Are certificates valid now, and are warnings raised before expiry?
Trusted issuers	Does the service trust only the issuers required for its boundary?
Audience and scopes	Are accepted audiences and scopes explicit, not wildcarded?
Contract versions	Are produced and consumed event/API versions compatible?
Logging safety	Are secret values redacted from logs, exceptions, health checks, and diagnostics?

Health checks should not leak secrets

Health checks are useful, but they are also common leakage points. A health endpoint may reveal which vault is used, which certificate is missing, which database account failed, or which secret name could not be read. That information may be helpful to an operator and too helpful to an attacker. The external result should be safe; the internal diagnostic should be protected.

```
public sealed record DependencyHealth(
    string Name,
    HealthStatus Status,
    string PublicMessage,
    string? ProtectedDiagnosticCode);

// Public: "Audit signing material unavailable."
// Protected diagnostics: "SecretRef AuditSigningKeyRef not readable by RootCauseService identity"
```

The same discipline applies to logs. Logging that a certificate will expire soon is helpful. Logging the private key path, raw thumbprint list, bearer token, or secret value is not. The diagnostic trail must support operations without turning the log store into a second secret store.

NEXUS-1 minimum boundary map

Boundary	Configuration owned locally	Secret/trust material protected separately
RootCause	Outbox cadence, verdict event version, Compliance endpoint.	DB writer credential, broker publisher credential, Audit signing key, client certificate.
Compliance	Review SLA, accepted verdict versions, escalation timers.	DB credential, service client credential, trusted RootCause publisher material.
Audit	Batch size, retention class, read limits.	Audit DB credential, sealing key, trust bundle for allowed appenders.
Reporting	Projection lag threshold, export limits, refresh interval.	Read-store credential, export storage secret, trusted data-source callers.
Operator Console BFF	UX cache rules, downstream base URLs, feature gates.	Confidential client credential, token validation settings, service-call certificate.

The minimum rule set for Chapter 50

Rule	Meaning
Do not store secret values as ordinary settings.	A settings file may hold a reference, not the protected value itself.
Validate configuration at startup.	A broken setting should fail early, before partial behavior appears.
Treat certificates as service identity material.	They need ownership, expiry monitoring, rotation, and audit.
Design rotation before production.	Every secret or certificate should have a replacement path.
Keep health checks safe.	Public status should not expose credential names, tokens, or internal trust details.
Do not let configuration drift silently.	Contract versions, issuers, endpoints, and scopes must remain observable.

The next chapter narrows the same security discipline to authorization between services: least privilege. Once the credentials exist, the important question becomes how little each service is allowed to do with them.

Chapter 50 checkpoint

CHECKPOINT

- 1. Explain why a secret is not the same kind of thing as a normal configuration value.
- 2. Why should configuration hold secret references rather than raw secret values?
- 3. What can go wrong if certificates are treated as simple thumbprint strings?
- 4. Why must rotation be designed before production, not after the first expiry?
- 5. Name three NEXUS-1 settings that are safe behavior configuration and three values that must be treated as protected secret or trust material.

PART VIII - SECURITY, IDENTITY, AND ACCESS BOUNDARIES

51. Least Privilege Between Services

A service identity should never mean service omnipotence. In a distributed system, the safest permission is the one a service never receives.

The previous chapter separated settings, secrets, certificates, and trust material. That gave every service a disciplined way to prove who it is and to protect the material that lets it call databases, brokers, APIs, and other services. But identity by itself is still not permission.

Least privilege is the next security boundary. It says that every service, every worker, every database login, every broker credential, and every service-to-service client should receive only the permissions required for its specific job - no more, no earlier than needed, and no longer than needed. That sounds simple until a system is under pressure. Then teams create one powerful credential, share it between services, and call the resulting convenience architecture.

PLAIN EXPLANATION

Least privilege means that RootCauseService should be able to do RootCause work, not every kind of work. It may publish a verdict event; it should not be able to rewrite Compliance reviews, alter Reporting projections directly, delete Audit entries, or read every table because all services happen to live in the same platform.

The convenience trap

The convenience trap begins with a reasonable sentence: this service needs to reach that dependency. Then the permission is made broader than the use case. A database login receives db_owner because migrations were easier during development. A broker credential can publish to every topic because it avoided one naming mistake. A service token receives an admin scope because one endpoint was temporarily blocked. Months later, the temporary shortcut has become the production trust model.

Microservices amplify this mistake because there are more identities and more paths. In a modular monolith, over-permission may be hidden inside one process. In a distributed system, over-permission becomes a lateral movement path: once one service identity is compromised, the attacker or bug can use that identity to reach things the service never needed.

Convenient permission	Why it feels harmless	What it breaks
One shared database login for all services	All services can connect immediately.	Data ownership, audit attribution, blast-radius control.
Broker publisher can write all topics	No topic-specific credential management.	Event contract ownership and producer accountability.

Convenient permission	Why it feels harmless	What it breaks
Service token has platform.admin	Every internal call works.	Authorization boundaries and incident containment.
Reporting can query operational stores directly	Dashboards are easy to build.	Source-of-truth boundaries and delayed-truth honesty.
Audit writer can update or delete evidence rows	Maintenance scripts are convenient.	Tamper resistance and accountability.

Least privilege in NEXUS-1 terms

NEXUS-1 already has the language needed to apply least privilege. AlarmManagement owns alarm lifecycle facts. RootCause owns diagnostic cases, hypotheses, evidence, and verdicts. Compliance owns review workflow. Audit owns evidence of what happened. Reporting owns projections. The permission model should follow those ownership lines instead of flattening them into one internal super-user.

NEXUS-1 SENTENCE TO REMEMBER

A service may be trusted to publish the facts it owns; it should not be trusted to mutate the facts owned by every downstream consumer.

Service identity	Allowed examples	Not allowed examples
AlarmManagementService	Create AlarmEvent; publish AlarmFloodDetected.v1; read own alarm definitions.	Open RootCause cases; close Compliance reviews; write Reporting projections directly.
RootCauseService	Open and update RootCauseCase; publish RootCauseVerdictIssued.v1; append RootCause outbox rows.	Delete Audit entries; change Compliance findings; publish arbitrary event types.
ComplianceService	Create and update compliance reviews; consume verdict events idempotently; publish review status facts.	Rewrite RootCause verdicts; edit AlarmEvent history; impersonate Audit writers.
AuditService	Append audit evidence; seal batches; expose protected read APIs.	Drive business workflow; delete or rewrite old audit evidence.
ReportingService	Read approved projections and build read models; expose dashboards.	Mutate source-of-truth tables; call command APIs as a shortcut for report repair.
Operator Console BFF	Compose UI views; call allowed APIs on behalf of authenticated users.	Bypass service policy; use its own identity to perform operator-only actions.

Database permissions should reflect data ownership

Database-per-service is not only about schema neatness. It is also a permission boundary. If RootCauseService owns its own database, its application identity can have the minimum rights required to load cases, persist case changes, append outbox rows, and read its own reference data. It does not need write access to Compliance tables or Audit tables. If it needs another service to act, it calls an API or publishes an integration event.

Data store	Service account	Minimum rights
RootCauseDb	rootcause-app	Read/write RootCause tables; append outbox; read local lookups; no schema ownership in runtime.
ComplianceDb	compliance-app	Read/write Compliance reviews; append inbox/outbox; read local policy lookups.
AuditDb	audit-writer	Append audit entries; read own batch metadata; no update/delete for sealed evidence.
ReportingReadStore	reporting-projection-worker	Upsert projection tables; read projection metadata; no source-system writes.
Broker	rootcause-publisher	Publish RootCauseVerdictIssued.v1 only, not every platform event.
Broker	compliance-consumer	Consume assigned verdict topic/queue; dead-letter only to its own DLQ.

DESIGN RULE

Runtime credentials should be weaker than deployment credentials. A service that runs the application should not automatically be able to create schemas, alter tables, grant permissions, or manage every queue.

Command permissions are not the same as read permissions

Least privilege becomes much clearer when commands and queries are separated. A service may need to read a public status from another service without being allowed to command that service. The Operator Console BFF may read RootCause case status for display, but an operator action still has to pass through the RootCause API policy. Reporting may read projection feeds; it should not gain command authority because it builds dashboards.

Interaction	Read permission	Command permission
BFF -> RootCause	Read visible case status for authorized operator.	Close case only when user has RootCause.CloseCase and RootCause policy accepts the state.
Compliance -> RootCause	Read verdict summary and evidence references.	No permission to change verdict. Correction requires RootCause-owned command or compensating event.
Reporting -> Compliance	Read projection feed or approved export contract.	No permission to approve, reject, or escalate a review.
Audit -> Services	Expose evidence for authorized investigation.	No permission to trigger normal business workflow.

Scoped service tokens

A service token should carry the smallest useful claim set. It should identify the calling service, the intended audience, the allowed action or scope, and enough correlation to audit the call. It should not carry broad administrative claims just because the caller is internal.

```
// Example service-to-service policy shape, not a framework mandate.
public static class ServiceScopes
{
    public const string RootCauseVerdictPublish = "rootcause.verdict.publish";
    public const string ComplianceReviewRequest = "compliance.review.request";
    public const string AuditEvidenceAppend = "audit.evidence.append";
    public const string ReportingProjectionWrite = "reporting.projection.write";
}

public sealed record ServicePrincipal(
    string ServiceName,
    string Audience,
    IReadOnlySet<string> Scopes,
    string CorrelationId);

public static bool CanAppendAudit(ServicePrincipal caller) =>
    caller.Audience == "audit-service" &&
    caller.Scopes.Contains(ServiceScopes.AuditEvidenceAppend);
```

This example is deliberately small. Real systems also validate issuer, signature, expiry, key id, tenant or environment, certificate binding, and token replay rules. The lesson is still the same: the receiving service must check a narrow permission, not just the existence of any internal token.

Broker permissions are part of the security model

Messaging systems often become accidental superhighways. A publisher that can write every topic can impersonate any producer. A consumer that can read every queue can observe data it does not own. A dead-letter handler that can republish anywhere can accidentally become a hidden workflow engine. Least privilege applies to the broker as strongly as it applies to HTTP APIs.

FIGURE 51.1 - LEAST PRIVILEGE ON THE EVENT BACKBONE	
RootCauseService identity	
publish allowed: RootCauseVerdictIssued.v1	
publish denied : ComplianceReviewApproved.v1, AuditEntryWritten.v1	
v	
Event broker enforces topic or exchange permissions	
v	
ComplianceService identity	
consume allowed: RootCauseVerdictIssued.v1 subscription	
consume denied : private RootCause internal queue	
v	
AuditService identity	
append allowed through audit append API or approved writer channel	
rewrite denied by storage and service policy	

Figure 51.1 - The event backbone is not trusted just because it is internal. Producers and consumers need narrow permissions.

Least privilege and human access

Service privilege and human privilege must not be confused. A human operator may be allowed to request a root-cause closure. The BFF may be allowed to forward that request. RootCauseService may be allowed to execute it only after validating both the user authority and the case state. None of those permissions automatically implies the others.

Actor	Can do	Cannot do
Operator user	Request actions permitted by role and current case state.	Borrow service credentials or bypass service validation.
Operator Console BFF	Transmit user context and call allowed APIs.	Convert its service identity into a universal operator identity.
RootCauseService	Validate RootCause commands and update RootCause-owned state.	Assume the BFF already authorized everything correctly.

Actor	Can do	Cannot do
Support engineer	Read protected diagnostics under controlled access.	Use production secret material for normal investigation convenience.

Deny by default, allow by reason

A least-privilege model is easier to review when it is expressed as denial by default. A new service starts with no access to databases, no broker topics, no internal APIs, and no privileged scopes. Access is then granted by reason: which use case requires it, which data owner approves it, which endpoint or topic is involved, and how the call is audited.

Grant question	Healthy answer
Which use case requires this permission?	Named use case, not general convenience.
Which service owns the target data or contract?	The owner is explicit and approves the access.
Is the permission read, write, publish, consume, append, or administer?	The verb is narrow; administer is rare.
Can the permission be scoped by topic, table, endpoint, or action?	Yes, and the scope is the default choice.
How is use audited?	Service name, user context if present, correlation id, endpoint/topic, result.
How is the permission removed?	Revocation path exists and is tested.

HONEST BOUNDARY

Least privilege can make development slower if tooling is weak. That is not a reason to remove it. It is a reason to automate local setup, permission templates, environment-specific grants, and reviewable access manifests. Security that depends on manual heroics will be bypassed.

A permission manifest shape

One practical way to make least privilege visible is to keep a service permission manifest. This is not necessarily the deployment tool itself. It is a review artifact that states what the service is expected to access. Platform automation can then translate the manifest into database roles, broker policies, API scopes, and certificate issuance.

```
service: RootCauseService
identity: svc-rootcause
runtime-permissions:
  databases:
    - name: RootCauseDb
      rights: [read-own, write-own, append-outbox]
  broker:
    publish: [RootCauseVerdictIssued.v1, RootCauseVerdictCorrected.v1]
    consume: [AlarmFloodDetected.v1]
  apis:
    call:
      - AuditService.AppendEvidence
      - ComplianceService.RequestReview
  denied:
    - ComplianceDb.write
    - AuditDb.update
    - ReportingReadStore.source-write
    - broker.publish.*
review:
  owner: RootCause team
  approved-by: Security Architecture
  rotation-owner: Platform Operations
```

The manifest makes excess visible. A reviewer can ask why RootCauseService needs wildcard broker publishing, why Reporting needs write access to source stores, or why a runtime identity can alter schemas. Without a manifest, the permissions hide inside deployment scripts and only appear during an incident.

NEXUS-1 least-privilege checklist

Check	Question
Service identity	Does every deployable service have its own identity, not a shared internal account?
Database rights	Can each service write only the data it owns?
Broker rights	Can publishers publish only their own event contracts and consumers read only their subscriptions?
API scopes	Does each receiving service enforce narrow scopes or policies?
Human context	Are user permissions preserved separately from service permissions?
Auditability	Can every privileged action be traced to service identity, user context where relevant, and correlation id?
Revocation	Can a service permission be removed without redesigning the platform?
Development parity	Do local/dev environments preserve the shape of least privilege, even if credentials are different?

The next chapter turns the security discussion into a NEXUS-1 security boundary map. It will gather the service identities, trust directions, access decisions, and protected resources into one architecture view before the book moves into observability and operations.

Chapter 51 checkpoint

CHECKPOINT

1. Why is service identity not enough without narrow authorization?
2. Name three permissions RootCauseService should have and three it should not have.
3. Why should broker permissions be treated as part of the security model?
4. Explain the difference between human authority, BFF authority, and receiving-service authority.
5. What should a permission manifest make visible before deployment?

PART VIII - SECURITY, IDENTITY, AND ACCESS BOUNDARIES

52. NEXUS-1 Security Boundary Map

A security model that cannot be drawn cannot be reviewed. The map is where service identities, trust directions, protected resources, and denial rules become visible.

Part VIII has separated several ideas that are often collapsed into one word: security. Authentication proves the caller. Authorization decides what the caller may do. Service-to-service authentication gives deployable services their own identities. Claims, roles, and policies describe authority without letting authority leak everywhere. Secrets and certificates protect the material that proves identity. Least privilege prevents that identity from becoming omnipotence.

This chapter gathers those ideas into a security boundary map for NEXUS-1. The map is not a certified security architecture and it is not a claim about an operational plant. It is a software architecture artifact for the demonstrator: a way to see who calls whom, what identity is used, which resource is protected, which service owns the decision, and where the system must deny by default.

PLAIN EXPLANATION

A security boundary map is a drawing of trust. It shows where a request crosses from one authority zone into another, which identity travels with it, which policy is checked, and what must not be assumed simply because the call is internal.

Why the map is necessary

A distributed system creates more edges than a modular monolith. The Operator Console does not simply call one application. It may call a BFF. The BFF may call AlarmManagement, RootCause, Compliance, Reporting, and Audit. RootCause may publish events. Compliance may consume them. Reporting may build projections. Audit may append evidence. Every edge is an opportunity for a correct decision or an unreviewed assumption.

Without a map, teams tend to secure endpoints one by one. That feels practical, but it hides the architecture. One endpoint validates a user token. Another trusts a service token. A worker reads a queue. A projection job writes a table. A support API reads audit evidence. Each piece may look reasonable alone, while the whole system quietly develops a path nobody intended.

Unmapped assumption	What the map should force the team to ask
The BFF is internal, so its call is safe.	Which user is it acting for, and which receiving service policy approves the action?

Unmapped assumption	What the map should force the team to ask
RootCause published an event, so Compliance must accept it.	Is the event signed or broker-protected, versioned, idempotent, and from the allowed producer?
Reporting reads many things, so it can read everything.	Which projections are public read models and which stores remain source-of-truth?
Audit is central, so every service can write to it directly.	Is audit append allowed through a narrow API/channel, and are updates/deletes denied?
A service runs in the platform network, so it is trusted.	What identity, audience, scope, certificate, and policy are validated by the receiver?

The NEXUS-1 security zones

The map begins by naming zones. A zone is not only a network segment. It is a place where the same security assumptions hold. The Operator Console is a user-facing zone. The BFF is an edge application zone. Domain services are service zones. Message brokers and databases are infrastructure zones. Audit is a protected evidence zone. Reporting is a read-model zone. Each zone has different rules.

Zone	Contains	Primary security concern
Operator edge	Browser console, user session, UI composition.	Do not confuse displayed capability with approved authority.
BFF/API gateway	Operator-facing API, token translation, request shaping.	Forward user context without becoming a universal admin.
Domain services	AlarmManagement, RootCause, Compliance, Audit, Reporting.	Enforce local policy at the owning service boundary.
Event backbone	Broker topics, queues, subscriptions, DLQs.	Restrict producers and consumers by contract and direction.
Data stores	Service-owned databases, read stores, outbox/inbox tables.	Protect source-of-truth ownership and least-privilege writes.
Evidence zone	Audit append path, sealed audit records, correlation chain.	Append-only evidence; deny workflow authority and tampering.

Security boundary map - first pass

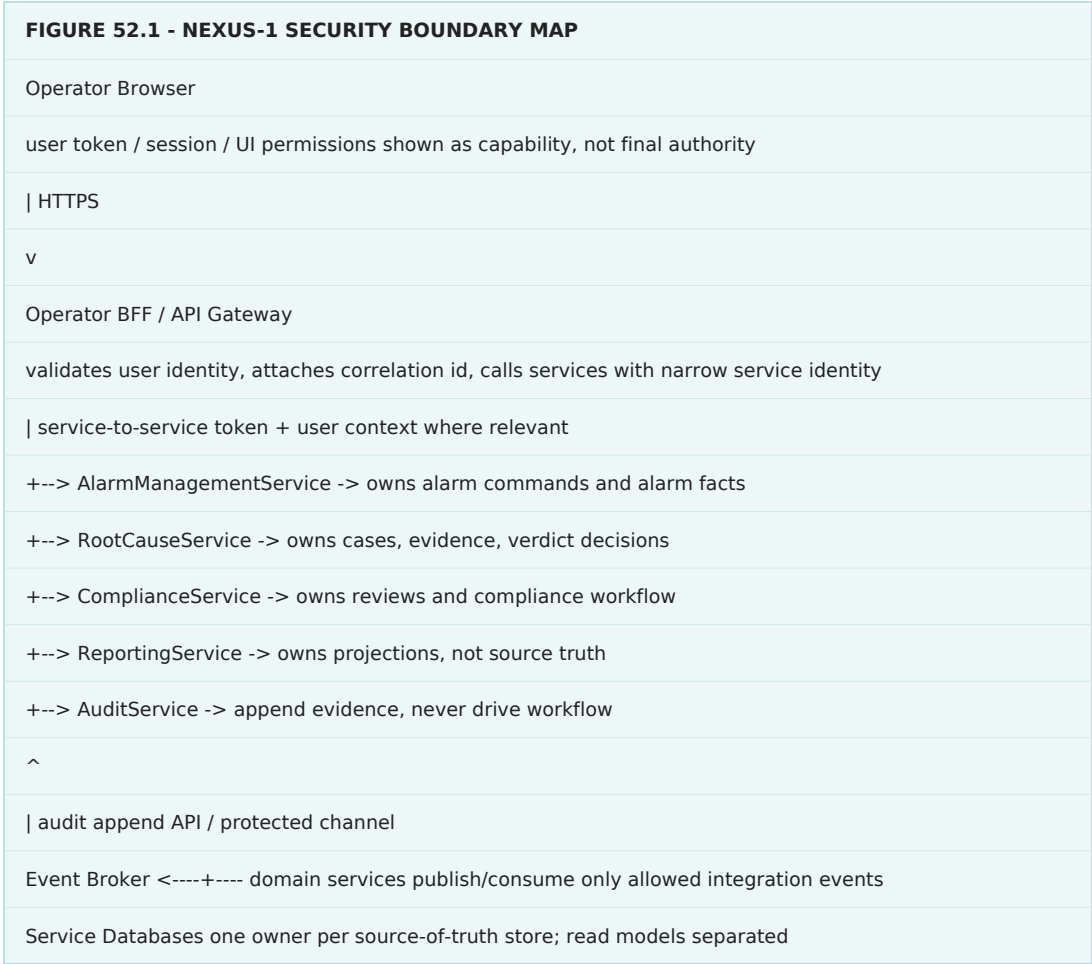


Figure 52.1 - The map is a review artifact. It shows identity flow, policy ownership, event boundaries, and protected stores without pretending that network location equals trust.

Trust directions

The most important arrows in a security map are not call directions; they are trust directions. A call from the BFF to RootCause does not mean RootCause trusts the BFF completely. It means RootCause receives a request, verifies the caller identity, validates the user context when the action is user-driven, checks RootCause policy, and decides locally. The call direction is BFF to RootCause. The decision authority stays in RootCause.

Edge	Identity carried	Decision owner
Browser -> BFF	User identity/session.	BFF validates session and request shape; no domain decision yet.
BFF -> RootCause command API	BFF service identity plus user context.	RootCause authorizes command against case state and user authority.
RootCause -> Broker publish	RootCause publisher identity.	Broker permits only RootCause-owned public contracts.

Edge	Identity carried	Decision owner
Compliance consumer -> Compliance DB	Compliance worker identity.	Compliance owns review state; worker uses narrow database rights.
Any service -> Audit append	Calling service identity and correlation context.	Audit accepts append-only evidence through protected contract.
Reporting -> Read store	Projection worker or reporting API identity.	Reporting writes/reads projections, not source-of-truth stores.

NEXUS-1 SENTENCE TO REMEMBER

The receiving service is the policy owner. The caller may request, prove identity, and provide context; it does not bring permission with it as a gift the receiver must accept.

Boundary decisions by service

A useful map does not stop at boxes and arrows. It states what each service is allowed to decide. This matters because security failures often begin as ownership failures. If Reporting can decide compliance status because it has a convenient projection, the security boundary has already failed. If Audit can trigger workflow because it sees evidence, the evidence boundary has failed. If the BFF can close a root-cause case using its own identity, the user boundary has failed.

Service	May decide	Must not decide
AlarmManagement	Whether an alarm exists, is acknowledged, or forms a flood.	Root cause verdict, compliance review outcome, audit evidence meaning.
RootCause	Whether a case can open, accept evidence, issue or correct a verdict.	Compliance approval, reporting truth, audit record mutation.
Compliance	Whether a review is required, pending, accepted, rejected, escalated.	Original root-cause truth or alarm lifecycle history.
Audit	Whether evidence is accepted into append-only audit trail.	Business workflow decisions that belong to domain services.
Reporting	How to present projections and lag indicators.	Source-of-truth mutation or operational decisions.
BFF	Which UI composition to return and which service call to make.	Domain approval merely because the user can see a button.

Protected resources

The map must name the things being protected. Abstract statements such as secure the platform are too broad to review. NEXUS-1 has concrete resources: RootCauseCase state, verdict events, audit entries, compliance reviews, read models, secrets, certificates, database credentials, broker topics, DLQs, and correlation records. Each protected resource needs an owner and a rule.

Protected resource	Owner	Primary rule
RootCauseCase	RootCauseService	Only RootCause commands can change case state; evidence required before verdict.

Protected resource	Owner	Primary rule
RootCauseVerdictIssued.v1	RootCauseService	Only RootCause publisher identity may publish canonical verdict events.
ComplianceReview	ComplianceService	Only Compliance policy changes review lifecycle; events request review, not approve it.
AuditEntry	AuditService	Append-only path; no normal update/delete authority for sealed records.
ReportingProjection	ReportingService	Projection is read-side truth with lag; never source-of-truth mutation.
Secrets and certificates	Platform/Security operations	Stored outside code; rotated; scoped to service and environment.
Broker DLQ	Owning consumer service	Triage and replay require explicit permission and audit trail.

A map should show denied paths

Good security diagrams show allowed paths. Better ones also show denied paths. The denied paths are where design discipline becomes visible. If the diagram only shows how things work when everything is allowed, it does not teach reviewers where trust stops.

Denied path	Reason
ReportingService -> RootCauseDb write	Reporting owns projections, not diagnostic truth.
BFF service identity -> CloseRootCauseCase without user authority	BFF identity is transport authority, not operator authority.
RootCauseService -> ComplianceDb update	Verdict may request review; Compliance owns review state.
ComplianceService -> publish RootCauseVerdictIssued.v1	Only RootCause owns the canonical verdict contract.
AuditService -> trigger compliance workflow	Audit records evidence; it does not drive business process.
Any service -> broker.publish.*	Wildcard publishing destroys event ownership and producer accountability.

DESIGN RULE

Draw denied paths deliberately. A reviewer should be able to point at the map and say: this route is not merely absent; it is forbidden by ownership, policy, and credentials.

Boundary map as code-adjacent artifact

The security boundary map should not live only as a picture in a slide. It should connect to code and deployment artifacts: service permission manifests, policy names, broker permissions, database roles, certificate subjects, token audiences, and audit fields. The drawing teaches the architecture; the manifests make it enforceable.

```
# Security boundary map fragment - review artifact, not a product-specific format.
service: RootCauseService
zone: DomainServices
owns:
  - RootCauseCase
  - RootCauseVerdictIssued.v1
receives:
  http:
    - OpenRootCauseCase          # user context required
    - AttachEvidence             # user context required
    - IssueVerdict               # user context + case policy required
  events:
    - AlarmFloodDetected.v1     # broker-permitted consumer
publishes:
  - RootCauseVerdictIssued.v1   # broker-permitted publisher
calls:
  - AuditService.AppendEvidence
  - ComplianceService.RequestReview
denied:
  - ComplianceDb.write
  - AuditEntry.update
  - ReportingProjection.source-write
  - broker.publish.*
audit:
  required-fields: [service, user, correlationId, causationId, action, resource, result]
```

Correlation is part of the boundary

A security boundary map should include correlation, not as observability decoration but as accountability. When a user action becomes a service call, then an event, then a compliance review, then a reporting projection, the system must preserve enough context to explain the chain. Without correlation, least privilege may still block many bad actions, but investigation becomes guesswork.

Field	Why it belongs on the map
CorrelationId	Groups all work caused by the same operator-visible action or process.
CausationId	Links a message or command to the immediate event/request that caused it.
UserId / Actor	Preserves the human authority when a user-driven action crosses services.
ServiceName	Identifies the deployable service identity that made the call.
PolicyName	Shows which authorization rule accepted or rejected the action.
ResourceId	Names the case, review, audit entry, projection, or topic affected.
DecisionResult	Records allow, deny, abstain, timeout, duplicate, or compensation result.

Using the map in review

The map is most valuable before the system is deployed. During design review, each new service, endpoint, topic, subscription, database role, and projection should be checked against it. During incident review, the same map becomes a guide: which identities could have reached this resource, which policies should have blocked it, which events could have triggered it, and which audit trail proves what happened?

Review question	What a good answer looks like
Which boundary is crossed?	Named zone and service boundary, not vague internal call.
Who is the caller?	Human, BFF, service identity, worker identity, or support identity are separated.
Who owns the decision?	Receiving service or data owner is explicit.
What is the protected resource?	Case, event contract, review, audit entry, projection, secret, topic, or table is named.
What is denied?	At least one forbidden path is documented for the same area.
How is it audited?	Correlation, actor, service, resource, policy, and result are recorded.

Part VIII checkpoint

Part VIII began by separating authentication from authorization and ends by drawing the boundaries that make that separation enforceable. The result is not a certified plant security model. It is a disciplined software map: who calls, who decides, what is protected, what is denied, and how the chain is later explained.

HONEST BOUNDARY

This chapter does not replace threat modelling, penetration testing, secure infrastructure design, secrets-management operations, incident response planning, or regulatory security review. It gives the microservice architecture a visible security shape so those later activities have something concrete to challenge.

The next part of the book can now move from service boundaries and security boundaries to the runtime discipline that keeps those boundaries observable, deployable, and repairable when the system is under pressure.

Chapter 52 checkpoint

CHECKPOINT

1. Why is a security boundary map more useful than a list of secured endpoints?
2. What is the difference between call direction and trust direction?
3. Name three protected NEXUS-1 resources and their owners.
4. Why should denied paths appear explicitly on the map?
5. Which fields are needed to trace a user action across services, events, audit, and reporting?

PART IX

Observability and Operations

The first eight Parts designed the boundaries: service candidates, data ownership, APIs, messages, workflows, consistency, read models, and security. Part IX asks the operational question that turns all of that design into something a team can run: when the system is distributed, how does it explain itself?

WHAT PART IX ADDS

Observability is not a dashboard added at the end. It is the evidence layer of a distributed architecture: logs for facts, metrics for trends, traces for journeys, health checks for readiness, resilience policies for controlled failure, and deployment units for independent release.

CHAPTERS 53-59	WHAT THE READER SHOULD UNDERSTAND
Logs, Metrics, and Traces	The three basic signals and the different question each one answers.
Correlation IDs and Causality Chains	How one business flow is followed across commands, events, consumers, and projections.
Health Checks and Readiness Checks	Why alive is not the same as ready, and why readiness must be dependency-aware.
Resilience Patterns	Retry, timeout, and circuit breaker as explicit policies, not magic safety.
Deployment and Orchestration	How independently deployable services become independently observable services.
NEXUS-1 Dashboards	How the demonstrator would make service health, lag, failures, and causality visible.

This Part continues the same standing boundary: the book demonstrates architecture and implementation shape. It does not claim a certified operations stack, a production nuclear monitoring system, or live control authority.

PART IX - OBSERVABILITY AND OPERATIONS

53. Logs, Metrics, and Traces

A service that cannot explain what happened is not operationally independent. Logs record facts, metrics show pressure, and traces connect the journey across boundaries.

Why this chapter exists

Earlier chapters were careful about boundaries. RootCause owns diagnostic decisions. Compliance owns review decisions. Reporting owns projections. Audit owns append-only evidence. Security owns identity and access rules. But after those contexts become separate runtime units, another responsibility appears: every service must make its behaviour visible without a developer attaching a debugger or reading a private database directly.

That is the reason observability belongs in the microservice book, not in a final appendix. Once the system is distributed, a failure is no longer one stack trace in one process. It may be a command accepted by one service, an integration event delayed in an outbox, a consumer retrying in another service, a projection lagging behind, and an operator console showing stale-but-valid information. The architecture needs a way to explain that path.

PLAIN EXPLANATION

Observability means the system emits enough evidence that a team can ask what happened, where it happened, why it is slow or wrong, and which boundary owns the next action - without guessing from symptoms.

The naive approach: logs everywhere

The first naive solution is to add more log lines. Every method starts logging input, output, exceptions, and timing. At first this feels safer because the console is full of text. In practice, it becomes a second alarm flood. The text is real, but it is not structured enough to answer architectural questions.

A log line can say that Compliance received RootCauseVerdictIssued.v1. It cannot by itself show whether the event belongs to the same alarm-to-compliance flow as the operator is investigating. It cannot show whether the event consumer is falling behind. It cannot show whether all requests to the RootCause API have slowed or only those that call a downstream service. Logs are necessary. They are not sufficient.

SIGNAL	QUESTION IT ANSWERS	NEXUS-1 EXAMPLE
Log	What specific fact happened?	RootCause published RootCauseVerdictIssued.v1 with EventId and CaselId.

Metric	How often, how fast, how full, how late?	Outbox lag is 412 messages; Compliance consumer retries rose this hour.
Trace	Which steps did one flow cross?	Alarm flood -> root-cause case -> verdict -> compliance review -> reporting projection.

Logs: facts, not prose

A log in a distributed system should be a structured fact. It should have a stable event name, a severity, a timestamp, a service name, a correlation identifier, and the relevant business identifiers. The message text may be readable, but the fields are what make the record queryable.

```
logger.LogInformation(  
    "Root-cause verdict event published {@Event}",  
    new {  
        EventType = "RootCauseVerdictIssued.v1",  
        EventId = envelope.EventId,  
        CorrelationId = envelope.CorrelationId,  
        CausationId = envelope.CausationId,  
        RootCauseCaseId = verdict.CaseId,  
        UnitId = verdict.UnitId,  
        Producer = "RootCauseService"  
    });
```

The important detail is not the exact logging library. The important detail is the shape. A plain sentence such as verdict published may help a developer reading one file. A structured log can be filtered by CorrelationId, grouped by EventType, joined mentally with traces, and compared with metrics.

Metrics: pressure over time

Metrics are not individual stories. They are measurements over time. A service can be producing correct log entries and still be unhealthy because the rate, duration, queue depth, or error percentage has changed. Metrics tell the team that a boundary is under pressure before the business flow visibly breaks.

METRIC	TYPE	WHY IT MATTERS
http.server.request.duration	Histogram	Shows whether a service endpoint is becoming slow, not just failing.
outbox.pending.count	Gauge	Shows events committed locally but not yet published.
consumer.retry.count	Counter	Shows delivery friction or downstream rejection.
projection.lag.seconds	Gauge	Shows how far a read model is behind source truth.
deadletter.message.count	Counter	Shows messages that automation could not safely handle.
authz.denied.count	Counter	Shows whether policy boundaries are blocking calls as expected or unexpectedly.

A metric should not be invented only when something fails. The most useful metrics are designed with the boundary. If RootCause owns verdict publication, it should expose outbox publication lag. If Compliance consumes verdict events, it should expose consumer retry rate and review creation latency. If Reporting builds projections, it should expose projection freshness.

Traces: the path through the system

A trace connects operations that belong to one journey. In a monolith, the journey often fits inside one process and one call stack. In microservices, the call stack disappears at the network boundary. The trace is the replacement: a distributed tree of spans showing what happened first, what called what, what waited, what failed, and how long each step took.

SPAN	OWNER	WHAT IT TELLS US
POST /alarms/floods/{id}/root-cause-cases	AlarmManagement API	The user or process requested case opening for a flood.
OpenRootCauseCase handler	RootCause service	The root-cause case was created inside its own consistency boundary.
Outbox publish RootCauseCaseOpened.v1	RootCause outbox worker	The public event left the service boundary.
Compliance consume RootCauseVerdictIssued.v1	Compliance consumer	The review workflow reacted to the verdict.
Reporting projection update	Reporting service	The read model caught up for the operator console.

NEXUS-1 observability map

The NEXUS-1 flow gives observability a concrete shape. The question is not only whether each service is running. The question is whether the alarm-to-compliance story can be reconstructed from evidence without violating service ownership.

SERVICE	LOGS SHOULD NAME	METRICS SHOULD MEASURE	TRACES SHOULD CONNECT
AlarmManagement	AlarmFloodDetected.v1, flood id, unit id	Alarm rate, flood detection duration	Signal ingestion -> flood detection -> event publish
RootCause	Case opened, evidence attached, verdict issued	Case creation latency, outbox lag	Flood event -> case -> verdict event
Compliance	Review requested, review created, escalation	Consumer retries, review SLA age	Verdict event -> compliance review
Audit	Append accepted, append rejected	Append throughput, failed append attempts	Every evidence write as a span or linked span
Reporting	Projection updated, stale view served	Projection lag, rebuild duration	Event consumed -> read model refreshed

NEXUS-1 RULE

A service may expose its health and telemetry, but it should not make other services query its private database to understand what happened. Observability signals cross boundaries; private tables do not.

A small .NET implementation shape

In .NET, the implementation shape is usually built around structured logging, metrics instruments, and OpenTelemetry tracing. The exact vendor can change. The architectural rule should not: every deployable service emits the same basic envelope and uses the same correlation vocabulary.

```
builder.Services.AddOpenTelemetry()
    .WithTracing(t => t
        .AddAspNetCoreInstrumentation()
        .AddHttpClientInstrumentation()
        .AddSource("Nexus.RootCause")
        .AddOtlpExporter())
    .WithMetrics(m => m
        .AddAspNetCoreInstrumentation()
        .AddRuntimeInstrumentation()
        .AddMeter("Nexus.RootCause")
        .AddOtlpExporter());

using var activity = RootCauseActivity.StartActivity(
    "RootCause.IssueVerdict",
    ActivityKind.Internal);

activity?.SetTag("nexus.correlation_id", command.CorrelationId);
activity?.SetTag("nexus.case_id", command.RootCauseCaseId);
activity?.SetTag("nexus.unit_id", command.UnitId);
```

This is still only an implementation shape. The book does not claim a production observability stack. The important architectural point is that observability is designed as part of the service boundary. A RootCause service without its own traces and metrics is not truly operationally independent; it is merely separately deployed.

Anti-patterns this chapter prevents

ANTI-PATTERN	WHY IT HURTS
Log-only observability	Text grows into an alarm flood without rates, latency, or causal path.
Metrics without business identifiers	The team sees pressure but cannot connect it to the business flow.
Traces without correlation discipline	Spans exist, but the alarm-to-compliance journey cannot be followed.
Private database debugging	Teams bypass service boundaries to diagnose incidents, weakening ownership.
Observability after production	The system fails first, then the team discovers it did not emit the evidence needed to understand the failure.

Checkpoint - Chapter 53

Before moving on, you should be able to answer these without looking back:

1. What different question does a log answer compared with a metric and a trace?
2. Why are more log lines not the same as observability?
3. Which NEXUS-1 services should expose outbox lag, consumer retry count, projection lag, and dead-letter count?
4. Why should observability signals cross service boundaries while private tables remain protected?
5. How does traceability help explain delayed truth in the operator console without pretending the read model is source-of-truth?

HONEST BOUNDARY

This chapter introduces the three signals and the operational vocabulary. It does not yet build the full causality chain across every service. The next chapter focuses on CorrelationId, CausationId, and how one NEXUS-1 business flow is followed from alarm to compliance review.

PART IX - OBSERVABILITY AND OPERATIONS

54. Correlation IDs and Causality Chains

A distributed system needs more than timestamps. It needs a stable way to say which operations belong to the same business journey and which action caused the next one.

Why this chapter follows logs, metrics, and traces

Chapter 53 introduced the three observability signals: logs for facts, metrics for pressure, and traces for journeys. This chapter gives those signals a language. Without correlation discipline, the system may emit excellent telemetry and still fail to answer the most important question during an incident: are these events part of the same story?

In NEXUS-1, the story is familiar by now. A signal changes. An alarm is raised. A flood is detected. A root-cause case opens. Evidence is attached. A verdict is issued. Compliance review starts. Reporting eventually catches up. If each service invents its own identifier at each step, every page of telemetry is technically correct and practically disconnected.

PLAIN EXPLANATION

A CorrelationId says: these operations belong to the same business conversation. A CausationId says: this operation happened because of that earlier command or event. Together they turn many separate service records into one explainable chain.

Three identifiers, three jobs

The mistake is to make one identifier do every job. A business case id, an event id, a trace id, and a correlation id may all appear near the same flow, but they are not interchangeable. They answer different questions and they live at different levels of meaning.

IDENTIFIER	QUESTION ANSWERED	NEXUS-1 EXAMPLE
EventId / MessageId	Which single message is this?	The unique RootCauseVerdictIssued.v1 envelope published by RootCause.
CorrelationId	Which wider business conversation does this belong to?	The alarm-to-compliance flow started by AlarmFloodDetected.v1.
CausationId	Which command or event directly caused this one?	RequestComplianceReview caused ComplianceReviewRequested.v1.
Business id	Which domain object is involved?	RootCauseCaseId, AlarmFloodId, ComplianceReviewId, UnitId.
TraceId / SpanId	Which technical execution path did this request follow?	HTTP request into RootCause and its internal handler span.

Correlation is not causation

Two operations can share a `CorrelationId` without one causing the other. That is why `CausationId` exists. Correlation groups the conversation; causation connects the immediate parent to the immediate child. In a distributed workflow, this distinction prevents a common analytical error: assuming that every event in the same case caused every later event.

Consider the moment after a root-cause verdict is issued. Compliance may start a review. Reporting may update a projection. Audit may append evidence. Notifications may alert a supervisor. These activities share a correlation because they belong to the same business flow. They do not all cause one another. They are siblings caused by the verdict event or by separate process-manager decisions.

NEXUS-1 RULE

Use correlation to gather the flow. Use causation to explain the next step. Never infer causal order from timestamp order alone, and never infer causation only because two messages share the same `CorrelationId`.

The envelope as the boundary contract

The simplest place to enforce correlation discipline is the public message envelope. The payload says what the event means in the domain. The envelope says how the message travels through the distributed system and how it can be traced later.

```
public sealed record IntegrationEventEnvelope<TPayload>(
    Guid MessageId,
    string EventType,
    int SchemaVersion,
    Guid CorrelationId,
    Guid? CausationId,
    string ProducerService,
    DateTime OccurredAtUtc,
    TPayload Payload);

public sealed record RootCauseVerdictIssuedV1(
    Guid RootCauseCaseId,
    Guid AlarmFloodId,
    int UnitId,
    string VerdictCode,
    string Summary);
```

This envelope does not make the domain depend on messaging infrastructure. The domain event can remain internal, such as `VerdictIssued`. The public integration event becomes `RootCauseVerdictIssued.v1` only when the service publishes a stable contract outside its boundary. The correlation fields belong to that boundary-crossing contract.

A NEXUS-1 causality chain

The table below shows one simplified alarm-to-compliance path. The identifiers are illustrative, but the discipline is the point: the CorrelationId remains stable; each new message records the direct cause that produced it.

STEP	OPERATION	MESSAGE / ACTION	CORRELATION	CAUSED BY
1	Alarm flood detected	AlarmFloodDetected.v1	C-0418	-
2	Case opening requested	OpenRootCauseCase	C-0418	AlarmFloodDetected.v1
3	Root-cause case opened	RootCauseCaseOpened.v1	C-0418	OpenRootCauseCase
4	Evidence attached	EvidenceAttached	C-0418	Telemetry evaluation
5	Verdict issued	RootCauseVerdictIssued.v1	C-0418	CloseWithVerdict command
6	Compliance requested	RequestComplianceReview	C-0418	RootCauseVerdictIssued.v1
7	Review created	ComplianceReviewRequested.v1	C-0418	RequestComplianceReview
8	Projection updated	ReportingProjectionUpdated	C-0418	ComplianceReviewRequest ed.v1

A real implementation would use GUIDs or another globally unique format, not short labels such as C-0418. Short labels are used here only to make the example readable on the page.

HTTP headers and message metadata

Correlation must survive both synchronous and asynchronous boundaries. An HTTP request carries it in headers. A message carries it in envelope metadata. A background worker reads the incoming values and creates new outgoing values without losing the chain.

```
// Incoming HTTP request
X-Correlation-Id: 71c70881-6b8d-4b18-b2d8-3b1e8c1a9b10
X-Causation-Id: bfa6806e-d9d1-4fc1-8e4b-e1a2fe390f3a

// Outgoing integration event envelope
new IntegrationEventEnvelope<RootCauseVerdictIssuedV1>(
    messageId: Guid.NewGuid(),
    eventType: "RootCauseVerdictIssued.v1",
    schemaVersion: 1,
    correlationId: context.CorrelationId,
    causationId: command.CommandId,
    producerService: "RootCauseService",
    occurredAtUtc: clock.UtcNow,
    payload: payload);
```

The rule is simple but strict. A service may create a new `MessageId` for the message it produces. It should not create a new `CorrelationId` unless the business conversation genuinely starts there. It should set `CausationId` to the command, event, or scheduled action that directly caused the new operation.

Fan-out: causality becomes a tree

A distributed workflow is rarely a single line. One event may produce several reactions. After RootCauseVerdictIssued.v1, Compliance may create a review, Reporting may update a read model, Audit may append an explanatory record, and Notification may alert a supervisor. The shared CorrelationId lets the team find the family. The CausationId values show the branches.

REACTION TO ROOTCAUSEVERDICTISSUED.V1	SAME CORRELATION?	SAME CAUSE?	WHY
Compliance review requested	Yes	Yes	It reacts directly to the verdict event.
Reporting projection refresh	Yes	Yes	It also reacts directly to the verdict event.
Supervisor notification	Yes	Possibly	It may be caused by the verdict or by a process-manager escalation rule.
Audit append	Yes	Possibly	It records evidence, but should not become the workflow coordinator.

IMPORTANT DISTINCTION
AuditEntry remains evidence, not workflow. It may store correlation fields so that evidence can be replayed, but it should not be the thing that drives Compliance, Reporting, or RootCause behaviour.

The correlation context in .NET

A practical .NET implementation usually creates a small correlation context and makes it available to application handlers, outbox writers, logging enrichers, and message publishers. The context should be boring. Its value is consistency, not cleverness.

```
public sealed record CorrelationContext(
    Guid CorrelationId,
    Guid? CausationId,
    Guid? UserId,
    string? TenantCode);

public interface ICorrelationContextAccessor
{
    CorrelationContext Current { get; }
}

public sealed class OpenRootCauseCaseHandler
{
    public async Task Handle(OpenRootCauseCase command)
    {
        var correlation = correlationAccessor.Current;
        var caseOpened = RootCauseCaseOpenedV1.From(command);

        await outbox.AddAsync(caseOpened, envelope => envelope with {
            CorrelationId = correlation.CorrelationId,
            CausationId = command.CommandId
        });
    }
}
```

This is not a full framework. It is an implementation shape. The same principle applies whether the transport is HTTP, RabbitMQ, Azure Service Bus, Kafka, or an in-process mediator during the modular-monolith stage.

What correlation must not become

Correlation identifiers are powerful because they connect evidence. That power makes them dangerous when misunderstood. A correlation id is not an authorization token. It is not a business key. It is not proof that a message is valid. It is not a replacement for idempotency, versioning, or domain validation.

MISTAKE	CORRECTION
Using RootCauseCaseld as the CorrelationId	A case id identifies one domain object. A correlation id identifies the wider conversation, which may involve many objects.
Generating a new CorrelationId at every service	Generate at the start of the business conversation and propagate across boundaries.
Treating Traceld as the business correlation	Traceld is technical execution evidence. CorrelationId is business-flow evidence. They can be linked, but not confused.
Trusting a client-supplied correlation without validation	Accept or replace according to gateway policy; never let it carry authority.
Logging correlation but not storing it in the outbox	If the message is retried later, the chain must survive outside the original process memory.

NEXUS-1 correlation policy

The following policy is the minimum rule set a NEXUS-1 microservice implementation should enforce. It keeps the causal chain readable without leaking internal domain models or turning the observability layer into business logic.

RULE	POLICY
Creation	Create a CorrelationId when a new business conversation begins, such as alarm flood detection or a user command that starts a case.
Propagation	Propagate CorrelationId through HTTP headers, message envelopes, outbox records, inbox records, logs, traces, and audit evidence.
Causation	Set CausationId to the immediate parent command, event, schedule, or human action.
Storage	Store correlation metadata in outbox and inbox tables so retries preserve the chain.
Security	Never use CorrelationId as proof of identity, permission, or message authenticity.
Queries	Allow operational tools to search by CorrelationId, but do not let read models pretend correlation is source-of-truth.

Checkpoint - Chapter 54

Before moving on, you should be able to answer these without looking back:

1. What is the difference between EventId, CorrelationId, CausationId, business id, and TraceId?
2. Why is correlation not the same as causation?
3. Where should correlation metadata appear in HTTP calls, message envelopes, logs, outbox records, and inbox records?
4. Why should RootCauseCaseId not automatically become the CorrelationId?
5. Why must AuditEntry remain evidence instead of becoming the workflow coordinator?

HONEST BOUNDARY

This chapter defines the diagnostic chain. It does not yet decide whether a service instance is healthy, ready, degraded, or unavailable. The next chapter moves from following one business journey to checking whether a service can safely receive work at all.

PART IX - OBSERVABILITY AND OPERATIONS

55. Health Checks and Readiness Checks

A service may compile, start, and still not be safe to receive work. Health checks tell the platform what the service can do now, not what the domain means.

What you already know

You have seen applications that are technically running and still useless. The process is alive, but the database connection is broken. The API returns a home page, but its message broker credentials have expired. A worker is consuming messages, but the outbox table has stopped draining. From outside, the container looks up. From inside, the service is not ready.

Microservices make this distinction unavoidable. In a modular monolith, one broken dependency often becomes one visible application failure. In a distributed system, one service may be alive, another may be ready, another may be degraded, and another may be deliberately removed from traffic while it catches up. The platform needs a small, boring, reliable language for those states.

PLAIN EXPLANATION

A health check is an operational question, not a domain verdict. It asks whether the service process, dependencies, queues, storage, and background workers are in a condition that lets the platform route traffic or continue work safely.

Liveness, readiness, and startup are not the same

The first discipline is naming. A single endpoint called /health is often too vague. Does it mean the process has not crashed? Does it mean the service can accept user traffic? Does it mean migrations completed? Does it mean the broker is reachable? These are different operational questions.

CHECK TYPE	QUESTION	TYPICAL PLATFORM ACTION	NEXUS-1 EXAMPLE
Liveness	Is the process still alive enough to recover by itself?	Restart the instance if it fails repeatedly.	RootCauseService thread pool is not deadlocked and the process responds.
Readiness	Can this instance receive traffic or consume work now?	Remove instance from load balancing until ready.	RootCauseService can reach its own database and publish to the outbox path.
Startup	Has the instance finished bootstrapping?	Do not test liveness too early during slow startup.	Configuration loaded, migrations checked, contract registry initialized.
Degradation	Can the service run with reduced capability?	Keep serving limited traffic and surface warning.	Reporting projection lag is high, but query API can still serve stale labelled data.

The danger of deep health checks

A health check should reveal operational state without becoming a hidden business workflow. It should not run a root-cause inference. It should not replay an incident. It should not write audit entries merely to prove that audit works. It should not call every downstream service in the platform and amplify an outage.

Deep checks look attractive because they seem more honest. In practice, they often become noisy, slow, fragile, and dangerous. A readiness endpoint that waits on six downstream services can remove a perfectly useful service from traffic because an optional projection is delayed. A liveness endpoint that performs database queries can trigger needless restarts during a temporary SQL pause. The check must match the operational action attached to its result.

NEXUS-1 RULE

Do not let a health check become a control path. It may observe and report operational state. It must not make domain decisions, create workflow facts, or issue recommendations.

A service health model

A practical service health model separates local process state, owned dependency state, optional dependency state, and backlog pressure. That separation lets the platform make better decisions than a single green or red dot.

DIMENSION	FAILURE MEANS	STATUS
Process	The service cannot execute requests or workers safely.	Unhealthy for liveness.
Owned database	The service cannot read or write its own source of truth.	Not ready. Often unhealthy for write services.
Message broker publish path	The service cannot publish integration events or drain its outbox.	Not ready or degraded depending on the flow.
Optional downstream API	A read-side enrichment or notification dependency is unavailable.	Usually degraded, not dead.
Projection lag	Read models are behind the source of truth.	Degraded with visible stale-data labelling.
Configuration / secrets	The service cannot load required configuration or credentials.	Unhealthy or startup failure.

NEXUS-1 readiness by service

Each NEXUS-1 service candidate has a different readiness profile because each owns a different kind of truth. RootCause readiness is not Reporting readiness. Compliance readiness is not DigitalTwin readiness. Treating them the same would hide the architecture the book has spent fifty-four chapters making visible.

SERVICE	MUST BE READY FOR	MAY BE DEGRADED BY
AlarmManagement	Owned alarm store, alarm-rule configuration, outbox write path.	Delayed notification delivery or slow reporting projection.
RootCause	RootCause database, evidence store, outbox, contract metadata for RootCauseVerdictIssued.v1.	Optional explanatory enrichment or delayed reporting consumption.
Compliance	Compliance database, review workflow configuration, inbox idempotency store.	Delayed reporting projection or unavailable dashboard export.
Reporting	Projection store and query model availability.	Lag behind source events; stale but labelled read models.
Audit	Append-only audit store and write path.	Slow search index or delayed long-term archive export.
Operator Console BFF	Upstream APIs needed for the requested screen.	One panel unavailable while other panels remain readable.

CONNECTIVITY CHECK
A service is ready for the responsibilities it owns, not for the whole plant. If RootCause is unavailable, Reporting should not claim root-cause truth. If Reporting is delayed, RootCause should not stop issuing correct verdict events.

ASP.NET Core implementation shape

In .NET, health checks usually become small, named checks registered at startup and exposed through endpoints. The important part is not the library call. The important part is the mapping between check result and operational action.

```
builder.Services
    .AddHealthChecks()
    .AddCheck<RootCauseDatabaseHealthCheck>("rootcause-db")
    .AddCheck<OutboxDrainHealthCheck>("outbox-drain")
    .AddCheck<BrokerPublishHealthCheck>("broker-publish");

app.MapHealthChecks("/health/live", new HealthCheckOptions
{
    Predicate = check => check.Tags.Contains("live")
});

app.MapHealthChecks("/health/ready", new HealthCheckOptions
{
    Predicate = check => check.Tags.Contains("ready")
});
```

The example is intentionally simple. A real implementation would decide which checks carry the live tag and which carry the ready tag. It would also shape the response payload so that humans and dashboards can see why a service is degraded without exposing secrets, connection strings, or internal topology.

A custom readiness check

The custom check below illustrates the level of work a readiness check should usually do. It asks whether the outbox is draining within an acceptable threshold. It does not publish a test message. It does not mutate domain state. It reads operational evidence and returns a state.

```
public sealed class OutboxDrainHealthCheck : IHealthCheck
{
    private readonly NexusDbContext db;

    public async Task<HealthCheckResult> CheckHealthAsync(
        HealthCheckContext context,
        CancellationToken cancellationToken = default)
    {
        var oldestPending = await db.OutboxMessages
            .Where(x => x.Status == OutboxStatus.Pending)
            .OrderBy(x => x.OccurredAtUtc)
            .Select(x => x.OccurredAtUtc)
            .FirstOrDefaultAsync(cancellationToken);

        if (oldestPending == default)
            return HealthCheckResult.Healthy("Outbox is empty.");

        var age = DateTime.UtcNow - oldestPending;

        if (age > TimeSpan.FromMinutes(10))
            return HealthCheckResult.Unhealthy("Outbox is not draining.");

        if (age > TimeSpan.FromMinutes(2))
            return HealthCheckResult.Degraded("Outbox is delayed.");

        return HealthCheckResult.Healthy("Outbox is within threshold.");
    }
}
```

This check is operational, not ceremonial. If the outbox stops draining, events may be committed locally but not published. That is exactly the kind of hidden distributed failure Part V warned about.

Readiness should be local-first

A service should usually check the resources it owns before checking services it merely talks to. The owner of RootCause truth must know whether it can access RootCause storage. It should not declare itself dead merely because Reporting is behind. A service that is unable to reach an optional downstream dependency may need to degrade one feature, not disappear from the platform.

DEPENDENCY TYPE	CHECK STYLE	WHY
Owned database	Readiness check with short timeout.	Without it the service cannot protect its own truth.
Outbox / inbox store	Readiness or degradation check.	Without it message reliability may be broken.
Broker connectivity	Readiness for producers and consumers, often with cached result.	Publishing and consuming must not silently fail.
Optional remote service	Degradation check or dashboard signal.	Do not remove the whole service for an optional panel.
External notification channel	Degraded unless the service exists only to notify.	Notification failure is important, but not always fatal.

This local-first rule is the operational version of data ownership. A service is first accountable for the truth it owns and the contracts it publishes. It observes remote truth; it does not become invalid because every remote observer is imperfect.

Health status in the operator console

The operator console should not reduce the distributed platform to a single green badge. It should show what is healthy, what is degraded, and what is stale. A root-cause panel can be unavailable while the fleet overview still works. A reporting panel can be stale while the audit trail remains append-only and correct.

CONSOLE STATE	MEANING	USER MESSAGE
Healthy	The panel is backed by a ready service and current projection.	Current data.
Degraded	The service can answer, but with reduced capability or delayed dependency.	Available with reduced detail.
Stale	The read model is behind the source event stream.	Last updated at the visible timestamp.
Unavailable	The required service or owned store is not ready.	Panel unavailable; source service not ready.
Unknown	The console cannot determine status safely.	Status unknown; do not infer truth from this panel.

USER EXPERIENCE RULE

Never hide uncertainty behind a green screen. If the platform is degraded, stale, or partially unavailable, say so in the language of the screen. Operational truth is part of the model.

What a health check cannot prove

Health checks are necessary, but they are not sufficient. A healthy RootCauseService can still contain a modelling bug. A ready ComplianceService can still receive a malformed but syntactically valid event. A live ReportingService can still display a projection that is correct only up to a visible timestamp. Health checks show operational conditions. They do not prove the domain is correct.

HEALTH CHECK CAN SAY	HEALTH CHECK CANNOT SAY
The process responds.	The domain model is correct.
The database is reachable.	The stored facts are semantically true.
The outbox is draining.	Every consumer interpreted the event correctly.
The broker can be reached.	The whole workflow will finish successfully.
The projection is delayed.	The source-of-truth event was wrong.

That boundary matters in NEXUS-1 because the domain carries safety-adjacent language even though the software is educational. The book must not let an operational endpoint sound like certification. Ready means ready to receive software work inside the demonstrator, not fit to control anything real.

Checkpoint - Chapter 55

Before moving on, you should be able to answer these without looking back:

1. What is the difference between liveness, readiness, startup, and degradation?
2. Why should a readiness check not run a full business workflow?
3. Which resources should RootCauseService check before accepting work?
4. Why should Reporting lag degrade a reporting panel rather than invalidate RootCause truth?
5. What should the operator console show when a service is healthy, degraded, stale, unavailable, or unknown?

HONEST BOUNDARY

This chapter explains how services report operational state. It does not yet define how the system should react to repeated failure, retry safely, stop calling a failing dependency, or recover after a timeout. That is the work of the next chapter: resilience patterns.

PART IX - OBSERVABILITY AND OPERATIONS

56. Resilience: Retry, Timeout, Circuit Breaker

A distributed system does not become reliable because it tries again. It becomes reliable when every retry, timeout, and circuit breaker is tied to ownership, idempotency, and visible failure.

What you already know

You have already debugged the small version of this problem. A database call pauses. A remote API takes longer than usual. A background job fails once and succeeds on the second attempt. A user clicks refresh and the screen suddenly works. The temptation is to turn that observation into a rule: if something fails, try again.

In a single process, trying again is often harmless. In a microservice architecture, trying again can be either the cure or the disease. A retry can survive a temporary network blip. The same retry can also duplicate a command, overload a failing service, hide a broken dependency, and make a delayed incident harder to explain.

PLAIN EXPLANATION

Resilience is not pretending failure did not happen. It is the discipline of bounding failure: how long we wait, when we retry, when we stop calling, what we preserve, and how we make the degraded state visible.

The three basic questions

Before choosing a library, the service must answer three architectural questions. First, how long is this operation allowed to consume attention? Second, is it safe to repeat the operation? Third, what should happen when the dependency keeps failing? Timeouts, retries, and circuit breakers are only the technical shape of those answers.

QUESTION	PATTERN	ARCHITECTURAL MEANING	NEXUS-1 EXAMPLE
How long may we wait?	Timeout	Every operation has a time budget. Silence is not allowed to become infinite waiting.	The Operator Console BFF waits briefly for Reporting, then labels the panel unavailable or stale.
May we try again?	Retry	A repeated attempt is allowed only when the action is safe or protected by idempotency.	A read query may retry once; OpenRootCauseCase must not be blindly repeated without an idempotency key.
When do we stop calling?	Circuit breaker	Repeated failure is treated as state, not as a surprise each time.	Compliance does not hammer RootCause if verdict lookups are failing repeatedly.
How do we reduce pressure?	Backoff / jitter	Retries spread out instead of synchronizing into a second outage.	Multiple services waiting for Audit do not all retry at the same millisecond.
How do we protect neighbours?	Bulkhead	One failing dependency cannot consume every worker, connection, or thread.	Reporting export cannot starve alarm ingestion workers.

Timeouts: the first resilience policy

The timeout is the most honest resilience pattern because it admits that waiting is not free. A service that waits forever is not patient; it is unavailable in slow motion. Threads, connections, request slots, message locks, and user attention are all finite.

Timeouts should be chosen from the use case, not from a default copied across the solution. A synchronous query powering a console panel may have a short user-facing timeout. An outbox publisher may have a different timeout because it runs outside the user request. A long-running export may have a job timeout, not an HTTP request timeout.

OPERATION	TIMEOUT STYLE	WHY
Operator Console query to BFF	Short user-facing timeout.	The screen should degrade quickly instead of freezing the whole console.
BFF call to Reporting query API	Short remote-call timeout plus stale fallback.	Reporting is a projection; a delayed panel must not block every other panel.
Outbox publish attempt	Bounded broker-call timeout.	The worker can retry later without holding a user transaction open.
Compliance review command	No remote distributed transaction; local command timeout only.	The service owns its own command and publishes later through outbox.
Audit write inside owner service	Local storage timeout with fail-fast behaviour.	Audit evidence cannot be silently dropped or postponed without being visible.

NEXUS-1 RULE

No user-facing request should wait for the whole alarm-to-compliance world to become current. The console can show partial truth, stale truth, unavailable panels, and correlation links; it must not simulate single-database immediacy.

Retries: useful only when repetition is safe

A retry is a deliberate second attempt, not a nervous reflex. It belongs where failure is likely to be transient and where repeating the operation cannot create a second business fact. That makes idempotency the gatekeeper. If the operation is not idempotent, the retry policy must be extremely conservative or must be moved behind an idempotent mechanism such as an outbox, inbox, or command idempotency key.

ACTION	RETRY?	REASON
GET /root-cause/cases/{id}	Usually yes, with a small limit.	A read query is normally safe to repeat.
POST /root-cause/cases/open	Only with idempotency key.	Repeating may create duplicate cases unless the command identity is stable.
Publish RootCauseVerdictIssued.v1 from outbox	Yes, from the outbox worker.	The message has a durable identity and consumer inboxes protect duplicates.
RequestComplianceReview command	Only if the command handler is idempotent.	A duplicate review request must not create duplicate workflow obligations.
Export a large report file	Retry the job step, not the user request.	Long work belongs to a recoverable job, not a blocking HTTP retry.

The outbox and inbox chapters already prepared the ground for this rule. Once a service has committed local truth and an outbox row, it can retry publication without re-running the domain decision. Once a consumer has an inbox record, it can ignore a duplicate without pretending the duplicate never arrived.

Backoff and jitter

Immediate retries can turn a small outage into a storm. If one service fails and one hundred clients retry immediately, the dependency receives the original load plus the retry load exactly when it is least able to handle it. Backoff spaces attempts out. Jitter adds randomness so clients do not synchronize their second attempt.

```
// Implementation shape, not a complete production policy.
var retryPolicy = Policy
    .Handle<HttpRequestException>()
    .Or<TimeoutRejectedException>()
    .WaitAndRetryAsync(
        retryCount: 3,
        sleepDurationProvider: attempt =>
        {
            var baseDelay = TimeSpan.FromMilliseconds(100 * Math.Pow(2, attempt));
            var jitter = TimeSpan.FromMilliseconds(Random.Shared.Next(0, 75));
            return baseDelay + jitter;
        });
```

The code shape is less important than the constraint: the policy must not be universal. A query retry policy, a broker publish retry policy, and a command retry policy are different because their failure modes are different. Copying one policy across every HTTP client is an architectural smell.

CONNECTIVITY CHECK

Retries are part of connectivity, but they are not a substitute for contract design. If a retry can create a second domain fact, the contract is not ready for that retry.

Circuit breakers: stop making the failure worse

A circuit breaker remembers that a dependency is failing and temporarily refuses to call it. That sounds harsh until you remember what the alternative is: every request discovers the same failure again, consumes time, creates logs, holds resources, and adds pressure to the broken dependency.

A breaker usually has three states. Closed means calls are allowed. Open means calls are blocked for a while because recent failures crossed a threshold. Half-open means a small number of trial calls are allowed to test recovery. This is operational state, not domain state. It should be visible in metrics and dashboards, not hidden as a silent skip.

STATE	WHAT HAPPENS	NEXUS-1 INTERPRETATION
Closed	Calls are attempted normally.	Reporting can query Compliance projection service.
Open	Calls fail fast or use fallback until the break duration passes.	The BFF marks the compliance panel unavailable instead of making every user wait.
Half-open	A limited test call checks whether the dependency recovered.	One probe may refresh the panel; a flood of users is not released at once.
Fallback	The caller uses stale data, partial data, or an explicit unavailable state.	The console shows last known projection timestamp with a warning.

Bulkheads: prevent one failure from consuming the platform

A bulkhead is the operational version of a bounded context. It says that one failing path should not consume all resources. Separate connection pools, worker queues, thread limits, and concurrency limits keep a slow reporting export from starving alarm ingestion or root-cause evidence handling.

RESOURCE	WITHOUT BULKHEAD	WITH BULKHEAD
HTTP client pool	A slow downstream service exhausts all outbound connections.	Each dependency has bounded client resources.
Background workers	Large report exports block outbox publishing.	Export workers and outbox workers have separate queues.
Database connections	Projection rebuild starves command handling.	Read-model maintenance has its own pool or throttle.
Message consumers	Poison messages occupy every consumer slot.	DLQ routing and concurrency limits isolate the failed stream.
Operator Console panels	One failed panel freezes the whole screen.	Panels degrade independently.

Bulkheads connect back to the main thesis of the book. Microservices are useful only when they make ownership and failure boundaries clearer. If a system is distributed but every failure can still consume every shared resource, the boundary is mostly cosmetic.

NEXUS-1 resilience policy map

The following map is intentionally conservative. It prefers visible degradation, idempotent publication, and bounded waiting over aggressive retries. The aim is not to make every flow succeed immediately. The aim is to keep the platform explainable when something cannot succeed immediately.

FLOW	TIMEOUT	RETRY	BREAKER / FALLBACK
Console -> BFF status query	Short.	One small retry for transient network error.	Panel-level unavailable or stale state.
BFF -> Reporting projection API	Short.	Limited query retry only.	Show stale read model with timestamp.
RootCause command handling	Local command timeout.	No blind command retry; idempotency key required.	Return accepted/failed state; do not create duplicates.
Outbox -> broker publish	Bounded broker timeout.	Worker retry with backoff and durable message identity.	Outbox backlog alert; service may degrade.
Consumer -> inbox processing	Bounded processing timeout.	Retry according to message type; poison to DLQ.	DLQ and human review for repeated failure.
Compliance review trigger	Local inbox/process timeout.	Idempotent retry only.	Escalation if review cannot be started.

DESIGN RULE

Put resilience where the ownership is. A service retries its own publication from its own outbox. A consumer protects itself with its own inbox. A BFF degrades its own screen composition. No service should pretend to own another service's truth.

Implementation shape in ASP.NET Core

In a .NET implementation, resilience policies normally sit near the adapter that performs the remote call, not inside the domain model. The domain should not know that HTTP, a broker, or a circuit breaker exists. The application layer asks for a port. The infrastructure adapter protects the remote call and returns an explicit result.

```
public interface IComplianceReviewClient
{
    Task<StartReviewResult> RequestReviewAsync(
        RootCauseVerdictIssuedV1 verdict,
        CancellationToken cancellationToken);
}

public sealed class ResilientComplianceReviewClient : IComplianceReviewClient
{
    private readonly HttpClient http;
    private readonly AsyncPolicy<HttpResponseMessage> policy;

    public async Task<StartReviewResult> RequestReviewAsync(
        RootCauseVerdictIssuedV1 verdict,
        CancellationToken ct)
    {
        using var request = new HttpRequestMessage(HttpMethod.Post, "/reviews")
        {
            Content = JsonContent.Create(verdict)
        };

        request.Headers.Add("Idempotency-Key", verdict.EventId.ToString());

        var response = await policy.ExecuteAsync(
            token => http.SendAsync(request, token), ct);

        return StartReviewResult.FromHttp(response.StatusCode);
    }
}
```

The important line is not the policy object. It is the idempotency key. Without a stable command identity, resilience becomes guesswork. With a stable identity, retry can be an engineering choice rather than a duplicate-fact generator.

What resilience must not hide

A resilience policy can make a system calmer. It can also make a system dishonest. Too many retries hide latency. Too long a timeout hides overload. Too generous a fallback hides stale data. A circuit breaker without a visible dashboard hides partial outage. The policy is only acceptable when the user, operator, developer, or auditor can see what happened.

PATTERN MISUSED	WHAT IT HIDES	BETTER DESIGN
Silent retry	The dependency is unstable and every request is slower.	Retry with metrics, limit, and correlation ID.
Very long timeout	The service is not responsive under load.	Short timeout plus explicit degraded state.
Fallback to stale data without label	The user believes old projection is current truth.	Show timestamp and source of staleness.
Circuit breaker without dashboard	A service is unavailable but nobody sees the operational state.	Expose breaker state in metrics and console diagnostics.
Retrying non-idempotent command	Duplicate domain facts.	Idempotency key, outbox, inbox, or no retry.

NEXUS-1 keeps the same discipline here that it used for events and read models: delayed truth is allowed; hidden truth is not. Resilience is a way to keep the system usable while remaining honest about what is delayed, unavailable, duplicated, retried, or rejected.

Checkpoint - Chapter 56

Before moving on, you should be able to answer these without looking back:

1. Why is a timeout the first resilience policy rather than an implementation detail?
2. When is a retry safe, and when can it create duplicate domain facts?
3. What does a circuit breaker protect, and why must its state be visible?
4. How do backoff and jitter prevent synchronized retry storms?
5. Why does a bulkhead belong to the same architectural family as a bounded context?
6. Which NEXUS-1 calls should degrade a panel instead of blocking the whole console?

HONEST BOUNDARY

This chapter explains resilience policies at the service-call and worker level. It does not yet define the physical deployment unit, release boundary, container shape, or orchestration model. That comes next: deployment units and independent release.

PART IX - OBSERVABILITY AND OPERATIONS

57. Deployment Units and Independent Release

A microservice is not independent because it has its own repository. It is independent only when it can be built, released, observed, rolled back, and owned without asking the rest of the platform to move with it.

What you already know

You have probably seen a system called microservices where every service still has to be deployed at the same time. The code lives in many projects. The APIs have different URLs. The teams may even own different repositories. But the release calendar tells the truth: a change in one place forces a coordinated release of five others. The runtime was distributed; the release was not.

That is the subject of this chapter. A deployment unit is the thing that can be shipped as one operational artifact. A release boundary is the promise that this artifact can change without forcing unrelated artifacts to change at the same moment. Independent release is therefore not primarily a DevOps phrase. It is architecture made operational.

PLAIN EXPLANATION

A service is not only code behind an endpoint. It is an independently releasable unit of responsibility: code, configuration, database migration discipline, public contracts, health checks, logs, metrics, ownership, and rollback behaviour.

Deployment unit, service boundary, and bounded context

The book has repeated one sentence many times: a bounded context is not automatically a microservice. The same caution applies here. A bounded context is a language and ownership boundary. A service boundary is a runtime responsibility boundary. A deployment unit is the package that carries that responsibility into production. These three things should usually align, but they are not the same thing.

IDEA	QUESTION IT ANSWERS	NEXUS-1 EXAMPLE
Bounded context	Who owns the language and rules?	RootCause owns RootCauseCase, Evidence, Hypothesis, and VerdictIssued.
Service boundary	Who owns the runtime contract and failure behaviour?	RootCause service owns root-cause commands, queries, and RootCauseVerdictIssued.v1 publication.
Deployment unit	What can be built, configured, shipped, scaled, monitored, and rolled back together?	The RootCause API, worker, outbox publisher, configuration, and migrations form a candidate release unit.
Release boundary	Can this change move without forcing unrelated changes?	A RootCause internal model change should not force Compliance to redeploy if RootCauseVerdictIssued.v1 remains compatible.
Operational owner	Who is paged when it fails?	The team owning root-cause diagnosis owns its service health, logs, and outbox backlog.

DESIGN RULE

If a service cannot be released without redeploying its consumers, it is not yet independent. It may still be a good module, but the deployment boundary has not matured.

What belongs inside a deployment unit

The deployment unit should be boring to describe. That is a sign that the architecture has become explicit. It should include the executable process or processes, the configuration it needs, the public contracts it exposes, the migrations it owns, the health/readiness checks it publishes, the observability signals it emits, and the runbook a human can follow when it fails.

ELEMENT	BELONGS TO THE UNIT?	WHY IT MATTERS
API process	Yes.	The public command/query surface is part of the runtime responsibility.
Worker process	Often yes.	Outbox publishing and message consumption usually belong to the same service ownership area.
Service database schema or database	Yes, by ownership.	The service owns its local truth and its migrations.
Public API contracts	Yes.	Independent release requires stable contracts and compatibility discipline.
Integration event contracts	Yes.	Published messages are part of the release promise.
BFF or UI composition layer	Usually no.	It may be a separate deployment unit that composes multiple services.
Shared domain model library	Avoid.	A shared domain library couples releases and leaks internal language.
Runbook and dashboard	Yes.	A service no one can operate independently is not independently releasable.

For NEXUS-1, this matters most in the alarm-to-compliance flow. RootCause may own case decisions. Compliance may own review obligations. Reporting may own projections. Audit may own immutable evidence. Each can become a deployment unit only if its contract and data ownership allow it to move without dragging the others behind it.

The release boundary test

A simple way to test a proposed microservice is to ask what happens when it changes. Suppose RootCause changes how it ranks hypotheses internally. If Compliance must redeploy because it used an internal RootCause class, the boundary is broken. If Reporting must change a query over RootCause tables, the data boundary is broken. If the Operator Console fails because one panel expected a private field, the API boundary is broken.

CHANGE	INDEPENDENT RELEASE IF...	BOUNDARY FAILURE IF...
Internal aggregate refactoring	Public commands, queries, and events stay compatible.	Consumers use shared domain objects or database tables.
New field in RootCauseVerdictIssued.v1	Consumers ignore unknown fields or use versioned contract handling.	Every consumer must be redeployed immediately.
Database migration inside RootCause	Only RootCause code and workers depend on the changed schema.	Reporting queries RootCause tables directly.
Compliance review workflow update	RootCause still publishes the same verdict event.	RootCause calls internal Compliance workflow states synchronously.
Operator Console panel change	The BFF adapts projection shape without changing domain services.	The UI depends on internal service DTOs from many services.

CONNECTIVITY CHECK
Independent release is connected to almost every previous part of the book: data ownership, public contracts, outbox, inbox, versioning, idempotency, traceability, health checks, and resilience. Without those patterns, release independence is mostly a diagram.

NEXUS-1 candidate deployment units

The following map is a candidate view, not a final production prescription. It keeps the educational boundary of the series: NEXUS-1 is not operational plant software. The purpose is to show how an advanced DDD model could be carried toward runtime responsibility without pretending that every context must be extracted at once.

CANDIDATE UNIT	PRIMARY RESPONSIBILITY	RELEASE DEPENDENCY TO WATCH
AlarmManagement Service	Raise alarms, detect floods, publish AlarmFloodDetected.v1.	Must not require RootCause redeploy when alarm threshold internals change.
RootCause Service	Open cases, attach evidence, issue verdicts, publish RootCauseVerdictIssued.v1.	Must protect its aggregate model from Compliance and Reporting.
Compliance Service	Start and track compliance review obligations.	Must consume verdict events idempotently and not depend on RootCause tables.
Audit Service	Record immutable evidence and correlation lineage.	Must not become a synchronous bottleneck for every business action.
Reporting Projection Service	Build read models for dashboards and exports.	Must accept eventual consistency and rebuild projections without owning truth.
Operator BFF	Compose console views across services.	Must degrade panels independently instead of coupling all services to one UI release.

This table also shows why some contexts may remain inside a modular monolith for a long time. If a candidate cannot pass the release-dependency test, keeping it in-process may be the more honest architecture until its contracts and ownership are mature.

Databases, migrations, and release order

A service that owns data must own its migration discipline. This does not mean every service has to use a physically separate database on day one. It means no other service may treat that schema as its private dependency. When the database changes, the service must be able to apply the migration safely and remain compatible with the contracts it has already published.

MIGRATION STYLE	SAFE WHEN	DANGEROUS WHEN
Expand/contract	A new column or table is added, code writes both shapes if needed, old shape is removed later.	The old shape is removed before all readers are safe.
Backward-compatible event payload	New optional fields are added and old consumers ignore them.	Required fields change meaning without a new event version.
Projection rebuild	Reporting can rebuild from events or source APIs without blocking commands.	Rebuild requires direct access to owner tables or locks business writes.
Blue/green release	Both old and new versions can run during transition.	Two versions disagree about command semantics.
Rollback	The previous version can run against the current data shape.	The migration destroys data needed by the old version.

NEXUS-1 RULE

A release must not force the platform into a moment where every service must be correct at exactly the same time. That is the old monolith release risk, now distributed across more moving parts.

Independent release does not mean independent meaning

The word independent can mislead. A service can be independently released; it cannot be semantically isolated from the system it serves. RootCause can release its internal ranking model without coordinating every service. It cannot redefine what a verdict means without changing the public language of the platform. Compliance can change its internal review workflow. It cannot silently reinterpret RootCauseVerdictIssued.v1.

CAN CHANGE INDEPENDENTLY	MUST REMAIN COHERENT
Internal class names, folder structure, private tables.	Public commands, queries, and event contract meaning.
Internal ranking heuristics or evidence scoring implementation.	The definition of verdict, evidence, corrected verdict, and recommendation.
Storage engine, indexes, worker concurrency.	Idempotency keys, correlation IDs, and event identifiers.
Dashboard layout or panel composition.	Truth labels: source-of-truth, projection, stale, unavailable.
Deployment frequency and scaling settings.	The shared operational story visible in logs, metrics, traces, and runbooks.

This is the difference between autonomy and chaos. Microservices need enough autonomy to release safely and enough shared language to remain one system. From Context to Flow prepared that language. This volume now turns it into release discipline.

Implementation shape: service manifest

A practical team can make release independence visible with a simple service manifest. The manifest is not a framework requirement. It is an architectural checklist written in a form that humans can review. It says what the service owns, what it exposes, what it consumes, how it is deployed, and what evidence proves that release is safe.

```
service: RootCause
boundedContext: RootCause
deploymentUnit: nexus-rootcause-service
ownerTeam: DiagnosisPlatform

publicApis:
- POST /root-cause/cases
- POST /root-cause/cases/{caseId}/evidence
- POST /root-cause/cases/{caseId}/verdict
- GET /root-cause/cases/{caseId}

publishes:
- RootCauseVerdictIssued.v1
- RootCauseVerdictCorrected.v1

consumes:
- AlarmFloodDetected.v1

ownedData:
- RootCause schema or database
- Outbox table
- Inbox table

releaseEvidence:
- contract tests pass
- migration is backward compatible
- health and readiness checks pass
- outbox backlog dashboard exists
- rollback plan documented
```

The value of a manifest like this is not the file format. The value is that independence becomes reviewable. If the manifest says the service owns RootCause data but Reporting still reads RootCause tables directly, the contradiction is visible before the release fails.

Operational evidence before release

A deployment unit should not be released merely because its tests passed. Tests prove design assumptions under controlled conditions. Release evidence proves that the service can be operated after it leaves the developer machine. That evidence includes contract compatibility, migration safety, dashboard visibility, alert routes, health checks, readiness checks, and a rollback path.

EVIDENCE	QUESTION ANSWERED
Contract tests	Can consumers still understand the API or event contract?
Migration rehearsal	Can the data shape move forward without breaking the old version too early?
Readiness check	Should the platform send traffic to this service now?
Dashboard	Can an operator see latency, error rate, backlog, breaker state, and dependency health?
Trace sample	Can a root-cause verdict be followed across RootCause, Compliance, Audit, and Reporting?
Rollback or roll-forward plan	What happens if the service is correct in tests but wrong in production behaviour?
Runbook	What should the human do when the service is degraded, not down?

HONEST BOUNDARY
This chapter describes deployment units and release independence at architecture level. It does not yet teach container images, orchestration primitives, pods, services, ingress, Helm charts, or Kubernetes operations. Those concepts come next.

Checkpoint - Chapter 57

Before moving on, you should be able to answer these without looking back:

1. What is the difference between a bounded context, a service boundary, and a deployment unit?
2. Why does independent release require stable public contracts?
3. Why can a shared domain library damage release independence?
4. How do migrations affect the release boundary of a service that owns data?
5. Which NEXUS-1 candidates look like deployable units, and which dependencies must be watched?
6. What operational evidence should exist before a service is released independently?

NEXT STEP
A deployment unit is still abstract until it has a runtime shape. The next chapter introduces containers and orchestration concepts without turning the book into a Kubernetes manual.

PART IX - OBSERVABILITY AND OPERATIONS

58. Containers and Orchestration Concepts

Containers make a service portable enough to run the same way in more than one place. Orchestration decides where that container runs, how it is restarted, how it is reached, and how it is observed. Neither one decides whether the service boundary is correct.

Why this chapter exists

By this point in the book, NEXUS-1 has service candidates, data ownership rules, public contracts, integration events, process managers, read models, security boundaries, health checks, resilience policies, and deployment units. That is the right moment to speak about containers and orchestration. They come after service design, not before it.

A common mistake is to treat containerization as the proof that microservices exist. A team puts each Web API into a Docker image, pushes those images to a registry, writes a few YAML files, and calls the result a microservice architecture. The runtime may be modern, but the model may still be shared, synchronous, fragile, and unclear. Containers package software. They do not fix ownership.

STANDING RULE

Do not let the runtime vocabulary replace the domain vocabulary. Pod, container, image, deployment, service, ingress, queue, and replica are operational words. They are useful only when they preserve the boundaries already established by AlarmManagement, RootCause, Compliance, Reporting, Audit, and the operator-console BFF.

What a container is - in plain language

A container is a packaged process with its dependencies, configuration surface, network boundary, and filesystem assumptions made explicit. For a .NET service, that usually means a compiled application, an ASP.NET Core runtime or self-contained output, environment variables, exposed ports, health endpoints, and a startup command. The container image is the packaged artifact. The running container is an instance of that artifact.

Runtime word	Plain meaning	NEXUS-1 interpretation
Image	The packaged service artifact.	RootCauseService image version containing API, handlers, outbox publisher, and health endpoints.
Container	A running process created from the image.	One running RootCauseService instance handling commands and publishing verdict events.
Registry	Where images are stored and versioned.	The place where released NEXUS-1 service images are promoted between environments.
Environment variables	Runtime configuration inputs.	Connection strings, broker endpoints, feature flags, and OpenTelemetry endpoints - never hardcoded secrets.
Volume	Mounted storage external to the container.	Rare for stateless APIs; more relevant for local dev, diagnostics, or sidecar patterns.
Network	How containers reach other processes.	Service-to-service calls, broker access, database access, and observability export paths.

The most important architectural consequence is this: if a service cannot start from a clean image with explicit configuration, then it is not operationally independent. It may still work on one developer machine or one server, but that is not the same as being deployable as a service.

What an orchestrator adds

An orchestrator manages containers as a fleet. It decides how many instances should run, where they should run, how they are restarted, how they receive traffic, how configuration and secrets are injected, how health checks affect routing, and how upgrades happen. Kubernetes is the most common vocabulary, but the architectural ideas are broader than any one product.

Orchestration concern	Question it answers	NEXUS-1 example
Scheduling	Where should this container run?	Run RootCauseService on nodes allowed to reach its database and broker.
Replica management	How many instances should exist?	Scale Reporting projection readers separately from RootCause command handling.
Service discovery	How does one service find another?	BFF reaches RootCause API through a stable service name, not a machine address.
Traffic routing	Which instance receives the request?	Route operator-console API traffic only to ready instances.
Configuration injection	How does runtime config enter?	Set broker topic names, database endpoints, and observability exporters by environment.
Secret injection	How are credentials supplied?	Mount certificates and credentials from the platform secret store.
Rolling updates	How is a version replaced?	Deploy a new Reporting API without forcing RootCause or Compliance to redeploy.
Health-based recovery	What happens when a service fails?	Restart failed consumers; stop routing to unready APIs; expose degraded dependencies.

The container boundary is not the service boundary

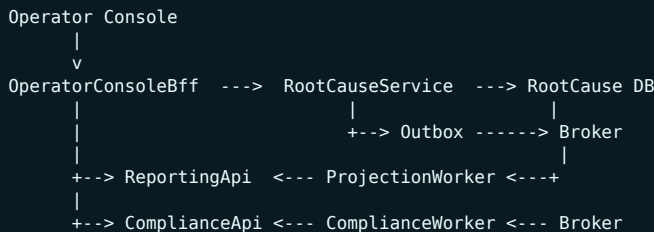
A container boundary is physical: one process image, one runtime instance, one startup command. A service boundary is conceptual and organizational: one language owner, one data owner, one contract owner, one failure responsibility, one operational owner. The two should eventually align, but they are not the same thing.

Boundary type	What it separates	Danger when confused
Bounded context	Language and model ownership.	Assuming every context deserves independent deployment.
Service boundary	Runtime ownership, contracts, data, failure responsibility.	Shipping a network boundary before ownership is mature.
Container boundary	A packaged process and its runtime dependencies.	Calling a container a microservice even when it shares database truth.
Pod/task boundary	One or more co-scheduled containers.	Hiding multiple responsibilities inside one runtime unit.
Deployment boundary	What can be released independently.	Forcing unrelated services to move together because they share build or data contracts.

NEXUS-1 RULE
RootCauseService in a container is not automatically a microservice. It becomes a credible microservice only if RootCause owns its data, protects RootCauseCase invariants, publishes RootCauseVerdictIssued.v1, consumes only stable contracts, and can be deployed, observed, and rolled back without changing AlarmManagement or Compliance.

A simple NEXUS-1 runtime map

The simplified map below is not a production deployment prescription. It is a thinking tool. It shows how the already-designed services might be packaged and operated without pretending that orchestration itself solves data ownership, event contracts, or security.



Each box may become a container image.

Each running image may have one or more replicas.

The broker, databases, and observability stack are infrastructure dependencies.

The domain truth still belongs to the owning service, not to the orchestrator.

Dockerfile as an implementation shape

A container recipe should be boring. It should build the service, publish it, run it with an explicit entry point, and leave environment-specific decisions outside the image. The image should not contain production secrets, machine-specific paths, or hidden assumptions about where it will run.

```

# Example shape only - not a complete production Dockerfile.
FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
WORKDIR /src
COPY . .
RUN dotnet restore Nexus.RootCause.Api/Nexus.RootCause.Api.csproj
RUN dotnet publish Nexus.RootCause.Api/Nexus.RootCause.Api.csproj -c Release -o /app/publish --no-restore

FROM mcr.microsoft.com/dotnet/aspnet:9.0 AS runtime
WORKDIR /app
COPY --from=build /app/publish .
ENV ASPNETCORE_URLS=http://+:8080
EXPOSE 8080
ENTRYPOINT ["dotnet", "Nexus.RootCause.Api.dll"]
  
```

HONEST BOUNDARY

A Dockerfile proves only that the service can be packaged. It does not prove that the service owns its data, that the API contract is stable, that events are versioned, that secrets are safe, or that the service is observable under failure.

Orchestration descriptors should express operational intent

An orchestration descriptor is not only a way to start a container. It is a contract with the runtime: how many instances are desired, when the instance is considered ready, what resources it expects, which configuration it receives, which secrets it can see, and which ports are exposed. The descriptor should therefore reflect the architectural decisions already made in the book.

Descriptor element	Architecture decision it should reflect
Image tag	The released service version and contract compatibility.
Replicas	Expected load and whether the service can safely run concurrently.
Readiness probe	Whether the service can accept traffic without breaking the user flow.
Liveness probe	Whether the runtime should restart the process.
Environment values	Non-secret configuration: topic names, feature flags, endpoints, log levels.
Secret references	Credentials and certificates with scoped access.
Resource limits	Operational budget and noisy-neighbor protection.
Labels/annotations	Ownership, service catalog metadata, traceability, and deployment automation hooks.

```
# Kubernetes-like fragment, simplified for explanation.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: rootcause-service
  labels:
    nexus.context: RootCause
    nexus.owner: diagnostics-team
spec:
  replicas: 2
  template:
    spec:
      containers:
        - name: rootcause-api
          image: registry.example/nexus/rootcause-service:1.4.0
          ports:
            - containerPort: 8080
          readinessProbe:
            httpGet: { path: /health/ready, port: 8080 }
          livenessProbe:
            httpGet: { path: /health/live, port: 8080 }
          env:
            - name: OTEL_SERVICE_NAME
              value: RootCauseService
            - name: CONTRACT_VERSION
              value: RootCauseVerdictIssued.v1
```

Stateful dependencies are not made simple by containers

APIs and workers are often easy to containerize because they are mostly stateless processes. Databases, brokers, observability stores, and secret stores are different. They are stateful infrastructure. They need backup, retention, recovery, storage

guarantees, access controls, and operational ownership. A development environment may run them in containers; a serious environment must treat them as durable platform dependencies.

Component	Container-friendly in dev?	Production caution
RootCauseService API	Yes	Keep it stateless; scale only if concurrency rules and idempotency are correct.
ComplianceWorker	Yes	Protect idempotency and message ordering assumptions.
Reporting projection worker	Yes	Lag, rebuild, and replay rules must be explicit.
SQL database	Sometimes	Data durability, backup, restore, encryption, and upgrades dominate the design.
Message broker	Sometimes	Topic permissions, retention, DLQ policy, and cluster durability matter.
Observability stack	Sometimes	Retention, sampling, cardinality, and access to traces must be governed.
Secret store	No casual treatment	Secrets require platform-grade access control, rotation, audit, and recovery.

Scaling replicas changes the failure model

Orchestration makes it easy to increase replicas. Architecture decides whether that is safe. A command-handling service that protects invariants may scale horizontally only if its database transactions, concurrency tokens, idempotency keys, and message publication rules still hold. A projection worker may scale only if partitioning, ordering, duplicate handling, and checkpointing are understood.

Scaling target	Possible benefit	Hidden risk to check
RootCause API replicas	More command throughput and availability.	Concurrent verdict attempts, duplicate commands, outbox transaction discipline.
Outbox publishers	Faster event publication.	Duplicate publication, lock strategy, ordered contract assumptions.
Inbox consumers	Higher event-processing throughput.	Partitioning, idempotency, poison message isolation, replay behaviour.
Reporting workers	Faster projection rebuild.	Out-of-order projection updates and misleading freshness indicators.
BFF replicas	More UI request capacity.	Session assumptions, cache coherence, user-context propagation.

CHECK BEFORE SCALING

Before increasing replicas, ask: Is the handler idempotent? Is the database update protected? Is ordering assumed? Is the outbox safe? Is the inbox safe? Does the user experience explain delayed truth? If the answer is unclear, more replicas may create more failure, not more capacity.

What orchestration must not hide

A workflow engine must not hide domain decisions; an orchestrator must not hide operational decisions. Restarts, retries, rolling updates, autoscaling, and service discovery are valuable, but they can also make failure look cleaner than it is. A restarted consumer may process the same message again. A rolling update may run two contract versions at once. A readiness probe may pass while a downstream dependency is degraded. The system still needs explicit design.

Runtime feature	What it can hide if abused
Automatic restart	A crash loop that repeatedly reprocesses the same poison message.
Rolling update	Mixed versions with incompatible event or API contracts.
Autoscaling	Pressure caused by a downstream bottleneck rather than real demand.
Service discovery	Synchronous dependency chains that still amplify failure.
Readiness probes	A service that can answer HTTP but cannot complete the business action.
Central logs	Logs without correlation, actor, resource, and causation context.
Sidecars	Infrastructure behaviour that developers stop understanding because it is outside the service code.

NEXUS-1 containerization checklist

The following checklist keeps containerization tied to the service-readiness thesis of the book. It asks whether the service can be packaged, configured, secured, observed, scaled, and released without breaking the model it is supposed to protect.

Question	Good answer
What is the image?	One deployable service or worker with a clear owner and version.
What does it own?	A named domain responsibility, data ownership boundary, or projection responsibility.
What configuration is external?	Environment-specific endpoints, topic names, feature flags, log level, OTEL exporter.
Where are secrets?	Injected by platform secret mechanism, never baked into the image.
How is readiness known?	A readiness endpoint that checks the minimum dependencies needed to accept work.
How is liveness known?	A liveness endpoint that says whether the process should be restarted.
How is it traced?	Service name, correlation id, causation id, actor context, and operation names are propagated.
Can it run twice?	Concurrency, idempotency, inbox/outbox, and locking rules survive more than one replica.
Can it be rolled back?	Contracts, database migrations, and events tolerate rollback or safe forward recovery.

Chapter 58 checkpoint

CHECKPOINT
1. Why is a container not the same thing as a microservice? 2. What does an orchestrator add beyond running a process? 3. Why should orchestration come after service-boundary design, not before it? 4. Which NEXUS-1 components are good container candidates, and which stateful dependencies require special caution? 5. What must be checked before increasing the replica count of a command service, outbox publisher, or projection worker?

The next chapter closes Part IX by turning the same operational discipline into dashboards: not beautiful screens for their own sake, but evidence panels that show whether NEXUS-1 services, contracts, queues, projections, and user-facing flows are healthy enough to trust.

PART IX - OBSERVABILITY AND OPERATIONS

59. Operational Dashboards for NEXUS-1

A dashboard is not a decoration placed on top of the system. It is the operational form of the architecture: the place where ownership, flow, delay, failure, recovery, and confidence become visible.

Why this chapter exists

The previous chapters made NEXUS-1 observable, traceable, resilient, deployable, and container-ready. That is still not enough. Operators, developers, support engineers, and architects need a place where the runtime tells the truth about itself. Not the business truth of a RootCauseCase, not the compliance truth of a review, and not the reporting truth of a projection - the operational truth of whether the distributed system is healthy enough to trust.

In a modular monolith, many failures remain local and immediate. In a service architecture, failure can be partial, delayed, hidden behind retries, or pushed into a queue. A service may be alive but unready. A projection may be current for one unit and stale for another. A saga may be waiting correctly, or it may be stuck. A broker may be accepting messages while a poison message quietly fills the dead-letter queue. Operational dashboards exist because distributed systems do not fail in one obvious place.

STANDING RULE

Do not build dashboards that merely make the system look impressive. Build dashboards that reveal whether the service boundaries, contracts, queues, projections, retries, compensations, and human escalations are behaving as designed.

The naive dashboard

The naive dashboard asks one question: are the services up? It shows green boxes for AlarmManagement, RootCause, Compliance, Audit, Reporting, and the Operator Console BFF. That is useful as a first glance, but it is not operational understanding. A green service can still be publishing no events. A healthy API can still be connected to a broken broker. A live projection can still be ten minutes behind. A process manager can be running while every process instance is waiting forever.

Naive signal	Why it is insufficient	Better operational question
HTTP 200 on /health/live	The process is not dead, but it may not be ready to do useful work.	Can the service perform its current responsibility?
CPU and memory look normal	Normal resource usage can coexist with silent business failure.	Are commands, events, projections, and workflows moving?
No exceptions in logs	Retries and compensations may hide the problem until later.	Are retry counts, DLQ counts, and stuck states increasing?
Queue has messages	A queue can mean healthy buffering or an accumulating blockage.	Is queue age, lag, and consumer throughput within the expected band?
Dashboard is mostly green	A single red dependency may be the one that matters to the user flow.	Which user-facing flow is degraded, and why?

Operational dashboards are not reporting screens

NEXUS-1 already has reporting and compliance views. Those views answer domain and governance questions: what happened, what verdict was issued, what review is open, what evidence exists, and what projection the operator can read. An operational dashboard answers a different question: can the distributed runtime currently carry those facts across services without losing, duplicating, hiding, or delaying them beyond the agreed boundary?

Screen type	Primary audience	Question answered	Source of truth
Operator console	Operator / learner	What is the state of the demonstrator flow?	Domain APIs and read models.
Reporting screen	Analyst / management	What facts and trends have been projected?	Reporting projections.
Compliance view	Compliance reviewer	What needs review and what evidence exists?	Compliance service plus audit evidence.
Operational dashboard	Support / platform / architect	Is the distributed system healthy, delayed, blocked, or degraded?	Telemetry, traces, health checks, queues, logs, deployment metadata.

NEXUS-1 SENTENCE TO REMEMBER
Reporting tells what the system knows. Operational dashboards tell whether the system is still able to know, move, explain, and recover.

The dashboard layers

A useful dashboard set should not be one giant page. It should be layered. A first page shows the health of the whole platform. A flow page follows the alarm-to-compliance path. A messaging page shows broker lag, dead letters, retries, and outbox publication. A projection page shows delayed truth. A deployment page shows versions and rollout state. A security page shows denied calls, expired credentials, and policy failures.

Dashboard layer	What it watches	Why it matters
Platform overview	Service readiness, dependencies, version spread, global incidents.	Shows whether the platform is broadly usable.
Alarm-to-compliance flow	Events, process-manager stages, pending commands, timeout states.	Shows whether the central NEXUS-1 story is moving.
Messaging reliability	Outbox age, publish rate, inbox duplicates, DLQ count, poison patterns.	Shows whether asynchronous communication is healthy.
Projection freshness	Read-model lag, rebuild progress, last consumed event, stale screens.	Shows delayed truth before it becomes user confusion.
Security boundary	Authentication failures, authorization denials, certificate expiry, secret rotation.	Shows whether service trust is working safely.
Deployment and release	Image versions, replicas, rollout progress, rollback markers.	Shows whether independent deployment is real or only claimed.
Cost and capacity	CPU, memory, queue depth, request rate, latency percentiles.	Shows where operational pressure is accumulating.

A NEXUS-1 operational overview

The overview should be deliberately simple. It should not try to explain every trace, every log line, or every metric. It should show the state of the service map and point the reader toward the page that explains the problem. The aim is fast orientation, not decorative density.

```
NEXUS-1 Operational Overview

[Operator BFF]  ready  v1.8.0  p95 42 ms
|
+--> [AlarmManagement]  ready  events/min 18  outbox age 03 s
|
+--> [RootCause]        ready  cases open 4  verdict event lag 06 s
|
+--> [Compliance]       degraded  review worker lag 02 m 14 s
|
+--> [Audit]            ready  append latency p95 31 ms
|
+--> [Reporting]        stale  projection lag 05 m 42 s

Flow status: Alarm-to-compliance degraded, not stopped.
User message: compliance and reporting views may be delayed.
Current suspect: ComplianceWorker consuming slowly from broker partition 2.
```

This sketch makes several architectural decisions visible. Reporting is allowed to be stale, but the staleness must be named. Compliance can be degraded without stopping alarm detection or root-cause case management. Audit is watched as evidence infrastructure, not as workflow owner. The BFF can expose a careful user message instead of pretending every screen is equally current.

Metrics that should exist because of the architecture

The best metrics are not generic. CPU, memory, request count, and error rate matter, but they are not enough. A NEXUS-1 dashboard should include metrics that follow from the service design: outbox age, inbox duplicates, process-manager waiting state, projection lag, correlation completeness, dead-letter growth, contract version spread, and evidence append latency.

Metric	Owner	Healthy interpretation
nexus_outbox_oldest_unpublished_seconds	Publishing service	Small and stable; outbox is draining after commits.
nexus_inbox_duplicate_total	Consuming service	Nonzero is normal; sudden growth means upstream retry or redelivery pressure.
nexus_dlq_messages_total	Broker/platform	Zero or explicitly explained; growth requires triage.
nexus_process_waiting_total	Process manager	Some waiting is normal; old waiting instances need attention.
nexus_projection_lag_seconds	Reporting / projection owner	Within user-facing freshness policy.
nexus_trace_missing_correlation_total	All services	Should be near zero; missing correlation breaks auditability.
nexus_auth_denied_total	Security boundary	Expected denials are normal; unexpected spikes indicate policy or credential problem.
nexus_audit_append_latency_ms	Audit service	Evidence writes stay inside the agreed operational envelope.

Small .NET instrumentation shape

The chapter does not need a full observability stack implementation, but the code shape should be concrete. A service should expose architectural metrics from the same points where it already knows architectural facts: after publishing from the outbox, after consuming into the inbox, after recording a process step, after updating a projection checkpoint, and after rejecting a request at a security boundary.


```
// Example shape only: architectural metrics, not business decisions.
public sealed class OutboxPublisherMetrics
{
    private readonly Histogram<double> _publishLatency;
    private readonly ObservableGauge<long> _oldestUnpublishedAge;

    public OutboxPublisherMetrics(IMeterFactory meters,
                                  IOutboxDiagnostics diagnostics)
    {
        var meter = meters.Create("Nexus.Messaging");

        _publishLatency = meter.CreateHistogram<double>(
            "nexus_outbox_publish_latency_ms",
            unit: "ms");

        _oldestUnpublishedAge = meter.CreateObservableGauge<long>(
            "nexus_outbox_oldest_unpublished_seconds",
            () => diagnostics.GetOldestUnpublishedAgeSeconds());
    }

    public void Published(string contractName,
                          string serviceName,
                          TimeSpan elapsed)
    {
        _publishLatency.Record(
            elapsed.TotalMilliseconds,
            new KeyValuePair<string, object?>("contract", contractName),
            new KeyValuePair<string, object?>("service", serviceName));
    }
}
```

DESIGN BOUNDARY

A metric should describe operational behavior. It should not become a second business model. Do not encode RootCause verdict logic, compliance policy, or audit semantics inside the dashboard layer. Measure the movement of facts; do not redefine the facts.

Dashboards need thresholds, not vibes

A dashboard without thresholds becomes a painting. It may look sophisticated, but the reader still has to guess what matters. NEXUS-1 should state operational thresholds in the language of the architecture: maximum outbox age, maximum projection lag, maximum process waiting time before escalation, maximum failed authorization spike, maximum DLQ growth, and minimum correlation completeness.

Signal	Example threshold	Meaning when breached
Outbox oldest unpublished age	> 60 seconds	Service commits are not being turned into published messages quickly enough.
Compliance review worker lag	> 5 minutes	Reviews may be delayed; user-facing compliance status should show degraded freshness.
Reporting projection lag	> 10 minutes	Reporting is stale; read model should display freshness warning.
Process waiting without reason	> configured timeout	Workflow may be stuck; escalation or operator review is needed.
DLQ growth	> 0 unexplained or increasing	A contract, poison message, or consumer defect needs triage.
Missing correlation id	> 0.1 percent of requests/events	Traceability is damaged; audit reconstruction may be incomplete.
Certificate expiry	< 14 days	Service-to-service trust is approaching failure; rotate before outage.

The flow dashboard

The most important operational dashboard for this book is not service-by-service. It is flow-by-flow. The recurring NEXUS-1 story crosses Instrumentation, AlarmManagement, RootCause, Audit, Compliance, Reporting, and the Operator Console. The dashboard should show that flow as a chain with state, lag, and ownership at each boundary.

```
Alarm-to-Compliance Operational Flow

Signal received      Instrumentation      OK      last event 11:42:03
Alarm raised         AlarmManagement      OK      outbox age 02 s
Flood detected       AlarmManagement      OK      window complete
Case opened          RootCause             OK      case RCA-2026-0418
Evidence attached    RootCause             OK      evidence count 7
Verdict issued       RootCause             OK      RootCauseVerdictIssued.v1
Audit entry written  Audit                 OK      append latency 28 ms
Review requested     ProcessManager        WAIT    command pending
Compliance opened    Compliance            LAG     worker lag 02 m 14 s
Projection updated   Reporting             STALE   lag 05 m 42 s
Console displayed    BFF                   WARN    freshness banner shown

The question is not "which service is red?"
The question is "where is the flow, who owns the wait, and what should the user be told?"
```

This form is more valuable than a grid of green boxes because it follows the actual architecture. It connects the user-facing state to the underlying service responsibility. It also prevents a dangerous support habit: blaming the last visible screen. If Reporting is stale, the dashboard should show whether Reporting is slow, Compliance is delayed, the process manager is waiting, or RootCause never published the event.

Drill-down without losing the story

A dashboard must allow drill-down, but the drill-down should preserve context. If an operator clicks a delayed compliance review, the next view should keep the correlation

id, incident id, service owner, last successful event, pending command, retry count, and user-facing freshness message. Jumping from a red tile to an unfiltered log search is not observability; it is archaeology.

Drill-down target	Must preserve
Trace view	CorrelationId, CausationId, EventId, service names, operation names, elapsed time.
Log view	Same correlation id, actor context where allowed, severity, structured event names.
Queue view	Topic/queue, partition, message age, delivery count, DLQ reason, contract version.
Process state view	Current stage, pending command, timeout deadline, escalation owner, last handled event.
Projection view	Last event checkpoint, lag seconds, rebuild state, freshness label shown to user.
Deployment view	Image version, rollout status, replica readiness, contract compatibility note.

Who owns the dashboard?

Operational dashboards often fail because nobody owns them. They are built during a project, admired during a demo, and forgotten until production pain returns. In NEXUS-1, the owner of a dashboard panel should match the owner of the responsibility it exposes. RootCause owns root-cause service health and verdict publication. Compliance owns review lag and failed review commands. Reporting owns projection freshness. Audit owns append latency and evidence ingestion. The platform owns broker, runtime, certificates, deployment, and shared telemetry infrastructure.

Dashboard area	Primary owner	Escalation partner
RootCause verdict flow	RootCause service owner	AlarmManagement, Compliance, Audit.
Compliance review lag	Compliance service owner	Process manager, broker/platform.
Reporting freshness	Reporting service owner	Source service owners whose events feed projections.
Audit append health	Audit service owner	All write-side service owners.
Broker and DLQ	Platform owner	Producing and consuming service owners.
Security failures	Security/platform owner	Affected service owner.
Operator-console warnings	BFF/UI owner	Owning backend service for stale or degraded data.

OWNERSHIP RULE
Every red panel should have an owner. If nobody owns a dashboard signal, nobody owns the failure it reveals.

Anti-patterns

The most common dashboard anti-pattern is visual noise: too many charts, too many colors, too many low-level counters, and no architectural story. The second anti-pattern is vanity: showing uptime and request volume while hiding lag, retries, dead letters, stale projections, and failed compensations. The third is disconnected drill-down: a red tile that sends the reader to unscoped logs instead of preserving correlation and causation.

Anti-pattern	Why it hurts	Better practice
Everything dashboard	Important signals drown in chart noise.	Layer dashboards by overview, flow, messaging, projections, security, deployment.
Green-box theatre	Service liveness hides business-flow failure.	Show flow health, lag, stuck stages, and user-facing freshness.
Unowned alerts	Pages fire but no team knows whether it is their responsibility.	Attach owner labels and escalation policy to each panel.
Log archaeology	Troubleshooting starts from raw text instead of preserved context.	Drill down by CorrelationId, CausationId, EventId, and service owner.
No freshness labels	Users read stale projections as current truth.	Expose projection lag and user-facing freshness state.
No contract visibility	Mixed versions and DLQ events become mysterious.	Show contract version, incompatible messages, and schema failures.

Chapter 59 checkpoint

CHECKPOINT
1. Why is an operational dashboard different from a reporting screen? 2. Why is a grid of green service boxes insufficient in a distributed architecture? 3. Which NEXUS-1 dashboard should follow the alarm-to-compliance flow, and what should it show? 4. Name four architecture-specific metrics that matter more than generic CPU and memory for this book. 5. Why must a drill-down preserve correlation id, causation id, event id, process stage, and service owner?

Part IX has now made NEXUS-1 observable and operable enough to discuss migration. The next part changes the question. Instead of asking how a service runs, it asks how a system already running as a modular monolith can be moved toward services without tearing the domain model, contracts, data ownership, and audit trail apart.

PART X

Migration Strategy

The previous parts have designed the service world: boundaries, ownership, contracts, messages, workflows, consistency, security, observability, and operations. Part X asks the practical question: how does a system get there without pretending it can be replaced in one heroic cutover?

Migration is not a second architecture. It is the path that protects the first one while runtime boundaries change. In NEXUS-1, the modular core already contains disciplined contexts: AlarmManagement, RootCause, Compliance, Audit, Reporting, DigitalTwin, and the Operator Console. The task is not to scatter them across the network. The task is to extract only where independence has become worth the cost.

WHAT PART X GIVES YOU
Part X turns readiness into movement. It explains how to move from a modular monolith to services, how to use a strangler approach, how to choose the first extraction, how to keep the old system alive, how to test contracts during migration, how to roll back safely, and how to express a NEXUS-1 migration roadmap without damaging the domain model.

PART X - MIGRATION STRATEGY

60. From Modular Monolith to Microservices

A microservice migration should not begin with deleting the monolith. It should begin with proving which parts of the modular core already have enough language, ownership, contracts, and operational discipline to survive outside it.

Why this chapter exists

The earlier chapters deliberately refused to treat microservices as a starting point. They first asked whether a bounded context is ready to become a service, who owns the data, what the public contracts are, how messages are delivered, how workflows survive partial failure, how users are told about delayed truth, how service trust is protected, and how operations can see what is happening. Only now is it safe to ask the migration question.

The migration from a modular monolith to microservices is not a change from bad architecture to good architecture. A disciplined modular monolith can be better than a poorly separated service architecture. The point of migration is not to prove that the monolith was wrong. The point is to move selected responsibilities behind runtime boundaries because the cost of keeping them together has become higher than the cost of separating them.

NEXUS-1 SENTENCE TO REMEMBER

A modular monolith is not a failure state. It is the safest place to learn boundaries before paying the distributed-system price.

The wrong migration story

The wrong story says: we have a monolith; therefore we must split it. That story is dangerous because it turns deployment topology into the goal. It produces teams that create many small APIs while still sharing the same database, leaking the same domain objects, hiding workflow state in controllers, and calling each other synchronously until the system is slower and less reliable than before.

Naive move	Why it looks attractive	Why it damages the system
Create one API per module	Each module now has an endpoint.	The module may still share tables, transactions, and internal domain language.
Copy the database schema per service	Each service appears independent.	Copied truth becomes divergent truth; ownership is still unclear.
Replace method calls with HTTP calls	The architecture looks distributed.	Latency, partial failure, retries, and timeouts appear without better boundaries.
Extract everything at once	The plan looks decisive.	Every bug becomes both a migration bug and a business bug.
Keep the UI unchanged by hiding many calls behind the BFF	The console keeps working.	The BFF becomes the new monolith if orchestration and domain rules leak into it.

The correct migration story

The correct story begins with a map. The modular monolith already has boundaries. Those boundaries must be checked against service-readiness criteria before they are moved. The migration is then performed one seam at a time: protect the contract, separate the data owner, publish the event, build the projection, run the new service beside the old path, compare behaviour, move traffic gradually, and keep a rollback path until the new owner is proven.

Migration sequence, expressed as architecture rather than tooling

1. Name the candidate boundary.

2. Prove the language and rule owner.

3. Freeze or publish the public contract.

4. Identify the data that the candidate owns.

5. Build the new service behind an adapter or route.

6. Run old and new paths side by side where possible.

7. Compare commands, events, projections, and audit records.

8. Move one flow, one user path, or one event consumer at a time.

9. Observe lag, errors, duplicate handling, and rollback readiness.

10. Remove the old path only after the new service has earned trust.

MIGRATION RULE

Extraction should reduce coupling that already exists in the model. It should not invent runtime separation where the model still depends on shared language, shared transactions, and shared data ownership.

NEXUS-1 as a migration candidate

NEXUS-1 is a useful teaching case because it has both strong modular boundaries and strong reasons not to distribute everything. ReactorFleet and Instrumentation contain foundational reference and signal structure. AlarmManagement produces events. RootCause owns the causal case and verdict. Compliance reacts to public facts. Audit records evidence. Reporting builds read models. The Operator Console composes views.

Some of these are plausible service candidates; some should remain close until their operational benefit is clear.

Context	Why it may move	Why it may stay for now
AlarmManagement	Owns alarm events, flood detection, and a clear public event stream.	It depends closely on instrumentation semantics and may need careful load/latency design.
RootCause	Owns RootCauseCase, evidence, hypotheses, verdicts, and canonical events.	It is central to the teaching narrative; extraction must not break auditability.
Compliance	Naturally reacts to issued verdicts and can lag without blocking the case owner.	Its review state must remain understandable to the operator console.
Reporting	Designed for projections and delayed truth; good candidate for independent read-side evolution.	It must not become a hidden source of truth.
Audit	Cross-cutting evidence infrastructure with append-only behaviour.	Premature extraction can turn every write into a remote dependency.
Operator BFF	Can protect the UI from service topology.	It can become a coordination dumping ground if not kept thin.

Three migration modes

A mature migration plan does not have only one tool. Some responsibilities are extracted as full services. Some are split first as read-side projections. Some are left inside the modular core and protected with clearer internal contracts. The decision should be explicit. Not extracting a context is a decision, not a weakness.

Mode	Meaning	NEXUS-1 example
Internal hardening	Keep the context inside the modular monolith but enforce stronger internal boundaries.	Keep ReactorFleet inside the core while removing direct table access from other contexts.
Read-side extraction	Move projections or query models first, while command ownership remains in the core.	Extract Reporting projections fed by RootCauseVerdictIssued.v1.
Full service extraction	Move command handling, data ownership, contracts, and operations behind a runtime boundary.	Extract Compliance review handling after RequestComplianceReview is stable.

The first extraction should be boring

The first service extraction should not be the most dramatic part of the system. It should be the part whose failure is easiest to understand, whose data ownership is clear, whose contract is stable, whose rollback path is possible, and whose operational signals are already visible. In NEXUS-1, that often points toward a read-side or downstream context before a central decision owner.

A tempting first extraction is RootCause because it is conceptually rich. A safer first extraction may be Reporting or Compliance because they can be eventually consistent

and downstream. Reporting can consume events and build projections without owning verdict truth. Compliance can react to a verdict and open a review without changing the verdict. RootCause, by contrast, owns central decision state; extracting it too early risks turning every surrounding context into a remote dependent before the migration discipline is proven.

PRACTICAL RULE

The first extracted service should teach the organization how to migrate safely, not prove architectural bravery.

Side-by-side operation

A safe migration often runs old and new paths together for a period. This does not mean both paths are allowed to write the same truth. It means one path remains authoritative while the other path shadows, compares, projects, or handles a controlled subset. Side-by-side operation is valuable only when the comparison criteria are explicit.

Side-by-side pattern	Allowed behaviour	Not allowed behaviour
Shadow consumer	New service consumes events and logs what it would do.	It must not change authoritative state.
Projection comparison	New read model is compared with old query output.	Users must not unknowingly read unverified data.
Route slice	A small class of requests is routed to the new service.	Traffic must not be split without correlation and rollback.
Contract compatibility run	Old and new producers/consumers validate the same event contract.	Contract mismatches must not be hidden behind adapter hacks.

Migration telemetry

A migration without telemetry is a cutover by faith. The previous part built logs, metrics, traces, health checks, resilience patterns, deployment visibility, and dashboards. During migration, those are not optional. They are the proof that the new boundary behaves as designed. The dashboard should show both old-path and new-path activity, contract failures, lag, duplicates, rollback status, and user-facing freshness.

```
Migration dashboard sketch

Candidate: Compliance service extraction
Authoritative path: modular core until cutover flag is enabled
Shadow path: new Compliance service consumes RootCauseVerdictIssued.v1

Old path reviews opened:      42
New path would open reviews:  42
Contract validation failures:  0
Duplicate event handling:      7 ignored safely
Average event-to-review lag:    4.2 s
Max event-to-review lag:       11.8 s
Rollback state:                ready
Cutover decision:              not yet - wait for 7 clean days
```

What must not change during migration

The most important migration discipline is knowing what must remain stable. The public language should not drift. RootCauseVerdictIssued.v1 should not be renamed because a new service boundary appears. A recommendation should still not become a command. AuditEntry should remain evidence, not workflow. A read model should still not become source of truth. The migration changes runtime ownership; it must not rewrite the meaning of the domain to make extraction easier.

Stable thing	Why it must stay stable
Ubiquitous language	Without stable language, migration becomes semantic drift.
Public event contracts	Consumers must survive the producer moving behind a new boundary.
Audit semantics	Evidence must remain replayable across old and new paths.
Source-of-truth ownership	A projection must not become truth just because it is easier to query.
Correlation discipline	Old and new paths must remain traceable as one business flow.
Rollback path	Safety requires a controlled return path until the new service is proven.

Chapter 60 checkpoint

CHECKPOINT
1. Why is a modular monolith not a failure state? 2. What is wrong with replacing method calls by HTTP calls without changing ownership? 3. Why should the first extracted service often be boring rather than central? 4. What is side-by-side operation, and what must it never allow? 5. Which NEXUS-1 meanings must remain stable while runtime boundaries change?

The next chapter introduces the strangler fig pattern. That pattern gives migration a shape: instead of replacing the old system in one step, the new service boundary grows around selected flows until the old path can be retired deliberately.

PART X - MIGRATION STRATEGY

61. Strangler Fig Pattern

A strangler migration does not ask the old system to disappear in one dramatic replacement. It lets new services grow around selected seams until traffic, ownership, and trust move gradually to the new boundary.

Why this pattern belongs here

Chapter 60 explained that the migration from a modular monolith to microservices should begin with boundaries, not enthusiasm. The strangler fig pattern is the migration shape that best matches that discipline. It does not require the whole NEXUS-1 core to be rewritten. It lets one capability be placed behind a new route, adapter, or facade while the old path remains available and measurable.

The name is useful because it describes the architecture honestly. In nature, a strangler fig grows around an existing tree. In software, the new system grows around an existing application. At first it is only a small external layer. Later it owns more traffic. Eventually some old paths can be removed. The important word is eventually. The pattern is not a shortcut. It is controlled replacement.

NEXUS-1 SENTENCE TO REMEMBER

The strangler pattern is not a rewrite strategy. It is a routing, ownership, and evidence strategy for moving one capability at a time.

The naive replacement trap

The trap is to believe that a service extraction begins by creating a new service and copying the old code. That creates two versions of the same truth with no clear cutover rule. The old module still works. The new service almost works. The database may still be shared. The UI may call both. A bug report then becomes hard to interpret because nobody knows which path produced the result.

Replacement move	Why it feels decisive	What goes wrong
Build the new service beside the old module	Work can start immediately.	There may be no route rule deciding which path is authoritative.
Copy tables into the new service database	The service appears independent.	Copied rows diverge unless ownership and synchronization rules are explicit.
Switch the UI directly to the new service	The console appears migrated.	Rollback becomes a UI problem and business behaviour is harder to compare.
Delete the old module early	The architecture looks clean.	The team loses the safest fallback path before production evidence exists.

The strangler shape

A disciplined strangler migration has four visible parts. First, an entry point decides whether a request goes to the old module or the new service. Second, the new service owns only the capability being extracted. Third, comparison and telemetry prove whether the new path behaves correctly. Fourth, rollback remains possible until the old path has no remaining reason to exist.

Part of the pattern	NEXUS-1 meaning	Rule
Entry point	API Gateway, BFF, or adapter route in front of the Operator Console flow.	The route must be observable and reversible.
New service	A candidate such as Reporting Projection Service or Compliance Review Service.	It owns a narrow capability, not a vague copy of a context.
Old path	The existing modular core path remains available during comparison.	It is deprecated only after evidence, not after optimism.
Evidence loop	Logs, correlation IDs, contract tests, projection comparison, and operator-visible status.	Migration decisions are made from observed behaviour.

A simple routing model

The strangler pattern is easiest to understand as a routing rule. A request reaches a stable entry point. The entry point checks the route policy. Some requests still go to the old modular core. Selected requests go to the new service. The client does not need to know that the architecture behind the route is changing.

```
// Conceptual routing logic, not a framework prescription.
public sealed class OperatorConsoleRoutePolicy
{
    public Target Resolve(RequestContext request)
    {
        if (request.Path == "/reporting/projections" &&
            FeatureFlags.ReportingServiceExtraction.Enabled &&
            request.UnitId is not null)
        {
            return Target.NewReportingService;
        }

        return Target.ModularCore;
    }
}
```

HONEST BOUNDARY

This code is intentionally small. Real routing would also consider authentication, tenant or plant partition, request type, failure state, circuit-breaker status, canary percentage, and rollback configuration. The architectural point is the explicit decision, not this exact implementation.

Choosing a strangler candidate

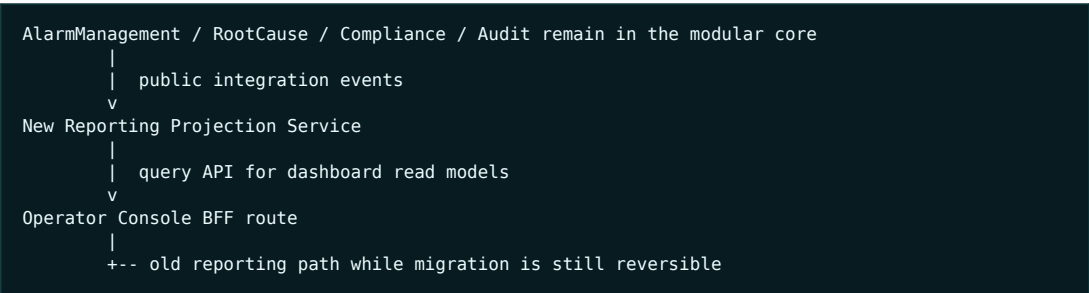
Not every NEXUS-1 capability is equally suitable for a strangler extraction. A good early candidate is useful but not life-or-death, has a clear public contract, can be compared against the old path, and can tolerate eventual consistency. A poor candidate sits inside the strongest consistency boundary, owns rules that are still changing, or requires too many synchronous calls back into the core.

Candidate	Strangler suitability	Reason
Reporting projections	High	Read-side output can be rebuilt, compared, and rolled back without changing command ownership.
Compliance review trigger	Medium to high	The event contract can be clear, but human review and audit evidence must be protected.
RootCause verdict issuing	Medium	Strong domain ownership exists, but correctness, evidence, and audit make extraction more sensitive.
Audit entry writing	Low as an early move	Audit is cross-cutting and must not become a fragile remote dependency for every command.
Operator Console UI composition	Medium	A BFF can route views, but it must not become the hidden workflow engine.

NEXUS-1 example: strangling reporting first

Suppose the first extraction is not RootCause itself but Reporting. The modular core continues to own commands, cases, verdicts, evidence, compliance review triggers, and audit entries. A new Reporting Projection Service subscribes to public integration events and builds read models for the Operator Console. The BFF can then route selected dashboard queries to the new service while keeping command flows inside the old core.

This is a conservative migration because it does not move the domain decision first. It moves a read-side capability that can be rebuilt from events and compared with existing reporting screens. If the new projection is wrong, the command truth remains intact. If the projection lags, the console can say so. If the service fails, the old reporting path can still be used during migration.



Data during a strangler migration

The most dangerous part of a strangler migration is data confusion. The new service must not quietly become a second owner of the same truth. During migration, it may hold projections, snapshots, caches, and replicated reference data. It must not accept writes that belong to the old owner unless the ownership decision has already moved.

Data type	Allowed in new service during early strangler phase?	Condition
Read projection	Yes	It is derived, rebuildable, and labelled as a projection.
Reference snapshot	Yes	It carries version and freshness metadata.
Copied command table	No	It creates duplicate ownership unless cutover is complete.
Audit copy	Only as read projection	The append-only audit owner remains explicit.
Integration event log	Yes	It supports replay, idempotency, and projection recovery.

Running old and new paths side by side

The strangler pattern becomes safe when old and new paths can be compared before the old path is retired. For a read-side extraction, comparison is straightforward: ask both paths for the same view, normalize harmless differences, and alert on meaningful divergence. For command-side extraction, comparison is harder because commands change state; shadow execution must never write twice.

Comparison level	Example	Safe use
Contract shape	Does the new API return the agreed fields and versions?	Run in CI and deployment gates.
Projection content	Does the new reporting view match the old read model for the same incident?	Run in shadow mode with sampled traffic.
Latency and availability	Does the new service meet the console expectation?	Observe before moving more traffic.
Operator meaning	Does the screen communicate delayed truth clearly?	Review with UI language and honest-boundary rules.

CHECKPOINT
A strangler path is not ready for traffic simply because it compiles. It is ready when the route is reversible, the contract is stable, the data owner is clear, the old path can still answer, and comparison tells the team what changed.

Feature flags and route percentages

A strangler migration often uses feature flags or route percentages. These are not cosmetic switches. They are operational controls. A flag can route only internal users first, then one unit, then one dashboard, then a percentage of requests. A percentage

can be increased only if health, latency, error rate, projection divergence, and support feedback remain inside agreed limits.

```
route reporting-dashboard:
  old_target: modular-core-reporting
  new_target: reporting-projection-service
  enabled_for:
    - internal-users
    - unit: U1
  traffic_percent: 10
  rollback_on:
    error_rate_above: 1.0%
    p95_latency_ms_above: 800
    projection_divergence_above: 0.5%
```

The exact mechanism might be an API gateway, a BFF, a service mesh, an application-level router, or a deployment platform. The architectural rule is independent of the tool: the movement of traffic must be explicit, observable, reversible, and owned.

When to retire the old path

The old path should not be removed when the new path first works. It should be removed when it has stopped carrying meaningful traffic, when the new owner has operated through normal incidents, when rollback no longer depends on the old code, when monitoring shows stable behaviour, and when the documentation says where the truth now lives.

Retirement question	Reason
Has the new path handled normal production load?	A successful demo is not operational proof.
Can support staff explain where the truth lives now?	Migration must change understanding, not only routing.
Are old database writes disabled or removed?	Leaving two write paths creates future corruption.
Are dashboards, alerts, and runbooks updated?	Operations must follow the new boundary.
Is rollback now a forward fix rather than a return to the old path?	Only then has the old path really become obsolete.

What must not be strangled

Some parts of the system should not be treated as easy strangler candidates. A domain invariant that protects correctness cannot be split across old and new paths casually. A transaction boundary cannot be half-moved without a compensation story. Audit cannot be reduced to an optional remote call. Security cannot be strangled by allowing two contradictory authorization models to coexist without a clear decision rule.

HONEST BOUNDARY

The strangler pattern is powerful because it slows migration down. If the team uses it only as a polite name for duplicating code and switching routes blindly, it becomes another form of rewrite risk.

Chapter checkpoint

Question	Expected answer
What is the strangler fig pattern?	A gradual replacement strategy where new paths grow around selected old paths until traffic and ownership move safely.
Why does it fit NEXUS-1?	NEXUS-1 already has modular boundaries and can move one capability at a time without replacing the whole core.
What is the safest early candidate?	Usually a read-side or projection capability, such as Reporting, because it is rebuildable and comparable.
What is the main data risk?	Creating two owners of the same truth during migration.
When can the old path be retired?	Only after traffic, evidence, monitoring, documentation, and ownership have moved to the new path.

PART X - MIGRATION STRATEGY

62. Extracting the First Service

The first extracted service is not the most exciting service. It is the service that can prove the migration method without putting the domain truth at risk.

The first extraction is an architectural exam

After the strangler pattern has created a safe route around the old system, the next question is practical: which service do we actually extract first? The answer should not be the service that sounds most important, or the one whose name is easiest to sell in a diagram. The first service is the proof that the team can move a capability without breaking language, ownership, contracts, data, observability, security, and rollback.

In NEXUS-1, the temptation would be to extract RootCause first, because it is the dramatic centre of the alarm-to-compliance flow. That would be possible later, but it is not the safest first move. RootCause owns evidence, hypotheses, verdicts, and a strong correctness story. A first extraction should teach the migration mechanics with lower risk. It should be narrow, observable, and reversible.

NEXUS-1 SENTENCE TO REMEMBER
The first service should prove the extraction method, not prove the ambition of the architecture.

Selection criteria for the first service

The first extracted service should satisfy seven conditions. It should have a clear business capability, a stable public contract, limited write authority, measurable output, a reversible route, manageable data ownership, and a failure mode that the operator console can explain. If one of those conditions is missing, the extraction may still be possible, but it should not be the first one.

Criterion	Question	Why it matters
Capability clarity	Can we describe the service in one sentence without using implementation words?	A vague service becomes a distributed module with no owner.
Contract stability	Are the requests, responses, and events known well enough to version?	Changing contracts during extraction multiplies risk.
Limited write authority	Can the service start with projections, requests, or workflow records rather than core truth?	The first cut should not split the strongest invariant.
Observable output	Can old and new results be compared?	Migration needs evidence, not only deployment success.
Rollback path	Can traffic return to the modular core quickly?	First extractions must be reversible.
Data ownership rule	Is every table classified as source truth, projection, snapshot, or reference copy?	This prevents two services from owning the same fact.
User-facing explanation	Can the console explain delay or fallback honestly?	Distributed truth changes the user experience.

The recommended first NEXUS-1 service

For this book, the safest first extraction is the Reporting Projection Service. It is useful, visible, and technically meaningful, but it does not issue verdicts, mutate root-cause cases, command the plant, or write the authoritative audit trail. It consumes events and builds read models. That makes it an ideal first service because it teaches integration, idempotency, schema ownership, operational monitoring, and UI routing without moving the most sensitive domain decisions first.

Candidate	First extraction rating	Reason
Reporting Projection Service	Best first move	Read-side, rebuildable, comparable, and visible in the console.
Compliance Review Service	Good second candidate	Meaningful workflow ownership, but it touches human review and evidence.
RootCause Service	Later candidate	Strong domain value, but verdict correctness makes first extraction too risky.
Audit Service	Not first	Cross-cutting and too important to turn into an early remote dependency.
Alarm Management Service	Later candidate	High-volume event and operator visibility make it operationally sensitive.

Define the service before creating the project

A service extraction does not begin with a new repository, a Dockerfile, or a Kubernetes manifest. It begins with a service definition. The definition states what the service owns, what it consumes, what it exposes, what it refuses to own, how it fails, and how it can be

rolled back. Without that definition, the project structure will look professional while the boundary remains confused.

Service definition field	Reporting Projection Service answer
Purpose	Build read models for dashboards, exports, and operator views from public integration events.
Owns	Projection tables, projection rebuild state, event-consumption checkpoints, read-side query API.
Does not own	Root-cause verdict truth, alarm truth, compliance decision truth, audit truth.
Consumes	RootCauseVerdictIssued.v1, AlarmFloodDetected.v1, RequestComplianceReview, RootCauseVerdictCorrected.v1.
Publishes	ProjectionRebuilt.v1, ProjectionLagDetected.v1, ReportingProjectionFailed.v1, if these events are needed by operations.
Failure behaviour	Show stale or unavailable projection status; do not block command flows.
Rollback	Route console reporting queries back to the modular core read model.

The first project shape

The first service should be small enough that its structure can be inspected by a human. For NEXUS-1, the Reporting Projection Service can use the same clean boundary discipline as the backend trilogy: domain language inside, application handlers around it, infrastructure for persistence and messaging, and an API adapter for queries. The point is not to reproduce every layer mechanically; the point is to keep dependencies readable.

```
Nexus.ReportingProjectionService/  
  src/  
    ReportingProjection.Api/  
      Controllers/  
      Contracts/  
      Program.cs  
    ReportingProjection.Application/  
      EventHandlers/  
      Queries/  
      ProjectionRebuild/  
    ReportingProjection.Domain/  
      ProjectionState.cs  
      DashboardReadModel.cs  
      ProjectionLag.cs  
    ReportingProjection.Infrastructure/  
      Persistence/  
      Messaging/  
      Idempotency/  
  tests/  
    ReportingProjection.ContractTests/  
    ReportingProjection.ApplicationTests/  
    ReportingProjection.IntegrationTests/
```

HONEST BOUNDARY

This is a service skeleton, not a deployment proof. Chapter 62 is about extracting the first service boundary. Production packaging, scaling, secret management, and platform deployment remain operational concerns, not reasons to skip the boundary definition.

Move read responsibility, not domain authority

The first extraction must not accidentally move authority. Reporting may display a root-cause verdict, but it does not issue one. It may display compliance-review status, but it does not decide compliance. It may show audit evidence, but it does not become the audit owner. This distinction is the same language discipline used throughout the NEXUS-1 series: a read model is not the source of truth.

Displayed by reporting	Authoritative owner	Rule
Current verdict text	RootCause	Reporting stores a projection of RootCauseVerdictIssued.v1.
Alarm flood summary	AlarmManagement	Reporting displays a derived view, not an alarm ledger.
Compliance review status	Compliance	Reporting displays the state; it does not trigger or close review by itself.
Audit trail excerpt	Audit	Reporting can query or project evidence, but Audit remains append-only truth.
Projection freshness	Reporting Projection Service	This is the service own truth: how current its own read model is.

The event contract boundary

The first service should consume the canonical public contracts already established by the previous volume. The service must not subscribe to internal domain events by reaching into another model. It consumes integration events, validates versioned payloads, and treats unknown versions as something to reject or quarantine, not as something to guess.

```
public sealed record RootCauseVerdictIssuedV1(
    Guid EventId,
    string CaseId,
    string UnitCode,
    string VerdictCode,
    string VerdictText,
    DateTime IssuedAtUtc,
    string IssuedBy,
    string CorrelationId,
    int SchemaVersion = 1);

public sealed class RootCauseVerdictIssuedConsumer
{
    public async Task Handle(RootCauseVerdictIssuedV1 message)
    {
        await inbox.EnsureNotProcessed(message.EventId);
        await projection.Apply(message);
        await inbox.MarkProcessed(message.EventId);
    }
}
```

Notice the deliberate absence. The event does not carry the full RootCauseCase aggregate. It does not expose private hypothesis structures. It does not ask reporting to decide whether the verdict is valid. It carries the public fact required to build a read model.

Data split for the first service

The Reporting Projection Service needs its own database, but that database should not be mistaken for a copy of the old one. It contains projection tables, checkpoints, idempotency records, and projection metadata. It may contain replicated reference snapshots, but those snapshots must carry freshness and source information. It must not contain command tables copied from the modular core.

Table category	Example table	Ownership meaning
Projection	Reporting.CaseDashboardProjection	Owned by the reporting service and rebuildable from events.
Checkpoint	Reporting.EventConsumerCheckpoint	Owned by the service; records how far each consumer has progressed.
Inbox	Reporting.ProcessedMessage	Owned by the service; protects idempotency.
Reference snapshot	Reporting.UnitReferenceSnapshot	Copy with version and last refreshed time.
Forbidden copy	RootCause.RootCauseCase	Not owned here; would duplicate domain truth.

Cutover sequence

The extraction should move through visible stages. First, create the service without routing production traffic to it. Second, feed it events in shadow mode. Third, compare its projections with the old reporting path. Fourth, route a narrow read endpoint through the BFF. Fifth, increase traffic only when monitoring and support evidence agree. Finally, retire the old reporting path when it has no remaining ownership.

Stage	Action	Exit condition
1. Build	Create service skeleton, contracts, projection tables, inbox, and health checks.	Service runs locally and contract tests pass.
2. Shadow	Consume real or replayed events while the old path still serves users.	Projection divergence is measured and understood.
3. Compare	Run old and new read models side by side for selected cases.	Differences are either fixed or documented as intentional.
4. Route	Send one dashboard query through the BFF to the new service.	Rollback switch is tested.
5. Expand	Increase traffic by unit, screen, or user group.	Latency, errors, lag, and support signals remain acceptable.
6. Retire	Remove old reporting route and old projection writer.	Ownership and runbooks point only to the new service.

The BFF route during extraction

The Operator Console should not be forced to understand every migration detail. A Backend-for-Frontend or gateway route can hide the temporary split while still exposing freshness and fallback information to the UI. The route decides whether a query goes to the old modular core or the new projection service; the response still tells the user whether the data is current, stale, rebuilding, or unavailable.

```
GET /operator-dashboard/root-cause-summary?caseId=RC-2026-0418

BFF route policy:
  if ReportingProjectionService is healthy
    and projection lag < configured threshold
    and feature flag enabled for this unit:
      call new Reporting Projection Service
  else:
    call modular core reporting path
    mark response as fallbackUsed = true
```

OPERATOR-CONSOLE RULE
The user does not need to see the migration topology, but the user does need honest status. If the read model is stale, rebuilding, or served through fallback, the console should say so.

Failure modes of the first service

A first service extraction is successful only when its failure modes are known. The new service may fail to consume events, build a wrong projection, fall behind, reject a contract version, lose database connectivity, or return slower than the console can tolerate. These failures should not be discovered by operators first. They should be designed into health checks, dashboards, alerts, and rollback policy.

Failure	Visible symptom	Expected response
Consumer stopped	Projection lag grows.	Alert operations; route reads back to old path if freshness threshold is crossed.
Duplicate event	Same message arrives twice.	Inbox prevents double projection changes.
Unknown event version	Consumer rejects payload.	Move to quarantine/dead-letter path and alert contract owner.
Projection bug	New read model differs from old path.	Stop traffic expansion; repair and replay projection.
Service unavailable	BFF call fails.	Fallback to modular core while migration is active.

What counts as done

The first service is not done when it has a repository and an endpoint. It is done when the team can operate it. That means contracts are versioned, events are consumed idempotently, projections can be rebuilt, health checks distinguish live from ready, dashboards show lag and failure, rollback has been rehearsed, and the old ownership has been removed or clearly marked as legacy.

Done condition	Evidence
Contract is public and versioned	RootCauseVerdictIssued.v1 and query DTOs are tested as contracts.
Projection is rebuildable	A rebuild can recreate the dashboard state from the event log.
Idempotency is proven	Duplicate events do not change the projection twice.
Routing is reversible	BFF can return to modular core without code changes.
Operations can see the service	Dashboards show lag, health, errors, latency, and dead-letter count.
Ownership is documented	Service catalog says what Reporting owns and what it only displays.

Chapter checkpoint

Question	Expected answer
Why should the first service be conservative?	Because it proves the extraction method before sensitive domain authority is moved.
Which NEXUS-1 service is the recommended first extraction here?	Reporting Projection Service.
What must it not own?	Root-cause verdict truth, alarm truth, compliance truth, or authoritative audit truth.
Why does it need an inbox?	Because messages may be delivered more than once and projection updates must be idempotent.
What makes the extraction safe?	A reversible route, stable contracts, clear data ownership, comparison with the old path, monitoring, and rollback.

PART X - MIGRATION STRATEGY

63. Keeping the Old System Alive During Migration

A migration is not successful because a new service exists. It is successful when the old system can keep serving truth while responsibility moves, one seam at a time.

Why the old system must remain alive

After the first service is extracted, the most dangerous sentence a team can say is: now we can switch everything over. That sentence treats extraction as replacement. In a serious system, extraction is only the beginning of a period where the old modular core and the new service world must coexist. During that period, the old system remains the safety net, the comparison point, the rollback target, and often the source of truth for capabilities that have not yet moved.

NEXUS-1 makes this especially important because the alarm-to-compliance flow is not a casual workflow. It contains alarms, evidence, root-cause verdicts, recommendations, audit entries, compliance review, and reporting projections. Even in a demonstrator, the architecture must show the right discipline: new services may grow around the old core, but the old core cannot be abandoned until every responsibility has been deliberately moved.

MIGRATION RULE
Do not weaken the old system in order to make the new system look successful. The old system stays alive until the new ownership has proven it can carry the same responsibility.

Coexistence is an architectural state

Coexistence is not a messy temporary accident. It is a designed state of the system. The old modular core and the new services must both be understandable, routable, observable, and supportable. The team must know which capability lives where today, which one is moving next, which route is active, which database owns truth, and which fallback path will be used if the new path fails.

Coexistence concern	Question to answer	NEXUS-1 example
Route ownership	Which path serves this request today?	The Operator Console may call the BFF, which routes reporting reads to the new service or old core.
Truth ownership	Which store owns the authoritative fact?	RootCause still owns verdict truth while Reporting owns its projection freshness.
Fallback	Where does traffic go when the new path fails?	Dashboard queries can return to the modular core read path during migration.
Comparison	How do we prove old and new agree?	Projection output is compared for selected RootCauseCase views.
Support	Who investigates a production issue?	Runbooks identify whether a symptom belongs to routing, event consumption, projection, or old core logic.

Keep the old route boring

The old route should become boring, not neglected. During migration, the old path should receive only safe maintenance: bug fixes, security fixes, compatibility shims, observability improvements, and small refactorings that reduce extraction risk. It should not keep receiving major new domain behaviour that belongs to the service being extracted. Otherwise the target keeps moving and the migration becomes a race against the old system itself.

Allowed in the old core during migration	Avoid in the old core during migration
Security patches and dependency updates.	New ownership rules for a capability already assigned to a service.
Bug fixes that preserve existing behaviour.	New public contracts that should belong to the extracted service.
Logging, metrics, and correlation improvements.	New reporting projections once Reporting has been extracted.
Compatibility adapters needed for the strangler route.	Direct database access from new services into old internal tables.
Tests that document legacy behaviour before extraction.	Silent rewrites that change old behaviour without comparison evidence.

A legacy system is not automatically wrong

The word legacy is often used carelessly. In this book, legacy does not mean bad. It means already carrying responsibility. The modular core may be less distributed, less independently deployable, and less operationally refined than the target architecture, but it may still be the place where the domain rules are most complete. Until the extracted service proves otherwise, the old system deserves respect.

HONEST BOUNDARY

Keeping the old system alive is not an excuse to avoid migration. It is the discipline that lets migration happen without gambling with truth.

The compatibility layer

A compatibility layer protects both sides. It lets the new service talk to the old system through explicit contracts instead of reading old internals. It lets the old system publish integration events without exposing private domain objects. It lets the BFF route requests without forcing the operator console to know whether a capability has moved. It is not glamorous, but it is what prevents migration from becoming a hidden coupling exercise.

```
// A compatibility adapter keeps old internals behind a deliberate seam.
public interface ILegacyRootCauseReadPort
{
    Task<RootCauseSummaryDto?> GetSummaryAsync(
        string caseId,
        CancellationToken cancellationToken);
}

public sealed class LegacyRootCauseReadAdapter : ILegacyRootCauseReadPort
{
    public Task<RootCauseSummaryDto?> GetSummaryAsync(
        string caseId,
        CancellationToken ct)
    {
        // Calls the old modular-core API or an approved legacy query seam.
        // It does not expose RootCauseCase internals to the new service.
        return legacyClient.GetRootCauseSummaryAsync(caseId, ct);
    }
}
```

The adapter is deliberately narrow. It does not grant the new service permission to browse every old table. It gives the migration one approved path. That is the difference between coexistence and uncontrolled coupling.

Feature flags and route switches

The old system stays alive because routing can change without redeploying everything. Feature flags allow the team to choose which unit, screen, tenant, user group, or capability uses the new service. Route switches allow fast rollback when the new path is unhealthy. These switches should be operational controls, not scattered if-statements hidden inside application code.

Switch	Purpose	Example
Capability flag	Enable the new service for one feature.	Use ReportingProjectionService for root-cause summary.
Unit flag	Limit migration to selected plant units.	Enable for U1 and U2 only.
User-group flag	Expose to a small operator or tester group.	Enable for architecture-test users.
Freshness threshold	Fallback when the projection is too stale.	If projection lag exceeds 60 seconds, route to old core.
Kill switch	Disable the new route immediately.	Return all reporting reads to modular core.

OPERATOR-CONSOLE RULE
The route may change behind the scenes, but the response must remain honest. If fallback was used, if data is stale, or if a projection is rebuilding, the console should display that state.

Shadow mode keeps the old system in charge

Shadow mode is the safest coexistence technique. The old system still serves the user, while the new service receives the same inputs and produces output for comparison. The new output is not trusted by users yet. It is trusted only as test evidence. This is how a team learns whether the new service is ready without putting the user in the middle of the experiment.

Stage	Old system role	New service role	Decision rule
Shadow consume	Serves all users.	Consumes events and builds projections.	No user traffic moves yet.
Compare	Provides current read model.	Produces equivalent read model.	Differences are measured and classified.
Narrow route	Remains fallback.	Serves one low-risk route.	Rollback is tested before expansion.
Expanded route	Serves only fallback and not-yet-migrated paths.	Serves selected dashboards or users.	Freshness, latency, and correctness remain within threshold.
Retire path	Stops serving the moved capability.	Owns the route fully.	Old code is removed or made explicitly legacy.

Data during coexistence

The hardest coexistence mistakes usually involve data. If both old and new systems write the same truth, the migration has created a split-brain domain. If the new service reads old tables directly, it has copied the monolith boundary into the network. If reference data is replicated without version and freshness, the system will eventually disagree without knowing why. Coexistence needs a data rule for every table touched by the migration.

Data type	During coexistence	Rule
Authoritative domain table	Remain owned by old core until explicit ownership transfer.	Do not write from both systems.
Projection table	May be owned by new service.	Rebuildable from events; not source truth.
Reference snapshot	May be copied to new service.	Carry source version and refreshed timestamp.
Inbox/outbox table	Owned locally by the service that uses it.	Do not share message processing tables.
Audit evidence	Keep authoritative ownership clear.	A service may emit audit facts, but Audit owns the append-only record.

Do not let the old system become invisible

A common migration failure is that the new service gets dashboards while the old system becomes a black box. That is backwards. During coexistence, the old path must be at least as observable as the new path because it is still part of the production architecture. If traffic falls back to the old core, operations must see it. If the old core publishes events slowly, the projection service must expose lag. If an adapter call fails, the trace must show where the failure crossed the boundary.

```
Trace example during coexistence:

OperatorConsole request
-> BFF route decision
  -> ReportingProjectionService query
    -> Projection freshness check failed
  -> fallback to ModularCore.Reporting endpoint
    -> LegacyRootCauseReadAdapter
  -> response: fallbackUsed = true, freshness = legacy-current

The trace is not decoration. It explains why the user saw old-core data.
```

NEXUS-1 coexistence map

For NEXUS-1, the coexistence map should be printed before each extraction wave. It makes the migration visible enough to discuss. It shows which capabilities still live in the modular core, which have moved, which are shadowing, and which remain intentionally untouched. This map prevents the architecture from becoming a collection of partial memories.

Capability	Migration state	Current owner	Next action
Reporting dashboard read models	Extracted / expanding	Reporting Projection Service	Increase route coverage after comparison.
Root-cause verdict issuance	Not moved	Modular core RootCause	Prepare contracts, but do not extract yet.
Compliance review workflow	Candidate	Modular core Compliance	Shadow event-driven trigger first.
Audit append-only trail	Not moved	Audit sector in modular core	Keep authoritative until audit service plan is complete.
Operator Console composition	BFF-routed	Operator BFF	Hide topology while surfacing freshness and fallback.

When to retire the old path

The old path should be retired only when the new path has proven ownership, not merely when it has handled traffic. For a read-side service, this means the projection can be rebuilt, compared, monitored, and supported. For a command-side service, the bar is higher: invariants, authorization, audit, idempotency, compensation, rollback, and operational support must all be proven. Retirement is a decision, not a cleanup task.

Retirement question	Required evidence
Can the new path serve all intended reads or commands?	Traffic history, error rate, latency, and support review.
Can the new data be rebuilt or recovered?	Replay or restore procedure tested.
Can operations diagnose failures?	Dashboards, alerts, traces, and runbook exist.
Can rollback still happen if needed?	Rollback procedure tested until the retirement point; after retirement, recovery plan replaces it.
Has ownership been updated?	Service catalog, documentation, and on-call responsibility reflect the new owner.

Chapter checkpoint

Question	Expected answer
Why must the old system remain alive during migration?	Because it still carries truth, fallback, comparison, and support responsibilities until ownership moves.
What is coexistence?	A designed state where old and new paths are routable, observable, supportable, and governed by ownership rules.
Why is shadow mode useful?	It lets the new service produce evidence without serving users yet.
What is the main data danger?	Two systems writing the same authoritative truth.
When can an old path be retired?	When the new owner has proven contracts, data recovery, operations, support, and responsibility transfer.

PART X - MIGRATION STRATEGY

64. Contract Tests During Extraction

A contract test is the migration agreement made executable. It protects the old system, the new service, and the client from discovering a broken promise only after traffic has moved.

Why extraction needs executable contracts

During extraction, a team is not merely moving code from one project to another. It is replacing an internal call with a public boundary. What used to be protected by compilation, shared memory, one database transaction, and local refactoring now has to survive a network, versioning, serialization, authorization, retries, and independent deployment. A contract test is the discipline that says: this boundary is not a conversation in a meeting; it is an executable promise.

In NEXUS-1, this matters because the first extracted services are not decorative. Reporting projections, compliance views, root-cause summaries, audit traces, and operator-console composition all depend on names, shapes, meanings, and timing. If one service silently changes the meaning of status, freshness, verdict, or recommendation, the console may still render a screen - but the screen will no longer mean what the operator thinks it means.

MIGRATION RULE
Every boundary that is used during extraction must have a testable contract before traffic is moved through it. A route switch without a contract test is a guess with a feature flag.

The contract is not the implementation

A common mistake is to test the provider by calling its current implementation and declaring the contract safe. That proves only that today's code behaves like today's code. A contract test should describe what the consumer is allowed to rely on and what the provider must continue to supply. The provider may refactor, split storage, change indexes, move projection code, or change its deployment model. The contract should still hold.

Thing	What it describes	What it must not become
Implementation	How the provider currently produces the response.	The public promise consumed by other services.
API contract	Request path, method, schema, status codes, errors, version.	A dump of internal entities.
Event contract	Public event name, version, payload, semantics, compatibility rules.	A serialized domain event.
Consumer expectation	The subset of the contract the consumer actually needs.	A demand to freeze everything forever.
Contract test	Executable verification that the promise still holds.	A broad integration test that hides the boundary.

Consumer-driven contracts

A provider can publish a contract, but consumers know which parts matter. The operator console may care about freshness and display status. Compliance may care about verdict identity and audit evidence. Reporting may care about stable projection fields. A consumer-driven contract captures those expectations explicitly. The provider then verifies, before release, that its current implementation still satisfies the consumers it has promised to serve.

Consumer	Contract expectation	Why it matters
Operator BFF	Root-cause summary contains case id, verdict status, freshness, and fallback marker.	The screen must explain whether the displayed truth is current, stale, or fallback.
Compliance service	RootCauseVerdictIssued.v1 includes verdict identity, issued time, issuing context, and evidence reference.	A review must be triggered from the correct public fact, not an internal case state.
Reporting projection	Event ordering metadata and idempotency key are present.	The read model must ignore duplicates and rebuild safely.
Audit service	Correlation id and causation id are carried across calls/events.	The trail must be replayable across service boundaries.

REST contract tests

For synchronous APIs, contract tests should verify more than a happy-path JSON example. They should verify route, method, required headers, accepted versions, successful response shape, expected error shape, authorization assumptions, and compatibility rules for optional fields. They should also verify what is deliberately absent. If the RootCause API returns the full RootCauseCase aggregate, the API has leaked the domain model. A contract test should catch that by asserting the public DTO shape.

```
// Consumer expectation: the Operator BFF needs a stable summary DTO.
[Fact]
public async Task RootCause_summary_contract_is_stable()
{
    var response = await client.GetAsync(
        "/api/root-cause/cases/RC-2026-0418/summary",
        headers: new { Accept = "application/vnd.nexus.rootcause-summary.v1+json" });

    response.StatusCode.Should().Be(200);
    response.Json.Should().ContainProperties(
        "caseId", "verdictStatus", "summary", "freshness", "correlationId");

    response.Json.Should().NotContainProperties(
        "_evidence", "domainEvents", "rowVersion", "internalHypotheses");
}
```

PUBLIC API DISCIPLINE

A contract test should protect both sides: consumers receive what they need, and providers are protected from consumers reaching into internals they were never promised.

Event contract tests

Event contracts are stricter than many teams expect because an event may outlive the version of the service that published it. If RootCause publishes RootCauseVerdictIssued.v1, consumers may store, replay, and project that event long after the RootCause service has changed internally. The test must therefore verify not only the JSON shape, but the meaning of the event: this is a public verdict fact, not a private notification that a case happened to close.

Event contract rule	Test should verify
Name is canonical	RootCauseVerdictIssued.v1 is used; RootCauseCaseClosed is not substituted as the public event.
Version is explicit	The version appears in the type name, topic, envelope, or schema metadata.
Payload is public	Only stable identifiers, timestamps, verdict fields, evidence references, and correlation metadata are exposed.
Idempotency is possible	Event id or deterministic idempotency key exists.
Ordering assumptions are visible	Sequence, causation id, or occurred-at semantics are explicit where required.

```
// Event contract sketch for a public integration event.
public sealed record RootCauseVerdictIssuedV1(
    string EventId,
    string CorrelationId,
    string CausationId,
    string CaseId,
    string UnitId,
    string VerdictCode,
    DateTime IssuedAtUtc,
    IReadOnlyList<string> EvidenceReferenceIds);

// This is not the RootCauseCase aggregate. It is a published fact.
```


Backward compatibility rules

Contract tests are also where compatibility rules become enforceable. A service can usually add an optional response field without breaking consumers. It cannot rename `caseld` to `rootCauseCaseldIdentifier` without a versioned transition. It cannot change a status code from 404 to 200 with an empty body unless consumers have agreed. It cannot turn a timestamp from UTC into local time because the value still looks like a date. Compatibility is not a feeling; it is a set of rules that tests can check.

Change	Compatibility judgment	Required discipline
Add optional response field	Usually safe.	Do not require consumers to understand it.
Remove existing field	Breaking.	New version or staged deprecation.
Rename field	Breaking.	Keep old field or version contract.
Change enum meaning	Breaking even if the string remains valid.	Version, migration note, and consumer tests.
Add new enum value	Potentially breaking.	Consumers must handle unknown values or contract must version.
Change timestamp semantics	Breaking.	Keep UTC semantics explicit.
Change error shape	Breaking for clients that explain errors.	Version or maintain compatible envelope.

Contracts during strangler migration

In a strangler migration, the same capability may be reachable through the old core and the new service for a while. Contract tests should run against both where appropriate. That does not mean old and new implementations must be identical internally. It means that, from the consumer boundary, they must return equivalent public meaning. The BFF route switch should be able to choose old or new without forcing the operator console to change its interpretation.

Migration stage	Contract test purpose	Failure meaning
Before extraction	Capture the old public behavior that consumers rely on.	The team does not know what must be preserved.
Shadow mode	Compare new-service output with old-route output.	New service is not semantically equivalent yet.
Partial traffic	Verify contract before each route expansion.	A consumer may be sent to a route it cannot safely read.
Old path retirement	Confirm new service satisfies every required consumer contract.	Retirement would remove the last known safe path.

Schema contracts for projections

A reporting projection has a contract even when it is not exposed as a public API. Dashboards, exports, and compliance views may depend on columns, meaning,

freshness, and rebuild behaviour. During extraction, projection schema tests should verify that read models remain rebuildable, nullable fields are intentional, and source metadata is preserved. A projection that looks correct but cannot explain when it was built is not a trustworthy read model.

```
-- Projection contract checks are simple but valuable.
SELECT ProjectionName, SourceEventType, SourceVersion,
       LastRebuiltAtUtc, MaxSourceOccurredAtUtc, LagSeconds
FROM Reporting.ProjectionStatus
WHERE ProjectionName = 'RootCauseSummary';

-- Contract expectation:
-- 1. Source version is recorded.
-- 2. Rebuild time is known.
-- 3. Lag is measurable.
-- 4. Missing freshness is a failure, not an empty string.
```

NEXUS-1 contract-test matrix

The extraction plan should keep a small matrix of contracts. This matrix is not bureaucracy. It is the evidence that the architecture knows which promises exist and who is protected by them. For NEXUS-1, the matrix starts around the alarm-to-compliance path because that is where the strongest cross-service meaning sits.

Boundary	Contract type	Protected consumer	Minimum proof
BFF -> Reporting service	REST DTO	Operator Console	Summary, freshness, fallback marker, stable errors.
RootCause -> Compliance	Integration event	Compliance workflow	RootCauseVerdictIssued.v1 payload and idempotency.
RootCause -> Audit	Audit fact/event	Audit service	Correlation, causation, actor, and evidence reference.
AlarmManagement -> RootCause	Command/event pair	RootCause service	AlarmFloodDetected.v1 and OpenRootCauseCase semantics.
Reporting -> Export	Projection schema	Report/export generator	Rebuildable read model and source freshness metadata.

Where contract tests belong in the pipeline

Contract tests should run before deployment, before route expansion, and before old-path retirement. They should not be reserved for a late end-to-end test environment where every failure is expensive to understand. Provider verification should happen in the provider pipeline. Consumer expectations should be versioned with the consumer. Compatibility checks should be fast enough to run often and strict enough to stop a release when a public promise is broken.

Pipeline point	Test responsibility
Consumer build	Publish or validate consumer expectations.
Provider build	Verify provider still satisfies registered consumer contracts.
Pre-release gate	Check breaking changes, event schema compatibility, and API version rules.
Route expansion gate	Run contract tests against the route that will receive traffic.
Retirement gate	Confirm no remaining consumer depends on the old contract path.

HONEST BOUNDARY

Contract tests do not prove the whole system is correct. They prove that a boundary promise has not silently changed. Integration tests, resilience tests, security tests, and operational checks still have their own work to do.

Chapter checkpoint

Question	Expected answer
Why are contract tests essential during extraction?	Because an internal call becomes a public boundary that must survive independent deployment.
What does a consumer-driven contract protect?	The subset of provider behavior that the consumer is allowed to rely on.
Why is an event contract more than JSON shape?	Because event name, version, meaning, idempotency, ordering metadata, and compatibility rules matter.
What is a breaking change even if code still compiles?	Renaming fields, changing meanings, removing fields, changing error shape, or changing time semantics.
When should contract tests run?	Before provider release, route expansion, and old-path retirement.

PART X - MIGRATION STRATEGY

65. Rollback and Safe Deployment

A safe deployment is not the one that never fails. It is the one whose failure path was designed before release, tested before traffic moved, and recorded well enough that the team can explain what happened afterwards.

Rollback is an architectural concern

Rollback is often discussed as a deployment feature: click the previous version, redeploy an older container, switch the route back, or restore a package. In a microservice migration, that is not enough. The application may already have published events, written rows, updated projections, invalidated caches, advanced workflow state, or taught users that a new route is available. Rolling back a binary is only one part of recovery. The architecture must know which facts can be reversed, which can only be compensated, and which must remain as audit evidence.

This distinction is especially important in NEXUS-1 because the alarm-to-compliance path is evidence-bearing. A root-cause verdict, a compliance review request, an audit entry, and a reporting projection are not all the same kind of output. Some can be rebuilt. Some can be superseded. Some must never be removed. The deployment plan must respect those differences.

DEPLOYMENT RULE
Do not ask whether the new service can be deployed. Ask first what will happen if it must be removed, bypassed, paused, or corrected while messages, projections, and users are already in motion.

Rollback, roll-forward, and compensation

The word rollback is overloaded. In a monolith, rollback often means returning the whole application and database to a previous release state. In a distributed system, that is rarely possible. A service may need to roll back its binary, roll forward with a corrective release, pause consumption, replay events, rebuild a projection, or publish a correction event. The safest recovery path depends on the type of change and the type of truth involved.

Recovery action	What it means	Where it fits
Binary rollback	Run the previous service version.	Safe for code-only changes when data and contracts remain compatible.
Route rollback	Send traffic back to the old path or modular core.	Useful during strangler extraction and BFF route switching.
Roll-forward	Deploy a corrective new version.	Best when old versions cannot understand already-written data.
Consumer pause	Stop reading from a queue/topic temporarily.	Useful when downstream effects would amplify a problem.
Projection rebuild	Recreate a read model from source events.	Safe for reporting and dashboard views when source truth is intact.
Compensation event	Publish a correcting public fact.	Required when an emitted fact must be superseded, not deleted.

The three compatibility gates

Safe deployment depends on compatibility in three places: API contracts, event contracts, and data shape. A release may pass unit tests and still be unsafe because the previous version cannot read the new data, a consumer cannot understand the new event, or the BFF cannot display the new error envelope. These gates must be checked before route expansion, not after an incident.

Gate	Question before release	Safe answer
API contract	Can existing consumers still call the service and understand responses?	Yes, or the change is versioned and routed explicitly.
Event contract	Can existing consumers ignore, accept, or version the event change?	Yes, and idempotency keys still work.
Data compatibility	Can old and new service versions coexist with the stored data?	Yes, or rollback is replaced by roll-forward.

HONEST WARNING

The most dangerous release is the one that looks reversible because the container image can be rolled back, while the data and messages it produced are not reversible.

Expand-and-contract deployment

A database or schema change used by an extracted service should usually follow an expand-and-contract pattern. First add what the new service needs without breaking the old path. Then deploy code that can use both old and new shapes. Then move traffic. Only after the system has run safely should the obsolete shape be removed. This is slower than a single migration, but it preserves the ability to retreat during the dangerous middle phase.

```
// Release sequence for a compatible extraction change.
1. Expand: add nullable column, new table, or new projection metadata.
2. Dual-read / dual-write where required, with explicit audit markers.
3. Deploy new service version behind a disabled route flag.
4. Verify contract tests, health checks, and shadow comparison.
5. Move a small slice of traffic through the new route.
6. Observe errors, lag, duplicate handling, and business meaning.
7. Expand traffic or route back.
8. Contract only after the old path is no longer needed.
```

Feature flags are not safety by themselves

Feature flags and route switches are essential during migration, but they are not magic. A flag can redirect requests, disable a new panel, or choose the old implementation. It cannot automatically undo messages that have been published, rows that have been written, or human decisions that have already been made from a screen. A flag is safe only when its downstream effects have been classified.

Flag type	What it controls	Risk if misunderstood
Route flag	Old path vs new service for a request.	Assumes both paths have equivalent public meaning.
Read flag	Old projection vs new projection.	Can hide freshness or source differences.
Publish flag	Whether a new integration event is emitted.	Can create split-brain if some consumers expect it and others do not.
UI flag	Whether a console panel displays a new view.	Can hide an unsafe backend state rather than fix it.
Worker flag	Whether a background consumer is active.	Can stop amplification, but may increase backlog and lag.

A NEXUS-1 rollback scenario

Suppose RootCause summary retrieval is extracted behind the Operator BFF. In shadow mode the new service builds the same summary that the modular core builds, but the console still reads from the old route. The team then moves ten percent of summary traffic to the new service. A problem appears: the new route marks a verdict as current while the reporting projection is still rebuilding. The safe response is not to delete anything. The route flag sends console reads back to the old path, the new worker is paused, the projection freshness rule is fixed, the affected summaries are rebuilt, and the audit trail records the route rollback.

Signal	Recovery move	Reason
Freshness marker is wrong	Route rollback to old summary path.	Protects the operator console first.
Projection rebuild incomplete	Pause worker or stop route expansion.	Prevents the wrong read model from spreading.
Bad output already displayed	Write audit note and operator-facing correction if needed.	Human decisions must be explainable.
Contract test missed condition	Add a new contract expectation.	The incident becomes executable protection.
Fix prepared	Roll forward with corrected freshness semantics.	The old binary may not know how to repair new metadata.

Deployment evidence checklist

A safe deployment should leave evidence behind. That evidence is not only for auditors; it is for the team that must decide whether to continue, pause, roll back, or roll forward. In NEXUS-1 language, deployment evidence is part of the same accountability discipline as audit trails, correlation IDs, and read-model freshness. If the team cannot reconstruct what changed, when it changed, and how traffic moved, the deployment was not observable enough.

Evidence item	Purpose
Release version and image digest	Identifies exactly what was running.
Schema migration id	Connects code release to data-shape change.
Route flag state history	Shows when traffic moved and when it moved back.
Contract-test result	Proves public promises were checked before release.
Health/readiness history	Shows whether the service was eligible for traffic.
Correlation samples	Proves a request can be followed across boundaries.
Projection lag and backlog	Shows whether delayed truth was under control.
Incident or rollback note	Explains the human decision to pause, revert, or continue.

Rollback plan before deployment

The rollback plan should be written before the release begins. It should name the trigger conditions, the owner, the action, and the evidence that confirms recovery. Without those details, rollback becomes a debate during stress. With them, rollback becomes a practiced operation. The important part is not that every release expects failure; it is that every serious release respects failure enough to plan for it.

```
Release gate: RootCause summary extraction

Proceed only if:
- Provider contract tests pass for BFF and Reporting consumers.
- New route is disabled by default.
- Old route remains available.
- Projection freshness is visible in the response.
- Rollback action is documented: set BFF route flag to OldCore.
- Worker pause action is documented: disable RootCauseSummaryProjector.
- Verification query confirms projection lag and error count.
- Audit note template exists for rollback or route expansion.
```

NEXUS-1 RULE

A rollback that cannot be explained later is not an accountable recovery path. The migration should leave behind the same kind of evidence that the platform demands from alarms, verdicts, compliance reviews, and audits.

What not to roll back

Some things should not be rolled back by deletion. Audit entries, emitted public facts, compliance review requests, and user-visible decisions require correction or supersession, not erasure. The old path may resume responsibility, but the record of what happened must remain. This protects the system from rewriting history in the name of operational convenience.

Artifact	Do not do this	Safer action
Audit entry	Delete the entry because the release was reverted.	Append a rollback/recovery audit entry.
Published integration event	Pretend the event never existed.	Publish a correction or superseding event if business meaning changed.
Compliance review request	Remove it silently.	Cancel or close with reason if it was triggered incorrectly.
Reporting projection	Patch rows by hand without source evidence.	Rebuild from source events or apply a documented correction.
Operator-visible state	Hide the panel and say nothing.	Display freshness, fallback, or correction status where needed.

Chapter checkpoint

Question	Expected answer
Why is rollback architectural?	Because code, data, events, projections, routes, and human-visible decisions must recover together.
When is roll-forward safer than rollback?	When new data or emitted facts cannot be understood by the old version.
What does expand-and-contract protect?	It keeps old and new paths compatible during staged migration.
Why are feature flags insufficient alone?	They control routes or behavior, but do not undo downstream effects.
What should never be erased during rollback?	Audit facts, public events, compliance records, and user-visible decisions; they need correction evidence.

PART X - MIGRATION STRATEGY

66. NEXUS-1 Migration Roadmap

A migration roadmap is not a calendar of enthusiasm. It is an ordered argument: what moves first, why it is safe to move now, what remains behind, and what evidence proves that the old and new systems still describe the same truth.

Why the roadmap comes last in the migration part

The previous chapters in this part deliberately avoided starting with a target architecture diagram. They walked through the migration discipline first: strangler routing, extracting a first service, keeping the old system alive, contract tests, rollback, and safe deployment. Only now is it reasonable to draw a roadmap. Before that, a roadmap would have been a list of wishes. After those chapters, it can become a controlled sequence of moves.

For NEXUS-1, the roadmap must respect the shape already built in the earlier parts of this book. Service extraction begins where ownership is clear, contracts can be named, read-side behaviour can be compared safely, and failure does not command the plant. It does not begin with the hardest domain decision or the most central write model. The first service is chosen because it teaches the migration safely.

ROADMAP RULE
Move first what can be compared, routed, observed, and rolled back. Move later what owns irreversible decisions, public facts, or cross-service workflow authority.

The migration direction

The roadmap does not turn every NEXUS-1 context into a service immediately. Some contexts remain modules for a long time; some may never deserve an independent runtime boundary. The direction is evolutionary: first isolate contracts and read paths, then extract services with clear ownership, then distribute workflow responsibility only when the system can observe, retry, compensate, and explain the flow.

Phase	Migration intent	Primary proof
0. Modular core baseline	Keep the modular monolith coherent while contracts are identified.	Current behaviour, tests, context map, data ownership map.
1. Read-side extraction	Extract low-risk query/projection services first.	Old/new response comparison and route rollback.
2. Ownership extraction	Move a context that owns data and language clearly.	Database ownership, public API, public events, independent deployment.
3. Workflow distribution	Move process managers only after messaging and compensation are ready.	Idempotency, outbox/inbox, retries, timeout policy, human review.
4. Operational hardening	Treat services as runtime responsibilities, not just projects.	Health checks, traces, dashboards, release evidence.
5. Final service catalogue	Document the stable service landscape.	Service catalog, API/event catalog, failure-mode catalog.

Candidate order for NEXUS-1

A useful roadmap names the candidate order explicitly. That order is not a judgement of importance. It is a judgement of migration risk. Reporting and operator-facing read composition can be extracted earlier because their outputs can be compared and rebuilt. Audit is central, but it should not be casually separated until append-only evidence and correlation semantics are fully stable. RootCause is powerful, but its write-side verdict ownership must not be split before its public contracts and correction events are mature.

Order	Candidate	Why this position
1	Operator BFF / UI composition	Routes and shapes responses without owning domain truth; easy to compare old and new views.
2	Reporting projections	Read models can be rebuilt from source truth and marked with freshness.
3	Compliance review view/service	Consumes public facts and human workflow state; useful but must not own root-cause truth.
4	RootCause read API	Can expose cases, evidence, and verdicts while the write model remains protected.
5	AlarmManagement event publisher	Owns alarm facts and flood events; extraction requires strong event-contract discipline.
6	RootCause write service	Owns verdict decisions; should move only after correction, audit, and compensation paths exist.
7	Audit service	Central evidence store; extraction must preserve append-only guarantees and correlation identity.
8	DigitalTwin and RL advisory services	Need clear advisory boundaries so recommendations never become commands.

Phase 0 - prepare the modular core

The first phase is not extraction. It is preparation. The modular core must expose seams without pretending that those seams are already services. Interfaces, DTOs, public

contract names, route flags, and comparison tests are introduced while the old deployment remains the authority. The goal is to make the future cut visible before the runtime cut is made.

```
Phase 0 checklist
- Public DTOs separated from domain entities.
- Integration event names fixed: RootCauseVerdictIssued.v1, AlarmFloodDetected.v1.
- Old and new response envelopes normalized.
- CorrelationId appears in logs, audit entries, messages, and BFF responses.
- Projection freshness fields exist before users depend on projections.
- Contract tests describe the boundary before the service exists.
- Route flags can select old core, shadow service, or active service.
```

HONEST BOUNDARY

A module with a route flag is not yet a microservice. It is a prepared seam. That preparation is valuable precisely because it lets the team delay distribution until the evidence supports it.

Phase 1 - extract read paths first

The safest first extraction usually belongs to a read path. A read path can run in shadow mode, compare responses against the modular core, record differences, and be removed from active routing without rewriting domain history. In NEXUS-1, this makes the Operator BFF and Reporting projections strong early candidates. They teach service deployment, routing, health checks, correlation, and dashboards without moving the authority to decide a root cause.

Read-side candidate	Shadow comparison	Rollback path
Operator BFF status view	Compare old console payload and new composed payload.	Route BFF back to old core endpoint.
Reporting projection	Compare aggregate counts, timestamps, and freshness markers.	Rebuild projection and read from old query temporarily.
Compliance dashboard view	Compare review counts and linked evidence references.	Hide new view or route to old read model.
RootCause case summary	Compare case id, verdict text, evidence count, and correction state.	Return to old root-cause summary route.

Phase 2 - extract a context that owns truth

After read-side extraction has taught the operational mechanics, the roadmap can move toward a true ownership service. This is the point where the earlier warnings of the book matter most. The service must own its database, protect its language, publish public events rather than internal domain objects, and define what happens when consumers are absent or slow. For NEXUS-1, AlarmManagement is a plausible early write-side candidate because alarms and flood detection have clear public events, while RootCause write ownership should wait until correction and compliance paths are fully mature.

Context	Owns	Must publish/serve
AlarmManagement	Alarm definitions, alarm events, flood detection state.	AlarmFloodDetected.v1 and alarm query API.
RootCause	Root-cause cases, evidence, hypotheses, verdicts.	RootCauseVerdictIssued.v1 and correction events.
Compliance	Review requests, findings, review state.	RequestComplianceReview command handling and review status API.
Audit	Append-only evidence records and correlation chain.	Audit query API and immutable evidence writes.

Phase 3 - distribute workflows carefully

Distributed workflows are not a reward for finishing service extraction. They are a new source of failure. The alarm-to-compliance path should become distributed only where the process manager, saga policy, or choreography has a clear reason to exist. The process manager should store its own state, not hide domain decisions. Each step must name timeout behaviour, retry behaviour, idempotency behaviour, compensation behaviour, and human review conditions.

Workflow step	Owner	Roadmap condition before distribution
Alarm flood detected	AlarmManagement	Event contract versioned and outbox-backed.
Root-cause case opened	RootCause	OpenRootCauseCase command is idempotent and traceable.
Evidence attached	RootCause	Evidence references are stable and source ownership is clear.
Verdict issued	RootCause	RootCauseVerdictIssued.v1 and correction semantics are stable.
Compliance review triggered	Compliance / process manager	Timeout, escalation, and human review policy are explicit.
Audit written	Audit	Append-only evidence cannot be erased by rollback.

Roadmap decision matrix

The migration roadmap should be reviewed at every phase against the same decision matrix. This prevents architectural momentum from becoming automatic extraction. A context that fails the matrix should remain in the modular core until the missing evidence exists.

Question	Ready signal	Delay signal
Can the boundary be named in business language?	A non-technical owner can explain what it owns.	The boundary is mostly a folder or controller name.
Does it own data?	Source-of-truth tables or event stream are assigned.	It reads and writes other contexts casually.
Is the public contract stable?	API and event contracts are versioned and tested.	Internal domain objects leak out.
Can it fail independently?	Timeouts, retries, DLQ, health, and rollback are known.	A downstream failure becomes a system-wide incident.
Can users understand delay?	Freshness and pending states are visible.	The UI pretends all truth is immediate.
Can the team operate it?	Dashboards, runbooks, and ownership exist.	No team owns the service after deployment.

The NEXUS-1 roadmap, written plainly

A realistic NEXUS-1 roadmap would therefore not begin by decomposing all seventeen sectors into seventeen services. It would begin with one or two routes, one projection, one public event, one independently deployable read service, and one operational dashboard. Only after that path has survived shadowing, traffic movement, rollback, and evidence review should the roadmap proceed to write-side ownership and distributed workflow.

```
Roadmap summary
1. Stabilize modular core boundaries and public language.
2. Introduce route flags, contract tests, correlation IDs, and freshness fields.
3. Extract Operator BFF and selected reporting/read projections.
4. Extract one ownership service with low ambiguity, likely AlarmManagement.
5. Move RootCause read APIs before RootCause write authority.
6. Publish RootCauseVerdictIssued.v1 only through outbox-backed contracts.
7. Introduce distributed process management for alarm-to-compliance flow.
8. Extract Audit only when append-only evidence and correlation rules are mature.
9. Document the resulting service catalog, API catalog, event catalog, data ownership catalog, and failure-mode
```

What should remain monolithic longer

A good roadmap also names what should not move yet. That is not lack of ambition; it is architectural judgement. Anything whose language is still unstable, whose data ownership is shared, whose failure would require manual reconstruction, or whose operational owner is unclear should remain in the modular core. The roadmap is strongest when it refuses premature extraction.

Keep inside longer	Reason
Unstable domain rules	A service freezes confusion behind a network boundary.
Shared lookup governance	Reference data may need central discipline before replication.
Safety-adjacent advisory boundaries	Recommendations must remain clearly non-commanding.
Cross-cutting audit semantics	Append-only evidence must be protected before runtime separation.
Experimental RL policy tuning	Advisory models should not be distributed before their contracts and limits are explicit.

Chapter checkpoint

Question	Expected answer
Why does the roadmap come after the migration patterns?	Because a roadmap without rollback, contract tests, coexistence, and evidence is only a wish list.
What should move first?	Read paths and composition layers that can be shadowed, compared, routed, and rebuilt.
What should move later?	Decision-owning write services, audit evidence, and distributed workflow authority.
Why not extract all NEXUS-1 sectors immediately?	Because bounded contexts are not automatically microservices and some boundaries are not operationally ready.
What does Chapter 66 close?	Part X: it turns migration patterns into an ordered NEXUS-1 roadmap before the final architecture catalog begins.

PART X CHECKPOINT
At the end of this part, the reader should understand that microservice migration is not a rewrite. It is a sequence of reversible, observable, contract-protected moves from a coherent modular core toward selected independent services.

PART XI

Final Architecture

The earlier parts made the decisions separately: boundaries, ownership, contracts, messages, workflows, read models, security, observability, and migration. Part XI draws them together. It does not announce that every NEXUS-1 sector is now a microservice. It records which sectors are strong candidates, which are conditional, and which should remain protected until a real reason appears.

Question	What Part XI records
What can become a service?	A candidate map, not an automatic context-to-service conversion.
What does each service own?	A catalog of data, APIs, events, and operational responsibility.
What can fail?	A failure-mode catalog that names the system behaviour before production pressure arrives.
What remains before implementation?	A final checklist that prevents architecture diagrams from becoming premature code.

PART XI RULE
The final architecture is not the drawing with the most boxes. It is the smallest distributed map that keeps ownership, traceability, reliability, and language discipline intact.

Chapters 67-72 close the design loop. They translate the accumulated reasoning into candidate services, service catalog entries, public APIs, integration events, data ownership records, failure modes, and a pre-implementation checklist.

PART XI - FINAL ARCHITECTURE

67. The NEXUS-1 Microservice Candidate Map

A microservice candidate map is a record of architectural judgement. It says which boundaries are ready to carry runtime independence, which boundaries still need protection, and which parts should remain deliberately inside the modular core.

The problem this chapter solves

By this point in the book, the reader has seen every major force that makes microservices difficult: ownership, separate data, remote failure, public contracts, asynchronous messages, process managers, eventual consistency, security boundaries, observability, and migration risk. The natural temptation is to finish by drawing a clean diagram with one service for every NEXUS-1 context.

That would be the wrong ending. It would confuse a conceptual boundary with a deployment boundary. The final map must not say ReactorFleet, Instrumentation, DigitalTwin, AlarmManagement, RootCause, Compliance, Audit, Reporting, and ReinforcementLearning all become services because they already have names. It must ask which of them are ready to own data, publish contracts, tolerate failure, justify independent deployment, and survive operational accountability.

CANDIDATE MAP DEFINITION
A microservice candidate map is a decision table and architecture diagram that classifies bounded contexts as strong service candidates, conditional candidates, or deliberately retained modules. It explains the reason for each classification.

The naive map

The naive map is simple: one bounded context equals one microservice. It looks clean on a slide and dangerous in a system. A context may own a language and still have no reason to deploy independently. A context may have tables and still be too coupled to shared simulation state. A context may be important and still be safer as a module until its public contract has matured.

Naive assumption	Why it fails in NEXUS-1
Every context becomes a service.	Some contexts express language boundaries, not runtime boundaries.
Every table group becomes a database.	Separate storage without ownership clarity creates a distributed shared database by another name.
Every event becomes public.	Internal domain events are not stable integration contracts.
Every UI screen calls every service.	The operator console would become a brittle distributed query engine.
Every service can decide.	Only the owning context may issue a decision; observers and projections must not promote themselves.

HONEST BOUNDARY

This chapter does not implement the service landscape. It classifies candidates. Implementation begins only after the next chapters define the service catalog, API and event catalog, data ownership catalog, and failure-mode catalog.

The evaluation criteria

A candidate map needs visible criteria, otherwise it becomes taste. The following criteria are applied repeatedly across NEXUS-1. No single criterion is enough. A service candidate becomes strong only when several forces point in the same direction.

Criterion	Question asked
Business capability clarity	Can a non-technical stakeholder understand what this service owns?
Data ownership clarity	Can this service own its write model without borrowing tables from another service?
Language independence	Does the context use words with meanings that must be protected?
Contract stability	Can the public API or integration event stay stable while the internal model evolves?
Failure isolation value	Does separating the service reduce blast radius rather than increase it?
Operational responsibility	Can a team monitor, deploy, and support this service independently?
Consistency cost	Can the system tolerate eventual consistency at this boundary?
Audit and traceability	Can the service preserve correlation, evidence, and replay across boundaries?

The candidate classes

The map uses three classifications. Strong candidate means the context has a clear reason to become a service once the platform has the operational discipline described in Parts V-IX. Conditional candidate means the context may become a service only if a specific pressure appears: scale, team ownership, deployment cadence, data isolation, or integration need. Retained module means distribution would probably damage clarity or safety for now.

Class	Meaning	Architectural posture
Strong candidate	Runtime independence is plausible and valuable.	Design service contract, data ownership, events, and operational dashboard.
Conditional candidate	Service extraction depends on real pressure.	Keep modular; define seams; postpone deployment split.
Retained module	Distribution creates more risk than value.	Protect inside modular core; expose only through carefully controlled facades.

NEXUS-1 candidate map

The first pass candidate map is intentionally conservative. It does not maximize the number of services. It maximizes architectural honesty. The map below is a design position, not a deployment command.

Area	Candidate class	Primary reason
AlarmManagement	Strong	Owens alarm lifecycle, flood detection language, alarm state, and the AlarmFloodDetected.v1 integration fact.
RootCause	Strong	Owens cases, evidence, hypotheses, verdicts, corrections, and the RootCauseVerdictIssued.v1 contract.
Compliance	Strong	Owens review workflow triggered by public facts; can evolve separately from root-cause reasoning.
Audit	Strong but protected	Owens append-only evidence and traceability; must be extracted with strict correlation and immutability rules.
Reporting	Strong read-side	Owens projections and dashboards; can rebuild views and explain freshness without owning source truth.
DigitalTwin	Conditional	May require scale and model isolation, but has tight coupling to instrumentation and simulation semantics.
Instrumentation	Conditional	High-volume ingestion may justify separation, but tag and signal meaning must remain governed.
ReinforcementLearning Advisory	Conditional	Advisory policy can be isolated if training, versioning, and recommendation contracts mature.
ReactorFleet	Conditional / reference authority	Central physical identity is widely referenced; extraction demands careful reference-data strategy.
Operator BFF	Supporting service	Useful runtime composition layer; not source of domain truth.
Simulation internals	Retained module	Highly coupled demonstration internals should not be distributed for cosmetic architecture.

A map in words before a diagram

The strongest service boundary in this book remains the alarm-to-compliance flow: AlarmManagement detects a flood, RootCause investigates and issues a verdict, Compliance opens review, Audit records evidence, and Reporting builds read models. That sequence is not a single service. It is also not a free-for-all. Each service owns one part of the truth and publishes only the public facts that other services may consume.

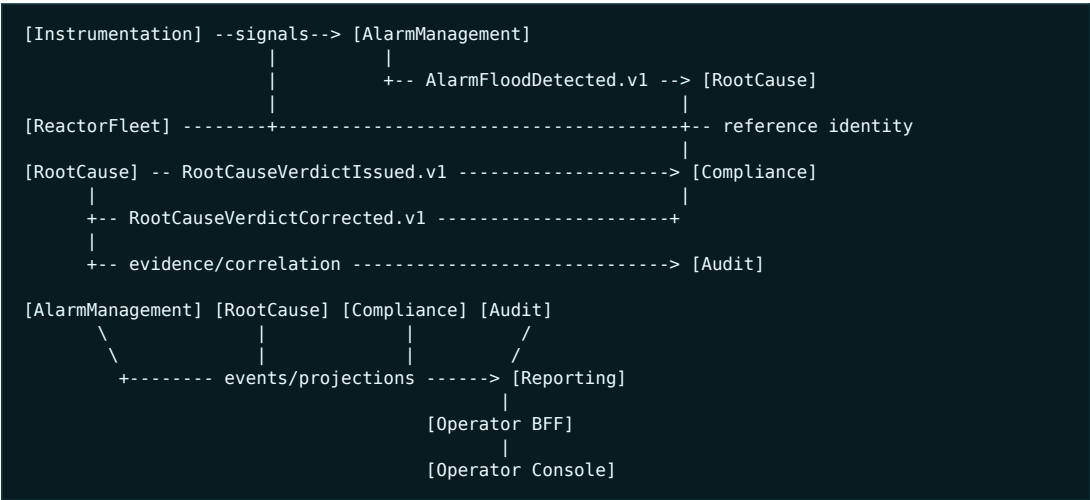
Flow step	Owning service candidate	Public boundary
Alarm flood detected	AlarmManagement	AlarmFloodDetected.v1
Root-cause case opened	RootCause	OpenRootCauseCase command or internal application API
Verdict issued	RootCause	RootCauseVerdictIssued.v1 / RootCauseVerdictIssuedV1
Compliance review requested	Compliance	RequestComplianceReview
Verdict corrected	RootCause	RootCauseVerdictCorrected.v1
Audit evidence appended	Audit	Append-only audit evidence, not workflow control
Operator view updated	Reporting / BFF	Projection freshness and correlation ID

LANGUAGE DISCIPLINE

AuditEntry is evidence, not workflow. A recommendation is not a command. A read model is not source of truth. RootCauseCaseClosed is not the canonical public event when the public fact is RootCauseVerdictIssued.v1.

Candidate map as deployment topology

A text map is often more precise than an early diagram. The following sketch shows direction, not physical network detail. The arrows are not database joins. They are public contracts, messages, or controlled query composition paths.



Notice what the sketch does not show. It does not show the operator console calling every service directly. It does not show Reporting owning the verdict. It does not show Audit deciding workflow. It does not show ReactorFleet copied everywhere as uncontrolled local truth. The absence of those arrows is as important as the arrows that remain.

Strong candidates in detail

The strong candidates are strong for different reasons. AlarmManagement is a candidate because it owns a clear operational capability and can publish a stable fact when a flood is detected. RootCause is a candidate because it owns a decision boundary: evidence, hypotheses, verdicts, and corrections. Compliance is a candidate because it has a human-review workflow that should evolve without changing the causal engine. Reporting is a candidate because read models can be rebuilt and compared. Audit is a candidate because evidence deserves a protected append-only boundary - but it is also the one that must be handled with the least tolerance for casual redesign.

Candidate	Owns	Must not own
AlarmManagement	Alarm definitions, alarm events, flood detection state.	Root-cause verdicts or compliance decisions.
RootCause	Cases, evidence, hypotheses, verdicts, verdict correction.	Alarm lifecycle, compliance review state, operator UI composition.
Compliance	Review requests, findings, control checks, human review state.	Causal truth or audit immutability.
Reporting	Read projections, dashboards, export snapshots, freshness metadata.	Write-side domain truth.
Audit	Evidence trail, append-only records, correlation, replay support.	Business workflow decisions.

Conditional candidates in detail

Conditional candidates require a trigger. DigitalTwin may need separation if model updates, divergence calculations, or twin snapshots grow independently. Instrumentation may need separation if ingestion volume or historian integration becomes a scaling pressure. ReinforcementLearning Advisory may need separation if training, policy publication, versioning, and recommendation review become a separate lifecycle. ReactorFleet may remain a reference authority for a long time because physical identity is a passport used everywhere.

Candidate	Extract only if...	Keep modular if...
DigitalTwin	Twin model cadence, snapshot size, or divergence processing requires independent scale.	It remains tightly coupled to in-process demonstration simulation.
Instrumentation	Signal ingestion volume or external historian integration needs its own runtime.	The system only needs demonstration-scale signal handling.
ReinforcementLearning Advisory	Policy training, approval, and advisory delivery require independent release.	The policy remains a small advisory feature of the core.
ReactorFleet	A stable physical-reference service is needed by many teams or systems.	It mostly supplies reference identity inside one controlled solution.

What stays out of the map

A final candidate map also names what does not move. This matters because microservice enthusiasm often hides inside ambiguous words like platform, shared, common, or core. In NEXUS-1, highly coupled simulation internals, safety-like control paths, and shared demonstration scaffolding should not be turned into services just to make the architecture look complete.

WHAT SHOULD NOT BE DISTRIBUTED

Do not distribute safety-like control logic, tightly coupled simulation internals, uncontrolled shared utilities, or UI convenience code that has no independent ownership. Runtime separation must buy clarity, not cosmetic complexity.

A small .NET shape for the map

A candidate map can be made explicit in code without pretending that services already exist. The point is not runtime discovery. The point is a reviewable architecture record that can later drive service catalog generation, contract tests, and migration planning.

```
public enum ServiceCandidateStatus
{
    Strong,
    StrongButProtected,
    Conditional,
    Supporting,
    RetainedModule
}

public sealed record MicroserviceCandidate(
    string Name,
    ServiceCandidateStatus Status,
    string Owns,
    string[] PublicContracts,
    string[] MustNotOwn,
    string ExtractionReason);

public static class NexusServiceCandidateMap
{
    public static readonly MicroserviceCandidate RootCause = new(
        Name: "RootCause",
        Status: ServiceCandidateStatus.Strong,
        Owns: "cases, evidence, hypotheses, verdicts, corrections",
        PublicContracts: new[] { "RootCauseVerdictIssued.v1",
                                "RootCauseVerdictCorrected.v1" },
        MustNotOwn: new[] { "alarm lifecycle", "compliance review state",
                            "operator UI composition" },
        ExtractionReason: "clear decision boundary and stable public facts");
}
```

How to read the map before implementation

The map should be read as a gate. A strong candidate is not automatically extracted tomorrow. It becomes eligible for the next chapter: a service catalog entry. That entry will define its purpose, owners, APIs, events, data, dependencies, operational checks, and failure behaviour. A conditional candidate is not rejected; it is protected from

premature extraction. A retained module is not a failure; it is a disciplined refusal to make the system worse for the sake of distribution.

If the map says...	Then the next architectural action is...
Strong	Create a service catalog entry and define public API/event contracts.
Strong but protected	Define immutability, audit, correlation, and recovery rules before extraction.
Conditional	Define seams and triggers; keep deployment modular until pressure is real.
Supporting	Define runtime purpose, but do not grant source-of-truth authority.
Retained module	Keep inside the modular core and document why distribution is harmful.

Checkpoint

Question	Expected answer
Why is a bounded context not automatically a service?	Because language ownership and runtime independence are different decisions.
Which NEXUS-1 services are strongest candidates?	AlarmManagement, RootCause, Compliance, Reporting, and Audit, with Audit protected by stricter evidence rules.
Why is ReactorFleet conditional?	Its identity is central and widely referenced; extraction requires careful reference-data ownership.
What must Reporting never become?	The source of truth for verdicts, alarms, compliance decisions, or audit evidence.
What does the map do next?	It feeds the service catalog, API/event catalog, data ownership catalog, failure-mode catalog, and final checklist.

CHAPTER 67 SUMMARY
The NEXUS-1 microservice candidate map is conservative on purpose. It turns the entire book so far into a decision record: strong candidates, conditional candidates, supporting runtime components, and retained modules. The final architecture begins by refusing to distribute what has not earned distribution.

PART XI - FINAL ARCHITECTURE

68. Service Catalog

A candidate map says which services may exist. A service catalog says what each of those services means, who owns it, what it exposes, what it consumes, what it stores, and how it behaves when the rest of the system is not perfect.

The problem this chapter solves

Chapter 67 produced a candidate map. That map was deliberately conservative: some NEXUS-1 sectors were strong service candidates, some were conditional, some were supporting runtime components, and some were retained modules. But a candidate map alone is not enough to guide implementation. A box named RootCause Service does not yet tell a developer what the service owns, which contracts are public, which tables are private, which events it consumes, which events it publishes, or which operational signals prove it is healthy.

A service catalog turns the final architecture from a diagram into a controlled record. It is the place where a team can ask: what is this service for, what does it own, what may it never own, what is its public language, and what must another service never assume about it?

SERVICE CATALOG DEFINITION
A service catalog is a structured record for every service candidate. It captures purpose, ownership, public contracts, consumed events, published events, data ownership, dependencies, security posture, observability, failure behaviour, and extraction status.

Why a catalog is better than a diagram alone

A diagram is excellent for orientation and poor as a contract. It can show that RootCause talks to Compliance, but it cannot reliably show whether the connection is a command, a query, an integration event, a projection feed, or a manual review workflow. It can show that Audit receives information, but it cannot remind the reader that AuditEntry is evidence, not workflow. The catalog carries these distinctions explicitly.

Diagram says...	Catalog must also say...
RootCause -> Compliance	Which public event triggers review, whether delivery is asynchronous, and what idempotency key protects duplicates.
RootCause -> Audit	Whether the audit record is evidence, who appends it, and why Audit must not drive workflow.
Reporting reads many services	Which data is a projection, how fresh it is, and which service owns the source truth.
Operator BFF composes screens	Which queries it may call, which data it may cache, and why it has no domain authority.
ReactorFleet is referenced by all	Whether it is a service, a reference authority, or still a module during this phase.

CATALOG RULE
No service entry is complete until it names both authority and prohibition: what the service owns, and what it must never decide.

The catalog entry shape

The NEXUS-1 service catalog uses the same grammar for every candidate. The goal is not bureaucracy. The goal is comparability. If two services use the same headings, reviewers can see immediately which boundary is mature and which boundary is still only a guess.

Field	Question answered
Service name	What runtime boundary or candidate is being described?
Candidate status	Strong, strong but protected, conditional, supporting, or retained module?
Purpose	What business or architectural capability does it provide?
Owns	Which decisions, data, and language are source-of-truth here?
Must not own	Which decisions belong elsewhere even if this service observes them?
Public commands / APIs	Which synchronous contracts may outside callers use?
Published events	Which integration facts become public after local commit?
Consumed events	Which external facts may influence local process state?
Private data	Which tables or storage are not shared?
Operational checks	Which health, readiness, trace, and dashboard signals matter?
Failure posture	What should happen when dependencies are unavailable?
Migration note	How should this boundary be extracted or protected?

A C# shape for the catalog

The catalog can be represented as a simple architecture record. It is not meant to drive runtime routing directly. It is meant to keep decisions reviewable, testable, and

comparable with contract tests, deployment manifests, dashboards, and documentation.

```
public sealed record ServiceCatalogEntry(
    string Name,
    ServiceCandidateStatus Status,
    string Purpose,
    string Owns,
    string[] MustNotOwn,
    string[] PublicApis,
    string[] PublishedEvents,
    string[] ConsumedEvents,
    string[] PrivateStores,
    string[] OperationalChecks,
    string FailurePosture,
    string MigrationNote);

public enum ServiceCandidateStatus
{
    Strong,
    StrongButProtected,
    Conditional,
    Supporting,
    RetainedModule
}
```

Core service entries

The first catalog entries are the strong candidates from the previous chapter. They are written with enough detail to prevent the most common microservice error: allowing every service to know a little too much about every other service.

Service	Status	Purpose
AlarmManagement	Strong	Own alarm lifecycle, alarm state, flood detection, acknowledgement, and public alarm-flood facts.
RootCause	Strong	Own root-cause cases, evidence, hypotheses, verdicts, corrections, and causal decision language.
Compliance	Strong	Own review workflow, findings, control checks, and human review state triggered by public facts.
Audit	Strong but protected	Own append-only evidence, correlation, replay support, and immutable accountability records.
Reporting	Strong read-side	Own projections, dashboards, export snapshots, freshness metadata, and read-side reconstruction.

AlarmManagement service catalog entry

AlarmManagement is a strong candidate because it owns a clear operational capability: alarm definitions, alarm events, alarm lifecycle, flood detection, and alarm-state language. It may publish that an alarm flood was detected. It must not decide root cause, compliance state, or audit evidence meaning.

Catalog field	Entry
Name	AlarmManagement Service
Status	Strong service candidate
Owns	Alarm definitions, alarm events, alarm lifecycle, flood grouping, acknowledgement state.
Must not own	Root-cause verdicts, compliance review status, audit immutability, operator UI composition.
Public APIs	RaiseAlarm, AcknowledgeAlarm, QueryActiveAlarms, QueryAlarmFloods.
Published events	AlarmFloodDetected.v1; optionally AlarmAcknowledged.v1 if a later flow needs it.
Consumed events	Reference updates from ReactorFleet or Instrumentation only if governed as reference data.
Private stores	AlarmDefinition, AlarmEvent, AlarmFlood, AlarmAcknowledgement, alarm lifecycle lookups.
Failure posture	Keep accepting local alarm writes if downstream RootCause or Reporting is unavailable; publish through outbox.

RootCause service catalog entry

RootCause is a strong candidate because it owns a decision boundary. A root-cause verdict is not a display value and not a reporting projection. It is a domain decision based on evidence, hypotheses, and rule-governed closure. For that reason, RootCause owns the public fact RootCauseVerdictIssued.v1.

Catalog field	Entry
Name	RootCause Service
Status	Strong service candidate
Owns	RootCauseCase, Evidence, Hypothesis, Verdict, verdict correction, causal case language.
Must not own	Alarm lifecycle, compliance finding state, audit storage authority, recommendation command execution.
Public APIs	OpenRootCauseCase, AttachEvidence, RejectHypothesis, PromoteHypothesis, IssueVerdict, CorrectVerdict.
Published events	RootCauseVerdictIssued.v1; RootCauseVerdictCorrected.v1.
Consumed events	AlarmFloodDetected.v1; possibly reviewed reference updates.
Private stores	RootCauseCase, Evidence, Hypothesis, CausalNode, CausalEdge, VerdictHistory.
Failure posture	Never lose committed verdicts; publish integration events through outbox; allow downstream lag.

CANONICAL EVENT DISCIPLINE
The public integration event is RootCauseVerdictIssued.v1 and the C# public contract type is RootCauseVerdictIssuedV1. RootCauseCaseClosed and RootCauseVerdictPublished should not replace the canonical event name.

Compliance, Audit, and Reporting entries

Compliance, Audit, and Reporting sit downstream of domain facts but they do not mean the same thing. Compliance owns review state. Audit owns evidence. Reporting owns read models. Confusing those three roles would undo the careful boundary work of the previous parts.

Service	Owns	Must not own	Primary public input
Compliance	Review requests, findings, controls, human review status.	Root-cause truth or audit immutability.	RootCauseVerdictIssued.v1 or RequestComplianceReview.
Audit	Append-only evidence, correlation chains, replay support.	Workflow decisions or compliance status.	Evidence append requests and event metadata.
Reporting	Projections, dashboard views, exports, freshness metadata.	Source truth for alarms, verdicts, reviews, or evidence.	Integration events and query-side projection feeds.

Conditional and supporting entries

A catalog should also record the boundaries that are not yet services. Conditional candidates are not ignored; they are described in a way that makes future extraction possible without pretending that the pressure already exists. Supporting services are also described, but without giving them domain authority.

Candidate	Catalog status	Catalog note
DigitalTwin	Conditional	Extract if model cadence, snapshot size, or divergence processing needs independent scale.
Instrumentation	Conditional	Extract if historian integration or ingestion volume becomes a separate operational pressure.
ReinforcementLearning Advisory	Conditional	Extract if training, policy versioning, approval, and advisory delivery become separate lifecycle concerns.
ReactorFleet	Conditional / reference authority	Treat as protected identity source; extraction requires a reference-data strategy.
Operator BFF	Supporting	Compose operator views and shield UI from service topology; never become source of truth.
Simulation internals	Retained module	Keep coupled demonstration internals inside the modular core.

Service catalog as an implementation gate

The catalog should be reviewed before code is split. A team should not create a new repository, database, or deployment pipeline until the service entry is complete enough to answer ownership and failure questions. In NEXUS-1, this is especially important because the same flow crosses alarm, root-cause, compliance, audit, reporting, and operator-console concerns.

Gate question	Pass condition
Purpose is clear?	The entry can be explained without naming technology first.
Data ownership is clear?	The service has private write storage and does not share tables.
Contracts are named?	Public APIs and events are listed with versions where needed.
Prohibitions are listed?	The service states what it must never decide.
Failure posture exists?	The entry says what happens when dependencies fail or lag.
Observability exists?	Health, readiness, traces, metrics, and dashboard signals are named.
Migration path exists?	The entry says how extraction can happen safely from the current system.

A service catalog excerpt in code

The catalog can be written as documentation, YAML, database seed data, C# records, or a combination. The format matters less than the discipline: the same fields must be present for every candidate.

```
public static class NexusServiceCatalog
{
    public static readonly ServiceCatalogEntry RootCause = new(
        Name: "RootCause",
        Status: ServiceCandidateStatus.Strong,
        Purpose: "Investigate alarm floods and issue grounded verdicts.",
        Owns: "cases, evidence, hypotheses, verdicts, corrections",
        MustNotOwn: new[] { "alarm lifecycle", "compliance findings",
            "audit storage authority", "UI composition" },
        PublicApis: new[] { "OpenRootCauseCase", "AttachEvidence",
            "IssueVerdict", "CorrectVerdict" },
        PublishedEvents: new[] { "RootCauseVerdictIssued.v1",
            "RootCauseVerdictCorrected.v1" },
        ConsumedEvents: new[] { "AlarmFloodDetected.v1" },
        PrivateStores: new[] { "RootCauseCase", "Evidence", "Hypothesis",
            "VerdictHistory", "OutboxMessage" },
        OperationalChecks: new[] { "readiness", "outbox lag", "verdict publish age",
            "correlation continuity" },
        FailurePosture: "commit local verdict, publish by outbox, allow downstream lag",
        MigrationNote: "extract after contract tests protect public event shape");
}
```

How the catalog connects to the remaining chapters

The service catalog is not the final artifact. It is the parent of three catalogs that follow. Chapter 69 will separate APIs from events. Chapter 70 will turn service entries into data ownership records. Chapter 71 will name failure modes. Chapter 72 will use all of them as a final pre-implementation checklist.

Catalog artifact	What it adds
Service Catalog	Purpose, authority, prohibitions, dependencies, operational posture.
API and Event Catalog	Public commands, queries, event names, versions, payload ownership.
Data Ownership Catalog	Private stores, source truth, references, projections, replication rules.
Failure Mode Catalog	Unavailable dependencies, duplicate events, stale read models, recovery path.
Final Checklist	A go/no-go decision before implementation or extraction begins.

Checkpoint

Question	Expected answer
Why is a service catalog needed after the candidate map?	The map says what may become a service; the catalog says what each service means and owns.
What two things must every service entry name?	Authority and prohibition: what it owns and what it must never decide.
Why is Audit marked strong but protected?	Because it owns evidence and replay, but extraction requires stricter immutability and correlation discipline.
What must the Operator BFF never become?	A source of domain truth or a hidden decision engine.
What does the catalog feed next?	The API/event catalog, data ownership catalog, failure-mode catalog, and final checklist.

CHAPTER 68 SUMMARY
A service catalog turns the final architecture into a reviewable record. It prevents service names from becoming vague boxes and forces each candidate to state purpose, ownership, contracts, private data, dependencies, operational checks, failure behaviour, and migration posture.

PART XI - FINAL ARCHITECTURE

69. API and Event Catalog

A service catalog says what a service is. An API and event catalog says how the rest of the system is allowed to talk to it, react to it, and depend on it without stealing its model.

The problem this chapter solves

After Chapter 68, NEXUS-1 has a service catalog: a disciplined record of candidate services, responsibilities, prohibitions, public surfaces, dependencies, and operational posture. But implementation teams do not integrate with a service catalog directly. They integrate through public APIs and public events. That is where many microservice designs quietly lose their boundaries.

A service can have a clear purpose and still leak its domain through controllers, DTOs, message payloads, or convenience queries. A RootCause service can accidentally expose Hypothesis as if every downstream consumer should understand its internal lifecycle. An AlarmManagement service can publish an event that looks like a database row instead of a public fact. A Compliance service can accept a command that allows another service to decide compliance review state for it. The catalog in this chapter prevents that drift.

CATALOG PURPOSE
The API and event catalog lists the public commands, public queries, public integration events, versions, payload owners, idempotency keys, correlation fields, consumers, and stability rules for every service boundary.

APIs and events are different public promises

A synchronous API and an integration event are both public contracts, but they are not the same kind of promise. An API is a request made to a service now. An event is a fact the service has already committed locally and now publishes for others to observe. Confusing those two shapes produces brittle workflows.

Contract kind	Meaning	Wrong use
Command API	Ask the owning service to attempt a state-changing decision.	Letting a downstream service set state it does not own.
Query API	Ask for a current view owned or exposed by the service.	Using another service query as a hidden join inside a transaction.
Integration event	Announce a committed fact for observers.	Treating the event as a remote command that must be handled immediately.
Callback/event reply	Report outcome of an async process.	Inventing distributed request/response without timeouts or idempotency.
Projection feed	Supply data for a read model.	Letting the projection become source of truth.

LANGUAGE RULE

Commands use imperative names because they request an action. Events use past-tense fact names because they report something that has happened. Queries describe a view or retrieval need. Mixing these grammars is a boundary smell.

The catalog entry shape

The catalog is intentionally plain. Every row should be reviewable by a developer, architect, tester, and operator. The important question is not whether the catalog is stored in Markdown, YAML, JSON, C#, or a database table. The important question is whether every contract has a named owner and a stability rule.

Field	Why it exists
Contract name	Gives the API, query, or event a stable public name.
Contract kind	Separates command, query, integration event, projection feed, or callback.
Owning service	States who may change the contract and who owns the language.
Version	Makes breaking change explicit instead of accidental.
Payload owner	States whether the payload is public language or internal model leakage.
Idempotency key	Protects retries, duplicates, and replay.
Correlation fields	Allows a flow to be traced across services.
Consumers	Names known downstream responsibilities without giving them ownership.
Stability rule	Says whether the contract is stable, candidate, deprecated, or internal-only.

Canonical NEXUS-1 contracts

The NEXUS-1 series already established several names that must remain disciplined. This book preserves that terminology rather than inventing new names for convenience. The most important example is the distinction between the internal domain event VerdictIssued and the public integration event RootCauseVerdictIssued.v1. The first belongs inside the RootCause model. The second belongs at the service boundary.

Name	Kind	Owner	Public meaning
OpenRootCauseCase	Command API	RootCause	Request the RootCause service to open a case for a validated alarm flood.
AlarmFloodDetected.v1	Integration event	AlarmManagement	An alarm flood has been detected and can be investigated.
VerdictIssued	Domain event	RootCause	Internal fact: the aggregate issued a verdict. Not a public contract.
RootCauseVerdictIssued.v1	Integration event	RootCause	A root-cause verdict is now publicly available for downstream observers.
RootCauseVerdictCorrected.v1	Integration event	RootCause	A previously issued verdict was corrected with new evidence.
RequestComplianceReview	Command/API or process message	Compliance	Ask Compliance to create or evaluate a review request.
AuditEntryWritten	Audit fact	Audit	Evidence has been appended. It must not become business workflow.

TERMINOLOGY DISCIPLINE

RootCauseCaseClosed should not replace RootCauseVerdictIssued.v1 as the canonical public event. Closing a case and issuing a verdict are not the same public fact. The downstream flow cares about the verdict and its evidence, not merely about a lifecycle transition.

Command APIs in the catalog

A command API should name an intention and preserve the authority of the owning service. The caller does not decide the result. It asks the owner to attempt a decision under its own rules. The response may be accepted, rejected, queued, or delayed. This is especially important for RootCause and Compliance, where rule ownership must remain local.

Command	Service	Idempotency	Important rule
OpenRootCauseCase	RootCause	AlarmFloodId or request id	Reject if the alarm flood is not valid or already represented by a case.
AttachEvidence	RootCause	EvidenceId or evidence fingerprint	Accept evidence only while the case allows change.
IssueVerdict	RootCause	Caseld + verdict attempt id	A case cannot close without evidence; the aggregate decides.
CorrectVerdict	RootCause	CorrectionId	Correction must preserve history and publish a correction event.
RequestComplianceReview	Compliance	ReviewRequestId	Compliance owns review state; RootCause only requests review.
AcknowledgeAlarm	AlarmManagement	AlarmEventId + actor id	Alarm acknowledgement does not decide root cause.

Query APIs in the catalog

Queries are tempting because they feel harmless. In a distributed architecture they can become hidden coupling. A query that is used occasionally by an operator screen is different from a query that is used inside another service transaction. The catalog therefore records purpose, freshness expectation, and whether the query is allowed for runtime orchestration.

Query	Provider	Intended consumer	Freshness expectation
GetRootCauseCaseStatus	RootCause	Operator BFF, Reporting projection	Current according to RootCause service.
GetAlarmFloodSummary	AlarmManagement	Operator BFF, RootCause intake view	Current enough to open or display investigation.
GetComplianceReviewStatus	Compliance	Operator BFF, Reporting	May lag behind verdict publication.
GetAuditTrailForCorrelation	Audit	Operator BFF, reviewer tools	Append-only evidence view, not workflow truth.
GetOperationalDashboardSnapshot	Reporting	Operator dashboard	Projection with visible generated-at timestamp.

QUERY BOUNDARY

A service should not use another service query as if it were a local table inside the same transaction. If the decision requires local certainty, the owning service needs a local copy, a snapshot, an event, or a redesigned boundary.

Integration events in the catalog

Integration events are the public memory of a distributed flow. They must be stable enough for downstream services to consume without knowing the publisher internal model. That means event payloads should contain public facts, public identifiers, timestamps, correlation identifiers, version information, and enough business context to let consumers act idempotently.

Event	Publisher	Known consumers	Required catalog fields
AlarmFloodDetected.v1	AlarmManagement	RootCause, Reporting, Operator BFF projection	FloodId, UnitId, detected time, alarm count, severity summary, correlation id.
RootCauseVerdictIssued.v1	RootCause	Compliance, Audit, Reporting, Operator BFF projection	Caseld, verdict id, cause code, confidence band, issued time, evidence refs, correlation id.
RootCauseVerdictCorrected.v1	RootCause	Compliance, Audit, Reporting	Correction id, prior verdict id, new verdict id, reason, evidence refs, correlation id.
ComplianceReviewRequested.v1	Compliance	Audit, Reporting, Operator BFF projection	Review id, source case id, requested by, due window, correlation id.
ComplianceReviewCompleted.v1	Compliance	Audit, Reporting	Review id, outcome, finding references, completed time, correlation id.

A public contract sketch

The public event type should look deliberately smaller and more stable than the aggregate that produced it. It should not carry internal collections, private enum names, database row shapes, or navigation properties. The public contract is a boundary object.

```
public sealed record RootCauseVerdictIssuedV1(
    Guid EventId,
    Guid CorrelationId,
    Guid CausationId,
    string CaseId,
    string UnitCode,
    string VerdictId,
    string PublicCauseCode,
    string ConfidenceBand,
    IReadOnlyList<string> EvidenceReferences,
    DateTimeOffset IssuedAtUtc,
    int SchemaVersion = 1);

// Contract rule:
// This is not RootCauseCase serialized over the wire.
// It is the public fact downstream services are allowed to know.
```

PAYLOAD RULE
The payload must be enough for consumers to react safely, but not so much that it becomes a copy of the publisher private domain model.

Versioning and deprecation

A catalog entry must say what happens when a public contract changes. Adding optional information is usually easier than removing or renaming fields. Changing meaning is more dangerous than changing shape. The version suffix is not decoration; it is a promise that existing consumers will not be surprised silently.

Change	Catalog response
Add optional field	Allow within same version if old consumers ignore it safely.
Rename field	Create a new version or compatibility mapping; do not surprise consumers.
Change meaning of field	Create new version and document semantic break.
Remove field	Deprecate first, measure consumers, then retire only after migration.
Split event	Publish both during transition; document causation and correlation rules.
Merge commands	Check authority and idempotency; merged commands often hide two decisions.

NEXUS-1 API and event catalog excerpt

Service	Public APIs	Published events
AlarmManagement	RaiseAlarm, AcknowledgeAlarm, GetAlarmFloodSummary	AlarmRaised.v1, AlarmFloodDetected.v1
RootCause	OpenRootCauseCase, AttachEvidence, IssueVerdict, CorrectVerdict, GetCaseStatus	RootCauseVerdictIssued.v1, RootCauseVerdictCorrected.v1
Compliance	RequestComplianceReview, CompleteComplianceReview, GetReviewStatus	ComplianceReviewRequested.v1, ComplianceReviewCompleted.v1
Audit	AppendAuditEntry, GetAuditTrailForCorrelation	AuditEntryWritten.v1 or internal audit stream
Reporting	GetDashboardSnapshot, GetCaseTrend, GetComplianceSummary	Usually none; publishes refreshed projection notifications only if needed.
Operator BFF	GetOperatorCaseView, GetAlarmToComplianceView	Usually none; composes views, does not own domain facts.

Review questions before implementation

Before a service is implemented, its public contracts should survive review. A contract that cannot answer these questions is not ready to become a dependency for another service.

Question	Why it matters
Who owns this contract?	A public surface without an owner will drift.
Is this command, query, or event?	The interaction shape controls reliability, timing, and authority.
What is the version?	Consumers need a stable expectation.
What is the idempotency key?	Retries and duplicate delivery are normal.
What correlation fields are present?	The alarm-to-compliance flow must be traceable.
What must not be in the payload?	This prevents domain-model leakage.
Who consumes it today?	Known consumers shape compatibility and deprecation planning.
What happens if it is delayed?	Public contracts create operational expectations.

Checkpoint

Question	Expected answer
Why separate the API and event catalog from the service catalog?	Because service purpose is not enough; consumers depend on concrete public contracts.
What is the difference between VerdictIssued and RootCauseVerdictIssued.v1?	VerdictIssued is internal domain truth; RootCauseVerdictIssued.v1 is the public integration event.
Why should commands be imperative and events past-tense?	The grammar protects authority and timing semantics.
What does idempotency protect?	Retries, duplicate requests, duplicate events, and replay.
What is the main payload danger?	Leaking the publisher internal domain model as a public contract.

CHAPTER 69 SUMMARY

The API and event catalog is the public contract ledger of the architecture. It names commands, queries, integration events, versions, owners, payload rules, idempotency keys, correlation fields, consumers, and deprecation posture so that services can communicate without stealing each other models.

PART XI - FINAL ARCHITECTURE

70. Data Ownership Catalog

A microservice architecture is not complete when every service has a name. It is complete only when every important piece of data has an owner, every copy has a reason, and every consumer knows whether it is reading truth or a projection.

The problem this chapter solves

The previous chapter catalogued APIs and events. This chapter catalogues data ownership. The distinction matters because a service can expose a clean contract and still hide a shared-data dependency underneath it. If two services both believe they own the same row, the system has not become distributed. It has become confused with network latency.

In NEXUS-1, this problem appears immediately. Unit, Signal, AlarmEvent, RootCauseCase, AuditEntry, ComplianceReview, and DashboardSnapshot are all useful to many parts of the platform. That does not make them collectively owned. It means the architecture must say who may write each fact, who may copy it, who may project it, who may correct it, and who must never treat a copy as source-of-truth.

CHAPTER ARGUMENT

A data ownership catalog is the final architecture ledger that prevents a microservice system from silently returning to a shared database mindset. It names the source-of-truth service, the owned entities, allowed replicas, projection rules, correction rules, retention rules, and forbidden writes.

Ownership catalog vs database catalog

A schema catalog tells where tables live. A service catalog tells which deployable owns a capability. A data ownership catalog connects the two and adds runtime rules. It asks whether a table is local truth, a read model, a snapshot, a reference copy, an audit record, or a temporary cache. These are not interchangeable categories.

Catalog	Question it answers	Danger if missing
Schema catalog	Which database object stores this shape?	Teams know storage but not ownership.
Service catalog	Which service owns this capability?	Services are named but their data remains ambiguous.
API/event catalog	How may others interact with this service?	Consumers depend on private models or unstable contracts.
Data ownership catalog	Who owns, copies, projects, corrects, and retires this data?	Shared database behaviour returns through projections, snapshots, and undocumented writes.

The ownership states

A data item can appear in several places, but it cannot have several equal owners. The catalog uses explicit states so that a developer knows what they are allowed to do with a row or message payload.

State	Meaning	Write authority
Local truth	The service authoritative model for a business fact.	Only the owning service writes it.
Reference copy	A small copied fact needed for local validation or display.	Owner publishes changes; receiver updates its copy through controlled integration.
Snapshot	A point-in-time copy used to preserve the context of a decision.	Written by the receiving service as evidence; not synchronized as current truth.
Projection	A read model built from events or queries for display/reporting.	Projection builder writes it; source remains elsewhere.
Cache	A performance copy with expiry and no business authority.	Regenerated or invalidated; never corrected manually as truth.
Audit evidence	Append-only proof that something happened.	Audit writes evidence; it does not own the business state described.

OWNERSHIP RULE
Copies are allowed. Ambiguous authority is not. The problem is not that Reporting stores a case summary; the problem is when Reporting starts acting as if that summary is the case.

Required catalog fields

A useful ownership catalog is not a diagram alone. It is a reviewable table with enough detail to guide implementation, testing, and operations. The fields below should be present for every serious service candidate.

Field	Purpose
Owned data	The aggregates, tables, documents, or streams the service is allowed to write as local truth.
Public identifiers	The identifiers other services may keep without owning the entity.
Reference copies	The foreign facts the service keeps locally, with update source and freshness expectation.
Snapshots	The point-in-time facts recorded as evidence for decisions.
Published events	The public facts through which other services learn about changes.
Inbound dependencies	Events, commands, or queries required to maintain local data.
Correction path	How wrong data is corrected without rewriting another service truth.
Retention rule	How long local truth, projections, audit evidence, and copies are kept.
Forbidden writes	The tables or facts this service must never update directly.

NEXUS-1 data ownership catalog excerpt

The catalog below is not every column of the future implementation. It is the architectural ownership map that prevents the final architecture from becoming a distributed monolith. It says which service writes which truth and which services only observe, copy, or project it.

Service	Local truth	Allowed copies / projections	Forbidden ownership
ReactorFleet	Unit, Reactor, major equipment identity, lifecycle status.	Public UnitRef snapshots consumed by AlarmManagement, RootCause, DigitalTwin, Reporting.	It does not own alarms, verdicts, compliance reviews, or dashboard projections.
Instrumentation	Signal, sensor metadata, readings, signal quality state.	SignalRef copied by AlarmManagement; measurement windows copied as evidence snapshots.	It does not decide alarm floods or root cause.
AlarmManagement	AlarmDefinition, AlarmEvent, AlarmFlood, acknowledgement state.	UnitRef and SignalRef for local decisions; flood summaries projected by Reporting.	It does not own RootCauseCase or ComplianceReview.
RootCause	RootCauseCase, Evidence, Hypothesis, Verdict, Correction.	AlarmFloodSnapshot, TelemetryEvidenceSnapshot, UnitRef.	It does not update alarm state, compliance findings, or audit evidence directly.
Compliance	ComplianceReview, Finding, ReviewOutcome, required follow-up.	Verdict snapshot from RootCauseVerdictIssued.v1; audit references.	It does not change RootCause verdict truth.
Audit	AuditEntry, audit batch, hash chain or lineage evidence.	Public identifiers and payload fingerprints.	It does not drive workflow state or decide business outcomes.
Reporting	Read models, dashboard snapshots, generated report artifacts.	Projection copies from events and queries.	It never becomes source-of-truth for operational decisions.

Public identifiers and private keys

The catalog must separate internal database keys from public identifiers. A service may use a local integer key for persistence, but other services should not build their own state around that private shape. Public identifiers should be stable, opaque enough to avoid coupling, and meaningful enough to support traceability.

Data item	Public identifier	Private or local shape
Root-cause case	RootCauseCaseId / CaseCode	RootCause service table key, hypothesis rows, evidence rows.
Alarm flood	AlarmFloodId	Alarm event rows and detection window internals.
Compliance review	ComplianceReviewId	Reviewer assignment, finding rows, control mapping.
Audit entry	AuditEntryId plus correlation id	Hash-chain storage and internal batch sequence.
Dashboard snapshot	SnapshotId plus generated-at timestamp	Projection tables and denormalized display rows.

IDENTITY BOUNDARY
A public identifier is a passport. It allows another service to reference a fact without receiving permission to own the private table structure behind that fact.

Snapshots as decision evidence

Some copied data must not stay current. That sounds strange until a decision is audited. If RootCause issues a verdict using an alarm-flood summary and telemetry window, the evidence should preserve what the service saw at that time. Replacing the snapshot later with a current value would corrupt the explanation of the decision.

Snapshot	Created by	Why it exists
AlarmFloodSnapshot	RootCause when a case opens	Preserves the flood shape that justified investigation.
TelemetryEvidenceSnapshot	RootCause when evidence is attached	Preserves the signal window used to evaluate the hypothesis.
VerdictSnapshot	Compliance when review is requested	Preserves the public verdict fact that triggered review.
AuditPayloadFingerprint	Audit when event is recorded	Preserves evidence without owning the business state.
DashboardProjectionSnapshot	Reporting during build	Records what the dashboard knew and when it knew it.

A small .NET implementation shape

The catalog can begin as documentation, but serious teams eventually make parts of it executable. Contract tests can verify that services do not write forbidden tables, public events carry required identifiers, and projections are marked as projections. The example below is deliberately small: it captures the ownership vocabulary that architecture reviews can use.

```
public enum DataOwnershipKind
{
    LocalTruth,
    ReferenceCopy,
    Snapshot,
    Projection,
    Cache,
    AuditEvidence
}

public sealed record DataOwnershipEntry(
    string ServiceName,
    string DataName,
    DataOwnershipKind Kind,
    string OwnerService,
    string PublicIdentifier,
    string FreshnessExpectation,
    IReadOnlyList<string> ForbiddenWriters);

// Example: Reporting may store a projection of a root-cause verdict,
// but RootCause remains the owner of verdict truth.
```

Correction without stealing ownership

Distributed systems need correction paths because events can be wrong, late, superseded, or misunderstood. The correction must go through the owner. Reporting does not correct a RootCause verdict. Compliance does not rewrite RootCause evidence. Audit does not erase a mistaken entry. The owner publishes a correction, the observers rebuild or mark stale, and the audit trail explains the change.

Problem	Wrong response	Ownership-preserving response
A verdict was issued with incomplete evidence.	Compliance edits the verdict fields in its database.	RootCause publishes RootCauseVerdictCorrected.v1; Compliance updates its review context.
A projection contains stale alarm counts.	Operator BFF writes directly to Reporting tables.	Reporting rebuilds from source events and marks generated-at time.
An audit payload contains a typo.	Update the old audit row in place.	Append a correction entry that points to the original evidence.
A Unit display name changes.	Every service manually updates whatever rows it can find.	ReactorFleet publishes reference-data change; consumers update copies or preserve snapshots.

Data ownership smells

The catalog also works as a smell detector. If a proposed service cannot fill its ownership row, it is not ready. If every service wants to write the same table, the boundary is false. If a projection is used for command decisions, the system has lost the difference between current truth and convenient display.

Smell	What it usually means
Two services write the same business table.	The service boundary is not real or the ownership decision has been avoided.
A service needs another service database to validate every command.	The aggregate boundary, snapshot strategy, or service split is wrong.
Reporting is used as operational truth.	A projection has been promoted beyond its authority.
Audit entries are replayed to infer workflow state.	Process state was not modelled explicitly.
Reference data changes break consumers silently.	Public identifiers and reference-update contracts are missing.
Every event carries entire aggregate graphs.	Public contracts are leaking private models.

Review questions before implementation

Before the final architecture becomes code, every service candidate should pass a data ownership review. The purpose is not to make a perfect catalog on day one. The purpose is to make ambiguity visible before it becomes production coupling.

Question	Expected answer
Which service owns this fact?	Exactly one owner is named.
Who may write it?	Only the owner, except for local projections/caches marked as such.
Who may copy it?	Consumers are named with copy type and freshness expectation.
Is this value current truth or historical evidence?	The catalog distinguishes live references from snapshots.
How is it corrected?	Through owner-published correction or append-only evidence, not direct cross-service edits.
Can the service work when the owner is temporarily unavailable?	It has a snapshot, cache, event stream, fallback, or explicit unavailable state.
What must never be written here?	Forbidden writes are listed and testable.

Checkpoint

Question	Expected answer
Why is data ownership more than database location?	Because copies, projections, snapshots, caches, and audit evidence can exist outside the owner.
Can a service store data it does not own?	Yes, if the data is explicitly a copy, snapshot, projection, cache, or evidence.
Why should Reporting not be source-of-truth?	Because Reporting optimizes reading and display; it does not own operational decisions.
Why preserve snapshots?	They explain what a service knew when it made a decision.
What is the main purpose of the catalog?	To prevent hidden shared-database thinking from reappearing inside a distributed architecture.

CHAPTER 70 SUMMARY

The Data Ownership Catalog closes one of the most dangerous gaps in microservice design. It says who owns each fact, who may copy it, who may project it, which snapshots are evidence, how corrections flow, and which writes are forbidden. Without this catalog, services can look independent while their data remains confused.

PART XI - FINAL ARCHITECTURE

71. Failure Mode Catalog

A final architecture is not proven by drawing the happy path. It is proven by naming the failures each service can suffer, deciding who notices them, and making the recovery path visible before the first deployment exists.

Why this catalog belongs near the end

The previous catalogs named services, APIs, events, and data ownership. Those catalogs make the architecture visible in normal operation. This chapter adds the missing half: what happens when the normal path breaks. A microservice architecture multiplies the number of places where failure can occur. That is not an argument against services; it is an argument against pretending that service boundaries are complete without failure boundaries.

In NEXUS-1, the alarm-to-compliance flow crosses Instrumentation, AlarmManagement, RootCause, Recommendation, Audit, Compliance, Reporting, and the Operator Console. Any one of those services can be slow, unavailable, inconsistent, behind in projection, blocked by a poison message, or working with an older contract version. The Failure Mode Catalog says what each service is allowed to do in those conditions and what it must never invent.

CHAPTER ARGUMENT
A Failure Mode Catalog turns distributed risk into named architectural knowledge. It lists failure types, detection signals, user-visible behaviour, retry policy, escalation path, audit evidence, recovery ownership, and the forbidden shortcuts that would damage trust.

Failure modes are architectural facts

A failure mode is not merely an exception class. It is a repeated shape of system behaviour that deserves a planned answer. The answer may be a retry, a timeout, a circuit breaker, a dead-letter route, a manual review queue, an unavailable-state banner, a compensating action, or a deliberate refusal to continue. What matters is that the answer is designed before production pressure forces an improvised one.

Failure mode	Typical symptom	Architectural question
Service unavailable	Call fails or times out.	Can the caller continue, defer, or must it stop?
Slow downstream	Thread pools, queues, or UI wait states grow.	What timeout protects the caller?
Duplicate message	The same event arrives more than once.	Which idempotency key makes the second delivery harmless?
Poison message	A message always fails processing.	Where is it isolated and who reviews it?
Projection lag	Read model is behind source truth.	How is delayed truth shown to the user?
Contract drift	Producer and consumer disagree on shape or meaning.	Which contract test should have caught this?
Partial workflow	Some steps succeeded and later steps failed.	Which service owns recovery and escalation?

The NEXUS-1 failure-mode template

Every failure-mode entry should be short enough to read and strict enough to guide implementation. It is not a long incident report. It is a design record for how a known class of failure is handled.

Field	Meaning
Mode name	The stable name used in code, dashboards, and documentation.
Trigger	The condition that makes the mode active: timeout, broker rejection, version mismatch, repeated handler failure, stale projection, missing dependency.
Detection signal	Metric, log, trace, health check, DLQ count, retry count, or domain status that proves the mode exists.
Immediate behaviour	What the service does now: fail fast, defer, retry, enqueue, compensate, degrade, or pause.
User-visible state	What the operator console says instead of pretending everything is current.
Audit evidence	Which event, record, or correlation chain proves what happened.
Owner	The team or service responsible for resolution.
Forbidden shortcut	The tempting action that must not be taken because it corrupts truth, hides risk, or breaks ownership.

Failure catalog for the alarm-to-compliance flow

The recurring NEXUS-1 flow is useful because every failure has a different consequence. A missing recommendation is not the same as a missing audit entry. A delayed compliance projection is not the same as a lost verdict. The catalog keeps those differences explicit.

Step	Failure mode	Safe behaviour
AlarmManagement publishes AlarmFloodDetected.v1	Outbox row cannot be published to broker.	Keep the outbox row pending, expose publishing lag, do not open a RootCause case from memory.
RootCause consumes alarm flood event	Duplicate delivery.	Use event id / correlation id idempotency; return success for already-opened case.
RootCause issues RootCauseVerdictIssued.v1	Consumer contract mismatch.	Reject to DLQ with version reason; do not translate private domain object ad hoc.
Recommendation service reacts to verdict	Recommendation engine unavailable.	Mark recommendation unavailable; do not fabricate advice.
Audit records evidence	Audit write fails.	Stop workflow continuation where audit is mandatory; raise operational alert.
Compliance review requested	Compliance service unavailable.	Queue request and surface pending compliance review state.
Reporting projection updates	Projection lag.	Display last-updated timestamp and source event watermark.
Operator console reads combined status	One backend-for-frontend dependency fails.	Render partial state with explicit degraded panel.

NEXUS-1 RULE

A service may degrade presentation, defer work, retry, isolate a poison message, or ask for human review. It may not invent domain truth, silently mark a workflow complete, treat an audit entry as a command, or write another service's data to escape a failure.

A catalog entry in implementation shape

The catalog is not only prose. A small typed representation lets the team keep failures close to code, tests, dashboards, and runbooks. The exact C# shape can change later; the important point is that failure behaviour is first-class, not hidden inside scattered catch blocks.

```
public sealed record FailureModeCatalogEntry(
    string ModeCode,
    string ServiceName,
    string Trigger,
    string DetectionSignal,
    string ImmediateBehaviour,
    string UserVisibleState,
    string AuditEvidence,
    string RecoveryOwner,
    string ForbiddenShortcut);

public static class RootCauseFailureModes
{
    public static readonly FailureModeCatalogEntry DuplicateVerdictEvent = new(
        ModeCode: "RCA_DUPLICATE_VERDICT_EVENT",
        ServiceName: "RootCause.Service",
        Trigger: "RootCauseVerdictIssued.v1 is delivered more than once.",
        DetectionSignal: "Inbox contains the same EventId or CorrelationId.",
        ImmediateBehaviour: "Return success without applying the verdict twice.",
        UserVisibleState: "No duplicate verdict is shown to the operator.",
        AuditEvidence: "Inbox idempotency record and correlation trace.",
        RecoveryOwner: "RootCause service owner.",
        ForbiddenShortcut: "Creating a second compliance review for the same verdict.");
}
```

Detection and observability mapping

Every failure mode needs a signal. If the only proof is a user complaint, the catalog is incomplete. Some signals are technical, such as HTTP 5xx rates, retry counts, circuit-breaker state, queue age, and dead-letter counts. Others are domain signals, such as root-cause cases stuck in AwaitingEvidence, compliance reviews pending beyond a threshold, or reports with stale event watermarks.

Mode category	Primary signal	Secondary evidence
Unavailable service	Failed health check, circuit breaker open, timeout count.	Trace spans ending at dependency boundary.
Message delivery failure	Outbox publish age, broker nack, DLQ count.	Outbox row state and retry history.
Consumer failure	Inbox retry count, handler exception class.	Dead-letter payload and correlation chain.
Projection lag	Event watermark behind source stream.	Dashboard last-updated timestamp.
Contract drift	Consumer deserialization / validation failure.	Contract-test build failure or version mismatch.
Human-review backlog	Queue age and assignment state.	Escalation audit entry.

Recovery ownership

A catalog that only describes failure is incomplete. It must also say who owns recovery. In a monolith, one team may inspect the whole process. In a service architecture, recovery crosses boundaries, so ownership must be visible. The service that owns the failed fact is not always the service that noticed the failure.

For example, the Operator Console may notice that a compliance projection is stale, but it does not own compliance truth. Reporting may notice that a RootCause verdict event has not arrived, but it does not own the verdict. The catalog prevents the observing service from becoming a shadow owner.

Observed by	Owned by	Correct recovery path
Operator Console	Backend-for-Frontend / source services	Show degraded panel; request fresh state through public APIs.
Reporting	RootCause	Wait for event or request re-projection; do not query RootCause database.
Compliance	Audit	Require evidence record before closing review; do not synthesize audit proof.
Audit	Publishing service	Record failed publication evidence; publishing service repairs outbox.
RootCause	AlarmManagement	Reject malformed alarm-flood event; AlarmManagement owns correction.

Forbidden shortcuts

The most important column in the Failure Mode Catalog may be the forbidden shortcut. Distributed systems fail under pressure, and pressure creates temptations: write directly into another service database, suppress an error because the UI looks better, replay a message by hand without idempotency, treat a stale projection as truth, or close a workflow step without audit evidence. The catalog gives the team permission to refuse those shortcuts.

HONEST BOUNDARY

This chapter does not implement a production incident-management platform. It defines the architectural catalog that such a platform would consume. Real deployment would add alert routing, ownership groups, SLOs, runbooks, on-call rotation, incident severity, and post-incident review.

NEXUS-1 failure-mode catalog excerpt

Mode code	Service	Short rule
ALM_OUTBOX_STUCK	AlarmManagement	Pending alarm-flood events must remain visible until published; no consumer may infer them from tables.
RCA_DUPLICATE_EVENT	RootCause	Duplicate verdict or evidence events must be idempotent and auditable.
REC_ENGINE_DOWN	Recommendation	Recommendation unavailable is an honest state; never fabricate advice.
AUD_WRITE_FAILED	Audit	If audit evidence is mandatory, workflow continuation pauses.
CMP_REVIEW_DELAYED	Compliance	Delayed review is surfaced as pending; reporting must not mark complete.
RPT_WATERMARK_STALE	Reporting	Read models show watermark and last update; stale projection is not source truth.
BFF_PARTIAL_STATE	Operator BFF	Render partial state with explicit degraded panels and correlation id.
CAT_VERSION_MISMATCH	Contract Catalog	Unsupported contract versions go to rejection path; no private-object guessing.

Checkpoint

Question	Expected answer
Why is failure cataloguing part of architecture?	Because distributed systems have predictable failure shapes that must be designed before deployment.
What is the difference between a failure mode and an exception?	A failure mode includes detection, behaviour, user state, audit evidence, owner, and forbidden shortcut.
Why include user-visible state?	Because hidden failure creates false confidence in the operator console.
Why include recovery ownership?	Because the service that observes a failure may not own the truth that must be repaired.
What should never happen during failure?	A service must not invent truth, write another service's data, hide lag, or skip mandatory audit evidence.

CHAPTER 71 SUMMARY

The Failure Mode Catalog completes the final architecture from the negative side. It does not ask only what services do when everything works. It asks how they fail, how they are detected, what the user sees, what evidence is kept, who recovers them, and which shortcuts are forbidden.

PART XI - FINAL ARCHITECTURE

72. Final Pre-Implementation Checklist

The last architectural chapter is not a celebration of microservices. It is a refusal to start before the evidence is strong enough. A service is ready only when boundaries, contracts, data, failure behaviour, security, observability, migration, and ownership can all be checked.

Why the book ends with a checklist

A checklist can look modest after chapters about bounded contexts, databases per service, API contracts, message brokers, sagas, projections, identity, observability, migration strategy, catalogs, and failure modes. That modesty is exactly why it belongs here. The final decision to extract a service should not depend on architectural enthusiasm or on a diagram that looks clean. It should depend on a list of questions whose answers are visible.

For NEXUS-1, the checklist is a guardrail against a common mistake: taking the names ReactorFleet, AlarmManagement, RootCause, Compliance, Reporting, and Audit and assuming that every name deserves its own deployment immediately. The previous chapters argued the opposite. A service candidate earns runtime independence only when it can protect its own truth, publish stable contracts, survive remote failure, prove its behaviour through tests, and be operated by a team without stealing another service's database or language.

FINAL ARGUMENT
Microservices begin after readiness, not before it. The checklist is the architectural gate between a service candidate and a deployable service.

The seven readiness gates

The checklist is organized as seven gates. A candidate does not need perfection in every column, but any weak column must be named as a risk, not hidden as future work. A weak gate may mean the candidate should remain inside the modular monolith, or that it should be extracted only behind a strangler path with additional supervision.

Gate	Question it answers	Evidence required
Boundary readiness	Does the service have a real business responsibility?	Context map, capability map, ownership statement, language boundary.
Data readiness	Can it own its data without shared-table escape routes?	Source-of-truth table list, reference strategy, migration path, read-model rules.
Contract readiness	Can others use it without knowing its internals?	Public APIs, integration events, versioning rules, contract tests.
Reliability readiness	Can it survive remote failure and messaging realities?	Timeouts, retries, outbox, inbox, DLQ rules, idempotency keys.
Security readiness	Can it authenticate, authorize, and protect secrets?	Service identity, claims policy, least privilege, certificate / secret handling.
Observability readiness	Can it be operated when something goes wrong?	Logs, metrics, traces, health checks, dashboards, correlation ids.
Migration readiness	Can it be introduced without breaking the old system?	Strangler route, coexistence plan, rollback, release gates, recovery evidence.

Gate 1 - boundary readiness

A service boundary is ready when it represents ownership, not convenience. The candidate must have a vocabulary that does not constantly borrow words from another context. It must contain rules that clearly belong inside it. It must have reasons to evolve independently. Most important, it must be able to say no to requests that would turn it into a remote helper for another model.

Check	Ready sign	Not-ready sign
Business capability	The service performs a recognizable domain responsibility.	It is mostly a bundle of CRUD endpoints around arbitrary tables.
Language ownership	Terms have stable meaning inside the service.	The same words change meaning depending on the caller.
Decision ownership	The service is allowed to make specific decisions.	Decisions are scattered across callers or workflow scripts.
Team ownership	A team can own change, tests, and operations.	Every change requires coordination with half the platform.

NEXUS-1 EXAMPLE

RootCause is a stronger service candidate than a generic Evidence service. RootCause owns cases, hypotheses, evidence rules, verdict language, and the public RootCauseVerdictIssued.v1 event. A generic Evidence service would own a noun, not a decision boundary.

Gate 2 - data readiness

The database-per-service rule is not a technology rule. It is a rule about ownership. A service that still needs direct access to another service's tables is not independent; it is a distributed module with a shared hidden database. Before extraction, the team must name source truth, local copies, reference snapshots, projections, and forbidden joins.

Data item	Ownership decision before implementation
RootCauseCase	Owned by RootCause; no other service closes it or rewrites evidence.
AlarmEvent	Owned by AlarmManagement; RootCause receives stable event or query contract, not table access.
AuditEntry	Owned by Audit; used as evidence, not as a workflow command.
ComplianceReview	Owned by Compliance; Reporting projects it but does not decide it.
Operator status read model	Owned by BFF or Reporting projection; explicitly marked as read-side, not source truth.
Reference plant identity	Referenced by stable ids and snapshots; changes propagate through contracts, not cross-database joins.

Gate 3 - contract readiness

A service is not ready merely because it has controllers. It is ready when its public contracts can be read without opening its domain model. Commands, queries, and integration events must be named differently, versioned deliberately, and tested outside the private implementation. The canonical public event names from the previous volume must remain stable unless a new version is declared.

Contract type	Checklist item	NEXUS-1 rule
Command API	Does the request express an intention, not a row mutation?	OpenRootCauseCase is a command; InsertRootCauseCaseRow is not.
Query API	Does the response protect internal aggregate shape?	Case summary DTOs do not expose private entity structure.
Integration event	Is the event public, versioned, and stable?	RootCauseVerdictIssued.v1 is the contract; VerdictIssued remains internal.
Correction event	Is correction explicit instead of overwriting history?	RootCauseVerdictCorrected.v1 is separate from the original verdict event.
Contract test	Can a consumer validate the shape without running the service internals?	Consumer-driven tests sit at the boundary, not inside the aggregate.

```
public sealed record ServiceReadinessCheck(
    string ServiceCandidate,
    bool BoundaryReady,
    bool DataReady,
    bool ContractsReady,
    bool ReliabilityReady,
    bool SecurityReady,
    bool ObservabilityReady,
    bool MigrationReady)
{
    public bool CanExtract =>
        BoundaryReady && DataReady && ContractsReady &&
        ReliabilityReady && SecurityReady &&
        ObservabilityReady && MigrationReady;
}
```

Gate 4 - reliability readiness

A remote service introduces remote failure. Before implementation, the team should be able to answer what happens if the call times out, if the message is delivered twice, if the broker is unavailable, if a poison message blocks consumption, if a projection lags, and if a human review step waits longer than expected. These are not operational details added later. They are part of the service contract.

Reliability question	Acceptable answer
What happens if publication fails?	Outbox row remains pending, age is monitored, and the service does not pretend the event was delivered.
What happens if the event arrives twice?	Inbox/idempotency record makes the second delivery harmless.
What happens if the downstream service is unavailable?	Timeout and retry policy is bounded; caller knows whether to defer, degrade, or fail fast.
What happens to poison messages?	They move to a DLQ with reason, payload, version, and correlation id.
What happens when a workflow is incomplete?	Process manager state remains visible and escalation rules apply.

Gate 5 - security readiness

Security readiness means more than putting authentication in front of an API. A service must know which identity it trusts, which claims it accepts, which roles remain local policy decisions, how service-to-service credentials rotate, and which permissions are deliberately not delegated. The checklist protects the system from turning every service into an administrator because integration is difficult.

Security area	Checklist question
Authentication	Can the service distinguish user identity from service identity?
Authorization	Are policy decisions local to the service that owns the rule?
Least privilege	Does the service account have only the permissions it needs?
Secrets	Are certificates, keys, and connection strings outside source code and rotation-ready?
Audit linkage	Can every privileged action be traced to user, service, request, and correlation id?

Gate 6 - observability readiness

A service cannot be safely extracted if it cannot be observed. Logs explain what happened at a single point. Metrics explain whether the system is healthy over time. Traces explain how a request or workflow crossed service boundaries. Health checks explain whether the service should receive traffic. Dashboards explain the operational state in language humans can act on.

Signal	Minimum readiness condition
Logs	Structured logs include service name, operation, correlation id, and failure mode where relevant.
Metrics	Request rate, error rate, latency, queue age, retry count, DLQ count, and projection lag are measurable.
Traces	Cross-service calls and message handlers propagate correlation and causation ids.
Health checks	Liveness, readiness, and dependency checks are separated.
Dashboards	The service has an owner-visible dashboard before production use.

Gate 7 - migration readiness

The first implementation should be reversible. The strangler route, routing rules, coexistence plan, compatibility layer, contract tests, fallback behaviour, and rollback plan must be written before traffic moves. Migration readiness is the discipline that prevents a microservice extraction from becoming a one-way door.

Migration item	Ready when...
Strangler route	Traffic can be redirected gradually and observed.
Coexistence plan	Old and new paths can run without corrupting the same truth.
Backfill / migration	Data movement is repeatable, verified, and auditable.
Contract tests	Old clients and new clients can be tested against the public boundary.
Rollback	Rollback restores routing and protects already-published events.
Release gate	Operational evidence, not optimism, decides whether to proceed.

NEXUS-1 final candidate checklist

The table below is not a final implementation promise. It is the architectural posture at the end of this book. It says which candidates are strongest, which need caution, and which should remain supporting modules until the forces for extraction become real.

Candidate	Readiness posture	Implementation note
RootCause.Service	Strong candidate	Clear language, verdict ownership, public event, audit need, and meaningful independent evolution.
AlarmManagement.Service	Strong but dependency-heavy	Must protect signal/alarm ownership and publish flood events without leaking tables.
Compliance.Service	Good candidate	Review workflow is distinct, but relies on evidence, audit, and reporting projections.
Reporting.Service	Read-side candidate	Should own projections, not truth; must display watermark and freshness.
Audit.Service	Special candidate	Append-only evidence service; must avoid becoming workflow engine.
DigitalTwin.Service	Cautious candidate	High coupling to instrumentation and simulation state; extract only when runtime need is strong.
ReinforcementLearning.Service	Advisory candidate	Recommendations are not commands; extraction must preserve advisory boundary.
Operator BFF	Useful edge service	Composes UI state; must not become hidden domain owner.

What should still not be implemented yet

A checklist is valuable only if it can stop work. Some things should remain outside implementation until the necessary evidence exists. Do not build a separate service merely to match a diagram. Do not split data before ownership is clear. Do not introduce a broker before the event contracts are stable. Do not add a workflow engine before the process states and forbidden decisions are understood. Do not deploy independently before health, rollback, and dashboards exist.

HONEST BOUNDARY

This chapter closes the architecture book, not the implementation. It does not claim that the services are built, deployed, containerized, monitored, or production-ready. It defines the conditions under which implementation would be responsible.

The final checklist page

Before extracting a NEXUS-1 service, confirm...	Status
The candidate owns a business capability and language boundary.	Required
The service can own its source-of-truth data without shared-table access.	Required
Commands, queries, and integration events are separate public contracts.	Required
Outbox, inbox, idempotency, DLQ, timeout, and retry rules are defined where needed.	Required
Security identity, authorization policy, least privilege, and secrets handling are explicit.	Required
Logs, metrics, traces, health checks, and dashboards exist before production traffic.	Required
Migration path, coexistence strategy, contract tests, rollback, and release gates are written.	Required
The operator console can show delayed truth honestly.	Required
Audit evidence is preserved across boundaries and never treated as workflow command.	Required
The team can name what remains in the modular monolith and why.	Required

Checkpoint

Question	Expected answer
Why does the final chapter use a checklist?	Because extraction should be gated by evidence, not by fashion or a clean-looking diagram.
What are the seven readiness gates?	Boundary, data, contract, reliability, security, observability, and migration readiness.
Why can a checklist say no?	Because some candidates are conceptually bounded but not operationally ready.
What is the strongest NEXUS-1 first candidate?	RootCause.Service is the strongest architectural candidate, provided contracts and audit/reliability rules are implemented.
What is the final rule before implementation?	Do not distribute a boundary until ownership, truth, failure, security, operation, and migration are visible.

CHAPTER 72 SUMMARY
The final pre-implementation checklist turns the entire book into a decision gate. NEXUS-1 is ready to discuss microservices only where boundary, data, contracts, reliability, security, observability, and migration can all be made explicit. Anything weaker remains a module until the evidence improves.

BACK MATTER

Glossary of Microservice Terms

A controlled vocabulary for the service architecture described in this volume.

This glossary is not a general dictionary of distributed systems. It defines the words as this book uses them, inside the NEXUS-1 microservice design argument. The purpose is consistency: when the book says service, contract, event, projection, ownership, or audit, the reader should know which meaning is intended and which meaning is deliberately excluded.

GLOSSARY RULE

A term is useful only when it prevents confusion. If a word hides ownership, failure, or responsibility, it is not architectural language; it is decoration.

A - C

Term	Meaning in this book	NEXUS-1 usage
ACL / Anti-Corruption Layer	A translation boundary that protects one model from another model's vocabulary, assumptions, and data shape.	Used when NEXUS-1 consumes or exposes information without letting foreign language leak into RootCause, Compliance, or DigitalTwin models.
API Gateway	A front door that routes or aggregates requests for clients without becoming the owner of domain decisions.	The operator console may call a gateway, but the gateway must not decide root cause, compliance status, or audit truth.
At-least-once delivery	A delivery guarantee where a message may arrive more than once. The system must be safe under duplicates.	RootCauseVerdictIssued.v1 consumers use inbox and idempotency rules because the same public event can be delivered repeatedly.
Audit entry	An append-only evidence record of what happened, not a command and not a workflow trigger by itself.	Audit records verdicts, review requests, corrections, and operator actions; it does not open compliance reviews merely because a row exists.
Backend-for-Frontend / BFF	A client-specific composition service that prepares API responses for a UI while leaving domain ownership elsewhere.	The NEXUS-1 operator console can use a BFF to compose RootCause, Compliance, Reporting, and Audit status into one screen.
Bounded context	A boundary where a word has one controlled meaning and a model owns its rules. It is not automatically a deployable service.	AlarmManagement, RootCause, Compliance, Reporting, Audit, and DigitalTwin are contexts first; only some become service candidates.

Term	Meaning in this book	NEXUS-1 usage
Business capability	A stable business responsibility that can justify a service boundary better than a table or controller name can.	RootCause investigates evidence and issues verdicts; Compliance reviews obligations and findings; Reporting builds read models.
Circuit breaker	A resilience pattern that stops repeated calls to an unhealthy dependency for a period of time.	Used to prevent the operator BFF or a process manager from hammering an unavailable downstream service.
Claim	A signed statement about a user or service identity, used by authorization policies but not blindly trusted as domain truth.	A claim can say a user has a role; RootCause still owns whether that user may close a case in the current state.
Command	A request for a service to do something. It may be rejected by the service that owns the rule.	OpenRootCauseCase and RequestComplianceReview are commands; RootCauseVerdictIssued.v1 is not a command.
Compensation	A forward recovery action that corrects or supersedes an earlier distributed step without pretending time can be rolled back.	If a verdict is corrected, NEXUS-1 publishes a correction event and triggers review adjustment rather than erasing the old fact.
Consumer	A component or service that reacts to a message or event.	Compliance consumes RootCauseVerdictIssued.v1; Reporting consumes public facts to update projections.

D - F

Term	Meaning in this book	NEXUS-1 usage
Database per service	An ownership rule: one service owns writes to its data. It is not mainly a rule about physical database products.	RootCause owns RootCauseCase data; Compliance must not update RootCause tables directly.
Dead-letter queue / DLQ	A holding place for messages that cannot be processed after the allowed attempts or validation checks.	Poison versions of RootCauseVerdictIssued.v1 move to a DLQ with reason, correlation id, payload, and handler evidence.
Delayed truth	A user-facing state where one context has committed while another context or projection has not yet caught up.	The console may show verdict issued while compliance review is pending and reporting projection is refreshing.
Deployment unit	The artifact that can be released independently: service, worker, BFF, gateway, or projection updater.	RootCause.Service and Reporting projection workers may be separate deployment units only when operational reasons exist.
Distributed monolith	A system split into deployable pieces that still behave like one tightly coupled application.	A chain where OperatorConsole -> Gateway -> RootCause -> Compliance -> Reporting must all be up synchronously is a distributed monolith smell.
Distributed transaction	A transaction that tries to commit changes across multiple services as if they were one database.	This book avoids relying on distributed transactions; outbox, inbox, idempotency, and compensation carry the design instead.

Term	Meaning in this book	NEXUS-1 usage
Domain event	An internal fact raised inside a bounded context after a domain change. It is not necessarily a public contract.	VerdictIssued is internal to RootCause and should not be treated as the integration event.
Event contract	The stable public shape of an integration event, including name, version, identifiers, timestamps, and payload meaning.	RootCauseVerdictIssued.v1 is a contract; changing it requires versioning discipline.
Eventual consistency	A consistency model where different services become current at different times through durable messages and projections.	RootCause can be current while Compliance and Reporting are still catching up.
Failure isolation	The ability for one part of the system to fail without collapsing every caller and downstream consumer.	Reporting lag should not prevent RootCause from issuing a verdict.
Fallback	A safe alternative response when a dependency is unavailable. It must not lie about freshness or truth.	The operator console may show last-known projection with a stale marker rather than pretending the data is current.

G - I

Term	Meaning in this book	NEXUS-1 usage
Gateway aggregation	Combining several service responses for a client-facing view. It is composition, not domain ownership.	The operator gateway can combine alarm status, root-cause status, and compliance status for display.
Health check	A technical endpoint or probe that reports liveness, readiness, or dependency status.	NEXUS-1 services separate liveness from readiness so traffic is not routed to a service that cannot process safely.
Idempotency	The property that repeating the same request or message does not create a second business effect.	Compliance does not open two reviews because the same verdict event is delivered twice.
Inbox	A consumer-side record of received or processed message ids, used to make message handling duplicate-safe.	Compliance stores the event id before or during handling so duplicate RootCauseVerdictIssued.v1 messages can be ignored safely.
Independent deployment	The ability to release a service without redeploying the whole system. It is valuable only if contracts and operations support it.	RootCause.Service is a candidate only if its public contracts remain stable for its consumers.
Integration event	A public fact published for other services after the local service commits its state.	RootCauseVerdictIssued.v1 announces the public verdict fact to Compliance, Audit, Reporting, and other interested consumers.
Internal model	The private domain objects, invariants, tables, and language of a service.	RootCauseCase is internal to RootCause; its private aggregate shape should not become a public DTO.
Invariant	A rule that must always hold inside a boundary.	A RootCauseCase cannot close without evidence; a recommendation is not an actuator command; audit evidence is append-only.

Term	Meaning in this book	NEXUS-1 usage
Identity propagation	Carrying user identity, service identity, correlation id, and causation id across calls and messages.	A compliance review can be traced back to the user action, service handler, verdict event, and audit entry that led to it.

L - O

Term	Meaning in this book	NEXUS-1 usage
Least privilege	Granting a service only the permissions it needs to perform its own responsibility.	Reporting can read projections; it should not receive write permission over RootCauseCase or ComplianceReview ownership tables.
Local truth	The authoritative state inside the service that owns a decision or data item.	RootCause is locally true about verdicts; Compliance is locally true about review state.
Message broker	Infrastructure that stores and delivers messages between services. It is not the owner of domain meaning.	RabbitMQ-like examples help explain delivery, retries, and DLQs; NEXUS-1 contracts should remain understandable without a specific broker.
Microservice	A separately deployable service with clear ownership, data control, public contracts, failure behaviour, observability, and operational responsibility.	A NEXUS-1 context becomes a microservice only when it passes readiness gates, not because it has a folder or controller.
Migration roadmap	A staged plan for moving from modular monolith to services while preserving operation, audit, and rollback.	The NEXUS-1 roadmap extracts candidates gradually using strangler routing, contract tests, coexistence, and rollback evidence.
Modular monolith	One deployable application with internal modular boundaries. It can be the better architecture when distribution cost is not yet justified.	NEXUS-1 begins from a modular core because language, ownership, and flows must mature before runtime separation.
Observability	The ability to understand system behaviour from logs, metrics, traces, health checks, and dashboards.	Operators and developers must trace the alarm-to-compliance flow across services when it is delayed or failed.
Operator console	The UI that displays plant/twin/service status. It composes views; it must not invent domain truth.	The console displays delayed truth with freshness, progress, and correlation information.
Outbox	A local table or store that records messages to publish in the same transaction as the state change.	RootCause saves the verdict and the RootCauseVerdictIssued.v1 outbox row together before a publisher sends the event.

P - S

Term	Meaning in this book	NEXUS-1 usage
Poison message	A message that repeatedly fails because of invalid payload, unsupported version, missing data, or handler defect.	A malformed event is not retried forever; it is quarantined with evidence in a DLQ.

Term	Meaning in this book	NEXUS-1 usage
Process manager	A component that tracks a long-running cross-service process and sends commands or reacts to events without owning the domain rules.	The alarm-to-compliance process manager follows AlarmFloodDetected.v1 and RootCauseVerdictIssued.v1 and requests review when appropriate.
Projection	A read-side model built from source facts for queries, dashboards, or reports. It is not the source of truth.	Reporting projections support dashboards but do not decide verdicts or compliance findings.
Public contract	The stable shape a service exposes to others: API, command, query, or integration event.	RootCauseVerdictIssuedV1 is public; RootCauseCase private fields are not.
Readiness check	A health check that says whether a service can safely receive traffic now, including dependency and configuration readiness.	A service may be alive but not ready if its broker connection or database migration state is not safe.
Reference data	Stable shared facts such as unit identifiers, engineering unit codes, or status catalogues.	Services may replicate or snapshot reference data rather than join across service databases.
Remote truth	Information owned by another service and observed through a contract, projection, snapshot, or event.	Compliance sees RootCause verdicts as remote facts and should not update RootCause truth.
Retry	A repeated attempt after a transient failure, bounded by policy to avoid storms.	Publishing or consuming a message may retry, but synchronous service chains should not amplify retries blindly.
Saga	A distributed consistency process made of local transactions and compensating actions.	The alarm-to-compliance flow uses saga thinking without pretending there is one transaction across RootCause and Compliance.
Service catalog	A registry of service candidates, responsibilities, owned data, contracts, dependencies, risks, and readiness status.	The NEXUS-1 service catalog records RootCause.Service, AlarmManagement.Service, Audit.Service, Reporting.Service, and conditional candidates.
Shared database	A database used by multiple services as if table access were integration. It breaks ownership even if the schema is tidy.	Two services updating the same RootCauseCase or AlarmEvent tables would not be microservices in this book.
Source of truth	The place that owns a fact and is allowed to change it.	RootCause owns verdict truth; Reporting owns display projections; Audit owns evidence history.
Strangler fig pattern	A migration pattern that routes parts of the system to new services gradually while the old system continues to run.	NEXUS-1 can route selected RootCause operations to a new service while the modular core remains alive.

T - V

Term	Meaning in this book	NEXUS-1 usage
Timeout	A bounded waiting period after which a caller stops waiting and chooses a defined failure path.	A BFF call to a reporting projection must time out rather than hold the operator screen forever.

Term	Meaning in this book	NEXUS-1 usage
Trace	A linked record of work across service boundaries, calls, messages, and handlers.	A correlation id and spans connect alarm flood detection, root-cause verdict, compliance review, audit write, and reporting projection.
Traceability	The ability to follow a case, decision, or message across services and time.	The operator can trace from RootCauseCase to verdict event, audit evidence, compliance review, and projection update.
Versioning	Changing a public API or event contract without breaking existing consumers unexpectedly.	RootCauseVerdictIssued.v1 remains stable; incompatible changes create a new version rather than silently changing v1.
Workflow engine	A runtime for coordinating process state. It must not hide domain decisions that belong inside services.	NEXUS-1 may coordinate alarm-to-compliance flow, but RootCause still decides verdict validity and Compliance decides review validity.
Write ownership	The rule that only the owning service changes its own source-of-truth data.	Compliance can request information from RootCause, but only RootCause changes RootCauseCase verdict data.

GLOSSARY CHECKPOINT

The vocabulary of this book keeps returning to the same discipline: a service owns language, data, contracts, failure behaviour, security, observability, and migration responsibility. If one of those words is unclear, the architecture is not ready to be distributed.

BACK MATTER

Sources and Reference Notes

Conceptual background, internal continuity sources, and the boundary of what these references do - and do not - prove.

This section records the sources behind the architectural vocabulary used in From Flow to Services. The references do not certify NEXUS-1, and they do not turn the demonstrator into operational plant software. They explain the software-architecture ideas that the book applies: domain boundaries, service ownership, asynchronous messaging, distributed consistency, contracts, security boundaries, observability, migration, and operational discipline.

SOURCE RULE

A reference is used here as conceptual support, not as a substitute for judgement. The book remains responsible for its own boundary decisions, service candidates, diagrams, and NEXUS-1 terminology.

A. Domain-Driven Design and boundary language

Reference / source	How it is used in this volume
Eric Evans, Domain-Driven Design: Tackling Complexity in the Heart of Software, Addison-Wesley, 2003.	Foundational vocabulary for domain, model, ubiquitous language, aggregate, repository, bounded context, and strategic design. The book uses this vocabulary to keep service decisions rooted in model ownership rather than deployment fashion.
Vaughn Vernon, Implementing Domain-Driven Design, Addison-Wesley, 2013.	Practical treatment of bounded contexts, context maps, aggregates, domain events, and collaboration between models. Used especially when deciding whether a NEXUS-1 context is mature enough to become a service.
Vaughn Vernon, Domain-Driven Design Distilled, Addison-Wesley, 2016.	Concise DDD framing that supports the school-style explanations in earlier NEXUS-1 volumes and in the opening parts of this book.
Martin Fowler, Patterns of Enterprise Application Architecture, Addison-Wesley, 2002.	Background for service layer, repository, unit of work, data mapper, identity map, transaction boundary, and enterprise persistence vocabulary.
Martin Fowler, public essays on Bounded Context, CQRS, Event Sourcing, Unit of Work, and Microservice Premium.	Plain-language framing for context boundaries, read/write separation, service cost, and the warning that distribution has an architectural price.
Alberto Brandolini, EventStorming materials and talks.	Influence on reasoning from business events, commands, policies, external systems, and hotspots before drawing service boundaries.

B. Microservices, service boundaries, and migration

Reference / source	How it is used in this volume
Sam Newman, Building Microservices, O'Reilly, 2015; 2nd edition, 2021.	Core background for service independence, service ownership, decomposition, databases per service, integration choices, and the operational cost of distributed systems.

Reference / source	How it is used in this volume
Sam Newman, Monolith to Microservices, O'Reilly, 2019.	Main influence for the migration chapters: extraction, strangler thinking, coexistence, deployment safety, and migration risk.
Chris Richardson, Microservices Patterns, Manning, 2018.	Service decomposition, API composition, database per service, saga patterns, transactional outbox, idempotent consumers, and microservice migration patterns.
Martin Fowler and James Lewis, Microservices: a definition of this new architectural term.	Used as a compact framing source for independently deployable services organized around business capability and decentralized governance.
Michael Nygard, Release It!, Pragmatic Bookshelf, 2007/2018.	Operational stability patterns: timeouts, circuit breakers, bulkheads, retry discipline, cascading failure, and production readiness.
Eric Evans and domain-driven design community discussions on strategic design.	Supports the recurring rule that service boundaries follow true language and ownership boundaries, not entity names or folders.

C. Messaging, distributed consistency, and reliability

Reference / source	How it is used in this volume
Gregor Hohpe and Bobby Woolf, Enterprise Integration Patterns, Addison-Wesley, 2003.	Message channels, routers, translators, correlation identifiers, message stores, idempotent receivers, dead-letter channels, and messaging vocabulary used throughout Parts V and VI.
Pat Helland, Life Beyond Distributed Transactions: an Apostate's Opinion.	Conceptual support for avoiding distributed transactions and accepting local transactions plus messages, idempotency, and compensation.
Hector Garcia-Molina and Kenneth Salem, Sagas, ACM SIGMOD, 1987.	Background for long-running transactions, compensation, and distributed workflow recovery. The book uses saga thinking without pretending a saga is a magic rollback.
Martin Kleppmann, Designing Data-Intensive Applications, O'Reilly, 2017.	Reliability, replication, ordering, logs, delivery semantics, consistency trade-offs, and the discipline of making assumptions visible.
Microsoft documentation and .NET guidance on hosted services, messaging, background workers, and resilient HTTP calls.	Practical .NET implementation shape for outbox publishing, background consumers, retry policies, and service-to-service calls.
RabbitMQ and message-broker documentation as conceptual background.	Broker vocabulary for queues, exchanges, acknowledgements, retries, dead-letter routing, and poison-message handling. The book avoids binding NEXUS-1 to one broker product.

D. APIs, contracts, and versioning

Reference / source	How it is used in this volume
Roy Fielding, Architectural Styles and the Design of Network-based Software Architectures, 2000.	REST constraints and the distinction between resource-oriented interaction and arbitrary remote procedure calls.
OpenAPI Specification.	Background for documenting public HTTP contracts, request/response shapes, versioned APIs, and consumer-facing service descriptions.
Microsoft REST API Guidelines and ASP.NET Core Web API documentation.	Practical .NET-oriented contract design, route shape, status codes, validation responses, and API versioning background.
Consumer-driven contract testing literature and tools such as Pact.	Conceptual basis for contract tests during extraction, especially when old and new systems coexist during strangler migration.
RFC 9110 and related HTTP Semantics specifications.	Protocol background for HTTP methods, status codes, caching semantics, headers, and request/response behaviour.

E. Security, identity, and service trust

Reference / source	How it is used in this volume
OAuth 2.0 and OpenID Connect specifications.	Background for authentication, authorization delegation, identity tokens, access tokens, scopes, and claims propagation.
JWT specifications and common implementation guidance.	Vocabulary for signed claims and the boundary between identity evidence and domain authorization decisions.
OWASP Application Security Verification Standard and OWASP API Security Top 10.	Security checklists for API exposure, authorization mistakes, authentication weaknesses, sensitive data exposure, and insecure configuration.
NIST SP 800-207, Zero Trust Architecture.	Conceptual influence for service-to-service trust, least privilege, explicit verification, and refusing to treat network location as sufficient trust.
Microsoft identity platform and ASP.NET Core authorization documentation.	Practical .NET examples for policies, claims, roles, authorization handlers, certificates, secrets, and service-to-service credentials.
CNCF and cloud-native security guidance.	General background for secret management, certificate rotation, service identity, and operational security hygiene.

F. Observability, operations, and runtime discipline

Reference / source	How it is used in this volume
OpenTelemetry documentation and semantic conventions.	Primary vocabulary for traces, spans, metrics, logs, baggage, context propagation, correlation IDs, and causality chains across service boundaries.
Google Site Reliability Engineering books and public SRE materials.	Operational thinking: service-level indicators, alerting discipline, error budgets, toil, incident response, and reliability as an engineering function.
Prometheus, Grafana, and dashboarding documentation.	Conceptual background for metrics, dashboards, alerting, service health, and operational visibility in NEXUS-1 examples.
Kubernetes documentation.	Basic orchestration vocabulary: pod, deployment, service, readiness probe, liveness probe, rollout, rollback, and configuration. The book uses these concepts without requiring Kubernetes-specific implementation.
Docker documentation.	Container vocabulary used in the container and deployment chapters: image, container, registry, runtime boundary, and environment configuration.
Polly and .NET resilience guidance.	Retry, timeout, circuit breaker, hedging, and resilience-pipeline concepts for .NET-oriented implementation examples.

G. NEXUS-1 internal companion sources

The following internal NEXUS-1 artefacts provide continuity for the examples. They are not external validation. They define the demonstrator vocabulary, screens, schema, earlier architectural decisions, and boundaries that this microservice volume extends.

Reference / source	How it is used in this volume
NEXUS-1 Phase-0 browser console, index.html.	The running demonstration anchor: plant/twin screens, alarm flows, root-cause views, reporting panels, audit/compliance panels, and the visual system that the service architecture is designed to explain.
From Grid to Core.	Physical and simulation vocabulary: unit, reactor, sensor, power, grid, instrumentation, digital-twin boundary, and the non-operational educational scope.
From Flood to Cause.	Root-cause reasoning vocabulary: alarm flood, causal graph, evidence, hypothesis, verdict, auditability, and the alarm-to-root-cause case flow used repeatedly in this book.
From Trial to Policy.	Advisory learning vocabulary: policy, Q-table, recommendation, bounded advice, and the strict distinction between recommendation and actuator command.
From Domain to Twin.	DDD foundation: entities, value objects, aggregates, domain events, bounded contexts, context maps, language discipline, and the first NEXUS-1 domain model.
From Context to Flow.	Immediate predecessor: domain events vs integration events, outbox, inbox, process managers, sagas, compensation, anti-corruption layers, published language, and eventual consistency before microservices.
From Table to Twin.	Data backbone reasoning: one schema, two paths, SQL Server/EF Core modelling, audit and history, lookup discipline, and performance framing.
From Schema to System.	Seventeen-sector schema atlas, typed lookups, explicit keys, cross-schema passports, auditability, reporting, and data ownership evidence.
From Entity to Context.	EF Core configuration discipline: one entity, one mapping, schema folders, explicit table names, indexes, relationships, and migration readability.
From Blueprint to Core.	Backend-core bridge: Clean/Onion structure, domain invariants, CQRS application layer, tests before persistence, and the C# shape that this microservice volume assumes.

H. Notes on terminology and deliberate exclusions

The terminology in this book is intentionally narrow. A bounded context is not automatically a service. A service is not merely a controller. A database per service is a rule about write ownership, not a command to multiply database products. A read model is not source of truth. Audit evidence is not workflow orchestration. A recommendation is not a command. These distinctions are repeated because distributed systems punish vague words.

CANONICAL NEXUS-1 CONTRACT NAMES

Internal domain event: VerdictIssued. Public integration event: RootCauseVerdictIssued.v1. C# public contract type: RootCauseVerdictIssuedV1. Related public contracts: AlarmFloodDetected.v1, RootCauseVerdictCorrected.v1. Commands include OpenRootCauseCase and RequestComplianceReview. RootCauseCaseClosed and RootCauseVerdictPublished are not used as the main canonical public names in this book.

The book also avoids presenting microservices as a universal upgrade. When a modular monolith protects the domain better than a distributed runtime, the monolith remains the more honest architecture. The final architecture is therefore a candidate map, not a command to split every sector.

I. Source boundary

HONEST BOUNDARY

The references above support software architecture reasoning. They do not make NEXUS-1 safety-class software, certified operational guidance, a licensed plant information model, or a system connected to any live facility. NEXUS-1 remains an educational digital-twin and architecture demonstrator.

A final practical rule closes this section: any claim in the book that depends on NEXUS-1 itself should be traceable to one of the companion volumes, the project description, or the Phase-0 console. Any claim that depends on general architecture should be traceable to the conceptual sources above. Where neither is true, the book should either revise the claim or mark it explicitly as a design proposal.

REFERENCE CHECKPOINT

Before continuing to the working index, the reader should know which ideas are imported from general software architecture, which facts belong to the NEXUS-1 demonstrator, and which decisions are original design judgments of this volume.

BACK MATTER Working Index

A practical working index for the draft. Page references point to the current PDF pagination and will be tightened again during final pagination.

A

- access tokens 506, 511, 517
- actor identity 492, 505, 530
- advisory recommendation 198, 216, 492
- AlarmFloodDetected.v1 300, 371
- AlarmManagement service candidate 64, 75, 120, 649
- alarm-to-compliance flow 50, 407, 441
- anti-corruption layer 64, 198, 617
- API and event catalog 663
- API gateway 275
- API versioning 239
- asynchronous communication 287, 288
- Audit service 485, 538, 656
- AuditEntry as evidence 485, 492, 700
- authorization 505, 517, 530

B

- backend-for-frontend 275
- bounded context 27, 64, 120
- business capability boundary 75
- business decision ownership 216, 433, 441
- business workflow 98, 388

C

- C# public contract type 216, 371, 700
- candidate map 649
- causality chain 554
- checklist discipline 682
- circuit breaker 570
- claims 517
- commands 216, 407
- Compliance service 478, 538, 656
- compensation 417
- configuration 523
- consumer idempotency 336
- containers 589
- contract tests 630

- correlation ID 492, 554
- cross-service identity 159
- customer-facing delayed truth 498

D

- dashboard 597
- data ownership 86, 136, 670
- database per service 137
- dead-letter queue 360
- delayed truth 464, 498
- deployment unit 580
- DigitalTwin service candidate 120, 649, 656
- distributed process manager 389, 441
- distributed transaction 417
- duplicate message 336, 348

E

- event catalog 663
- event contracts 371
- eventual consistency 455, 464
- evidence 407, 485
- extraction strategy 606, 617

F

- failure catalog 676
- failure isolation 64, 120
- failure mode 676
- final architecture 648
- first extracted service 617

G

- glossary 689
- grounded service boundary 120

H

- health checks 562
- human review 425
- hybrid migration 624

I

- idempotent consumer 336

inbox pattern 336
independent release 580
Instrumentation service candidate 86, 649
integration event 300, 371
internal domain event 300, 700

L

least privilege 530
local truth 170
logs 546

M

message broker 312
message ordering 348
metrics 546
microservice candidate 64, 649
migration roadmap 642
modular monolith 40, 606

N

NEXUS-1 service catalog 656
NEXUS-1 security boundary map 538
NEXUS-1 data ownership map 189
NEXUS-1 migration roadmap 642

O

observability 545, 546
OpenRootCauseCase 216, 700
operational dashboard 597
operator console 275, 464
orchestration 389, 399, 441
outbox pattern 324

P

Part I - microservice temptation 15
Part II - service boundaries 63
Part III - data independence 136
Part IV - synchronous communication 197
Part V - messaging 287
Part VI - distributed workflows 388
Part VII - read models 455
Part VIII - security 504
Part IX - operations 545
Part X - migration 605
Part XI - final architecture 648

poison message 360
policy boundary 517
process manager 389, 441
public contract 216, 239, 371

R

read model 455, 470
readiness check 562
recommendation not command 198, 216, 700
reference data 178
remote truth 170
Reporting service 470, 656
resilience 570
REST APIs 228
retry 570
rollback 636
RootCause service 64, 120, 649, 656
RootCauseVerdictIssued.v1 300, 371, 700

S

safe deployment 636
saga coordination 399
scope note 3
secret management 523
security boundary 504, 538
service authentication 511
service catalog 656
service-to-service trust 511
shared database smell 148
snapshot 178, 470
sources 695
Strangler Fig pattern 611
synchronous call 251, 263

T

timeouts 425, 570
traceability 492
tracing 546, 554

V

VerdictIssued 300, 700
versioning 239

W

workflow boundary 98

workflow engine 433

working index 701

write ownership 137, 670

BACK MATTER

Final Consistency Audit

A final review of the book as an architectural argument: language, ownership, contracts, data, reliability, operations, migration, and honest boundaries.

This audit is not a new chapter and not a marketing conclusion. It is the same discipline applied throughout the NEXUS-1 series turned back onto the manuscript itself. The question is simple: does From Flow to Services still say what it promised to say, after seventy-two chapters and back matter?

AUDIT RULE

A microservice claim is acceptable only when the page identifies responsibility, data ownership, contract shape, failure behavior, and operational cost. A distributed decision that names only the tool is incomplete.

Audit area	Result
Central thesis	The book consistently treats microservices as a consequence of mature boundaries, not as an upgrade slogan.
Series continuity	The book begins where From Context to Flow ended: service readiness after context maps, events, outbox, inbox, process managers, and eventual consistency.
NEXUS-1 boundary	The educational demonstrator boundary remains explicit: no live plant, no safety-class control, no operational nuclear guidance.
Implementation level	The examples are practical and .NET-oriented, but the book remains an architecture volume rather than a broker, Kubernetes, or API tutorial.

FINAL AUDIT

A. Terminology Canon

Concept	Canonical wording kept by the book	Avoid / audit warning
Bounded context	A language and ownership boundary first	Treating every context as a service automatically
Microservice	A deployable service with owned data, contract, runtime responsibility, and failure behavior	A small Web API with a shared database
Internal domain event	VerdictIssued inside RootCause	Publishing internal aggregate events directly
Public integration event	RootCauseVerdictIssued.v1	RootCauseCaseClosed as the main canonical public event
Public contract type	RootCauseVerdictIssuedV1	Serializing RootCauseCase or private domain objects
Audit evidence	AuditEntry records evidence	Using audit rows as workflow commands
Recommendation	Advisory outcome	Command to plant equipment or safety automation
Read model	Projection for querying and display	Source of truth for decisions
Reliability pattern	Outbox, inbox, idempotency, DLQ, retry with limits	Exactly-once assumptions or retry storms

TERMINOLOGY RESULT

The book keeps one public event language for the alarm-to-compliance flow and separates domain events, integration events, audit evidence, read models, recommendations, and commands.

FINAL AUDIT

B. Structure and Connectivity

Area	Connectivity result
Front matter	Cover, dedication, boundary, readable contents, preface, reading guidance, and series positioning are present.
Part I	Establishes why microservices are not the starting point and why a modular monolith may remain correct.
Part II	Converts context maps into service-candidate judgement using capability, data, workflow, team, and deployment boundaries.
Part III	Separates data ownership from table ownership and introduces local truth, remote truth, snapshots, reference data, and replication.
Part IV	Explains synchronous APIs, public contracts, versioning, API gateway, and BFF concerns without leaking private domain models.
Part V	Builds asynchronous messaging discipline: integration events, broker guarantees, outbox, inbox, ordering, DLQ, and event contracts.
Part VI	Shows distributed workflow without distributed transactions: process managers, sagas, compensation, timeouts, human review, and NEXUS-1 example.
Parts VII-IX	Connect consistency, reporting, security, identity, observability, health checks, resilience, deployment, and operations.
Parts X-XI	Move from migration strategy to final architecture catalogues and pre-implementation checklist.
Back matter	Glossary, sources, working index, and final consistency audit are present; final checkpoint remains the next explicit step.

FINAL AUDIT

C. Claim and Boundary Audit

The book uses NEXUS-1 as a coherent software demonstrator, not as a claim about a real facility. The microservice architecture remains a design exercise built from domain boundaries, service ownership, public contracts, and operational responsibility.

Claim type	Allowed claim	Not claimed
Architecture	The book can propose service candidates and boundary maps.	It does not prove that every candidate must be deployed as a service.
Domain model	The book can protect RootCause, Compliance, Audit, Reporting, and other models from leakage.	It does not claim one global model should be shared everywhere.
Data	Database-per-service is explained as ownership, not a fashion rule.	It does not require physical separation before the service boundary is mature.
Messaging	Outbox/inbox patterns are presented as recovery evidence and idempotency discipline.	It does not promise exactly-once delivery.
Security	Identity, claims, service-to-service authentication, secrets, and least privilege are explained architecturally.	It does not implement a production identity platform.
Operations	Logs, metrics, traces, health checks, resilience, and dashboards are treated as design requirements.	It does not claim production operations are solved by diagrams.
Nuclear boundary	NEXUS-1 remains educational, advisory, and demonstrator-grade.	It is not certified control software and does not connect to a live plant.

FINAL AUDIT

D. Service Decision Audit

The final architecture does not split NEXUS-1 evenly. It evaluates each responsibility against ownership, language stability, deployment value, failure isolation, integration cost, and auditability.

Decision group	Candidate services / responsibilities	Reasoning status
Strong candidates	AlarmManagement, RootCause, Compliance, Audit, Reporting	Clear responsibility, strong public contracts, meaningful event flows, and independent operational value.
Conditional candidates	DigitalTwin, Instrumentation, ReinforcementLearning Advisory, ReactorFleet	Useful boundaries exist, but coupling, data volume, model freshness, and demonstration scope must be priced carefully.
Keep close / protect	Simulation internals, UI composition, shared demonstration infrastructure, safety-like control logic	Distribution would add risk or false authority without a justified operational benefit.
Catalogues	Service Catalog, API and Event Catalog, Data Ownership Catalog, Failure Mode Catalog	These convert architecture from prose into reviewable evidence before implementation begins.
Migration path	Strangler extraction, coexistence, contract tests, rollback, safe deployment, roadmap	Extraction is presented as staged migration, not a big-bang rewrite.

DECISION RESULT

The book does not confuse a context map with a deployment map. It uses the context map as evidence, then asks whether runtime separation earns its cost.

FINAL AUDIT

E. Operational Readiness and Final Items

Operational concern	Audit result
Traceability	CorrelationId, CausationId, EventId, audit entries, and traces are treated as the connective tissue of the distributed system.
Recovery	Outbox, inbox, DLQ, retry limits, idempotency, compensation, rollback plans, and release gates prevent happy-path architecture.
User truth	The operator console is expected to show freshness, pending states, delayed truth, and failure reasons rather than pretending every context is current.
Security	Authentication, authorization, service trust, secrets, certificates, least privilege, and boundary maps are not postponed to production.
Observability	Logs, metrics, traces, health checks, dashboards, deployment units, and runtime supervision are part of service design, not decorations.

FINAL HONEST BOUNDARY

From Flow to Services is a software architecture book. It proposes how the NEXUS-1 demonstrator could move from mature DDD flow to service-oriented runtime architecture. It does not certify an operational system, implement production infrastructure, or claim that distribution makes the system safer by itself.

ITEMS LEFT FOR FINAL EDITION

Final checkpoint page; figure and table lists if required; deeper index density for code names and event names; page audit after final pagination; optional formal bibliography formatting.

BACK MATTER

Final Checkpoint

A final reader-facing checkpoint for the complete draft: what the book should now allow you to judge, design, and refuse.

This checkpoint closes the architectural journey. It does not introduce a new pattern. It asks whether the reader can now defend the whole move from modular DDD flow to microservice architecture without relying on slogans, tool names, or deployment fashion.

THE FINAL QUESTION

Can this boundary survive as an independently deployed service without losing language discipline, data ownership, contract clarity, auditability, reliability, security, and operational visibility?

You should now be able to...	Evidence in the book
Say when a bounded context should remain a module.	Parts I-II: context readiness, boundary checklists, and modular-monolith discipline.
Explain why database-per-service is about ownership.	Part III: local truth, remote truth, replication, snapshots, and data ownership maps.
Design public APIs without leaking domain models.	Part IV: commands, queries, contracts, REST, versioning, gateway, and BFF.
Publish and consume events safely.	Part V: integration events, broker guarantees, outbox, inbox, ordering, duplicates, and DLQ.
Coordinate long-running flows without distributed transactions.	Part VI: process managers, sagas, compensation, timeouts, human review, and NEXUS-1 distributed flow.

FINAL CHECKPOINT

The Refusal Checklist

A mature microservice book should not only teach what to build. It should also teach what to refuse. The following refusals are part of the final architecture, because every distributed system becomes dangerous when it cannot say no.

Do not...	Reason
Do not turn every bounded context into a service.	A context is a language and ownership boundary first; runtime separation is more expensive.
Do not share a database and call the result microservices.	Shared tables erase ownership and create hidden coupling.
Do not expose aggregates as public contracts.	Contracts must be stable, intentional, versioned, and smaller than private models.
Do not let audit rows drive workflow.	Audit is evidence; workflow belongs to process managers and application logic.
Do not treat recommendations as commands.	NEXUS-1 advisory outcomes do not become plant-control authority.
Do not hide delayed truth from users.	Distributed systems must show pending states, freshness, and uncertainty.
Do not ship a service without recovery evidence.	Outbox, inbox, idempotency, DLQ, rollback, and dashboards must be designed before celebration.
Do not confuse deployment with architecture.	A container is a packaging unit, not proof of service autonomy.

FINAL CHECKPOINT

The NEXUS-1 Readiness Gate

Before any NEXUS-1 service candidate is extracted, the candidate should pass a written gate. The gate is intentionally stricter than a diagram: it asks for concrete evidence that the service can stand alone, fail alone, and be understood alone.

Gate	Question
Language	Does the service own a vocabulary that does not need private terms from another context to make sense?
Data	Does it own the data it decides with, and does it know which remote data is only a snapshot or projection?
Contract	Are commands, queries, and events public, versioned, and separated from the internal domain model?
Reliability	What happens when publication fails, consumption repeats, order changes, or a downstream service is unavailable?
Security	Which identity, claims, secrets, certificates, and least-privilege rules apply at the boundary?
Observability	Can an operator trace one case across services with correlation, causation, logs, metrics, traces, and audit evidence?
Migration	Can the old system and the extracted service coexist, be tested together, and be rolled back safely?
Human meaning	Does the operator console explain delayed truth, pending work, and uncertainty instead of hiding it?

FINAL CHECKPOINT

Closing Statement

The book began with a refusal: microservices are not the starting point. They are not the proof that an architecture is mature. They are one possible consequence of maturity, and only when the system can name its boundaries, contracts, events, data ownership, failure behavior, and operating cost clearly enough to survive distribution.

For NEXUS-1, that means the alarm-to-compliance flow must remain explainable after it crosses runtime boundaries. An alarm may be raised in one service, a root-cause verdict issued in another, a compliance review triggered elsewhere, a reporting view updated later, and an audit trail written across the whole path. The distributed system is acceptable only if the reader can still replay the story.

FINAL HONEST BOUNDARY

This book closes at architectural readiness. It does not claim that the NEXUS-1 demonstrator has become a production microservice platform. It gives the map, the rules, the catalogues, the checkpoints, and the refusal discipline needed before such an implementation should begin.

Final sentence

A service boundary is ready only when it can be deployed independently without making the system less understandable, less accountable, or less honest.